APPLIED SOFTWARE MEASUREMENT

Global Analysis of Productivity and Quality

THIRD EDITION

- Based on statistics from more than 12,000 software projects
- Includes comprehensive international data
- Covers metrics on the latest technologies, including Agile, Extreme (XP), and ERP

CAPERS JONES

FOREWORD BY Doug Brindley, President, Software Productivity Research, LLC

ABOUT THE AUTHOR

CAPERS JONES (Narragansett, Rhode Island) is a well-known author, consultant, and speaker in the world of software measurement, metrics, productivity, and quality. He has also appeared as an expert witness in a number of software lawsuits. He was the founder and chairman of Software Productivity Research (SPR), where he retains the title of Chief Scientist Emeritus of SPR. He was awarded a lifetime membership in the International Function Point User's Group (IFPUG). Jones gives presentations at conferences such as IEEE Software, International Function Point Users Group (IFPUG), the Project Management Institute (PMI), the Software Process Improvement Network (SPIN), the Japanese Software Symposium on Testing (JaSST), and at scores of in-house corporate and government events.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

Applied Software Measurement

Global Analysis of Productivity and Quality

Capers Jones

Third Edition

New York Chicago San Francisco Lisbon London Madrid
Mexico City Milan New Delhi San Juan Seoul
Singapore Sydney Toronto

Copyright © 2008 by The McGraw-Hill Companies. All rights reserved. Manufactured in the United States of America. Except as permitted under the United States Copyright Act of 1976, no part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written permission of the publisher.

0-07-164386-9

The material in this eBook also appears in the print version of this title: 0-07-150244-0.

All trademarks are trademarks of their respective owners. Rather than put a trademark symbol after every occurrence of a trademarked name, we use names in an editorial fashion only, and to the benefit of the trademark owner, with no intention of infringement of the trademark. Where such designations appear in this book, they have been printed with initial caps.

McGraw-Hill eBooks are available at special quantity discounts to use as premiums and sales promotions, or for use in corporate training programs. For more information, please contact George Hoare, Special Sales, at george_hoare@mcgraw-hill.com or (212) 904-4069.

TERMS OF USE

This is a copyrighted work and The McGraw-Hill Companies, Inc. ("McGraw-Hill") and its licensors reserve all rights in and to the work. Use of this work is subject to these terms. Except as permitted under the Copyright Act of 1976 and the right to store and retrieve one copy of the work, you may not decompile, disassemble, reverse engineer, reproduce, modify, create derivative works based upon, transmit, distribute, disseminate, sell, publish or sublicense the work or any part of it without McGraw-Hill's prior consent. You may use the work for your own noncommercial and personal use; any other use of the work is strictly prohibited. Your right to use the work may be terminated if you fail to comply with these terms.

THE WORK IS PROVIDED "AS IS." McGRAW-HILL AND ITS LICENSORS MAKE NO GUARANTEES OR WARRANTIES AS TO THE ACCURACY, ADEQUACY OR COMPLETENESS OF OR RESULTS TO BE OBTAINED FROM USING THE WORK, INCLUDING ANY INFORMATION THAT CAN BE ACCESSED THROUGH THE WORK VIA HYPERLINK OR OTHERWISE, AND EXPRESSLY DISCLAIM ANY WARRANTY, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. McGraw-Hill and its licensors do not warrant or guarantee that the functions contained in the work will meet your requirements or that its operation will be uninterrupted or error free. Neither McGraw-Hill nor its licensors shall be liable to you or anyone else for any inaccuracy, error or omission, regardless of cause, in the work or for any damages resulting therefrom. McGraw-Hill has no responsibility for the content of any information accessed through the work. Under no circumstances shall McGraw-Hill and/or its licensors be liable for any indirect, incidental, special, punitive, consequential or similar damages that result from the use of or inability to use the work, even if any of them has been advised of the possibility of such damages. This limitation of liability shall apply to any claim or cause whatsoever whether such claim or cause arises in contract, tort or otherwise.

DOI: 10.1036/0071502440

Mc Graw Hill Professional

Want to learn more?

We hope you enjoy this McGraw-Hill eBook! If you'd like more information about this book, its author, or related books and websites, please click here.

This book is dedicated to Allan Albrecht of IBM, a pioneer in bringing economic measurement to software. Allan and his colleagues were the first to develop function point metrics and therefore allow standard economic measures to be applied to software.

This book is also dedicated to the officers and members of the International Function Point Users Group (IFPUG) who have continued to advance the cause of measuring software economics. The existence of function points and the IFPUG organization have led to further measurement studies carried out by Peter Hill and his colleagues at the International Software Benchmark Standards Group (ISBSG), as well to studies carried out by many affiliated national groups such as the Australian, Netherlands, Japanese, and Brazilian software metrics associations.

This page intentionally left blank

Contents at a Glance

Chapter 1.	Introduction	1
Chapter 2.	The History and Evolution of Software Metrics	71
Chapter 3.	United States Averages for Software Productivity and Quality	185
Chapter 4.	The Mechanics of Measurement: Building a Baseline	351
Chapter 5.	Measuring Software Quality and User Satisfaction	433
Chapter 6.	Measurements, Metrics, and Industry Leadership	509
Chapter 7.	Summary of Problems in Software Measurement	549
Appendix.	Rules for Counting Procedural Source Code	635
Index		645

This page intentionally left blank

For more information about this title, click here

Contents

Foreword xiii
Preface to the Third Edition xvii
Acknowledgments xxxi

Chapter 1. Introduction 1

Applied Software Measurement 2
Planning and Estimation 3
Management and Technical Staffs 4
Organization Structures 4
Methodologies and Tools 4
The Office Environment 5
Reusability 5
The Essential Aspects of Applied Software Measurement 6
What Do Companies Measure? 17
Benchmarks and Industry Measures 22
Measurement and the Software Life Cycle 44
The Structure of a Full Applied Software Measurement System 48
The Sociology of Software Measurement 53
The Sociology of Data Confidentiality 54
The Sociology of Using Data for Staff Performance Targets 55
The Sociology of Measuring One-Person Projects 56
The Sociology of MIS vs. Systems Software 57
The Sociology of Measurement Expertise 57
Justifying and Building an Applied Software Measurement Function 58
Applied Software Measurement and Future Progress 66
Suggested Readings 67
Additional Readings on Software Measurement and Metrics 68

Chapter 2. The History and Evolution of Software Metrics 71

Evolution of the Software Industry and Evolution of Software Measurements 72
The Cost of Counting Function Point Metrics 78
The Paradox of Reversed Productivity for High-Level Languages 87

The Varieties of Functional Metrics Circa 2008 104
Variations in Application Size and Productivity Rates 159
Future Technical Developments in Functional Metrics 164
Summary of and Conclusions About Functional Metrics 172
Software Measures and Metrics Not Based on Function Points 173
Suggested Readings on Measures and Metrics 182

Chapter 3. United States Averages for Software Productivity and Quality 185

Sources of Possible Errors in the Data 190
Significant Software Technology Changes Between 1990 and 2008 226
Changes in the Structure, Format, and Contents of the Third Edition 244
Variations in Software Development Practices Among Seven Sub-Industries 256
Ranges, Averages, and Variances in Software Productivity 268
The Impact of Technology on Software Productivity and Quality Levels 322
Technology Warnings and Counterindications 342
Using Function Point Metrics to Set "Best in Class" Targets 344

Chapter 4. The Mechanics of Measurement: Building a Baseline 351

Software Assessments 352
Software Baselines 356
Software Benchmarks 357
What a Baseline Analysis Covers 378
Developing or Acquiring a Baseline Data Collection Instrument 380
Administering the Data Collection Questionnaire 383
Analysis and Aggregation of the Baseline Data 430
Suggested Readings 431
Additional Readings 431

Chapter 5. Measuring Software Quality and User Satisfaction 433

New Quality Information Since the Earlier Editions 437
Quality Control and International Competition 451
Defining Quality for Measurement and Estimation 454
Five Steps to Software Quality Control 458
Software Quality Control in the United States 460
Measuring Software Defect Removal 471
Measuring Defect Removal Efficiency 475
Finding and Eliminating Error-Prone Modules 480
Using Metrics to Evaluate Test-Case Coverage 481
Using Metrics for Reliability Prediction 482
Measuring the Costs of Defect Removal 483
Evaluating Defect Prevention Methods 487
Measuring Customer-Reported Defects 488

Measuring Invalid Defects, Duplicate Defects, and Special Cases 491
Measuring User Satisfaction 492
Combining User Satisfaction and Defect Data 499
Summary and Conclusions 501
Reading List 501
Suggested Readings 507
Additional References on Software Quality and Quality Measurements 507

Chapter 6. Measurements, Metrics, and Industry Leadership 509

What Do Companies Measure? 511
Measures and Metrics of Industry Leaders 529
Measures, Metrics, and Innovation 532
Measurements, Metrics, and Outsource Litigation 535
Measurements, Metrics, and Behavioral Changes 536
Topics Outside the Scope of Current Measurements 542
Cautions Against Simplistic and Hazardous Measures and Metrics 543
Commercial Software Measurement Tools 546
Summary and Conclusions 547
Suggested Readings on Measurement and Metrics 548

Chapter 7. Summary of Problems in Software Measurement 549

Synthetic vs. Natural Metrics 550
Ambiguity in Defining the Nature, Scope, Class, and Type of Software 552
Ambiguity in Defining and Measuring the Activities and Tasks of Software Projects 561
False Advertising and Fraudulent Productivity Claims 564
The Absence of Project Demographic and Occupation Group Measurement 566
Ambiguity in the Span of Control and Organizational Measurements 567
The Missing Link of Measurement: When Do Projects Start? 568
Ambiguity in Measuring Milestones, Schedules, Overlap, and Schedule Slippage 568
Problems with Overlapping Activities 573
Leakage from Software Project Resource Tracking Data 573
Ambiguity in Standard Time Metrics 575
Inadequate Undergraduate and Graduate Training in Software Measurement and Metrics 578
Inadequate Standards for Software Measurement 579
Lack of Standardization of "Lines of Source Code" Metrics 580
The Hazards and Problems of Ratios and Percentages 589
Ambiguity in Measuring Development or Delivery Productivity 590
Ambiguity in Measuring Complexity 593
Ambiguity in Functional Metrics 594
Ambiguity in Quality Metrics 599
Ambiguity with the Defects per KLOC Metric 599

Ambiguity with the Cost per Defect Metric 600
Failure to Measure Defect Potentials and Defect Removal Efficiency 602
The Problems of Measuring the Impact of "Soft" Factors 603
Problems in Measuring Software Value 607
Lack of Effective Measurement and Metrics Automation 609
Social and Political Resistance to Software Measurements 617
Ambiguity in Software Measurement and Metrics Terminology 620
Failure to Use Metrics for Establishing Goals and Targets 624
Summary and Conclusions 629
Suggested Readings 630
Additional References on Software Measurements 632

Appendix. Rules for Counting Procedural Source Code 635

Project Source Code Counting Rules 637
General Rules for Counting Code Within Applications 639
Examples of the SPR Source Code Counting Rules 640
Software Productivity Research COBOL-Counting Rules 642

Index 645

Foreword

A few years before Bill Gates and his friends started Microsoft—well, in 1604 actually—William Shakespeare wrote *Measure for Measure*, his penetrating meditation on the relationship between justice and mercy. It is a complicated tale involving pompous authority figures of unbending moral certainty, delegation of power to the ill-prepared, concealed and mistaken identities, abuse of the innocent, virtue corrupted, reputations ruined, and, of course, the ever-popular lust and greed. Now, everyone agrees that Shakespeare was a genius, but how could he so perfectly describe the modern software development environment *four hundred years in advance?*

Well, just as Shakespeare believed there must be a balanced, measured link between justice and mercy, we know that our IT endeavors must also seek balance among numerous observable, measurable, and often conflicting motives and influences. From function points to McCabe's complexity metrics, from Putnam's Productivity Index to Orthogonal Defect Classification, from Six-Sigma analysis to CMMI process improvement initiatives, the list of measurement and assessment tools seems nearly endless. While software metrics are critical to good project estimation, project management, product implementation, quality assurance, customer care, and outsource governance, how does an organization make informed and effective decisions as to which metrics to use, when to use them, and how to interpret them?

As it turns out, the unfortunate history of software development has been to avoid answering such questions for fear, it would seem, of "wasting time" on measuring and/or from an unhealthy anxiety for what the measures may actually reveal. Of course, at times the available software metrics and measurement methods have not always been robust enough to support the promise of their usefulness. Nevertheless, IT organizations on the leading edge of performance are the leaders in fearlessly gathering, studying, and using software metrics.

As Shakespeare said in *Measure for Measure,* "Truth is truth to the end of reckoning." Blaming the metrics or trembling at the prospect

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

of being measured (and perhaps judged) will not alter the underlying facts about an IT organization. In short, top performers learn to work with the metrics tools at hand, as any craftsman must do, and as a result, they have the most self-knowledge and insight, leading them to a better understanding of what they are doing, what is working, and what is not.

In 1991, my friend and colleague Capers Jones published the first edition of *Applied Software Measurement*, which comprehensively surveyed the field, describing the range of options available to IT organizations, as well as then current levels of performance across the industry. In so doing, Capers (perhaps to his surprise) created what many came to revere as the "bible" of software metrics. His revised second edition in 1996 covered even more ground and found even more adherents. As he has done since his early days at IBM decades ago, where he first applied his linguistics training to studying the relative productivity rates among programming languages, in his books Capers confronts conventional wisdom head on and above all challenges IT organizations and their leaders to measure their activities as a light on the path to understanding and improving them.

The degree to which the earlier editions of *Applied Software Measurement* influenced the global IT marketplace is difficult to overstate. An episode from my own experience may serve to illustrate.

In 1994, I was the Director of Corporate Software Appraisals for Electronic Data Systems. We were working on a rather cumbersome acquisition involving entities in multiple countries and a diverse portfolio of software. My team's objective was to support the independent appraiser with all relevant, auditable software metrics such that he could determine the fair market value of the software assets. Much depended upon this valuation.

The first big conference to plan and execute the appraisal was held late that year in Texas, and there were more attorneys present than one typically sees at a nationally televised Congressional hearing. Anxiety and apprehension were palpable among all assembled. Who knew how the appraisal would turn out? Would it make the deal, or would it sink it? Everything rested on the independent appraiser's viewpoint and methodology.

The appraiser was a delightful chap from Sydney, Australia, and he entered the room with a surprisingly slim briefcase. By contrast, many of the attorneys had junior associates wheeling around suitcases full of documents and reference materials. The appraiser asked if we had metrics on the software, including size, age, relative complexity, rates of change, and underlying technologies. You never saw such glazing over of the eyes among the lawyers, but one word stuck in their minds: "metrics." That, they all knew, was the responsibility of the fellow over

in the corner (me!), so all eyes shifted quickly in my direction. I informed the appraiser that we had all of those things and more, and I waved generally at a massive pile of reports and printouts.

"Then, ladies and gentlemen, our problem is practically solved," the Australian said. There were numerous audible gasps.

"You may ask the basis of my confidence, so I'll reveal it immediately," he said, and he opened his little briefcase. "Here it is!" he exclaimed, and he raised a dog-eared copy of the yellow, now so familiar 1991 edition of *Applied Software Measurement,* its pages clogged with clusters of sticky notes, and (as I learned later) filled with vigorous highlighting marks and penciled annotations. "This is the bible, mates," the appraiser told us. "With the rigorous measurements you've taken, I can tell you almost anything about this software—and all according to Capers Jones' documented techniques and recommendations!"

The gasps turned to smiles, and pretty soon the lawyers decided we could break for lunch early!

That was my first introduction to *Applied Software Measurement,* and although the triumphant occasion didn't win any of us a seat on the board of directors, it certainly confirmed the importance of software metrics and the broad scope of their application to business problems. (Incidentally, in time it also led me to working closely with Capers and eventually to becoming a partner in his company.)

Over the years numerous similar episodes have occurred, including most memorably the CIO of one the largest food and beverage conglomerates in the world openly weeping when I presented a signed copy of the revised 1996 edition. Dramatics aside, however, this new, completely revised edition of *Applied Software Measurement* will surely take an important place alongside its predecessors. At Software Productivity Research, we live by the following mantra, which is integral to every aspect of this book and comes from Jones—not Shakespeare:

Measure to Know
Know to Change
Change to Lead.

–Doug Brindley
President and CEO
Software Productivity Research, LLC

This page intentionally left blank

Preface to the Third Edition

The first edition of *Applied Software Measurement* was published in 1991. The second edition was published in 1996. Thus about 18 years have passed since the original data was first collected, put into book form, and published.

Many changes have occurred in the software industry between 1991 and 2008. In 1991, personal computers were in their infancy and the World Wide Web was just beginning to move from Europe to the United States, where an early version was installed at the Stanford Linear Accelerator (SLAC) in California.

In 1991, the function point metric was just starting to become the de facto standard for measuring software productivity and quality, in place of the older and troublesome "lines of code" metric.

In 1991, the Software Engineering Institute (SEI) was about six years old and was just starting to reach national prominence because of the publication of its "capability maturity model" (CMM), which was among the first approaches able to classify the sophistication of software development methods.

The data available for the first edition in 1991 consisted primarily of several thousand software projects studied by the author and his colleagues at Software Productivity Research (SPR).

By the time of the second edition in 1996, not only had SPR examined several thousand more projects but the Web was sophisticated enough so that queries could be made to other companies about available data, which increased the numbers of projects available for analysis.

Today, as the book is written in 2008, the Web has become the primary research tool for scientists in every discipline. Therefore, dozens of colleagues and hundreds of web sites can be contacted with very little effort.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

However, software measurement is still somewhat less than optimal—both in terms of the measurement techniques themselves and also in terms of the volume and reliability of published data.

In 1997, a nonprofit organization called the International Software Benchmarking Standards Group (ISBSG) was formed to collect information about software productivity and quality. This is the first "public" source of benchmark data, where clients can buy both the data itself and also books and reports derived from the data.

In 2008, the ISBSG organization is continuing to expand in terms of numbers of projects and also in terms of statistical analysis of results. It is useful to the industry to have a consolidation point for software productivity and quality data.

However, the ISBSG organization uses data that is gathered and reported by hundreds of separate companies. Since it is a known fact that many corporate and government cost and effort tracking systems "leak" and omit portions of actual effort, the precision of the ISBSG data collection is somewhat ambiguous in 2008. The most likely issue with the ISBSG data is that "leakage" from resource tracking systems might artificially indicate higher productivity levels than really occur.

Since the normal practice of corporate measurement is to "leak" unless great care is taken, perhaps 25 percent of the ISBSG projects would be fairly complete, but the remainder might omit some activities. Unfortunately, it would be necessary to go onsite and interview team members to find out what has been included and what has been left out.

In general, data reported via questionnaire or emailed surveys tends to be less accurate than data collected via face-to-face interviews. This is not a criticism of the ISBSG questionnaires. For example, the main questionnaire for development projects is 38 pages in length and quite complete in coverage. The uncertainty of precision in the ISBSG data centers on whether the clients who use the questionnaire actually have enough solid data to answer the questionnaire fully and accurately.

Although the precision of the ISBSG data is somewhat uncertain in 2008, the organization itself is continuing to grow and evolve and no doubt will work to improve precision in the future. As this book is written in 2008, the ISBSG data consists of almost 4,000 projects and is growing rapidly. The various ISBSG reports and data collections have already become valuable to the industry.

The Software Engineering Institute (SEI) has also undertaken a significant amount of data collection and analysis. Additionally, SEI has developed a newer form of process improvement methodology called the "capability maturity model integration" (CMMI). This new

model builds on the success of the older CMM and adds additional features. Both the CMM and CMMI have demonstrated tangible improvements for large software projects in the 10,000 function point size range and higher.

Building on the concepts of the CMM, Watts Humphrey has developed team software process (TSP) and personal software process (PSP) concepts that are showing excellent results in terms of both productivity and quality. These new approaches also utilize some special metrics and measures, as will be discussed later in the book.

What would be useful to the industry would be consolidating some of the data collected by the SEI with data collected by ISBSG and perhaps with other benchmarking groups as well, such as Software Productivity Research, the David Consulting Group, and Quantitative Software Management, among others. This is not likely to happen in the near term because some of these organizations are business competitors, whereas others regard their data as valuable and proprietary.

Starting about 2003, Dr. Victor Basili of the University of Maryland developed a software metrics approach called "goal/question metrics" that attempts to utilize special customized metrics that match the business goals of the application. This method is growing in popularity and usage.

In about 2004, Tony Salvaggio the president of Computer Aid Inc. (CAI) created an interesting research organization named the "Information Technology Metrics and Productivity Institute" or ITMPI. This new group began to assemble reference material about software engineering and software management topics. ITMPI also organized a series of seminars and webinars with some of the top-ranked software experts in the world.

Today, in 2008, the ITMPI organization has become one of the major industry sources of data on software methodologies and process improvement topics. Within a few years if ITMPI adds actual benchmark data, this organization will become an even more valuable industry resource.

Another interesting technology that has emerged since the second edition of this book was published in 1996 is that of "Agile development." The phrase "Agile development" is an umbrella term for a number of different software development approaches such as Extreme programming (XP), Adaptive Software Development, feature-driven development, and Crystal Clear.

These Agile methods have in common the view that conventional requirements and design approaches are time consuming and ineffective. Rather than attempting to document requirements in paper form, the Agile methods include full-time user representatives as part

of the team and develop working models or prototypes of features as quickly as possible. Frequent team meetings or "Scrum" sessions are also a key component of the Agile approach.

The Agile methods also have developed some unique measurement (and estimation) methods. These will be discussed later in the book.

The Agile methods started with fairly small projects less than 1,000 function points in size or the opposite end of the spectrum from the SEI's capability maturity model which started with large systems in the 10,000 to 100,000 function point range. Although the Agile approach and the SEI started at different ends of the spectrum, both are now covering software projects whose sizes range from less than 1,000 function points to more than 100,000 function points.

There are dozens of software methodologies in existence as this book is written. There were dozens of methods during the previous editions too, but many have fallen out of favor such as CASE and RAD which are now being subsumed by Agile and Extreme programming. An interesting trend in 2008 is that of "process fusion" or the merging useful portions of several different methodologies. In fact, this approach is becoming so popular that the consultant Gary Gack has even formed a company named "Process Fusion." In the future, service-oriented applications (SOA) are emerging, and even newer methods such as agent-based development are being discussed.

Some of the methodologies with complementary features include Six-Sigma and the CMMI, Six-Sigma and Watts Humphrey's team software process (TSP), Agile development and "lean Six-Sigma," and the CMMI merged with aspects of the Rational Unified Process (RUP). Overall, there are scores of possible mergers of various development methods.

Whereas the Agile approaches concentrate on software development, the Information Technology Infrastructure Library (ITIL) concentrates on maintenance, services, and customer support. This book will discuss some of the kinds of metrics associated with service management under the ITIL approach.

Another major industry change since 1991 and since 1996 as well has been in the development of function point metrics themselves. When the first edition of this book was published in 1991, function points were used primarily for information technology projects. Today, in 2008, they are used for all forms of software, including web projects, systems software, embedded software, and even military weapons systems.

In 1996, at the time of the second edition, the International Function Point Users Group (IFPUG) was the dominant source of function point data in the United States and indeed in much of the world.

The British Mark II function point metric was widely used in the United Kingdom, and there were also a few users of the "feature point" metric, which was similar to IFPUG function points but aimed at systems software.

Over the years the IFPUG counting standards have been updated and modified as new kinds of software and new topics needed to be included. As this book is written, the IFPUG counting rules are currently at version 4.2. When the first edition was published in 1991, the IFPUG counting rules in effect were version 2.0. By 1996, the IFPUG rules in effect were at version 4.0. This meant that the data in the current edition needed minor recalibration.

Today, in 2008, there are at least 22 variant forms of function point metrics including (in alphabetical order):

- 3D function points
- Backfiring (mathematical conversion from lines of code)
- Bang metrics (DeMarco function points)
- COSMIC function points
- Engineering function points
- Feature points
- Full function points
- Function points by analogy
- Function points "light"
- IFPUG function points
- ISO Standard 19761 for functional sizing
- Mark II function points
- Micro function points for very small projects
- Netherlands Software Metrics Association (NESMA) function points
- Object points
- Partial function points (reconstruction of missing elements)
- Quick Sizer function points
- Software Productivity Research (SPR) function points
- Story points
- Unadjusted function points
- Use case points
- Web Object Points

Unfortunately, there are few rules for converting from one function point variant to another. As it happens, many of the variants were developed because of a feeling that standard IFPUG function points undercounted certain kinds of software, such as real-time and embedded software. Since this feeling was the main motive for creating a number of variants, some of the project size values for these variants are perhaps 15 percent larger than IFPUG function points for the same applications.

Exceptions to this rule include feature points, SPR function points, and Quick Sizer function points, which were intended to yield results that were in close proximity to standard IFPUG function points.

Standard IFPUG function points are not practical below a size of about 15 function points, due to limits on the lower range of adjustment factors. For many years, there has been a gap below 15 function points, which means that small maintenance projects have not been able to be evaluated using functional metrics. The new "micro function point" introduced in this edition can span a range that runs from 0.001 function points up to about 20 function points and are intended for use on very small projects below the level of normal counting. This method is based on standard IFPUG function points, version 4.2. Micro function points utilize two (or more) decimal places of precision and can, therefore, deal with very small values such as a small update of 0.25 function points in size.

The explosion of function point clones raises an interesting technical issue. Some kinds of software projects are much harder to develop than other kinds, due to more complex logical problems, more complex algorithms, and more complex data structures. For these difficult kinds of software, is it best to use standard IFPUG function points and accept a lower productivity rate, or should a special kind of function point be developed that yields larger values for more complex applications?

In other words, if application A is twice as difficult as application B, is it best to express the difficulty in terms of 50 percent reduction in the development productivity rate, or should the difficulty be expressed in terms of a 100 percent increase in function point size?

From an economic standpoint, it would seem more reasonable to express the increased difficulty by means of a lower productivity rate. That way the costs of complex software applications will be put in proper perspective compared to simpler applications.

If the object to be constructed is a tangible object such as a house, it is easier to understand the situation. Suppose house A, which is 3,000 square feet, is going to be built on a very difficult and remote mountainous site that will raise construction costs. Now suppose that a similar house B, also 3,000 square feet, is going to be built on a normal suburban lot.

The cost per square foot for house A might be $500 whereas the construction cost per square foot for house B might be only $250. For houses, it is obvious that added complexity and difficulty in construction will result in a higher cost per square foot. It is not possible to claim that houses that are difficult to build have more square feet than houses that are easy to build.

But software features and requirements are not tangible, so we face the ambiguity of determining whether increased difficulty and complexity should be measured in terms of lower productivity rates or in terms of larger function point values. Overall, it would seem advantageous to the software industry to utilize one specific kind of function point metric and deal with complexity and construction difficulties by adjusting productivity rates and costs per function points.

Although a great many function point clones have been developed in recent years, there are many other important software topics where some form of functional metric might be applied, but where there is little, if any, research. For example, as of 2008, there is no effective metric for measuring the size or volume of data in databases and repositories. As a result, there is no solid empirical information on data quality, data costs, or other economic issues associated with data. A "data point" metric using a format similar to function points would be extremely valuable to the industry.

Also, there are no effective metrics for dealing with the full spectrum of values that software applications return. Financial value can be measured reasonably well, but there are many kinds of value that do not lend themselves to simple financial terms: medical applications that can save lives, military applications for national defense, scientific applications such as the Hubble telescope, and a host of others. For non-financial value topics, a "value point" metric using the format of function point metrics would be an important addition to the software measurement suite.

As the software industry moves toward service-oriented offerings and service-oriented architectures, there is also a need for a form of "service point" metric to deal with some of the new issues. Even today, there are many kinds of services such as customer support that surround software projects. These need to be included in the economic measures of software.

What the software industry truly needs is not 22 flavors of function point metrics, but rather a suite of related functional metrics that can deal with software costs, data costs, service costs, and both software and data value in an integrated fashion. This book will discuss some of the features of these extended forms of functional metric.

Readers who are unfamiliar with software economic issues may wonder why the traditional "lines of code" metrics have been supplanted

by function point metrics for economic studies. Since function point metrics are somewhat difficult and expensive to count, what value do they provide that lines of code metrics do not provide?

An obvious issue with lines of code metrics is that there are more than 700 programming languages and dialects in use circa 2008. Of the approximately 700 programming languages and dialects in use today, there are formal code counting rules for less than 100. For at least 50 languages, such as Visual Basic, there are no effective counting rules for ascertaining source code size because much of the "programming" is done using alternatives such as pull-down menus and buttons rather than using procedural code.

Over and above the ambiguity in the way code is counted, there is a fundamental economic flaw with using lines of code metrics. High-level programming languages are penalized, whereas low-level languages appear to be more productive than they truly are.

The reason that LOC metrics give erroneous results with high-level languages is because of a classic and well known business problem: the impact of fixed costs. Coding itself is only a small fraction of the total effort that goes into software. Paperwork in the form of plans, specifications, and user documents often costs much more. Paperwork tends to act like a fixed cost and that brings up a well-known rule of manufacturing: "When a manufacturing process includes a high percentage of fixed costs and there is a reduction in the number of units manufactured, the cost per unit will go up."

Here is a simple example, showing both the lines-of-code results and function point results for doing the same application in two languages: basic Assembly and C++.

Assume that the Assembly language program required 10,000 lines of code, and the various paper documents (specifications, user documents, etc.) totaled to 100 pages. Assume that coding and testing required 10 months of effort, and writing the paper documents took 5 months of effort. The entire project totaled 15 months of effort and so has a productivity rate of 666 LOC per month. At a cost of $10,000 per staff month, the application cost $150,000. Expressed in terms of cost per source line, the costs are $15.00 per line of source code.

Assume that the C++ version of the same application required only 1,000 lines of code. The design documents probably were smaller as a result of using an OO language, but the user documents are almost the same size as in the previous case: assume a total of 75 pages were produced. Assume that coding and testing required 1 month and document production took 4 months. Now we have a project where the total effort was only 5 months, but productivity expressed using LOC has dropped to only 200 LOC per month. At a cost of $10,000 per staff

month, the application cost $50,000 or only one-third as much as the Assembly language version. The C++ version is a full $100,000 less expensive than the Assembly version, so clearly the C++ version has much better economics. But the cost per source line for this version has jumped to $50.00.

Even if we measure only coding, we still can't see the value of high-level languages by means of the LOC metric: the coding rates for both the Assembly language and C++ versions were both identical at 1,000 LOC per month, even though the C++ version took only 1 month as opposed to 10 months for the Assembly version.

Since both the Assembly and C++ versions were identical in terms of features and functions, let us assume that both versions were 50 function points in size. When we express productivity in terms of function points per staff month, the Assembly version had a productivity rate of 3.33 function points per staff month. The C++ version had a productivity rate of 10 function points per staff month. When we turn to costs, the Assembly version cost $3,000 per function point whereas the C++ version cost $1,000 per function point. Thus, function point metrics clearly match the assumptions of standard economics, which define productivity as "goods or services produced per unit of labor or expense."

Lines of code metrics, on the other hand, do not match the assumptions of standard economics and, in fact, show a reversal. Lines of code metrics distort the true economic case by so much that their use for economic studies involving more than one programming language might be classified as professional malpractice.

The only situation where LOC metrics behave reasonably well is when two projects utilize the same programming language. In that case, their relative productivity can be measured with LOC metrics. But if two or more different languages are used, the LOC results will be economically invalid.

In many fields, there are multiple metrics for the same phenomenon. Thus, we have statue and nautical miles, Fahrenheit and Celsius temperature metrics, and several varieties of horsepower metrics. At this point in the history of software, we will probably continue to have quite a few variants in the way function points are counted and continue with variants in counting lines of code. What the software industry does need are three things:

- Accurate conversion rules between all of the function point variants
- Accurate conversion rules between size measured in physical lines of code and measured in logical code statements
- Accurate conversion rules between source code measurements and function point measurements

Because there are at least 5 variations in source code counting methods and 20 variations in function point counting methods, the total number of conversion rules would be 100. It is obvious that the software industry needs to reduce the number of variations in how things are counted and begin to move toward a smaller set of metrics choices.

This book will include some provisional conversion rules, but as of 2008, there is insufficient data for rigorous conversion rules to be defined. Rigorous rules would require counts of the same applications by certified counters of the various metrics in question. For example, the same application would have to counted by certified counters in IFPUG function points, certified counters in COSMIC function points, and perhaps certified counters in other metrics as well.

The data in this book will all be presented using IFPUG counting rules, version 4.2. The practical reason for this is that the IFPUG function point method has more historical data available circa 2008 than all of the other variants put together, although use of COSMIC function points is expanding in Europe.

Since this book covers almost 20 years of software history and the author has been involved with software for more than 40 years, there are several troubling observations about the software industry circa 2008:

- At a national level, software productivity rates for traditional systems and information systems have stayed comparatively flat for the past 20 years.
- At a national level, software quality levels have only improved slightly for the past 20 years.
- From working as an expert witness in many software lawsuits, the issues that lead to litigation in 2008 are essentially identical to the issues that caused problems in 1987.
- Litigation for excessive cost and schedule overruns or total failure of software projects appears to be increasing rather than declining in 2008.
- The best companies in terms of software productivity and quality are about three times better than U.S. averages.
- The worst companies in terms of software productivity and quality are about 50 percent worse than U.S. averages.
- The number of companies that are improving productivity and quality seems to be only about 15 percent of all companies in 2008.
- The number of companies that are getting worse seems to be about 15 percent of all companies in 2008.

- The number of companies that are neither improving nor getting worse seems to be about 70 percent of all companies in 2008.
- Maintenance activities continue to grow at a faster rate than software development activities as portfolios age and decay.

The most troubling observation, and the one which explains the fact that national productivity and quality levels for many forms of software seem almost flat for the past 20 years, is the unexpected discovery that as many companies are regressing or moving backward as there are companies moving forward.

In fact, the main reason that national software productivity rates have improved is due to the emergence of web applications, which usually have high productivity in part due to their use of Agile methods. Some benefits also accrue from the SEI CMM and from Watts Humphrey's team software process (TSP) and personal software process (PSP).

This topic will be taken up later in the book. But what seems to happen is that after an initial success with process improvement, the original team and the original managers either change jobs or are promoted. The new managers and executives want to put in their own preferred methods, which often are not as successful as the ones being replaced.

This brings up a hidden weakness of the software industry, which is a major theme of this book: Good measurements have lasting value and can be used to perpetuate successful software practices.

One of the main reasons why successful process improvement activities tend to drop from use is because the new managers and team members have no solid data on the economic value of either the improved way of building and maintaining software or the replacement way. Thus changes in software methods tend to be regarded as sociological issues rather than the economic issues that they truly are.

The main purpose of the third edition of *Applied Software Measurement* is to show the kinds of measurements and metrics that can place software economics on a sound empirical base. The book will discuss the most effective quantitative measurement approaches for eight key software topics:

- Software application size
- Software development schedules
- Software development staffing
- Software development productivity
- Software maintenance productivity

- Software costs in all forms
- Software quality
- Software value

However, even good quantitative data is not enough to ensure success. It is also necessary to collect some qualitative subjective data as well. The qualitative issues that need to be collected include

- Development and maintenance team capabilities
- Requirements stability and changes over time
- Specialists available to aid in software development and maintenance
- Organizational structures in place for large applications
- Constraints such as legal deadlines or mandated features
- Standards that the project is required to adhere to
- International topics such as translations or global development
- Earned value measures

The phrase "applied software measurement" refers to measurements that can provide software managers, executives, developers, and clients with unambiguous economic information. The phrase also includes the concept of using quantitative data to ensure high levels of professionalism. In other words, software projects need to use state-of-the-art measurements as well as state-of-the-art development methods.

Without good measurements, the results of good development methods will be invisible or unconvincing. Without good development methods, even good measurements will only show schedule delays and cost overruns. Both measures and methods should be deliberately selected and applied to optimize the economic value of software projects and to minimize the risks of unsuccessful development.

As of 2008, a majority of software organizations have few measurements of any kind. Perhaps 20 percent of software organizations have some form of quality measurement and some form of productivity measurement using functional metrics. A dismaying 10 percent of organizations still attempt to measure using lines of code, with, of course, totally incorrect economic results. This is not a satisfactory situation, and poor measurement practices are part of the reason why so many software projects fail and so many run late and exceed their budgets.

Without solid empirical data based on valid measurement of results, progress in software (or any other industry) will be erratic at best. *Applied Software Measurement* will hopefully show the kinds of information that

should be collected and analyzed to put development, maintenance, and service process improvements on a firm empirical basis.

Because measurements themselves have costs, another topic that will be discussed in this book is the value of measurements. Because good measurements are a necessary precursor to successful process improvement work, the actual return on investment (ROI) in a solid measurement program is significant. The actual ROI varies with the size of the company and the nature of the work.

Overall, a good measurement program is an effective method for demonstrating the value of process improvements, new tools, better organizations, and many other business activities applied to software operations.

This page intentionally left blank

Acknowledgments

As ever, thanks to my wife Eileen for her support and patience when I'm writing my books.

Thanks also to my colleagues who provided some of the new information for the third edition: Michael Bragen, Doug Brindley, Gary Gack, Peter Hill, Watts Humphrey, Tom Love, Steve Kan, and Tony Salvaggio.

Because this book is a third edition and built on the existing data, thanks are also due to my many friends and colleagues from Software Productivity Research from 1984 to today. Some still work at SPR and others have formed their own companies or moved to existing companies, but without the solid contributions of many consultants and managers, the data would not have been collected. Thanks to Allan Albrecht, Michael Bragen, Doug Brindley, Tom Cagley, Mike Cunnane, Chas Douglis, Gail Flaherty, David Garmus, Scott Goldfarb, the late Bill Harmon, Deborah Harris, David Herron, Steve Hone, Richard Kauffman, Bob Kendall, David Longstreet, John Mulcahy, Mark Pinis, and John Zimmerman for their help in collecting and analyzing software data.

Thanks also to the rest of the SPR team who built the tools, sold the studies, and handled finances and administration: Ed Begley, Barbara Bloom, Julie Bonaiuto, Kristin Brooks, Lynne Caramanica, Sudip Chakraborty, Criag Chamberlin, Debbie Chapman, Carol Chiungos, Jon Glazer, Wayne Hadlock, Shane Hartman, Jan Huffman, Richard Kang-Tong, Peter Katsoulas, Scott Moody, Donna O'Donnel, and Susan Turner.

Thanks to Ajit Maira for suggesting the title *Applied Software Measurement* for the first edition of this book.

Thanks to Wendy Rinaldi and the McGraw-Hill production team for support of this book. Madhu Bhardwaj has worked on two of my books and the results have been excellent. The support of other McGraw-Hill

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

team members such as Janet Walden, Claire Splan, and LeeAnn Pickrell have made international production enjoyable.

Thanks to Michael Milutis and the CAI and ITMPI staff for providing excellent support for distributing this and many other software books and articles. The Information Technology Metrics and Productivity Institute is now a major portal for software metrics and measurement information.

Chapter

1

Introduction

The third edition of *Applied Software Measurement* confirms the conclusions of the first and second editions, that measurement is a key technology for successful software development and maintenance. Indeed, measurements have been a pivotal component in the progress of the software industry in the years between the publication of the first and second editions. In particular, the usage of function point metrics has exploded across the software world.

In 1990, function points were just beginning to expand in usage. By 1996, function point metrics had become the dominant metric in the United States and about 20 other countries. By 2007, function point metrics had become the main basis of software benchmarks, but an unexpected issue occurred. As of 2007 there were about 20 function point variants, and few conversion rules from one variant to another. The data in this book is based on the function point metrics defined by the International Function Point Users Group (IFPUG) version 4.2 of the counting rules.

Yet another trend occurred that also affects measurement technology. As new software development methodologies have been created, they tend to create their own special kinds of measurements. For example, the object-oriented method has created some specialized OO metrics. Many of the organizations that deploy "use cases" now estimate and measure with "use case point." Some of the Agile methods also utilize special metrics, as do the practitioners of Watts Humphrey's team software process (TSP).

The problem with these specialized metrics that have evolved in tandem with development methods is that many of them have such a narrow focus that they only measure one technology. For example, it is not possible to compare two projects using "use case points" if one of the projects handled design with use cases and the second handled design with ordinary structured design.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

As of 2008 the software industry has perhaps 50 kinds of development methods to choose from, and that means we have almost as many metrics and measurement approaches. It is not possible to create overall industry benchmarks using 50 different kinds of metrics. This book is based on the assumption that in spite of the local value of specialized metrics for specific technologies, industry-wide data has to be expressed using a single metric and measurement approach. Therefore, IFPUG function points, using the version 4.2 counting rules, have been chosen to express all of the data in this book.

Software development and maintenance started to become major corporate concerns in the last half of the 20th century. Although most companies could not survive or compete successfully without software and computers, even in 2008 senior executive management remains troubled by a set of chronic problems associated with software applications: long schedules, major cost overruns, low quality, and poor user satisfaction.

These problems have occurred so widely that it is fair to characterize them as a "corporate epidemic." Yet software is not out of control in every company. The companies that have been most successful in bringing software under control tend to share most of these seven characteristics:

- They measure software productivity and quality accurately.
- They plan and estimate software projects accurately.
- They have capable management and technical staffs.
- They have good organization structures.
- They have effective software methods and tools.
- They have adequate staff office environments.
- They are able to reuse substantial volumes of software deliverables.

All seven characteristics are important, but the first is perhaps the most significant of all, since it tends to permeate and influence the others. Let us consider each of the seven in turn.

Applied Software Measurement

Measurement is the basis of all science, engineering, and business. Unfortunately, software developers lagged far behind other professions in establishing both standard metrics and meaningful targets. The phrase "applied software measurement" refers to the emerging discipline associated with the accurate and meaningful collection of information that has practical value to software management and staffs. The goal of applied software measurement is to give software managers and

professionals a set of useful, tangible data points for sizing, estimating, managing, and controlling software projects with rigor and precision.

For many years, measuring software productivity and quality was so difficult that only very large companies such as IBM attempted it. Indeed, one of the reasons for IBM's success was the early emphasis the company put on the applied measurement of quality and productivity, which gave IBM the ability to use the data for corrective purposes. But stable metrics and accurate applied measurement of software have been possible since 1979, and now every company can gain the benefits and insights available from applied software measurement.

The problem today is not a deficiency in software measurement technology itself; rather, it is cultural resistance on the part of software management and staff. The resistance is due to the natural human belief that measures might be used against them. This feeling is the greatest barrier to applied software measurement. The challenge today is to overcome this barrier and demonstrate that applied software measurement is not harmful, but as necessary to corporate success as standard financial measurements.

What tends to separate leading-edge companies from trailing-edge companies are not only technical differences but cultural differences as well. Project managers in companies at the leading edge, such as Microsoft, IBM, DuPont, and Hewlett-Packard, may have ten times as much quantified, historical information available to them to aid in project planning as their peers in companies at the trailing edge.

Managers in leading-edge companies also have accurate demographic and morale information available, which is largely missing at the trailing edge. Not only is such information absent at the trailing edge, but the managers and executives within trailing-edge companies are often deluded into thinking their companies are much better than they really are!

Planning and Estimation

Planning and estimation are the mirror images of measurement. The factors and metrics that were recorded during project development are now aimed toward the future of uncompleted projects. There is a perfect correlation between measurement accuracy and estimation accuracy: Companies that measure well can estimate well; companies that do not measure accurately cannot estimate accurately either.

Commercial-grade estimation tools have been available since the mid-1970s, and they are now becoming widely deployed. Here too measurement is significant, since only companies with accurate historical data can validate estimates and judge their accuracy. Leading-edge enterprises normally do not attempt to estimate large projects by hand; instead, they use either proprietary tools based on their own history or commercial estimating tools based on general industry data.

Management and Technical Staffs

Leading-edge companies tend to attract and keep good managers and good technical staffs. What attracts such people appears to be exciting projects, excellent working conditions, and the pleasure of working with capable colleagues. Although it is outside the scope of this book, leading-edge companies such as IBM, Google, and Microsoft tend to go out of their way to measure employee satisfaction by means of annual corporate opinion surveys. Trailing-edge companies have trouble keeping capable management and staff. What causes the dissatisfaction are poorly conceived or canceled projects, inadequate working conditions, primitive tools and methods, and the lack of stimulating colleagues and effective project management. Trailing-edge companies are seldom aware of these problems because they lack any form of measurement or opinion survey.

Organization Structures

Software circa 2008 is becoming specialized just as medicine and law have become specialized. As special technical skills are needed, such as those of database administrators, quality assurance specialists, human factors specialists, and technical writers, it becomes more and more important to plan organization structures carefully. Indeed, among the hallmarks of the larger leading-edge corporations are measurement specialists and measurement organizations. One of the useful by-products of measurement is the ability to judge the relative effectiveness of organization structures such as hierarchical vs. matrix management for software projects and centralization vs. decentralization for the software function overall. Here too, measurement can lead to progress and the lack of measurement can lead to expensive mistakes.

Methodologies and Tools

The labor content of software projects is extraordinarily high. Very few artifacts require as much manual labor as a large software system. Many software methodology, tool, language, and technology vendors claim to displace human effort through automation with 10- or 20-to-1 improvements in productivity. Are such claims justified? Generally, they are not. Only companies that measure software productivity and quality can find their way through the morass of conflicting assertions and pick a truly effective path. As it turns out, heavy investment in tools prior to resolving organizational and methodological issues is normally counterproductive and will improve neither quality nor productivity. Only accurate measurements can navigate a path that is both cost-effective and pragmatic.

The Office Environment

Yet another aspect of leading-edge companies is a surprising one: The companies tend to have adequate office space and good physical environments. It appears that, for knowledge workers such as software professionals, the impact of physical office environments on productivity may be as great as the impact of the tools and methods used. Open offices and overcrowding tend to lower productivity, whereas private offices and adequate space tend to augment it. Findings such as that are possible only from accurate measurements and multiple-regression studies of all the factors that influence human endeavors.

Art is normally free-form and unconstrained; business is normally under management control. In companies with no measurement practices, software projects are essentially artistic rather than business undertakings. That is, there is no way for management to make an accurate prediction of the outcome of a project or to exert effective control once the project has been set in motion. That is not as it should be. Software should be a normal business function with the same rigor of planning, estimating, risk, and value analysis as any other corporate function. Only measurement, carefully applied, can convert software production and maintenance from artistic activities into business activities.

Although many different sociological and technological steps may be needed to bring software under management control in a large company, all of them require accurate measurement as the starting point. That is true of all physical and social systems: Only measurement can assess progress and direction and allow feedback loops to bring deviations under control.

Reusability

A topic of growing importance to the software community is the ability to reuse a number of software artifacts. Because manual development of software applications is highly labor-intensive, the ability to reuse material is one of the critical steps that can improve both software quality and productivity simultaneously. Twelve software "artifacts" are potentially reusable, and a successful reuse program will include all twelve:

- Reusable requirements
- Reusable architecture
- Reusable plans
- Reusable cost estimates
- Reusable designs
- Reusable source code

- Reusable data elements
- Reusable interfaces
- Reusable screens
- Reusable user manuals
- Reusable test plans
- Reusable test cases

Software reusability was largely experimental when the first edition of this book was published, but under the impact of many new tools and also new programming languages such as Java, Visual Basic and object-oriented (OO) languages, reuse is starting to enter the mainstream.

The Essential Aspects of Applied Software Measurement

Three essential kinds of information must be considered when dealing with applied software measurement, or the measurement of any other complex process involving human action:

- Hard data
- Soft data
- Normalized data

All three kinds of information must be recorded and analyzed in order to gain insights into productivity or quality problems.

Hard Data

The first kind of information, termed *hard data*, refers to things that can be quantified with little or no subjectivity. For the hard-data elements, high accuracy is both possible and desirable. The key hard-data elements that affect software projects are

- The number of staff members assigned to a project
- The effort spent by staff members on project tasks
- The schedule durations of significant project tasks
- The overlap and concurrency of tasks performed in parallel
- The project document, code, and test case volumes
- The number of bugs or defects found and reported

Another key "hard data" element is that of the costs expended for each activity, and the total costs for the project. Unfortunately, costs are more tricky than other forms of hard-data measurement because software tends to utilize rather large quantities of unpaid overtime. Thus, effort measured in dollars and effort measured in hours are not always the same.

Although hard data can in theory be measured with very high accuracy, most companies are distressingly inaccurate in the data they collect. Factors such as unpaid overtime by professional staff members, management costs, user involvement, and frequent project accounting errors can cause the true cost of a software project to be up to 100 percent greater than the apparent cost derived from a normal project-tracking system. That fact must be evaluated and considered when starting a serious corporate measurement program: It will probably be necessary to modify or replace the existing project cost-tracking system with something more effective.

The solution to inaccurate tracking accuracy is not technically difficult, although it may be sociologically difficult. It is only necessary to establish a standard chart of accounts for the tasks that will normally be performed on software projects and then collect data by task. As of 2008 there are commercial tracking tools for software development and maintenance work that are easy to use and facilitate collecting data with minimal intrusion on ordinary tasks.

The sociological difficulty comes in recognizing that manual tracking systems tend to omit large volumes of unpaid overtime, user effort on projects, managerial effort, and often many other activities. Senior executives and software project managers must have access to an accurate accounting of what the true cost elements of software projects really are.

One of the most frequent problems encountered with project historical data that lowers data accuracy is a simple lack of granularity. Instead of measuring the effort of specific activities, companies tend to accumulate only "bottom line" data for the whole project without separating the data into meaningful subsets, such as the effort for requirements, design, and coding. Such bottom-line data is essentially worthless because there is no real way to validate it.

For purposes of schedule and management control, it is common to break software projects into somewhere between five and ten specific phases such as "requirements, design, coding, testing, and installation." Such phase structures are cumbersome and inadequate for cost measurement purposes. Too many activities, such as production of user documentation, tend to span several phases, so accurate cost accumulation is difficult to perform.

Table 1-1 is an example of the standard chart of accounts used by Software Productivity Research when collecting the project data shown in the later sections of this book. It illustrates the kind of granularity by activity needed for historical data to be useful for economic studies. This chart of accounts is based on activities rather than phases. An "activity" is defined as a bounded set of tasks aimed at completing a significant project milestone. The milestone might be completion of requirements, completion of a prototype, or completion of initial design. As can easily be seen, variations in the activities that are usually performed are one of the primary reasons why productivity varies from industry to industry and class to class of software.

TABLE 1-1 Example of Standard Software Charts of Accounts for Six Domains

Activities Performed	Web	MIS	Outsource	Commercial	Systems	Military
1. Requirements		X	X	X	X	X
2. Prototyping	X	X	X	X	X	X
3. Architecture			X	X	X	X
4. Project plans		X	X	X	X	X
5. Initial design		X	X	X	X	X
6. Detail design		X	X	X	X	X
7. Design reviews			X	X	X	X
8. Coding	X	X	X	X	X	X
9. Reuse acquisition	X		X	X	X	X
10. Package purchase		X	X		X	X
11. Code inspections				X	X	X
12. Independent verification and validation						X
13. Configuration management		X	X	X	X	X
14. Formal integration		X	X	X	X	X
15. User documentation	X	X	X	X	X	X
16. Unit testing	X	X	X	X	X	X
17. Function testing		X	X	X	X	X
18. Integration testing		X	X	X	X	X
19. System testing		X	X	X	X	X
20. Field testing				X	X	X
21. Acceptance testing		X	X		X	X
22. Independent testing						X
23. Quality assurance			X	X	X	X
24. Installation and training		X	X		X	X
25. Project management		X	X	X	X	X
Activities	5	15	20	21	22	25

The six kinds of software illustrated by Table 1-1 include projects for the Web, management information systems (MIS), outsourced projects, commercial software, systems software, and military software.

Although Table 1-1 uses 25 standard activities, that does not imply that only 25 things must be done to develop a software project. The 25 activities are accounting abstractions used to accumulate costs in a convenient manner. Any given activity, such as "requirements," obviously consists of a number of significant subactivities or specific tasks. Indeed, even small software projects may require hundreds of specific tasks in a full work breakdown structure; large projects may require many thousands. A task-level chart of accounts, although capable of high precision, tends to be very cumbersome for cost accumulation, since project staff must record times against a very large number of individual tasks.

The 25 activities listed in Table 1-1 illustrate the level of granularity needed to gain economic insights into the major costs associated with software.

Unless a reasonably granular chart of accounts is used, it is difficult to explore economic productivity in ways that lead to insights. For example, it has been known for many years that military software projects typically have lower productivity rates than civilian software projects of the same size. One of the key reasons for that can easily be seen by means of a granular chart of accounts.

MIS projects normally perform only 15 of the 25 standard activities; systems software projects perform about 22; and military projects normally perform all 25. The need to perform almost twice as many activities explains why military projects are usually quite low in productivity compared to civilian projects.

Soft Data

The second kind of information, termed *soft data*, comprises topics in which human opinions must be evaluated. Since human opinions will vary, absolute precision is impossible for the soft data. Nonetheless, it is the soft data, taken collectively, that explains variations in project outcomes. The key soft data elements that affect software projects are

- The skill and experience of the project team
- The constraints or schedule pressures put on the team
- The stability of project requirements over time
- User satisfaction with the project
- The expertise and cooperation of the project users
- Adequacy of the tools and methods used on the project

- The organization structure for the project team
- Adequacy of the office space for the project team
- The perceived value of the project to the enterprise

Another form of soft data concerns the level of the organization using the capability maturity model (CMM) or the newer capability maturity model integrated (CMMI) published by the Software Engineering Institute (SEI). The entire CMM/CMMI concept is an example of the utility and power of soft data.

Although soft data is intrinsically subjective, it is still among the most useful kinds of information that can be collected. Soft data is the major source of information that can explain the variations in productivity and quality without which a measurement program cannot lead to insights and improvements. Therefore, in a well-designed applied software measurement program, much effort and care must be devoted to selecting the appropriate sets of soft factors and then developing instruments that will allow this useful data to be collected and analyzed statistically. That is the most difficult intellectual task associated with measurement programs.

Normalized Data

The third kind of information, termed *normalized data*, refers to standard metrics used for comparative purposes to determine whether projects are above or below normal in terms of productivity or quality. This form of information was very troublesome for the software industry, and more than 40 years went by before satisfactory normalization metrics were developed.

The historical attempts to use lines of code for normalization purposes failed dismally because of the lack of international standards that clearly defined what was meant by a "line of code" in any common language and because of serious mathematical paradoxes. Indeed, lines-of-code metrics are technically impossible for studying economic productivity, which is defined as the "goods or services produced per unit of labor expense." Lines of code are neither goods nor services, and for many large software projects less than 20 percent of the total effort is devoted to coding.

The mathematical paradoxes and the reversal of apparent economic productivity associated with lines-of-code metrics totally negate the validity of such data for statistical analysis. The most troubling aspect of the paradox is the tendency for lines-of-code productivity rates to penalize high-level languages. The reason for this paradox was first described by the author in 1978 in the *IBM Systems Journal*. However, the fundamental mathematics of the paradox had been worked out

during the industrial revolution, and the basic reason for the problem has been known by economists and industrial engineers for more than 200 years!

In any manufacturing process in which fixed costs are significant, a reduction in the number of units constructed will raise the average cost per unit. For software, more than half of the project costs can go to noncoding tasks such as requirements, specifications, documentation, and testing. These noncoding tasks tend to act like fixed costs, raise the cost per source line, and lower the source lines per person-month rates for projects written in high-level languages.

Table 1-2 illustrates the paradox of an increase in real economic productivity but a decline in source lines per person-month for high-level languages. Assume that two projects are identical in functionality, but one is written in a low-level language such as Assembler and the second written in a high-level language such as Java. The noncoding tasks stay constant between the two examples and tend to act as fixed costs. The reduction in the number of source code statements in the Java example, therefore, tends to act like a reduction in manufactured units.

Note that although the Java version required only four person-months of total effort and the Assembler version required six person-months, the lines-of-code metric appears to be twice as good for the Assembler version. That violates the standard economic definition of productivity and common sense as well.

Since October 1979, when A. J. Albrecht of IBM first publicized function points, the new function-based metrics, such as function points for MIS projects and feature points for systems software, have become the preferred choice for software normalization. They have substantially replaced the older lines-of-code metric for purposes of economic and productivity analysis.

Note that function points are independent of lines of code, so the function point total for both the Assembler and the Java versions of the example in Table 1-2 would be the same: 35 function points in this case. Measures based on function points, such as cost per function point and function points per staff month, are much more reliable than the older lines of code for economic purposes.

TABLE 1-2 Example of the Mathematical Paradox Associated with Lines-of-Source-Code Metrics

	Assembler Version	Java Version
Lines of source code in project	7500	2500
Noncoding effort in person-months	3	3
Coding effort in person-months	3	1
Total project effort in person-months	6	4
Net source lines per person-month	1250	625

Functional metrics are artificial business metrics equivalent perhaps to cost per square foot in home construction or the Dow Jones stock indicator. Contractors do not build houses a square foot at a time, but nonetheless cost per square foot is a useful figure. Software professionals do not build software one function point at a time, but cost per function point is a useful economic metric.

As mentioned, both function points and feature points are totally independent of the number of source code statements, and they will stay constant regardless of the programming language used. Table 1-3 illustrates the same pair of example programs as shown in Table 1-2, only this time productivity is expressed in function points per person-month. Since both the Assembler version and the Java version are identical in functionality, assume that both versions contain 35 function points.

Observe how the function point metric agrees with both common sense and the economic definition of productivity, since the same quantity of an economic unit (35 function points) has been produced for less labor in the Java example.

Now that function points have been in use for more than 30 years, some interesting findings have occurred. Indeed, whole new kinds of studies that were previously difficult are now being carried out on a routine basis.

In the first edition in 1990, the overall U.S. average based on the cumulative results of some 4,300 software projects was about 5.0 function points per staff month. For the second edition in 1996 the current U.S. average was about 5.7 function points per staff month based on the cumulative results of roughly 6,700 software projects. For this third edition in 2008 the average productivity is about 9.9 function points per staff month based on the cumulative results of about 12,500 projects. Additional details will be provided in Chapter 3.

But the increase is deceptive. Traditional systems and information systems applications have not improved by very much. But a host of new Web applications have been under development that demonstrate rather high productivity rates. Also, the increased usage of outsource vendors have helped to improve productivity. Agile methods and Watts Humphrey's team software process (TSP) and personal software process (PSP) have also benefited productivity, although as this book is written less than 15 percent of projects use them. The bottom line is that traditional software has been stagnant, but Web applications,

TABLE 1-3 Example of the Economic Validity Associated with Function Point Metrics

	Assembler Version	Java Version
Number of function points in project	35.0	35.0
Noncoding effort in person-months	3.0	3.0
Coding effort in person-months	3.0	1.0
Total project effort in person-months	6.0	4.0
Net function points per person-month	5.83	8.75

Agile applications (which are often Web applications), and the use of disciplined methods such as TSP have triggered the boost.

(The situation is not dissimilar to the physical health of United States citizens. As of 2008 obesity is a major problem and the average health of U.S. citizens is mediocre and even in decline. However, professional athletes such as football players, swimmers, and track stars are in much better shape than athletes 10 or 15 years ago, so that many previous records are falling. In both athletics and software, the technologies exist to do very well, but not many people out of the general population utilize state-of-the-art methods.)

As the third edition was being prepared, a number of companies were visited whose productivity and quality rates in 2007 were lower than they were in 1997. A troubling number of organizations have regressed rather than improved. Surprisingly, some successful approaches from more than a decade ago were abandoned, such as use of formal design and code inspections.

One of the reasons for this retrograde change in productivity and quality is because new executives and management arrived who did not understand the economic value of formal inspections and other process activities. A hidden but important reason for the negative changes is that companies in decline did not have economic measurements sufficient to prove the value of the better methods.

Table 1-4 shows the distribution of U.S. productivity rates at three intervals: 1990 when the first edition was being written, 1995 when the second edition was being written, and 2005 when the work started on the third edition.

The primary reason for the increased productivity rates can be attributed to three factors:

- The explosion of Web-based applications, many of which use Agile methods
- The increased use of outsource vendors, some of whom displaced organizations that had low productivity
- The rigor associated with the CMM/CMMI and the TSP, which benefited the larger and more complex projects that utilized these methods.

TABLE 1-4 Distribution of U.S. Productivity Rates in Function Points

Function Points per Month	1990	1995	2005	Average
1 to 5	15%	10%	8%	11%
5 to 10	50%	40%	38%	43%
10 to 25	20%	25%	26%	24%
25 to 50	10%	15%	17%	14%
> 50	5%	10%	11%	9%
	100%	100%	100%	100%

However, the overall ranges contained in the author's data are very broad: from a low of about 0.13 function point per staff month (more than 1,000 work hours per function point) to a high of more than 200 function points per staff month (0.66 work hour per function point).

These wide variances can be explained by careful analysis of the major factors that influence software productivity:

- Overall sizes of the applications
- Variations in the activities performed
- Variations in the "soft" factors associated with software development such as tools, languages, and processes
- Variations in the types of software developed

The fourth topic, variations in software types, is perhaps the most significant change. In 1990 when the first edition was in preparation there was no World Wide Web, and hence no Web-based applications. But today in 2007, Web applications comprise more than 20 percent of all of the new software packages in the United States. Web applications are growing so rapidly that within ten years they may top 50 percent of all applications. Table 1-5 shows the changes in software types over time.

Table 1-5 shows the explosive growth of Web-based applications. It also shows a steep decline in conventional MIS applications, because many of these are now being done by outsource vendors, whose market shares have tripled since the first edition. End-user applications have almost dropped from the picture, due to ownership problems coupled with quality issues.

Table 1-6 brings up another important set of industry changes since the first edition: the average age of software applications. Table 1-6 shows the approximate average ages of software applications in calendar years among the author's clients.

Web-based applications are the newest kind of business software, so their average ages are quite young. The most alarming data in Table 1-6

TABLE 1-5 Distribution of U.S. Software Applications by Type of Application

	1990	1995	2005	Average
End-user	10%	5%	1%	5%
Web	0%	5%	20%	8%
MIS	50%	43%	29%	41%
Outsourced	5%	12%	15%	11%
Systems	15%	15%	15%	15%
Commercial	10%	10%	10%	10%
Military	10%	10%	10%	10%
	100%	100%	100%	100%

TABLE 1-6 Average Age in Years of Software Applications Still in Use

	1990	1995	2005
End-user	1.50	2.00	2.00
Web	-	1.50	5.00
MIS	10.00	15.00	20.00
Outsourced	5.00	7.00	9.00
Systems	5.50	8.00	12.00
Commercial	2.00	2.50	3.80
Military	12.00	16.00	23.00
Average	5.14	7.43	10.69

is the extreme old age of MIS applications, systems software, and especially military software. Software does not age gracefully, and tends to become increasingly unstable and difficult to modify safely. The United States and indeed much of the world are facing some huge geriatric problems with their inventories of aging and decaying applications.

Table 1-6 leads directly to Table 1-7, which shows some interesting trends in the nature of software development and maintenance since the first edition was published. Note that in Table 1-7, a form software project called *renovation* appears. The word "renovation" indicates major overhaul of aging applications, including reverse engineering, restructuring, error-prone module removal, and other geriatric services.

New development of software applications has declined for every form of software other than Web-based applications, which are now the major kind of new development circa 2007. This means that maintenance, renovation, and enhancement work are now the dominant activities of the software industry, and will probably stay that way for the indefinite future.

Later in the book the details of productivity will be addressed in terms of ranges and standard deviations, but Table 1-8 shows simple averages of what productivity levels have looked like between the time of the first edition and the third edition. The averages are nothing more than the arithmetic averages of the rows and columns; they are not weighted averages. Therefore, Table 1-8 shows two totals: the second

TABLE 1-7 Distribution of U.S. Software Projects by Nature of Work

	1990	1995	2005	Average
New	55%	35%	23%	38%
Enhancement	25%	35%	39%	33%
Renovation	5%	7%	13%	8%
Maintenance (Defect Repairs)	15%	23%	25%	21%
Total	100%	100%	100%	100%

TABLE 1-8 Approximate U.S. Productivity Ranges by Type of Application (Data Expressed in Function Points per Staff Month)

	1990	1995	2005
End-user	47.00	52.00	52.00
Web	-	12.00	23.00
MIS	7.20	7.80	9.40
Outsourced	8.00	8.40	9.70
Systems	4.00	4.20	6.80
Commercial	5.10	5.30	7.20
Military	2.00	1.80	3.75
Average	10.47	13.07	15.98
Average without end-user software	4.38	6.58	9.98

total that excludes end-user development is more relevant to the business world, where end-user applications are really not part of corporate portfolios.

Web-based applications are fairly small, and are also supported by fairly powerful development tools and programming languages. Therefore, Web productivity explains much of the improvement since the first edition. Web applications are now the home of the Agile methods, which tend to be fairly productive, especially for smaller applications.

MIS productivity has improved primarily because many of the low-productivity organizations gave up and brought in outsource vendors. A number of MIS software organizations have lower productivity in 2007 than in 1995 for the same size of application, but are now doing smaller applications. A distressing number of MIS producers (perhaps 10 percent) appear to be less sophisticated in 2007 than in 1997, due to eliminating previously successful process improvement methods such as formal inspections before testing.

Outsource productivity has improved due to a combination of hiring fairly skilled people, using fairly sophisticated development methods, providing good tool sets, and instilling a very strong work ethic, which leads to quite a few hours of unpaid overtime. However, there are still many lawsuits between clients and outsource vendors for applications that either fail completely or never work well enough to be successfully deployed. Because outsource vendors are statistically somewhat better than their clients, that does not mean that every outsource vendor will succeed on every project.

Systems software and military software productivity has improved under the combined rigor of the SEI CMM and Watts Humphrey's TSP/PSP and other technologies such as Six-Sigma. Part of the success of the systems software world can be attributed to better than average quality

assurance teams coupled with fairly rigorous defect removal activities such as formal inspections and formal testing by test specialists.

Commercial software productivity has improved because many companies, such as Microsoft, are building new versions of older applications so team experience is quite good. Also, the commercial software world has a very intense work ethic, so quite a bit of unpaid overtime contributes to their results. The commercial world also attracts very bright individuals, and offers better than average pay and benefits. That being said, the commercial world still needs to catch up with systems software in terms of quality control and measurement technologies.

The averages associated with these six categories would be unexplainable unless it were known that the average size of military software projects is almost 25 times larger than the average size of end-user software projects, and the volume of specifications and written material is more than 200 times larger in the military domain than in the end-user domain. Also, the military domain includes a large number of oversight and contractual requirements that are basically unique to the military world. The military world also includes activities, such as independent verification and validation (IV&V), that are almost never found with civilian software projects.

In other words, it is not enough just to have accurate quantitative data. It is also necessary to collect enough information about the factors that influence the data to explain *why* the results vary. Quantitative benchmark data without "soft" assessment data is insufficient to deal with the variations that occur. Conversely, process assessment data without quantification is also insufficient.

What Do Companies Measure?

Measurement is not just a software activity. A full set of corporate measurements includes financial measures, customer satisfaction measures, warrant measures, market share measures, and a host of others. In fact, the total costs of measurement in a Fortune 500 corporation can top 6 percent of sales when all forms of measurement are examined.

It is interesting to provide a context for software measurements by showing some of the many other kinds of measurements that are found in major corporations.

The best way for a company to decide what to measure is to find out what the "best in class" companies measure and do the same things. Following are the kinds of measurements used by companies that are at the top of their markets and succeeding in global competition. If possible, try to visit companies such as GE, Microsoft, IBM, AT&T, or Hewlett-Packard and find out first hand what kinds of measurements tend to occur.

Business and Corporate Measures

There are a number of important measurements at the corporate level. Some measurements are required by government regulatory agencies. But many metrics and measurements are optional and utilized to provide visibility into a variety of topics. Here are just a few samples of corporate measures to illustrate what the topics of concern are.

Sarbanes-Oxley (SOX) Measures The Sarbanes-Oxley Act was passed in 2002 and became effective in 2004. The SOX measures attempt to expand and reform corporate financial measurements to ensure absolute honesty, integrity, and accountability. Failure to comply with SOX criteria can lead to felony charges for corporate executives, with prison terms of up to 20 years and fines of up to $500,000. However, the Sarbanes-Oxley measures apply only to major public companies with revenues in excess of $75,000,000 per annum. The first implementation of SOX measures seemed to require teams of 25 to 50 executives and information technology specialists working for a year or more to establish the SOX control framework. Many financial applications required modification, and of course all new applications must be SOX-compliant. The continuing effort of administering and adhering to SOX criteria will probably amount to the equivalent of perhaps 20 personnel full time for the foreseeable future. Because of the legal requirements of SOX and the severe penalties for non-compliance, corporations need to get fully up to speed with SOX criteria. Legal advice is very important.

Balanced Scorecard Measures Dr. Robert Kaplan and Dr. David Norton of the Harvard Business School are the originators of the "balanced scorecard" approach. This approach is now found in many major corporations and some government agencies as well. The balanced scorecard approach is customized for specific companies, but includes four major measurement topics:

- The learning and growth perspective
- The business process perspective
- The customer perspective
- The financial perspective

Although the balanced scorecard approach is widespread and often successful, it is not a panacea.

Financial Measures Corporations have been measuring revenues, expenses, profits, losses, cash flow, capital expenditures, and a host of other

financial topics for many centuries. Financial measurements are usually the most complete of any form of corporate measurement. Financial measurements are also governed by a variety of regulations and policies as demanded by the Internal Revenue Service, Securities and Exchange Commission, state agencies, and other governmental bodies. Of course, the new Sarbanes-Oxley (SOX) Act introduced the most important changes in financial measures in U.S. history. SOX was enacted because historical financial measurements were not always done well. Indeed, the recent spate of bankruptcies, lawsuits, and criminal charges levied against corporate financial officers and other executives indicate how imperfect financial measures can sometimes be in large corporations. Financial measures are normally in accordance with generally accepted accounting principles (GAAP), which are set by the Financial Accounting Standards Board (FASB) in the United States and by similar organizations abroad. Non-GAAP measures are also utilized. Of course, auditors and accountants have also been implicated in some of the recent corporate financial scandals. In the U.S. the Securities and Exchange Commission (SEC) requires a very detailed financial report known as 10-K. These reports discuss the corporation's evolution, management structure, equity, subsidiaries, earnings per share, and many other topics.

Return on Investment (ROI) The measurement of return on investment or ROI has long been an important corporate measurement. The mathematics for calculating internal rates of return (IRR) and net present value (NPV) have been in use for more than a hundred years. This is not to say that ROI calculations are simple and easy. Often, long-range assumptions or even guesses must be utilized. Some companies include total cost of ownership (TCO) as part of their ROI calculations, while others prefer a limited time horizon of three to five years for a positive ROI. Because of unique situations, each company, and sometimes each division or business unit, may calculate ROI differently. Additionally, not all value is financial. When investments or potential development projects benefit corporate prestige, customer loyalty, staff morale, safety, security, health, or perhaps national defense, then financial ROI calculations are not completely adequate. ROI is one of the key factors that affect corporate innovation, because almost all companies (more than 75 percent at last count) require formal ROI studies for technology investments.

Shareholder Measures Public companies in the U.S. are required to produce reports on their financial and equity conditions for all shareholders. Some of the elements included in annual and quarterly reports to shareholders include revenues, assets, expenses, equity, any litigation in progress or coming up, and a variety of other topics. One of the items

of interest to shareholders include market capitalization, or the number of shares in existence multiplied by the current price per share. Another item of interest is total shareholder return, or the change in share price from the original purchase date (plus dividends accrued) divided by the original price. The total number of topics is too large for a short discussion, but includes economic value added (EVA), cash flow return on investment (CFROI), economic margin, price-earnings ratio (P/E ratio), and many others.

Market Share Measures The industry and global leaders know quite a lot more about their markets, market shares, and competitors than the laggards. For example, industry leaders in the commercial software domain tend to know how every one of their products is selling in every country, and how well competitive products are selling in every country. Much of this information is available from public sources. Subscriptions to various industry reports and customized studies by consulting groups also provide such data.

Competitive Measures Few companies lack competitors. The industry leaders know quite a bit of information about their competitors' products, market shares, and other important topics. Much of this kind of information is available from various industry sources such as Dun & Bradstreet, Mead Data Central, *Fortune* magazine and other journals, and from industry studies produced by organizations such as Auerbach, Meta Group, the Gartner Group, and others. For many topics, industry benchmarks performed by neutral consulting groups provide information to many companies, but conceal the specifics of each company in order to protect proprietary information.

Human Resource Measures Since good people are the cornerstone of success, industry leaders strive to hire and keep the best knowledge workers. This means that measurements of morale and attrition are important. Other personnel measures include demographic measures of the number and kinds of personnel employed, skills inventories, and training curricula.

Research and Development (R&D) Measures Some companies have substantial investments in research facilities and employ scientists and researchers as well as white- and blue-collar workers. The IBM Research Division and Bell Labs are prominent examples of major corporate research facilities. Because research and development projects are not always successful, simple measurements are not sufficient. Some of the specialized measures associated with R&D laboratories include topics such as patents, invention disclosures, publications, and the migration

of research topics into industrial products. For large companies R&D measures provide at least a glimpse of organizations that are likely to be innovative. For example, a company that files for 150 patents each year can probably be assumed to have a higher level of innovation than a similar company filing for five patents each year.

Manufacturing Measures Companies that build products from scratch have long found it valuable to measure their development cycles from end to end. Manufacturing measures include procurement of raw materials, shipping and freight, manufacturing steps, scrap and rework, inventory and warehousing, and a host of other topics. The specific kinds of measurements vary based on whether the manufacturing cycle is discrete or involves continuous flow processes. A variety of commercial tool vendors and consultants have entered the manufacturing measurement domain. Enterprise Resource Planning (ERP) packages are now available to handle many key measurements in the manufacturing sector, although such packages are not perfect by any means.

Outsource Measures Outsourcing within the United States has been common for many years. Outsourcing abroad to low-cost countries such as India, China, Pakistan, Russia, Guatemala, etc. has also become very common and appears to be accelerating rapidly. Specific measures for outsourcing vary with the nature of the work being outsourced. For example, outsourcing human resources or payroll administration is very different from outsourcing software development or manufacturing. For software outsourcing, which is the area Software Productivity Research specializes in, some of the key measurement topics include service-level agreements, quality agreements, productivity agreements, schedule agreements, and rates of progress over time. Also important are some specialized topics such as growth rates of requirements, dealing with project cancellations or termination, and methods for handling disagreements between the outsource vendor and the client.

Supply Chain Measures Measuring the performance of an entire supply chain for a major corporation is a challenging task. The full supply chain may include dozens of prime contractors and scores of subcontractors, possibly scattered around the globe. In 1996 and 1997 an organization called the Supply Chain Council was formed. Some of the founders included perhaps a dozen large companies such as Bayer, AMR Research, Procter & Gamble, Lockheed Martin, and similar companies. The council developed a set of measurement methods called the Supply Chain Operations Reference (SCOR). Some of the topics that are significant in dealing with supply chain measures include orders filled without

backordering, orders filled without errors, arrival time of materials from subcontractors, defects or returns to subcontractors, on-time delivery to customers, delivery speed, costs of materials, freight and storage costs, inventory size, and inventory turnover. There are also some specialized measures having to do with taxes paid by supply chain partners. Some of these measures overlap traditional measures used by individual companies of course. Supply chain measures are a fairly new phenomenon and are evolving rapidly.

Warranty and Quality Measures For both manufactured goods and custom built products such as software, quality and warranty costs are important topics and should be measured fully and accurately. The topic of warranty measures is fairly straightforward, and includes the total costs of customer support and product replacements under warranty guarantees, plus any additional costs attributed to good will. The topic of quality itself is large and complex. Many quality metrics center around defect levels and defect repairs. The costs of repairing defects are often handled under the topic of "cost of quality," which was made famous by Phil Crosby of ITT in his book *Quality Is Free*. It is somewhat ironic that the cost of quality topic is really about the cost of poor quality. Traditional cost of quality measures cover four topics:

- External failures
- Internal failures
- Inspection costs
- Prevention costs

there are many variations and extensions to Crosby's original themes. Some more recent approaches to quality measures are those associated with Total Quality Management (TQM), Six-Sigma Quality, Quality Function Deployment (QFD), and the quality standards in the ISO 9000-9004 set. However, these topics are too complex and extensive to discuss here.

Benchmarks and Industry Measures

Most companies are part of industries that contain similar companies. It is often of interest to have comparative data that can show how a specific company is performing against the background of many similar companies. Since some or all of the companies of interest may be competitors, special methods are needed to gather data from companies without revealing proprietary information. It is also necessary to avoid even the appearance of collusion or price fixing or sharing insider information.

The most common way for carrying out multi-company benchmarks is to use an independent consulting group that specializes in benchmarking various topics. Examples of such benchmark organizations include the Gartner Group, Meta Group, the Standish Group, Forrester Research, the International Function Point Users Group (IFPUG), and of course, the author's company, Software Productivity Research. Some of the measurements normally collected by independent consulting groups include the following.

Compensation and Benefits

Corporations cannot legally share compensation and benefit data with competitors, nor would it be prudent to do so. Compensation and benefit studies are normally "blind" studies where the names of the participants are concealed. Companies within the same industry report compensation and benefit amounts to a neutral independent consulting group, which then averages and reports on the results. Each company that contributes information receives a report showing how their compensation compares to group averages but the specific values for each company are concealed from the others in the group.

Attrition and Turnover

Every large company keeps internal statistics on attrition rates. But to find out whether a company is better or worse than others in the same industry requires a neutral independent company to collect the information and perform statistical analysis. The results are then reported back to each company that contributes data to the study, but the specifics of each company are concealed from the others.

Research and Development Spending Patterns

Companies with substantial investments in research and development are always interested in how they compare to similar companies. As with other topics involving multiple companies in the same industry, the R&D studies are performed blind with each company contributing information about R&D spending and personnel.

Software Measures and Metrics

As the name of the author's company implies, Software Productivity Research is primarily involved with measurements, assessments, estimation, and process improvements in the software domain. Of course, software development is only one of many organizations in modern enterprises. However, software is notoriously difficult. The failure rate

of software projects is alarmingly high. Even when software projects are successfully completed, poor quality, high maintenance costs, and customer dissatisfaction are distressingly common. Therefore, measurement of software projects, and especially key indicators affecting quality and risk, are vital to major corporations.

Software metrics have been troublesome over the years. The first attempt at a metric for measuring software quality and productivity was lines of code (LOC). This was appropriate in the 1960s when only a few low-level programming languages were used. By the end of the 1970s, hundreds of programming languages were in use. For some high-level programming languages such as Visual Basic, there were no effective rules for counting lines of code. As the work of coding became easier thanks to high-level programming languages, the costs and quality of handling requirements and design became more important. Lines of code metrics could not be used for measuring non-coding work.

In the mid-1970s IBM developed a general-purpose metric for software called the *function point metric*. *Function points* are the weighted totals of five external aspects of a software project: inputs, outputs, logical files, interfaces, and inquiries. Function point metrics quickly spread through the software industry. In the early 1980s a non-profit corporation of function point users was formed: the International Function Point Users Group. This association expanded rapidly and continues to grow. As of 2004 there are IFPUG affiliates in 23 countries and the IFPUG organization is the largest software measurement association in the world.

Since about 1990 function points have been replacing lines of code metrics for studies of software economics, productivity, and quality. Today in 2008 there is more data expressed in function point form than all other metrics put together. This is not to say that function point metrics are perfect. But they do have the ability to measure full software life cycles including project management, requirements, design, coding, inspections, testing, user documentation, and maintenance. With function point metrics it is possible to show complete expense patterns for large software projects from early requirements all the way through development and out into the field.

Thanks to function point metrics it is now known that for large software projects, the cost of producing paper documents is more expensive than the code itself. But outranking both paper and code, the cost of repairing defects is the most expensive single activity. It is also known that for large software projects requirements grow and change at a rate of about 2 percent per calendar month.

The software world has two major segments: systems software and information technology (IT) projects. Systems software controls physical devices and includes operating systems, telephone switching systems,

embedded software, aircraft flight controls, process control, and the like. IT software controls business functions and includes payrolls, accounting packages, ERP packages, and the like.

Many large companies such as IBM, AT&T, and Microsoft produce both kinds of software. Interestingly, the systems software groups and the IT groups tend to be in separate organizations, use different tools and programming languages, and perform different kinds of measurements.

The systems software domain is much better in measuring quality and reliability than the IT domain. The IT domain is much better in measuring productivity and schedules, and is much more likely to use function point metrics. The sociological differences between the systems software and IT domains have caused problems in putting together overall data about software in very large companies.

Every "best in class" company measures software quality. There are no exceptions. If your company does not do this it is not an industry leader and there is a good chance that your software quality levels are marginal at best. Quality is the most important topic of software measurement. The reason for this is because, historically, the costs of finding and fixing software defects have been the most expensive activity during both software development and software maintenance. Over a typical software life cycle, more than 50 percent of the total accrued costs will be associated with finding and repairing defects. Therefore, it is obvious that attempts to prevent defects from occurring, and to remove them as quickly as possible, are of major economic importance to software.

Following are some of the more important software measurements circa 2008.

Software Benchmarks Because software is such a troublesome topic, large companies are anxious to find out if other companies have their software under control. Therefore, software benchmarks for IT investments, for quality, productivity, costs, and customer satisfaction are very common. Another form of benchmark in the software domain involves demographics. As of 2008 large corporations can employ more than 70 kinds of specialized personnel in their software organizations. As with other kinds of benchmarks, software benchmarks are usually carried out by independent consulting organizations. Data is collected from a sampling of companies, analyzed, and then reported back to the participants.

Portfolio Measures Major corporations can own from 250,000 to more than 10,000,000 function points of software, apportioned across thousands of programs and dozens to hundreds of systems. Leading enterprises know the sizes of their portfolios, their growth rate, replacement cost, quality levels, and many other factors. For companies undergoing

various kinds of business process reengineering, it is important to know the quantity of software used by various corporate and business functions such as manufacturing, sales, marketing, finance, and so forth. Portfolio measures are often subset into the various categories of software that corporations build and acquire. Some of the categories include in-house systems software, in-house information systems, in-house Web applications, outsourced software of various kinds, and vendor-provided applications.

For vendor-acquired applications, reported defects, acquisition costs, and maintenance costs are the most common data points. Since most vendors don't report on the sizes of their packages in terms of LOC or function points, normalized analysis is usually impossible.

Earned Value Measurements Traditional methods for monitoring progress on various kinds of projects involved estimating costs before the projects began, and accumulating actual costs while the projects were underway. At monthly intervals, the estimated costs are compared to the actual accrued costs. Any deviations were highlighted in the form of variance reports.

The problem with this methodology is that it did not link either estimates or accrued costs to actual deliverable items or work products. In the late 1960s defense and civilian engineered projects began to utilize a somewhat more sophisticated approach termed *earned value*. Under the earned value approach, both time and cost estimates were made for specific deliverables or work products such as requirements, initial functional specifications, test plans, and the like.

As projects proceed, actual costs and schedules are recorded. But actual accrued costs are linked to the specific deliverables that are supposed to be produced under the earned value approach. This linkage between planned costs, accrued costs, and actual completion of work packages or deliverables is a better indicator of progress than the older method of simply estimating costs and accruing actual costs without any linkage to milestones or work packages.

Balanced Scorecards for Software The balanced scorecard approach was not developed for software. However, it can be applied to software organizations as it can to other operating units. Under this approach conventional financial measures are augmented by additional measures that report on: the learning and growth perspective, the business process perspective, the customer perspective, and the financial perspective.

Function point metrics are starting to be used experimentally for some aspects of the balanced scorecard approach. Some of the function point metrics associated with the balanced scorecard approach include

- Learning and Growth Perspective
  - Rate at which users can learn to use software (roughly 1 hour per 100 function points)
  - Tutorial and training material volumes (roughly 1.5 pages per 100 function points)
- Business Process Perspective
  - Application and portfolio size in function points
  - Rate of requirements creep during development
  - Volume of development defects per function point
  - Ratios of purchased function points to custom-developed function points
  - Ratios of reused function points to custom-developed function points
  - Annual rates of change of applications or portfolios in function points
  - Productivity (work hours per function point or function points per staff month)
- Customer Perspective
  - Number of function points delivered
  - Number of function points used by job title or role
  - Customer support costs per function point
  - Customer-reported defects per function point
- Financial perspective
  - Development cost per function point
  - Annual maintenance cost per function point
  - Termination or cancellation cost per function point for retired projects

The balanced scorecard approach and function point metrics are just starting to be linked together circa 2008.

Goal Question Metrics Dr. Victor Basili of the University of Maryland developed a general methodology for linking important software topics to other business issues using a technique called *goal question metrics*. This method is increasing in usage and popularity circa 2008. Important business or technical goals are stated, and then various questions are posed as to how to achieve them. The combination of goals and questions leads to sets of metrics for examining progress.

What would be an interesting adjunct to this method would be link it to the North American Industry Classification System (NAICS) developed by the Department of Commerce. Since companies within a given industry tend to have similar goals, developing common sets of metrics within the same industry would facilitate industry-wide benchmarks.

Software Outsource Measures About 70 percent of software outsource agreements are satisfactory to both sides. For about 15 percent, one side or both will not be pleased with the agreement. For about 10 percent, the agreement will be terminated within two years. For about 5 percent, disagreements may even reach the level of litigation for breach of contract or even fraud. For software outsourcing, which is the area Software Productivity Research specializes in, some of the key measurement topics include service-level agreements, quality agreements, productivity agreements, schedule agreements, and rates of progress over time. Also important are some specialized topics such as growth rates of requirements, handling new and changing requirements, handling project cancellations or termination, and methods for handling disagreements between the outsource vendor and the client if they occur.

In the outsource litigation where the author and his colleagues have been expert witnesses, the major issues that caused the litigation were outright failure of the project; significant cost and schedule overruns of more than 50 percent; delivery of software that could not be used as intended; excessive error content in delivered software; and high volumes of requirements changes.

Customer Satisfaction Leaders perform annual or semi-annual customer satisfaction surveys to find out what their clients think about their products. Leaders also have sophisticated defect reporting and customer support information available via the Web. Many leaders in the commercial software world have active user groups and forums. These groups often produce independent surveys on quality and satisfaction topics. There are also focus groups, and some large software companies even have formal "usability labs" where new versions of products are tried out by customers under controlled conditions.

Software Usage Measures A new kind of analysis is beginning to be used within the context of business process reengineering. The function point metric can be used to measure the quantity of software used by various workers within corporations. For example, project managers often use more than 10,000 function points of tools for planning, estimating, sizing, measuring, and tracking projects. Such information is starting to be available

for many other occupations including accounting, marketing, sales, various kinds of engineering, quality assurance, and several others.

Defect Quantities and Origins Industry leaders keep accurate records of the bugs or defects found in all major deliverables, and they start early during requirements or design. At least five categories of defects are measured:

- Requirements defects
- Design defects
- Code defects
- Documentation defects
- Bad fixes or secondary bugs introduced accidentally while fixing another bug

Accurate defect reporting is one of the keys to improving quality. In fact, analysis of defect data to search for root causes has led to some very innovative defect prevention and defect removal operations in many SPR client companies. Overall, careful measurement of defects and subsequent analysis of the data is one of the most cost-effective activities a company can perform.

Defect Removal Efficiency Industry leaders know the average and maximum efficiency of every major kind of review, inspection, and test and they select optimum series of removal steps for projects of various kinds and sizes. The use of pre-test reviews and inspections is normal among Baldrige winners and organizations with ultra-high quality, since testing alone is not efficient enough. Leaders remove from 95 percent to more than 99 percent of all defects prior to delivery of software to customers. Laggards seldom exceed 80 percent in terms of defect removal efficiency, and may drop below 50 percent.

Delivered Defects by Application Industry leaders begin to accumulate statistics on errors reported by users as soon as the software is delivered. Monthly reports are prepared and given to executives, which show the defect trends against all products. These reports are also summarized on an annual basis. Supplemental statistics such as defect reports by country, state, industry, client, etc. are also included.

Defect Severity Levels All of the industry leaders, without exception, use some kind of a severity scale for evaluating in-coming bugs or defects reported from the field. The number of plateaus vary from one to five.

In general, Severity 1 covers problems that cause the system to fail completely, and the severity scale then descends in seriousness.

Complexity of Software It has been known for many years that complex code is difficult to maintain and has higher than average defect rates. A variety of complexity analysis tools are commercially available that support standard complexity measures such as cyclomatic and essential complexity. It is interesting that the systems software community is much more likely to measure complexity than the information technology (IT) community.

Test Case Coverage Software testing may or may not cover every branch and pathway through applications. A variety of commercial tools are available that monitor the results of software testing, and help to identify portions of applications where testing is sparse or nonexistent. Here too the systems software domain is much more likely to measure test coverage than the IT domain.

Cost of Quality Control and Defect Repairs One significant aspect of quality measurement is to keep accurate records of the costs and resources associated with various forms of defect prevention and defect removal. For software, these measures include

- The costs of software assessments
- The costs of quality baseline studies
- The costs reviews, inspections, and testing
- The costs of warranty repairs and post-release maintenance
- The costs of quality tools
- The costs of quality education
- The costs of your software quality assurance organization
- The costs of user satisfaction surveys
- The costs of any litigation involving poor quality or customer losses attributed to poor quality

In general the principles of Crosby's cost of quality topic apply to software, but most companies extend the basic concept and track additional factors relevant to software projects.

Information Technology Infrastructure Library (ITIL) Measures Although some of the ITIL materials originated in the 1980s and even before, the ITIL books started to become popular in the 1990s. Today in 2007 the ITIL materials are probably in use by more than 30 percent of

large corporations in Europe and North America. Usage is higher in the United Kingdom and Europe because many of the ITIL books were developed in the United Kingdom. The ITIL library focuses on "service-oriented" measurements. Thus, some of the ITIL measurement topics deal with change requests, incidents, problem reports, service or "help" desks, availability, and reliability.

In spite of the usefulness of the ITIL materials, there are still some notable gaps in coverage. For example, there are no discussions of "error prone modules," whose presence in large systems is a major factor that degrades reliability and availability. Neither is there good quantitative coverage of the growth in complexity and size of legacy applications over time. Also, the very important topic of "bad fix injection" is not discussed either. Since about 7 percent of all changes and defect repairs contain new defects, this is a critical omission. The bottom line is that the ITIL materials alone are not yet a full solution for achieving optimal service levels. Additional materials on software quality, defect removal efficiency, Six-Sigma, and other topics are needed also.

Service Desk Measures Under the ITIL concept a "service desk" is the primary point of contact between a user of applications and the software engineers who maintain the applications and add new features. Thus service desks are the main point for handling both change requests and defect reports. Unfortunately, most service desks for commercial software are severely understaffed, and also may be staffed by personnel who lack both the knowledge and tools for providing effective responses. In order to minimize the wait time for actually talking to someone at a service desk, the staffing ratio should be in the order of one service specialist for about every 150 customers, adjusted for the quality levels of the software they support. Very few service desks have staffing ratios of more than about one service specialist per 1,000 customers. Even fewer make adjustments for poor or marginal quality of the software being supported. As a result, very long wait times are common.

Change Requests Under both the ITIL concept and normal maintenance concepts, change requests are submitted by authorized users to request new features for both existing software applications and also for applications currently under development. Using function points as a unit of measure, development changes average about 2 percent per calendar month from the end of requirements through the design and coding phases. Thus, if an application is nominally 1,000 function points in size at the end of the requirements phase, changes amounting to about 20 function points in size will occur every month during the subsequent design and coding phases. For legacy applications that are already in use, changes amount to about 7 percent per calendar year. Thus, for

an installed application of a nominal 1,000 function points in size, about 70 new and changed function points will occur on an annual basis for as long as the application is used. Failure to anticipate and plan for change requests is a major cause of cost overruns and schedule slips. Failure to include effective methods for sizing and costing change requests is a cause of litigation between outsource vendors and clients.

Problem Management Under both the ITIL concept and normal maintenance concepts, a "problem" is an event that either stops a software application from running, or causes the results to deviate from specified results. That being said, problems vary widely in severity levels and overall impact. The worst kinds of problems are high-severity problems that stop an application dead. An equally bad kind of problem is one that destroys the validity of outputs, so that the application cannot be used for its intended purposes. Minor problems are those that might degrade performance slightly, or small errors that do not affect the usability of the application. In addition to valid and unique problems, there are also many invalid problems reported. Invalid problems are situations that, upon investigation, turn out to be hardware issues, user errors, or in some cases reports submitted by accident. There are also many duplicate problem reports. These are common for applications with hundreds or thousands of users. Since software applications normally find only about 85 percent of known bugs before delivery to customers, there will obviously be many problem reports after deployment.

Service Level Agreements One of the key concepts of the ITIL approach, and also normal maintenance, is to define and measure service levels of software applications. These levels include reliability measures, availability measures, and performance measures. Sometimes security measures are included as well. One of the gaps of the ITIL materials is the failure to provide quantitative data on the very strong correlations between defect levels and reliability levels. Reliability is inversely proportional to the volume of delivered defects.

Security Measures In the modern world where viruses are daily occurrences, identify theft is a global hazard, and spyware is rampant, all applications that utilize important business, personal, or defense information need to have fairly elaborate security plans. These plans may include encryption of all data, firewalls, physical security, and other approaches of varying degrees of sophistication.

Application Deliverable Size Measures Industry leaders measure the sizes of the major deliverables associated with software projects. Size data is kept in two ways. One method is to record the sizes of actual

deliverables such as pages of specifications, pages of user manuals, screens, test cases, and source code. The second way is to normalize the data for comparative purposes. Here the function point metric is now the most common and the most useful. Examples of normalized data would be pages of specifications produced per function point, source code produced per function point, and test cases produced per function point. The function point metric defined by IFPUG is now the major metric used for software size data collection. The total number of projects sized with function points circa 2008 probably exceeds 60,000 in the U.S. and 100,000 on a worldwide basis.

Activity-Based Schedule Measures Some but not all leading companies measure the schedules of every activity, and how those activities overlap or are carried out in parallel. The laggards, if they measure schedules at all, simply measure the gross schedule from the rough beginning of a project to delivery, without any fine structure. Gross schedule measurements are totally inadequate for any kind of serious process improvements. One problem however is that activities vary from company to company and project to project. As of 2008 there are no standard activity definitions for software projects.

Activity-Based Cost Measures Some but not all leaders measure the effort for every activity, starting with requirements and continuing through maintenance. When measuring technical effort, leaders measure *all* activities, including technical documentation, integration, quality assurance, etc. Leaders tend to have a rather complete chart of accounts, with no serious gaps or omissions. Laggards either don't measure at all or collect only project or phase-level data, both of which are inadequate for serious economic studies. Three kinds of normalized data are typically created for development productivity studies:

- Work hours per function point by activity and in total
- Function points produced per staff month by activity and in total
- Cost per function point by activity and in total

Maintenance Productivity Measures Because maintenance and enhancement of aging software is now the dominant activity of the software world, companies also measure maintenance productivity. An interesting metric for maintenance is that of *maintenance assignment scope*. This is defined as the number of function points of software that one programmer can support during a calendar year. Other maintenance measures include numbers of customer supported per staff member, numbers of defects repaired per time period, rate of growth of applications over time.

Collecting maintenance data has led to some very innovative results in a number of companies. IBM commissioned a major study of its maintenance operations some years ago, and was able to reduce the defect repair cycle time by about 65 percent. This is another example of how accurate measurements tend to lead to innovation.

Indirect Cost Measures The leading companies measure costs of both direct and indirect activities. Some of the indirect activities, such as travel, meeting costs, moving and living, legal expenses, and the like, are so expensive that they cannot be overlooked.

Gross Productivity Measures In addition to measuring the productivity of specific projects, it is also interesting to measure gross productivity. This metric is simple to calculate. The entire work force from the chief information officer down through secretaries is included. The total effort expended by the entire set of personnel is divided into the total number of function points developed in the course of a year. The reason that gross productivity is of interest is because it includes overhead and indirect effort and thus provides a good picture of overall economic productivity. However, compared to net project productivity rates, the gross figure will be much smaller. If a company averages ten function points per staff month for individual projects, it is unlikely that they would top two function points per staff month in terms of gross productivity. This is because of all of the work performed by executives, managers, secretaries, and administrative personnel.

Rates of Requirements Change The advent of function point metrics has allowed direct measurement of the rate at which software requirements change. The observed rate of change in the United States is about 2 percent per calendar month. The rate of change is derived from two measurement points:

- The function point total of an application when the requirements are first defined
- The function point total when the software is delivered to actual customers

By knowing the size of the initial requirement, the size at delivery, and the number of calendar months between the two values, it is possible to calculate monthly growth rates.

Measurement of the rate at which requirements grow and change can also reveal the effectiveness of various methods that might slow down change. For example, it has now been proven that joint application design (JAD) and prototypes can reduce the rate of requirements

change down to below 10 percent in total. Here too collecting data and analyzing it is a source of practical innovations.

Software Assessment Measures Even accurate quality and productivity data is of no value unless it can be explained *why* some projects are visibly better or worse than others. The domain of the influential factors that affect the outcomes of software projects is normally collected by means of software assessments, such as those performed by the Software Engineering Institute (SEI), SPR, R.A. Pressman Associates, Howard Rubin Associates, Quantitative Software Management (QSM), Real Decisions, or Nolan & Norton. In general, software process assessments cover the following topics.

Capability Maturity Model (CMM) Level In 1985 the non-profit SEI was chartered by DARPA to find ways of making significant improvements in the development of military software engineering. One of the first efforts by SEI was the creation of a schema for evaluating software development expertise or "maturity." This method of evaluation was published under the name of *capability maturity model*, or *CMM*. The CMM approach assigns organizations to one of five different levels of software development maturity:

1. Initial
2. Repeatable
3. Defined
4. Managed
5. Optimizing

The specifics of each level are too complex for this report. But solid empirical evidence has been collected that demonstrates organizations at levels 3, 4, and 5 are in fact much more successful at building large and complex software projects than organizations at levels 1 and 2. Ascertaining the CMM level of a company is a task normally carried out by external consulting groups, some of which are licensed by the SEI to perform such assessments. It is significant that levels 3, 4, and 5 have very good and complete measurements of software quality and productivity. Indeed, one of the criteria for even reaching level 3 is the presence of a good measurement system. Because the CMM originated for military and defense software, it is still much more widely used in the defense sector than it is for pure civilian organizations. Comparatively few information technology groups have been assessed or have CMM levels assigned. The systems software community, on the other hand, has also adopted the CMM approach fairly widely.

Software Processes This topic deals with the entire suite of activities that are performed from early requirements through deployment. How the project is designed, what quality assurance steps are used, and how configuration control is managed are some of the topics included. This information is recorded in order to guide future process improvement activities. If historical development methods are not recorded, there is no statistical way for separating ineffective methods from effective ones.

Software Tool Suites There are more than 2,500 software development tools on the commercial market, and at least the same number of proprietary tools, which companies have built for their own use. It is of considerable importance to explore the usefulness of the available tools and that means that each project must record the tools utilized. Thoughtful companies identify gaps and missing features, and use this kind of data for planning improvements.

Software Infrastructure The number, size, and kinds of departments within large organizations are an important topic, as are the ways of communication across organizational boundaries. Whether a project uses matrix or hierarchical management, and whether or not a project involves multiple cities or countries exert a significant impact on results.

Software Team Skills and Experience Large corporations can have more than 70 different occupation groups within their software domains. Some of these specialists include quality assurance, technical writing, testing, integration and configuration control, network specialists, and many more. Since large software projects do better with specialists than with generalists, it is important to record the occupation groups used.

Staff and Management Training Software personnel, like medical doctors and attorneys, need continuing education to stay current. Leading companies tend to provide from 10 to 15 days of education per year, for both technical staff members and for software management. Assessments explore the topic. Normally, training takes place between assignments and is not a factor on specific projects, unless activities such as formal inspections or joint application design are being used for the first time. It is an interesting fact that companies providing about 10 days or more of training each year to their technical staff members have higher productivity rates than similar companies that provide little or no training. Thus, measurements have demonstrated a positive return on investment in staff education.

Environment and Ergonomics The physical office layout and noise levels exert a surprisingly strong influence on software results. The best in class

organizations typically have fairly good office layouts, while laggards tend to use crowded cubicles or open offices that are densely packed. There may also be telecommuters or remote personnel involved, and there may be subcontractors at other locations involved. Several studies have been made of office organizations, and the conclusion for software projects is that private offices lead to higher productivity and quality levels than shared offices, although this finding may be counter intuitive.

Assignment Scopes and Production Rates Two secondary measures that are extremely useful are those termed "assignment scope" and "production rate." An *assignment scope* is the amount of some deliverable for which one person will normally be held fully responsible. For new development projects, programmers are normally assigned between 50 and 100 function points as typical workloads. This is equivalent to between about 5,000 and 15,000 code statements, based on programming languages and experience of the developer.

For the purposes of maintaining existing software (fixing bugs and making small changes), an ordinary programmer will normally be responsible for perhaps 300 to 500 function points if the application is in a normal low-level language such as Cobol. This is equivalent to between 30,000 and 75,000 code statements. Some "top gun" maintenance teams have assignment scopes that top 1,500 function points or 150,000 code statements.

The maintenance assignment scope is a particularly useful statistic, because most large companies have production libraries that total from 1,500,000 to more than 10,000,000 million function points. The average maintenance assignment scope is a key factor for predicting future maintenance staffing requirements.

Once assignment scope data is collected, the assignment scope is a key metric in determining how many technical staff members will be needed for both development and maintenance projects. Dividing the total size of a new project by the average assignment scope derived from similar historical projects will generate a useful estimate of the average staff size to be required.

The *production rate* is the amount of some deliverable that one person can produce in a standard time period such as a work-month. The previously mentioned U.S. average of about ten function points per staff month is an example of a production rate. The production rate is a key metric in determining how much effort will be needed in terms of person-months, since the total size of the project divided by the average production rate will generate the amount of effort needed. Thus, a project of 50 function points in size divided by an average rate of ten function points per person-month should require a total of five person-months to be completed.

Once the staffing and person-month of effort values have been created for a project, the approximate schedule can quickly be determined by simply dividing the effort by the staff. For example, a 24-person-month project to be completed by a staff of four people should take about six calendar months. Once their logic has become assimilated, assignment scopes and production rates lead to very useful quick-estimating capabilities.

Strategic and Tactical Software Measurement

In military science, strategy is concerned with the selection of overall goals or military objectives, whereas tactics are concerned with the deployment and movement of troops toward those goals. There is a similar dichotomy within corporations that affects the measurement function. A corporate strategy will concern the overall business plan, target markets, competitive situations, and direction of the company. Corporate tactics will concern the specific steps and movement taken to implement the strategy.

For the purposes of measurement, strategic measurements are normally those that involve the entire corporation and the factors that may influence corporate success. Tactical measurements are those that concern specific projects, and the factors that can influence the outcomes of the projects (see Table 1-9).

TABLE 1-9 Strategic and Tactical Software Measures

Kind of Data	Strategic Measures	Tactical Measures
Hard	Total staff size	Staffing by activity or task
	Occupation groups	Effort by activity or task
	Portfolio size	Costs by activity or task
	User support	Project deliverables
	Market share studies	Defect rates and severities
	Profitability studies	Function or feature points
	Cancellation factors	Staff assignment scopes
	Annual software costs	Staff production rates
	Annual hardware costs	Project risk analysis
	Annual personnel costs	Project value analysis
Soft	Morale surveys	User satisfaction
	Incentive plans	Effectiveness of tools
	Annual education	Usefulness of methods
	Corporate culture	Appropriate staff skills
	Executive goals	Environment adequacy
	Competitive analysis	Project constraints
Normalized	Total function points	Project size
	Annual function points	Productivity rate(s)
	Function points per user	Defect rate(s)
	Function points used	Cost rate(s)

A full applied software measurement program will include both strategic and tactical measurements. Some of the more common forms of strategic measurement include an annual survey of total data processing expenses vs. competitive companies; an annual survey of staff demographics and occupation groups vs. competitive companies; and an annual survey of the size, mix, quality, current status, and backlog associated with the corporate portfolio.

For productivity itself, sometimes the differences between the strategic concept of productivity and the tactical concept can be surprising. For example, when companies start to think in terms of productivity measurement, most of them begin tactically by measuring the efforts of the direct staff on a set of successfully completed projects. That might generate a number such as an average productivity rate of perhaps 10 to 15 function points per person-month for the projects included in the tactical study. Tactical project measures are a reasonable way to measure successfully completed projects, but what about projects that are canceled or not successfully completed? What about indirect staff such as executives, administrators, and secretarial people whose salaries are paid by the software budget but who are not direct participants in tactical project work?

A strategic or corporate productivity measurement would proceed as follows: The entire quantity of function points delivered to users in a calendar year by the software organization would be counted. The total software staff effort from the senior software vice president down through secretarial support would be enumerated, including the efforts of staff on canceled projects and the staff on incomplete projects that are still being built. Even user effort would be counted if the users participated actively during requirements and project development. If there were any canceled projects, the effort would be collected even though the projects were not delivered.

The strategic or corporate productivity metric would be calculated by dividing the total quantity of delivered function points in a calendar year by the total number of person-months of effort expended by the whole organization. This, of course, will generate a much lower number than the previous tactical rate of 10 to 15 function points per person-month, and a normal strategic corporate rate might be only from one to three function points per person-month.

Both strategic and tactical measurements are important, but they tend to give different insights. The strategic measures tend to be very important at the CEO and senior executive levels; the tactical measures tend to be very important at the group, unit, and project levels.

Plainly, a corporation must pay the salaries and expenses of its entire software organization from the vice presidents downward. It must also

pay for canceled projects and for projects that are under development but are not yet complete. The strategic form of productivity measurement tends to be a very useful indicator of overall corporate efficiency in the software domain.

Suppose a company successfully completed 50 projects in 2007, all of which were 1,000 function points in size, and completed at a production rate of 20 function points per staff month. The total size would be 50,000 function points and the total effort would be 2,500 months. With an average assignment scope of 250 function points, each project would have a staff of four people—200 people in all. At a cost of $7,500 per staff month, these successful projects would have cost $18,750,000. The cost per function point would have been only $375.00, which is quite low.

However, for every software developer the software organization probably employed one additional person in administrative, finance, and management work. Thus, at a strategic level, an organization of 400 people delivered 50,000 function points. This doubles the cost per function point and halves the productivity rate.

Now suppose that the same company had a major failure in 2007 and canceled a massive 50,000 function point project. Suppose this one project by itself had a staff of 200 people. When this canceled project is included, the annual expense for 2007 has ballooned to $56,250,000.

When the successful 50,000 function points are divided into the total software cost of $56,250,000 the cost per function point soars to $1,125. Productivity has diminished from 20 function points per staff month to only 6.9 function points per staff month.

This small example illustrates that while having a high productivity rate for successful projects is good, it is not the whole story. Administrative and management costs plus the expenses for canceled projects and failures must also be evaluated and measured.

It cannot be overemphasized that both the strategic and tactical forms of measurement are useful, but each serves its own purpose. The strategic form is of great interest to senior executives, who must pay for all software staff and expenses. The tactical form is of great interest to project and divisional managers.

Prior to the advent of function points, it was not technically possible to carry out large-scale strategic measurement studies at the corporate, industry, or national level. Now, however, the functional metrics have been widely deployed, and it is possible to make at least the first steps in exploring productivity differences by company, by industry, and by nation.

Current Measurement Experiences by Industry

The leading high-technology companies that produce both equipment and software, such as IBM, Hewlett-Packard, Lockheed, Raytheon, and

Boeing, tend to measure both software productivity and quality and to use the data to make planned improvements. They are also very good at measuring user satisfaction and they are comparatively good at project estimating. The trailing companies within this segment produce only partial quality measurements and little or nothing in the way of productivity measurement. Sociologically, quality tends to receive greater emphasis in high-technology companies than elsewhere because the success of such companies' products demands high quality.

There seems to be a fairly good correlation between high technology and measurement, and the companies that have active research and development programs under way in technical areas, such as DuPont, General Electric, and Motorola, are often innovative in software measurements too.

The telecommunications manufacturing business, on the whole, has been one of the leaders in software measurement technology. Companies such as AT&T, GTE, Lucent, Motorola, and Northern Telecom have long known that much of their business success depends on quality, and they have therefore been pioneers in quality and reliability measures. Most have also started exploring productivity measures, although they tend to lag somewhat in adopting function-based metrics because of the preponderance of systems software.

The telecommunication operating companies such as Bell South and Pacific-Bell, on the other hand, have tended to be quite sophisticated with productivity measurements and were early adopters of function points, perhaps because, with thousands of software staff members, productivity is a pressing concern.

When airlines and automotive manufacturers were extremely profitable, neither software productivity nor measurement tended to be emphasized. In the wake of deregulation and reduced earnings for airlines, and in the wake of enormous overseas competition in the automotive segment, both kinds of manufacturers are now attempting to make up for lost time by starting full quality and productivity measurement programs. Airlines such as Delta, American, Quantas, and British Air are taking active steps to enter the software measurement arena, as are automotive manufacturers such as Ford.

Some years ago, energy and oil production companies, also in the wake of reduced earnings, started to move quickly into the domain of productivity measurement. Even now with profits at record levels, the oil industry still measures quality and user satisfaction as well. Companies such as Exxon and Amoco were early students of software productivity measurement, and they have been moving into quality and user satisfaction as well.

When the first edition of this book was published in 1991, the pure software houses such as Microsoft and Computer Associates were lagging other industry segments in terms of software measurement technology.

However, as the overall size of commercial software began to approach and then exceed the overall size of mainframe software, the commercial software world moved rapidly toward full measurements of both software quality and software productivity. With large systems now being developed for personal computers, the commercial software world has as many problems with schedules as any other industry segment, so topics such as cost estimating are now increasingly important in the commercial software domain.

One of the most impressive advances in software measurement technology is that of the contract and outsource domain. When the first edition of this book was published in 1991, measurements were used for internal purposes within major outsource vendors such as Andersen, Electronic Data Systems (EDS), Keane, and IBM's ISSC division. However, now that global competition is heating up, most of the major outsourcers are making rapid strides in software measurement technology. Thus Accenture, Computer Aid Inc., and other outsourcers are moving quickly into the measurement domains. This is true internationally as well as domestically, and offshore outsourcers such as TATA in India are also advancing rapidly into full software productivity and quality measurements. Indeed, the use of function point measurements is now a global competitive weapon among international outsource vendors. Within the United States, the outsource community is now one of the most sophisticated of any segment in terms of software measurement technology.

Management consulting companies such as Software Productivity Research, Accenture, Lockheed, Gartner, Boston Consulting, and Quantitative Software Management have often been more effective than universities both in using metrics and in transferring the technologies of measurement throughout their client base.

The defense industry has long been active in measurement, but in part because of government requirements, often attempts to both measure and estimate productivity by using the obsolete lines of code metric with unreliable results. Even the measurement initiatives in defense research establishments such as the Software Engineering Institute tend to lag behind the civilian sectors. SEI was slow in adopting any of the economically sound function-based metrics, although the attempts of Watts Humphrey of SEI to measure the stages of software maturity are attracted much attention. The defense segment is also spotty and incomplete in terms of quality measurement. That is unfortunate, given the size and resources of the defense community. The defense community is among the world leaders in terms of estimating automation, however, and most large defense contractors have professional estimating staffs supported by fully automated software-estimating packages.

That does not, of course, mean that the defense industry has an excellent record of estimating accuracy, but it does mean that estimating is taken seriously.

The leading insurance companies, such as Aetna, Blue Cross, Hartford Insurance, UNUM, USF&G, John Hancock, and Sun Life Insurance, tend to measure productivity, and they are now stepping up to quality and user satisfaction measures as well. The trailing companies within the insurance segment seem to measure little or nothing. There is a general correlation with size, in that the larger insurance companies with several thousand software professionals are more likely to use measures than the smaller companies. Insurance is a very interesting industry because it was one of the first to adopt computers and one of the few in which there have been significant correlations between effective measurements and overall corporate profitability.

Banking and financial companies for many years tended to lag in terms of measurement, although there were some exceptions. In the wake of increased competition and dramatic changes in banking technology, the financial institutions as a class are attempting to make up for lost time, and many are mounting large studies in attempts to introduce productivity, quality, and user satisfaction measures as quickly as possible. Interestingly, Canadian banks such as CIBC and the Bank of Montreal may be ahead of U.S. banks such as the Bank of America in software measurement.

In the manufacturing, energy, and wholesale-retail segments, the use of software productivity measurement appears to be proportional to the size of the enterprise: The larger companies with more than 1,000 software professionals, such as Sears Roebuck and J.C. Penney, measure productivity, but the smaller ones do not. Quality and user satisfaction measurement are just beginning to heat up within these industry segments.

Such public utilities as electric, water, and some telephone operating companies have started to become serious students of measurement in the wake of deregulation, and they are taking productivity measurement quite seriously. Such companies as Consolidated Edison, Florida Power and Light, and Cincinnati Gas and Electric are becoming fairly advanced in those measurements. Here too, however, quality and user satisfaction measures have tended to lag behind.

In the publishing business, the larger newspapers such as *The New York Times* have tended to be fairly active in both estimating and measurement, as have publishers of specialized documents such as telephone directories. Book publishers, on the other hand, have tended to be very late adopters of either measurement or estimation. It is surprising

that some of the leading publishers of software engineering and measurement books are not in fact particularly innovative in terms of their own software methods and metrics!

Federal, state, and local government agencies have not as a rule spent much energy on measuring either software productivity or quality. That is perhaps due to the fact that they are not in a competitive environment. There are some interesting exceptions at the state level, where such government agencies as Human Resources in Florida are starting to measure and estimate well, but by and large government tends to lag behind the private sector in these concepts. At the national or federal level, it is interesting that the internal revenue services in both the United States and Australia tend to be fairly active in both software measurement and software estimating technologies.

Academic institutions and universities are distressingly far behind the state of the art in both intellectual understanding of modern software measurements and the actual usage of such measurements in building their own software. The first college textbook on function points, Dreger's text on *Function Point Analysis*, was not published until 1989, a full ten years after the metric was placed in the public domain by IBM. The number of major U.S. universities and business schools that teach software measurement and estimation concepts appears to be minuscule, and for the few that do the course materials appear to be many years out of date. The same lag can be observed in England and Europe. Interestingly, both New Zealand and Australia may be ahead of the United States in teaching software measurement concepts at the university level.

Measurement and the Software Life Cycle

An effective project management measurement system adds value to all of the major activities of software development life cycles. Table 1-10 illustrates the major kinds of measurements associated with each development activity. Note that Table 1-10 shows a full set of 25 activities. In real life, not every project uses all 25. Some, such as Agile projects, use only a subset.

Note also that due to overlap between activities, the Schedule Months column is not the arithmetic sum of the activity durations. Since design starts before requirements are complete, and since other activities also overlap their predecessors, the effective total schedule is only 40 to 50 percent of the arithmetic sum.

Table 1-10 illustrates one of the main virtues of function point metrics: they can be used to measure every single activity in a software life cycle, without exceptions.

For normal manual counting, during the requirements phase, function or feature points are enumerated and then the first formal cost estimate is normally prepared. For new projects, the requirements phase is the normal point at which the project cost tracking system should be initialized. It is also appropriate to initialize the project defect and quality tracking system, since requirements problems are often a major source of both expense and later troubles.

As the requirements are refined, the next aspect of measurement deals with whether the project should be in the form of a custom application or in the form of a package that is acquired and modified. The risks and value of both approaches are considered.

If the decision is to construct the project, then a second estimate should be prepared in conjunction with the logical design of the application. Since from this point on defect removal can be the most expensive element, it is imperative to utilize reviews and inspection and record defect data. A second and more rigorous cost estimate should be prepared at this time. During physical design, reviews and inspections are also valuable and defect data will continue to accumulate.

The coding or construction phase of an application can be either troublesome or almost effortless depending upon the rigor of the preceding tasks. A third formal cost estimate should be prepared; it will be very rigorous in accumulating costs to date and very accurate in estimating costs to the completion of the project. Defect and quality data recording should also be kept during code reviews or inspections. Complexity measures of the code itself can now be performed as well.

The testing phase of an application can range from a simple unit test by an individual programmer to a full multistage formal test suite that includes function test, integration test, stress test, regression test, independent test, field test, system test, and final acceptance test. Both the defect data and the cost data from the testing phase should be measured in detail and then analyzed for use in subsequent defect prevention activities.

During the maintenance and enhancement phase, both user satisfaction measures and defect measures should be carried out. It is at this point that it becomes possible to carry out retrospective analyses of the defect removal efficiencies of each specific review, inspection, and test and of the cumulative efficiency of the overall series of defect removal steps. A useful figure of merit is to strive for 95 percent cumulative defect removal efficiency. That is, when defects found by the users and defects found by the development team are summed after the first year of usage, the development team should have found 95 percent of all defects.

TABLE 1-10 Example of Activity-Based Costs for Software Development

Average monthly salary =	$5,000
Burden rate =	50%
Fully burdened monthly rate =	$7,500
Work hours per calendar month =	132
Application size in FP =	1,500
Application type =	Systems
CMM level =	1
Programming lang. =	C
Function point type =	IFPUG 4.2

Activities	Staff Funct. Pt. Assignment Scope	Monthly Funct. Pt. Production Rate	Work Hours per Funct. Pt.	Burdened Cost per Funct. Pt.	Schedule Months	Staff	Effort Months	Effort Hours	Effort Percent
01 Requirements	500	200	0.66	$37.50	2.50	3.00	7.50	990	3.00%
02 Prototyping	500	150	0.88	$50.00	3.33	3.00	10.00	1,320	4.00%
03 Architecture	1,000	300	0.44	$25.00	3.33	1.50	5.00	660	2.00%
04 Project plans	1,000	500	0.26	$15.00	2.00	1.50	3.00	396	1.20%
05 Initial design	250	175	0.75	$42.86	2.86	3.00	8.57	1,131	3.43%
06 Detail design	250	150	0.88	$50.00	1.67	6.00	10.00	1,320	4.00%
07 Design reviews	200	225	0.59	$33.33	0.89	7.50	6.67	880	2.67%
08 Coding	150	25	5.28	$300.00	6.00	10.00	60.00	7,920	24.00%
09 Reuse acquisition	500	1,000	0.13	$7.50	0.50	3.00	1.50	198	0.60%

10 Package purchase	2,000	2,000	0.07	$3.75	1.00	0.75	0.75	99	0.30%
11 Code inspections	150	75	1.76	$100.00	2.00	10.00	20.00	2,640	8.00%
12 Ind. verif. & valid.	1,000	250	0.53	$30.00	4.00	1.50	6.00	792	2.40%
13 Configuration mgt.	1,500	1,750	0.08	$4.29	0.86	1.00	0.86	113	0.34%
14 Integration	750	350	0.38	$21.43	2.14	2.00	4.29	566	1.71%
15 User documentation	1,000	75	1.76	$100.00	13.33	1.50	20.00	2,640	8.00%
16 Unit testing	200	150	0.88	$50.00	1.33	7.50	10.00	1,320	4.00%
17 Function testing	250	150	0.88	$50.00	1.67	6.00	10.00	1,320	4.00%
18 Integration testing	250	175	0.75	$42.86	1.43	6.00	8.57	1,131	3.43%
19 System testing	250	200	0.66	$37.50	1.25	6.00	7.50	990	3.00%
20 Field (beta) testing	1,000	250	0.53	$30.00	4.00	1.50	6.00	792	2.40%
21 Acceptance testing	1,000	350	0.38	$21.43	2.86	1.50	4.29	566	1.71%
22 Independent testing	750	200	0.66	$37.50	3.75	2.00	7.50	990	3.00%
23 Quality assurance	1,500	250	0.53	$30.00	6.00	1.00	6.00	792	2.40%
24 Installation/training	1,500	250	0.53	$30.00	6.00	1.00	6.00	792	2.40%
25 Project management	1,000	75	1.76	$100.00	13.33	1.50	20.00	2,640	8.00%
Cumulative Results	211	6.00	22.00	$1,249.94	35.21	7.10	249.99	32,998	100.00%

The Structure of a Full Applied Software Measurement System

A full applied software measurement system for an entire corporation or government agency is a multifaceted undertaking that will include both quality and productivity measures and produce both monthly and annual reports. Figure 1-1 shows the overall schematic of a full enterprise software measurement system. Let us consider the essential components of a full measurement system for software.

Quality Measures

Starting at the left of Figure 1-1, there are two major components of a quality measurement program: user satisfaction measures and defect measures. User satisfaction is normally assessed once a year by means of interviews; actual users are asked to give their opinions of operational applications. User satisfaction is by definition a soft measure, since opinions are the entire basis of the results.

The second quality factor or defect counts are continuously recorded during project life cycles starting as early as requirements reviews

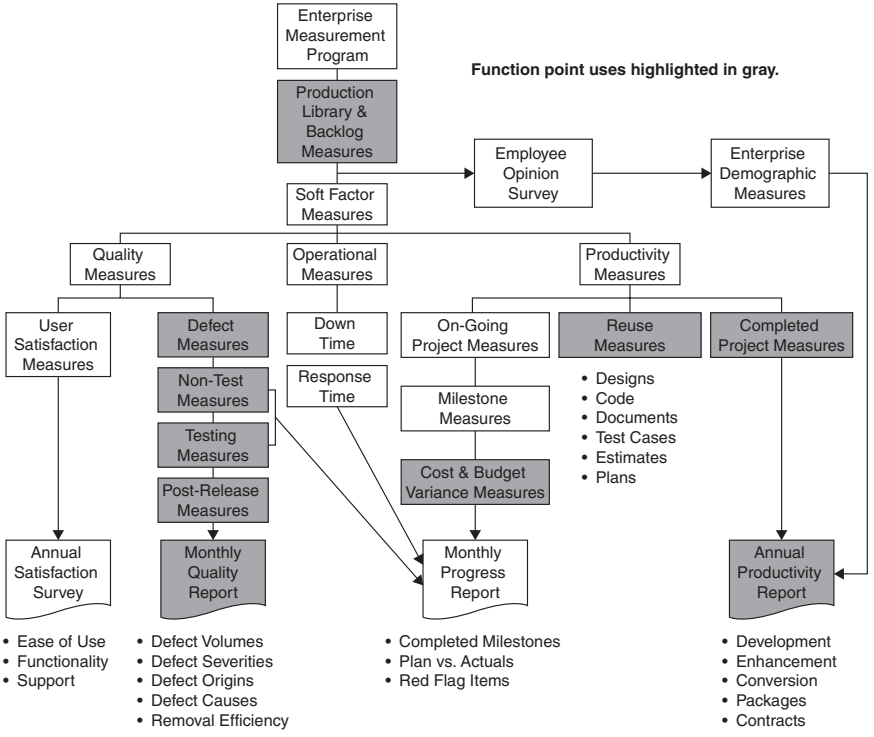

Figure 1-1 Full Applied Software Measurement System

and continuing through maintenance. Defect measures are normally reported on a monthly basis. In a well-planned measurement program, defect counts are one of the key hard-data measures. In trailing-edge companies, either quality in terms of defects is not measured at all or the measurements start so late that the data is woefully incomplete.

One of the most useful by-products of defect measurement is termed "defect removal efficiency." This is defined as the ratio of bugs found prior to installation of a software application to the total number of bugs in the application. Leading-edge enterprises are able to find in excess of 95 percent of all bugs prior to installation, whereas trailing-edge enterprises seldom exceed 70 percent in defect removal efficiency. There appears to be a strong correlation between high defect removal efficiency and such other factors as user satisfaction and overall project costs and schedules, so this is a very important metric indeed.

The measurement of defect removal efficiency is a sure sign that a company is at the leading edge, and only a handful of major corporations such as IBM and AT&T have carried out these powerful measures. Such companies are aware that most forms of testing are less than 30 percent efficient or remove less than one bug of every three, so they have long been augmenting testing with full reviews and inspections. Here too, accurate quantitative data gives the managers and staff of leading-edge companies insights that their peers in the trailing-edge companies do not even know exist!

Productivity Measures

Moving to the right in Figure 1-1, there are two major components of a productivity measurement program: ongoing projects and completed projects. Ongoing projects are normally measured, on a monthly basis, in terms of milestones successfully completed or missed and planned vs. actual expenditures. It is also possible to measure accumulated effort and cost in some normalized form such as "work hours expended to date per function point" or "dollars expended to date per function point." The monthly project reports normally contain a mixture of soft subjective information such as problem statements and hard data such as dollars expended during the month.

Completed projects are normally measured once a year. Typically, in the first quarter of a calendar year all projects that were completed and installed in the previous year will be measured and analyzed. This annual measurement of completed projects provides an ever-growing database of historical data and becomes the heart of the enterprise measurement program. Once an enterprise completes its first annual productivity report, it can use that as the baseline for judging improvements over time.

In producing an annual productivity report, all of the relevant soft factors need to be included, and the hard data for the project in terms of deliverables, schedules, staffing, and so forth should be extremely accurate. It is also desirable to convert the hard data into normalized form, such as cost per function point, for comparative purposes.

The annual productivity report will contain both strategic and tactical data, just as a corporate annual report will contain both. Indeed, companies such as IBM, ITT, and AT&T tend to create their annual software productivity reports on the same schedules as the corporate annual reports to stockholders and even to adopt similar formats and production techniques such as the use of high-quality paper, professional graphics, and excellent layouts.

Because new development projects, enhancement projects, maintenance projects (defect repairs), package acquisition projects, and projects involving contract personnel tend to have widely different productivity profiles, it is desirable to segregate the annual data very carefully. This is the area where normalization is most important and where function-based metrics are starting to provide new and sometimes surprising insights into productivity and quality topics.

Production Library and Backlog Measures

Large corporations tend to own many thousands of small programs and some hundreds of large systems. IBM, AT&T, Lockheed, and the like own some 650 million lines of source code, which is equivalent to about 500,000,000 function points. This software can be spread over more than 50 subsidiary locations. There may be as many as 50,000 small programs and 25,00 large systems. The replacement cost for such a production library would be in excess of $12 billion.

There also may be backlogs of potential applications awaiting development. These might consist of more than 60 large new systems and more than 3,500 small new programs. In addition, more than 150 large systems and about 5,000 small programs might be awaiting updates and enhancements. Such backlogs can top about 50 million source code statements or 1,000,000 function points. Such a backlog might take about four calendar years to implement and about 12,500 labor-years. The costs for building the backlog would have approximate $1,200,000,000.

An interesting but seldom performed form of production library measurement study is that of the usage patterns of programs and systems, as described by Kendall and Lamb. From a year-long analysis of IBM's data centers, they found that less than 5 percent of the company's applications used more than 75 percent of its machine capacity.

Not only that, but standard packages such as the operating systems, sorts, and commercial databases utilized more machine capacity than all custom applications put together. An even more surprising finding was that, of the custom applications owned by IBM, more than two-thirds appeared to be dormant and were not executed at all in the course of the year.

That kind of strategic information is becoming increasingly vital as companies all over the world depend upon computing and software for their operational control and their new products as well. Production library and backlog analyses should become standard strategic measures in all medium-size to large corporations and should be performed on an annual or semiannual basis.

Soft-Factor Measures

Even accurate recording of quality and productivity data cannot answer questions about why one project is better or worse than another. To answer such questions, it is necessary to come to grips with one of the most difficult aspects of measurement: how to capture soft factors or subjective opinions in a way that can lead to useful insights. Soft-data collection is a necessary adjunct to both quality and productivity measures, and it can be stated that without effective soft-data measurement, the hard data will be almost useless.

This topic of measuring soft factors devolves into two related subtopics:

- What soft factors should be measured?
- What is the best way to collect soft data?

Every known factor that can influence software productivity and quality, of which there are more than 200, is a potential soft factor for measurement purposes. The primary soft factors are those that have the greatest known impacts, and this set of primary factors includes the skill and experience of staff, the cooperation of users during requirements and design, schedule or resource constraints, methods employed on the project, tools available, appropriate choice of programming language(s), problem complexity, code complexity, data complexity, project organization structures, and the physical environment.

Although the soft data is subjective, it must be recorded in a way that lends itself to statistical analysis. Since free-form text responses are difficult to analyze, this requirement normally leads to the creation of a multiple-choice questionnaire, so that all of the recorded information

can be analyzed by computer. Following is an example of a typical soft-factor multiple choice question to illustrate the principle:

User involvement during development?

1. User involvement is not a major factor.
2. Users are heavily involved during early stages.
3. Users are somewhat involved during early stages.
4. Users are seldom involved during early stages.
5. User involvement is not currently known.

A normal soft-factor questionnaire will contain from 10 to more than 200 questions, such as the one developed by Software Productivity Research.

Operational Measures

Operational measures are those that concentrate on the adequacy and responsiveness of the computing environment. They normally include measures of:

- Computer availability and downtime
- Response time for users and development staff
- Data storage volumes
- Data storage access
- Telecommunications traffic, if any

Operational measures have traditionally been the first form of metrification used by companies, because computer room efficiency has been studied since the 1950s. Most of the operational measures consist of hard data, but the more sophisticated companies augment simple monthly reports with personal interviews of users to collect soft data on user satisfaction with turnaround and computer room performance. Operational measures are normally considered to be tactical.

Enterprise Opinion Survey

Leading-edge companies are aware that taking good care of employees pays off in both higher productivity and lower voluntary attrition rates. A normal part of taking good care of employees is an annual opinion survey, which is normally conducted by a personnel group. This, of course, is one of the purest forms of soft data, since it deals primarily with subjective opinions. Opinion surveys are also strategic measures, since staff feelings and opinions have a wide and pervasive influence.

It is important that, once such a survey has been conducted, change should follow swiftly. Nothing is more debilitating to morale than an opinion survey followed by inaction. Some of the kinds of topics included in the opinion survey include satisfaction with salary and benefits plans, physical office environments, company polices, and overall management direction. Although by their nature opinion surveys deal with soft or subjective opinions, they differ from the project-related tactical soft-factor studies in that they concentrate on issues that are general or corporate in nature.

Enterprise Demographic Measures

Now that software is approaching 50 years of age as an occupation, the same kind of specialization is occurring for software professionals that manifested itself for other knowledge workers such as doctors, attorneys, and engineers. It is highly desirable to perform an annual census of the kinds of software specialists needed and employed by the enterprise. This is one of the most useful kinds of strategic hard data that a company can collect for long-range planning purposes.

Some of the kinds of specialists that might be included are quality assurance specialists, technical writers, database administrators, estimating specialists, maintenance specialists, systems programmers, application programmers, human factors specialists, performance specialists, testing specialists, and planning specialists.

In all human activities that have been measured accurately, specialists tend to outperform generalists. Among large corporations with more than 1,000 software professionals employed, those with generalists often lag behind those with specialists in terms of software productivity. An annual demographic survey can become a significant tool leading to improvement.

The Sociology of Software Measurement

Establishing an applied measurement program for software requires sensitivity to cultural and social issues. The normal reaction to a measurement program by both project management and staff is apprehension, and only when it is shown that the data will be used for beneficial purposes rather than punitive purposes will the apprehension subside.

The sociology of measurement implies a need for high-level corporate sponsorship of the measurement program when the program is first begun, since the normal reactions of subordinate managers whose projects will actually be measured are dismay, resistance, and apprehension. Normally, either the CEO or an executive vice president would be the overall measurement sponsor and would delegate responsibilities for specific kinds of measures to those lower down in the hierarchy.

Indeed at such companies as IBM in the 1960s, ITT in the 1970s, and Hewlett-Packard in the 1980s, it was the demand for accurate measures from the CEO level that started the corporate measurement programs in the first place. The IBM corporate software measurement program has not yet been fully described for external publication, but a description of the Hewlett-Packard software measurement system has been published by Grady and Caswell.

In a well-designed applied measurement program, staff and management apprehension or opposition is very transitory and lasts for only a month or so prior to start-up, after which the real value of accurate measures makes the system expand spontaneously. At Hewlett-Packard, for example, a small experiment in software project measurement was so useful and so successful that over a period of several years it expanded on a voluntary basis into a major international study including virtually all of Hewlett-Packard's software development laboratories. Indeed, the internal measurements have proved to be so valuable that Hewlett-Packard began to offer the same kind of software project measurement services to their customers.

What causes the transition from apprehension to enthusiasm is that a well-designed applied measurement program is not used for punitive purposes and will quickly begin to surface chronic problems in a way that leads to problem solution. For example, excessive schedule pressures, inadequate office space, and insufficient computer turnaround may have been chronic problems for years and yet been more or less invisible. But a good measurement program can spot the impact of such problems and quantify the benefits of their solution.

It is an interesting phenomenon that new commercial software measurement tools have been entering the market at almost monthly intervals during the years from 1995 through 2007. A significant percentage of such tools are from start-up companies founded by former employees of companies such as AT&T, IBM, Hewlett-Packard, Motorola, or other companies with well-developed in-house measurement programs.

The rationale for these start-ups is that measurements were so valuable when they exist that the commercial marketing of measurement tools should be profitable. The general growth and success of the measurement subindustry to date make it look as if this rationale is a valid one.

The Sociology of Data Confidentiality

In many companies, corporate politics have such prominence that project managers and some executives will be afraid to submit their data to a corporate measurement group unless the confidentiality of their data is guaranteed by the measurement group. That is, each manager

will want to find out personally how his or her data compares to the corporate or group average but will not want that data distributed to other project groups or to "rival" managers.

Although it is sometimes necessary for reasons of corporate culture to start a measurement program on a confidential basis, the approach is both sociologically and technically unsound. In a mature and well-managed enterprise, software productivity and quality measurements are normal business tools and should have about the same visibility and the same security classification as corporate financial data. A branch sales manager, for example, could hardly insist on the confidentiality of the branch's quarterly profit-and-loss data.

Group, divisional, and corporate executives should receive productivity and quality reports on all projects and units within their scope of responsibility, just as they receive profit-and-loss reports or normal financial reports. A well-designed software measurement program will not be a punitive weapon; it will identify all weaknesses that need correction and point out all strengths that need encouragement.

Another disadvantage of data confidentiality is that it tends to lower the credibility of the measures themselves. For the first year of ITT's corporate measurement program in 1980, the data was held in confidence. The consequence was that no one really cared about the results. In the second year, when the projects were explicitly identified, acceptance of the measurements as important to managers and executives increased dramatically.

The Sociology of Using Data for Staff Performance Targets

Once a company begins to collect software productivity and quality data, there is a natural tendency to want to use the data to set staff performance targets. That, of course, is one of the reasons for apprehension in the first place. Leading-edge companies such as IBM and Hewlett-Packard do set performance targets, but for sociological and business reasons the targets should be set for executives at the director and vice presidential level, rather than for the technical staff.

The major reason for that is that executives are in a much better position to introduce the changes necessary to achieve targets than are technical staff members or first-line managers. Neither the technical staff nor subordinate managers are authorized to purchase better tools and workstations, stop work and receive necessary education, or introduce new practices such as full design and code inspections. Executives, on the other hand, can do all those things.

A secondary reason for establishing executive targets is likely to become more and more important in the future: Corporate officers have a legal

and fiduciary duty to achieve professional levels of software quality, and if they do not, both their companies and themselves may find expensive lawsuits and perhaps even consequential damages in their futures!

Perhaps the single event that more than any other made IBM a leader in software quality for many years was the establishment in 1973 of numeric quality targets for software executives and the inclusion of those targets in their performance and bonus plans. Prior to that time, IBM, like many other companies, talked about achieving high quality, but when the pressure of business caused a choice between opting for high quality or skipping something like inspections to try to shorten delivery dates, quality seldom won. Once IBM's vice presidents and directors had quality goals in their performance plans, however, quality was no longer just being given lip service but became a true corporate incentive.

The Sociology of Measuring One-Person Projects

More than half of all software projects in the world are small projects that are carried out by a single programmer or programmer-analyst. This situation requires special handling, since it is obvious that all data collected on one-person projects can easily be used for appraisal purposes. The delicacy of measuring one-person projects is especially sensitive in Europe, where some countries prohibit the measurement of an individual worker's performance either because of national law, as in Sweden, or because the software staffs are unionized and such measurements may violate union agreements, as in Germany.

The normal solution to this problem in large companies such as IBM and ITT can be one or more of several alternatives: The basic alternative is to establish a cutoff point of perhaps two person-years and simply not measure any project that is smaller. This solution tends to concentrate the measurements on the larger and more costly projects, where, indeed, the value of measurement is greatest. A second solution is to collect one-person project data on a voluntary basis, since many programmers are perfectly willing to have their work measured. It is, however, tactful to ask for volunteers. A third solution, possible only in very large companies, is to aggregate all small one-person projects and then create an overall set of small-project statistics that does not drop below the division or laboratory level.

Of course, it is also possible to bite the bullet and use one-person project data for appraisal purposes, and some companies indeed do that. It is, however, very likely to lead to morale problems of a significant nature and perhaps even to lawsuits by indignant staff members who may challenge the measurements in court.

The Sociology of MIS vs. Systems Software

Many large high-technology corporations produce both management information systems (MIS) projects and also systems software, such as operating systems or telecommunication systems. Some also produce other kinds of software as well: process control, scientific software, mathematical analysis, and so on.

Generally speaking, the MIS staffs and the systems software staffs have such difficulty communicating and sharing technical ideas that they might as well inhabit different planets. The dichotomy will affect measurement programs too, especially since systems software productivity is normally somewhat lower than MIS productivity because of the larger number of tasks performed and the effect of the soft factors. The natural reaction by the systems software groups to this fact is to assert that systems software is much more complex than MIS applications. Indeed, many systems software producers have rejected function-based metrics for two reasons: Function points originated in the MIS domain, and MIS projects normally have higher productivity rates.

This kind of dispute occurs so often that companies should plan remedial action when beginning their measurement programs. There are several possible solutions, but the most pragmatic one is simply to segregate the data along clear-cut lines and compare MIS projects primarily to other MIS projects and systems software primarily to other systems software. A more recent solution is to adopt the feature point metric for systems software productivity measures, since the built-in assumptions of feature points about algorithmic complexity tend to generate higher totals for systems software than for MIS projects.

Whatever solution a company decides on, the problem of needing to be sensitive to the varying software cultures needs attention right from the start.

The Sociology of Measurement Expertise

The managers and technical staff workers who embark on a successful measurement project are often surprised to find permanent changes in their careers. There is such a shortage of good numerical information about software projects and such enormous latent demand by corporate executives that, once a measurement program is started, the key players may find themselves becoming career measurement specialists. This phenomenon has affected careers in surprising ways. From informal surveys carried out by Software Productivity Research, almost half of the measurement managers are promoted as a result of their work. About a third of the managers and technical staff workers who start corporate measurement programs work in the measurement area for

more than five years thereafter. Both A. J. Albrecht and the author began their measurement careers with short measurement projects intended to last for less than two months. In both cases, the demand for more measurement information led to long-term careers.

Justifying and Building an Applied Software Measurement Function

For most U.S. companies other than those at the very leading edge such as Microsoft, Google, IBM, Hewlett-Packard, AT&T, and a few others, a software measurement program will be a new and perhaps exotic concept. It will be necessary to justify the costs of measurement and to plan the staffing and sequence of establishing the measurement function with great care. The following are the major topics that must be considered.

The Value of Applied Software Measurement Programs

It is, of course, not possible to perform direct productivity or quality comparisons between companies that measure and companies that do not, since only the ones that measure have any data. This phenomenon creates a trap of circular reasoning within companies that do not measure: Executives tend to say "prove to me that measurements will be valuable." But since the same executives have no idea of their current levels of productivity, quality, or user satisfaction, there is no baseline against which the proofs can be made.

Project managers and executives in companies that do not measure software numerically actually have a vested interest in preventing measurements from occurring. They suspect, rightly, that their performances will not be shown in a favorable light if measurements occur, and so they tend to obstruct metrics work rather than support it.

To make an initial case for the value of measurements, it is often necessary to depend on such indirect factors as market share, user satisfaction, and profitability. It is also necessary to gain the support of executives high enough in the company, or secure enough in self-esteem, that they will not feel threatened by the advent of a measurement program.

Software measurement is a very powerful defect prevention technology, and also can assist in raising defect removal efficiency. Software measurement can minimize poor investments and optimize good investments. As a result, software quality measurements have one of the best returns on investment of any software technology, and software productivity measurements are useful also.

Following are the approximate returns on each $1 invested in ten current technologies. The investments are the results after four years

of usage of the technology. This data is extracted from the author's book *Assessment and Control of Software Risks* (Prentice Hall, 1994).

Technology	Return on Investment, $, for Each $1 After 4 Years of Usage
Full software reusability	30
Agile methods	17
Software quality measurements	17
Software estimation tools	17
Formal design inspections	15
Formal code inspections	15
Object-oriented programming	12
Software productivity measurements	10
Software process assessment	10
Functional metrics	8

Software quality measurements provide one of the highest ROIs of any technology and are far easier to get started than a full reusability program or a full object-oriented approach.

Let us consider the value of measurement to a specific major corporation that has measured for many years: IBM. It is often said that "knowledge is power," and perhaps no other company in history has had available so much knowledge of its software. One of the major reasons for IBM's long dominance of the computer industry is that its founder, Thomas Watson, Sr., was personally insistent that IBM strive for the highest levels of quality and user satisfaction possible, and he introduced quality measures very early in the corporation's history. Both Watson, his son Thomas Watson, Jr., and the other IBM chairmen since the early days have continued that trend. It is revealing to consider some of the kinds of measurement data available within IBM but not necessarily available to most of IBM's major competitors.

IBM's quality data includes full defect recording from the first requirements review through all forms of inspection and all forms of testing and then all customer-reported bugs as well. IBM probably knew the measured efficiencies of every kind of software review, inspection, and test before any other corporation in the world, and it was able to use that data to make net corporate software quality improvements of about 5 to 1 during the late 1960s and early 1970s. Although IBM put much of this data into the public domain, very few competitors bothered to replicate IBM's findings! Of course, even IBM puts out some low-quality products and makes mistakes. But in terms of the percent of all products in the market that have high quality, few companies can equal IBM's overall rates.

IBM was also the first company to discover that software quality and software productivity were directly coupled and that the projects with the

lowest defect counts by customers were those with the shortest schedules and the highest development productivity rates. This phenomenon, discovered by IBM in the early 1970s and put in the public domain in May 1975, is still not understood by many other software-producing enterprises that tend to think that quality and productivity are separate issues.

IBM's customer-reported defect database is so powerful and sophisticated that IBM can produce reports showing the origins and severities of defects for any product, for any month of any year, in any country, and in every major city. The data can also be sorted by industry as well. Software managers and senior executives in IBM receive such data monthly, and they also receive cumulative trends that show long-term progress for each product, each laboratory, each division, and the corporation as a whole.

IBM's user satisfaction surveys also revealed, long before most other companies realized this point, that user satisfaction and numeric defect rates were directly correlated. Projects with low defect counts tended to be high in user satisfaction; projects with high defect counts tended to be low in user satisfaction. This discovery also was made in the early 1970s.

IBM's employee demographic and opinion survey data can identify all technical occupation groups within the company and both the current and past morale history of every laboratory, branch office, and manufacturing location within the corporation. It is also possible for IBM executives to gain other kinds of useful information, such as the annual attrition rates of each location by occupation group within the company.

In addition to these software and demographic measurements, IBM managers and executives have an enormous quantity of economic and market data available to them, including the histories of the economic trends of all countries, demographic and population statistics of all cities in more than 100 countries, and the sales statistics of every IBM product in every geographic region of the world, every industry, and every time period for more than 15 years in the past and projected forward for more than 10 years in the future.

IBM's software productivity measures included systems, military, and MIS software projects. The multiple regression techniques used by Felix and Walston of IBM's Federal Systems Division were published in 1977 as a landmark study that showed the factors that influenced software productivity. The analysis of the mathematical errors of lines of code was first published by the author in the *IBM Systems Journal* in 1978; it proved conclusively that lines of code could never match economic productivity assumptions. Function points were invented within IBM for MIS projects by A. J. Albrecht of IBM's Data Processing Services Division and placed in the public domain in October 1979. The insights and correlations based on that metric have been used by IBM to make far-reaching improvements in software methods and tools.

Of course, even with all of this data IBM can sometimes make mistakes and can sometimes be surprised, as by the success of the original personal computer or by the decline in large mainframe sales in the middle 1980s. Nonetheless, IBM managers and executives tend to have at least ten times as much valid measurement-based data available to them as do equivalent managers and executives in most other companies, and IBM's long-term success is in large part due to that wealth of factual information.

Another well-known company that is beginning to become aware of the value of software measurement is Microsoft. Microsoft's products have grown from small applications of a few hundred function points up to more than 5,000 function points for Excel and Microsoft Word, more than 50,000 function points for Windows 95, more than 100,000 function points for Windows XP, and more than 150,000 function points for Windows Vista. Microsoft is moving very rapidly into the software measurement arena. For example, as of 2007, Microsoft has one of the best quality and defect measurement systems in the software world.

The only lagging portion of Microsoft's measurement program is the lack of widespread support for function point metrics. The reason for this, perhaps, is that Microsoft has been one of the most productive companies for small applications in the world. Therefore, productivity measurements were more or less irrelevant, since there was no serious competitor to be compared against.

Now that Microsoft is developing applications that involve hundreds of personnel and require several years of development, it may be that productivity measurements based on function points will become as important in Microsoft as elsewhere in the software world.

The Costs of Applied Software Measurement Programs

Accurate and complete measurements of software are not inexpensive; indeed, the costs can approximate the costs of a corporate cost accounting function. In the companies that have full applied measurement programs for software, the annual costs can sometimes exceed 4 to 6 percent of the total software budget, with about 2 percent being spent on measuring productivity and 2 to 3 percent spent on measuring quality and user satisfaction. By coincidence, the same breakdown often occurs in soft- and hard-data collection: about 2 percent for collecting the soft factors and about 2 to 3 percent for collecting the hard data from completed projects on an annual basis.

Very large corporations such as Google, Microsoft, IBM, Hewlett-Packard, and AT&T can have permanent corporate measurement staffs

in excess of a dozen individuals, regional or laboratory measurement staffs of half a dozen at each major site, and intermittent involvement in measurements by several hundred managers and staff members. The companies that have full software measurement programs are also the companies with the best track records of success in terms of both bringing software projects to completion and achieving high levels of user satisfaction afterward. They also tend to be industry leaders in respect to morale and employee satisfaction.

The Skills and Staffing of a Measurement Team

Most universities and academic institutions have no courses at all in the measurement of software quality, productivity, or user satisfaction, so it is seldom possible to hire entry-level personnel with anything like an adequate academic background for the work at hand. Business schools and MBA programs also are deficient in these topics, so most companies are forced to substitute on-the-job training and industry experience in software management for formal credentials.

Some of the skills available in measurement teams such as those at IBM, AT&T, DuPont, Hewlett-Packard, and ITT include a good knowledge of statistics and multivariate analysis, a thorough grounding in the literature of software engineering and software project management, a knowledge of software planning and estimating methods and the more powerful of the available tools, a knowledge of forms design, a knowledge of survey design, a knowledge of quality control methods including reviews, walk-throughs, inspections, and all standard forms of testing, a knowledge of the pros and cons of all software metrics including the new function-based metrics, and knowledge of accounting principles.

The special skills and knowledge needed to build a full measurement program are so scarce in the United States as a whole that many companies begin their measurement programs by bringing in one or more of the management consultants who specialize in such tasks. Once the consulting group assists in the start-up phase, the corporate measurement team takes over the future studies and measurements.

The Placement and Organization of the Measurement Function

Measurement of software productivity, quality, and user satisfaction works best with a dedicated staff of professionals, just as cost accounting and financial measurement works best with a dedicated staff of professionals. Leading-edge companies that recognize this fact will normally establish a corporate measurement focal point under an executive at about the level of a director or third-line manager. This focal point will often report

to someone at the level of a vice president, executive vice president, or chief information officer (CIO). The corporate measurement group will coordinate overall measurement responsibilities and will usually produce the annual productivity report. As with finance and cost accounting, the larger units and subordinate organizations within the corporation may have their own local measurement departments as well.

The raw data collected from tracking systems, in-depth studies, surveys, interviews, and other sources should be validated at the source prior to being sent forward for aggregation and statistical analysis. However, some wrong data seems to always slip by, so the corporate group must ensure that all incoming data is screened, and questionable or incorrect information must be corrected. The raw data itself can either be collected by local personnel on the scene, by traveling data collection specialists from the unit or corporate measurement function, or even by outside consultants if the enterprise is just getting started with measurement.

If the corporation has a formal quality assurance (QA) function, the defect-related data will normally be collected by QA personnel. Quality data can, of course, be reported separately by the QA staff, but it should also be consolidated as part of the overall corporate reporting system.

User satisfaction data for commercial software houses and computer companies is often collected by the sales and marketing organization, unless the company has a human factors organization. Here too, the data can be reported separately as needed, but it should be consolidated as part of the overall corporate reporting system.

If the company does not have either a sales and marketing organization or a human factors organization, it would be normal to bring in a management consulting group that specializes in user satisfaction measurements to aid during the start-up phase.

The Sequence of Creating an Applied Software Measurement Program

For sociological reasons, measurement programs are often established in a sequence rather than as an attempt to measure all factors simultaneously. As a rule of thumb, companies at the extreme leading edge such as IBM will have all nine measurements deployed. To be considered a leading-edge enterprise at all, a company will have at least five of the nine measurement classes operational. Trailing companies will usually have no more than the first two forms of measurement deployed. At the extreme rear of the trailing edge are the unfortunate companies that have no measurements at all. They tend to be short-lived organizations whose future is not likely to be happy, and they stand a good chance of failing or being acquired by better-managed competitors.

The time span for creating a full software measurement program will vary with the urgency and executive understanding of the value of measurement. IBM's measurement program tended to evolve naturally over many years, and it started with opinion surveys even before the computer era. IBM got into quality measures even before software and computers became prominent in its product line, and it was perhaps the first U.S. company to actually measure software quality. Its software quality measures were started in 1964 for commercial and systems software, and it added productivity measures in 1968. Function points were invented within IBM's DP Services Division in about 1975 and placed in the public domain in October 1979. Thus, for more than 40 years, IBM's management and executives have had useful data available to aid in improving software quality and productivity.

For a major corporation, starting an applied software measurement program and selecting the initial team will usually take about six weeks. Collecting data and producing an initial report usually takes an additional two to three months, based on the number of operating units included and their geographic separation.

The observed sequence of measurement in successful large enterprises tends to follow this pattern:

1. **Operational Measures** Historically, operational measures have been first. Most companies already record the key operational measures of computer utilization, downtime, and response time. These measures may be used for charge-backs, and they normally serve to keep tabs on the overall health of the computing complex. Operational measures have been common since the 1950s.
2. **Ongoing Project Measures** Many large companies already require monthly status reports from project managers on accomplished milestones or planned vs. actual expenditures. Informal monthly ongoing project measures are common, but they are not always very effective as early warning indicators because of a natural human tendency to conceal bad news if possible. On-going project measures have been fairly common since the 1950s in large or very large corporations.
3. **Production Library and Backlog Measures** When the CEO and senior executives of corporations begin to sense how much money is tied up in software, they naturally want to find out the true dimensions of the corporation's investment. When they ask the CIO or senior software vice president, the initial answer is likely to be "I don't know." This embarrassing exposure quickly tends to trigger a full production library and backlog study, which will often be performed by an outside management consulting group that specializes in such tasks.

4. **User Satisfaction Measures** The next measurement that companies tend to implement is that of user satisfaction. It is a basic metric for enterprises that market software and an important metric for internal information systems as well. Effective measurement of user satisfaction normally requires actual interviews with users. Forms and questionnaires alone are seldom sufficient to find out what really needs to be known, although such information is certainly helpful. User satisfaction surveys for software and computing products started in the late 1950s and 1960s.
5. **Completed Project Measures** Now that function-based metrics have become widespread, many companies have started counting the function point totals of completed projects and accumulating resource data as well. This form of measurement can be useful, but neither function points nor resource data alone can deal with the issue of why some projects succeed and others fail. Nonetheless, it is a sign of growing sophistication when a company begins to collect accurate hard data and functional metrics from completed projects. Although some companies such as IBM have been measuring since the 1960s, completed project measures have only started to become common during the 1980s as a by-product of the development of function-based metrics.
6. **Soft-Factor Measures** When a company begins to strive for leadership, it is natural to want to know everything that is right and everything that is wrong about the way it does business. At this point, such a company will start an in-depth survey of all of the soft factors that influence software projects. That is, it will perform a project-by-project survey of the methods, tools, skills, organization, and environment available for software development and maintenance. The soft factors can be used to eliminate weaknesses and augment strengths. The soft factors and the completed project data can be collected at the same time, and they can even be part of the same survey questionnaire or instrument. Soft-factor measures started to undergo serious study in the 1970s, and they matured in the 1980s.
7. **Software Defect Measures** Only the true industry leaders such as IBM, Lockheed, and Raytheon have stepped up to the task of measuring software defect rates, and this is a partial explanation of IBM's long-term success. Since the cost of finding and fixing bugs has historically been the largest software cost element, quality control is on the critical path to productivity control. Also, there is a strong observed correlation between defect levels and user satisfaction; users seldom give favorable evaluations to software products with high defect rates. Only a handful of U.S. companies have accurate measures of software defect rates and defect removal, and they tend

to dominate their industry segments. The leading-edge U.S. companies began their defect measures in the 1960s, but for many others, this will be a topic of the 1990s.

8. **Enterprise Demographic Measures** Very few companies realize how important their employees truly are to corporate success. Those that do tend to perform annual demographic surveys of exactly how many employees they have in the skill classes that are relevant to corporate goals. The data can then be used for long-range projections over time. Unfortunately, some otherwise very sophisticated companies have not been able to carry out demographic surveys because of their tendency to lump all staff members under such job titles as "member of the technical staff." Since more than 40 kinds of specialists are associated with software, it will become increasingly important to include demographic measures as part of the overall corporate measurement program in the future. The military services and some government agencies have been keeping track of job categories since prior to World War II, but for many companies this will be a task for the 1990s.
9. **Enterprise Opinion Survey** An opinion survey is last on the list not because it is least important, but because it requires the greatest amount of lead time and is the greatest change in corporate culture to begin implementation. Opinion surveys, of course, affect all employees and not just the software staffs, so it is necessary to have support and backing from the entire executive ranks. It is also necessary to have the survey instruments acquired or produced by personnel experts, or the results at best will be misleading and at worst may be harmful. Finally, it is necessary for the company to face reality and try to solve any major problems that the opinion survey uncovers. Opinion surveys are, of course, older than the computer era, and industry leaders have been using them since the 1950s. For many companies, opinion surveys must be a topic to be addressed in the 1990s.

Applied Software Measurement and Future Progress

Progress in all scientific and engineering work has been closely coupled to accurate measurements of basic phenomena. Without the ability to measure voltage, resistance, and impedance, there could be no electrical engineering. Without the ability to measure temperature, blood pressure, and blood types, medical practice could scarcely exist. Without the ability to measure barometric pressure, wind velocity, and wind direction, meteorology would be even more imperfect than it is today.

Software is at a pivotal point in its intellectual history. For the first 45 years of existence, it achieved a notorious reputation as the worst-measured engineering discipline of the 20th century. Now that accurate and stable software measures are possible, the companies that seize the opportunity to base their improvements on quantified, factual information can make tremendous progress.

The companies that wish to improve but do not measure are at the mercy of fads and chance. Progress may not be impossible, but it is certainly unlikely. Only when software engineering is placed on a base of firm metrical information can it take its place as a true engineering discipline rather than an artistic activity, as it has been for much of its history. Measurement is the key to progress, and it is now time for software to learn that basic lesson.

Suggested Readings

Arthur, Jay. *Measuring Programmer Productivity and Software Quality*. Indianapolis, IN: Wiley Press, 1985. Jay Arthur is a software engineering researcher at U.S. West. In his book he discusses the pros and cons of various measurement techniques from the standpoint of how real companies are likely to use the information.

Conte, S. D., H. E. Dunsmore, and V. Y. Shen. *Software Engineering Metrics and Models*. Menlo Park, Calif.: The Benjamin/Cummings Publishing Company, Inc., 1986. This book contains descriptions of most of the relevant metrics that can be used on software projects, together with suggestions for their applicability. It also contains discussions of statistical sampling, validation of data, and other useful information. Although aimed more at software engineering than at management information, it nonetheless covers the field in more depth than almost any other source. It is a good book for anyone getting started in metrics selection or evaluation.

Garmus, David, and David Herron. *Function Point Analysis*. Boston: Addison Wesley, 2001. David Herron and David Garmus are both long-term members of the International Function Point Users Group. Both have served as officers and committee members. This book is an excellent introduction to function point analysis co-authored by two of the top expert in the field.

Jones, Capers. *Program Quality and Programmer Productivity, Technical Report TR 02.764*. San Jose, Calif.: IBM Corporation, 1977. This report reveals as much as IBM has ever chosen to reveal about the internal measurements of large systems software projects within the company. It includes data on a number of related topics, including productivity, quality, machine utilization, and the technologies that IBM had concluded were beneficial or harmful. For competitors of IBM, it is significant to note that the report, although published in 1977, contained more than ten years worth of historical information that had already been available within the company.

———. *A History of Software Engineering in IBM from 1972 to 1977*. Burlington, Mass.: Software Productivity Research, Inc., 1989. This report is the history of a critical five-year period in IBM, during which time software evolved from a relatively low-key support function for hardware devices into a true strategic product line. In 1972, software projects had grown enormously in size and complexity, but IBM's methods of managing and measuring progress were still groping with the changes. Each year during the period, a major problem was addressed and brought under control. In every case, the availability of measured data provided a background of facts that enabled IBM's senior management to make usually sound business decisions.

———. *A 10 Year Retrospective of Software Engineering Within ITT from 1979 to 1989*. Burlington, Mass.: Software Productivity Research, Inc., 1989. This report shows the evolution of software engineering methods within a major corporation, and it illustrates

how measurement data became one of the most powerful tools for making rapid improvements in both quality and productivity of software projects. It also discusses the creation and functions of the well-known ITT Programming Technology Center, which was one of the premier R&D laboratories for software in the United States prior to ITT's sale of several divisions to Alcatel.

———. *Assessment and Control of Software Risks*. Englewood Cliffs, N.J.: Prentice Hall, 1994. This book discusses some 65 risk factors noted during SPR's software process assessments. The risks are both technical and social in nature. Technical risks include those of poor quality, long schedules, and inadequate planning and estimating. Social risks include friction between clients and software groups, and the very serious situation of friction between software executives and corporate executives. For each risk, there is a discussion of how the risk might be prevented or controlled. Other information includes discussions of the return on investment in various software technologies.

———. *Patterns of Software System Failure and Success.* Boston: Thomson International, 1996. This book is based on research into two extreme conditions at opposite ends of the spectrum: (1) projects that set new records for software quality and productivity; (2) projects that failed totally and were canceled without being completed. Both technical and social factors contributed to both conditions. Poor management practices tended to outweigh technical factors for canceled projects, with planning, estimating, and quality control approaching or exceeding malpractice thresholds. Successful projects, as might be expected, were much better at estimation, planning, and quality control. They also tended to have reduced levels of creeping requirements and larger volumes of reusable materials.

———. *Software Assessments, Benchmarks, and Best Practices.* Boston: Addison Wesley, 2000. This book contains fairly detailed discussions of productivity and quality levels achieved by systems software, information systems, embedded software, commercial software, outsourcers, and military software. It also discusses the technologies utilized to achieve good results.

———. *Estimating Software Costs*. New York: McGraw-Hill, 2007. This book is a companion to *Applied Software Measurement*, and covers similar ground only from the standpoint of estimation rather than of measurements. Measurement and estimation use many of the same metrics, only with estimation the metrics are pointed forward toward the future, while with measurement the metrics are aimed backward at history.

Kan, Stephen. *Metrics and Models in Software Quality Engineering, Second Edition*. Boston: Addison Wesley, 2003. Stephen Kan is one of IBM's top software quality measurement and metrics experts. This book contains a rich mine of useful data on both measuring quality and achieving high levels of quality. It also contains excellent discussions of quality measurement methods.

Sayward, F. G., and M. Shaw. *Software Metrics.* Cambridge, Mass.: MIT Press, 1981. Fred Sayward was one of the researchers at the well-known ITT Programming Technology Center. This book is yet another of the dozen or so written or edited by researchers of that organization. It contains a very useful discussion on the design of experiments and on ensuring that measured data is not biased by accident or poor collection techniques. It also contains useful discussions of many standard software engineering metrics.

Additional Readings on Software Measurement and Metrics

Albrecht, A. J.. "Measuring Application Development Productivity." *Proceedings of the Joint SHARE, GUIDE, and IBM Application Development Symposium, October 1979*. Reprinted in Capers Jones, *Programming Productivity—Issues for the Eighties*, IEEE Press, Catalog Number EHO239-4, 1986, pp. 35–44.

Boehm, Barry. *Software Engineering Economics.* Englewood Cliffs, N.J.: Prentice Hall, 1981.

_____. *Software Cost Estimation with COCOMO II*. Englewood Cliffs, N.J.: Prentice Hall, 2000.

Cohn, Mike. *Agile Estimating and Planning.* (Robert C. Martin Series) Englewood Cliffs, N.J.: Prentice Hall PTR, 2005.

Coombs, Paul. *IT Project Estimation: A Practical Guide to the Costing of Software*. Melbourne, Australia: Cambridge University Press, 2003.
DeMarco, Tom. *Controlling Software Projects.* New York: Yourdon Press, 1982.
———. *Deadline.* New York: Dorset House Press, 1997.
Department of the Air Force. *Guidelines for Successful Acquisition and Management of Software Intensive Systems, Vols. 1 and 2.* Software Technology Support Center, Hill Air Force Base, Utah, 1994.
Dreger, Brian. *Function Point Analysis.* Englewood Cliffs, N.J.: Prentice Hall, 1989.
Dreger, J. Brian. *Function Point Analysis*. Englewood Cliffs, N.J.: Prentice Hall, 1989.
Galorath, Daniel D. and Michael W. Evans. *Software Sizing, Estimation, and Risk Management*. Philadelphia: Auerbach, 2006.
Garmus, David, and David Herron. *Measuring the Software Process: A Practical Guide to Functional Measurement.* Englewood Cliffs, N.J.: Prentice Hall, 1995.
Grady, Robert B. *Practical Software Metrics for Project Management and Process Improvement.* Englewood Cliffs, N.J.: Prentice Hall, 1992.
——— and Deborah L. Caswell. *Software Metrics: Establishing a Company-Wide Program.* Englewood Cliffs, N.J.: Prentice Hall, 1987.
Grady, R. B., and D. C. Caswell. *Software Metrics: Establishing a Company-Wide Program.* Englewood Cliffs, N.J.: Prentice Hall, 1987.
Howard, Alan, ed. *Software Metrics and Project Management Tools.* Phoenix, Ariz.: Applied Computer Research (ACR), 1997.
Humphrey, W. *Managing the Software Process.* Reading, Mass.: Addison-Wesley, 1989.
IFPUG Counting Practices Manual, Release 4. Westerville, Oh.: International Function Point Users Group, April 1995.
Jones, Capers. *Critical Problems in Software Measurement.* Carlsbad, CA: Information Systems Management Group, 1993a.
Jones, Capers. *U.S. Industry Averages for Software Productivity and Quality*. Version 4.0. Burlington, Mass.: Software Productivity Research, Inc., December 1989.
Jones, Capers. "Measuring Programming Quality and Productivity," *IBM Systems Journal*, Vol. 17, no. 1, 1978. Armonk, N.Y.: IBM Corporation.
———. *Software Productivity and Quality Today—The Worldwide Perspective.* Carlsbad, CA: Information Systems Management Group, 1993b.
———. *Assessment and Control of Software Risks.* Englewood Cliffs, N.J.: Prentice Hall, 1994.
———. *New Directions in Software Management.* Carlsbad, CA: Information Systems Management Group, 1993.
———. *Patterns of Software System Failure and Success.* Boston: International Thomson Computer Press, 1995.
———. *Applied Software Measurement, Second Edition.* New York: McGraw-Hill, 1996.
———. *The Economics of Object-Oriented Software.* Burlington, Mass.: Software Productivity Research, April 1997a.
———. *Software Quality—Analysis and Guidelines for Success.* Boston: International Thomson Computer Press, 1997b.
———. *Software Assessments, Benchmarks, and Best Practices.* Boston: Addison Wesley, 2000.
Kan, Stephen H. *Metrics and Models in Software Quality Engineering, Second Edition.* Boston: Addison-Wesley, 2003.
Kemerer, C. F. "Reliability of Function Point Measurement—A Field Experiment." *Communications of the ACM,* **36:** 85–97 (1993).
Kendall, R. C., and E. C. Lamb, "Management Perspectives on Programs, Programming, and Productivity." Presented at *GUIDE 45*, Atlanta, Ga., 1977. Reprinted in Capers Jones, *Programming Productivity—Issues for the Eighties*, IEEE Press, Catalog Number EHO239-4, 1986, pp. 35–44.
Keys, Jessica. *Software Engineering Productivity Handbook.* New York: McGraw-Hill, 1993.
Laird, Linda M. and Carol M. Brennan. *Software Measurement and Estimation: A Practical Approach.* New York: John Wiley & Sons, 2006.
Lewis, James P. *Project Planning, Scheduling & Control.* New York: McGraw-Hill, 2005.

Marciniak, John J., ed. *Encyclopedia of Software Engineering, Vols. 1 and 2*. New York: John Wiley & Sons, 1994.

McConnell, Steve. *Software Estimation: Demystifying the Black Art*. Redmond, WA: Microsoft Press, 2006.

Mertes, Karen R. *Calibration of the CHECKPOINT Model to the Space and Missile Systems Center (SMC) Software Database (SWDB).* Thesis AFIT/GCA/LAS/96S-11, Air Force Institute of Technology (AFIT), Wright-Patterson AFB, Ohio, September 1996.

Ourada, Gerald, and Daniel V. Ferens. "Software Cost Estimating Models: A Calibration, Validation, and Comparison," in *Cost Estimating and Analysis: Balancing Technology and Declining Budgets.* New York: Springer-Verlag, 1992, pp. 83–101.

Pressman, Roger. *Software Engineering: A Practitioner's Approach with Bonus Chapter on Agile Development.* New York: McGraw-Hill, 2003.

Putnam, Lawrence H. *Measures for Excellence—Reliable Software on Time, Within Budget.* Englewood Cliffs, N.J.: Yourdon Press/Prentice Hall, 1992.

———, and Ware Myers. *Industrial Strength Software—Effective Management Using Measurement.* Los Alamitos, Calif.: IEEE Press, 1997.

Reifer, Donald, ed. *Software Management, Fourth Edition*. Los Alamitos, Calif.: IEEE Press, 1993.

Rethinking the Software Process. CD-ROM. Lawrence, Kans.: Miller Freeman, 1996. (This CD-ROM is a book collection jointly produced by the book publisher, Prentice Hall, and the journal publisher, Miller Freeman. It contains the full text and illustrations of five Prentice Hall books: Capers Jones, *Assessment and Control of Software Risks*; Tom DeMarco, *Controlling Software Projects*; Brian Dreger, *Function Point Analysis*; Larry Putnam and Ware Myers, *Measures for Excellence*; and Mark Lorenz and Jeff Kidd, *Object-Oriented Software Metrics.*)

Rubin, Howard. *Software Benchmark Studies for 1997.* Pound Ridge, N.Y.: Howard Rubin Associates, 1997.

Stutzke, Richard D. *Estimating Software-Intensive Systems: Projects, Products, and Processes.* Boston: Addison Wesley, 2005.

Software Productivity Research, Inc., *CHECKPOINT™ Questionnaire*. Version 1.2. Burlington, Mass.: Software Productivity Research, Inc., 1989.

Symons, Charles R. *Software Sizing and Estimating—Mk II FPA (Function Point Analysis),* Chichester, U.K.: John Wiley & Sons, 1991.

Walston, C., and C. P. Felix. "A Method of Programming Measurement and Estimation." *IBM Systems Journal*, vol. 10, no. 1, 1977. Reprinted in Capers Jones, *Programming Productivity—Issues for the Eighties*, IEEE Press, Catalog Number EHO239-4, 1986, pp. 60–79.

Wellman, Frank. *Software Costing: An Objective Approach to Estimating and Controlling the Cost of Computer Software*. Englewood Cliffs, N.J.: Prentice Hall, 1992.

Yourdon, Ed. *Death March—The Complete Software Developer's Guide to Surviving "Mission Impossible" Projects.* Upper Saddle River, N.J.: Prentice Hall PTR, 1997.

Zells, Lois. *Managing Software Projects—Selecting and Using PC-Based Project Management Systems.* Wellesley, Mass.: QED Information Sciences, 1990.

Zvegintzov, Nicholas. *Software Management Technology Reference Guide.* New York: Dorset House Press, 1994.

Chapter

2

The History and Evolution of Software Metrics

The software industry is almost 60 years old, which makes it a fairly mature industry. One would think that after 60 years the software industry would have well-established methods for measuring productivity and quality, and also a large volume of accurate benchmark data derived from thousands of measured projects. However, this is not quite the case.

There are a number of proprietary collections of software benchmark data, such as those collected by the Software Engineering Institute (SEI), Gartner Group, Software Productivity Research (SPR), the David Consulting Group, Quantitative Software Management (QSM), and a number of others. Some of theses collections are large and may top 10,000 projects. However, the details from these benchmark collections are provided to clients, but not to the general public other than the data that gets printed in books such as this one.

Only the non-profit International Software Benchmark Standard Group (ISBSG) has data that is widely and commercially available. As this book is written in 2008, the ISBSG collection of benchmark data contains perhaps 4,000 projects. The rate of growth is about 500 projects per year. The majority of ISBSG projects are measured using IFPUG function points, but some data is also available using COSMIC, NESMA, Mark II, and other common function point variants.

As of 2008 the software industry has dozens of metrics available, some of which have only a handful of users. Very few software metrics and measurement practices are supported by formal standards and training. Benchmarks vary from quite complete to so sparse in terms of what is collected that the value is difficult to ascertain.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

Major topics such as service-oriented metrics, database volumes, and quality have a severe shortage of both research and published results. Today in 2008 as this book is written, the total volume of reliable productivity and quality data for the entire industry is less than 1 percent of what is really needed.

In order to understand the lack of solid measurements in the software industry, it is useful to look at the history of software work from its origins just after World War II through 2008.

Evolution of the Software Industry and Evolution of Software Measurements

For the first 10 years or so of the software industry starting at around 1947 through 1957, most applications were quite small: the great majority were less than 1,000 source code statements in size. All of these were written in low-level assembly languages and some were patched in machine language, which is even more difficult to work with.

The first attempts to measure productivity and quality used "lines of code" measures and at the time (circa 1950) that metric was fairly effective. Coding took about 50 percent of the effort to build an application; debugging and testing took about 40 percent, and everything else took only about 10 percent.

In this early era, productivity measures based on lines of code and bug counts based on bugs or defects per 1,000 lines of code (KLOC) were the standard metrics of the era and were fairly effective. This is because in the early days of software, coding bugs were the most common and the most troublesome.

Between 1957 and 1967, the situation began to change dramatically. Low-level assembly languages started to be replaced by more powerful procedural languages such as COBOL, FORTRAN, and APL. As computers were applied to business issues such as banking and manufacturing, application sizes grew from 1,000 lines of code up past 100,000 lines of code.

These changes in both programming languages and application sizes began to cause problems for the lines of code metric. By 1967 coding itself was beginning to drop below 30 percent of application development effort, while production of requirements, specifications, plans, and paper documents began to approach 40 percent. Testing and debugging took about 30 percent. Adding to the problem, some applications were written in two or more different programming languages.

The lines of code metric continued to be used as a general productivity metric, but some weaknesses were being noted. For example, it was not possible to do direct measurement of design and documentation

productivity and quality with LOC metrics, because these paper activities did not involve coding.

By the mid-1970s more serious problems with LOC metrics were noted. It is reasonable to assume that high-level programming languages improve development productivity and quality, which indeed they do. But attempts to measure these improvements using LOC metrics led to the discovery of an unexpected paradox: LOC metrics penalize high-level languages and give the best results for low-level languages.

Assume you have two identical applications, with one written in assembly language and one written in COBOL. The assembly version required 10,000 lines of code but the COBOL version only required 3,000 lines of code. When you measure coding rates, both languages were coded at a rate of 1,000 lines of code per month. But since the COBOL version was only one-third the size of the assembly version, the productivity gains can't be seen using LOC metrics.

Even worse, assume the specifications and documents for both versions took 5 months of effort. Thus, the assembly version required a total of 15 months of effort while the COBOL version required only 8 months of effort. If you measure the entire project with LOC metrics, the assembly version has a productivity rate of 666 lines of code per month, but the COBOL version only had a productivity rate 375 lines of code per month. Obviously, the economic benefits of high-level programming languages disappear when measured using LOC metrics.

These economic problems are what caused IBM to assign Allan Albrecht and his colleagues in White Plains to try and develop a useful software metric that was independent of code volumes, and which could measure both economic productivity and quality without distortion.

After several years of effort, what Albrecht and his colleagues came up with was a new kind of metric termed *function points*. Function point metrics are based on five external aspects of software applications: inputs, outputs, inquiries, logical files, and interfaces.

After being used internally within IBM for several years, function points were discussed publicly for the first time in October 1979 in a paper which Albrecht presented at a joint SHARE/GUIDE/IBM conference held at Monterey, California.

Between the publication of the first edition in 1991 and this third edition in 2008, function point metrics have become the dominant measurement instrument for software in the United States, Canada, Australia, New Zealand, South Africa, and much of Europe. Function point measurements are also expanding rapidly in the Pacific Rim, India, and South America.

Once an application's function point total is known, the metric can be used for a variety of useful economic purposes, including:

- **Studies of software production**
  - Function points per person-month
  - Work hours per function point
  - Development cost per function point
  - Maintenance cost per function point
- **Studies of software consumption**
  - Function points owned by an enterprise
  - Function points needed by various kinds of end users
  - Build, lease, or purchase decision making
  - Contract vs. in-house decision making
  - Software project value analysis
- **Studies of software quality**
  - Test cases and runs required per function point
  - Requirements and design defects discovered per function point
  - Coding defects per function points
  - Documentation defects per function point

When he invented function points, Albrecht was working for IBM's Data Processing Services group. He had been given the task of measuring the productivity of a number of software projects. Because IBM's DP Services group developed custom software for a variety of other organizations, the software projects were written in a wide variety of languages: COBOL, PL/I, RPG, APL, and assembly language, to name but a few, and some indeed were written in mixed languages.

Albrecht knew, as did many other productivity experts, that it was not technically possible to measure software production rates across projects written in different levels of language with the traditional lines-of-code measures.

Other researchers knew the problems that existed with lines-of-code measures, but Albrecht deserves the credit for going beyond those traditional and imperfect metrics and developing a technique that can be used to explore the true economics of software production and consumption.

Albrecht's paper on function points was first published in 1979 in the conference proceedings, which had only limited circulation of several hundred copies. In 1981, with both IBM's and the conference organization's permission, the paper was reprinted in the IEEE tutorial entitled *Programming Productivity: Issues for the Eighties* by the author.

This republication by the IEEE provided the first widespread circulation of the concept of function point metrics outside IBM.

The IEEE tutorial brought together two different threads of measurement research. In 1978, the author had published an analysis of the mathematical problems and paradoxes associated with lines-of-code measures. That article, also included in the 1981 IEEE tutorial, proved mathematically that lines of code were incapable of measuring productivity in the economic sense. Thus it provided strong justification for Albrecht's work on function point metrics, which were the first in software history that could be used for measuring economic productivity.

It should be recalled that the standard economic definition of productivity is: "Goods or services produced per unit of labor or expense." A line of code is neither goods nor services in the economic sense. Customers do not buy lines of code directly, and they often do not even know how many lines of code exist in a software product. Also, lines of code are not the primary deliverables of software projects, so they cannot be used for serious studies of the production costs of software systems or programs.

The greatest bulk of what is actually produced and what gets delivered to users of software comprises words and paper documents. In the United States, sometimes as many as 400 English words will be produced for every line of source code in large systems. Often more than three times as much effort goes into word production as goes into coding. Words are obviously not economic units for software, since customers do not buy them directly, nor do they have any real control over the quantity produced. Indeed in some cases, such as large military systems, far too many unnecessary words are produced.

As already mentioned, customers do not purchase lines of code either, so code quantities have no intrinsic value to users. In most instances, customers neither know nor care how much code was written or in what language an application is embodied. Indeed, if the same functionality could be provided to users with less code by means of a higher-level language, customers might benefit from the cost reductions.

If neither of the two primary production units of software (words and code) is of direct interest to software consumers, then what exactly constitutes the "goods or services" that make software a useful economic commodity? The answer, of course, is that users care about the functions of the application.

Prior to Albrecht's publication of the function point metric, there were only hazy and inaccurate ways to study software production, and there was no way at all to explore the demand or consumption side of the software economic picture.

Thus, until 1979 the historical problem of measuring software productivity could be stated precisely: "The natural units of software production

(words and code) were not the same as the units of software consumption (functions)." Economic studies require a standard definition of both what is produced and also of what is consumed.

Since neither words nor lines of code are of direct interest to software consumers, there was no tangible unit that matched the economic definition of goods or services that lent itself to studies of software's economic productivity.

Recall from earlier that a function point is an abstract but workable surrogate for the goods that are produced by software projects. Function points are the weighted sums of five different factors that are of interest to users:

- Inputs
- Outputs
- Logical files (also called user data groups)
- Inquiries
- Interfaces

Function points were defined by Albrecht to be "end-user benefits," and they are now serving as the economic units that customers wish to purchase or to have developed. That is, function points are beginning to be used in contract negotiations between software producers and their clients.

Clients and developers alike can discuss an application rationally in terms of its inputs, outputs, inquiries, files, and interfaces. Further, if requirements change, clients can request additional inputs or outputs after the initial agreement, and software providers can make rational predictions about the cost and schedule impact of such additions, which can then be discussed with clients in a reasonable manner.

Function points, unlike lines of code, can also be used for economic studies of both software production costs and software consumption. For production studies, function points can be applied usefully to effort, staffing, and cost-related studies. Thus, it is now known that the approximate U.S. average for software productivity at the project level is 5 function points per person-month. At the corporate level, where indirect personnel such as executives and administrators are included as well as effort expended on canceled projects, the U.S. average is about 1.5 function points per person-month. Function points can also be used to explore the volumes and costs of software paperwork production, a task for which lines of code were singularly inappropriate.

For consumption studies, function points are beginning to create an entirely new field of economic research that was never before possible.

It is now possible to explore the utilization of software within industries and the utilization of software by the knowledge workers who use computers within those industries. Table 2-1 shows the approximate quantity of function points utilized by selected enterprises in the United States at three time periods: 1990, 1995, and 2005. The data were derived from studies of the production libraries of representative companies.

The overall growth of software as a business tool has been extremely rapid. Software utilization appears to be growing larger every year without showing any signs of reaching a stable point.

Although the margin of error in Table 2-1 is high, this kind of large-scale study of the volumes of software portfolios required by industries could not easily be performed prior to the advent of the function point metric. Although lines of code might be attempted, the kinds of companies shown in Table 2-1 typically use up to a dozen or more different languages: COBOL, C, Assembler, SQL, and so on.

Table 2-2 shows yet another new kind of consumption analysis possible from the use of function points. It illustrates the approximate number of function points required to support the computer usage of selected occupation groups in the United States. Here too the margin of error is high and the field of research is only just beginning. But research into information-processing consumption was not technically possible prior to the advent of the function point metric. It appears that function points may be starting to shed light on one of the most difficult economic questions of the century: how to evaluate the business value of software applications.

Every working day most knowledge workers now spend at least three hours using their computers. Modern business could not operate without computers and software, nor could government operations.

TABLE 2-1 Approximate Number of Function Points Owned by Selected U.S. Companies

Enterprise	1990	1995	2005
Small local bank	40,000	125,000	250,000
Medium commercial bank	150,000	350,000	700,000
Large international bank	200,000	450,000	1,500,000
Medium life insurance company	200,000	400,000	1,200,000
Large life insurance company	250,000	550,000	1,750,000
Large telephone company	350,000	450,000	1,250,000
Large telephone manufacturer	350,000	600,000	1,800,000
Medium manufacturing company	75,000	200,000	1,000,000
Large manufacturing company	125,000	375,000	1,300,000
Large computer company	500,000	1,650,000	5,000,000
Department of Defense	1,000,000	3,000,000	12,000,000

TABLE 2-2 Function Points Used by Selected U.S. Occupations

	1990	1995	2005
Airline reservation clerk	5,000	30,000	60,000
Travel agent	5,000	35,000	60,000
Corporate controller	10,000	20,000	200,000
Bank loan officer	6,000	15,000	50,000
Aeronautical engineer	5,000	25,000	50,000
Electrical engineer	5,000	25,000	50,000
Telecommunications engineer	5,000	20,000	50,000
Software engineer	3,000	15,000	50,000
Mechanical engineer	2,500	12,500	50,000
First line software manager	1,000	3,500	25,000
Second line software manager	1,000	3,500	25,000
Corporate CIO	2,000	15,000	100,000
Corporate CEO	1,000	3,000	60,000
Physician for diagnostic work	1,000	15,000	200,000
Municipal law enforcement	1,000	5,000	20,000

Since software economic consumption studies are not widespread even in 2008, the information in Tables 2-1 and 2-2 must be regarded as preliminary and as containing a high margin of error. Nonetheless, it is a powerful illustration of the economic validity of function points that such studies can be attempted at all.

In conclusion, although function points are an abstract and synthetic metric, they are no less valid for economic purposes than many other standard economic metrics that also are abstract and synthetic, such as the Dow Jones stock indicator, cost per square foot for construction projects, accounting rates of return, internal rates of return, net present value, and the formulas for evaluating the net worth of an enterprise. Function points are starting to point the way to the first serious economic analyses of software production and software consumption since the computer era began.

The Cost of Counting Function Point Metrics

There has been a long-standing problem with using function point metrics. Manual counting by certified experts is fairly expensive. Assuming that the average daily fee for hiring a certified function point counter in 2008 is $2,500 and that manual counting using the IFPUG function point method proceeds at a rate of about 400 function points per day, the result is that manual counting costs about $6.25 per function point. Both the costs and comparatively slow speed has been an economic barrier to the widespread adoption of functional metrics.

These statements are true for IFPUG, COSMIC, Mark II, NESMA, and other major forms of function point metrics.

There are some alternate methods for deriving function point counts that are less expensive, although perhaps at the cost of reduced accuracy. Table 2-3 shows the current range of function point counts using several alternative methods. Note that Table 2-3 has a high margin of error. There are broad ranges in counting speeds and also in daily costs for every function point variation. Also, Table 2-3 is not an accurate depiction because each "Agile story point" reflects a larger unit of work than a normal function point. A story point may be equal to at least 2 and perhaps more function points. A full Agile "story" may top 20 function points in size.

The term *Agile story points* refers to a metric derived from Agile stories, the method used by some Agile projects for deriving requirements. An Agile story point is usually somewhat larger than a function point, and is perhaps roughly equivalent to two IFPUG function points, or perhaps even more.

Manual counting implies analyzing and enumerating function points from requirements and specifications by a certified counter. The accuracy is good, but the costs are high. (Note function point counts by uncertified counters are erratic and unreliable. However, counts by certified counters have been studied and achieve good accuracy.) Since COSMIC, IFPUG, Mark II, and NESMA function points all have certification procedures, this method works with all of the common function point variants. Counting use case points is not yet a certified activity as of 2008.

Automatic derivation refers to experimental tools that can derive function points from written requirements and specifications.

TABLE 2-3 Range of Costs for Calculating Function Point Metrics

Method of Counting	Function Points Counted per Day	Average Daily Compensation	Cost per Function Point	Accuracy of Count
Agile story points	50	$2,500	$50.00	5%
Use case manual counting	250	$2,500	$10.00	3%
Mark II manual counting	350	$2,500	$7.14	3%
IFPUG manual counting	400	$2,500	$6.25	3%
NESMA manual counting	450	$2,500	$5.55	3%
COSMIC manual counting	500	$2,500	$5.00	3%
Automatic derivation	1,000	$2,500	$2.50	5%
"Light" function points	1,500	$2,500	$1.67	10%
NESMA "indicative" counts	1,500	$2,500	$1.67	10%
Backfiring from LOC	10,000	$2,500	$0.25	50%
Pattern-matching	300,000	$2,500	$0.01	15%

Such tools have been built experimentally, but are not commercially available. IFPUG function points has been the major metric supported. These tools require formal requirements and/or design documents such as those using structured design, use cases, and other standard methods. This method has good accuracy, but its speed is linked to the rate at which the requirements are created. It can go no faster. Even so, there is a great reduction in manual effort.

The phrase *"light" function points* refers to a method developed by David Herron of the David Consulting Group. This method simplifies the counts to a higher level and therefore is more rapid. The "light" method uses average values for the influential factors. As this book is written, the "light" method shows promise but is still somewhat experimental.

The phrase *NESMA indicative* refers to a high-speed method developed by the Netherlands Function Point Users Group (NESMA) that uses constant weights and concentrates on counts of data files.

Backfiring is the oldest alternative to manual counting, and actually was developed in the mid-1970s when A.J. Albrecht and his colleagues first developed function point metrics. During the trials of function points, both lines of code and function points were counted, which provided approximate ratios between LOC metrics and function point metrics.

However, due to variations in how code is counted and variations in individual programming "styles," the accuracy of backfiring is not high. At best, backfiring can come within 10 percent of manual counts, but at worst the difference can top 100 percent. Backfiring works best when logical statements are counted and worst when physical lines of code are counted. This means that backfiring is most effective when automated code counting tools are available. Note also that backfiring ratios are not the same for IFPUG, COSMIC, Mark II, or NESMA function points. Therefore, each function point method requires its own tables of backfiring values. For that matter, when counting rules change, all of the backfiring ratios would change at the same time. Backfiring works best when the code itself is counted automatically by a tool that supports formal counting rules.

The new *pattern-matching* approach is based on the fact that many thousands of projects have now been counted with function points. By using a formal taxonomy to match new projects with similar older projects, and by making some adjustments for complexity (problem, code, and data complexity specifically), pattern-matching works by aligning new projects with similar historical projects. Pattern-matching offers a good combination of speed and accuracy. The method is still experimental as this book is written in 2008, but the approach seems to offer solid economic advantages coupled with fairly good accuracy.

Pattern-matching also has some unique capabilities, such as being useful for legacy applications where there may be no written specifications. It can also be used for commercial software packages, such as large ERP applications, office suites, and operating systems where the specifications are not available and where the vendors have not provided function point sizes.

However, as of 2008, the only two methods that have widespread utilization are normal counting and backfiring. Both have accumulated data on thousands of projects.

The cost and speed of counting function points is a significant barrier to usage for large applications. For example, a major ERP package such as SAP or Oracle is in the range of 275,000 function points in size. To calculate function points manually for such a large system would take up to six months by a team of certified counters, and cost more than half a million dollars. There is an obvious need for quicker, cheaper, but still accurate methods of arriving at function point totals before really large applications will utilize these powerful metrics.

The high costs and low speed of manual counting explain why backfiring has been popular for so many years. The costs and speed are quite good, but unfortunately accuracy has lagged. The pattern-matching approach, which can be tuned to fairly good precision, is a promising future method.

Problems with and Paradoxes of Lines-of-Code Metrics

One of the criticisms sometimes levied against function points is that they are subjective whereas lines of code are considered to be objective. It is true the function point counting to date has included a measure of human judgment, and therefore includes subjectivity. (The emergence of a new class of automated function point tools is about to eliminate the current subjectivity of functional metrics.) However, it is not at all true that the lines-of-code metric is an objective metric. Indeed, as will be shown, in the past 1,000 years of human history there has not been a metric as subjective as a line of code since the days when the yard was based on the length of the arm of the king of England!

Also, code counting is neither inexpensive nor particularly accurate. Table 2-4 shows similar information to Table 2-3, only this time the various methods for counting code are illustrated.

Manual counting of code is fairly expensive and not particularly accurate. In fact for some "visual languages" such as Visual Basic, code counting is so difficult and unreliable that it almost never occurs. Code counting works best when it is automated by tools that can be programmed to follow specific rules for dealing with topics such as dead code, reused code, and blank lines between paragraphs.

TABLE 2-4 Range of Costs for Calculating Lines-of-Code (LOC) Metrics

Method of Counting	Lines of Code Counted Per Day	Average Daily Compensation	Cost per Line of Code	Accuracy of Count
Manual count of "visual" languages	500	$2,500	$5.00	50%
Manual count of physical LOC	5,000	$2,500	$0.50	10%
Manual count of statements	2,500	$2,500	$1.00	10%
Automated physical LOC counts	50,000	$2,500	$0.05	2%
Automated logical counts	50,000	$2,500	$0.05	2%
Reverse backfiring to LOC	200,000	$2,500	$0.01	50%
Pattern-matching	1,000,000	$2,500	$0.0025	15%

Some compilers produce source code counts. There are many commercial tools that can count source code statements, and also calculate complexity values such as cyclomatic and essential complexity. Multiple programming languages such as Java and HTML add a measure of complexity. Also, topics such as comments, reused code, dead code, blank lines between paragraphs, and delimiters between logical statements tend to introduce ambiguity into code counts. Even so, automated code counting is both faster and more reliable than manual counts. In fact the phrase "manual count" is something of a misnomer. What usually happens for large applications is that a small sample is actually counted, and then those values are applied to the rest of the application.

The phrase "reverse backfiring" merely indicates that the formulae for converting code statements into function points are bi-directional and work from either starting point.

The pattern-matching approach is similar for code counts as for function points. It operates by comparing the application in question to older applications of the same class and type written in the same programming language or languages.

To understand the effectiveness of function points, it is necessary to understand the problems of the older lines-of-code metric. Regretfully, most users of lines of code have no idea at all of the subjectivity, randomness, and quirky deficiencies of this metric.

As mentioned, the first complete analysis of the problems of lines-of-code metrics was the previously mentioned study by the author in

the *IBM Systems Journal* in 1978. In essence there are three serious deficiencies associated with lines of code:

- There has never been a national or international standard for a line of code that encompasses all procedural languages.
- Software can be produced by such methods as program generators, spreadsheets, graphic icons, reusable modules of unknown size, and inheritance, wherein entities such as lines of code are totally irrelevant.
- Lines-of-code metrics paradoxically move backward as the level of the language gets higher, so that the most powerful and advanced languages appear to be less productive than the more primitive low-level languages. That is due to an intrinsic defect in the lines-of-code metrics. Some of the languages thus penalized include Ada, APL, C++, Java, Objective-C, SMALLTALK, and many more.

Let us consider these problems in turn.

Lack of a Standard Definition for Lines of Code The software industry will soon be 70 years of age, and lines of code have been used ever since its start. It is surprising that, in all that time, the basic concept of a line of code has never been standardized.

Counting Physical or Logical Lines The variation that can cause the greatest apparent difference in size is that of determining whether a line of code should be terminated physically or logically. A physical termination would be caused by the ENTER key of a computer keyboard, which completes the current line and moves the cursor to the next line of the screen. A logical termination would be a formal delimiter, such as a semicolon, colon, or period.

For languages such as Basic, which allow many logical statements per physical line, the size counted by means of logical delimiters can appear to be up to 500 percent larger than if lines are counted physically. On the other hand, for languages such as COBOL, which utilize conditional statements that encompass several physical lines, the physical method can cause the program to appear perhaps 200 percent larger than the logical method. From informal surveys of the clients of Software Productivity Research carried out by the author, it appears that about 35 percent of U.S. project managers count physical lines, 15 percent count logical lines, and 50 percent do not count by either method.

Counting Types of Lines The next area of uncertainty is which of several possible kinds of lines should be counted. The first full explanation of the variations in counting code was perhaps that published by the author in 1986, which is surprisingly recent for a topic almost 65 years of age.

Most procedural languages include five different kinds of source code statements:

- Executable lines (used for actions, such as addition)
- Data definitions (used to identify information types)
- Comments (used to inform readers of the code)
- Blank lines (used to separate sections visually)
- Dead code (code left in place after updates)

Again, there has never been a U.S. standard that defined whether all five or only one or two of these possibilities should be utilized. In typical business applications, about 40 percent of the total statements are executable lines, 35 percent are data definitions, 10 percent are blank, and 15 percent are comments. For systems software such as operating systems, about 45 percent of the total statements are executable, 30 percent are data definitions, 10 percent are blank, and 15 percent are comments. However, as applications age, dead code begins to appear and steadily increases over the years.

Dead code consists of code segments that have been bypassed or replaced with newer code following a bug fix or an update. Rather than excising the code, it is often left in place in case the new changes don't work. It is also cheaper to leave dead code than to remove it. The volume of dead code increases with the age of software applications. For a typical legacy application that is now ten years old, the volume of dead code will probably approximate 15 percent of the total code in the application. Dead code became an economic issue during the Y2K era, when some Y2K repair companies were charging on the basis of every line of code. It was soon recognized that dead code was going to be an expensive liability if an application was turned over to a commercial Y2K shop for remediation based on per line charges.

From informal surveys of the clients of Software Productivity Research carried out by the author, it appears that about 10 percent count only executable lines, 20 percent count executable lines and data definitions, 15 percent also include commentary lines, and 5 percent even include blank lines! About 50 percent do not count lines of code at all.

Counting Reusable Code Yet another area of extreme uncertainty is that of counting reusable code within software applications. Informal code reuse by programmers is very common, and any professional programmer will routinely copy and reuse enough code to account for perhaps 20 to 30 percent of the code in an application when the programming is done in an ordinary procedural language such as C, COBOL, or FORTRAN. For object-oriented languages such as SMALLTALK, C++, and Objective C, the volume of reuse tends to exceed 50 percent because

of the facilities of inheritance that are intrinsic in the object-oriented family of languages. Finally, some corporations have established formal libraries of reusable modules, and many applications in those corporations may exceed 75 percent of the total volume of reused code.

The problem with measuring reusability centers around whether a reused module should be counted at all, counted only once, or counted each time it occurs. For example, if a reused module of 100 source statements is included five times in a program, there are three variations in counting:

- Count the reused module at every occurrence.
- Count the reused module only once.
- Do not count the reused module at all, since it was not developed for the current project.

From informal surveys of the clients of Software Productivity Research carried out by the author, about 25 percent would count the module every time it occurred, 20 percent would count the module only once, and 5 percent would not count the reused module at all. The remaining 50 percent do not count source code at all.

Applications Written in Multiple Languages The next area of uncertainty, which is almost never discussed in the software engineering literature, is the problem of using lines-of-code metrics for multi-language applications. From informal surveys of the clients of Software Productivity Research, it appears that about a third of all U.S. applications include more than one language and some may include a dozen or more languages. Some of the more common language mixtures include

- Java mixed with HTML
- Java mixed with C
- COBOL mixed with a query language such as SQL
- COBOL mixed with a data definition language such as DL/1
- COBOL mixed with several other special-purpose languages
- C mixed with Assembler
- Visual Basic mixed with HTML
- Ada mixed with Assembler
- Ada mixed with Jovial and other languages

Since there are no U.S. standards for line counting that govern even a single language, multi-language projects show a great increase in the number of random errors associated with lines-of-code data.

Additional Uncertainties Concerning Lines of Code Many other possible counting variations can affect the apparent size of applications in which lines of code are used. For example:

- Including or excluding changed code for enhancements
- Including or excluding macro expansions
- Including or excluding job control language (JCL)
- Including or excluding deleted code
- Including or excluding scaffold or temporary code that is written but later discarded

The overall cumulative impact of all of these uncertainties spans more than an order of magnitude. That is, if the most verbose of the line-counting variations is compared to the most succinct, the apparent size of the application will be more than 10 times larger than the most succinct! That is an astonishing and even awe-inspiring range of uncertainty for a unit of measure approaching its 60th year of use!

Unfortunately, very few software authors bother to define which counting rules they used. The regrettable effect is that most of the literature on software productivity that expresses the results in terms of lines of code is essentially worthless for serious research purposes.

Size Variations That Are Due to Individual Programming Style A minor controlled study carried out within IBM illustrates yet another problem with lines of code. Eight programmers were given the same specification and were asked to write the code required to implement it. The amount of code produced for the same specification varied by about 5 to 1 between the largest and the smallest implementation. That was due not to deliberate attempts to make productivity seem high, but rather to the styles of the programmers and to the varying interpretations of what the specifications asked for.

Software Functions Delivered Without Producing Code As software reuse and service-oriented architecture (SOA) become more common, it is possible to envision fairly large applications that consist of loosely coupled reusable components. In other words, some future applications can be constructed with little or no procedural code being written.

A large-scale study within ITT is in which the author participated found that about 26 percent of the approximately 30,000 applications owned by the corporation had been leased or purchased from external vendors rather than developed internally. Functionality was being delivered to the ITT software users of the packages, but ITT was obviously not producing the code. Specifically, about 140,000 function points out of the corporate total of 520,000 function points had been delivered to

users in the form of packages rather than being developed by the ITT staff. The effective cost per function point of unmodified packages averaged about 35 percent of the cost per function point of custom development. However, for packages requiring heavy modification, the cost per function point was about 105 percent of equivalent custom development. Lines-of-code metrics are essentially impossible for studying the economics of package acquisitions or for make-vs.-buy productivity decisions. The vendors do not provide code counts, so unless a purchaser uses some kind of code-counting engine there is no convenient way of ascertaining the volume of code in purchased software.

The advent of the object-oriented languages and the deliberate pursuit of reusable modules by many corporations is leading to the phenomenon that the number of unique lines of code that must actually be hand-coded is shrinking, whereas the functional content of applications continues to expand. The lines-of-code metric is essentially useless in judging the productivity impact of this phenomenon. The use of inheritance and methods by object-oriented languages, the use of corporate reusable module libraries, and the use of application and program generators make the concept of lines of code almost irrelevant.

As the 21st century progresses, an increasing number of graphics or icon-based "languages" will appear, and in them application development will proceed in a visual fashion quite different from that of conventional procedural programming. Lines of code, never defined adequately even for procedural languages, will be hopeless for graphics-based languages.

The Paradox of Reversed Productivity for High-Level Languages

Although this point has been discussed several times earlier, it cannot be overemphasized: The LOC metrics penalize high-level languages and make low-level languages look artificially better than they really are.

Although lack of standardization is the most visible surface problem with lines of code, the deepest and most severe problem is a mathematical paradox that causes real economic productivity and apparent productivity to move in opposite directions! This phenomenon was introduced and illustrated in Chapter 1, but it deserves a more elaborate explanation.

The paradox manifests itself under these conditions: As real economic software productivity improves, metrics expressed in both lines of source code per time unit and cost per source line form will tend to move backward and appear to be worse than previously. Thus, as real economic productivity improves, the apparent cost per source line will be higher and the apparent lines of source code per time unit will be lower than before even though less effort and cost were required to complete an application.

Failure to understand the nature of this paradox has proved to be embarrassing to the industry as a whole and to many otherwise capable managers and consultants who have been led to make erroneous recommendations based on apparent productivity data rather than on real economic productivity data. The fundamental reason for the paradox has actually been known since the industrial revolution, or for more than 200 years, by company owners and manufacturing engineers. The essence of the paradox is this: If a product's manufacturing cycle includes a significant proportion of fixed costs and there is a decline in the number of units produced, the cost per unit will naturally go up.

For software, a substantial number of development activities either include or behave like fixed costs. For example, the applications requirements, specifications, and user documents are likely to stay constant in size and cost regardless of the language used for coding. This means that when enterprises migrate from a low-level language such as assembly language to a higher-level language such as COBOL or Ada or Java, they do not have to write as many lines of source code to develop applications but the paperwork costs are essentially fixed. In effect, the number of source code units produced declines in the presence of fixed costs.

Since so many development activities either include fixed costs or behave like fixed costs, the cost per source line will naturally go up. Examples of activities that behave like fixed costs, since they are independent of coding, include user requirements, analysis, functional design, design reviews, user documentation, and some forms of testing such as function testing.

Table 2-5 is an example of the paradox associated with lines of source code metrics in a comparison of Assembler and Ada. Assume $5,000 per month is the fully burdened salary rate in both cases. Note that Table 2-5

TABLE 2-5 The Paradox of Lines-of-Code Metrics and High-Level Languages

	Assembler Version	Java Version	Difference
Source code size	100,000	25,00	–75,000
Activity, in person-months:			
Requirements	10	10	0
Design	25	25	0
Coding	100	20	–80
Documentation	15	15	0
Integration and testing	25	15	–10
Management	25	15	–10
Total effort	200	100	–100
Total cost	$1,000,000	$500,000	–$500,000
Cost per line	$10	$20	+$20
Lines per month	500	250	–250

is intended to illustrate the mathematical paradox, and it exaggerates the trends to make the point clearly visible.

As shown in Table 2-6, with function points the economic productivity improvements are clearly visible and the true impact of a high-level language such as Ada can be seen and understood. Thus, function points provide a better base for economic productivity studies than lines-of-code metrics.

To illustrate some additional findings vis-à-vis the economic advantages of high-level languages as explored by function points, a larger study covering ten different programming languages will be used.

Some years ago the author and his colleagues at Software Productivity Research were commissioned by a European telecommunications company to explore an interesting problem.

Many of this company's products were written in the CHILL programming language. CHILL is a fairly powerful third-generation procedural language developed specifically for telecommunications applications by the CCITT, an international telecommunications association.

Software engineers and managers within the company were interested in moving to object-oriented programming using C++ as the primary language. Studies had been carried out by the company to compare the productivity rates of CHILL and C++ for similar kinds of applications. These studies concluded that CHILL projects had higher productivity rates than C++ when measured with the productivity metric LOC per staff month.

We were asked to explore the results of these experiments, and either confirm or challenge the finding that CHILL was superior to C++. We were also asked to make recommendations about other possible languages such as Ada83, Ada95, C, PASCAL, or SMALLTALK.

TABLE 2-6 The Economic Validity of Function Point Metrics

	Assembler Version	Java Version	Difference
Source code size	100,000	25,000	–75,000
Function points	300	300	0
Activity, in person-months:			
Requirements	10	10	0
Design	25	25	0
Coding	100	20	–80
Documentation	15	15	0
Integration and testing	25	15	–10
Management	25	15	–10
Total effort	200	100	–100
Total cost	$1,000,000	$500,000	–$500,000
Cost per function point	$3,333	$1,666	–$1,667
Function points per person-month	1.5	3.0	–1.5

As background information, we also examined the results of using macro-assembly language. All eight of these languages were either being used for telecommunications software, or were candidates for use by telecommunications software as in the case of Ada95, which was just being prepared for release.

Later, two additional languages were included in the analysis: PL/I and Objective C. The PL/I language has been used for switching software applications and the construction of PBX switches for many years. For example, several ITT switches were constructed in a PL/I variant called Electronic Switching PL/I (ESPL/I).

The Objective C language actually originated as a telecommunications language within the ITT Corporation under Dr. Tom Love at the ITT Programming Technology Center in Stratford, Connecticut. However, the domestic ITT research facilities were closed after Alcatel bought ITT's telecommunications business, so the Objective C language was brought to the commercial market by Dr. Love and the Stepstone Corporation. The data on Objective C in this section was derived from mathematical modeling, and not from an actual product.

The basic conclusion of the study was that object-oriented languages did offer substantial economic productivity gains compared to third-generation procedural languages, but that these advantages were hidden when measured with the LOC metric.

However, object-oriented analysis and design is more troublesome and problematic. The unified modeling language (UML) and the older Booch, Jacobsen, and Rumbaugh "flavors" of OO analysis and design had very steep learning curves and were often augmented or abandoned in order to complete projects that used them.

For a discussion of software failures and abandonment, refer to my book *Patterns of Software Systems Failure and Success* (Jones 1995).

Basis of the Study The kind of project selected for this study was the software for a private branch exchange switch (PBX), similar to the kinds of private switching systems utilized by the larger hotels and office buildings.

The original data on CHILL was derived from the client's results, and data for the other programming languages were derived from other telecommunication companies who are among our clients. (The Ada95 results were originally modeled mathematically, since this language had no available compilers at the time of the original study. The Objective C results were also modeled.)

To ensure consistent results, all versions were compared using the same sets of activities, and any activities that were unique for a particular project were removed. The data was normalized using the CHECKPOINT® measurement and estimation tool. This tool facilitates

comparisons between different programming languages and different sets of activities, since it can highlight and mask activities that are not common among all projects included in the comparison. The full set of activities that we studied included more than 20, but the final study used consolidated data based on six major activities:

- Requirements
- Design
- Coding
- Integration and testing
- Customer documentation
- Management

The consolidation of data to six major activities was primarily to simplify presenting the results. The more granular data actually utilized included activity and task-level information. For example, the cost bucket labeled "integration and testing" really comprised information derived from integration, unit testing, new function testing, regression testing, stress and performance testing, and field testing.

For each testing stage, data was available on test case preparation, test case execution, and defect repair costs. However, the specific details of each testing step are irrelevant to an overall economic study. So long as the aggregation is based on the same sets of activities, this approach does not degrade the overall accuracy.

Since the original study concentrated primarily on object-oriented programming languages as opposed to object-oriented analysis and design, data from OO requirements and analysis were not explored in depth in the earlier report.

Other SPR clients and other SPR studies that did explore various OO analysis and design methods found that they had steep learning curves and did not benefit productivity in the near term. In fact, the OO analysis and design approaches were abandoned or augmented by conventional analysis and design approaches in about 50 percent of the projects that attempted to use them initially.

The new unified modeling language (UML), which consolidates the methods of Booch, Rumbaugh, and Jacobsen, now has formal training available, so the learning curve has been reduced.

Metrics Evaluated for the Study Since the main purpose of the study was to compare object-oriented languages and methods against older procedural languages and methods, it was obvious that we needed measurements and metrics that could handle both the old and the new. Since the study was aimed at the economic impact associated with entire

projects and not just pure coding, it was obvious that the metric needed to be useful for measuring non-coding activities such as requirements, design, documentation, and the like. The metrics that were explored for this study included

- Physical lines of code, using the SEI counting rules
- Logical lines of code, using the SPR counting rules
- Feature points
- Function points
- MOOSE (metrics for object-oriented system environments)

The SEI approach of using physical lines (Park, 92) was eliminated first, since the variability is both random and high for studies that span multiple programming languages. Individual programming styles can affect the count of physical lines by several hundred percent. When multiple programming languages are included, the variance can approach an order of magnitude.

The SPR approach of using logical statements gives more consistent results than a count of physical lines and reduces the variations due to individual programming styles. The usage of logical statements also facilitates a technique called "backfiring," or the direct conversion of LOC metrics into functional metrics. The use of logical statements using the SPR counting rules published in the second edition of *Applied Software Measurement* (Jones, 96) but still used in 2008 for the third edition provides the basis of the LOC counts shown later in this report. However, LOC in any form is not a good choice for dealing with non-coding activities such as document creation.

The feature point metric was originally developed for telecommunications software, and indeed was used by some U.S. and European telecommunication companies for software measurement work. This metric is not as well known in Europe as the function point metric, however, so it was not used in the final report. (For those unfamiliar with the feature point metric, it adds a sixth parameter, a count of algorithms, to the five parameters used by standard function point metrics. This metric was also described in the second edition of *Applied Software Measurement*.)

The IFPUG function point metric was selected for displaying the final results of this study, using a constant 1,500 function points as the size of all eight versions. The function point metric is now the most widely utilized software metric in both the United States and some 20 other countries, including much of Europe.

More data is now being published using this metric than any other, and the number of automated tools that facilitate counting function

points is growing exponentially. Also, this metric was familiar to the European company for which the basic analysis was being performed.

Since a key purpose of the study was to explore the economics of object-oriented programming languages, it was natural to consider using the new "metrics for object-oriented system environments" (MOOSE) developed by Dr. Chris Kemerer of MIT (Kemerer and Chidamber, 1993).

The MOOSE metrics include a number of constructs that are only relevant to OO projects, such as Depth of the Inheritance Tree (DIT), Weighted Methods per Class (WMC), Coupling Between Objects (CBO), and several others.

However, the basic purpose of the study is a comparison of ten programming languages, of which six are older procedural languages. Unfortunately the MOOSE metrics do not lend themselves to cross-language comparisons between OO projects and procedural projects, and so had to be excluded.

In some ways, the function point metric resembles the "horsepower" metric. In the year 1783 James Watt tested a strong horse by having it lift a weight, and found that it could raise a 150 pound weight almost four feet in one second. He created the ad hoc empirical metric, "horsepower," as 550 foot pounds per second. The horsepower metric has been in continuous usage for more than 200 years and has served to measure steam engines, gasoline engines, diesel engines, electric motors, turbines, and even jet engines and nuclear power plants.

The function point metric also had an ad hoc empirical origin, and is also serving to measure types of projects that did not exist at the time of its original creation in the mid-1970s, such as client-server software, object-oriented projects, web projects, and multimedia applications.

Function point metrics originated as a unit for measuring size, but have also served effectively as a unit of measure for software quality, for software productivity, and even for exploring software value and return on investment. In all of these cases, function points are generally superior to the older LOC metric. Surprisingly, function points are also superior to some of the modern object-oriented (OO) metrics, which are often difficult to apply to quality or economic studies.

For those considering the selection of metrics for measuring software productivity and quality, eight practical criteria can be recommended:

- The metric should have a standard definition and be unambiguous.
- The metric should not be biased and unsuited for large scale statistical studies.
- The metric should have a formal user group and adequate published data.
- The metric should be supported by tools and automation.

- It is helpful to have conversion rules between the metric and other metrics.
- The metric should deal with all software deliverables, and not just code.
- The metric should support all kinds and types of software projects.
- The metric should support all kinds and types of programming languages.

It is interesting that the function point metric is currently the only metric that meets all eight criteria.

The feature point metric lacks a formal user group and has comparatively few published results but otherwise is equivalent to function points and hence meets seven of the eight criteria.

The LOC metric is highly ambiguous, lacks a user group, is not suited for large-scale statistical studies, is unsuited for measuring non-code deliverables such as documentation, and does not handle all kinds of programming languages such as visual languages or generators. In fact, the LOC metric does not really satisfy any of the criteria.

The MOOSE metrics are currently in evolution and may perhaps meet more criteria in the future. As this paper was written, the MOOSE metrics for object-oriented projects appeared unsuited for non-OO projects, do not deal with deliverables such as specifications or user documentation, and lack conversion rules to other metrics.

Results of the Analysis The first topic of interest is the wide range in the volume of source code required to implement essentially the same application. Note that the function point total of the PBX project used for analysis was artificially held constant at 1,500 function points across all ten versions in this example.

In real-life, of course, the functionality varied among the projects utilized. Table 2-7 gives the volumes of source code and the number of source code statements needed to encode one function point for the ten examples.

The next topic of interest is the amount of effort required to develop the ten examples of the project. Table 2-8 gives the effort for the six major activities and the overall quantity of effort expressed in terms of "staff months." Note that the term *staff month* is defined as a typical working month of about 22 business days, and includes the assumption that project work occurs roughly 6 hours per day, or 132 work hours per month. The data normalization feature of the CHECKPOINT® tool was used to make these assumptions constant, even though the actual data was collected from a number of companies in different countries, where the actual number of work hours per staff month varied.

TABLE 2-7 Function Point and Source Code Sizes for Ten Versions of the Same Project (A PBX Switching System of 1,500 Function Points in Size)

Language	Size in Function Point	Language Level	LOC per Function Point	Size in LOC
Assembly	1,500	1	250	375,000
C	1,500	3	127	190,500
CHILL	1,500	3	105	157,500
PASCAL	1,500	4	91	136,500
PL/I	1,500	4	80	120,000
Ada83	1,500	5	71	106,500
C++	1,500	6	55	82,500
Ada95	1,500	7	49	73,500
Objective C	1,500	11	29	43,500
SMALLTALK	1,500	15	21	31,500
Average	**1,500**	**6**	**88**	**131,700**

Note that since the data came from four companies, each of which had varying accounting assumptions, different salary rates, different work month definitions, work pattern differences, and other complicating factors, the separate projects were run through the CHECKPOINT® tool and converted into standard work periods of 132 hours per month, with costs of $10,000 per month.

None of the projects were exactly 1,500 function points in size, and the original sizes ranged from about 1,300 to 1,750 function points in size. Here, too, the data normalization feature was used to make all ten versions identical in factors that would conceal the underlying similarities of the examples.

TABLE 2-8 Staff Months of Effort for Ten Versions of the Same Software Project (A PBX Switching System of 1,500 Function Points in Size)

Language	Req. (Months)	Design (Months)	Code (Months)	Test (Months)	Doc. (Months)	Mgt. (Months)	TOTAL (Months)
Assembly	13.64	60.00	300.00	277.78	40.54	89.95	781.91
C	13.64	60.00	152.40	141.11	40.54	53.00	460.69
CHILL	13.64	60.00	116.67	116.67	40.54	45.18	392.69
PASCAL	13.64	60.00	101.11	101.11	40.54	41.13	357.53
PL/I	13.64	60.00	88.89	88.89	40.54	37.95	329.91
Ada83	13.64	60.00	76.07	78.89	40.54	34.99	304.13
C++	13.64	68.18	66.00	71.74	40.54	33.81	293.91
Ada95	13.64	68.18	52.50	63.91	40.54	31.04	269.81
Objective C	13.64	68.18	31.07	37.83	40.54	24.86	216.12
SMALLTALK	13.64	68.18	22.50	27.39	40.54	22.39	194.64
Average	**13.64**	**63.27**	**100.72**	**100.53**	**40.54**	**41.43**	**360.13**

It can readily be seen that the overall costs associated with coding and testing are much less significant for object-oriented languages than for procedural languages. However, the effort associated with initial requirements, design, and user documentation are comparatively inelastic and do not fluctuate in direct proportion to the volume of code required.

Note an interesting anomaly: The effort associated with analysis and design is higher for the object-oriented projects than for the procedural projects. This is due to the steep learning curve and general difficulties associated with the common "flavors" or OO analysis and design: the UML and the older Booch, Rumbaugh, and Jacobsen original methods.

The most interesting results are associated with measuring the productivity rates of the ten versions. Note how apparent productivity expressed using the metric "LOC per Staff Month" moves in the opposite direction from productivity expressed in terms of "Function Points per Staff Month."

The data in Table 2-9 is derived from the last column of Table 2-8, or the total amount of effort devoted to the ten projects. Table 2-9 gives the overall results using both LOC and function point metrics.

As can easily be seen, the LOC data does not match the assumptions of standard economics, and indeed moves in the opposite direction from real economic productivity. It has been known for many hundreds of years that when manufacturing costs have a high proportion of fixed costs and there is a reduction in the number of units produced, the cost per unit will go up.

TABLE 2-9 Productivity Rates for Ten Versions of the Same Software Project (A PBX Switching System of 1,500 Function Points in Size)

Language	Effort (Months)	Function Point per Staff Month	Work Hours per Function Point	LOC per Staff Month	LOC per Staff-Hour
Assembly	781.91	1.92	68.81	480	3.38
C	460.69	3.26	40.54	414	3.13
CHILL	392.69	3.82	34.56	401	3.04
PASCAL	357.53	4.20	31.46	382	2.89
PL/I	329.91	4.55	29.03	364	2.76
Ada83	304.13	4.93	26.76	350	2.65
C++	293.91	5.10	25.86	281	2.13
Ada95	269.81	5.56	23.74	272	2.06
Objective C	216.12	6.94	19.02	201	1.52
SMALLTALK	194.64	7.71	17.13	162	1.23
Average	**360.13**	**4.17**	**31.69**	**366**	**2.77**

The same logic is true for software. When a line of code is defined as the unit of production, and there is a migration from low-level procedural languages to object-oriented languages, the number of units that must be constructed declines. The costs of paper documents such as requirements and user manuals do not decline, and tend to act like fixed costs. This inevitably leads to an increase in the cost per LOC for object-oriented projects, and a reduction in LOC per staff month when the paper-related activities are included in the measurements.

On the other hand, the function point metric is a synthetic metric totally divorced from the amount of code needed by the application. Therefore, function point metrics can be used for economic studies involving multiple programming languages and object-oriented programming languages without bias or distorted results. The function point metric can also be applied to non-coding activities such as requirements, design, user documentation, integration, testing, and even project management.

In order to illustrate the hazards of LOC metrics without any ambiguity, Table 2-10 simply ranks the ten versions in descending order of productivity. As can be seen, the rankings are completely reversed between the function point list and the LOC list.

When using the standard economic definition of productivity, which is "goods or services produced per unit of labor or expense" it can be seen that the function point ranking matches economic productivity assumptions.

The function point ranking matches economic assumptions because the versions with the lowest amounts of both effort and costs have the highest function point productivity rates and the lowest costs per function point rates.

TABLE 2-10 Rankings of Productivity Levels Using Function Point Metrics and LOC Metrics

Productivity Ranking Using Function Point Metrics		Productivity Ranking Using LOC Metrics	
1	SMALLTALK	1	Assembly
2	Objective C	2	C
3	Ada95	3	CHILL
4	C++	4	PASCAL
5	Ada83	5	PL/I
6	PL/I	6	Ada83
7	PASCAL	7	C++
8	CHILL	8	Ada95
9	C	9	Objective C
10	Assembly	10	SMALLTALK

The LOC rankings, on the other hand, are the exact reversal of real economic productivity rates. This is the key reason why usage of the LOC metric is viewed as "professional malpractice" when it is used for cross-language productivity or quality comparisons involving both high-level and low-level programming languages.

When the costs of the ten versions are considered, the hazards of using "lines of code" as a normalizing metric become even more obvious. Table 2-11 shows the total costs of the ten versions, and then normalizes the data using both "cost per line of code" and "cost per function point."

Note that telecommunications companies, as a class, tend to be on the high side in terms of burden rates and compensation and typically average from $9,000 to more than $12,000 per staff month for their fully burdened staff compensation rates.

The ten examples shown in Table 2-11 are all arbitrarily assigned a fully burdened monthly salary rate of $10,000.

Note the paradoxical increase in cost per line of code for the more powerful languages, at the same time that both total costs and cost per function point decline. Using a metric such as LOC that moves in the opposite direction as economic productivity improves is a very hazardous practice.

Indeed, the LOC metric is so hazardous for cross-language comparisons that a strong case can be made that using LOC for normalization of data involving multiple or different languages should be considered an example of professional malpractice.

The phrase *professional malpractice* implies that a trained knowledge worker did something that was hazardous and unsafe, and that

TABLE 2-11 Cost of Development for Ten Versions of the Same Software Project (A PBX Switching System of 1,500 Function Points in Size)

Language	Effort (Months)	Burdened Salary(Months)	Burdened Costs	Burdened Cost per Function Point	Burdened Cost per LOC
Assembly	781.91	$10,000	$7,819,088	$5,212.73	$20.85
C	460.69	$10,000	$4,606,875	$3,071.25	$24.18
CHILL	392.69	$10,000	$3,926,866	$2,617.91	$24.93
PASCAL	357.53	$10,000	$3,575,310	$2,383.54	$26.19
PL/I	329.91	$10,000	$3,299,088	$2,199.39	$27.49
Ada83	304.13	$10,000	$3,041,251	$2,027.50	$28.56
C++	293.91	$10,000	$2,939,106	$1,959.40	$35.63
Ada95	269.81	$10,000	$2,698,121	$1,798.75	$36.71
Objective C	216.12	$10,000	$2,161,195	$1,440.80	$49.68
SMALLTALK	194.64	$10,000	$1,946,425	$1,297.62	$61.79
Average	**360.13**	**$10,000**	**$3,601,332**	**$2,400.89**	**$27.34**

the level of training and prudence required to join the profession should have been enough to avoid the unsafe practice. Since it is obvious that the LOC metric does not move in the same direction as economic productivity, and indeed moves in the opposite direction, it is a reasonable assertion that misuse of LOC metrics should be viewed as professional malpractice if a report or published data caused some damage or harm.

One of the severe problems of the software industry has been the inability to perform economic analysis of the impact of various tools, methods, or programming languages. It can be stated that the LOC metric has been a significant barrier that has slowed down the evolution of software engineering, since it has blinded researchers and prevented proper exploration of software engineering factors.

The use of software for switching applications is now a technology that is more than 50 years old. Although the productivity, staffing, and schedule results of the object-oriented versions are significantly better than for the procedural version, the OO versions are all fairly recent and were created by software teams with a great deal of experience in automated switching applications.

The age of the software applications also seems to play a major part in the volume of reusable materials utilized. One of the major claims of the OO paradigm is that increased volumes of reusable materials will be available. However, these claims are mostly subjective assertions that are not yet fully supported by empirical data.

Software reuse is a multi-faceted topic that involves much more than reusable source code. A really effective software reuse program can include a dozen artifacts, and at a very minimum should include five critical artifacts:

- Reusable requirements
- Reusable designs and specifications
- Reusable source code
- Reusable test cases
- Reusable user documentation

Unfortunately, much of the literature on software reuse, including the OO literature on software reuse, has concentrated exclusively on code reuse. For example, the OO literature is almost silent on reuse of user documentation and fairly sparse on the reuse of requirements, design, and test materials.

Hopefully, the unified modeling language will increase the emphasis on reusable design, but the OO literature is still sparse on the reuse of many other software artifacts.

Software Quality Analysis Another interesting aspect of the study was the exploration of quality results, using "defect potentials" and "defect removal efficiency" as the primary metrics. The *defect potential* of a software project is the sum of the errors found in requirements, design, code, user documentation, and bad fixes secondary errors introduced when repairing prior defects.

The *defect removal efficiency* of a project is the total percentage of defects eliminated prior to delivery of software to its intended clients. Defect removal efficiency is normally calculated on the anniversary of the delivery of software. For example, if the development team found a total of 900 bugs in a software project and the users reported a total of 100 bugs in the first year of usage, then the defect removal efficiency for that project would obviously be calculated to be 90 percent when both sets of bug reports are summed and the ratio of pre-release defects is calculated. Table 2-12 summarizes the defect potentials of the ten projects.

Note that the columns of requirements defects and documentation defects were held constant using the data normalization feature of the CHECKPOINT® tool. The older projects did not record this value, and since all of the projects did the same thing, it was reasonable to use a constant value across all ten versions of this similar project.

The large volume of coding defects for the assembly language and C language versions of the projects should come as no surprise to programmers who have used these rather taxing languages for complex real-time software such as switching systems.

Table 2-13 shows the defect removal efficiency levels, or the number of defects eliminated before deployment of the software via design inspections, code inspections, and testing. Note that telecommunications

TABLE 2-12 Defect Potentials for Ten Versions of the Same Software Project (A PBX Switching System of 1,500 Function Points in Size)

Language	Req. Defects	Design Defects	Coding Defects	Doc. Defects	Bad Fix Defects	TOTAL DEFECTS
Assembly	1,500	1,875	7,500	900	1,060	12,835
C	1,500	1,875	3,810	900	728	8,813
CHILL	1,500	1,875	3,150	900	668	8,093
PASCAL	1,500	1,875	2,730	900	630	7,635
PL/I	1,500	1,875	2,400	900	601	7,276
Ada83	1,500	1,875	2,130	900	576	6,981
C++	1,500	2,025	1,650	900	547	6,622
Ada95	1,500	2,025	1,470	900	531	6,426
Objective C	1,500	2,025	870	900	477	5,772
SMALLTALK	1,500	2,025	630	900	455	5,510
Average	**1,500**	**1,935**	**2,634**	**900**	**627**	**7,596**

TABLE 2-13 Delivered Defects for Ten Versions of the Same Software Project (A PBX Switching System of 1,500 Function Points in Size)

Language	Total Defects	Defect Removal Efficiency	Delivered Defects	Delivered Defects per Function Point	Delivered Defects per KLOC
Assembly	12,835	91.00%	1,155	0.77	3.08
C	8,813	92.00%	705	0.47	3.70
CHILL	8,093	93.00%	567	0.38	3.60
PASCAL	7,635	94.00%	458	0.31	3.36
PL/I	7,276	94.00%	437	0.29	3.64
Ada83	6,981	95.00%	349	0.23	3.28
C++	6,622	93.00%	464	0.31	5.62
Ada95	6,426	96.00%	257	0.17	3.50
Objective C	5,772	96.00%	231	0.15	5.31
SMALLTALK	5,510	96.00%	220	0.15	7.00
Average	**7,580**	**94.00%**	**455**	**0.30**	**3.45**

software, as a class, is much better than U.S. averages in terms of defect removal efficiency levels due in part to the wide-spread usage of formal design reviews, formal code inspections, testing specialists, and the use of state of the art defect tracking and test automation tools.

Table 2-13 uses the metric "defects per KLOC" rather than "defects per LOC." The term "KLOC" refers to units of 1,000 lines of code, and is preferred for quality metrics because defects per LOC would usually require three 0's before encountering a value and it is hard to visualize quality with so many decimal places.

It is immediately apparent that the metric "Defects per KLOC" is not suitable for measuring defect volumes that include errors made in requirements, design, and user documentation as the data in Table 2-13 does. For quality, just as for productivity, the LOC metric reverses the real situation and yields incorrect conclusions.

Although coding defects dominate in the older assembly and C projects, design defects dominate in the newer OO projects. These non-coding defects amount to more than 50 percent of the total defect load for the more recent projects, and may amount to more than 80 percent of the defect load for object-oriented projects.

The metric "Defects per Function Point" is a much better choice for dealing with all of the possible sources of software error, and especially so for OO projects.

There is a subtle problem with measuring quality in a modern OO environment. As might be expected, the defect potentials for the object-oriented versions are lower than for the procedural versions. If you examine pure coding defects, it is surprising that the overall defect

removal efficiency rates decline for projects using OO languages. This is a counter-intuitive phenomenon, but one which is supported by substantial empirical data.

The reason for this seeming quality paradox is actually very logical once it is understood. Coding defects are easier to get rid of than any other category. Defect removal efficiency for pure coding errors is usually higher than 95 percent. For requirements and design errors, defect removal is much tougher, and averages less than 80 percent. When OO programming languages are used, there is a significant reduction in coding errors as a result of the defect prevention aspects of OO languages. This means that the troublesome requirements and design errors constitute the bulk of the defects encountered on OO projects, and they are very hard to eliminate.

However, the telecommunications community has generally been aware of this problem and has augmented their testing stages by a series of pre-test design and code inspections. Also, for the more recent projects the set of requirements and design errors are reduced because these projects are no longer novel, but are being constructed as simply the most recent links in a growing chain of similar projects.

Conclusions of the Study The most obvious conclusions from the study were these four:

- Object-oriented programming languages appear to benefit both software productivity and software quality.
- There is not yet any solid data that indicates that object-oriented analysis and design methods benefit either software productivity or quality, although there is insufficient information about the unified modeling language (UML) to draw a conclusion.
- Neither the quality nor productivity benefits of object-oriented programming can be directly measured using LOC metrics.
- The synthetic function point metric appears to be able to bridge the gap between procedural and OO projects and can express results in a fashion that matches standard economic assumptions about productivity.

The function point metric is also a useful tool for exploring software quality, when used in conjunction with other metrics such as defect removal efficiency rates.

A somewhat surprising result of the original study was that productivity and quality cannot be compared between OO projects and non-OO projects using specialized object-oriented metrics such as MOOSE as they were currently described. It is possible to use OO metrics for comparisons within the OO paradigm, but not outside the OO paradigm.

Some less obvious conclusions were also reached regarding the completeness of the object-oriented paradigm itself. Because coding productivity tends to increase more rapidly than the productivity associated with any other activity with OO projects, it would be unsafe to use only coding as the basis for estimating complete OO projects.

This finding also has a corollary, which is that the OO paradigm is not yet fully developed for the paper-related activities dealing with requirements, design, and user documentation. The new unified modeling language (UML) may have a significant impact on OO analysis and design, but it is premature to know for sure as this report is written.

It should be pointed out that the OO literature has lagged in attempting to quantify the costs of the paper deliverables associated with software. A more subtle observation appears to be that both inspection methods and testing methods for OO projects are also in need of further research and development.

This study is not completely rigorous and does not attempt to control all sources of error. There may be a substantial margin of error with the results, but hopefully the major conclusions are reasonable.

Note also that the telecommunications industry is usually below U.S. averages in software productivity rates, although one of the top five U.S. industries in terms of software quality levels. The high complexity levels and high performance and reliability requirements of telecommunications software invoke a need for very careful development practices, and very thorough quality control.

In addition, telecommunications software ranks number two out of all U.S. industries in terms of the total volume of text documents and paper materials created for software projects (military software ranks number one in the paperwork production factor). Telecommunications companies are also above U.S. averages in terms of burdened salary rates. Therefore, the data shown here should not be used for general estimating purposes.

To summarize, economic productivity deals with the amount of effort required to produce goods or services that users consume or utilize. Studies of productivity, therefore, can be divided into those that concentrate on production efficiencies and those that concentrate on demand or consumption.

The first two editions of this book included a history of function points from 1979 through the date of publication. However, that approach is no longer suitable. As of 2008 there are more than 20 variants of function points, plus many other metrics as well. It seems more appropriate to discuss the major function point variants and the other emerging metrics, such as use case points, rather than to deal with the history of only function point metrics. A small amount of history will be retained just to show the original economic motives and goals for the development of functional metrics.

The Varieties of Functional Metrics Circa 2008

This discussion of various kinds of function point metrics does not attempt to teach them in detail. The counting rules for some of the function point methods, such as IFPUG, Mark II, and COSMIC, are more than 100 pages in length. It is much better to seek out a specific training manual or textbook for the method of interest, or even better to take formal classroom training from a certified instructor.

In its simplest and original form circa 1975, the function point metric was defined as consisting of the counts of five external attributes of software applications:

- Inputs
- Outputs
- Logical files
- Inquiries
- Interfaces

The numbers of these five attributes were totaled, some additional calculations were performed to adjust for complexity over a range of about 25 percent, and the final result was the "adjusted function point total" of the application.

As time passed many changes have occurred with function point metrics, including the development of more than 20 variations. These variations all have in common several kinds of extensions to the original approach:

- Making additions to the original five attributes
- Broadening the range of complexity adjustments
- Adding supplemental factors not included in the original
- Attempting to simplify or speed up the calculations

An interesting question, and one that is difficult to answer, is whether having so many variations of functional metrics is beneficial or harmful to the industry. It is a fact of science and engineering that there are often multiple metrics for the same values. For example, we have nautical miles and statute miles and also kilometers; we have Fahrenheit and Celsius temperature scales; we have both English gallons and American gallons and also liters; we have British and American tons; we have several methods for calculating octane ratings; and there are several kinds of horsepower.

However, the software industry seems to have an excessive number of choices in many technical areas. For example, there is no compelling reason why the software industry has developed more than 600 programming languages. There is no apparent reason to have more than 40 named methods for design techniques. To an impartial observer, there seems to be insufficient justification for having more than 20 flavors of functional metrics. For that matter, there seems to be insufficient justification for having at least five different methods for counting source code including both counts of physical lines and counts of logical statements.

My hypothesis for why the software industry has so many variations for performing basic tasks is that none of the methods are fully satisfactory solutions for the problems they address. Each variant solves some fragment of a problem, but not the entire problem.

This hypothesis is strengthened by looking at all of the flavors of functional metrics. If you consider only the external attributes the methods use, and compare them to the known issues that affect software, no single function point method includes every important topic. But the overall set of various functional methods seems to include every known factor. Consider the following short list of topics known to affect software projects:

Topic	Source
Inputs	IFPUG
Outputs	IFPUG
Logical files	IFPUG
Extended file types	Full function points
Interfaces	IFPUG
Entities/relationships	Mark II
Algorithms	Feature points
Data movement	COSMIC
Layers	COSMIC
Complexity	All methods

Each of the individual functional metrics deals with a subset of key topics. When all of the flavors of functional metric are considered at the same time, they capture essentially all topics of importance.

In terms of overall usage as this book is written, four of the function point flavors seem to account for more than 97 percent of total usage: IFPUG, COSMIC, MARK II, and the Netherlands NESMA approach. These four methods all have formal training and certified instructors. They have also been reviewed and endorsed by the International Standards Organization (ISO).

In time it may be that one of these four methods will dominate. However, given the history of the software industry, it is even more likely that some newer variation will occur in a few years and claim to be the most effective functional metric yet developed.

In a perfect world the major function point organizations (IFPUG, COSMIC, NESMA, ISO, etc.) would have a joint meeting and agree on a single consolidated functional metric. The probability of this occurring is only slightly better than the probability of achieving peace in the Middle East.

The 20 varieties of functional metric discussed in this chapter include, in alphabetical order:

- 3D function points
- Backfiring function points
- COSMIC function points
- DeMarco "bang" function points
- Engineering function points
- Feature points
- Full function points
- Function points "light"
- IFPUG function points
- ISO function point standards
- Mark II function points
- Micro function points
- Netherlands function points (NESMA)
- Object points
- Pattern-matching and function points
- SPR function points
- Story points
- Unadjusted function points
- Use case points
- Web object points

There are actually more varieties in existence than the 20 discussed here, but a sample of 20 is enough to illustrate the fact that we probably have more flavors of functional metric than may actually be needed. Readers can well imagine the difficulty and complexity of attempting to create industry-wide benchmarks when the data is divided among projects measured with 20 different metrics.

This section provides short discussions of the major flavors of functional metrics circa 2008. For additional and very current information, it is interesting to do web searches using phrases such as "COSMIC function points" or "IFPUG function points" as the search argument.

3D Function Points

This method was developed by Scott Whitemire for Boeing circa 1990. The phrase "3D" stands for three dimensions of data, function, and control. Because of the nature of Boeing's work, 3D function points were stated to be aimed at scientific and real-time software.

The 3D function point method was one of those created because of the feeling that standard IFPUG function points might undercount complex software such as embedded and real-time applications. From early results, the 3D approach did tend to produce larger totals for such applications. However, because of the specific aim of the 3D method at the engineering world, few trials were made to judge how IFPUG and 3D function points compared for information technology projects. It is likely that the 3D approach created larger totals across the board.

The 3D function point approach attracted some attention within the aerospace industry in the 1990s, but there are few recent citations to this method.

For web searching, the name "3D" was an unfortunate choice because web searches turn up hundreds of citations for three-dimensional plotting algorithms and software tools, but almost nothing about 3D function points per se. As of 2008, the 3D function point approach is probably no longer deployed. A web search using the phrase "3D function points" only turned up 12 citations, and none of them were recent.

Backfiring Function Points

The term "backfiring" refers to mathematical conversion from source code statements to equivalent function points. Because function point metrics are independent of source code, it is an obvious question as to why such a correlation even exists. Therefore, some background information is useful.

Before function points were invented, programming languages were roughly analyzed by either "generations" or by "levels." The word "generation" referred to the approximate year or era when the language was developed, but as languages multiplied this method fell out of use.

The "level" of a programming language was defined in terms of the number of code statements in basic assembly language that would provide about the same set of functions as one statement in a higher-level language. By trial and error experiments in the early 1970s, both COBOL and FORTRAN were determined to be level 3 languages

because it took about three statements in basic assembly language to provide the same functionality as one statement in either FORTRAN or COBOL. The APL language was defined as a level 10 language, using the same rationale.

The level method was used by IBM in the early 1970s for evaluating the productivity rates associated with various programming languages. For example, Assembly, FORTRAN, COBOL, and APL can all be coded at a rate of about 1,000 lines of code per month. Thus, using unadjusted lines of code, there is no visible difference in productivity. But when productivity is expressed in terms of "equivalent assembly statements," then basic assembly language has a productivity rate of 1,000 LOC per month; COBOL and FORTRAN of 3,000 LOC per month; and APL of 10,000 LOC per month. This method, although somewhat crude, at least enabled economic researchers to evaluate the power of higher-level programming languages.

In the 1970s when A.J. Albrecht and his colleagues within IBM were first developing function point metrics, they had access to many IBM historical projects where source code volumes had already been measured. When application sizes were compared using both function points and counts of logical source code statements, it was discovered more or less by accident that for many programming languages there were correlations between size measured using LOC metrics and size measured using the original IBM function point metrics. For example, COBOL applications seemed to have about 105 source code statements in the procedure and data divisions for every function point.

By the mid-1970s this kind of correlation between LOC and function points had been evaluated for about 30 common programming languages. These correlations led to the use of backfiring as early as 1975. Backfiring and function point metrics have co-existed for more than 30 years. Backfiring did not replace the older "level" concept but instead absorbed it and used levels as a convenient shortcut for calculating the ratio of source code statements to function points. For example all level 3 languages could be turned into approximate function points simply by dividing 320 by 3, which resulted in a ratio of 106.7 source code statements per one function point.

Although simultaneous measurements of function points and source code statements did find that the average value of the relationship between COBOL and function points was about 107 statements to 1 function point, the ranges spanned values from less than 70 statements per function point to almost 170 statements per function point. In general, the ranges were proportional to the usage of the language, i.e., languages with many users such as COBOL had wider variances than languages with only a few users such as MUMPS.

The technology of backfiring, or direct conversion of LOC data into the equivalent number of function points, was pioneered by A.J. Albrecht, the inventor of the function point metric as well as by the author of this book. The term "backfiring" was coined by the author circa 1985, and used in the first edition of *Applied Software Measurement* in 1991, although it had been discussed in the context of the SPQR/20 estimating tool in 1985. The first commercial software estimating tool to support backfiring was SPQR/20™, which came out in 1985 and supported bi-directional sizing for 30 languages. Today, backfiring is a standard feature for many commercial software estimating tools. In spite of the lack of real precision, the ease of use of backfiring had made it a popular approach.

From 30 languages in the 1980s, the number of languages that can be sized or backfired has now grown to more than 600 when all dialects are counted, and there is no limit to the applicability of the fundamental technique. Even combinations of multiple languages can be backfired, if their volumes of code are known. That is, combinations such as HTML and Java or SQL and COBOL can be dealt with simultaneously.

Software Productivity Research publishes an annual table of conversion ratios between logical lines of code and function points, and the current edition contains more than 600 programming languages and dialects. This table is available on SPR's web site (www.SPR.com). Similar tables are published by other software consulting companies such as the David Consulting Group and Gartner Group.

There are far too many programming languages to show more than a few examples here. Note also that the margin of error when backfiring is rather large. Even so, the results are interesting and now widely utilized. Table 2-14 lists examples taken from the author's original Table of Programming Languages and Levels, which is updated several times a year by Software Productivity Research. This table is extracted from the author's companion book *Estimating Software Costs* (McGraw-Hill, 2007)

This data indicates the ranges and median values in the number of source code statements required to encode one function point for selected languages. The counting rules for source code are based on logical statements rather than physical lines of code.

The best results from backfiring come within about 10 percent of matching function point counts by certified counters. The worst results, on the other hand, can deviate by more than 100 percent. The deviations are due to variations in individual programming styles, and to variations in how the code is actually counted. The best results are those where logical statements are counted by an automated tool. The worst results are those where physical lines are counted manually. With backfiring, the variances can be either positive or negative.

TABLE 2-14 Ratios of Source Code Statements to Function Points for Selected Programming Languages

		Source Statements per Function Point		
Language	Nominal Level	Low	Mean	High
1st Generation	1.00	220	320	500
Basic assembly	1.00	200	320	450
Macro assembly	1.50	130	213	300
C	2.50	60	128	170
BASIC (interpreted)	2.50	70	128	165
2nd Generation	3.00	55	107	165
FORTRAN	3.00	75	107	160
ALGOL	3.00	68	107	165
COBOL	3.00	65	107	150
CMS2	3.00	70	107	135
JOVIAL	3.00	70	107	165
PASCAL	3.50	50	91	125
3rd Generation	4.00	45	80	125
PL/I	4.00	65	80	95
MODULA 2	4.00	70	80	90
Ada83	4.50	60	71	80
LISP	5.00	25	64	80
FORTH	5.00	27	64	85
QUICK BASIC	5.50	38	58	90
C++	6.00	30	53	125
Ada 9X	6.50	28	49	110
Database	8.00	25	40	75
Visual Basic (Windows)	10.00	20	32	37
APL (default value)	10.00	10	32	45
SMALLTALK	15.00	15	21	40
Generators	20.00	10	16	20
Screen painters	20.00	8	16	30
SQL	27.00	7	12	15
Spreadsheets	50.00	3	6	9

That is, sometimes backfiring generates larger totals than manual counts of function points, and sometimes backfiring generates smaller totals.

Surprisingly, since backfiring is as old as function points themselves, none of the function point associations such as COSMIC, IFPUG, or NESMA have established backfiring committees or performed any formal evaluations of the method. As this book is written, there is no formal association of backfire users. Essentially, the backfiring method is an orphan.

Although there is no formal support for backfiring, the usage of the method is so widespread that probably as many as 75,000 projects have been "backfired" over the past 30 years. If so, then backfiring would actually be the most widely used method for ascertaining function points.

Although backfiring is not as accurate as actually counting function points, there is one special case where backfiring is more accurate: very small modifications to software applications that are less than ten function points in size. For very small maintenance changes of less than one function point, backfiring is the only current approach for deriving "micro function points" or fractional function points. For example, a small change of ten lines of COBOL would only be about 0.09 "micro function points." As of 2008 there are no actual rules for normal counting of "micro function points" from change requests or defect reports but these would not be hard to develop, as discussed later in this chapter.

The factors known to affect the accuracy of backfire ratios include

1. Source code counting rules used
2. Database structures associated with the application
3. Macro expansions and calls to external routines
4. Reused code in the application
5. Individual programming "styles"
6. Dead code still present in applications but no longer used
7. Cyclomatic and essential complexity levels
8. Specific dialects of programming languages
9. Changes in function point counting rules
10. Boundaries of the application being counted

It would not be difficult to perform controlled experiments on backfiring, but as of 2008 these have seldom been performed. The kinds of research needed on this topic include

- Recalibrating the basic "level" concept of programming language by coding sample problems in basic assembly and more modern languages such as Java, SMALLTALK, C++, and others. The research goal is to validate the number of statements in basic assembly required to encode one statement in the target language.
- Confirming or challenging "backfire" ratios by counting both logical statements and IFPUG function points for sample problems and the 4.2 counting rules.
- Developing "backfire" ratios for selected function point variations such as COSMIC function points, NESMA function points, web object points, use case points, etc.

- Developing effective ratios between counts of logical statements using the SPR counting rules and counts of physical lines using the SEI counting rules. The goal of this research is to allow backfiring from either physical or logical code counts.
- Developing effective backfire ratios for languages such as Visual Basic where procedural code is augmented by buttons and pull-down menus.

As of 2008, backfiring is probably used more often than normal function point counting, because it is so much quicker and cheaper. For aging legacy applications where written specifications have withered away or are not current, backfiring is one of the few methods available for ascertaining function point sizes.

However, it is important to note that backfire ratios are not constant. They must be adjusted for each flavor of function point, such as COSMIC and IFPUG, and they must be adjusted each time there is a significant change in counting rules. For example, backfire ratios that were valid for IFPUG function points in 1995 using version 3 of the counting rules are not valid today in 2008 when using version 4.2 of the IFPUG counting rules.

As this book is written in 2008, a web search on the phrase "backfiring function points" yields about 383,000 citations. Backfiring in 2008 remains a curious side-issue in the function point world. It is a method that is widely used and supported by many estimating tools and many articles, but more or less unacknowledged by any of the actual function point associations.

COSMIC Function Points

The term "COSMIC" stands for "Common Software Measurement International Consortium." The phrase "COSMIC function points" refers to one of the newer forms of functional metric first published circa 1999, although research on the COSMIC method started several years earlier.

Several of the developers of the COSMIC function point have had long experience with functional metrics. Two of these include Alain Abran, who developed the "full function point" in Canada, and Charles Symons, who developed the Mark II function point method in the United Kingdom. Some aspects of both Mark II and full function points are included in the newer COSMIC function point approach. Some of these features include expanded kinds of file types and expanded kinds of transaction types.

The stated motive for developing the COSMIC approach was to extend the range of projects that could be sized with functional metrics to include real-time and engineering applications. This motive assumes

that the older IFPUG function points could not be used for such applications. However, IFPUG function points are in fact capable of being utilized for exactly the same kinds of software that COSMIC function points are used for. In day-to-day use, both are used for business applications and also for real-time and embedded software.

Charles Symons, one of the COSMIC developers, was quoted in a 2006 interview as saying the COSMIC function points were not yet suitable for applications such as weather forecasting where algorithms were numerous and complex.

This is somewhat surprising because the older "feature point" metric circa 1986 included algorithms as a specific parameter. Rules for counting feature point algorithms were first published in the second edition of this book in 1996. Also, the "engineering function point" circa 1994 included specific counts of algorithms, and also included definitions and ways of counting algorithms.

Both COSMIC and IFPUG function points tend to put a great deal of emphasis on data and information. With IFPUG the emphasis is on numbers of logical files, while with COSMIC the emphasis is on data movement and transformation.

As of 2008 the COSMIC approach is gaining users and growing fairly rapidly in Europe and the Pacific Rim.

Some experiments have been performed to compare counts between IFPUG and COSMIC function points, but with somewhat ambiguous results. For systems, real-time, and embedded applications, COSMIC results are often perhaps 15 to 50 percent larger than IFPUG results. For management information systems, the results between the two can be close, or can vary in either direction by perhaps 10 percent.

The total number of projects measured using the COSMIC method is not known exactly as this is written in 2008, but probably exceeds 2,000 software projects. The most widespread usage of the COSMIC method seems to be in Europe, the United Kingdom, and the Pacific Rim. To date, use of the COSMIC method in the United States is sparse and probably experimental.

The growing popularity of the COSMIC function point can be seen from the fact that a web search turned up about 1,780,000 citations in mid-2007. Users who have tried both IFPUG and COSMIC function points report that the COSMIC method is slightly quicker than the IFPUG method, although not as quick as either the "function point light" method or the NESMA high-speed method.

DeMarco "Bang" Function Points

A.J. Albrecht and his colleagues were not the only researchers who were attempting to go beyond the imperfect lines-of-code metric and move

toward functional metrics. In 1982 the well-known software author and researcher, Tom DeMarco, published a description of a different kind of functional metric that he initially termed the "bang metric." This unusual name was derived from the vernacular phase "getting more bang for the buck." A less jocular name for the metric would be the "DeMarco functional metric."

Although DeMarco and Albrecht were acquainted with each other and their metrics were aimed at the same problem, the bang metric and the function point metric are somewhat different in form and substance. (The two appear to be totally independent inventions, incidentally.) DeMarco's consulting had often taken him into the domain of systems software and some of the more complex forms of software engineering, rather than pure MIS projects. His bang metric was the first attempt to apply functional metrics to the domain of systems and scientific software. In the metric, the basic elements to be counted are the following:

- Functional primitives
- Modified functional primitives
- Data elements
- Input data elements
- Output data elements
- Stored data elements
- Objects (also termed "entities")
- Relationships
- States in a state transition model of the application
- Transitions in a state transition model
- Data tokens
- Relationships involving retained data models

As can be seen, the DeMarco bang metric is a considerable superset of the Albrecht function point metric, and it contains such elements as data tokens and state transitions that are normally associated with the more complex forms of systems software such as operating systems and telecommunication systems.

The full set of things that can be counted by using the bang metric is of imposing if not intimidating length. However, DeMarco has pointed out that applications can be conveniently segregated into those that are "function strong" and those that are "data strong." That is, most applications will emphasize either functionality or files and data.

It is not impossible to relate the bang metric to function points. Since DeMarco has stated that subsets are acceptable, it is possible to select an exact match of bang and function point parameters. However, complexity adjustments of the two methods would still be different because the IFPUG adjustments are rule-driven and the DeMarco method is largely subjective.

Although the DeMarco bang metric is technically interesting and can lead to valuable insights when utilized, it fell far behind the Albrecht function point metric in terms of the numbers of users and practitioners once IBM began to offer function point courses as part of its data processing education curriculum. The metric also fell behind in convenience when numerous software packages that could aid in the calculation of IBM's function point metrics began to be marketed while the bang metric still required manual methods.

Although there are no controlled studies comparing DeMarco function points with IFPUG or other flavors, it would seem that the factors included in the DeMarco function point would generate totals perhaps 10 percent to 15 percent larger than IFPUG function points.

In one other way, the DeMarco metric fell behind in the ability to evolve when IFPUG was formed. There is no equivalent for bang metrics to the IFPUG Counting Practices Committee, which serves to both enhance function points and provide standard definitions and examples. A web search in mid-2007 indicated only 877 citations for the DeMarco Bang metric, which indicates that the metric is losing ground compared to many other functional metrics.

Engineering Function Points

The topic of engineering function points was described by Donald Umholtz and Arthur Leitgeb circa 1994. The engineering function point method was aimed at engineering and scientific software.

The engineering function point itself is a blend of function point concepts and the older "software science" metric of the late Dr. Maurice Halstead of Purdue University. The software science contribution was a method of enumerating algorithms by counting operators and operands. For example, in the simple algorithm "A + B = C" there are two operators and three operands.

The function point contribution included system characteristics, but for scientific software the original function point set was replaced by a variant that included

- Communications
- Memory limitations
- Man-machine interface

- Configuration options
- Network transaction rates
- Operator data entity
- Processing of information variables
- Required performance
- Complex processing
- Software reusability: previously written
- Software reusability: for use by future applications
- Real-time updates
- Multiple development or testing sites

There were also changes to other function point elements, such as replacing the phrase "logical files" with "tables." The engineering function point method was used experimentally on several government and military projects in the mid-1990s, but seems to have no recent citations. A web search in mid 2007 on the phrase "engineering function points" picked up about a dozen references and citations, but none since 2000.

Earlier in 1987, the well-known software researcher Don Reifer published a description of a metric that was based on the concept of merging the Albrecht function point technique with the older Halstead software science metric. The latter is based on the work of the late Dr. Maurice Halstead of Purdue University.

Like many researchers, Halstead was troubled by the ambiguity and paradoxical nature of "lines of code." His technique was an attempt to resolve the problems by looking at the specific subelements of lines of code. He divided code into two atomic units: the executable or command portion (which he termed "operators"), and the data descriptive portion (which he termed "operands"). The Halstead metric centered on counts of four separate values:

- The total number of unique operators
- The total number of unique operands
- The total quantity of operators in an application
- The total quantity of operands in an application

From those four counts, a number of supporting metrics are derived including:

- The program's vocabulary (sum of unique operators and operands)
- The program's length (sum of total operators and total operands)

There are a number of conceptual and practical difficulties about attempting to merge function points with the Halstead software science technique. From a conceptual standpoint, function points are intended to be independent of the programming language used and capable of being applied early in a project's life cycle, as during requirements and design.

Since the Halstead software science metric is basically only a more sophisticated way of counting lines of code, it appears to be an inappropriate choice for metrics applied early in the life cycle and also to be counter to the essential philosophy of function points as being language-independent.

The practical difficulties lie in the ambiguities and uncertainties of the Halstead software science metric itself. An attempt by ITT statisticians in 1981 to replicate some of the published findings associated with the Halstead software science metric uncovered anomalies in the fundamental data and a number of questionable assertions. The final conclusion was that the Halstead software science metric was so intrinsically ambiguous and studies using it were so poorly constructed and controlled that the results were useless for serious economic study purposes.

There are also a few problems of a historical nature with the fundamental assertions of the software science metric. For example, the software science literature has made the correct assertion that there is a strong relation between the length and vocabulary of a program.

That is, large systems will use a richer set of operator and operand constructs than small programs. Although that observation is correct, it had actually been noted in 1935 by the linguist George Zipf for natural languages such as English and Mandarin Chinese. Indeed, Zipf's law on the relation of vocabulary and length covers the topic. As it happens, there appears to be a constant relation between length and vocabulary that is true for all natural languages, and no doubt for all programming languages as well. In fact, it would be true even if the language consisted of random characters divided into words or random lengths!

Unfortunately, few of the software science articles and reports build on those findings from conventional linguistics, and it is fair to say that the whole software science concept suffers from a tendency to be unfamiliar with conventional linguistics, even though the two domains are covering the same grounds.

For engineering and systems software projects, the engineering function point seems to generate totals perhaps 20 percent larger than standard IFPUG function points. Because the engineering method was aimed specifically at systems and embedded software, there are no examples of side-by-side usage for information system projects.

Feature Points

A.J. Albrecht is of course identified with the invention of function point metrics in the mid-1970s. Not many people know that Albrecht also participated in the development of feature point metrics circa 1986 after he left IBM and was working at Software Productivity Research.

Feature points were developed primarily to solve a psychological problem. The systems and embedded software world had a fixed idea that function points "only worked for IT projects." This idea was due to the historical fact that IT projects happened to be the first kinds of applications that used function point metrics. Albrecht himself, an electrical engineer by training, had always envisioned function points as being suitable for any kind of software.

Because AT&T, ITT, Siemens, and other telecommunications companies use the word "feature" to define the various components and services provided by telecommunications systems, a change in terminology from "function points" to "feature points" was made primarily to match the terminology of the telecommunications companies.

Another aspect of systems and embedded software is that such applications often contain significant numbers of algorithms or mathematical formulae. Because algorithms are so common in the systems software world, it seemed appropriate to include counts of algorithms in the feature point metric.

The hope was that feature points would open the technical community to the value of functional metrics. The target for feature points included real-time software such as missile defense systems, systems software such as operating systems, embedded software such as radar navigation packages, communications software such as telephone switching systems, process control software such as refinery drivers, engineering applications such as CAD and CIM, discrete simulations, or mathematical software.

When standard function points are applied to such systems, they, of course, generate counts. It often happens that standard function points and feature points generated exactly the same total in terms of size. But by changing the name to "feature points" and by including algorithms, it was hoped that the feature point metric would break down the psychological barrier that had kept the real-time domain from using functional metrics. From both a psychological and practical vantage point, the harder kinds of systems software seem to require a counting method that is equivalent to function points but is sensitive to the difficulties brought on by high algorithmic complexity.

The SPR feature point metric was a superset of the standard function point metrics. It introduced a new parameter, algorithms, in addition to the five standard function point parameters. The algorithms parameter was assigned a default weight of 3. The feature point method

also reduced the empirical weights for logical data files from standard function points from 10 down to a value of 7 to reflect the somewhat reduced significance of logical files for systems software vis-à-vis information systems.

As can be seen, for applications in which the number of algorithms and logical data files are roughly the same, function points and feature points will generate the same numeric totals. But when there are many more algorithms than files, which is not uncommon with systems software, the feature points will generate a higher total than function points. Conversely, if there are only a few algorithms but many files, which is common with some information systems, feature points will generate a lower total than function points.

When feature points and function points were used on classical MIS projects, the results are often almost identical. For example, one small MIS project totaled 107 function points and 107 feature points. However, when applied to the harder forms of system software, the feature point counts were usually higher. For a PBX telephone switch, the function point total was 1845 but the feature point total was 2300 because of the high algorithmic complexity of the application.

Since feature points are driven by algorithmic complexity, a definition of "algorithm" is appropriate. An *algorithm* is defined as the set of rules that must be completely expressed in order to solve a significant computational problem. For example, both a square root extraction routine and a Julian date conversion routine would be considered algorithms.

Some additional examples of typical algorithms include calendar date routines, square root extraction routines, and overtime pay calculation routines. For feature point counting purposes, an algorithm can be defined in the following terms: "An algorithm is a bounded computational problem which is included within a specific computer program."

Although more than 50 software engineering books that describe and discuss algorithms are in print, it is interesting that there is no available taxonomy for classifying algorithms other than purely ad hoc methods based on what the algorithm might be used for.

The basis for the feature point counts for algorithms was twofold: (1) the number of calculation steps or rules required by the algorithm, and (2) the number of factors or data elements required by the algorithm. To illustrate the terms "rule" and "factor," consider the following example taken from an algorithm that selects activities in a software estimating tool: "If class is equal to 'military' and size is equal to 100 feature points, then independent verification and validation will be performed." The example is a single rule, and it contains two factors: class and size.

There were some supplemental rules for determining what algorithms are countable and significant:

- The algorithm must deal with a solvable problem.
- The algorithm must deal with a bounded problem.
- The algorithm must deal with a definite problem.
- The algorithm must be finite and have an end.
- The algorithm must be precise and have no ambiguity.
- The algorithm must have an input or starting value.
- The algorithm must have output or produce a result.
- The algorithm must be implementable and capable of execution on a computer.
- The algorithm can include or call upon subordinate algorithms.
- The algorithm must be capable of representation via the standard structured programming concepts of sequence, if-then-else, do-while, CASE, etc.

Although the software engineering literature is plentifully supplied with illustrations and examples of algorithms, there is currently no complete catalog of the more common algorithms that occur with high frequency in software applications. Since it is always more useful to have tangible examples of concepts, the following are discussions of some common algorithm types.

Sorting Sorting is one of the earliest forms of algorithm created during the computer era. Although physical sorting of records and files has been carried out for thousands of years, it was not until mechanical tabulating equipment was developed that sorting became anything other than a brute force manual task. With the arrival of electronic computers, the scientific study of sorting began. Sorting itself and the development of ever faster sorting algorithms have been among the triumphs of the software engineering community. Prior to about 1950, sorting methods were primarily simple and ad hoc. During the 1960s, 1970s, and on through to today, whole new families of sorting methods and improved algorithms were developed, including selection sorts, insertion sorts, bubble sorts, quicksort, and radix sorting.

Searching Two of the primary functions of computers in their normal day-to-day business applications are sorting and searching. Here again, physical files have been searched for thousands of years, and techniques for facilitating the storage and retrieval of information long outdate the computer era. However, it was only after the emergence of electronic

computers that the study of searching algorithms entered a rigorous and formal phase. This new research into sorting methods led to the development of binary searches, tree searches, indirect tree searches, radix searches, and many others.

Step-Rate Calculation Functions Under the concept of the graduated income tax, a certain level of taxable income is related to a certain tax rate. Higher incomes pay higher tax rates; lower incomes pay lower tax rates. The same logic of dealing with the dependent relations of two variables is perhaps the commonest general form of algorithm for business software. This logic, termed a "step-rate calculation function," which is used for income tax rates, can also apply to the rates for consuming public utilities such as electricity and water, for salary and performance calculations, and for dividends.

Feedback Loops Feedback loops of various kinds are common algorithms in process control applications, hospital patient monitoring applications, and many forms of embedded software such as that for fuel injection, radar, and navigation. Classic feedback loops are much older than the computer era, of course, and one of the clearest examples is provided by the automatic governors on steam engines. Such governors were normally rotating metal weights whose rotation was driven by escaping steam. As the velocity of the steam increased when pressures went higher, the governors opened more widely and allowed excess steam to escape. This same concept of feedback between two variables is one of the major classes of algorithms for sensor-based applications.

Function Point Calculations It is appropriate to conclude the discussion of representative algorithms with the observation that the calculation sequence for function points is itself an algorithm. Let us consider two practical examples. Suppose you were writing a computer program to calculate function points by using IBM's original 1979 methodology. The calculation sequence would be to multiply the raw data for inputs, outputs, inquiries, and master files by the empirical weights Albrecht derived, thus creating a subtotal of unadjusted function points. You would then multiply the unadjusted subtotal by a user-specified complexity factor to create the final adjusted function point total. This entire calculating sequence would comprise only one algorithm, the "function point calculation algorithm." Since the calculations consist of only five simple multiplications and two additions, the weight for this algorithm can be viewed as minimal and be assigned a feature point weighting value of 1.

Now let us suppose you were writing a computer program to calculate function points by using the current IFPUG methodology version 4.2 circa 2008. The calculation sequence today would be to first multiply

and sum the file and data element references to determine high, low, or medium complexity of the five input parameters. You would then multiply raw data for inputs, outputs, inquiries, data files, and interfaces by the separate values for high, low, and medium complexity to quantify the unadjusted function point total.

Then the 14 influential factors would be summed and multiplied by 0.01 and the constant 0.65 would be added to create the influence multiplier weight. Finally, the unadjusted function points would be multiplied by the influence weight to yield the adjusted function point total. Calculating the function point total is still, of course, a single algorithm, but now the weight today would appropriately be set at 3 to reflect the increased difficulty of the calculation sequence.

Additional considerations in the domain of algorithms include whether the algorithm in question lends itself to sequential or parallel processing. In addition, it is desirable to consider these aspects of algorithms:

- Uniqueness
- Correctness
- Computational complexity
- Length
- Performance
- Sensitivity
- Selectivity

The research centering around the practical applications of feature points has opened up some interesting and potentially significant gaps in the software engineering literature. Notably, there seems to be no standard taxonomy for classifying algorithms, and there is no standard weighting scale for judging their difficulty or complexity.

From discussions with a number of hi-tech companies that experimented with the feature point method, the most common question asked concerns the level of granularity of the algorithms to be counted. If the basic concept is held in mind that an "algorithm" should be bounded and complete and should perform a fairly significant business or technical function, then some practical illustrations of algorithms can be given. For example:

- In telephone switching systems, call routing is an appropriate algorithmic topic.
- In PC operating systems, floppy disk formatting is an example of an algorithm.

- In payroll programs, the calculations for hourly, exempt, managerial, and contractor pay are examples of normal algorithms.
- In process control applications, pressure monitoring and feedback are examples.

To give a somewhat more detailed example of typical algorithms, consider the kinds of algorithms in a basic software cost estimating tool:

Examples of Algorithms in a Software Estimating Program

Algorithm	Algorithm Weight
1. Defect potential prediction	3
2. Defect removal prediction	2
3. Function point calculation	3
4. Source code size prediction	2
5. Backfire function point prediction	1
6. Document size prediction	3
7. Test case and test run prediction	3
8. Reliability prediction	2
9. Paid overtime impact on project	1
10. Unpaid overtime impact on project	2
11. Development staff, effort, and schedule prediction	4
12. Activity schedule overlap prediction	4
13. Annual maintenance effort and staff prediction	3
14. Annual enhancement effort and staff prediction	3
15. Overall aggregation of project effort and costs	2
16. Overall calculation of project schedule	2
17. Effort and cost normalization	1
18. International currency conversion	1
19. Inflation rate calculation	1
20. Normalization of data to selected base metric	1

As can be seen, the level of granularity of typical algorithms is reasonably fine but not excessively so. (From examining the actual source code of the project used to provide the data, the algorithms weighted 1 all took fewer than 25 statements in C to implement, the level-2 weighted algorithms usually took less than 50 C statements, the level-3 weighted algorithms usually took fewer than 100 C statements, and so on.)

Feature points did attract interest in the real-time and systems software domain and therefore aided in removing the psychological barrier for using functional metrics with real-time software. When IFPUG began to produce examples and counting rules in the early 1990s for

systems and embedded software, the reason for using feature points gradually disappeared. During the period between about 1986 and 1992, perhaps 150 projects were measured using the feature point method. But after IFPUG began to circulate guidelines and examples for counting function points with systems and real-time software, usage of feature points essentially stopped.

When the first edition of this book was published in 1991, standard IFPUG function points had been used very seldom for real-time and embedded software. By the time of the second edition in 1996, function points were starting to expand in all software domains. Now in 2008 with the third edition, function points are the dominant software metric in much of the world and are used for information systems, real-time applications, weapons systems, embedded software, medical instruments, and essentially every kind of software. In fact, the only form of software where functional metrics have not yet been utilized is that of computer games. The probable reason is that the game industry is not currently interested in topics that function points support. Function point metrics would work for games too, if anyone chooses to use them in that domain.

The standard IFPUG function point metric and COSMIC function points actually work very well for real-time and embedded software. However, it is necessary to expand some of the definitions of "inputs" and "outputs" to encompass things such as sensor-based information, hardware interrupt signals, voltage changes, and so forth.

Standard IFPUG function points have been successfully applied to military software such as the Tomahawk cruise missile, the software on board Aegis-class naval vessels, fuel injection systems in Ford automobiles, software embedded in medical instruments such as CAT scan equipment, both public and private telephone switching systems, and computer operating systems such as IBM's MVS and Microsoft's Windows.

A web search using the keywords "feature points" turned up an astonishing 151,000,000 citations. However, upon looking at the first 50 or so, only five of these citations actually referred to the software "feature points" discussed in this book. The term "feature point" is also common in optics, projective geometry, and other sciences and these uses have millions of web citations.

Function Points Light (FP Lite™)

The function point variant known as "function point light" or FP Lite was developed by David Herron of the David Consulting Group. David is himself an IFPUG certified function point counter and indeed a former officer and committee member of the IFPUG organization.

The FP Lite approach is intended to come close to matching the formal IFPUG counting method in terms of sizing, but to be somewhat quicker and easier to calculate. The main difference between the FP Lite method

and standard function points is that the "light" method uses average values for the five main function point parameters, rather than dividing them into simple, average, and complex categories. This approach speeds up the calculation speed. (Another attempt at improving calculation speed is that of "unadjusted function points," which will be discussed later in this chapter.)

Herron has commented that dealing with the complexity adjustments usually takes more time than anything else. Of course, being quick is irrelevant if accuracy poor, but the FP Lite approach strikes a good balance between speed and accuracy. Further, the Herron method can also be done earlier and with less documentation than normal function point counts.

A study of 30 enhancement projects performed by David Herron reported an average difference of about 8 percent between the FP Lite approach and standard function points, with a maximum difference of about 24 percent. Interestingly, the larger difference was for the smallest project. For projects larger than about 150 function points, the average difference between the two methods was only about 4 percent.

A second and larger study of 95 projects showed a difference of about 10 percent between the FP Lite approach and standard IFPUG function points. Since the accuracy of standard function point counts by certified counters has a variance of about 3 percent, it would seem that the FP Lite method is worthy of serious consideration.

The time required to count function points with the FP Lite method varied from project to project, but generally required only 50 percent as much time as the standard IFPUG method. Interestingly, the time savings increased as application size increased. This is a very promising result, since the time required to count large projects has been a significant barrier to the adoption of function point metrics. If you project the results to date, the savings will increase as function point totals grow larger.

As of 2008 the FP Lite approach is promising, but not yet endorsed by IFPUG or supported by automated function point counting tools.

Full Function Points

The variation called "full function points" was developed by Alain Abran and his colleagues at the University of Quebec in Montreal circa 1997. As with many of the other variations, the motive for developing full function points was the perception that normal IFPUG function points were not suitable for real-time and systems software. The full function point approach expands on the set of 14 influential factors used by IFPUG, and also expands on the kinds of transactions included.

An interesting paper in 1999 by Serge Oligny and Alain Abran provided a useful comparison of the methods of operations of IFPUG and full function point metrics. Full function points are described as a superset of the IFPUG function point metrics, with extensions in

terms of influential attributes, file types, transaction or process types, and the inclusion of a sub-process object type that was not present in IFPUG. The full function points metric also expands on the IFPUG definition of "user" to include a broader range such as other applications or even machines.

One interesting point made by Fred Bootsma of Nortel, a full function point user, is that full function points can be used to measure projects that don't actually add or subtract function points from an application. It often happens that changes are made to applications that modify report formats or change file structures, but don't add or subtract input and output elements. These changes are termed "churn" rather than "creep" because the IFPUG function point totals are not effects. However, with full function points these changes do affect the function point counts.

When used with systems and real-time software, the full function points metric seems to generate counts that run between 40 and 60 percent larger than IFPUG function points. This is one of the largest variations of any of the function point metrics.

Because many of the full function point concepts migrated into COSMIC function points, the number of pure full function point users circa 2008 is probably in decline as COSMIC usage ramps up. A web search using the phrase "full function points" in mid-2007 showed 151,000,000 citations. However, many of these citations were not actually about full function points but rather about some of the other forms of functional metric as well.

IFPUG Function Points

By 1986, several hundred companies, many but not all of them clients of IBM, had been using function points. The association of IBM's commercial clients, GUIDE, had established a working group on function points in 1983, but by 1986 a critical mass of function point users had occurred. It was then decided to form a new nonprofit organization devoted exclusively to the utilization of function points and the propagation of data derived from function point studies.

The new organization was named the International Function Point Users Group. Because of the length of the name, the organization is more commonly identified by the abbreviation, IFPUG, as it quickly came to be called.

IFPUG originated in Toronto, Canada, but moved its offices to the United States in 1988. IFPUG has evolved from its informal beginnings into a major new association concerned with every aspect of software measurement: productivity, quality, complexity, and even sociological implications. As this third edition is written in 2008, IFPUG has more

than 3,000 individual members and affiliate organizations in more than 20 countries. IFPUG is the largest software metrics association in the world.

The IFPUG counting practices committee has become a de facto standards group for normalizing the way function points are counted, and it has done much to resolve the variations in terminology and even the misconceptions that naturally occur when a metric gains wide international use.

The version of function points that IFPUG took over from IBM was the last IBM revision in 1984. In 1984, Albrecht and IBM published a major revision of the function point method that significantly revised the technique. It is the basis of the current IFPUG function point methodology, although there have been several updates since.

Also in 1984, IBM started to include courses in function points as part of its data processing education curriculum, which created a quantum leap in overall utilization of the technique. IFPUG took over the training in function point metrics too. Additionally, in about 1988, IFPUG began to administer certification exams for those wishing to count function points. The combination of training and certification leveled the playing field, and achieved an excellent commonality in terms of results. Certified counters with formal training tend to produce the same function point totals within about 3 percent accuracy. This is better accuracy than many kinds of accounting, and far better accuracy than some other complex calculations such as filling out federal income tax returns.

In the 1984 revision, the impact of complexity was broadened so that the range increased from plus or minus 25 percent to about 250 percent. To reduce the subjectivity of dealing with complexity, the factors that caused complexity to be higher or lower than normal were specifically enumerated and guidelines for their interpretation were issued. Instead of merely counting the number of inputs, outputs, master files, and inquiries as in the original function point methodology, the current methodology requires that complexity be ranked as low, average, or high. In addition, a new parameter, interface files, has been added. These changes are shown in Table 2-15.

TABLE 2-15 The Initial IFPUG Version of the Function Point Metric

Significant Parameter	Low Complexity	Medium Complexity	High Complexity
External input	$\times 3$	$\times 4$	$\times 6$
External output	$\times 4$	$\times 5$	$\times 7$
Logical internal file	$\times 7$	$\times 10$	$\times 15$
External interface file	$\times 5$	$\times 7$	$\times 10$
External inquiry	$\times 3$	$\times 4$	$\times 6$

Each major feature such as external inputs must be evaluated separately for complexity. In order to make the complexity evaluation less subjective, a matrix was developed for each feature that considers the number of file types, record types, and/or data element types. For example, Table 2-16 shows a matrix dealing with the complexity of external inputs.

The treatment of complexity is still subjective, of course, but it is now supported by guidelines for interpretation. Examples of inputs include data screens filled out by users, magnetic tapes or floppy disks, sensor inputs, and light-pen or mouse-based inputs. The adjustment table for external outputs is shown in Table 2-17, and it is similar to the input adjustment table.

Examples of outputs include output data screens, printed reports, floppy disk files, sets of checks, or printed invoices. The adjustment table for logical internal file types is shown in Table 2-18, and it follows the same pattern.

TABLE 2-16 Adjustment Weights for External Inputs

File Types Referenced	Data Element Types		
	1–4	5–15	⇒ 16
0–1	Low	Low	Average
2	Low	Average	High
⇒ 3	Average	High	High

TABLE 2-17 Adjustment Weights for External Outputs

File Types Referenced	Data Element Types		
	1–5	6–19	⇒ 20
0–1	Low	Low	Average
2–3	Low	Average	High
⇒ 4	Average	High	High

TABLE 2-18 Adjustment Weights for Logical Internal Files

File Types Referenced	Data Element Types		
	1–19	20–50	⇒ 51
0–1	Low	Low	Average
2–5	Low	Average	High
⇒ 6	Average	High	High

Examples of logical internal files include floppy disk files, magnetic tape files, flat files in a personal computer database, a leg in a hierarchical database such as IBM's IMS, a table in a relational database such as DB2, and a path through a net in a network-oriented database. Table 2-19 shows the adjustments for external interfaces, and it follows the same pattern as the others.

Examples of an interface include a shared database and a logical file addressable from or to some other application. The last parameter is inquiries, which is similar to the others but is divided into the two sub-elements of the input portion and the output portion, as shown in Tables 2-20 and 2-21.

Examples of inquiries include user inquiry without updating a file, help messages, and selection messages. A typical inquiry might be illustrated by an airline reservation query along the lines of "What Delta flights leave Boston for Atlanta after 3:00 p.m.?" as an input portion. The response or output portion might be something like "Flight 202 at 4:15 p.m."

TABLE 2-19 Adjustment Weights for Interface Files

File Types Referenced	Data Element Types		
	1–19	20–50	⇒ 51
0–1	Low	Low	Average
2–5	Low	Average	High
⇒ 6	Average	High	High

TABLE 2-20 Adjustment Weights for Inquiry Input Portions

File Types Referenced	Data Element Types		
	1–4	5–15	⇒ 16
0–1	Low	Low	Average
2	Low	Average	High
⇒ 3	Average	High	High

TABLE 2-21 Adjustment Weights for Inquiry Output Portions

File Types Referenced	Data Element Types		
	1–5	6–19	⇒ 20
0–1	Low	Low	Average
2–3	Low	Average	High
⇒ 4	Average	High	High

The initial IFPUG version of the function point counting rules introduced yet another change that is minor in technical significance but important from a human factors standpoint. It adopted a set of standard full-length descriptions and standard abbreviations for the factors and variables used in function point calculations. Since examples and discussions of function points by experienced users tend to make use of those abbreviations, novice users of function points should master them as soon as possible.

Inputs, outputs, inquiries, logical files, and interfaces obviously comprise many different kinds of things, so the generic word "type" was added to the 1984 full nomenclature. However, since it is necessary to use some of the terms many times when giving examples, the revision introduced standard abbreviations as well. The changes in nomenclature and abbreviations are shown here:

Original Nomenclature	Revised	Abbreviations
Inputs	External input type	IT
Outputs	External output type	OT
Logical files	Logical internal file type	FT
Interface	External interface file type	EI
Inquiries	External inquiry type	QT

The terminology of function points has tended to change frequently, although the fundamental concepts have been remarkably consistent. Unfortunately, novices tend to be put off by the complexity of the terms and by the many arcane abbreviations, even though the terms are no more difficult, for example, than those associated with stocks or corporate finance.

Some of the terms and abbreviations that must be understood circa 2008 include the following, in alphabetical order:

A	Average (used for complexity)
DET	Data element type
EI	External input
EIF	External interface file
EO	External output
EQ	External query
FPA	Function point analysis
FTR	File types referenced
GSC	General system characteristics
H	High (used for complexity)
ILF	Internal logical file

L	Low (used for complexity)
RET	Record element type
TDI	Total degree of influence
UFP	Unadjusted function points
VAF	Value adjustment function points

Because much of the literature on function point analysis uses the abbreviations without defining them, the results may be confusing to those unfamiliar with the terminology and the abbreviations that go with them.

The 14 Influential or Value Adjustment Factors Another significant change in the IFPUG version was an expansion in the range of complexity adjustments and the rigor with which the adjustments are carried out. Recall that, in the original version of function points, complexity was a purely subjective adjustment with a range that spanned ±25 percent. In the IFPUG revision, it is derived from the overall impact of 14 influential factors, and the total range of adjustment of the complexity multiplier runs from 0.65 to 1.35. In recent years the influential factors have been called *value adjustments*.

The 14 influential complexity factors are evaluated on a scale of 1 to 5 (with a 0 being used to eliminate factors that are not present at all). The 14 influential complexity factors are assigned shorthand identifiers that range from C1 to C14 for accuracy and convenience when referencing them.

C1	Data communications
C2	Distributed functions
C3	Performance objectives
C4	Heavily used configuration
C5	Transaction rate
C6	On-line data entry
C7	End-user efficiency
C8	On-line update
C9	Complex processing
C10	Reusability
C11	Installation ease
C12	Operational ease
C13	Multiple sites
C14	Facilitate change

In considering the weights of the 14 influential factors, the general guidelines are these: score a 0 if the factor has no impact at all on the application; score a 5 if the factor has a strong and pervasive impact; score a 2, 3, 4, or some intervening decimal value such as 2.5 if the impact is something in between. This is, of course, still subjective, but the subjectivity is now spread over 14 different factors.

Various tutorial materials provide the following general suggestions for assigning weights to the 14 influential factors:

0 Factor not present or without influence
1 Insignificant influence
2 Moderate influence
3 Average influence
4 Significant influence
5 Strong influence

Let us consider the maximum ranges of the 14 influential factors and look in detail at the scoring recommendations for the first two:

C1 Data Communication Data communication implies that data and/or control information would be sent or received over communication facilities. This factor would be scored as follows:

0 Batch applications
1 Remote printing or data entry
2 Remote printing and data entry
3 A teleprocessing front end to the application
4 Applications with significant teleprocessing
5 Applications that are dominantly teleprocessing

C2 Distributed Functions Distributed functions are concerned with whether an application is monolithic and operates on a single contiguous processor or is distributed among a variety of processors. The scoring for this factor is as follows:

0 Pure monolithic applications
1 Applications that prepare data for other components
2 Applications distributed over a few components
3 Applications distributed over more components
4 Applications distributed over many components
5 Applications dynamically performed on many components

C3 Performance Objectives Performance objectives are scored as 0 if no special performance criteria are stated by the users of the application and scored as 5 if the users insist on very stringent performance targets that require considerable effort to achieve.

C4 Heavily Used Configuration Heavily used configuration is scored as 0 if the application has no special usage constraints and as 5 if anticipated usage requires special effort to achieve.

C5 Transaction Rate Transaction rate is scored 0 if the volume of transactions is not significant and 5 if the volume of transactions is high enough to stress the application and require special effort to achieve desired throughputs.

C6 On-Line Data Entry On-line data entry is scored 0 if none or fewer than 15 percent of the transactions are interactive and 5 if all or more than 50 percent of the transactions are interactive.

C7 End-User Efficiency Design for end-user efficiency is scored 0 if there are no end users or there are no special requirements for end users and 5 if the stated requirements for end-user efficiency are stringent enough to require special effort to achieve them.

C8 On-Line Update On-line update is scored 0 if there is none and 5 if online updates are both mandatory and especially difficult, perhaps because of the need to back up or protect data against accidental change.

C9 Complex Processing Complex processing is scored 0 if there is none and 5 in cases requiring extensive logical decisions, complicated mathematics, tricky exception processing, or elaborate security schemes.

C10 Reusability Reusability is scored 0 if the functionality is planned to stay local to the current application and 5 if much of the functionality and the project deliverables are intended for widespread utilization by other applications.

C11 Installation Ease Installation ease is scored 0 if this factor is insignificant and 5 if installation is both important and so stringent that it requires special effort to accomplish a satisfactory installation.

C12 Operational Ease Operational ease is scored 0 if this factor is insignificant and 5 if operational ease of use is so important that it requires special effort to achieve it.

C13 Multiple Sites Multiple sites is scored 0 if there is only one planned using location and 5 if the project and its deliverables are intended for many diverse locations.

C14 Facilitate Change Facilitate change is scored 0 if change does not occur, and 5 if the application is developed specifically to allow end users to make rapid changes to control data or tables that they maintain with the aid of the application.

Using the 14 Influential Factors for Complexity Adjustment When all of the 14 factors have been considered and scores assigned individually, the sum of the factors is converted into a final complexity adjustment by the following procedure:

1. Multiply the sum of the factors by 0.01 to convert the sum to a decimal value.
2. Add a constant of 0.65 to the decimal value to create a complexity multiplier.
3. Multiply the unadjusted function point total by the complexity multiplier to create the final adjusted function point total.

It can be seen that the 14 influential factors yield a multiplier that has a range of from 0.65 to 1.35. If none of the factors were present at all, the sum would be 0, so only the constant of 0.65 is used as the multiplier. If all 14 factors were strongly present, their sum would be 70. Using the procedure of 70*0.01 + 0.65 = 1.35, the final multiplier in this case is 1.35.

Here is an example of the calculation sequence used to derive a function point total by using the early IFPUG method. Assume an average project with 10 inputs, 10 outputs, 10 inquiries, 1 data file, and 1 interface. Assume average complexity for the five primary factors and a range of weights from 0 to 5 for the 14 influential factors, so that the sum of the 14 influential factors totals to 40 influence points.

Basic Counts	Elements		Weights		Results
10	Inputs	×	4	=	40
10	Outputs	×	5	=	50
10	Inquiries	×	4	=	40
1	Logical file	×	10	=	10
1	Interface	×	7	=	7
Unadjusted Total					147

The influential factor calculations are as follows:

Data communications	0
Distributed functions	0
Performance objectives	4
Heavily used configuration	3
Transaction rate	3
On-line data entry	4
End-user efficiency	4
On-line update	2
Complex processing	3
Reusability	0
Installation ease	4
Operational ease	4
Multiple sites	5
Facilitate change	4

The sum total of the influential factors is 40. Then,

$$40*0.01 = 0.40 + 0.65 \text{ [constant]} = 1.05 \text{ [complexity multiplier]}$$

The final adjustment is

$$147 \text{ [unadjusted total]}*1.05 \text{ [complexity multiplier]} = 154 \text{ [adjusted function points]}$$

Although the steps with the IFPUG method can be time-consuming, the calculation sequence for producing function points is fairly simple to carry out. Not quite so simple is reaching a clear agreement on the exact number of inputs, outputs, inquiries, data files, and interfaces in real-life projects. This is why formal training and a certification exam are so useful.

When first starting with function points users should be cautioned not to get bogged down in rules and determinations of exact weights. If you think clearly about the application, your counts are likely to be acceptably accurate. In summary form, here are the basic concepts used when counting with function points.

- **Inputs** Inputs are screens or forms through which human users of an application or other programs add new data or update existing data. If an input screen is too large for a single normal display (usually 80 columns by 25 lines) and flows over onto a second screen, the set counts as 1 input. Inputs that require unique processing are what should be considered.

- **Outputs** Outputs are screens or reports which the application produces for human use or for other programs. Note that outputs requiring separate processing are the units to count. In a payroll application, an output function that created, say, 100 checks would still count as one output.
- **Inquiries** Inquiries are screens that allow users to interrogate an application and ask for assistance or information, such as help screens.
- **Data Files** Data files are logical collections of records that the application modifies or updates. A file can be a flat file such as a tape file, one leg of a hierarchical database such as IMS, one table within a relational database, or one path through a CODASYL network database.
- **Interface** Interfaces are files shared with other applications, such as incoming or outgoing tape files, shared databases, and parameter lists.

As can be seen, the IFPUG function point methodology is substantially more rigorous than the original implementation, and the rigor has added considerably more work before an application's function points can be totaled. The effort involved to count the function points of a large system by using the current IFPUG methodology amounts to several days, sometimes spread out over several weeks. Counting function points by using the current IFPUG method also requires trained and certified function point specialists to ensure consistency of the counts.

As of 2008 there are probably around 1,500 certified function point counters in the United States, and another 1,000 in Europe, Asia, and South America. Some are independent consultants and some work for larger companies.

Because the IFPUG method is the oldest form of function point counting, it has been in continuous use since IFPUG was formed. By 2008, probably more than 50,000 projects have been counted using the IFPUG method in the United States alone, and another 30,000 projects in Europe, the Pacific Rim, and South America. Of course, many of these projects and their counts are proprietary, but the public data available from the International Software Benchmarking Standards Group (ISBSG) tops 4,000 projects in 2008 and other public sources such as this book and similar books summarize the results from another 15,000 or so projects. With the exception of "backfiring," the IFPUG method has been used for more projects than all other function point variations added together.

A web search in mid-2007 on the phrase "IFPUG function points" turned up about 46,100 citations, with many of them including tutorial information.

ISO Standards on Functional Metrics

The International Organization for Standardization (commonly known as ISO) has published hundreds of standards on hundreds of topics. The value of having the ISO review and accept a particular measure or a particular kind of function point counting technique is beneficial because it does ensure that the method is effective and is fully defined.

However, from time to time there may be more than one metric that can be demonstrated to be effective and fully defined, so there may be more than one ISO standard. This situation has occurred with function points, and as of 2008 there are four varieties of function point metric certified by the ISO organization: COSMIC function points, IFPUG function points, Mark II function points, and Netherlands (NESMA) function points, in alphabetical order.

For the many methods that are not certified by ISO, such as backfiring, pattern-matching, use case points, "function points light," and 3D function points, the lack of certification does not imply that the methods don't work or are ineffective. The lack of certification only means that these other methods have not applied for certification as of 2008. In the case of backfiring, for example, there is no active user group or owner of the approach and therefore no organization has enough control over backfiring to apply for ISO certification. In the case of the FP Lite approach, the method is so new and experimental that it is still premature to consider applying for ISO certification.

Receiving ISO certification indicates that a method is mature enough and is defined well enough so that the technology can be taught and used without too much concern about random or capricious changes occurring.

Mark II Function Points

In January 1988, Charles Symons, at the time employed by Nolan, Norton & Company in London, published a description of his Mark II Function Point metric in the *IEEE Transactions on Software Engineering*. Although Symons' work had started in the early 1980s and was announced in England in 1983, it was not well known in the United States prior to the IEEE publication in 1988.

Symons had been carrying out some function point studies at Xerox and other companies in the United Kingdom, and he had formed the opinion that the 1984 IBM method might perhaps need to be modified for systems software. The essence of Symons' concerns were four:

- He wanted to reduce the subjectivity in dealing with files by measuring entities and relations among entities.
- He wanted to modify the function point approach so that it would create the same numeric totals regardless of whether an application was implemented as a single system or as a set of related sub-systems.

- He wanted to change the fundamental rationale for function points away from value to users and switch it to the effort required to produce the functionality.
- He felt that the 14 influential factors cited by Albrecht and IBM were insufficient, and so he added six factors.

When carried to completion, Symons' modifications of the basic function point methodology were sufficiently different from IBM's to merit the "Mark II" nomenclature. When counting the same application, the resulting function point totals differ between the IBM and Mark II by sometimes more than 30 percent, with the Mark II technique usually generating the larger totals.

(It is a curious historical fact that for many metrics the British versions are larger than the American versions. For example, the British ton is 2,240 pounds while the American ton is 2,000 pounds. The British Imperial gallon is larger than the American gallon, and the British Mark II function point produces larger totals than the American IFPUG function point.)

Viewed objectively, Symons' four concerns are not equal in their impact, and his modifications have pros and cons. His first concern, introducing entities and relationships, does add an interesting new dimension of rigor to function point counting, and his suggestion is starting to find widespread acceptance. His second concern, wanting total function point counts to stay constant regardless of whether an application is monolithic or distributed, is debatable and questionable.

For example, in a construction project involving physical objects such as living space, there will be very significant differences in providing 1,500 ft. of housing in the form of ten single-family homes or in the form of ten apartments in a single large building. It is obvious to architects and contractors that very different quantities of lumber, cement, roofing, and so on, will be required depending upon which construction choice is made.

In a parallel fashion, an application developed as an integrated, monolithic system will certainly have different needs and requirements than if the same functionality is implemented in ten independent programs. At the very least, the interfaces will be quite different. Therefore, attempting to generate a constant function point count regardless of whether an application is monolithic or distributed seems hazardous.

Symon's third concern, wishing to change the basis of the function point method from "user value" to "development effort," appears to be a step in a retrograde direction. To continue with the parallel of the building trade, the value of a home is only partly attributable to the construction costs. The other aspects of home value deal with the

architectural and design features of the home, the charm of the site, the value of surrounding homes, convenience of location, and many other topics.

In Albrecht's original concept, function points were analogous to the work of an architect in home construction: The architect works with the clients on the features and design that satisfy the clients' needs. In other words, the architect works with the client on the functionality required. What might result is an architectural plan for a home of say 3,000 square feet. Then contractors will be invited to bid on construction. However, the house will still be 3,000 square feet no matter what the construction costs are.

In Symons' Mark II concept, function points become analogous to the work of a contractor in home construction: The contractor brings in equipment and workers and constructs the home. In other words, the contractor builds the functionality required. Further, since different contractors will probably have different cost structures, it is better to know the square feet first and have all the bids against a constant size.

Albrecht's original concept of function points appears to be preferable to the Mark II concept: Function points measure the size of the features of an application that users care about. The costs, schedules, and efficiency with which those features are built is a separate topic and should not be mixed up with the size of the features themselves.

Symons' fourth modification, adding to IBM's 14 supplemental factors, is in keeping with his overall philosophy of switching function points from a metric dealing with value and size to a metric also including effort. The Mark II factors added by Symons are

- Software with major systems software interfaces
- Software with very high security considerations
- Software providing direct access for third parties
- Software with special documentation requirements
- Software needing special user training
- Software needing special hardware

The additional factors considered by Symons are in real life often significant. However, the Symons factors tend to affect the cost of construction rather than the intrinsic size of the application itself.

The disadvantage in the context of function points is twofold: (1) If additional factors that influence a project are considered thoroughly, 6 is insufficient and more than 100 such factors might be added; and (2) whenever such factors are added as complexity adjustments, they typically drive up the function point totals compared to the IFPUG standard. As more factors are added, the function point total will tend

to creep up over time for reasons that appear unjustified under the assumptions of the original IBM and later IFPUG assertions.

As one of the older function point variations, the Mark II approach probably has as many as 10,000 projects that have been measured with this metric, the majority in the United Kingdom. A web search using the phrase "Mark II function points" turned up an astonishing 14,800,000 citations. However, only about 100 of these were actual citations to the Mark II function point. The others were citations where "mark II" was used generically, such as the mark II version of a camera and the mark II versions of tools and electronic equipment.

Micro Function Points

When function points are calculated using the normal IFPUG rules, the complexity adjustments have a lower boundary. The effect of these lower limits is that about 15 function points is the lowest value that can be calculated by normal counting.

For maintenance and enhancement work, a great many changes are much smaller than 15 function points. For example, sometimes a bug repair will involve only changing one line of code. At first glance the lower limit on function point counts may seem to be a minor inconvenience. After all, changing a few lines of code, equivalent to a fraction of one function point, may take less than 30 minutes. Who cares whether or not function points can be used?

Individually, there is not a major need to count very small numbers of function points. However, when large applications such as ERP packages or Microsoft Vista are first released, there may be as many as 100,000 bugs or defects reported in the first year alone. The cumulative costs of dealing with very large numbers of very small changes can become a significant expense element of overall software budgets.

The phrase "micro function points" simply refers to function points where the counts have three decimal places of precision rather than integer values. As of 2008, there are no formal rules for counting fractional function points but the "backfiring" method is a reasonably effective surrogate.

Let us assume that you are interested in knowing the function point equivalent of adding ten lines of code to existing applications written in five different languages: Assembler, COBOL, PL/I, Java, and SMALLTALK.

If you have access to information on the "level" of programming languages or the number of statements per function point, the calculations are simple. Assuming that you were using the Software Productivity Research table of programming languages, the number of logical code statements per function point for these five languages are

Language	Logical Code Statements per Function Point
Assembler	320
COBOL	107
PL/I	80
Java	53
SMALLTALK	18

Since this thought experiment consists of adding ten lines of code in each language, then the function point equivalent can be calculated by dividing ten by the number of statements per function point. The results are

Language	Micro Function Points
Assembler	0.031
COBOL	0.094
PL/I	0.125
Java	0.188
SMALLTALK	0.563

Individually, micro function points are small, but suppose you are trying to estimate the annual maintenance costs of large systems. Suppose you are going to face 10,000 such changes in each language in the coming year. Now the results start to be significant:

Language	Annual Total of Micro Function Points	Cost per Function Point	Annual Costs
Assembler	310	$500	$155,000
COBOL	940	$450	$423,000
PL/I	1,250	$400	$500,000
Java	1,880	$350	$658,000
SMALLTALK	5,630	$300	$1,689,000

The bottom line is that small maintenance changes, individually, may not be very expensive. But if enough changes occur in a year, the cumulative costs need to be estimated, measured, and included in corporate warranty and rework costs. Therefore, large corporations that develop large applications need to have an effective metric that works for small updates. Micro function points are a step in that direction. As of 2008 the micro function point approach is experimental and has no web citations.

Since micro function points cover the range from a fraction of 1 function point up to the lower boundary of normal function points, which is about 15, it would be possible to develop actual counting rules for micro function points. Because we are dealing primarily with small changes

to existing software, what would seem to be a reasonable approach for counting micro function points would be to remove the weighting factors for inputs, outputs, inquiries, logical files, and interfaces and just count the number of occurrences of each parameter.

The weighting factors are the cause of the lower boundary of conventional function point counts, so they would have to be removed or reduced to eliminate that barrier.

For complexity adjustments, dropping the 14 influential factors and just using problem complexity, code complexity, and data complexity would also simplify the calculations. This approach would yield counts from a small fraction of 1 function point up to perhaps 20 function points.

It might be asked how this alternate counting method compares to normal IFPUG counts. Since normal IFPUG counts don't cover such small values, there is no direct comparison. Obviously, the simpler method would yield lower counts if used for large applications, but in the area below the cut-off for normal function points that does not matter a great deal.

Netherlands Function Points (NESMA)

The Netherlands Software Metrics Association (NESMA) has been expanding rapidly and carrying out a number of innovative programs. Along with other function point groups, NESMA has been concerned about both the accuracy of function point counts and also about the speed or cost of calculating function point metrics.

The NESMA organization recognizes three styles of function point counts, which they term detailed, estimated, and indicative. The detailed form is similar to normal IFPUG function points with some comparatively minor differences. The estimated function point method is somewhat quicker, and uses constant values for complexity ratings.

The most intriguing form is the indicative method, which is intended for high-speed early function point counts before all details are known. With this method, only the number of logical files is counted and then constant values are used to determine the approximate function point count. The internal logical files (ILF) are multiplied by 35 while the external interface files (EIF) are multiplied by 15. This method can be applied at least a month earlier than the other two, and generates results that often come within 15 percent of the other two. It is also perhaps twice as fast as the detailed counting method.

As of 2008 the various NESMA methods of counting function points have probably examined as many as 3,000 projects, and possibly more. A web search on the keywords "NESMA function points" generated about 16,000 citations, with most of them being relevant. The simple NESMA indicative version lends itself to automation, and is finding its way into various estimating tools such as some of the COCOMO models.

As an organization, the NESMA group is active and energetic and is having a significant impact on function point and metrics technology.

Object Points

The object point method is something of a hybrid that mixes some concepts from function points and some concepts from the object-oriented paradigm. However, and surprisingly, the word "object" is not used in the same context as it is in the object-oriented literature. It is used more as a generic term.

Objects in the object point metric can be screens, reports, and modules. They are counted and weighted in terms of complexity, and then a total is calculated called the *object point value*. The main use of the object-point method seems to be ascertaining productivity levels, and the reported ranges run from 4 object points per month to more than 50 object points per month. Coincidentally, that is similar to the range of standard IFPUG function points, although it is not at all clear if object points and function points are similar in size.

As of 2008 it is not clear how many projects have been measured with object points. The method is not ISO certified, and apparently has no formal user group. A web search on the phrase "object points" generated an astounding 74,900,000 citations, but only about five of the ones reviewed had any relevance. It happens that the same phrase "object points" is used in optics so most of the citations involved two- and three-dimensional ray tracing algorithms.

Pattern-Matching and Function Point Sizing

The pattern-matching approach to function point analysis is a very different way of ascertaining function point totals. The best analogy for the pattern-matching approach is to leave software and consider the well-known Kelley Blue Book for determining the probable selling price for a used automobile.

The Kelley Blue Book lists every model of every automobile sold in the United States. It also shows the + or – cost impacts of factors such as navigation packages, satellite radio systems, electric seat adjustments, and other luxury items. The Kelley Blue Book also has rules for ascertaining the effect of high or low mileage, external damages, geographic areas, and other factors. The data in the Kelley Blue Book comes from monthly statistical analysis of actual automobile sales throughout the United States.

As of 2008 thousands of new applications and thousand of enhancements have been sized and measured using function point analysis. The pattern-matching approach is based on accumulating a large catalog

of measured software projects whose function point totals have been determined by means of normal function point analysis.

To use the pattern-matching approach, rather than carrying out a conventional function point count the application to be sized is compared against the catalog of historical projects and matched against similar projects. The logic is exactly the same as looking up an automobile you are interested in buying in the Kelley Blue Book.

There are two critical topics required for the pattern-matching approach to be effective:

- A large collection of historical data
- A formal taxonomy of software projects to guide the search

Organizations such as the International Software Benchmarking Standards Group (ISBSG), SPR, the David Consulting Group, SEI, and several other organizations have sufficient data to support the pattern-matching approach. If the data from these organizations were consolidated, there would be more than enough information for pattern-matching to become a major sizing and estimating approach.

What has not been fully defined circa 2008 is a formal taxonomy for placing software projects in a logical structure so that pattern-matching can be automated and is easy to use. The taxonomy developed by the author for pattern-matching includes these elements:

- **Function Point Approach** The specific form of function point analysis of interest (COSMIC, IFPUG, Mark II, NESMA, etc.). This guides the search to the collection of data in the appropriate function point form.
- **Project Nature** Whether the project to be sized is new, an enhancement, maintenance, or something else. This guides the search to the appropriate subsection of the overall catalog of projects.
- **Project Scope** Whether the project to be sized is a module, component of a system, stand-alone program, system, enterprise system, etc. This guides the search to the approximate size range of similar applications.
- **Project Class** Whether the project to be sized is to be developed for internal use, for external use by other companies, put onto the web, bundled with hardware, etc. This narrows down the search to similar forms of software.
- **Project Type** Whether the project to be sized is systems software, embedded, real-time, neural net, client-server, multimedia, etc. This also narrows the search to similar forms of software.

If you start with a catalog of 5,000 projects in total, the five taxonomy characteristics discussed earlier will probably narrow down the choices to about 50 projects that are more or less similar to the application that is being sized.

Just as the Kelley Blue Book provides adjustments for factors such as mileage, external damages, and accessories, three adjustment factors are used to narrow the search still further. Three forms of complexity are considered using a scale of 1 to 10 for each. The number 1 indicates minimal complexity while the number 10 indicates maximum complexity.

- **Problem Complexity** This factor considers the difficulty of the problems and algorithms that are going to be used in the application to be sized. The lower scores imply few algorithms and simple calculations. The higher scores imply very complex logical or mathematical topics.
- **Code Complexity** This factor considers the complexity of the control flow and modules in the application that is going to be sized. This factor has the most significance for maintenance and enhancement projects. The lower scores imply few branches and low values for cyclomatic and essential complexity. The higher scores imply very complex branch logic and very complex modules as well.
- **Data Complexity** This factor considers the complexity of the files and data structures that the application to be sized will include and support. The lower scores imply few files and simple data structures. The higher scores imply extremely large numbers of files and very complicated data relationships, such as might be found in enterprise resource planning (ERP) applications.

Once the three complexity factors have been analyzed, the probable number of similar matching projects will usually be in the range of 3 to 10. It is interesting that experiments with the pattern-matching approach indicate that the projects that match all five criteria are usually within 15 percent of the same size. Projects that also match the three complexity factors are usually within about 5 percent of the same size.

Some additional factors are also useful in guiding the search to the appropriate section of the catalog. These additional factors do not affect the size of software applications, but they do affect their costs and schedules. These factors are

- **Country of Development** Because of major differences in work patterns and compensation levels, it is important to know whether a specific application being sized is going to be developed in the United States, China, Canada, Russia, India, England, or wherever.

Since some large projects are developed in more than one country, it is important to know this too.

- **Industry Code** Because of differences in costs structures and compensation levels by industry, it is important to know whether a specific application is being developed by a bank, a manufacturing company, a military service, a commercial software house, or whatever. The Quick Sizer prototype uses the North American Industry Classification Codes (NAIC) published by the U.S. Department of Commerce. This information is important because, for example, banks have higher compensation levels and higher burden or overhead rates than manufacturing companies. Thus an application of 10,000 function points developed by a large investment bank might cost 50 percent more than the same application developed by a mid-sized manufacturing company.
- **City or Geographic Region** There are portions of the United States where costs and compensation levels are much higher than other regions. For example, New York City, San Francisco, and Seattle are all more costly than Little Rock, Albany, and Hartford. The geographic area of a software project can affect costs by as much as 25 percent. There are similar geographic differences in other countries and with other cities. For example, Moscow, Tokyo, Geneva, Paris, Sao Paolo, and London are all very expensive locations for software development work.
- **Date of Release** Because data that is more than a few years old is often viewed as obsolete or unreliable (even if it is still valid) it is important to know the date that software applications were first released to customers. For applications that continue to evolve over many years and have had multiple releases, it is also important to know the date of the most recent release. This factor brings up the point that new releases add functionality and therefore change the function point totals of the historical projects.
- **Methodology** One of the most interesting questions faced by metrics and estimating groups is how various methodological approaches affect software projects. By including a "methodology" topic in the overall taxonomy, it is possible to seek out the appropriate subsections of the overall catalog of projects for those using object-oriented methods, Agile methods, Extreme programming, spiral development, or any known software methodology so long as there is historical data available.

The methodology parameter actually requires decomposition into a number of specific technical areas. It is obviously important to know requirements methodologies, design methodologies, programming languages, use of inspections, forms of testing, tool suites, and several other technical topics of interest.

One issue with the pattern-matching approach is the very large number of combinations. If you calculate the total number of possible patterns from the basic taxonomy and complexity factors, there are about 22,000,000 combinations. When the country, industry, geography, and methodology factors are included, the number of combinations soars above 1,000,000,000. It is of course unlikely and probably impossible for the software industry to ever have historical data on every combination. Indeed some rare combinations do not even occur in real life.

However, once a suitable catalog of projects has been gathered together, it is possible to perform a statistical analysis of the overall results and extract the fundamental sizes associated with each known combination. Then it is possible to develop mathematical approximations for all of the patterns that are not actually included in the historical data collection.

In other words, the pattern-matching approach leads to the creation of a new kind of sizing and estimating tool that uses a formal taxonomy to match the project to be estimated against projects that share the same pattern of taxonomy results.

A prototype pattern-matching tool called "Quick Sizer," developed by the author, has been used experimentally on more than 125 projects with fairly good results. However, when the prototype is used by people unfamiliar with the taxonomy, the results are not as favorable. There is a tendency to exaggerate the problem and data complexity questions, and these exaggerations result in matching projects that are much larger than the application that is being sized. A possible solution to this issue is to display applications of various complexity levels so that users of the approach can look for familiar or similar projects.

The experimental results from pattern-matching indicate that this is the fastest approach yet discovered for sizing, and reasonably accurate as well once the taxonomy is properly understood. For example, using pattern-matching, an ERP application of about 290,000 function points was sized in less than one minute. By comparison, manual function point counting of such a huge application would probably require a team of certified counters and take perhaps six calendar months. In fact, once the taxonomy is well understood, sizing any kind of application can be done in less than a minute.

The pattern-matching approach can be used as a front-end to many kinds of software estimating methods. However, since the historical data used in the development of the pattern-matching approach also includes staffing, effort, schedule, and cost information, it is very easy to display this kind of information as well as size information. In fact, total cost of ownership, quality data, warranty repairs, and even customer satisfaction data can be associated with many of the more common patterns where such historical data already exists.

As data becomes available, the pattern-matching approach can also be used with Agile projects, object-oriented projects, web applications, and even with service-oriented architecture (SOA) applications that are assembled from components rather than being developed in a conventional manner. In addition, the pattern-matching approach can also deal with the various capability levels of the Software Engineering Institute's CMM and CMMI approaches. Pattern-matching can encompass Six-Sigma, formal inspections, and essentially any form of software development. It is only necessary to have sufficient historical data on various forms of software, plus a taxonomy for guiding the search into the appropriate subsections of the overall catalog of historical projects.

Another useful aspect of the pattern-matching approach is the fact that it can be used even before all application requirements are known. In fact, the pattern-matching approach can be used perhaps six months earlier than any other approach for sizing and estimating. The Quick Sizer prototype can not only be used before requirements are fully known, it also predicts the rate at which requirements will grow and change after they are known.

Because the pattern-matching approach is based on a taxonomy of project attributes rather than on specific requirements, it can also be used to size applications whose requirements and specifications are not available. This means that pattern-matching can be used for commercial software such as Microsoft Office, for open-source software such as Linux, for ERP applications such as SAP and Oracle, for aging legacy applications whose specifications are missing, and even for applications whose specifications are classified and secret such as the FBI Carnivore application. A few sample sizes produced by the Quick Sizer prototype illustrate that pattern-matching works for any form of software:

Application	Size in IFPUG Function Points
SAP	296,764
Microsoft Vista	157,658
Microsoft Office 2007	93,498
FBI Carnivore	28,283
Apple I Phone	18,398
Google Search Engine	16,895
Apple Leopard	16,842
Linux	17,672
Denver Airport Luggage Handling	16,508
America OnLine (AOL)	14,761

This information is hypothetical because the problem, code, and data complexity adjustments were approximated by the author rather than being supplied by the development team. However, it is interesting that

the sizing for these very large applications took less than one minute each using the Quick Sizer prototype. Manual function point counting for any of these large applications would require teams of certified counters and take up to several months for the larger projects.

Pattern-matching can also be used for forensic analysis of software disasters, such as the failure of the software application supporting the luggage handling system of the Denver Airport. Usually, the sizes of cancelled software projects are unknown, so forensic analysis of disasters is difficult. Thus the pattern-matching approach is likely to prove useful for software litigation. If a project can be matched to a known taxonomy, then its size can be approximated with reasonable but not perfect precision.

The pattern-matching approach is experimental in 2008, but is likely to become an important method as more and more historical data becomes available. The entire topic of pattern-matching is an emerging technology with great potential for many aspects of software, including sizing and estimating.

Pattern-matching is most effective when the historical data is collected using the same taxonomy as that used for searching the catalog. Therefore, an emerging research topic of considerable economic importance is to perfect a formal taxonomy that can be used to define the nature of software projects in a precise and unambiguous manner. A formal software engineering taxonomy will be as important for software as the taxonomies developed for biological classifications of plants and animals.

SPR Function Points

As a topic of minor historical interest, the first commercial software estimating tool that utilized function point metrics for sizing and estimating was the "Software Productivity, Quality, and Reliability" estimating tool (SPQR/20) that was released in October of 1985 by Software Productivity Research. The number "20" in the name of the tool referred to the number of input questions that needed to be answered in order to generate an estimate.

The SPQR/20 tool was also the first time that function point calculations were automated and included in a commercial software cost-estimating tool. The SPQR/20 tool was also the first to support "backfiring" or conversion from logical code statements into equivalent function point metrics. The tool was also the first to include automatic sizing logic for paper deliverables such as requirements, specifications, and user manuals.

In order to speed up the calculation of function points, some changes were introduced into the normal function point calculation sequence. The SPQR/20 goal was not to produce counts that were different from standard IBM function points, but only to simplify usage and speed up the calculation sequence.

There were two significant changes:

- Complexity was handled by only 3 factors instead of 14.
- The five function point parameters were counted, but not adjusted for complexity.

The SPR function point variation simplified the way complexity was dealt with and reduced the human effort associated with counting function points. The SPR function point methodology yielded function point totals that were essentially the same as those produced by the IBM function point method. In several trials, the SPR method produced counts that averaged within 1.5 percent of the IBM method, with a maximum variation of about 15 percent.

The primary difference between the IBM and SPR function point methodologies in 1985 was in the way the two dealt with complexity. The IBM techniques for assessing complexity were based on weighting 14 influential factors and evaluating the numbers of field and file references. The SPR technique for dealing with complexity was to separate the overall topic of "complexity" into three distinct questions that can be dealt with intuitively:

- How complex are the problems or algorithms facing the team?
- How complex are the code structure and control flow of the application?
- How complex is the data structure of the application?

This simplification of the way complexity is treated allowed the SPR method to backfire function points. If the source code size of an existing application were known, then the SPR function point technique and its supporting software could automatically convert that size into a function point total.

With the SPR function point method, it was not necessary to count the number of data element types, file types, or record types as it was with the IBM method. Neither was it necessary to assign a low, average, or high value to each specific input, output, inquiry, data file, or interface or to evaluate the 14 influential factors as defined by the IBM method.

As a result of the reduced number of complexity considerations, the SPR method could also be applied somewhat earlier than the IBM method, such as during the preliminary requirements phase. The effort required to complete the calculations also was reduced.

The three SPR complexity parameters deal with the entire application rather than with the subelements of the application. Mathematically, the SPR function point methodology has a slightly broader range of adjustments than the IBM methodology (from 0.5 to 1.5), and it

produced function point totals that seldom differed by more than a few percent from the IBM methodology of 1985. The three SPR complexity questions for a new application look like this:

Problem complexity? _____

1. Simple algorithms and simple calculations
2. Majority of simple algorithms and calculations
3. Algorithms and calculations of average complexity
4. Some difficult or complex calculations
5. Many difficult algorithms and complex calculations

Code complexity? _____

1. Nonprocedural (generated, spreadsheet, query, etc.)
2. Well structured with reusable modules
3. Well structured (small modules and simple paths)
4. Fair structure, but some complex modules and paths
5. Poor structure, with large modules and complex paths

Data complexity? _____

1. Simple data with few variables and low complexity
2. Numerous variables, but simple data relationships
3. Multiple files, fields, and data interactions
4. Complex file structures and data interactions
5. Very complex file structures and data interactions

For fine tuning, the SPR complexity questions can be answered with decimal values, and answers such as 2, 2.5, and 3.25 were all perfectly acceptable.

The SPR function point questions themselves were similar to IBM's except that only one set of empirical weights was used. The SPR function point questions are shown in Table 2-22.

Since the SPR function point method was normally automated, manual complexity adjustments were not required when using it. For those who wished to use it manually, the SPR algorithm for complexity adjustment used the sum of the problem complexity and data complexity questions and matches the results to the values shown in Table 2-23.

Note that Table 2-23 shows only integer values for the complexity sum. For decimal results between integer values, such as 3.5, the SPQR/20 tool calculated a fractional adjustment factor. For example, if the complexity sum were in fact 3.5, the adjustment multiplier would be 0.75.

TABLE 2-22 The Original SPR Function Point Method

Significant Parameter		Empirical Weight	
Number of inputs?	_____	× 4 =	_____
Number of outputs?	_____	× 5 =	_____
Number of inquiries?	_____	× 4 =	_____
Number of data files?	_____	× 10 =	_____
Number of interfaces?	_____	× 7 =	_____
Unadjusted total			_____
Complexity adjustment			_____
Adjusted function point total			_____

TABLE 2-23 The SPR Complexity Adjustment Factors

Complexity Sum	Adjustment Multiplier
1	0.5
2	0.6
3	0.7
4	0.8
5	0.9
6	1.0
7	1.1
8	1.2
9	1.3
10	1.4
11	1.5

It may be asked why *code complexity* was one of the questions posed by the SPR method but omitted from the SPR adjustment calculations. The code complexity factor was not required by the logic of the function point metric when producing normal forward function point counts. However, for retrofitting or backfiring function points to existing software, code complexity is an important parameter, as will be seen later.

Because the SPQR/20 methodology was automated rather than manual, it did not require manual calculations at all. Because it dealt with complexity in terms of only three discrete and intuitive parameters and did not require segmentation of individual factors into low, average, and high complexity, it could create function point totals very rapidly. Users who were generally familiar with function point principles and who also know an application well enough to state how many inputs, outputs, inquiries, data files, and interfaces are involved could generate function point totals in less than a minute.

The SPQR/20 function point calculation method was released in October 1985, and it was the first available methodology to provide automatic source code size prediction. Although only 30 common languages were initially sized, the mathematical logic of the SPR size prediction technique could be applied to any or all of the 600 or so existing languages and dialects.

Size prediction was based on empirically derived observations of the level of languages and on the number of statements required to implement a single function point. The history of source code size prediction is older than function points themselves, and the coupling of these two fields of research has been very synergistic.

Prior to the invention of function points, the problems of using lines-of-code metrics had been explored at IBM's San Jose programming laboratory in the late 1960s and early 1970s. This research led to a technique of normalizing productivity rates by expressing all values in terms of "equivalent Assembler statements." For example, if a project were coded in FORTRAN and required 1,000 source code statements and two months of effort, the productivity rate in terms of FORTRAN itself would be 500 statements per month.

However, the same functionality for the application, had it been written in basic assembly language, would have required about 3,000 source code statements, or three times as much code as was actually needed to do the job in FORTRAN. Dividing the probable 3,000 assembly statements by the two months of observed effort generated a productivity rate of 1,500 "equivalent Assembler statements" per month.

The purpose of this conversion process was to express productivity rates in a constant fashion that would not be subject to the mathematical anomalies and paradoxes of using lines of code with the languages of varying power. In a sense, basic assembly language served as a kind of primitive form of function point: The number of assembly statements that might be required to produce an application stayed constant, although the actual number would fluctuate depending upon which language was actually used.

In working with this method, it was soon realized that the normalization technique provided a fairly rigorous way of assigning a numeric level to the power of a language. In the 1960s and 1970s, the terms "low-level language" and "high-level language" had become widespread but "level" had never been mathematically defined.

The level of a language was defined at IBM as the number of basic assembly language statements it would take to produce the functionality of one statement in the target language. Thus COBOL was considered to be level-3 because it would take about three assembly language statements to create the functionality of one COBOL statement. FORTRAN also was considered a level-3 language, since it took about three assembly

language statements to encode functionality available in one FORTRAN statement. PL/I was considered a level-4 language because it took about four assembly language statements to replicate the functions of one PL/I statement. Java is a level-6 language, and SMALLTALK is a level-10 language.

The simple mathematics associated with levels allowed very rapid size conversion from one language to another. For example, if a program was 10,000 statements in basic assembly language, then dividing 10,000 by 3 indicated that the same application would have taken about 3,333 FORTRAN statements. Dividing 10,000 by 4 indicated that about 2,500 PL/I statements might have been required.

By the mid-1970s IBM had assigned provisional levels to more than 50 languages and could convert source code sizes back and forth among any of them. However, source code size conversion is not the same as source code size prediction. It was still necessary to guess at how many statements would be required to build an application in any arbitrary language. Once that guess was made, size conversion into any other language was trivial.

After the publication of the function point metric in 1979, the situation changed significantly. Several researchers, including Albrecht himself began to explore function point totals and source code size simultaneously. The research quickly led to a new definition of language level: "the number of source code statements required to encode one function point." Since function point totals can be enumerated as early as the requirements phase, the new definition implied that true source code size prediction would now be possible.

Software Productivity Research merged the new definition of "level" with the old definition and produced an initial list of some 300 common languages in 1986 that showed both the average number of source code statements per function point and the number of assembly statements necessary to create the functionality of one statement in the target language. Thus, for example, COBOL remains a level-3 language and two facts can now be asserted:

- An average of three statements in basic assembly language would be needed to encode the functions of one COBOL statement.
- An average of 105 COBOL statements would be needed to encode one function point.

Empirically, languages of the same level require the same number of source code statements to implement one function point, but the complexity of the application and its code have a strong impact. Although COBOL, for example, averages about 105 source statements per function point, it has been observed to go as high as 170 source code statements per function point and as low as 50 source code statements per function point.

Because of individual programming styles and variations in the dialects of many languages, the relationship between function points and source code size often fluctuates widely, sometimes for reasons that are not currently understood. Nonetheless, the relation between function points and source code statements is extremely interesting.

Several hundred customers used the SPQR/20 estimating tool for several thousand projects between 1985 and 1991, when it was replaced by a more powerful tool with additional features. During that time other software estimating tools also began to support function point metrics and backfiring as well.

Because the method pioneered both function point sizing and backfiring, a web search for the phrase "SPR function points" found about 784,000 hits in mid-2007 with many of the citations discussing the early history of the approach.

Story Points

Story points are not function points at all, although the use of the term "points" makes that statement slightly ambiguous. Many Agile projects develop user requirements using "stories," which are verbal or text descriptions of what kinds of features and functions the software is likely to contain. Story points are subsets of full user stories, and are a kind of informal juxtaposition of size and complexity, although there seem to be no fixed rules about the mixture.

In Scrum sessions or team meetings, the large user stories are broken down into smaller "story points" that are then used for planning coding and other tasks. Some of the Agile literature suggests that a story is typically decomposed into about three story points. Other reports suggest about eight hours of work per story point, although this claim is frequently challenged. In fact, one way of ascertaining the work hours for story points is a kind of poker game where the developers meet and carry out a workflow analysis more or less similar to poker, with various bids and counter bids. Another description of a method for ascertaining the work hours associated with story points uses a form of the "rock, scissors, paper" game.

Story points are not formally certified by ISO, and although they have many users, there seems to be no formal story point organization circa 2008, unless the Agile Alliance itself is considered to be such an organization. Although spreadsheets and informal tools exist for calculating work effort associated with story points, the method itself is too flexible and informal circa 2008 to have found its way into formal estimating tools such as COCOMO, SLIM, KnowledgePlan, etc.

From looking at several sample stories and descriptions of story points, it appears that a story point is somewhat larger than a typical IFPUG function point: perhaps one story point is roughly equivalent to two function points, or maybe even more.

Because of the popularity of the Agile method, there are hundreds of users circa 2008 and probably more than 5,000 projects in the United States have been sized to date using story points. Europe and the Pacific Rim and South America have probably carried out a similar volume. So long as the Agile methods expand in use, story points should continue to expand.

Since several forms of function point metrics could be used with Agile projects, it would be useful to produce formal conversion rules between IFPUG, COSMIC, and NESMA function points and equivalent story points. Doing this would require formal function point counts of a sample of Agile stories, together with counts of story points. Ideally, this task should be performed by the Agile Alliance, on the general grounds that it is the responsibility of the developers of new metrics to provide conversion rules to older metrics.

A web search in mid-2007 using the keywords "story points" generated a very significant 156,000,000 web citations. Obviously, the story point approach is attracting a great deal of interest, as are the Agile methods.

Unadjusted Function Points

Because calculations involving the 14 value or influential factors are somewhat time consuming and also somewhat subjective, one approach to both speeding up function point calculations and also minimizing subjectivity is to stop when the unadjusted total has been reached. When this method is used experimentally, it is about 50 percent faster than completing the full function point count.

The 14 influential characteristics will of course vary with the application being sized. Sometimes for simple applications they might reduce the size, but more often than not for any application large enough to be of concern, the 14 characteristics will increase the size.

If the function point analysis stops after the unadjusted count, more than likely the results will be perhaps 25 percent smaller than if the count went all the way to completion, although there is no guarantee in which direction the final count might have gone.

Using unadjusted function points is more or less like changing a speedometer from miles to kilometers. If you are going from Boston to New York, the distance will be the same whether you measure in miles or kilometers but you will have a lot more kilometers on your odometer than miles.

As of 2008 probably 100 or so projects have used this method experimentally. The increased speed of using unadjusted function points is perhaps not quite enough of a benefit to make the method popular. A web search on the phrase "unadjusted function points" turned up

761,000 citations. However, many of these citations were discussing unadjusted function points as just one stage in a complete function point analysis.

Use Case Points

The phrase "use cases" refers to one of the methods incorporated into the unified modeling language (UML), although they can be used separately if desired. Because of the widespread deployment of the UML for object-oriented (OO) software, use cases have become quite popular since about 2000. A use case describes what happens to a software system when an actor (typically a user) sends a message or causes a software system to take some action.

Use case points start with some of the basic factors of IFPUG function points, but add many new parameters such as actor weights. Use case points also include some rather subjective topics such as lead analyst capability, motivation, and familiarity with UML.

Since both use case points and IFPUG function points can be used for object-oriented software, some side-by-side results are available. As might be expected from the additional parameters added, use case points typically generate larger totals than IFPUG function points by about 25 percent and sometimes more.

One critical problem limits the utility of use case points for benchmarks. Use case points have no relevance for software projects where use cases are not being utilized. Thus, for dealing with economic questions such as the comparative productivity of OO projects versus older procedural projects that don't capture requirements with use cases, the use case metric will not work at all. On the other hand, standard function points work equally well with UML projects and those using other methods.

Because of the popularity of the UML and use cases, the use case metric probably has several hundred users circa 2008 and perhaps as many as 5,000 projects have been measured to date. However, as of 2008 the use case metric is not one of those certified by the International Standards Organization, nor does there seem to be a formal user association such as IFPUG.

There are currently no certification exams for measuring the proficiency of metrics specialists who want to count with use cases. The approach is growing in numbers, but would benefit from more training courses, more text books, and either creating a user association or joining forces with an existing user group such as IFPUG, COSMIC, NESMA, or one of the others. Since use case metrics are irrelevant for older projects that don't deploy use cases, the lack of conversion rules from use case metrics to other metrics is a major deficiency.

Interest in use case points is illustrated by the fact that a mid-2007 web search using the phrase "use case points" turned up an astonishing 471,000,000 citations, which is more than any other function point variant. A review of several dozen of these citations indicate that the topic of use case points is indeed generating a lot of interest and excitement.

Web Object Points

Because web applications are often built using rather powerful templates and large volumes of reusable material, linking web "applets," and using very powerful graphic tools, they are not very similar to conventional applications developed using normal requirements, design, and coding activities. The volume of reusable material, templates, and frameworks tends to greatly reduce the amount of manual effort for web applications.

Although standard IFPUG and COSMIC function points could be used for web applications, there has been a push to develop special web object points and specialized web estimating tools. For one thing, web applications are often fairly small and are developed so quickly that there is a psychological resistance to the time it would take to calculate function points in the normal fashion.

The well-known consultant Donald Reifer and some other researchers have suggested that web object points might be preferable to standard function points for sizing and estimating web-based applications. Reifer's method is based in part on the older Halstead software science metric, which includes operators and operands. Reifer's suggested sizing method includes topics such as number of building blocks, number of purchased components, and number of graphic files.

Reifer's web estimating approach uses a number of rather cryptic abbreviations such as "PDIF" for platform difficulty, "PEFF" for process efficiency, "PERS" for personnel capability, "PREX" for personnel experience, and a number of others.

The Reifer approach has attracted quite a bit of interest from the web community, and has been used on enough projects (more than 50) to begin to see some interesting results. The literature on web objects and web object points is expanding. A web search turned up about 126,000,000 citations but the great majority of these deal with the phrase "web objects" in a general sense, not with web object points in the context of estimation.

It is difficult to make a direct comparison between web object points and normal function points, but a speculative analysis leads to the conclusion that a web object point might be roughly equivalent to perhaps two standard IFPUG function points. If that is so, then apparent

productivity expressed in terms of web object points would be about twice as fast as productivity expressed in terms of function points, using the metrics "web object points per staff month" and "function points per staff month."

The number of web applications is increasing geometrically, while there are actual declines in the numbers of several forms of traditional software development projects. In the future as web-based projects become the dominant form of software, there will be a pressing need for formal metrics and effective software estimation methods.

It would be useful for IFPUG, COSMIC, NESMA, and other function point organizations to pay very close attention to the special needs of web-based applications.

Variations in Application Size and Productivity Rates

It is interesting to experiment with the 20 variations of function point metrics and try them on the same projects. The following two tables of comparative data should be viewed as "thought experiments" rather than as actual counts of function points. The function point sizes are speculative and based on merely reading the counting rules and the supporting documents that describe the various methods. The results are not based on actual counts using the methods. Actually counting the same application using all 20 methods would require about two weeks of work. The applications themselves, although hypothetical, are derived from actual projects studied by the author in the past.

Three different kinds of projects are used in this experiment. Using the IFPUG 4.2 counting rules, all three would be sized at 1,500 function points. The three applications are

- A financial application of 1500 function points in size, developed using COBOL and SQL. This project used joint application design (JAD) and conventional structured design and development. It did not use Agile methods or use cases. This is an information technology project. This project had a productivity rate of 12 function points per staff month, and the total effort was 125 staff months.
- A PBX or private branch exchange switching system of 1,500 function points developed using the C++ programming language. This application did use some use cases, but it was developed in a traditional structured manner rather than using Agile methods. This application is a systems software project. The project had a productivity rate of 10 function points per staff month, and the total effort was 150 staff months.

- A web-based software cost-estimating tool of 1,500 function points developed using Java, HTML, Visual Basic, and CORBA. This application used a mix of story points and use cases. The application was an expert system with many complex algorithms. However, most of the web features were reusable and derived from CORBA and other sources of reusable material. This application is a web project. The project had a productivity rate of 25 function points per staff month, and the total effort was 60 staff months.

Table 2-24 starts with the size expressed in terms of IFPUG function points, and then shows the comparative sizes of the other methods listed below. Note that not every method works with every project.

TABLE 2-24 Size Comparison of Selected Function Point Variants

Function Point Variations	IT Financial Project	PBX Switching System	Web-Based Estimating Tool
IFPUG 4.2	1,500	1,500	1,500
3D function points	1,600	1,700	1,800
Backfiring	1,400	1,600	1,700
COSMIC	1,550	1,750	
DeMarco bang function points	1,700	1,900	1,850
Engineering function points		2,000	2,000
Feature points	1,475	1,650	1,750
FP Lite	1,550	1,550	1,575
Full function points	1,600	1,750	1,800
Mark II	1,600	1,800	1,650
NESMA	1,525	1,575	1,575
Pattern-matching	1,550	1,550	1,550
SPR function points	1,550	1,550	1,550
Story points			3,200
Unadjusted function points	1,300	1,400	1,400
Use case points		2,000	2,200
Web object points			1,800
AVERAGE	1,531	1,685	1,806
STANDARD DEVIATION	99.56	180.72	422.15
MINIMUM SIZE	1,300	1,400	1,400
MAXIMUM SIZE	1,700	3,000	3,200

For example, the COSMIC method was not used on the web project because the COSMIC literature states that it is not suitable for projects with large numbers of algorithms. The use case and story point metrics only work for projects that make use of these design methods. The engineering function point approach is aimed exclusively at systems software.

All of the variations in counting function points are fairly close together for the IT financial application. Recall that the stated goal of 3D, COSMIC, Mark II, full function points, and engineering function points is to produce larger counts than IFPUG for systems software. Therefore, the differences in size between the function point variations grow wider for the more complex forms of software.

The variations in size are accompanied by similar variations in productivity rates. Note that these variations in rates are due to the metrics themselves: the actual number of months of effort did not change at all. Table 2-25 illustrates variations in apparent productivity.

Here too the range of variation is smallest for the IT financial projects, and much larger for the systems software and for the web project. Recall that the productivity variations are entirely caused by the variations in apparent size. The actual effort for the projects is the same in every case.

Readers probably have questions as to which of the many forms of functional metric is most appropriate. This is not an easy question to answer. Any of the four methods that have been reviewed and certified by the ISO can be used with reasonable success: COSMIC, IFPUG, Mark II, and NESMA in alphabetical order.

Narrowing down the choices depends upon the primary purpose of a measurement program:

- For benchmarks and comparisons of productivity and quality levels, the IFPUG function point metric as the oldest form also has the largest number of measured projects. In fact, the IFPUG form has more historical data than all of the other metrics combined.
- If the effort and costs of counting function points are troublesome, then backfiring, FP Lite, and the NESMA indicative method are fairly quick and inexpensive. (The even faster pattern-matching approach is experimental as this book is written.)
- If you are using Agile methods or use cases, and you don't care about benchmarks but are concerned with estimating current projects, then story points and use case points are suitable choices.
- If you are building web applications and are looking for estimating information, then web object points and story points should be considered. Both are probably easier to use in web environments than the older forms of function point metric, although the older forms could certainly be used.

TABLE 2-25 Productivity Comparison of Selected Function Point Variants

Function Point Variations	IT Financial Project	PBX Switching System	Web-Based Estimating Tool
IFPUG 4.2	12.00	10.00	25.00
3D function points	12.80	11.33	30.00
Backfiring	11.20	10.67	28.33
COSMIC	12.40	11.67	–
DeMarco bang function points	13.60	12.67	30.83
Engineering function points	–	13.33	33.33
Feature points	11.80	11.00	29.17
FP Lite	12.40	10.33	26.25
Full function points	12.80	11.67	30.00
Mark II	12.80	12.00	27.50
NESMA	12.20	10.50	26.25
Pattern-matching	12.40	10.33	25.83
SPR function points	12.40	10.33	25.83
Story points	–	–	53.33
Unadjusted function points	10.40	9.33	23.33
Use case points	–	13.33	36.67
Web object points	–	–	30.00
AVERAGE	12.25	11.23	30.10
STANDARD DEVIATION	4.37	3.04	9.99
MINIMUM PRODUCTIVITY	10.40	9.33	25.83
MAXIMUM PRODUCTIVITY	12.80	20.00	53.33
STAFF MONTHS OF EFFORT	125	150	60

- Some of the older variations such as 3D function points, feature points, and full function points have more or less dropped out of use circa 2008. They could of course still be used, but finding any recent data for comparative purposes would be difficult.

Although the plethora of function point variations in 2008 is confusing, the class of function point metrics are much more effective for economic analysis and quality analysis than the older LOC metrics, which have severe counting problems and yield highly unreliable, paradoxical results.

Other scientific and engineering fields have dealt with multiple metrics for more than 100 years. If other engineers can deal with Fahrenheit and Celsius; with statute miles, nautical miles, and kilometers; with British and American tons and gallons; with yards and meters; then software engineers can deal with multiple flavors of function point metric.

Although the pattern-matching approach is experimental as this book is written, it offers the potential of being able to size any form of software project in less than one minute coupled with reasonably good precision. Pattern-matching is an emerging technology that is likely to enter the mainstream of software engineering within a very few years.

Because new languages continue to develop at a rate of more than three per year, a static table in a book is not sufficient to cover all of the latest language developments. The SPR web site contains an online table that now tops 600 programming languages and dialects, and continues to grow on an annual basis.

Why the software industry needs more than 600 programming languages is a question with no good answers. Old languages die out and stop being used and new ones are being frequently created. One of the major maintenance problems of the software industry in the 21st century is finding maintenance programmers who are knowledgeable in dead languages such as mumps.

However, there is another issue that needs to addressed. Many maintenance changes are quite small: sometimes only a few lines of code are changed. But due to the lower limits in calculating function point complexity, the lower limit for normal function point calculations is about 15 function points. What is needed is some form of "micro function point" that could deal with very small changes, perhaps even a fraction of a function point.

It is not hard to envision such a micro function point. It would be calculated from a bug report or change request, and would use the normal five factors. The only difference is that the complexity adjustments would have to be modified to produce very small function point counts.

Of course, backfiring can also be used to create micro function points. Suppose, for example, that a change to a COBOL application consists of 10 lines of code. Since COBOL requires about 105 statements per function point, this particular change would be about 0.095 function points.

Within a few years, however, the IFPUG organization had begun to provide counting rules for systems and embedded software. When that occurred, the need for feature points diminished. It is better to have a standard metric that works for all kinds of software than it is to have numerous specialized metrics that only work for one kind of software. This greatly simplifies benchmarking and statistical analysis.

Future Technical Developments in Functional Metrics

Functional metrics have made remarkable progress over the past 30 years. There are now major international societies of functional metric users. A working standards committee exists for both IFPUG and COSMIC function points, and they are actively addressing both extensions to the methodology and clarification of the topics that require it.

The functional metric techniques have spread from their origins in management information systems and are starting to be used for systems and scientific software as well. Exciting new developments in functional metrics are occurring on almost a daily basis. It is apparent that the following evolution of the functional metrics concepts should occur over the next few years.

Automatic Derivation of Function Points from Requirements and Design

Since the major factors that go into function point and feature point calculations are taken from the requirements and design of the application, it is an obvious step to forge a direct connection between design tools and a function point calculation tool.

This desirable step should enable automatic and nonsubjective counts of the basic functional metric parameters. Complexity adjustments, however, may still require some form of human intervention. It is not impossible to envision that even complexity adjustments may eventually become precise and objective as a by-product of research getting under way.

Software Productivity Research, for example, has begun a study of more than 150 specification and design methodologies with a view to extracting the basic function point parameters from the standard design representations or, alternatively, making minimal modifications to the standard design representations in order to facilitate direct extraction of function point or feature point parameters.

Automatic derivation should be fairly easy from formal requirements and design methods such as JAD, UML, and use cases. Automatic derivation would be more difficult from informal requirements and design methods such as Agile Scrum sessions.

Automatic Backfiring of Function Points and Feature Points from Source Code

Now that function points are becoming a de facto standard metric for productivity studies, there is significant interest in retrofitting function

points to aging applications that might have been created even before function points were invented or, in any case, that did not use function points during their development cycles.

Since backfiring of function points or converting source code size into function point totals is already possible, it is an obvious next step to automate the process. That is, it is technically possible to build a scanning tool that would analyze source code directly and create function point totals as one of its outputs.

Several existing complexity analysis tools such as Battlemap and the Analysis of Complexity Tool (ACT) would require only minor modifications to support backfiring. In fact, these tools would probably be far more accurate than manual methods because both of them capture cyclomatic and essential complexity measures, which can impact the backfiring ratios.

Automatic Conversion from IFPUG Function Points to COSMIC Function Points, Mark II Function Points, NESMA Function Points, and Other Variations

One unfortunate aspect of the rapid growth of functional metrics has been the proliferation of many variations, each of which uses different counting methods and creates different totals. It appears both technically possible and also desirable to establish conversion factors that will allow data to be mathematically changed from method to method.

Such conversion techniques exist between the IBM function point method and the SPR function and feature point methods, and indeed have been done automatically by the CHECKPOINT® software tool.

However, conversion factors for the COSMIC function point, story points, web object points, and many other variations have not yet been published.

Extension and Refinement of Complexity Calculations

It has been pointed out many times that the possible Achilles heel of functional metrics in general and function points in particular is the way complexity is treated. In the original 1979 version of function points, complexity was purely subjective and covered a very small range of adjustments of about 25 percent.

In the 1984 revision, and still today in 2008, the range of adjustments was extended and the rigor of complexity analysis was improved, but much subjectivity still remains. Today complexity adjustments can top 250 percent. The assertion of subjectivity is also true of the other flavors of functional metrics, such as the COSMIC, NESMA, and story point techniques.

There are several objective complexity metrics, such as the McCabe cyclomatic and essential complexity methods, that appear to be promising as possible adjuncts to functional metrics. Several researchers have started exploring possibilities for extending and refining business complexity concepts for use with functional metrics.

SPR has identified 20 different forms of complexity that are possible candidates for future coupling functional metrics, including computational complexity, semantic complexity, entropic complexity, and many others. In any event, complexity research in the context of functional metrics is undergoing energetic expansion.

Publication of Estimating Templates Based on Functional Metrics

From 1979 through 2008, function points had been applied to thousands of applications. Now that so many applications have been explored, a new form of research is starting to emerge: Patterns or "templates" of function point totals for common application types are discovered. It can be anticipated that the next decade will witness the publication of standard guidelines, empirically derived, for many different kinds of applications. These templates will allow very early estimating.

Utilization of Functional Metrics for Studies of Software Consumption

During the first decade of the growth of functional metrics, almost all the studies were aimed at exploring software development or production. However, functional metrics have very powerful, and currently only partly explored, capabilities for studying software consumption as well. It can be anticipated that the next decade will see a host of new studies dealing with usage patterns and the consumption patterns of software functions.

Utilization of Functional Metrics for Software Value Analysis

Assessing or predicting the value of software has been one of the most intractable measurement and estimation problems of the software era. Since the previous lines-of-code metric had essentially no commercial value and was neither the primary production unit nor the primary consumption unit of large software projects, it was essentially useless for value analysis. Although functional metrics are only just starting to be applied to value analysis, the preliminary results are encouraging enough to predict much future progress in this difficult area.

Extending the Concept of Function Points into Areas That Lack Effective Measurements

Function points have proven to be very successful for economic studies of software development and maintenance, and also for studies of software quality. This brings up the point that companies and government agencies have other topics that need effective measurements, but where none exist in 2008.

All large corporations own more data than they own software. But there is no effective size measure for the volume of data in a repository or database. Therefore, the industry lacks good economic data about the costs of data creation and ownership, and it lacks good quality information about "data quality." It would be possible to create a "data point" metric that uses a format similar to function points.

Another area lacking effective measurements is that of "value." Financial value such as revenue, profits, and losses can be measured with good precision. But there are many intangible, non-financial aspects of value that have no effective metrics. For example, what about the value of medical software that can save lives or diagnose illness? What about the value of military weapons systems that are used for defense. What about the value of software packages that are used to measure customer satisfaction or employee morale?

It would be possible to create a "value point" metric that integrated well-known financial values with intangible value topics. The goal would be to have an integrated value metric that could be used to size intangible values and therefore allow reasonable business analysis of non-financial value topics.

Overall Prognosis for Functional Metrics

The software industry suffered for more than 50 years from lack of adequate measurements. Now that functional metrics have made adequate measurements technically possible, it can be anticipated that the overall rate of progress in both software productivity and software quality will improve. Measurement alone can focus attention on areas of strength and weakness, and now that software can be measured effectively, it can also be managed effectively for the first time since the software industry began!

Selecting a Chart of Accounts for Resource and Cost Data

The function point method is a normalizing metric whose primary purpose is to display productivity and quality data. To be meaningful,

the effort and the cost data itself must be collected to a standard chart of accounts for all projects. One of the major problems with measuring software projects has been the lack of a standard chart of accounts. For example, the simplest and most primitive chart of accounts would merely be to accumulate all costs for a project without bothering to identify whether those costs were for requirements, design, coding, testing, or something else. This kind of data is essentially impossible to validate, and it is worthless for economic studies or any kind of serious exploration.

A slightly more sophisticated way to collect data would be to use a phase-level chart of accounts that segregated effort and costs among these five elements:

- Requirements
- Design
- Development
- Testing
- Management

This technique is better than having no granularity at all, but unfortunately it is insufficient for serious economic studies. Consider, for example, the testing cost bucket. The smallest number of tests performed on a software project can be a single perfunctory unit test. Yet some large projects may carry out a 12-step series that includes a unit test, function test, stress test, performance test, independent test, human factors test, integration test, regression test, system test, field test, and user acceptance test. If all testing costs are simply lumped together under a single cost bucket labeled "testing," there would be no serious way to study the economics of multistage test scenarios.

The smallest chart of accounts that has sufficient rigor to be used effectively with MIS projects, systems software, and military software contains 25 cost elements, with the total project serving as the twenty-sixth cost accumulation bucket. Software Productivity Research has produced such a uniform chart of accounts.

An early function point chart of accounts was developed by IBM primarily for MIS projects. This chart of accounts used 20 tasks that are not really suitable for military projects or systems software projects. For example, the IBM chart of accounts excludes quality assurance, independent verification and validation, independent testing, design and code inspections, and many other activities that are common outside the MIS world but not within it. The IBM chart of accounts also excludes tasks associated with really large systems, such as system architecture and project planning.

It should be clearly understood that because from 20 to 25 cost buckets are available for recording staffing, effort, schedule, and cost information, that does not imply that every project will in fact carry out all of the tasks. Indeed, MIS projects routinely perform only from 6 to 12 of the 25 tasks. Systems software projects tend to perform from 10 to 20 of the 25 tasks. Military projects tend to perform from 15 to all 25 of the tasks, which is one of the reasons for the high costs of military projects. When a project does not carry out one or more of the specific tasks in the chart of accounts, that task is simply set to contain a zero value. The following are the IBM and SPR charts of accounts for comparative purposes:

SPR Chart of Accounts	IBM Chart of Accounts
1. Requirements	1. Project management
2. Prototyping	2. Requirements
3. System architecture	3. System design
4. Project planning	4. External design
5. Initial analysis and design	5. Internal design
6. Detail design and specification	6. Program development
7. Design reviews and inspections	7. Detail design
8. Coding	8. Coding
9. Reusable code acquisition	9. Unit test
10. Purchased software acquisition	10. Program integration
11. Code reviews and inspections	11. System test
12. Independent verification and validation	12. User documentation
13. Configuration control	13. User education
14. Integration	14. File conversion
15. User documentation	15. Standard task total
16. Unit testing	16. Studies
17. Function testing	17. Package modification
18. Integration testing	18. Other
19. System testing	19. Nonstandard task total
20. Field testing	20. Development total
21. Acceptance testing	
22. Independent testing	
23. Quality assurance	
24. Installation and user training	
25. Project management	
26. Total project costs	

Note that the two charts shown above are essentially top-level charts. The SPR chart of accounts, for example, expands into a full work-breakdown structure encompassing more than 150 subordinate tasks.

In the modern world circa 2008 Agile projects are quite common. The Agile approach would need a slightly different chart of account than conventional waterfall or spiral projects. For example, one of the Agile methods is extreme programming (XP). A chart of accounts for an extreme programming application might resemble the following:

Agile Chart of Accounts

1. Initial planning
2. Test case development for initial iteration
3. Scrum session for testing
4. Development of initial iteration
5. Scrum session for development
6. Internal testing of initial iteration
7. Scrum session for internal testing
8. User documentation of initial iteration
9. Scrum session for user documentation
10. Release of initial iteration
11. Scrum session for release
12. User acceptance of initial iteration
13. Scrum session for acceptance test

The above Agile chart of accounts would be repeated for as many iterations as occur during the overall development of the application. For a typical Agile project of 1,500 function points in size, there would probably be five iterations, each of about 300 function points in size.

But suppose the Agile methods are going to be used on something really large, such as a 20,000-function point telephone switching system that might have thousand of users, so that it is unlikely that direct user participation will be effective.

In this case, some additional activities would have to be included in the chart of accounts. These additional activities might include

1. Configuration control
2. Change control
3. Integration
4. Software quality assurance (SQA)
5. Performance testing by specialists (not by developers)
6. System testing by specialists (not by developers)
7. Translation of documents into foreign languages

8. Cost and milestone data collection
9. Defect and quality data collection

What comes to mind as an analogy is construction of physical objects such as buildings. It is possible to construct a small building such as a tool shed of 100 square feet in a reasonably informal manner without using any specialists. Adhering to local building codes is the main regulation that would be required. Design changes during development are casual and decided by conversations between the owner and the carpenter.

But constructing an office building of 200,000 square feet is a much more serious and complicated construction job. For one thing there are dozens of different kinds of codes and inspections, such as electrical codes, plumbing codes, environmental code, and many more.

Further, many kinds of specialists will be needed for a large building, because generalists are prohibited by law from installing the electrical wires, transformers, and other electronics components. Generalists are prohibited by law from doing the plumbing too. If some or all of the contractors are in unions, there will also be union regulations that affect the kinds of work that can performed, overtime, and other topics.

In other words, building a large structure is not a casual undertaking and there will be many specialists and many requirements that are not associated with building small structures.

Once the chart of accounts is settled, the next topic is what data should be collected for each activity.

The data collected with the standard chart of accounts is the unambiguous hard data that is not likely to be colored by subjective personal opinions:

- The size of the staff for each activity
- The total effort for each activity
- The total cost for each activity
- The schedule for each activity
- The overlap or concurrency of activity schedules
- The deliverables or work products from each activity

If, as often happens, a project did not perform every activity in the standard 25-activity chart of accounts, those cost buckets are simply filled with zero values. If, as also happens, additional activities were performed below the level of the standard 25-activity chart of accounts, subordinate cost buckets can be created. Productivity studies have sometimes been carried out by using as many as 170 cost buckets for a chart of accounts.

It is dismaying and astonishing that in almost 50 years of software history, there has never been an industry standard for the chart of accounts that should be used to collect software project resource data! Now that functional metrics are becoming the new standard, it is hoped that the importance of standardizing a chart of accounts will soon be addressed.

Summary of and Conclusions About Functional Metrics

Functional metrics in all their forms provide the best capability for measuring economic productivity in software history. Although training is necessary before starting and care must be exercised to ensure consistency, function points are worth the effort. It is appropriate to end this section by summarizing 12 goals for functional metrics:

The 12 Essential Goals of Functional Metrics

1. The metric should deal with software's visible features.
2. The metric should deal with factors important to users.
3. The metric should be applicable early in the life cycle.
4. The metric should reflect real economic productivity.
5. The metric should be independent of source code.
6. The metric should be easy to apply and calculate.
7. The metric should assist in sizing all deliverables.
8. The metric should retrofit to existing software.
9. The metric should work for maintenance and enhancements.
10. The metric should work with all software types including MIS projects, systems software projects, real-time and embedded software projects, and military software projects.
11. Hard project data (schedules, staff, effort, costs, etc.) should be collected by using a standard chart of accounts.
12. Soft project data (skills, experience, methods, tools, etc.) should be collected in an unambiguous fashion that lends itself to multiple regression analysis.

Function points are still in evolution. Function points are providing new and clear insights into software productivity and quality. They are key steps leading to the development of software engineering as a true engineering profession.

In conclusion, measurement is the key to progress in software. Without accurate measurements in the past, the software industry has

managed through trial and error to make progress, but the progress has been slower than is desirable and sometimes erratic. Now that accurate measurements and metrics are available, it can be asserted that software engineering is ready to take its place beside the older engineering disciplines as a true profession, rather than an art or craft as it has been for so long.

Software Measures and Metrics Not Based on Function Points

Although various forms of functional metrics have become dominant for economic studies and are also widely used for quality studies, there are a number of other measures and metrics used in the software world that are completely detached from functional metrics.

As with the section on functional metrics, short discussions of these other approaches will be discussed here. For actually learning to use the approaches, full text books or courses by expert practitioners are recommended. Also recommended are web searches using the names of the metrics as keywords in the search argument. Some of the interesting non-functional metrics applied to software include, in alphabetical order:

- Agile project metrics
- Balanced scorecard metrics
- Defect removal efficiency metrics
- Earned value metrics
- Goal question metrics

Let us consider these interesting metrics in turn.

Agile Project Metrics

Because the Agile methods are fairly new and in rapid evolution as of 2008, the varieties of metrics used with Agile projects varies from project to project and method to method. Story points, which are often used with Agile projects, were discussed earlier in this chapter. There is also a form of the "earned value" metrics for Agile projects, and earned value metrics are discussed later in this chapter.

What seems to be missing as this book is written is a large collection of Agile benchmark data that show topics such as:

- Average size of Agile projects (in any known metric)
- Largest Agile project to date (in any known metric)
- Average schedules for Agile projects

- Longest schedule for Agile projects
- Average productivity for Agile projects
- Highest productivity for Agile projects
- Average defect potentials of Agile projects
- Average delivered defect rates for Agile projects
- Average bad-fix injection rates for Agile projects
- Comparisons of Agile results against similar non-Agile projects

Until this kind of basic benchmark data is assembled and validated, the true economic and business value of the Agile approach is merely speculative. Worse, until valid benchmark data is collected, the Agile approach is likely to be viewed as just another software fad such as I-CASE or RAD, i.e., methods that made vast claims, but that were never able to back those claims up with any proof of success.

ISBSG would be the logical place to serve as a repository for Agile project data. In fact, the ISBSG does have a few samples of Agile projects, but not really enough to draw firm conclusions. Scans of more than 100 articles and web sites on Agile topics did not turn up even a single citation that discussed benchmarks, or even any articles that acknowledged the existence of the ISBSG. The Agile approach cannot live in isolation, and there is a strong need to contribute project data to the ISBSG in order to demonstrate the economic validity of the Agile approach and reduce the perception that Agile may be just another software cult.

That being said, the Agile approach is exploring a number of interesting metrics, some of which may have relevance to non-Agile projects as well. One of these interesting metrics is "running tested features" or RTF. The RTF metric measures the numbers of tested features turned over to clients or users, measured against time. Since the philosophy of the Agile method is to produce running code quickly, the RTF approach is a good match to that philosophy. The RTF approach is experimental and there are not yet normative values, but what might seem reasonably congruent with the Agile concept is a rate of perhaps two per month.

Although the RTF approach does not specify corollary size metrics, examining a small sample of a dozen or so tested features indicates that a provisional size would be about 20 function points or roughly 1,000 Java statements.

Another interesting experimental metric is the "break-even point" or the spot in a development cycle where the value of what has been delivered to date equals the costs that have been expended to date. This metric is valid for both Agile and non-Agile projects. The expected results with Agile projects is to have a break-even point about 50 percent

earlier in time than a traditional waterfall project for applications of the same size. Is this really what happens? Well, as of 2008, it sometimes happens, but there needs to be a lot more and better data collected, as cited in the list of benchmark topics at the beginning of this section.

In general, function point metrics have not had much usage among the Agile community. The main reason for this is probably because the high costs and long schedules for counting function points are at odds with the basic Agile philosophy. Further, since accurate counts of function points require a certified counter, that would mean that using function points would entail either hiring an outside consultant or having a team member take a course in function point analysis and then pass the certification examination.

In order for function point metrics to become attractive and useful to the Agile community, much quicker counting methods are needed. The backfiring and pattern-matching approaches are both quick enough to be of possible interest to the Agile community, but neither method has yet been adopted by the Agile community as of 2008.

Among the most interesting of the Agile measurement approaches is not a specific metric, but a way of coordinating progress called "Scrum," which is a term taken from the game of Rugby.

In an Agile context, Scrum sessions are team meetings that occur often, usually every day, and that have specific purposes. Some of these purposes include

- Defining the backlog, or sets of features awaiting development
- Planning "sprints" or the development cycle for either one or a small number of features
- Discussing progress on a sprint that is currently underway
- Discussing problems or factors that may delay a sprint
- Discussing the results of a sprint that was just completed

To keep the sprints from becoming chaotic, they are controlled by a "Scrum master" who is empowered to set the agenda and keep the meetings focused on the relevant topics. The attendees at the Scrum sessions include the developers, the resident customer who is part of the team, and various specialists such as quality assurance, technical writing, or graphics design (if needed).

Overall, the Scrum sessions have been among the most significant contributions of the Agile approach to coordinating software projects. Because of the focus on important issues and because they occur often, Scrums have the practical effect of making all issues visible. Further, if problems occur, the Scrum sessions provide a group dynamic for dealing with them before the problems grow so large that they cannot be controlled.

As of 2008 the Agile methodologies (and there are many variants) are on a sharp upswing. A 2008 web search using the phrase "Agile software development" as the search argument turned up 5,600,000 citations. From reviewing about 100 of these, perhaps 60 percent were impassioned evangelical blogs by Agile missionaries. About 30 percent were fairly serious technical articles that dealt with practical matters. About 10 percent were complaints about gaps and shortcomings in the Agile approaches by those who feared that Agile offered more hot air than actual accomplishment.

However, if the Agile methods are going to last longer than the approximate ten-year life cycle of other software development methods, the Agile community is going to have to do a much better job of collecting cost, productivity, and quality data than has occurred to date.

Balanced Scorecard Metrics

Dr. Robert Kaplan and Dr. David Norton of the Harvard Business School are the originators of the balanced scorecard approach. This approach is now found in many major corporations and some government agencies as well. The balanced scorecard approach is customized for specific companies, but includes four major measurement topics: 1) the learning and growth perspective; 2) the business process perspective; 3) the customer perspective; and 4) the financial perspective. Although the balanced scorecard approach is widespread and often successful, it is not a panacea.

The balanced scorecard approach was not developed for software. However, the balanced scorecard approach can be applied to software organizations as it can to other operating units. Under this approach conventional financial measures are augmented by additional measures that report on the learning and growth perspective, the business process perspective, the customer perspective, and the financial perspective.

Although the balanced scorecard was developed without any reference to function points at all, it happens that in recent years function point metrics are starting to be used experimentally for some aspects of the balanced scorecard approach:

- Learning and Growth Perspective
  - Rate at which users can learn to use software (roughly 1 hour per 100 function points)
  - Tutorial and training material volumes (roughly 1.5 pages per 100 function points)
- Business Process Perspective
  - Application and portfolio size (can be measured in function points)
  - Rate of requirements creep during development (can be measured with or without function points)

  - Volume of development defects (can be measured with or without function points)
  - Ratios of purchased software to custom software (usually measured with dollars spent)
  - Ratios of software to custom software (can be measured with lines of code or function points)
  - Annual rates of change of applications (measured with dollars, code, or function points)
  - Productivity (can be measured several ways)
- Customer Perspective
  - Number of features delivered (can be measured with function points)
  - Software usage (can be measured with function points)
  - Customer support costs (can be measured with dollars or function points)
  - Customer-reported defects (can be measured with and without function points)
- Financial Perspective
  - Development costs (can be measured with dollars or function point)
  - Annual maintenance cost (can be measured with dollars or function point)
  - Termination or cancellation costs (can be measured with dollars or function points)

Probably 90 percent of the time the balanced scorecard method is used without function points at all. However, when it is augmented by functional metrics, the results are often useful. This combination of function points and the balanced scorecard is still evolving as of 2008.

The business significance of the balanced scorecard approach is demonstrated by the fact that a web search in 2008 using the phrase "balanced score card" turned up 2,680,000 citations.

Defect Removal Efficiency Metrics

One of the most powerful and useful software quality metrics does not require function points at all, or lines of code either. The only requirement for measuring defect removal efficiency is a careful accumulation of defect report counts both before delivery to customers and after customers begin to use the software.

Suppose in the course of development a new software project uses formal design and code inspections and four kinds of testing: unit test,

new function test, stress test, and system test. Data is collected on all defects found in every inspection and test stage.

After the customers receive the software application, all defects that they find and report are counted. Usually, a fixed period of 90 days is used for measuring defect removal efficiency, but sometimes a longer period of 6 to 12 months may be used.

After all development defects and customer-reported defects are accumulated, the formula for calculating defect removal efficiency is to divide the in-house defects by the total defects and express the result as a percentage.

Here is an example of how defect removal efficiency is usually measured:

Defect Removal Activities	Defects Found
Design Inspections	200
Code Inspections	400
Unit Test	125
New Function Test	75
Stress Test	25
System Test	75
Subtotal	900
Customer-reported defects(First 90 days of usage)	100
TOTAL DEFECTS FOUND	1,000

In this example, the defect removal efficiency is calculated by dividing 900 by 1,000, and the result is a 90 percent cumulative defect removal efficiency level. As of 2008 the approximate U.S. average for defect removal efficiency is only about 85 percent. Best in class organizations top 95 percent. Therefore, a defect removal efficiency of 90 percent would be better than average, and is actually a fairly respectable rate.

Note that even more interesting results are possible. The same data can be used to evaluate the defect removal efficiency levels of every specific form of inspections and test. For this kind of analysis, the defects that have already been found are subtracted from the total because they are already gone when the defect removal activity begins.

Let us bypass the initial design inspection, because it only finds design errors and not coding errors. All of the other forms of inspection test and find coding errors.

At the time code inspections were performed, there were still 800 latent defects in the application, and the data shows that the code inspections found 400, so the code inspections had a defect removal efficiency of 50 percent.

At the time unit testing was performed, there were still 600 latent defects in the application and the data shows that unit testing found 150, so the defect removal efficiency was 25 percent.

At the time new function testing was performed there were still 275 latent defects in the application and the data shows that new function testing found 75, so the defect removal efficiency was 27 percent.

At the time stress testing was performed there were still 200 latent defects in the application and the data shows that stress testing found 25, so the defect removal efficiency was 12.5 percent.

At the time system testing was performed there were still 175 latent defects in the application and the data shows that system testing found 75, so the defect removal efficiency was 43 percent.

Readers who have never measured defect removal efficiency levels will probably be shocked at how low most forms of testing are in terms of defect removal efficiency levels. However, the data in this simple example is fairly close to U.S. averages. Most forms of testing are not very efficient and seldom top about 35 percent, which is why so many different kinds of testing are needed.

It is also true that formal code inspections are about twice as efficient as most forms of testing. In fact, sometimes code inspections top 85 percent in defect removal efficiency, which is almost three times better than normal testing stages. As can easily be seen, measurements of defect removal efficiency levels provide a great deal of useful quality information for a comparatively low cost. These measures are easy to perform and very insightful. Also, they don't require function points to be useful.

A web search using the phrase "defect removal efficiency" in mid-2007 turned up a total of 1,640,000 citations. The value of defect removal efficiency is higher than almost any other form of metric, and the costs are quite low. As it happens, raising defect removal efficiency levels to about the 95 percent plateau is one of the most effective known approaches to software process improvement. Projects at the 95 percent plateau seldom overrun their schedules, have very high productivity rates, very good customer satisfaction, and very good team morale.

Earned Value Metrics

The method called "earned value" originated for defense projects and was not created specifically for software. As a result, the earned value approach is independent of function point measures, although function points can be used as a supplemental earned value metric if desired.

With the earned value approach, a project is divided into discrete work packets of fairly short durations and fairly concise deliverables. For software projects, the work packets might be based on common software deliverables. Once the work packets are defined, each has a planned schedule duration, planned effort, and planned cost attached to them, and also a planned value.

The earned value approach uses some special terms and acronyms that need to be understood. Although the terms are daunting when first encountered, they quickly become familiar. Also, there are dozens of commercial software tools that can calculate earned value so once a work breakdown structure and work packets are complete, the calculations are not too difficult. Although collecting the actual data can be difficult from time to time. The main terms and acronyms for the earned value method are

BCWS	Budgeted Cost of Work Scheduled
ACWS	Actual Cost of Work Scheduled
BCWP	Budgeted Cost of Work Performed
ACWP	Actual Cost of Work Performed
SV	Schedule Variance (BCWP-BCWS)
CV	Cost Variance (BCWP-ACWP)
SPI	Schedule Performance Index (BCWP/BCWS)
CPI	Cost Performance Index (BCWP/ACWP)
CSI	Cost Schedule Index (CPI * SPI)

Following are some representative work packets that might be defined for a software project:

Work Packet	Schedule in Calendar Months	Effort in Staff Months	Costs in Dollars
Requirements complete	3	15	$150,000
Design complete	6	24	$240,000
Initial code segments complete	5	30	$300,000
Final code segments complete	4	24	$240,000
Test complete	3	18	$180,000
User manuals complete	3	9	$90,000
TOTAL	24	120	$1,200,000

These planned values or budgets are compared against actual results as the project unfolds. Let us suppose that the first packet or "requirements complete" actually took 4 calendar months, 20 staff months, and $200,000 due to some unforeseen difficulty in ascertaining the requirements. As can be seen we have a schedule overrun of 1 month, an effort overrun of 5 months, and a cost overrun of $50,000.

Without going into the actual calculations for SPI, CPI, CSI, etc., the bottom line on the earned value approach, derived from hundreds of projects, is an important lesson: Early overruns in costs and budgets are difficult to recover from. Early overruns in cost will affect final costs. Early overruns in schedules will affect final schedules.

Do not expect miraculous recoveries for projects whose early work packets slip out of control.

Although so far the earned value method looks somewhat similar to ordinary cost vs. budget data, the additional rigor and the added dimension of careful planning has made the earned value approach a standard for defense projects, and useful for other kinds of projects as well. Essentially, the earned value approach leads to more careful up-front planning than is normal with software projects, and with better progress and cost tracking too.

A web search on the phrase "earned value metrics" in mid-2007 turned up about 1,200,000 citations. The topic is obviously one of considerable importance, and especially so in the defense sector where EVM is a requirement.

Goal Question Metrics (GQM)

The goal-question metric approach (GQM) was developed by the well-known software engineering research Dr. Victor Basili and his colleagues from the University of Maryland. The GQM operates in a hierarchical manner beginning with major business goals at the top level, followed by questions that are relevant to those goals, and ending up with metrics that can be used to measure progress toward those goals.

An example of this hierarchy by Dr. Basili himself illustrates the salient points. The business goal might be stated as "improve the speed of change request processing." A relevant question to this goal would be, "what is the current speed of processing change requests?" Metrics that might be used to ascertain progress toward the goal could include "average cycle time" and "standard deviation of cycle times."

Perhaps the most important aspect of the GQM approach is that it provides a linkage between executive concerns and the specific metrics and methods of improving the activities that cause those concerns. The bottom rung of the GQM approach includes many standard metrics such as return on investment (ROI), accounting rate of return, productivity, quality, customer satisfaction, staff morale, warranty costs, market share, and a host of others. It would be possible to utilize function point metrics since they are appropriate for dealing with quite a few kinds of executive goals and questions.

What would be an interesting adjunct to the GQM method would be to link it to the North American Industry Classification System (NAICS) developed by the Department of Commerce. Since companies within a given industry tend to have similar business goals, developing common sets of metrics within the same industry would facilitate industry-wide benchmarks.

Having done consulting work in dozens of insurance companies, almost a dozen telecom companies, more than a dozen banks, more than 50 manufacturing companies, and about 15 state governments, it is obvious that organizations in the same general business sector typically have common goals and common issues. Therefore, it would be possible to develop standard packaged GQM metrics that would probably be useful throughout all of the companies in the same industry.

A web search on the keywords "goal question metrics" performed in late 2007 turned up 1,950,000 citations, so there is a great deal of business interest in the GQM approach.

Suggested Readings on Measures and Metrics

Albrecht, A. J. "Measuring Application Development Productivity," *Proceedings of the Joint SHARE, GUIDE, and IBM Application Development Symposium, October 1979.* pp. 83–92.

Boehm, Barry W. *Software Engineering Economics.* Englewood Cliffs, N.J.: Prentice Hall, 1981.

Garmus, David and D. Herron. *Function Point Analysis*. Boston: Addison Wesley Longman, 2001.

————. *Measuring the Software Process: A Practical Guide to Functional Measurement.* Englewood Cliffs, N.J.: Prentice Hall, 1995.

Grady, Robert B. and D.L. Caswell. *Software Metrics: Establishing a Company-Wide Program.* Englewood Cliffs, N.J.: Prentice Hall, Inc., 1987.

Howard, Alan, ed. *Software Metrics and Project Management Tools: Applied Computer Research.* Phoenix, AZ: ACR, 1997.

IBM Corporation. *DP Services Size and Complexity Factor Estimator.* DP Services Technical Council, 1975.

International Function Point Users Group. *IT Measurement.* Boston: Addison Wesley Longman, 2002.

Jones, C. *Programming Productivity—Issues for the Eighties.* IEEE Computer Society, Catalog No. 391, 1981, Revised 2nd edition, 1986.

Jones, C. "Measuring Programming Quality and Productivity." *IBM Systems Journal,* vol. 17, no. 1, 1978, pp. 39–63.

Jones, C. *Programming Productivity.* New York: McGraw-Hill, 1986.

Jones, Capers. *Conflict and Litigation Between Software Clients and Developers*. Burlington, MA: Software Productivity Research, 2003.

————. *Estimating Software Costs, Second Edition.* New York: McGraw-Hill, 2007.

————. "Sizing Up Software." *Scientific American*. December 1998, Vol. 279 No. 6; pp. 104–109.

————. *Software Assessments, Benchmarks, and Best Practices.* Boston: Addison Wesley Longman, 2000.

————. *Software Systems Failure and Success*. Boston: International Thomson Computer Press, 1996.

Kan, Stephen H. *Metrics and Models in Software Quality Engineering, Second Edition.* Boston: Addison Wesley Longman, 2003.

Kaplan, Robert S. and D. B. Norton. *The Balanced Scorecard*. Cambridge, MA: Harvard University Press, 1996.

————. and D.B. Norton. *Strategy Maps: Converting Intangible Assets into Tangible Outcomes.* Boston: Harvard Business School Press, 2004.

Miller, Sharon E. and G. T. Tucker. *Software Development Process Benchmarking*. New York: IEEE Communications Society, 1991. (Reprinted from IEEE Global Telecommunications Conference, December 2–5, 1991.)

Pohlen, Terrance L. "Supply Chain Metrics." *International Journal of Logistics Management*. Vol. 2, No. 1, 2001; pp. 1–20.

Putnam, Lawrence H. *Measures for Excellence: Reliable Software on Time, Within Budget*. Englewood Cliffs, N.J.: Yourdon Press - Prentice Hall, 1992.

————. and W. Myers. *Industrial Strength Software: Effective Management Using Measurement.* Los Alamitos, CA: IEEE Press,1997.

This page intentionally left blank

Chapter

3

United States Averages for Software Productivity and Quality

The first version of *Applied Software Measurement* was published in 1991, and was based primarily on data gathered between the years 1985 and 1990, although it included older data too. The second edition of the book was published in 1996, and so was able to include almost five years of new findings. More than 2,000 new software projects were analyzed by the author and his company between 1991 and 1996. The third edition has been published in 2008, so once again new data and new projects are available. In addition to more than 4,000 new projects, a number of new development methods have achieved popularity in the time between the second and third editions. Some of these (in alphabetical order) include Agile development, Crystal development, Extreme programming (XP), the Rational Unified Process (RUP), and Watts Humphrey's team software process (TSP) and personal software process (PSP) for development. The even newer service-oriented architecture (SOA) has also emerged between the second and third editions. For maintenance, the Information Technology Information Resource Library (ITIL) has introduced new service-oriented metrics. Also, maintenance workbenches and renovation of aging software has become popular, and maintenance outsourcing is increasing in numbers of agreements.

Although the volume and quality of data on software projects has improved between the first, second, and third editions, it is still far from perfect. Few projects have consistent, accurate, and complete data associated with them. As mentioned in both the first and second editions, software measurement resembles an archeological dig. One shifts and examines large heaps of rubble, and from time to time finds a significant artifact.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

Two significant changes in measurement technologies have occurred over the past ten years. One of these was the creation of the International Software Benchmarking Standards Group (ISBSG) in 1997. The ISBSG is a nonprofit corporation and the first major organization to make benchmark data available to the public.

Older benchmark companies primarily made their data available to clients or in books such as this one. However actual detailed project data from the ISBSG can be acquired in CD form and also in published form from their books and articles. The ISBSG is a good step in the direction of creating a very useful collection of valuable data for the software industry. Readers are encouraged to visit the ISBSG web site, www.ISBSG.org, to find out more.

The second and more problematic change over the past ten years has been the fragmentation of functional metrics into more than 20 variations, all inconsistent and most lacking conversion rules to other metrics. In fact, some function point groups are competing with the others.

As of 2008, the function point metric defined by the International Function Point Users Group (IFPUG) has been joined by metrics defined by the COSMIC function point organization, the Netherlands Software Metrics Association (NESMA), the older Mark II function point method, full function points, and by other metrics that are discussed in print but which seem to lack formal organizations such as web-object points, story points, and use case points. It is hard enough to create meaningful benchmarks using a single metric. Attempting to create meaningful benchmarks with data scattered over 20 or so different metrics, which lack conversion rules, is an extremely taxing and difficult challenge.

Fortunately for the third edition of this book, which uses the function point method defined by the International Function Point Users Group (IFPUG), data using this method remains the largest of any of the variations and is continuing to expand in usage.

Many important technological changes have occurred in the software industry over the past 15 years. Many of the changes have been helpful and beneficial, but other changes have been harmful.

Following are some of the beneficial changes in software technologies since the first edition in 1991:

- Development of Agile software development methods
- Expansion of the CMM and development of the CMMI
- Expansion of functional metrics into systems and embedded software
- Development of the TSP and PSP
- Development and expansion of Six-Sigma for Software
- Creation of the International Software Benchmarking Standards Group (ISBSG)

- Development of maintenance renovation services and tools
- Development of an orthogonal defect classification system
- Explosive growth in use of web-based applications
- Explosive growth in use of web development tools
- Expansion of the Information Technology Infrastructure Library (ITIL)
- Emergence of SOA
- Development of automated testing tools
- Development of wiki discussion and network groups
- Development of webinars to augment live conferences
- Increase in numbers of software engineering and software management books
- Development of high-speed pattern-matching for functional metrics
- Development of the Information Technology Metrics and Productivity Institute (ITMPI) as a portal to recent and important software topics

Following are some of the harmful changes in software technologies since the first edition in 1991:

- Abandonment of successful process improvements due to executive changes
- Failure of the Agile community to provide effective economic data
- Failure of the object-oriented community to provide effective economic data
- Continued usage of "lines of code" metrics even though this has proven to be inaccurate
- Development of numerous function point variants without conversion rules
- Development of informal metrics without standard definitions (i.e., story points)
- Decline in the usage of formal design and code inspections
- Decline in the usage of joint application design (JAD)
- Decline in customer support and service by commercial software groups
- Decline in number of software quality assurance (SQA) personnel
- Continued marginal quality levels of commercial software applications
- Refusal to provide size and quality data by commercial software vendors

- Diversion of software resources due to the Euro rollout and the Y2K problem
- Sparse quantitative data on maintenance and enhancement work
- Lack of effective metrics for data volumes, data quality, and data costs
- Constant creation of new programming languages (greater than 700 circa 2007)
- A plethora of certifications for software specialists, but no formal licensing
- Increased losses of U.S. software jobs due to offshore outsourcing

On the whole, the overall results of these changes have benefited software productivity and reduced software schedules in several software domains. The CMM, CMMI, TSP, and PSP have benefited many military and systems projects and also some outsourcing, commercial, and MIS projects. The Agile methods have benefited many MIS and web projects. The use of very powerful development tools and substantial volumes of reuse have benefited web projects as well.

However, in spite of many improvements, some companies and projects are not as good in 2008 as they were in 1995. Also, litigation for canceled or delayed outsource projects has increased between 1990 and 2008, in part because there are so many more outsourced projects today than there were 18 years ago.

The most troubling of the detrimental changes was the discovery that a significant number of companies that had successfully applied software process improvement programs in the early 1990s had abandoned them by 2008. This phenomenon is so surprising that it deserves a word of explanation. Three key factors seem to be the root cause of the abandonment of successful software processes:

- A change of executives and management, with the original sponsors of the improvements retiring or changing jobs
- Failure to measure the economic value of the improvements with enough rigor to convince the new executives to continue making them
- Lack of continuing education in proven software methods for new managers and technical employees

To a surprising degree, software process improvements seem to depend on the personalities and enthusiasm of the original sponsors and development teams. When they change jobs (sometimes receiving promotions as a result of their hard work), the next generation does not share the same vision, and hence takes what seems to be the path of least resistance and eliminates effective techniques such as formal

design and code inspections, formal testing, and sometimes formal quality assurance.

Software quality, on the other hand, has declined in some segments such as information systems. However, the quality of commercial software (i.e., the packages sold over the counter such as spreadsheets or word processors) improved and overall results improved between 1990 and 2000, but seem to be in decline in 2008. The huge ERP applications continue to be delivered with thousands of latent defects. New commercial software such as the 2007 editions of Microsoft Vista, Microsoft Office, the Norton antivirus and Internet products, and many other commercial packages are still delivered in unstable and buggy condition. Even worse, getting help appears to be more difficult circa 2008 than it was ten years ago. A great deal of customer support is now outsourced abroad, so one problem with customer service today is the lack of English as a native language by many customer support specialists. Several major vendors such as Microsoft and Norton have made speaking to a live customer support representative either difficult or expensive, or both.

Table 3-1 shows the approximate net changes in U.S. average productivity rates between 1990 and 2005. The data from 1990 and 1995 are from the first two editions of this book. The data for 2000 is taken from the author's book *Software Assessments, Benchmarks, and Best Practices,* which was published in 2000. The newer data for 2005 is derived primarily from more recent studies by the author, augmented by some data from his colleagues at Software Productivity Research, and from external sources such as the International Software Benchmarking Standards Group (ISBSG).

Note that the data shown for the year 2000 includes some anomalies, such as the reduction in productivity for outsourced projects compared to both 1995 and 2005. This is due to the dilution of effort caused by the Euro rollout in 1999 and the Y2K problem, both of which absorbed

TABLE 3-1 Approximate U.S. Productivity Ranges by Types of Applications (Data Expressed in Function Points per Staff Month)

Software Types	1990	1995	2000	2005
Web	–	12.00	15.50	23.00
MIS	7.20	7.80	9.90	9.40
Outsourced	8.00	8.40	7.80	9.70
Systems	4.00	4.20	5.80	6.80
Commercial	5.10	5.30	9.20	7.20
Military	2.00	1.80	4.20	3.75
Average	**4.38**	**6.58**	**8.73**	**9.98**

thousands of outsource personnel and left many regular projects short-handed.

Table 3-1 illustrates projects that were put into production in calendar year 1990, 1995, 2000, and 2005. By limiting the results to a single year at five-year intervals, it is easier to see the rate of progress. Unfortunately, averages tend to be deceptive no matter how the information is displayed since the ranges are so broad. Readers can expect the best projects to be more than twice as good as the results in Table 3-1, and the worst projects would be only about half as good.

As the software industry continues to evolve, the overall results of positive and negative factors are hard to predict, but hopefully we will see continued improvements well into the middle of the 21st century.

Sources of Possible Errors in the Data

Readers should be aware that this study has the potential for a high error content. The raw data, although better than that available for the first two editions, remains partial and inconsistent in respect to the activities included, and it is often known to be incomplete and even wrong (i.e., the widespread failure to measure unpaid overtime).

Many different companies and government data have been examined. There are no current U.S. or international standards for capturing significant data items, or for including the same sets of activities when projects are measured. For costs and financial data, there are not even any recognized, standard accounting practices for dealing with the overhead or burden rates that companies typically apply. This fact is troublesome within the United States and even more troublesome for international cost comparisons where there are major differences in compensation, where currency exchange rates must be dealt with and where burden rate variances can be many hundred percent.

Since the data has such as high potential for error, it is reasonable to ask why it should be published at all. The reason for publishing is the same as the reason for publishing any other scientific results based on partial facts and provisional findings. Errors can be corrected by subsequent researchers, but if the data is not published there is no incentive to improve it.

For example when the Danish astronomer Olaus Roemer first attempted to measure the speed of light his results only indicated 227,000 km/s, which differs from today's value of 299,792 km/s by more than 24 percent. However, prior to Roemer's publication in 1676, most scholars thought the speed of light was infinite. Even Roemer's incorrect results started scientists along a useful path.

It is hoped that even if the data shown here is later proven wrong, that its publication will at least be a step toward new and correct data that will benefit the overall software community.

There are three major kinds of error that can distort the results published here:

- Errors in determining the size of the projects
- Errors in determining the effort or work content applied to the projects
- Statistical or mathematical errors in aggregating and averaging the results

Let us consider the implications of these three sources of error in turn.

Sizing Errors

The data contained in this book consists of four major categories in descending order of precision: (1) Projects actually measured by the author or by consultants employed by the author's company; (2) Projects measured by clients of the author's company and reported to the author; (3) Legacy systems where function point sizes were backfired by direct conversion from lines of code (LOC) data; (4) Projects collected from secondary sources such as the ISBSG database, from the Software Engineering Institute or from the general software literature.

In the first edition, about 75 percent of the data was originally recorded in terms of "lines of code" and the function point totals were derived via backfiring. In the second and third editions, the distribution among the four sizing methods is approximately the following: (1) About 30 percent of the size data was measured by Software Productivity Research (SPR) personnel who are certified function point counters; (2) About 15 percent of the data came from SPR clients who asserted that the sizes were derived using certified function point counters; (3) About 35 percent of the size data was derived from backfiring or direct conversion from the original LOC counts; (4) About 20 percent of the data was derived from secondary sources, such as studies reported to the International Function Point Users Group (IFPUG), the International Software Benchmarking Standards Group (ISBSG), or published in the function point literature.

The actual sizes of the projects used to produce this book varied, and possibly varied significantly. The normal precision of function point counting when performed by certified counting personnel is about plus or minus 3 percent. The normal precision of "backfiring" is only about plus or minus 25 percent, although exceptional cases can vary by more than 100 percent.

For very large projects in excess of 100,000 function points, a new method based on pattern-matching and mathematical analysis was developed by the author and is used for some large applications. This method is still

under development, and although it appears to yield sizes that come within about 15 percent of projects that have been measured using normal function point analysis, there are no actual projects above about 35,000 function points that have yet been sized by means of actual function point counting. The reasons are the high cost and long timespans that would be required.

The mixture of data from five discrete sources means that the actual sizes of the projects could vary from the stated sizes by a significant but unknown value. Hopefully, although this is uncertain, the possible errors from the five sources are not all in the same direction and hence may partially cancel out.

Effort, Resource, and Activity Errors

The second source of error concerns possible mistakes in collecting the effort and costs associated with software projects. It is a regrettable fact that most corporate tracking systems for effort and costs (dollars, work hours, person months, etc.) are incorrect and manage to omit from 30 percent to more than 70 percent of the real effort applied to software projects. Thus most companies cannot safely use their own historical data for predictive purposes. When the author or SPR personnel go on site and interview managers and technical personnel, these errors and omissions can be partially corrected by interviews. For secondary data, the errors are not so easily corrected.

The most common omissions from historical data, ranked in order of significance, include the following:

Sources of Cost Errors	Magnitude of Cost Errors
1. Unpaid overtime by exempt staff	(up to 25% of reported effort)
2. Charging time to the wrong project	(up to 20% of reported effort)
3. User effort on software projects	(up to 20% of reported effort)
4. Management effort on software projects	(up to 15% of reported effort)
5. Specialist effort on software projects: Human factors specialists Database administration specialists Integration specialists Quality assurance specialists Technical writing specialists Education specialists Hardware or engineering specialists Marketing specialists	(up to 15% of reported effort)
6. Effort spent prior to cost tracking start up	(up to 10% of reported effort)
7. Inclusion/exclusion of non-project tasks: Departmental meetings Courses and education Travel	(up to 5% of reported effort)
Overall Error Magnitude	(up to 110% of reported effort)

Not all of these errors are likely to occur on the same project, but enough of them occur so frequently that ordinary cost data from project tracking systems is essentially useless for serious economic study, benchmark comparisons between companies, or baseline analysis to judge rates of improvement.

A more fundamental problem is that most enterprises simply do not record data for anything but a small subset of the activities actually performed. In carrying out interviews with project managers and project teams to validate and correct historical data, the author and the consulting staff of SPR have observed the following patterns of incomplete and missing data, using the 25 activities of the standard SPR chart of accounts as the reference model:

Activities Performed	Completeness of Historical Data
01 Requirements	Missing or incomplete
02 Prototyping	Missing or incomplete
03 Architecture	Incomplete
04 Project planning	Incomplete
05 Initial analysis and design	Incomplete
06 Detail design	Incomplete
07 Design reviews	Missing or incomplete
08 Coding	Complete
09 Reusable code acquisition	Missing or incomplete
10 Purchased package acquisition	Missing or incomplete
11 Code inspections	Missing or incomplete
12 Independent verification and validation	Complete
13 Configuration management	Missing or incomplete
14 Integration	Missing or incomplete
15 User documentation	Missing or incomplete
16 Unit testing	Incomplete
17 Function testing	Incomplete
18 Integration testing	Incomplete
19 System testing	Incomplete
20 Field testing	Incomplete
21 Acceptance testing	Missing or incomplete
22 Independent testing	Complete
23 Quality assurance	Missing or incomplete
24 Installation and training	Missing or incomplete
25 Project management	Missing or incomplete
Total project resources, costs	Incomplete

When the author or SPR personnel collect data, at least we ask the managers and personnel to try and reconstruct any missing cost elements. Reconstruction of data from memory is plainly inaccurate, but

it is better than omitting the missing data entirely. Unfortunately, the bulk of the software literature and many historical studies only report information to the level of complete projects, rather than to the level of specific activities. Such gross "bottom-line" data cannot readily be validated and is almost useless for serious economic purposes.

For the seven software sub-industries or domains included in this edition, the overall performance in terms of resource and cost tracking accuracy is as follows:

Software Sub-industry	Resource and Cost Tracking Accuracy
Military software	Most complete cost data, although unpaid overtime is often missing. Usually has the most detailed activity costs.
Systems software	Fairly complete data, although unpaid overtime is often omitted. Management effort and some specialist effort may be omitted. Activity data is often good for testing and quality control.
Outsource/contract software	Fairly complete cost data, although unpaid overtime is often omitted. Seldom any real granularity in terms of distribution of effort by activity, although most activities are captured.
Commercial software	Fairly complete in large companies such as Microsoft. Cost data may not exist at all for small independent software vendors.
Management Information Systems	Woefully incomplete with many major errors and omissions, among them are unpaid overtime, management costs, user costs, and many specialists such as quality assurance (if any) or database administration.
Web software	Incomplete with major omissions, such as unpaid overtime, user costs, and costs of web content. Web content is outside the scope of normal software measurement and is not part of the software itself. However the costs can be much greater than the software costs.
End-User software	Usually no cost or resource data of any kind. Only the fact that developers of end user applications can reconstruct their time from memory makes any conclusions in this domain possible at all.

The primary impact of the missing and incomplete data elements is to *raise apparent productivity* and hence cause results to seem much better than they truly are. For example, SPR and a benchmarking competitor were both commissioned to study the productivity levels of the same corporation and compare them against our respective benchmarks within the same industry (financial services). The SPR benchmark for that industry averaged about 8 function points per staff month, but the other company's

benchmark for the same industry averaged more than 20 function points per staff month.

The reason for the difference was that the SPR data had used on-site interviews to correct and compensate for missing effort that was not part of normal cost-tracking system results. The competitive benchmark database, on the other hand, was based on surveys distributed by mail. There was no mechanism for validating the reported results by means of on-site interviews with project personnel to correct for missing or incomplete data elements. Since cost and resource tracking systems routinely omit from 30 percent to more than 70 percent of the total effort devoted to software projects, the apparent results can easily top 20 function points per staff month.

In real life there are many projects that can top 20 function points per staff month, and a few even top 100 function points per staff month. However, any project larger than 1,000 function points that reports productivity rates higher than 10 function points per staff month needs to be validated before the data can be accepted as legitimate. For Agile projects, applications larger than 1,000 function points that are higher than 15 function points per staff month need to be validated.

Given that the majority of activities in corporate tracking systems are missing or incomplete, the question arises as to just what value tracking systems have to American businesses. When used for cost control purposes, many tracking systems are so inaccurate that they seem to have no positive business value at all. Indeed, they are a source of major cost and schedule overruns because they provide such inaccurate data that when project managers attempt to use the data to predict new project outcomes, they place their projects in jeopardy.

Software Schedule Ambiguity Schedule data is also very troublesome and ambiguous. Considering how important software schedules and time to market considerations are, it is surprising that this topic has had very little solid, hard data in print for the past 50 years.

Even more surprising is the fact that so few companies track software development schedules, since this topic is visibly the most important to software managers and executives.

Failure to measure software schedules means that at least 85 percent of the software managers in the world jump into projects with hardly a clue as to how long they will take. This explains the phenomenon that about half of the major disasters associated with missed schedules and overruns can be traced back to the fact that the schedules were not competently established in the first place.

From collecting schedule information on historical projects, it happens that establishing the true schedule duration of a software project is one of the trickiest measurement tasks in the entire software domain.

When a software project ships is often fairly clear, but when it originated is the hardest and most imprecise data point in the software industry.

For most software projects, there is an amorphous and unmeasured period during which clients and software personnel grope with fundamental issues such as whether to build or buy, and what is really needed. Of several thousand software projects analyzed over the past 20 years by the author and his colleagues, less than 1 percent had a clearly defined starting point. To deal with this ambiguity, our pragmatic approach is to ask the senior software manager for a project to pick a date as the starting point, and simply use that. If the project had formal, written requirements, it is sometimes possible to ascertain the date that they started, or at least the date shown on the first printed version.

Surprisingly, more than 15 percent of software projects are also ambiguous as to when they were truly delivered. Some of the sources of ambiguity are whether to count the start of external beta testing as the delivery point or wait until the formal delivery when beta testing is over. Another source of ambiguity is whether to count the initial delivery of a software product, or whether to wait until deferred functions come out a few months later in what is usually called a "point release" such as "Version 1.1."

For determining the delivery of software, our general rule is to count the first formal release to paying customers or to users who are not participants in beta testing or prerelease field testing.

Not only are both ends of software projects ambiguous and difficult to determine, but the middle can get messy too. The confusion in the mid portion of software projects is because even with the "waterfall model" of development there is always overlap and parallelism between adjacent activities. As a general rule of thumb, software requirements are usually only about 75 percent defined when design starts. Design is often little more than 50 percent complete when coding starts. Integration and testing can begin when coding is less than 20 percent complete. User documentation usually starts when coding is about 50 percent done, and so forth. Due to overlap and parallelism, the end-to-end schedule of a software project is never the same as the duration of the various activities that take place.

The newer Agile, spiral, and iterative models of software development are even more amorphous, since requirements, design, coding, testing, and documentation can be freely interleaved and take place concurrently.

The schedule interval in this book runs from the nominal "start of requirements" until the "first customer ship" date of the project. The activities included in this interval are those of the 25 standard activities that SPR utilizes that are actually performed.

Software Cost Ambiguity Although productivity measurements based on human effort in terms of work hours or work months can now be measured with acceptable precision, the same cannot be said for software costs and prices as well.

There are several major problems in the cost domain that have existed for more than 50 years, but which escaped notice so long as software used inaccurate metrics like lines of code that tended to mask a host of other important problems.

These same problems occur for other industries besides software, incidentally. They tend to be more troublesome for software than for most other industries because software is so labor intensive.

A fundamental problem with cost measures is the fact that salaries and compensation vary widely from job to job, worker to worker, company to company, region to region, industry to industry, and country to country.

For example, among SPR's clients in the United States the basic salary of "senior systems programmer engineers" averages about $83,000 per year, and ranges from a low of about $45,000 per year to a high of over $115,000 per year.

When international clients are included, the range for the same position runs from less than $10,000 per year to more than $125,000 a year. The more expensive countries such as Belgium, Sweden, and Switzerland can average from 150 percent to 170 percent of United States norms. Countries such as Germany and Austria may be roughly equivalent to slightly higher than U.S. norms. However, countries such as China, India, and Russia may be as little as 15 percent to 20 percent of U.S. norms.

Industries vary too. For software executives such as chief information officers (CIO), the banking industry pays more than twice as much (sometimes more than $500,000) as public utilities or education ($90,000) but all industries pay more than government service.

There are also major variations associated with the sizes of companies. As a rule, large companies pay more than small companies. However, some small startup companies offer equity to founding employees, and that can translate into substantial money that is difficult to evaluate. (Microsoft is a famous example, since their employee stock option plan created more millionaires than any other company in history.)

Geographic regions vary too. Major urban areas such as New York and San Francisco have compensation packages that are about 30 percent higher than rural areas in the West and South.

Other software-related positions have similar ranges. This means that in order to do software cost studies it is necessary to deal with geographic differences in costs, industry differences in costs, size of company differences, and a number of other complicating factors.

Table 3-2 shows some of the author's findings on software positions and typical compensation packages for the United States for the year 2007.

Note that many companies offer employees stock option plans. The "equity" column in Table 3-2 refers to significant options whose cash value might very well exceed a year's compensation when exercised.

Most of the higher-paying software positions are managerial or executive in nature. However, there are a few technical jobs that have rather good compensation packages associated with them. These are positions such as "chief scientist" and "software architects" and the newer "Scrum master" position. Normally positions such as these are found in companies

TABLE 3-2 Approximate 2007 U.S. Software Compensation for Selected Positions

	Salary	Bonus	Total	Equity
CEO of a software company	$200,000	$40,000	$240,000	Yes
VP of Software Engineering	$190,000	$25,000	$215,000	Yes
Chief Information Officer (CIO)	$190,000	$25,000	$215,000	Yes
Director of a Software Lab	$175,000	$15,000	$190,000	Yes
VP of Software Quality Assurance	$170,000	$15,000	$185,000	Yes
3rd Line Software Manager	$165,000		$165,000	Yes
2nd Line Software Manager	$160,000		$160,000	
1st Line Software Manager	$145,000		$145,000	
Average	$174,375		$189,375	
Chief Scientist (software)	$170,000	$15,000	$185,000	Yes
Software architect	$165,000	$10,000	$175,000	
Scrum master	$145,000	$10,000	$155,000	
Senior systems programmer	$110,000		$110,000	
Senior web developer	$90,000		$90,000	
Senior systems analyst	$85,000		$85,000	
Systems programmer	$80,000		$80,000	
Systems or business analyst	$75,000		$75,000	
Application programmer/ analyst	$65,000		$65,000	
Testing specialist	$60,000		$60,000	
Software quality specialist	$55,000		$55,000	
Software metrics specialist	$55,000		$55,000	
Software technical writer	$55,000		$55,000	
Average	**$93,077**		**$95,769**	

that have major research laboratories such as AT&T's Bell Labs or IBM's Research Division. There are usually very few incumbents in these senior technical positions, but the fact that they exist at all is an interesting phenomenon.

Another and equally significant problem associated with software cost studies is the lack of generally accepted accounting practices for determining the burden rate or overhead costs that are added to basic salaries to create a metric called "the fully burdened salary rate" that corporations use for determining business topics such as the charge-out rates for cost centers. The fully burdened rate is also used for other business purposes such as contracts, outsource agreements, and return on investment (ROI) calculations.

Some representative items that are included in burden or overhead rates are shown in Table 3-2.1, although the data is hypothetical and generic.

TABLE 3-2.1 Generic Components of Burden or Overhead Costs

Large Company			Small Company		
Average Annual Salary	$75,000	100.00%	Average Annual Salary	$65,000	100.00%
Personnel Burden			**Personnel Burden**		
Payroll taxes	$7,500	10.00%	Payroll taxes	$6,500	10.00%
Bonus	$7,500	10.00%	Bonus	$0	0.00%
Benefits	$7,500	10.00%	Benefits	$3,250	5.00%
Profit sharing	$7,500	10.00%	Profit sharing	$0	0.00%
Subtotal	$30,000	40.00%	*Subtotal*	$9,750	15.00%
Office Burden			**Office Burden**		
Office rent	$25,000	20.00%	Office rent	$12,000	10.00%
Property taxes	$3,750	5.00%	Property taxes	$1,300	2.00%
Office supplied	$3,000	4.00%	Office supplied	$1,300	2.00%
Janitorial service	$1,500	2.00%	Janitorial service	$1,300	2.00%
Utilities	$1,500	2.00%	Utilities	$1,300	2.00%
Subtotal	$34,750	33.00%	*Subtotal*	$17,200	18.00%
Corporate Burden			**Corporate Burden**		
Information systems	$10,000	10.00%	Information systems	$0	0.00%
Finance	$7,500	10.00%	Finance	$0	0.00%
Human resources	$7,500	8.00%	Human resources	$0	0.00%
Legal	$6,000	6.00%	Legal	$0	0.00%
Subtotal	$31,000	34.00%	*Subtotal*	$0	0.00%
Total Burden	$95,750	107.00%	**Total Burden**	$26,950	41.46%
Salary + burden	$170,750	227.67%	*Salary + burden*	$91,950	141.46%
Monthly rate	$14,229		Monthly rate	$7,663	

The components of the burden rate are highly variable from company to company. Some of the costs included in burden rates can be: social security contributions, unemployment benefit contributions, various kinds of taxes, rent on office space, utilities, security, postage, depreciation, portions of mortgage payments on buildings, various fringe benefits (medical plans, dental plans, disability, moving and living, vacations, etc.), and sometimes the costs of indirect staff (human resources, purchasing, mail room, etc.)

Some former components of the burden rate are being discontinued by companies in the modern era of downsizing and cost reduction. For example, ten years ago many large corporations offered moving and relocation packages to new managerial and senior technical employees that included features such as

- Payment of real-estate commissions
- Moving fees for household goods
- Settling-in allowances
- Hotels or rent while searching for new homes

These moving and living packages are now in decline, and some companies no longer offer such assistance at all.

As can be seen from the right side of Table 3-2.1, small companies that are self-funded may have burden or overhead rates that are only a fraction of the amount shown in Table 3-2. On the other hand, some corporations can have burden rates that top 250 percent because they allocate their whole cost of doing business into the burden; i.e., travel, marketing, sales, can also be apportioned as part of this structure.

One of the major gaps in the software literature as of 1995, and for that matter in accounting literature as well, is the almost total lack of international comparisons of the typical burden rate methodologies used in various countries. So far as can be determined, there are no published studies that explore burden rate differences between countries such as the United States, Canada, India, the European Union countries, Japan, China, etc.

Among the author's clients, the range of average burden rates runs from a low of perhaps 15 percent of basic salary levels (for start-up companies operating out of the owner's homes) to a high of approximately 300 percent. In terms of dollars, that range means that the fully burdened compensation rate for a senior software engineer in the United States can run from a low of under $50,000 per year to a high of $250,000 per year.

There are no apparent accounting or software standards for what constitutes an "average" burden rate. Among SPR's clients, several companies include the entire costs of operating their businesses as part of their burden rate structure. For these two, their fully burdened monthly costs are in excess of $25,000. This is close to five times greater than

the actual average compensation levels, and reflects the inclusion of all indirect personnel, buildings, medical costs, taxes, utilities, and the like.

When the combined ranges of basic salaries and burden rates are applied to software projects in the United States, they yield about a 5 to 1 variance in costs for projects where the actual number of work months or work hours are identical! When the salary and burden rate ranges are applied to international projects, they yield about a 15 to 1 variance between countries such as India or Pakistan on the low end, and Germany or Switzerland or Japan on the high end.

Keep in mind that this 15 to 1 range of cost variance is for projects where the actual number of hours worked is identical. When productivity differences are considered too, there is more than a 100 to 1 variance between the most productive projects in companies with the lowest salaries and burden rates and the least productive projects in companies with the highest salaries and burden rates.

Expressed in terms of cost per function point, the observed range for software projects spans amounts that run from less than $100 per function point on the low end to more than $10,000 per function point on the high end. Although it is easily possible to calculate the arithmetic mean, harmonic mean, median, and mode of software costs, any such cost value would be dangerous to use for estimating purposes when the ranges are so broad. This is why the author is reluctant to publish general software cost data and instead uses work hours or person months for productivity studies.

Two other cost matters also are important. For long-term projects that may span several years, inflation rates need to be dealt with. Many commercial estimating and measurement tools, such as the older CHECKPOINT or newer KnowledgePlan tools that the author's company markets, have inflation rate adjustments as standard features.

For international software projects, currency conversions must be dealt with. Here too, commercial software cost-estimating tools such as CHECKPOINT or KnowledgePlan may have built-in currency conversion features. However, fluctuations in exchange rates are essentially daily topics and are not stable over long periods.

When clients ask for data on "average cost per function point" the only safe answer is that costs vary so much due to compensation and burden rate differences and to inflation that it is better to base the comparison on work effort. Cost data is much too variable for casual comparisons. Cost comparisons are possible, but they need a lot of preparatory work before they can be accurate.

Software Work Period Ambiguity One of the chronic problems of the software industry has been ambiguity in the various work periods such as work days, work weeks, work months, and work years applied to software projects.

The calendar year is 365.25 days long, and there are 52 weeks and 12 calendar months each year. These are essentially the only constant, unambiguous values associated with software work periods, and even these are not really constant due to leap years occurring every four years. In fact, given the slight irregularity in the earth's revolution around the sun, eventually even the length of a solar year might change.

Consider calendar year 2008 in terms of weekdays and holidays since 2008 is a leap year:

Months	Days	Weekdays	U.S. Public Holidays
January	31	23	2
February	29	21	1
March	31	21	0
April	30	22	1
May	31	23	1
June	30	20	0
July	31	23	1
August	31	22	0
September	30	21	1
October	31	23	1
November	30	21	1
December	31	22	1
Totals	**366**	**262**	**10**

Since there will be 366 calendar days and 262 weekdays in 2008, the average work month will be slightly longer than a non-leap year and will consist of 21.83 work days. Of course we don't really work every weekday, and this is where the ambiguity begins.

In the United States, the normal work week runs from Monday through Friday, and consists, nominally, of five 8-hour days, often cited somewhat humorously as "9 to 5." However, because lunch periods and coffee breaks are standard in the United States, many companies compensate for this by having nine-hour business days, such as opening at 8:30 AM and closing at 5:30 PM.

Although we may be physically at work for eight or nine hours a day, very few of us can or would want to work solidly for eight hours every day. The exact amount of time used for coffee breaks, lunch, rest breaks, and social matters varies from company to company, but can be assumed to average about two hours per day.

If we assume that we are in fact working for six hours each day, and that there will be 22 working days each month, then the monthly total would be 132 work hours each month. If we assume that we are going to work seven hours each day, the total jumps to 154 work hours each month.

However, the actual amount of work time is a very ambiguous quantity and simple assumptions are not accurate. Let us consider some of the reasons for variance.

Since there are 52 weeks in a year, that indicates 104 Saturdays and Sundays, leaving a residue of 261 business days when we subtract 104 from 365.

At this point, by dividing 261 days by 12 months, there would be an average of 21.75 working days each month. If you round 21.75 up to 22, and multiply by 6 work hours each day, you arrive at the value of 132 hours each month, which is a common default value used for rough cost and effort estimating purposes.

However, there are also national and some state public holidays, vacation days, sick days, special days away due to weather, and non-work days for events such as travel, company meetings, and the like. This is where the bulk of the ambiguity arises.

The major public holidays where business offices are typically closed in the United States total to about 10 days each year. This includes both national holidays such as the Fourth of July and Veterans Day and state holidays, which vary from state to state.

In the United States, the entry-level vacation periods are normally only 5 to 10 days per year. For journeymen with more than five years of work experience, the average is about 15 days per year. There are also senior personnel with extensive longevity who may have accrued vacation periods of 20 days per year. Let us assume that vacations average 15 days per year in the United States.

Vacations are difficult to deal with when estimating software projects. On "crunch" projects vacations may be deferred or prohibited. Even during ordinary projects, vacations may be temporarily put on hold until the project is over, or at least until the portions of the project are stable. Therefore vacations tend to come between projects and assignments, rather than in the midst of them. However, on large-scale multi-year projects vacations must be considered.

Summing both 10 public holidays and 15 vacation days and subtracting them from an annual 261 business days yields 236 days remaining. At this point, there would be an average of 19.66 working days per month. Rounding 19.66 up to 20 and multiplying by 6 hours a day yields a result of 120 work hours per month.

It is also necessary to consider sick leave, and days when companies are closed due to conditions such as snow storms or hurricanes. Let us assume an average of three sick days per year, and two days for unscheduled office closures due to weather conditions. Subtracting five more days brings us down to 231 working days in a year, which is equivalent to 19.25 working days per month. Rounding 19.25 down to 19 and multiplying by 6 hours each day yields a result of 114 work hours per month.

There are also days devoted to education, company meetings, appraisals, interviewing new candidates, travel, and other activities that are usually not directly part of software projects. These extraneous activities are difficult to generalize. However, let us assume that they total to 16 days per year. This now reduces the number of work days down to 215 per year. Dividing by 12 months, the number of work days per month is now 17.9. Rounding this to 18 and multiplying by 6 hours a day, the number of nominal work hours per month would be only 108.

Another source of great ambiguity is the amount of unpaid overtime applied to software projects. Software personnel, as a class, are a pretty hard-working group and unpaid overtime is very common. For example, an average of about 60 minutes every night during the week and half a day or four hours every weekend is a common value for unpaid overtime.

For "crunch" projects, vacations and some holidays may be suspended. There is also a tendency to work harder and take fewer breaks during the day. This means that projects under severe schedule pressure might accrue 7.5 hours of regular time and 2 hours of unpaid overtime during the week. On weekends, another 12 hours of unpaid overtime might be worked. Thus for "crunch" projects there might be as many as 165 regular work hours each month, and a total of perhaps 88 hours of unpaid overtime. This would amount to a combined value of 253 work hours per month. This is a difference of 121 hours or 92 percent compared to the nominal 132 hours per month that is frequently used.

The range of the calculations shown here indicate that the number of work hours per month in the United States can vary from a low of 108 to a high of 253. While intervening values such as 120 or 132 can be used as rough assumptions for estimation, the ambiguity of the situation is large enough so that "averages" are bound to be wrong and misleading. SPR and the author recommend a careful analysis of the available work periods for each project, rather than simply utilizing default or average values.

Table 3-3 lists some observations on the normal patterns of work time and unpaid overtime for six of the sub-industries covered in this book.

TABLE 3-3 Representative Software Work Hours per Day in Six Sub-Industries

	Work Hours per 8-Hour Day	Unpaid Overtime per Day	Total Work Hours per Day
End-user	3.5	0.0	3.5
MIS	5.5	1.0	6.5
Outsource	7.0	1.5	8.5
Commercial	7.0	2.0	9.0
System	6.0	1.0	7.0
Military	6.5	1.0	7.5
Average	**5.9**	**1.1**	**7.0**

With so much variability, it is easy to see why SPR recommends exploring the specifics of real software projects and attempting to match the actual pattern, rather than depending upon abstract averages.

It is interesting to consider the same kind of data for two new forms of software projects: Agile projects and web projects:

	Work Hours Per 8-Hours Day	Unpaid Overtime per Day	Total Work Hours per Day
Agile	7.0	3.0	10.0
Web	7.0	1.5	8.5

The Agile approach seems to attract a young and very energetic set of developers, who bring with them a rather intense work ethic. A part of the productivity improvements from the Agile methods are based on the technologies themselves, and part are based on old-fashioned hard work and a lot of unpaid overtime.

When international data is considered, variations in work periods are large enough to require careful handling. For example, the Japanese work week normally includes five and a half days, and much more overtime than U.S. norms; i.e., it runs to almost 50 hours a week.

The Canadian work day fluctuates between summer and winter, and is only about 7.5 hours as opposed to the U.S. norm of 8 hours and also has somewhat less unpaid overtime than U.S. or Japanese norms.

European vacations are much longer than U.S. norms and may average more than 20 days. On the other hand, vacations in Mexico are shorter than U.S. norms and average less than 10 days. Variations such as these require careful handling for international comparisons to be valid.

When examining the literature of other scientific disciplines such as medicine, physics, or chemistry, about half of the page space in journal articles is normally devoted to a discussion of how the measurements were taken. The remaining space discusses the conclusions that were reached. The software engineering literature is not so rigorous, and often contains no information at all on the measurement methods, activities included, or anything else that might allow the findings to be replicated by other researchers.

The situation is so bad for software that some articles even in refereed journals do not explain the following basic factors of software measurement:

- Which activities are included in the results; i.e., whether all work was included or only selected activities
- The assumptions used for work periods such as work months
- The programming language or languages used for the application

- Whether size data using lines of code (LOC) is based on
  - Counts of physical lines
  - Counts of logical statements
- Whether size data using function points is based on
  - IFPUG function point rules (and which version)
  - Mark II function point rules (and which version)
  - Other rules such as Boeing, SPR, DeMarco, IBM, or something else
- How the schedules were determined; i.e., what constituted the "start" of the project and what constituted the "end" of the project.

It is professionally embarrassing for the software community to fail to identify such basic factors. The results of this lack of rigor in measurement reporting leads to very wide apparent variances on top of very wide real variances.

These false variances based on miscounting, mis-sizing, or misstating can approach or exceed a 10 to 1 range. The real variances based on accurate and consistent measurements can also vary by about a 10 to 1 range. The overall permutations of these two ranges can lead to apparent productivity and quality differences of 100 to 1. This is far too broad a range to be acceptable for an occupation that aspires to professional status.

Statistical and Mathematical Errors

The statistical and mathematical methods used in the first edition of this book were mixed and not carefully controlled. That same criticism can be applied to this edition as well. The overall set of projects used in producing the data contained here is clearly biased. This is because the author and his company are management consultants rather than academics. We collect data when we are commissioned to do so by specific clients. This means that there are significant gaps in the data, and the sizes of applications we study tend to be much larger than the probable distribution in the United States as a whole. Table 3-4 shows the approximate numbers of software projects for this third edition.

Although the total number of projects may seem fairly large, it is actually only a very small sample of the overall numbers of software projects produced. Moreover, the sample is biased, as will shortly be discussed.

Over and above the projects examined by the author and his colleagues from their immediate clients and primary sources, another 5,000 or so projects from secondary sources such as the International Software Benchmark Standards Group (ISBSG), Software Engineering Institute (SEI), and other consulting companies have been considered. In particular, projects that use COSMIC function points, Mark II function points,

TABLE 3-4 Summary of Projects Used in the Third Edition

	Number	Percent
End-user	155	2%
Agile	125	2%
Web	120	1%
Systems	3,300	40%
MIS	3,000	36%
Commercial	650	8%
Outsource – U.S.	425	5%
Outsource – offshore	165	1%
Military	350	4%
Total	**8,290**	**100%**

story points, Use Case points, and other metrics that are not part of the author's consulting work have been derived from secondary sources.

Table 3-5 gives the author's hypothesis for the probable number of software projects produced in calendar year 2007 by U.S. companies.

A quick comparison of Tables 3-4 and 3-5 will show that the author's data is sparse on Agile projects, web projects, and offshore outsourced projects. On the other hand, the author's data for systems software is quite a large sample. The reason that the author's data is sparse for web, Agile, and offshore projects is because he has not been hired as a consultant or assessor for very many of these types of projects.

TABLE 3-5 Approximate Numbers of Projects Performed in 2007

	Number	Percent
End-user	2,500	2%
Agile	20,000	13%
Web	12,500	8%
Systems	8,750	6%
MIS	70,000	45%
Commercial	3,250	2%
Outsource – U.S.	17,500	11%
Outsource – offshore	15,000	10%
Military	7,750	5%
Total	**157,250**	**100%**

As can easily be seen, the author's and the SPR knowledge base under-represents every software domain except systems software in terms of relative percentages. The most severe under representation is the domain of end-user software, where SPR has never been commissioned to perform any formal assessments or benchmark studies. Only the fact that many of the author's personal friends and colleagues (and the author himself) write end-user applications provides any data at all. Of course, hardly anyone even cares about end-user software so this is not a serious issue. But a great many people care about Agile and web projects, so the shortage of data here is troubling. The fact of the matter is that the Agile community marches to the sound of a different drummer and to date has seldom used either function point metrics or measurement consultants.

The net results of the visible bias in the SPR knowledge base is that overall averages based on our knowledge base will probably appear to be *lower* than true national averages based on a perfect representation of software projects. Systems software is comparatively low in software productivity, whereas end-user applications are comparatively high in software productivity. Web and Agile projects are also high in productivity. If U.S. norms were calculated based on a more representative match of the distribution of projects, the probable results would be close to15 or 18 function points per staff month rather than the 9.98 function points per staff month shown in Table 3-1.

These biases are unfortunate. However, so far as can be determined there is no national database that is a perfect representation of either U.S. software patterns or those of any other country either.

For example, the 4,000 or so projects in the new database developed by the International Software Benchmarking Standards Group (ISBSG) probably has a distribution that more closely matches U.S. norms than the author's own data. However, the largest projects yet submitted to ISBSG are less than 20,000 function points in size, so there is a total absence of really large applications in the class of Microsoft Vista, SAP, major defense applications, and the other projects that top 300,000 function points in size.

There is a lesson to be learned and also a caution to readers: as of 2008 there are no truly accurate databases of software productivity and quality information anywhere in the world. To create a truly accurate database, we would need about five times more projects than have been measured to date, and we would need projects that range from less than 10 function points in size to more than 300,000 function points in size. The total number of projects should be at least 20,000 for a valid statistical sample that reflects the entire range of software projects in the United States.

A very relevant question is whether the sum total of the SPR knowledge base is even large enough to be statistically valid for a country the

size of the United States. The answer is that the SPR knowledge base is probably not large enough for statistical validity. Even so, if the results of the author's analysis were not published, there might be no incentive for other researchers to expand their knowledge bases or challenge and correct the author's conclusions.

So far as can be determined as of 2008, no known U.S., European, or Asian software knowledge base is probably valid either. It is even questionable if the sum of all major software knowledge bases in the entire world would be large enough to be statistically valid (i.e., the sum of the data maintained by the U.S. Air Force, by Gartner Group, by the ISBSG, by David Consulting Group, by IFPUG, by COSMIC, by NESMA, by Quantitative Software Management, by Howard Rubin Associates, by the Software Engineering Institute, and by Software Productivity Research.)

Age Ranges of the Software Productivity Research Knowledge Base The author and his colleagues are often asked about the ages of the projects in the SPR database. SPR began software productivity and quality data collection in volume in 1985. However, the author had been involved in software measurement work long before SPR was formed. He was part of various groups in IBM that collected productivity and quality data collection from 1968 through 1979. He was also part of the ITT corporate group that collected software quality and productivity data too from 1979 through 1983. He also worked at the consulting company of Nolan, Norton & Company during 1983 and 1984 in a consulting practice devoted to software data collection. In fact the oldest data actually comes from the early days of the computer industry, and the most recent data was collected earlier in the week that this paragraph was written in 2007.

Table 3-6 shows the approximate number of projects included in this book from the earliest to the most recent.

TABLE 3-6 Age Ranges of the Software Projects Cited

Time Period	Number of Projects	Percent of Total
1951–1960	100	1.20%
1961–1970	500	6.02%
1971–1980	1,000	12.05%
1981–1990	2,550	30.72%
1991–2000	3,350	40.36%
2001–2007	800	9.64%
Total	**8,300**	**100.00%**

Readers might be troubled by the comparatively small quantity of recent projects, but unfortunately that is a fact of life. It is difficult to examine more than about 200 projects in a calendar year, and this is true for all organizations that collect quantitative data such as SPR, the SEI, the David Consulting Group, and most others who gather data by means of onsite visits and interviews with project personnel.

The International Software Benchmark Standards Group (ISBSG) is able to collect data on more than 500 projects in a calendar year, but this is because the data is self-reported by clients and ISBSG does not do onsite interviews or data validation. The incoming questionnaires are screened for obvious errors in counting function points or incorrect answers to questions, but more subtle errors such as "leaks" from the clients' cost tracking systems probably go uncorrected. Since leakage is such a common problem (noted in more than 60 percent of corporations analyzed), the most likely result is that the productivity data reported to the ISBSG is probably somewhat higher than would actually be true if the missing data had been collected and supplied by clients.

Because of the large number of legacy applications contained in the SPR knowledge base, it is also of interest to consider the distribution of projects that are new, that consist of enhancements to existing applications, or are concerned with maintenance or defect repairs for existing applications. Table 3-7 gives the approximate overall distribution of the SPR knowledge base into these three categories.

It is obvious in 2007 that enhancements to existing software and maintenance work (fixing bugs) are the dominant activities for the software industry, and this trend will continue into the foreseeable future.

TABLE 3-7 Numbers of New, Enhancement, and Maintenance Projects

	New	Enhancement	Maintenance	Total
End-user	140	10	5	155
Agile	75	45	5	125
Web	70	45	5	120
Systems	750	1950	600	3,300
MIS	700	1700	600	3,000
Commercial	350	200	100	650
Outsource – U.S.	150	250	25	425
Outsource – offshore	65	5	5	75
Military	110	200	40	350
Total	**2,410**	**4,405**	**1,385**	**8,200**
Percent	29	54	17	100

As the software industry moves past the 60 year mark the number of legacy applications that are being enhanced or maintained will grow slightly larger every year.

Programming Languages Represented in the Data Another frequent question to the author concerns how the knowledge base reflects various programming languages of interest. Here there is far too much data to represent it all.

The author and his colleagues at Software Productivity Research developed and marketed the SPQR/20 estimating tool, the CHECKPOINT tool, and the newer KnowledgePlan software cost estimation tool.

From the beginning, one of our standard features was "support for all known programming languages." In order to back up this assertion, we have been producing an annual report of software programming languages since 1985. When this advertising claim was first made none of us at SPR knew exactly how many programming languages there were. For example, in 1985 I had thought that the world total of programming languages might be about 100. This turned out to be a major underestimation.

The initial version of the *Table of Programming Languages and Levels*, circa 1985, had about 150 programming languages. It quickly grew to more than 300 programming languages by 1990.

Version 15 of SPR's *Table of Programming Languages and Levels* is in preparation as this book is being written in 2008. Version 15 contains a total of almost 700 programming languages and dialects. As a general rule, new programming languages are created about once a month, and this statement has been true since 1985. If anything, the creation of new programming languages has accelerated since about 1990.

The topic of programming languages presents an unexplained mystery about software engineering. Since the author's study of programming languages began more than 25 years ago, new programming languages have been created at a rate of more than one per month. As of 2007, the total number of programming languages and dialects tops 700. The questions associated with this phenomenon are

- Why does the software industry need 700 programming languages?
- Will programming languages continue to be developed at a rate of greater than one per month?
- How many software applications use more than one programming language?
- What is the maximum number of programming languages in the same application?

- How many of the 700 programming languages are actually being used in 2007?
- How many of the 700 programming languages are as dead as Sanskrit?
- How do you maintain software written in dead programming languages?
- Is there any objective way for selecting the most suitable programming language?
- What is the actual impact of programming languages on productivity?
- What is the actual impact of programming languages on quality?

The author does not have answers to all of these questions, but the topics are important enough to discuss the answers that are actually known circa 2008.

First, the development of new programming languages seems to be based more on human psychology than on technical need. No programmer knows the capabilities of all 700 languages, so when he or she sees a need for some specialized programming feature it is easier to develop a new language than it is to explore the capabilities of existing languages. Also, being the developer of a new programming language brings with it a certain status in the software engineering world.

The topic of multiple programming languages in the same application is also interesting. As of 2008 about 50 percent of software applications use more than one programming language. Some common combinations include Java plus HTML or COBOL plus SQL for older applications. The maximum number of programming languages in the same application noted by the author is 12, but there may be applications with even more languages.

In its own way, the use of multiple languages in the same application is also puzzling. It is equivalent to going to a restaurant and ordering the appetizer in English, the main course in Japanese, and the dessert in French. The obvious technical reason for having more than one language is that most languages are optimized for a narrow range of functions, and if the application has a broad range of features, using more than one language is the probable result. In particular, queries and database access and text or graphics manipulation may utilize specialized languages.

Of the approximately 700 programming languages in the SPR table of languages, less than 200 are in actual use circa 2008 and almost 500 are "dead" languages or at least orphans. The most active programming languages circa 2008 include Ada, C, C#, C++, HTML, Java, and Visual Basic.

Dead languages provide a major problem for software maintenance. The average life expectancy of a large software application in the 10,000 function point size range tops 20 years. But the average life expectancy of many programming languages for development purposes is less than 15 years before they fall out of use. As a result, many large applications are difficult to maintain because of the lack of modern compilers combined with a lack of programmers who know the languages (or even want to know the languages).

Some examples of these "dead" languages that are hard to deal with circa 2008 include mumps, Jovial, the early versions of Basic, and the early versions of Algol. Even FORTRAN and COBOL, once two of the most widely used languages in the world; have declining numbers of available programmers circa 2008. They also have dead or dying variants that lack working compilers. This issue is partly met by companies and tools that can convert senile languages into more recent languages. For example, a company called Relativity Technologies can convert aging COBOL into Java.

The selection process for choosing programming languages for software is surprisingly subjective. The most common choices are simply the languages that are currently available to the company doing the development work. Occasionally there will be a deliberate analysis and selection of languages to match application needs, but the most common situation is that of just using the languages in which the organization's programmers have compilers and are already trained.

In the early days of software engineering when programs were small, the choice of a programming language exerted a tangible effect on both productivity and quality. Today in 2008 when many applications are large, the overall impact of programming languages on either productivity or quality is small. The programming language accounts for less than 10 percent of either productivity or quality results. As of 2008, programming is only the third most expensive activity out of the five major cost drivers:

- Finding and fixing defects
- Creating paper documents
- Programming
- Meetings and communications
- Project management.

For quality purposes, coding defects are the most numerous but also the easiest to find and eliminate due to sophisticated debugging tools, code inspections, and formal testing. The toughest kinds of defects are not in the code itself, but originate in requirements, design, and the

form of "bad fixes" or secondary defects that are introduced accidentally in the defect repairs themselves. By the time software is delivered to customers, more than 95 percent of coding bugs are usually eliminated, but less than 75 percent of requirement bugs and less than 80 percent of design bugs will have been found. Bad fix injections average about 7 percent, but sometimes top 20 percent.

What are important about languages are their structure and complexity. These are not intrinsic attributes of the languages themselves, but are based on individual programming styles. However it is possible to simplify complex code by either manual "refactoring" or by automated restructuring. Obviously the economics clearly favor automatic restructuring. In fact, all legacy applications that have strategic value and are likely to be used for another five years or so should undergo both complexity analysis and renovation, ideally using automated tools.

The total number of programming languages in projects discussed in this book totals to about 450 out of the world total of 700 or so. All of the common languages (Ada, APL, Algol, Abap, Assembler, Basic, C, C++, CHILL, COBOL, Forth, FORTRAN, HTML, Lisp, Natural, Objective C, Pascal, Perl, PL/I, Prolog, SMALLTALK, SQL, etc.) are represented. There are also some examples of newer languages such as C#, Haskell, PHP, and Ruby.

The legacy projects and data represented in this book also include a number of older languages that are either dead or orphans in 2008, due to lack of programmers, lack of working compilers or interpreters, or even a lack of hardware that the language can run on. Some of these include 1401 Auto coder, RM Cobol, GW Basic, Erlang, VAX ADE, Interpreted Basic, mumps, Simula, Jovial, turbo Pascal, and turbo Prolog.

There are also a few projects included that used special proprietary languages that were developed for a specific purpose by a specific company. One example is ESPL/I, a proprietary variant of PL/I developed by the ITT Corporation for electronic switching systems.

To conclude with the discussion of programming languages, the overall language situation in the software industry circa 2008 is about as strange and complicated as the biblical story of the Tower of Babel. In fact, surprisingly, the number of natural human languages and the number of programming languages are fairly close in number.

What the author recommends is a five-year cessation on developing new programming languages starting in the year 2010. During this five-year period a joint working group from industry, government, and academia would attempt to collect information on the number of applications in all active languages and also the number of legacy applications in dead languages. After this information is gathered, the next stage would be to create a national or international repository that

would contain the compilers and interpreters of all known languages; training manuals for all known languages, and an approximate census of the programmers who know these languages. Training materials for all languages should be converted to DVD or computerized form, so that they are readily available as needed.

In addition, the technical fields of complexity analysis and code restructuring should be made available to all orphan or dead languages that have a significant number of existing applications. This would need to be funded either by the government or by the owners of the applications. It cannot be expected that the software renovation companies would do this on a speculative basis. For example, both the State of New York and the U.S. Veterans Administration are maintaining large systems written in the mumps programming language. There are few programmers available who know this language and even fewer tools. It would be useful to be able to convert this language into a modern active language, but doing this by hand would be prohibitively expensive.

The purpose of this repository of languages, plus renovation, is to assist owners of major software applications that are written in orphan or dead languages. Since the life expectancy of large software applications is longer than the life of most programming languages, it can be anticipated that every single programming language in existence in 2008 will eventually become obsolete and pass away. Worse, the languages will become obsolete while many applications written in those languages are still actively utilized.

If the software industry continues to develop new programming languages in the future at the same rate they have been developed in the past, by the year 2050, we will be looking at close to 1,500 programming languages (with more than 1,200 being obsolete). By the end of the 21st century, we will be looking at more than 2,500 programming languages, with about 2,200 of these being either dead or orphan languages.

If the unabated proliferation of new programming languages does happen, which seems likely, then maintenance of legacy software is probably going to absorb about 85 percent of the world's total programming population, which will probably be at least three times greater than today's programming population. Very little new work will get done. The economics of the unchecked development of hundreds of programming languages growing to thousands of programming languages is likely to become catastrophic in a few years time.

A modern web application might require as many as 15 languages, including HTML, JavaScript, XML, C#, ASP, Shockwave, etc. Each of these languages deals with a narrow spectrum of the total needs for building and maintaining a web application. In general, neither the language developers nor the application developers pay much attention to downstream maintenance. As of 2008 most web applications and indeed

many web sites are so recent that long-range maintenance costs have not yet accrued. But in another ten years, it will be interesting to see what the maintenance costs are for Amazon, Google, AOL, etc., as the current languages die out and tools and compilers begin to disappear, while programmers skilled in the original languages retire or change jobs.

It should be recalled that the software industry has a higher percentage of failures than any other engineering discipline. Almost 50 percent of projects larger than 10,000 function points are canceled, and almost 100 percent of the remainder are late and over budget. A study of 600 Department of Defense software projects did not find a single project that was on time and within its budget. Software maintenance costs are also higher than many other engineered products, due to the huge number of latent defects delivered with software and the complexity of the applications themselves. The plethora of programming languages is merely one more indictment of software engineering as being a only a quasi-profession rather than a true profession built on a solid foundation of empirical information.

The task of creating a global repository and inventory of all known programming languages is so large and expensive that it could only be funded by the U.S. Department of Defense, by DARPA, or by a consortium of major companies such as IBM, Microsoft, EDS, etc. Major universities with large software engineering schools such as MIT, Carnegie Mellon, Harvard, the University of Florida, etc., should also participate.

Previous attempts to create a single universal programming language have failed. IBM was not successful with PL/I and the Department of Defense was not successful with Ada. The telecommunications industry was not successful with CHILL. All three languages are actually quite powerful and useful, but they were not able to overcome both social resistance and technical challenges to become truly universal.

Although a single universal language might be developed, that seems to be unlikely. What might be more easily achievable is to narrow down the set of programming languages from today's 700+ to perhaps 10. However even doing this will face both sociological and economic barriers.

If it were possible to narrow down the active programming languages to about ten, and if several hundred of the formerly popular dead and orphan languages were converted into this set of ten active languages, maintenance costs and effort would soon reverse the historic trend of increasing at about 3 percent to 5 percent per year, and would eventually decline at about the same rate.

However, it would also be necessary to slow the rate at which new programming languages are created (or even stop creating new languages). But the set of 10 active languages would have to be robust and useful enough so that they might last for 50 years or so instead of dropping

out of use in 15 years, which is the current norm. If these things could happen, then the effect on the software industry would be profound.

What would probably result would be a net reduction in overall maintenance work by more than 50 percent compared to 2008, coupled with an eventual decline in the total number of maintenance and enhancement software engineers on a global basis. Under this scenario of reduced creation of new languages plus conversion to a small set of active and stable languages, maintenance and enhancement work would soon drop below 50 percent of total employment. By the end of the 21st century maintenance and enhancement work would probably remain stable at about 12 percent to 15 percent of total software employment. This, of course, would free up about 4,000,000 maintenance software engineers on a global basis.

The bottom line is that the current situation, where hundreds of programming languages are created and quickly die out, is a major waste of money and resources. It is also professionally embarrassing since there are much more important problems that we should be addressing, such as the poor quality of many software applications and the tendency of major projects to be canceled.

The proliferation of programming languages to date has cost the software industry more than the Y2K problem and the Euro problem put together. In fact since software maintenance is one of the most labor-intensive occupations of the industrial era, a case can be made that the proliferation of programming languages is actually the most expensive business problem of the past 100 years.

The most surprising and alarming aspect of this problem is the fact that it has remained essentially invisible. There are many books on programming languages (hundreds in fact) but to date not one of these books has discussed the overall economic costs caused by the existence of hundreds of programming languages whose effective lives are less than 15 years before they stop being used for development. Another topic that lacks significant coverage in the software engineering literature is the cost of maintaining applications written in dead or orphan programming languages.

It is time for a change of viewpoint about the proliferation of programming languages. New programming languages are not part of the solution to software engineering problems; new programming languages are part of the problem faced by software engineers and especially by software engineers involved with maintenance and enhancements of legacy applications.

Software Technologies Represented in the Data The author is often asked about how the available software projects in the knowledge base relate to various technologies, such as Agile projects, service-

oriented architecture projects, client-server projects, or those using object-oriented methodologies, and so forth.

The data represented in this book is based on almost all of the major software development methods in use since about 1984 when Software Productivity Research was formed. Indeed, before the author founded Software Productivity Research, he was involved in process development work at IBM and ITT, so some data goes back as far as 1975.

In total, more than 100 software technologies are included in the data. Following are 30 of the key technologies represented in this book, in alphabetical order:

- Agile methods
- Assessments
- Capability maturity model (CMM)
- Capability maturity model integrated (CMMI)
- Code inspections
- Design inspections
- Extreme programming (XP)
- Information technology infrastructure library (ITIL)
- Joint application design (JAD)
- Object-oriented development (OO)
- Personal software process (PSP)
- Quality function deployment (QFD)
- Rapid application development (RAD)
- Rational unified process (RUP)
- Reengineering
- Refactoring
- Reusability
- Reverse engineering
- Scrum
- Service-oriented architecture (SOA)
- Six-Sigma for software
- Spiral development
- Stories
- Story points
- Team software process (TSP)

- TickIT
- Total quality management (TQM)
- Unified modeling language (UML)
- Use Cases
- Waterfall development

Here too, new information is coming in essentially on a daily basis. Unfortunately, the creation of development methods is almost as rapid as the creation of programming languages. Since data has been collected by the author since 1985, new development methods have been announced about once every two months. There is no end in sight. From the time the author turns the book over to the publisher at the end of 2007 until it is actually published in 2008, no doubt at least four and perhaps six new development methods will have surfaced.

The major concern for most readers of this book is how the various development methods compare in terms of productivity and quality. Table 3-8 provides an interesting but somewhat unreliable comparison of productivity rates for the most widely used current methods. The data in Table 3-8 is sorted in descending order for applications of 10,000 function points in size, which is the central column of the table. This is the size where failure and litigation are troublesome. It is also where rigorous development methods excel. Interestingly, hybrid methods appear to do an excellent job, such as Watts Humphrey's team software process (TSP) joined with Scrum, and CMM level 5 joined with Six-Sigma.

If the data had been sorted by the left column, or 1,000 function points, other methods such as Agile would have appeared on top.

The data in Table 3-8 is partly based on observations and partly on mathematical extrapolation from projects of different sizes. It has a high margin of error. The main value of Table 3-8 is the fact that it uses the same unit of measure for all of the methods.

The reason that the service-oriented architecture (SOA) has such high productivity rates is because applications using SOA are merely linked combinations of existing applications. If the original development of the SOA pieces were included, SOA would drop toward the bottom of the list and TSP/PSP + CMM Level 5 would be the best for the larger size ranges.

Since productivity without quality is a misleading and hazardous metric, Table 3-9 shows the approximate quality levels of the same technologies. Table 3-9 uses three important quality metrics: defect potentials, defect removal efficiency, and delivered defects. As with Table 3-8, Table 3-9 concentrates on the troublesome size plateau of 10,000 function points.

TABLE 3-8 Approximate Productivity Rates by Size of Application (Data Expressed in Terms of Function Points per Staff Month)

	1,000 Function Points	10,000 Function Points	100,000 Function Points
SOA	8.00	26.00	20.00
TSP/PSP + Scrum	15.75	10.00	9.00
CMM 5 + Six-Sigma	13.00	9.75	9.25
TSP/PSP	14.25	9.50	8.00
CMM Level 5	12.50	9.25	7.75
Six-Sigma for software	9.00	9.00	7.20
CMM 3 + OO	16.00	8.75	6.80
CMMI	11.25	8.25	7.50
Object oriented (OO)	15.50	8.25	6.75
Agile/Scrum + OO	24.00	7.75	7.00
RUP	14.00	7.75	4.20
CMM Level 4	11.00	7.50	5.50
CMM Level 3	9.50	7.25	4.50
Agile/Scrum	23.00	7.00	5.25
Spiral	15.00	6.75	4.25
Extreme XP	22.00	6.50	5.00
TickIT	8.75	6.25	3.90
Iterative	13.50	6.00	3.75
RAD	14.50	5.75	3.60
Waterfall + inspections	10.00	5.50	4.75
CMM Level 2	7.50	4.00	2.25
Waterfall	8.00	2.50	1.50
CMM Level 1	6.50	1.50	1.25
Average	**13.64**	**7.03**	**5.23**

It is useful to define the major terms used in Table 3-9. The term *defect potential* refers to the probable number of defects found in five sources: requirements, design, source code, user documents, and bad fixes. A *bad fix* is a new defect that is accidentally included in an attempt to repair previous defects. The data on defect potentials comes from companies that actually have lifecycle quality measures. Only a few leading companies have this kind of data, and they are among the top-ranked companies in overall quality: IBM, Motorola, AT&T, and the like.

The term *defect removal efficiency* refers to the percentage of defects removed before delivery of the software to its users or customers. If the development team found 900 defects and the users reported 100 defects in the first three months of usage, then the defect removal efficiency would be 90 percent. This is a simple but extremely powerful and useful metric.

TABLE 3-9 Approximate Quality Levels for Applications of 10,000 Function Points (Data Expressed in Terms of Defects per Function Point)

	Defect Potentials	Removal Efficiency	Delivered Defects	Total Defects Delivered	High Severity Defects
CMM 5 + Six-Sigma	4.80	98.00%	0.10	960	259
TSP/PSP + Scrum	4.90	97.00%	0.15	1,470	397
TSP/PSP	5.00	96.00%	0.20	2,000	540
CMM Level 5	5.50	96.00%	0.22	2,200	594
Six-Sigma for software	5.25	94.00%	0.32	3,150	851
CMMI	6.10	94.00%	0.37	3,660	988
CMM Level 4	6.00	93.00%	0.42	4,200	1,134
CMM 3 + OO	6.10	92.00%	0.49	4,880	1,318
CMM Level 3	6.25	92.00%	0.50	5,000	1,350
Waterfall + inspections	6.50	92.00%	0.52	5,200	1,404
Agile/Scrum + OO	5.30	90.00%	0.53	5,300	1,431
TickIT	6.10	88.00%	0.73	7,320	1,976
Extreme XP	6.25	88.00%	0.75	7,500	2,025
Agile/Scrum	6.00	87.00%	0.78	7,800	2,106
Iterative	6.25	86.00%	0.88	8,750	2,363
Spiral	6.50	85.00%	0.98	9,750	2,633
Object oriented (OO)	6.00	83.00%	1.02	10,200	2,754
RUP	6.75	84.00%	1.08	10,800	2,916
SOA	2.50	55.00%	1.13	11,250	3,038
CMM Level 2	7.00	80.00%	1.40	14,000	3,780
Waterfall	7.25	80.00%	1.45	14,500	3,915
RAD	7.25	77.00%	1.67	16,675	4,502
CMM Level 1	7.50	70.00%	2.25	22,500	6,075
Average	**5.96**	**86.83%**	**0.78**	**7,785**	**2,102**

Empirical data from many projects indicates that applications having about 95 percent defect removal efficiency levels have shorter schedules, lower costs, and happier customers than average applications of 85 percent defect removal efficiency. Surprisingly, increasing removal efficiency to 95 percent by means of design and code inspections improves productivity and schedules. This is a very important economic fact. Going beyond 95 percent defect removal efficiency up to 98 percent adds slightly to schedules and costs, but 95 percent removal efficiency is a "golden number" because it optimizes so many important parameters at the same time.

The term *delivered defects* refers to the number of latent defects still present in the software when it is delivered to users. Of course only

about 25 percent of these are of high severity that might cause the application to fail or produce incorrect results.

The best results in terms of having low levels of delivered defects comes from a synergistic combination of defect prevention methods and defect removal methods. For example, the daily meetings between clients and developers under the Agile method is a very effective defect prevention approach. Formal design and code inspections are very effective defect removal methods and are also excellent in terms of defect prevention.

Table 3-9 shows data in descending order based on numbers of delivered defects for applications of 10,000 function points in size. The margin of error is high, but the overall rankings are probably close to reality. Here too hybrid methods such as CMM Level 5 joined with Six-Sigma for software and TSP/PSP joined with Scrum tend to be at the top of the list in overall effectiveness.

As this book is written, the overall U.S. averages for defect potentials are about 5.0 per function point, and the average value for defect removal efficiency is about 85 percent. Any technology of serious value to the software industry needs to lower defect potentials below average values, raise defect removal efficiency levels, or both.

The Agile methods lower defect potentials for small applications and are slightly better than average for defect removal efficiency levels. The team software process (TSP) and higher CMM levels are the current winners in terms of overall defect prevention and defect removal efficiency levels and especially so when combined with Six-Sigma for software. Interestingly, the hybrid methods that combine standard approaches such as the CMM + Six-Sigma or the object-oriented approach are often better than the original methods in isolation.

Somewhat troubling to the author, and no doubt to readers as well, is that even the very best software quality control methods are not 100 percent effective in removing defects prior to delivery to clients.

Of course an application of 10,000 function points in size will probably have more than 1,000 users. Each user will probably only discover about 1 percent of the latent high-severity defects. That means that if a large software application is released with 1,000 high-severity latent defects, any given customer will probably encounter only 10 of them. Of course, some unlucky customers may encounter quite a few more than that, but other customers may actually encounter only 1 or 2, or even none at all.

Since latent defects are more common in seldom-used functions and features than in main-line functions and features, the users who are most likely to encounter latent defects in significant numbers are those who are the most sophisticated and hence use more of the application's features than average. The users who are least likely to encounter latent defects are those who use the application in the simplest and most straightforward fashion.

Variations in Data Representation and Averaging Techniques The first edition of this book utilized powers of 2 for graphing the results. The first edition also did not display specific results for applications larger than 10,240 function points in size. The second and third revised editions have switched over from powers of 2 to powers of 10. They also extend the upper range of displayed data up to 300,000 function points.

One of the many problems of exploring software productivity and quality is that of attempting to calculate "averages" in a way that makes sense and does not give misleading results.

There are of course a number of techniques for establishing "average" values: i.e., the arithmetic mean, the harmonic mean, the geometric mean, the mode, or the median. There is also the *inter-quartile range,* or the set of values within which half of the results might be found. There are also standard deviations and various tests of statistical precision.

The software world is not very sophisticated when it uses the word *average*. From examination of the software literature, the most common usage of *average* appears to be nothing more than the arithmetic mean of the results.

Table 3-10 lists a set of ten sample projects to illustrate some of the results that might occur based on which approaches are used to quantify averages.

TABLE 3-10 Examples of Means, Mode, and Median for Ten Software Projects

Project	Function Points	Effort (Months)	Productivity (Function points per staff month)	Productivity (Work hours per function point)
A	10	0.30	33.33	3.96
B	10	0.50	20.00	6.60
C	12	0.60	20.00	6.60
D	15	0.60	25.00	5.28
E	30	1.50	20.00	6.60
F	50	3.30	15.15	8.71
G	60	4.00	15.00	8.80
H	80	5.30	15.09	8.75
I	100	6.60	15.15	8.71
J	500	100.00	5.00	26.40
Totals	**867**	**122.70**		
	Arithmetic mean		18.37	9.04
	Standard deviation		7.43	6.32
	Harmonic mean		14.60	7.18
	Geometric mean		16.74	7.88
	Median		17.58	7.66
	Mode		20.00	6.60
	Weighted Average		7.07	18.68

The arithmetic mean of 18.37 function points per staff month is dominated by the large number of small, high productivity projects. The arithmetic mean is suitable if the purpose of the average is to explore the central tendency of productivity rates. The accompanying standard deviation of 7.43 reflects a fairly broad range of productivity rates that indeed are a common occurrence.

Note that if the arithmetic mean is used by companies that do not compensate for the approximate 50 percent "leakage" of effort noted with cost tracking systems in the United States, the apparent average productivity rate would be about 36 function points per staff month.

If the arithmetic mean is used by companies whose software measurements are built around "design, code, and unit test" or DCUT activities, which constitute only about 25 percent of total software development effort, the apparent productivity would be about 72 function points per staff month.

The arithmetic mean is dangerously misleading even if used with full project data, and becomes more so when applied to the partial project data that is so common in the software world.

The harmonic mean of 14.6 function points per staff month is more balanced than the arithmetic mean, but is still dominated by the smaller projects, which utilize very little of overall effort and have the highest productivity rates.

The geometric mean of 16.53 function points per staff month is derived by multiplying the 10 productivity values together and then extracting the 10th root. The geometric mean is seldom encountered in software studies, although it often is used for population studies.

The mode of 20 function points per staff month, and the median of 17.58 function points per staff month are seldom encountered in the software literature, although both are useful in certain situations and are used from time to time in this book.

The weighted average is probably of greatest interest to financial and executive personnel, who are concerned with overall expenses for software. It is the somewhat pessimistic weighted average results of 7 function points per staff month derived from dividing total function points (867) by total effort (122.7 months) that often causes executives to move toward outsourcing arrangements on the grounds that productivity is too low. This is an important point, because a few large systems that are sluggish or unsuccessful can make a hundred successful small projects invisible in the eyes of senior executives.

Software managers, executives, and technical workers should realize that many successful small projects are not as significant to corporate management as one huge disaster. Therefore, the weighted average is a relatively important business metric. More so, in fact, than the arithmetic mean.

Suppose there had been another large project the same size as project J, which was canceled and not delivered. Assume that this canceled project (project K) was also 500 function points in size and 100 months of effort were devoted to it prior to termination.

The function points for this failed project would not be part of the normal productivity results since it was incomplete and not delivered. However, it is perfectly reasonable to record the 100 months of effort because they were actually utilized.

Including 100 months of effort for a failed project affects the arithmetic only slightly: it drops from 18.37 to 16.36 function points per staff month. However, the inclusion of the additional 100 months of effort drives the weighted mean down to a dismal 3.9. It is highly likely that corporate management would wish to seek better methods and would probably invite various outsource vendors in for serious discussions as well.

The software industry has not been very careful or very conscientious in these measurement and statistical attributes:

- For 55 years the software industry has used a metric (lines of code) that was never standardized and violates the basic assumptions of economic productivity.
- For 55 years the software industry has tried to make do with gross project-level data, and has no standard chart of accounts or standard activity descriptions for activity-based costing.
- The software industry has failed to validate raw data prior to publication, and hence tends to overstate productivity and understate costs by using partial data.
- The software industry has seldom bothered to identify in print the nature of the statistical methods used to produce reports, benchmarks, and baseline studies.

The United States and other countries too should have national databases of software productivity and quality information. Since the first edition of this book was published, the U.S. Air Force has been attempting to establish a national database, with encouraging but not totally satisfactory results. The Australian Software Metrics Association (ASMA) has also attempted to quantify local results. The International Function Point Users Group (IFPUG) is also attempting this, although they must depend upon members to contribute information. The Software Engineering Institute (SEI) has also collected substantial volumes of data, primarily in an attempt to demonstrate the value of the CMM and CMMI. Members of the Agile Alliance publish data from time to time, but as yet there is not a large collection of Agile benchmark data.

As the third edition of this book is written in 2008, the most interesting new source of benchmark data is the International Software

Benchmark Standards Group (ISBSG). This is the only source of productivity and quality benchmarks available to the general public. Both CDs and published data are available from ISBSG. Currently about 4,000 projects are represented, and the rate of growth is perhaps 500 projects per year. ISBSG was founded in 1997, a year after the second edition of this book was published.

There are also a number of proprietary or semi-private databases owned by commercial companies such as the David Consulting Group, the Gartner Group, Quantitative Software Management (QSM), and Software Productivity Research.

However, a true national database would probably exceed the volume of all known databases to date. A true national database for software projects in the United States would probably need to contain data from about 5,000 organizations and perhaps 75,000 software projects. This is about three times larger than the sum of all known data on software productivity and quality. What would be helpful to the industry would be a joining or merging of data from the top benchmark organizations such as ISBSG, SEI, Gartner, the David Consulting Group, SPR, etc. However since some of these organizations are business competitors, merging their data is unlikely.

It would also be highly useful to establish both industry databases and individual company databases for software quality and productivity information. At the corporate level, several hundred U.S. companies now have at least rudimentary information on software topics and rather accurate function point counts (even if their cost and resource data are suspect). There is nothing to speak of at the industry level, although it would appear reasonable for software-intensive industries such as insurance, banking, and telecommunications to wish to do this.

Significant Software Technology Changes Between 1990 and 2008

Following are short summaries of the major technological changes between 1990 and 2008 that have affected overall national averages for software.

Agile Software Development

The previous edition of this book was published in 1996, so Agile methods were not discussed in it. In recent years, the Agile methods have been on an explosive growth path. As of 2008 perhaps as many as 25 percent of new information technology projects now use at least some aspect of the Agile methods. Because the Agile methods are so new, there is little data available about maintenance of Agile projects.

The history of the Agile methods is hazy because the Agile methods are somewhat diverse. However in 2001 the famous *Agile manifesto* was published. This provided the essential principles of Agile development. That being said, there are quite a few Agile variations that include Extreme programming (XP), Crystal development, adaptive software development, feature-driven development, and several others.

Some of the principal beliefs found in the Agile manifesto are

- Working software is the goal, not documents.
- Working software is the primary measure of success.
- Close and daily contact between developers and clients is necessary.
- Face to face conversation is the best form of communication.
- Small self-organizing teams give the best results.
- Quality is critical, so testing should be early and continuous.

Comparing the Agile methods with the CMM and the CMMI is interesting. Both the Agile methods and the CMM/CMMI are concerned with three of the same fundamental problems:

- Software requirements always change.
- Fixing software bugs is the most expensive software activity in history.
- High quality leads to high productivity and short schedules.

However the Agile method and the CMM/CMMI approach draw apart on two other fundamental problems:

- Paperwork is the second most expensive software activity in history.
- Without careful measurements continuous progress is unlikely.

The Agile methods take a strong stand that paper documents in the form of rigorous requirements and specifications are too slow and cumbersome to be effective. In the Agile view, daily meetings with clients are more effective than written specifications. In the Agile view, daily team meetings or "Scrum" sessions are the best way of tracking progress, as opposed to written status reports. The CMM and CMMI do not fully endorse this view.

The CMM and CMMI take a strong stand that measurements of quality, productivity, schedules, costs, etc., are a necessary adjunct to process improvement and should be done well. In the view of the CMM and CMMI, without data that demonstrates effective progress, it is hard to prove that a methodology is a success or not. The Agile methods do not fully endorse this view. In fact, one of the notable gaps in the Agile

approach is any quantitative quality or productivity data that can prove the success of the methods.

The Agile methods of development follow a different pattern from the conventional waterfall model. The Agile goal is to deliver running and usable software to clients as rapidly as possible. Instead of starting with requirements and moving to design before coding, what is most likely with the Agile methods is to divide an overall project into smaller projects, each of about 100 to 250 function points in size. In the Agile terminology, these small segments are termed iterations or sometimes *sprints*.

These small iterations or sprints are normally developed in a "time box" fashion that ranges between perhaps two weeks and three months based on the size of the iteration.

However in order to know what the overall general set of features would be, an Agile project starts with "Iteration 0" or a general planning and requirements-gathering session. At this session, the users and developers would scope out the likely architecture of the application and then subdivide it into a number of iterations.

Also, at the end of the project when all of the iterations have been completed, it is necessary to test the combined iterations at the same time. Therefore a release phase follows the completion of the various iterations. For the release, some additional documentation may be needed. Also, cost data and quality data needs to be consolidated for all of the iterations. A typical Agile development pattern might resemble the following:

- Iteration 0
  - General overall requirements
  - Planning
  - Sizing and estimating
  - Funding
- Iterations for each subset
  - User requirements for each iteration
  - Test planning for each iteration
  - Testing case development for each iteration
  - Coding
  - Testing
  - Scrum sessions
  - Iteration documentation
  - Iteration cost accumulation
  - Iteration quality data

- Release
  - Integration of all iterations
  - Final testing of all iterations
  - Acceptance testing of application
  - Total cost accumulation
  - Quality data accumulation
  - Final Scrum session

The most interesting and unique features of the Agile methods are these: (1) The decomposition of the application into separate iterations; (2) The daily face-to-face contact with one or more user representatives; (3) The daily "Scrum" sessions to discuss the backlog of work left to be accomplished and any problems that might slow down progress. Another interesting feature is to create the test cases before the code itself is written, which is a feature of Extreme programming (XP).

The rapid growth in the usage of Agile methods is partly because they improve the productivity of applications up to about 1,500 function points in size. For larger projects, portions of the Agile methods have been applied with good success. For example, the daily Scrum sessions are useful across all size ranges. However for really large projects in the 10,000 to 100,000 function point range, it is also necessary to use formal design and code inspections and have formal change control and configuration control methods to be successful.

Also, there are practical logistics issues for really large software projects. Applications in the 100,000 function point size range have development teams that can top 600, and may have more than 10,000 users. It is not feasible to have Scrum sessions for the entire team, nor can there be daily meetings with representative users when the number of possible users is so large.

Benchmarking

Benchmarking, or comparing software productivity, quality, schedules, salaries, methodologies, etc., between companies was rare when the data for the first edition of this book was assembled. Since 1990, benchmarking has been on an explosive growth path.

The probable reason for the accelerating interest is the fact that the function point metric makes benchmarking much more accurate and the data much more useful than was possible in the former "lines of code" (LOC) era. In any case, now that hundreds of companies are measuring software productivity and quality, the impact is beneficial in terms of leading to improvements in the factors that are measured.

The formation in 1997 of the nonprofit International Software Benchmarking Standards Group (ISBSG) has also accelerated software benchmarks. Unlike companies whose benchmark data is only available to a few specific clients, the ISBSG data is commercially available and can be published in both print and CD formats. The CD formats are particularly useful since they allow statistical manipulation of the data.

There are still problems with inconsistencies and incompatibilities among the various benchmarking consulting companies and the published results. For example, the David Consulting Group, the ISBSG, the SEI, SPR, and Gartner Group have different averages for several industries. The probable reason for the difference is the fact that some data is derived from onsite interviews that correct some of the anomalies and missing items that are normally omitted from software cost and resource tracking systems, and some data is reported by clients without onsite validation. Also, each of the major benchmarking companies have somewhat different sets of clients, although there is some overlap. Usually, however, a large corporation interested in benchmark comparisons will only hire one of the benchmark consulting companies.

Benchmarking does not have a direct, near-term impact on software quality or productivity rates. However, benchmarks are usually preludes to some kind of planned improvement program and hence almost always on the critical path for companies that do make improvements in software productivity and quality.

End-user Programming

The first edition of this book did not deal with end-user programming at all. However, the increase in the numbers of knowledge workers who are computer literate and can program is rising at double-digit rates. The tools and programming languages for end-user development have exploded under the impact of Visual Basic and its competitors. Even the macro languages for spreadsheet applications have become more user-friendly and less arcane.

There are now enough end-user applications and their typical productivity rates are high enough (in excess of 50 function points per staff month) to actually elevate national averages when end-user data is included. End-user programming is not always safe and effective, however, as will be discussed later in this book.

Somewhat surprisingly, the volume of reusable material in the end-user programming domain now tops every other due to the huge numbers of Visual Basic controls now being marketed by third-party vendors. This phenomenon is very significant.

A strong caution is needed regarding end-user software in corporate environments. The ownership of end-user applications probably belongs

to the company rather than the developer, due to standard employment agreements. That means when users change jobs, the software they develop stays behind. However, it is highly unlikely that the new employee taking over the same job will be able to use the application because end-user software almost never has a user's guides. It often uses quirky and nonstandard interfaces. Although quality is seldom measured for end-user software, the lack of formal testing and the total absence of quality assurance reviews means that many end-user applications are filled with bugs that no one knows about.

Client-server Applications

When this book was first published, scarcely 100 projects out more than 4,000 were client-server projects. Yet roughly 750 client-server projects have been explored in the past 15 years.

Client-server software productivity rates are typically about double that of mainframe COBOL productivity rates (i.e., 16 function points per staff month versus 8 function points per staff month). However, client-server software complexity is high and quality is not as good as for mainframe applications. About twice as many defects or bugs are present at delivery as are found in mainframe applications. The implication is that maintenance costs will be much higher for client-server software than for mainframe software, as today's client-server applications become tomorrow's legacy systems.

Since the year 2000, maintenance studies of client-server applications have revealed that indeed maintenance is difficult and troublesome. Error-prone modules are common, bad fix injections are high, and the overall costs of maintenance are greater than for simpler kinds of applications.

Commercial Software

When this book was first published in 1991, the quality levels of commercial software were not very impressive. Productivity was high, due in large part to massive bouts of unpaid overtime on the part of many commercial software engineers. Since commercial software houses as a class are sensitive to public opinion, the major players such as Microsoft, SAP, Oracle, Symantec, and Computer Associates have been trying energetically to improve their quality levels but not reduce their productivity or lengthen their schedules.

Typical Windows-based commercial software packages such as spreadsheets and word processors are now between 1,000 and 5,000 function points in size and can be purchased for prices in the range of $295 per copy (or less). Therefore the cost per function point for packaged applications

can be as low as $0.15. Packages are occupying increasingly large percentages of corporate portfolios. The commercial software business is now a significant component of the United States balance of trade.

However, at the upper end of the commercial software spectrum are really massive applications, some of which are among the largest software applications in world history. Microsoft Vista, for example, seems to top 150,000 function points. Microsoft Office 2007 Professional may top 100,000 function points. Even beyond this size, the massive ERP packages leased by SAP and Oracle may approach 300,000 function points in size.

Unfortunately as of 2007, the quality control methods used by software vendors for such large applications are not fully adequate. There are thousands of latent bugs in the large Microsoft packages, and also thousands of latent bugs in the massive ERP packages. Indeed, progress on this very book was slowed down by serious bugs in Microsoft Excel and Word. When pasting spreadsheets from Excel into Word, sometimes the formats were lost. Worse, some mathematical functions in Excel appeared to give incorrect results. For example when averaging columns in spreadsheets, sometimes the left-most column would show incorrect results. Of course the more famous "multiply" bug was widely discussed on the web (i.e., multiplying 850 by 77.1, Excel shows the answer to be 100,000 even though the correct answer is really 65,335).

Downsizings and Layoffs

The author has been in the software business for more than 35 years. Yet the period between 1990 and 2002 has been the most traumatic in terms of layoffs and downsizings of major corporations. A number of corporations such as Digital Equipment, Wang, Data General, Enron, Lechmere, and Anderson Consulting have gone out of business. Others such as IBM and EDS are smaller in 2008 than they were in 1997, in terms of employment.

Of course, some companies are growing and prospering. For example, the maintenance outsource company Computer Aid has been growing rapidly.

The immediate near-term impact of downsizings is a sharp reduction in productivity and an overall reduction of quality levels within the affected company, due in part to the fact that often the best technical and quality assurance personnel jump ship or are let go. If the enterprise survives, it can eventually regain its former productivity and quality levels. Indeed, it may even exceed them since average project sizes tend to decline during layoffs and downsizing operations.

In a curious way, downsizings have contributed to long-range national U.S. productivity gains. The reasons are fourfold: (1) More and more software is now being done by outsourcers who are usually more productive

than their clients; (2) As large companies shrink, average project sizes shrink too; (3) The reduction in the number of large companies building software means that smaller companies construct a larger percentage of total applications. Small companies are often more productive than large corporations; (4) A number of rather troublesome projects have been shipped offshore to India or China, so their long schedules and low productivity are removed from U.S. data points.

Also, during bouts of downsizings and layoffs, large companies seldom tackle the high-risk applications whose frequent failures tend to bring down corporate and national productivity averages.

The author has not been engaged to study very many projects in India, China, Russia, or the other major offshore outsource countries. A significant number of U.S. projects and also U.S. jobs have moved to other countries, and this trend is accelerating as of 2008. Whether offshore outsourcing will continue to accelerate is difficult to predict. However, since the inflation rates in the major offshore outsource countries are higher than in the United States, the cost differential between offshore work and U.S. work is rapidly declining. Indeed, as this book is being written the average cost of living in Shanghai has increased so much that Shanghai now costs more than New York. If such trends continue, it is not impossible that offshore outsourcing will decline by about 2015 and perhaps even reverse.

Function Point Expansion

When this book was first published, usage of function point metrics was growing but still a minority among the major software houses of the United States and abroad. Membership in the International Function Point Users Group (IFPUG) has been steadily expanding at about 50 percent per year. By about 1995, function point metrics became the most widely used software metric in the United States and IFPUG was the largest software measurement association.

Function point usage is still increasing, but the rate of increase has slowed down. Also, an unexpected phenomenon has occurred: the creation of more than 20 function point variants in addition to the original form of function point now maintained by the International Function Point Users Group (IFPUG).

As discussed elsewhere in this book, and also in the companion book *Estimating Software Costs* (McGraw-Hill, 2007) the standard IFPUG function point metric has been joined by COSMIC function points, NESMA function points, Mark II function points, full function points, engineering function points, web-object points, and many more. Unfortunately, most of these variants lack conversion rules to IFPUG function points.

For the United States, IFPUG remains the dominant form of function point, and as of 2007, the author estimates that at least 97 percent of all

projects use either IFPUG itself or a method derived from IFPUG such as backfiring and the newer pattern-matching approach.

But for Europe and the Pacific Rim, the situations are more fluid. The other function point variants are probably used for perhaps 15 percent of project measurements outside the United States.

The European software community is also adopting function point metrics rapidly, and function points are making significant progress in India and Japan as well. This rapid expansion of function point metrics means an exponential growth in the availability of software productivity and quality data.

The function point metric is contributing to software productivity and quality improvements in an interesting way. The older "lines of code" metric focused industry attention on the task of coding. The high costs associated with the production of more than 50 paper documents surrounding software was essentially invisible using LOC metrics. Also invisible were bugs found in requirements, specifications, and user documents. Function point metrics have thrown a spotlight on a topic that had not been examined, and revealed that software paperwork often costs more than source code. Also, defects in requirements and specifications outnumber coding defects and cannot easily be found by testing.

Hybrid Development Practices One of the major trends in the industry since about 1997 has been to couple the most effective portions of various software development methodologies to create hybrid approaches. As shown in several places in this book, the hybrid methods are quite successful and often achieve higher quality and productivity rates than the original "pure" methods. As of 2007, some interesting examples of the hybrid approach include, in alphabetical order:

- Agile joined with the CMM and CMMI
- Agile joined with lean Six-Sigma
- Agile joined with object-oriented development
- Agile joined with team software process (TSP) and personal software process (PSP)
- CMM and CMMI joined with ISO standard certification
- CMM and CMMI joined with ITIL
- CMM and CMMI joined with quality function deployment (QFD)
- CMM and CMMI joined with Six-Sigma
- CMM and CMMI joined with TSP and PSP
- Extreme programming (XP) joined with lean Six-Sigma

- Information Technology Infrastructure Library (ITIL) joined with Six-Sigma
- ISO joined with TickIT
- Object-oriented development joined with service-oriented architecture (SOA)
- Six-Sigma joined with ITIL
- Six-Sigma joined with quality function deployment (QFD)
- Six-Sigma joined with TSP/PSP
- SOA joined with ITIL
- TickIT joined with Six-Sigma

This list only links pairs of development methods that are joined together. From time to time, three or even four development practices have been joined together:

- Agile joined with OO, joined with lean Six-Sigma, joined with ITIL
- CMM joined with Agile, joined with TSP/PSP, joined with Six-Sigma, joined with QFD

The topic of hybrid approaches is not well covered in the software engineering literature. Neither is the topic of hybrid approaches well covered in terms of benchmarks and quantitative data, although some hybrid projects are present in the International Software Benchmark Standards Group (ISBSG) data, and also in the data presented in this book and collected by the author and his colleagues at Software Productivity Research. A new consulting company, Process Fusion, has been created specifically to aid clients in selecting the best combinations of factors from the various software development methods.

The hybrid approaches also present a challenge to software cost and schedule estimating tools as well. Most such tools have settings for specific methods such as the various CMM levels or OO, but need to be adjusted for predicting the results of hybrid methods.

If current trends continue, by about 2012, hybrid approaches will probably outnumber "pure" approaches for software development.

ISO 9000-9004 Standards

The international standards organization (ISO) has assumed a new and major importance in the context of the evolution of the European Union. The ISO 9000-9004 quality standards are now being implemented on a worldwide basis and are affecting both hardware and software products. In particular, ISO 9001 is affecting software projects.

Unfortunately, the ISO quality standards are not particularly effective for software projects and have managed to omit a number of important software quality topics. However, the ISO quality standards have had the beneficial impact of making quality an important business issue and raising the global awareness of the importance of quality.

There is no solid evidence that demonstrates that ISO certification actually improves software quality levels. Software defect potentials and defect removal efficiency levels are approximately the same in certified companies and in similar uncertified companies.

The ISO organization is an important organization to both the engineering and software world. ISO certification is not a panacea, but it does ensure that certified methods have been reasonably designed and are appropriate to use.

More recently than the ISO 9000-9004 standards, the ISO organization has certified four different forms of function point analysis: COSMIC, IFPUG, Mark II, and NESMA in alphabetical order.

Maintenance and Enhancements

When the first edition of this book was published in 1991, maintenance and enhancement work surpassed 35 percent of total software effort. Now in 2008, maintenance and enhancement work is the dominant software activity in the United States, Europe, the Pacific Rim, and most of the world.

This fact by itself should not be a surprise. Whenever an industry has more than 50 years of product experience, the personnel who repair existing products tend to outnumber the personnel who build new products. For example, there are more automobile mechanics in the United States who repair automobiles than there are personnel employed in building new automobiles.

The software industry somewhat resembles the automobile industry in another respect. Currently there are more than 700 programming languages in existence. The average programming language lasts about 15 years as a development tool, but large software applications last more than 20 years. This means that many legacy software applications are written in "dead" or orphan languages, where compilers no longer exist and skilled programmers are rare.

In the automotive world, if all of the models of all of the major manufacturers are coupled together, there are more than 700 distinct automobiles that are still being driven. For automobiles older than about 10 years, or for obscure brands (i.e., Yugo, Crosley Hotshot, Studebaker, etc.) parts are hard to find and knowledgeable mechanics may be very rare.

When the software industry was first growing in the 1960s and 1970s, a very harmful maintenance method evolved. In the days of unstructured "spaghetti bowl" code, safely maintaining a program was so

difficult that only the original developer could modify the code (and not even the original developer would always be safe).

As a result, the programmers who developed applications also took over maintenance responsibilities for the applications after they were delivered to customers. In fact, the author has known some programmers personally who worked on the same applications for more than 10 years.

If the original programmer should change jobs, become ill or incapacitated, or die, then the company that owned the software would be in a very bad situation. Sometimes teams of several top programmers were assigned for several months to go through the application and handle any changes that occurred.

This is not a cost-effective way of doing business. For one thing, being locked into the maintenance of a single application puts an abrupt end to the career path of the programmer. For another, maintenance and development are mutually antagonistic kinds of work, so both suffer when the same person has to do both jobs.

Software, like automobiles, needs to be designed and developed so that maintenance can be taken over by qualified specialists after delivery. It would be folly to have to return an automobile to the manufacturer for every repair. Any software application that is so complex or peculiar that only the original developer can fix it has something seriously wrong with it.

At the end of the 20th century, software maintenance grew rapidly during 1997–2008 under the impact of two "mass updates" that between them required modifications to about 85 percent of the world's supply of existing software applications.

The first of these mass updates was the set of changes needed to support the new unified European currency or Euro that rolled out in January of 1999. About 10 percent of the total volume of world software needed to be updated in support of the Euro. However in the European Monetary Union, at least 50 percent of the information systems required modification in support of the Euro.

The second mass-update to software applications was the "Y2K" or year 2000 problem. This widely discussed problem was caused by the use of only two digits for storing calendar dates. Thus the year 1998 would have been stored as 98. When the century ended, the use of 00 for the year 2000 violated normal sorting rules and hence caused unrepaired software applications to fail or to produce incorrect results.

The year 2000 problem affected as many as 75 percent of the installed software applications operating throughout the world. Unlike the Euro, the year 2000 problem also affected some embedded computers inside physical devices such as medical instruments, telephone switching systems, oil wells, and electric generating plants.

Although these two problems were taken care of, the work required for handling them triggered delays in other kinds of software projects and hence made software backlogs longer than normal.

Under the double impact of the Euro conversion work and year 2000 repair work, it appeared that more than 65 percent of the world's professional software engineering population were engaged in various maintenance and enhancement activities during 1999 and 2000.

Although the Euro and the Y2K problem are behind us, they are not the only mass-update problems that we will face. The U.S. Congress changed the starting date of daylight saving time in 2007, and that has already affected thousands of devices and software applications. By the year 2015, it may be necessary to add one or more digits to U.S. telephone numbers. The UNIX calendar expires in the year 2038 and could be troublesome like the year 2000 problem. By the year 2050, it may be necessary to add at least one digit to U.S. social security numbers, which may pose an even greater problem.

The imbalance between software development and maintenance is opening up new business opportunities for software outsourcing groups. It is also generating a significant burst of research into tools and methods for improving software maintenance performance.

Three interesting forms of companies have emerged in the wake of the huge maintenance "deficit" that the software industry has been accumulating. The first of these are companies such as Computer Aid Incorporated, which specialize in maintenance operations and which employ skilled maintenance programmers. Such companies are usually quite productive because their workers are concentrating primarily on maintenance and are not diverted by other tasks.

The second form of emerging companies is that of companies such as Relativity Technologies, which deal with tools and analytical methods for handling the inner problems of legacy applications. Legacy software may be high in complexity and may also contain "error-prone" modules. Using sophisticated parsing and code analysis tools, companies such as Relativity Technologies can tackle the major "R" issues of legacy applications: reverse engineering, restructuring, refactoring, recoding, reengineering, and full renovation. They can also isolate and surgically remove error-prone modules. In addition, they can translate some (but not all) older languages into more current languages, such as converting Natural into Java.

The third interesting form of company is exemplified by Shoulders Corporation. The view of Shoulders Corporation is that important legacy applications can be "mined" for business rules and key algorithms. Then using these rules and algorithms, plus new and additional requirements, it is possible to develop modern replacement versions using Agile methods.

In order for the replacements to fit within the fairly tight budgets of the legacy application owners, current maintenance costs need to be

reduced and the savings to be used to fund the replacement version. Obviously all maintenance cannot stop on the legacy application, but if maintenance can be limited to defect repairs and legally mandated enhancements, then the reductions should be sufficient to fund the replacement.

The productivity, costs, and schedule for the modern replacement version are likely to be much better than the original due to the combination of "data mining" plus the usage of Agile technologies. If the original legacy application required 48 calendar months and had a productivity rate of 8 function points per staff month, the new version can perhaps be brought to completion in 18 to 24 months with a productivity rate that might exceed 20 function points per staff month. About half of the improvements would be due to data mining and the other half to the use of the Agile methods.

Because quality is the weak link in most application development processes, the Shoulders Corporation methods are particularly rigorous in terms of quality. Among the methods used are: (1) Early appointment of a QA manager on day 1; (2) Developing test cases before the code itself is written; and (3) Having a ratio of testers to developers of about 1 to 1 to ensure that testing is both thorough and fast.

It is, of course, possible to blend all three forms of maintenance improvements; in other words, it is possible to combine the methods exemplified by Computer Aid, Relativity Technologies, and Shoulders Corporation.

The combination of the specialized maintenance outsource companies and the maintenance technology companies tend to be extremely cost effective compared to "traditional" maintenance where developers interleave enhancements and bug repairs, but have to work with obsolete languages, high levels of complexity, and some error-prone modules.

Before renovation, legacy applications typically have between a 7 percent and 20 percent bad-fix injection rate, when new bugs are accidentally introduced while fixing old bugs. The average maintenance assignment scope, or the amount of software that one person can keep running in the course of a year is at or below 750 function points. The number of bugs fixed per staff month is usually 8 or less.

If the fully burdened cost of a maintenance programmer is $7,500 per month or $90,000 per year and the maintenance assignment scope is only 750 function points, then the annual maintenance cost per function point will be $120. A combination of renovation and maintenance specialists can increase the maintenance assignment scope by as much as 25 percent per year for several years in a row. If the maintenance assignment scope reaches 1,500 function points, the annual maintenance cost per function point will be down to $60. If the maintenance assignment scope tops 3,000 function points, which is rare but possible, the cost will be down to $30 per function point per year.

The combination of maintenance specialists working with renovated code are much more efficient than ordinary maintenance methods by generalists. The bad-fix injection rates are usually less than 2 percent; the maintenance assignment scopes may top 3,000 function points, and the number of bugs fixed per staff month are in the range of 12 to 15. Annual maintenance costs should be well below $100 per function point.

Maintenance specialization augmented by sophisticated code analysis and restoration tools show marked economic and quality improvements. Maintenance assignment scopes are usually above 1,500 function points and occasionally top 3,000 function points. The largest maintenance assignment scope noted by the author is about 5,000 function points. This large value is achievable only for applications that are low in cyclomatic and essential complexity, have zero error-prone modules, and capable maintenance specialists.

Object-Oriented Paradigm

Since this book was first published in 1991, the object-oriented paradigm has emerged from cult status to become a mainstream methodology (although many of the OO conferences retain an interesting flavor of eccentricity and cultishness). Productivity rates of OO projects using languages such as Objective C, SMALLTALK, and C++ are generally significantly higher than those of similar projects developed using procedural languages such as C, FORTRAN, or PASCAL: 10 to 12 function points per staff month as opposed to 4 to 6 function points per staff month.

This means that the rapid expansion of the OO paradigm is having some impact on quantitative results and especially so for systems and commercial software where the OO paradigm is furthest up the learning curve. The OO analysis and design domain has had such a steep learning curve that the near-term impact is negative for productivity. Do not expect much in the way of productivity gains for the first year after moving into the OO domain.

As more companies move along the OO learning curve, and as class libraries of reusable objects improve in quantity and quality, it can be hypothesized that the OO domain will continue to expand in volume and in impact. (Surprisingly, however, the volume of reusable material available for the Visual Basic programming language currently exceeds the volume for any known OO language.)

In recent years since about 2000, the OO technologies have formed synergistic combinations with other technologies such as Agile, Extreme programming (XP), the Rational Unified Process (RUP), and the various levels of the capability maturity model (CMM). In fact, hybrid approaches of various kinds are now more common than single-flavor technologies.

Outsourcing

The outsourcing sub-industry has continued to grow and prosper in both the United States and abroad. In the 1991 edition of this book, outsourced or contract software projects were subsumed under the general headings of information systems, systems software, or military software. The outsource projects were broken out separately in the second edition.

The growth rate of the outsourcing domain and the comparatively high productivity and quality levels of the outsource community deserve to have outsource projects shown as a separate category. Since many outsourcers are, in fact, superior to their clients in software development methods, the overall results of the outsource domain contribute to the general improvements in software productivity and quality reflected in this edition.

In terms of work hours per function point, the outsource world typically exceeds their clients by somewhere between 15 percent and 50 percent. Of course, outsource charges and costs are higher so the costs per function point are similar.

The outsourcing community that serves particular industries such as banking, healthcare, insurance, and so forth, tends to accumulate substantial volumes of reusable material within those industries. Part of the reason for higher productivity levels achieved by major outsourcers is due to the large volumes of reusable specifications, source code, test cases, and user manual sections for software within specific industries.

An issue of great economic importance involves offshore outsourcing to countries such as India, China, Russia, the Ukraine, and many other countries. Because of the low labor costs of these countries, they have become the preferred destinations for outsource contracts since about 2000. However, the inflation rates of the successful outsource countries are higher than those of the United States. If this inflation continues, and it seems likely that it will, by about 2015 the cost differentials between offshore work and U.S. work will disappear. Even in 2008, as this book is being written, the cost of living in Shanghai has pulled ahead of the cost of living in New York city. Inflation is also driving up costs in India. Therefore U.S. companies should do long-range economic planning before making major investments in offshore companies, since the immediate cost savings are likely to be ephemeral.

Service-Oriented Architecture (SOA)

One of the more interesting technologies to surface since about 2000 is that of "service-oriented architecture" or SOA. The SOA concept elevates software reusability from assembling applications out of small reusable components into attempting to join together rather large programs or full-sized systems by piping together their inputs and outputs. This is

not as easy as it sounds, and the SOA concept is an emerging technology rather than a fully formed technology.

As SOA matures and experience is gathered, the method should add value to the software industry, primarily in the upper ranges where today massive enterprise resource planning (ERP) applications hold sway.

As of 2007 ERP applications are close to 300,000 function points in size, but still only support about 30 percent of the functionality needed to actually operate a major corporation. In the future, it is possible to envision SOA applications topping 1,000,000 function points or being roughly three times larger than any existing software in the world.

Building a monolithic application of 1,000,000 function points would take more than 10 years by almost 2,000 people and is, therefore, economically unsound. But linking together a large collection of 10,000 function points, applications could probably be built in less than 2 years, by a team of only a 100 or so. However, in 2008, SOA is not yet at that level of sophistication, but it is a technology with great promise.

Six-Sigma for Software

The concept of Six-Sigma originated for hardware development at Motorola. The concept is mathematically appealing and is defined as one defect in a million opportunities. The Six-Sigma approach has been applied to software in principle and has achieved good success in improving quality measurements and quality technologies. However, the actual mathematical achievement of only one defect per million opportunities is outside the state of the art of software practices.

In recent years, the Six-Sigma concepts have formed synergistic alliances with other technologies such as the capability maturity model (CMM and CMMI) of the Software Engineering Institute.

Software Reusability

Software reuse was largely a theoretical topic in 1991 when this book was first published. It is still not a pervasive technology in terms of day-to-day reuse by large numbers of corporations. However, reusability is now beginning to grow to significant proportions.

Successful reuse covers many artifacts and not just source code. Reusable plans, estimates, specifications, data, user documentation, and test materials are some of the other reusable materials. An emerging sub-industry of tools and vendors are beginning to coalesce around reuse. It is time that this technology is emphasized in terms of its pragmatic results.

It is an interesting observation that software projects that approach or exceed productivity rates of 100 function points per staff month typically use in excess of 50 percent reusable code and also have high levels

of other reusable material such as specifications, test materials, and sections of user manuals.

The previous topic of "service-oriented architecture" dealt with elevating reuse from small modules up to entire programs or systems that connect via pipelines that feed inputs and outputs from one to another.

Software Engineering Institute (SEI) Capability Maturity Model (CMM)

The SEI CMM was in its infancy when the first edition of this book was published. Now it has more than 20 years of continuous usage and several years of evolution and modification. In the past, the author of this book has been critical of the SEI CMM as being incomplete, arbitrary, and lacking effective quantification of results. These problems still remain, but the SEI has not been idle. New features are being added to the CMM, and the penetration of the CMM continues to grow. The new capability maturity model integrated (CMMI) has replaced the original CMM approach. Watts Humphrey's new team software process (TSP) and personal software process (PSP) have also joined with the CMMI method.

In 1991, less than 10 percent of the author's clients had even heard of SEI or the CMM and less than 2 percent had used the SEI assessment method. By 1995, more than 50 percent of the author's clients have heard about the SEI, and about 10 percent have experimented with an SEI-style assessment. By today in 2008 about 75 percent of U.S. software organizations are familiar with the SEI and CMM, and usage has increased about 15 percent.

In spite of its faults, the SEI CMM and CMMI concepts have raised the awareness levels of many software houses about topics such as process improvement, quality, measurement, and metrics. Thus the SEI has become a useful contributor to the progress of software in the United States, and even abroad. Hopefully, the SEI will continue to improve on the original CMM concepts and will adopt modern quantitative metrics such as function points instead of the flawed "physical lines of code" or LOC metrics they first endorsed.

A study of software quality performed by the author's company in 1994 found some overlap among the SEI CMM levels. The best software produced by Level 1 organizations actually had higher quality levels than the worst produced by Level 3 organizations. However, there was also evidence to suggest that average quality levels and also productivity levels increased with each maturity level.

Today in 2008, there is substantial empirical evidence that the higher CMM and CMMI Levels 3, 4, and 5 have improved quality and productivity of the projects using them. Interestingly, the value of the CMM and CMMI are directly proportional to the size of the software applications

being developed. For small applications of 1,000 function points or lower, the CMM and CMMI tend to reduce productivity but raise quality. This is due to the overhead built-in to the CMMI approach. However, for large systems in the 10,000 and 100,000 function point size categories, the higher CMM and CMMI levels are quite beneficial to both productivity and quality. In fact, for applications larger than 10,000 function points, CMM Level 5 has about the best record of success yet recorded.

World Wide Web

When the first edition of this book was published in 1991 the World Wide Web was just starting to become a reality. By 1996, the Web was still in its infancy. Today in 2008, the Web has become the most important research tool in human history. Never before has so much information and data been so readily available. Not only that but the information is available 24 hours a day, 7 days a week, from almost any place in the world that has telephone service. In fact, with satellite phones, the Web is accessible even from the middle of the ocean.

Not only has the Web become the premier source of data and information, but also web software applications are now the major new software projects being constructed. When the first edition of this book was published, there were almost no web applications, but today in 2008, web applications comprise more than 20 percent of all new software projects in the world, and their numbers are rising faster than any other form of software.

Web application development is an interesting technology. Most web applications are fairly small (below 1,500 function points in size) and are developed using very powerful tools, programming languages, and reusable materials. As a result, productivity rates topping 25 function points per staff month are not uncommon.

Although web applications themselves are often studied and described in the literature, web content (images, text, animation, sound, etc.) is outside the scope of software engineering research. As of 2008, there are no metrics for measuring web content, and there are no standards for measuring the quality of web content. As a result, there is very little known about web content costs or economics, web content quality, and web content maintenance.

Changes in the Structure, Format, and Contents of the Third Edition

When the first edition of this book was published in 1991, this chapter on U.S. national averages showed overall data from approximately 4,300 projects as a series of simple graphs and tables. Selected portions of the data were broken down into three subcategories:

- Management information systems
- Systems software
- Military software

That approach was acceptable at the time because there was no other comparable collection of quantitative data available. Now that software measurements and benchmarking are entering the mainstream of software technologies, many companies and many industries now have significant amounts of data on their own performance. Therefore, this third edition will display the data in a much more detailed and granular, although complicated, fashion.

In the second edition, data from six sub-industries or domains was shown separately. The six sub-industries were

- End-user software produced for personal use by the developers themselves
- Information systems produced for internal business use within enterprises
- Contract or outsourced software systems produced for some other enterprise
- Commercial software produced for licensing or sale to clients
- Systems software that controls physical devices such as computers
- Military software produced under various Department of Defense standards

These six categories are not the only kinds of software produced. However, they were the most common types of software development projects in North America, South America, Europe, Africa, India, the Middle East, and the Pacific Rim.

In the third edition, web applications are now included in some of the tables as a separate category. Web applications have been on an explosive growth path and are increasing in numbers faster than any other form of software.

Another difference in the third edition is the attempt to show U.S. job losses due to offshore outsourcing to China, India, Russia, Ireland, and other major countries in the outsource domain. Interesting, offshore outsourcing has been most common for MIS applications, followed by commercial software. Offshore outsourcing has been much less common for web applications, systems software, and military software.

An eighth category identified as "other" is shown in some of the tables. This eighth category is a catch-all for various kinds of software where the author and SPR have not done enough assessment and baseline studies to display detailed information or reach overall conclusions;

i.e., the computer gaming industry, the software used for entertainment such as the animation sequence of *Jurassic Park,* the music software industry, specialized medical and scientific software, and the like.

The approximate 2008 population of personnel in these various categories is shown in Table 3-11.

Table 3-11 lumps together software engineers and related specialists such as quality assurance personnel, testers, software project managers, and technical writers. The most interesting aspect of Table 3-11 is the loss of personnel in the MIS sub-industry. This is due to three phenomena:

- Offshore outsourcing
- Migration to web applications
- Domestic outsourcing

The data in Table 3-11 is derived from various sources, such as corporate annual reports; government employment statistics; and web searches. It is not particularly accurate, but the trends between 1995 and 2007 are probably in the right ballpark. Both web applications and offshore outsourcing have been growing at a rapid rate, whereas other segments have been growing slowly.

Following are short discussions of what each of the categories means in the context of this book.

TABLE 3-11 Comparison of 1995 and 2007 Software Populations by Industry Segment

Industry Segment	1995	2007	Difference	Percent
Web	25,550	275,900	250,350	979.84%
MIS	1,048,800	915,250	(133,550)	–12.73%
U.S. outsource	175,950	185,900	9,950	5.66%
Commercial	123,063	220,500	97,437	79.18%
Systems/embedded	495,570	535,600	40,030	8.08%
Military	482,560	560,500	77,940	16.15%
Other	105,525	110,500	4,975	4.71%
Subtotal	2,457,018	2,804,150	347,132	14.13%
Offshore outsource	85,750	295,750	210,000	244.90%
End-users	8,000,000	10,000,000	2,000,000	25.00%
Total U.S.	**10,457,018**	**12,804,150**	**2,347,132**	**22.45%**
Total + Offshore	**10,542,768**	**13,099,900**	**2,557,132**	**24.25%**

Web Applications

The world of web applications includes major commercial web sites such as Amazon, Dell, and EBay, informational web sites such as those maintained by government agencies; and smaller web sites developed and maintained by corporations, nonprofit organizations, hospitals, and affinity groups such as Second Life, You Tube, and the like. As of 2008, the author estimates that more than 90 percent of commercial companies, more than 70 percent of government agencies, and about 60 percent of nonprofit organizations maintain web sites. There are also thousands of web sites that are hard to classify.

The nature of building web applications is somewhat different than conventional software applications. Web construction is aided by many powerful tools and languages. There is little effort devoted to traditional requirements and specifications, since with web sites the goal is to quickly get a working site or prototype up and running and use that to iron out the details. Web development is also somewhat sparse in terms of documentation, quality control, planning, estimating, and measurement compared to traditional applications.

However, dwarfing all of the differences in development is the main purpose of the web and the web sites: the content. Web content consists of text, images, sounds, music, animation, and interactive features such as order forms, surveys, forums, chat rooms, the like. For every function point in a web application, there may be as many as a thousand "web content points" in the information used by the web site.

As this book is written in 2008, the domain of web content remains largely unmeasured and unmeasurable. There are no good metrics for web content size, web content quality, cost of content acquisition, maintenance, etc.

End-User Software

The phrase "end-user software" refers to applications written by individuals who are not programmers or software engineers as their normal occupations. Many knowledge workers such as accountants, physicists, medical doctors, managers, and other professionals have the technical ability to write software if they choose to do so or have the time to do so.

There are currently about 120,000,000 workers in the United States. The author estimates that there are perhaps 10,000,000 managers, engineers, architects, accountants, and other knowledge workers who know enough about programming to build end-user applications using tools such as spreadsheets Visual Basic, or even C#, and the like.

The upper limit of applications which end-users can build, using current technologies, is approximately 100 function points in size.

The average size for end-user applications is closer to 10 function points in size.

There is no exact census of the number of companies where end-users have developed applications, but SPR estimates the U.S. total to be in the range of 50,000 companies including a host of smaller companies that employ no professional software personnel at all.

Similar ratios of professional software personnel to end-user software personnel are found in the other industrialized countries. On a global basis, there are perhaps 12,000,000 professional software personnel but more than 25,000,000 end-users outside the U.S. who can program.

Interestingly, the end-user population seems to be growing at more than 10 percent per year, whereas the professional software population is now down to single-digit growth rates and even declining within some industries. The population of end-users who can program constitutes one of the largest markets in the world, and a host of vendors led by Microsoft are bringing out tools and products at an accelerating rate.

As this new edition of the book is written, there are more questions about the quality, value, and economics of end-user developed software than there are answers. Nonetheless, it is a topic of growing importance due to the fact that knowledge of computer programming is becoming a fairly common business skill.

Management Information Systems (MIS)

The phrase "management information systems" or MIS refers to the kinds of applications that enterprises produce in support of their business and administrative operations: payroll systems, accounting systems, front and back office banking systems, insurance claims handling systems, airline reservation systems, and the like.

Many government agencies also produce management information systems, such as the various kinds of taxation software produced at local, state, and national levels; social security benefit tracking systems; and drivers license and vehicle registration systems.

The class of information systems constitutes a very broad range of application sizes and types, including but not limited to large mainframe systems in excess of 250,000 function points at the upper end of the spectrum, and small personal computer applications of less than 100 function points at the low end of the spectrum.

The large employment of information systems personnel is also true for every industrialized country in Europe, South America, and the Pacific Rim. The global information systems community is the largest in every industrialized country. The exceptions to this rule include China, Russia, and other countries that have lagged in the usage of computers for business and service-related purposes, as opposed to military and manufacturing purposes.

The major employers of information systems software personnel are the industries that were early adopters of mainframe computers: banks, insurance companies, manufacturing companies, government agencies, and the like. Insurance, for example, has at least a dozen major companies that employ many thousands of software personnel: Aetna, Travelers, CIGNA, Hartford, Sun Life, etc.

As this book is written in 2008, traditional MIS software actually has declining numbers of personnel. This is partly due to increased offshore and U.S. outsourcing and partly due to the explosive growth of web applications.

Of course, many large corporations that are better known for other kinds of software also produce information systems in significant volumes: AT&T, IBM, Boeing Computer Services, are also information systems producers. For that matter, the DoD and the military services produce huge volumes of information systems.

The information systems community is strongly allied with the concepts of databases, repositories, and data warehouses. The bulk of all database tools, languages, and methods are aimed squarely at the management information systems domain.

As of 2008, maintenance and enhancement of legacy applications is the dominant work of the MIS world. New applications are declining, but maintenance of legacy applications requires more and more personnel due to the decay of the applications themselves.

United States Outsourced and Contract Software

The phrase "outsourced software" refers to software produced under a blanket contract by which a software development organization agrees to produce all, or specific categories, of software for the client organization.

The phrase "contract software" refers to a specific software project that is built under contract for a client organization. There are a number of forms under which the contract might be negotiated: time and materials, fixed cost, and work for hire to name but three.

Note that this book deals with civilian contracts and civilian outsourcing primarily in the information systems domain. The military world has unique standards and procurement practices, and even a body of special contract law that does not apply in the civilian world (although parts of it may show up in civilian contracts with the U.S. Federal Government).

The systems software domain also has some extensive contract and outsource experiences, and indeed is more likely to go abroad for outsourcing than any other domain. However, in the United States the large outsource vendors such as ISSC, EDS, and Keane tend to concentrate on the information systems market.

The contract and outsource houses are more likely to be hired for major applications than for small ones. The upper limit of the outsource size range is well in excess of 100,000 function points. As pointed out in another of the author's books, *Patterns of Software System Failure and Success* (Thomson International, 1995), the outsource community has a better record of success for large systems than do their clients. (Although no domain can build large systems without some kind of problems or troubles occurring.)

Contract software and outsourced software are alike in that the work is performed by personnel who are not employees of the client organization, using terms and conditions that are mutually agreed to and covered by contractual obligations. Contract software and outsourced software differ in the overall scope of the arrangement. Contracts are usually for specific individual projects, whereas the outsource arrangement might encompass hundreds or even thousands of projects under one general contractual umbrella.

The software outsourcing community is growing, although much of the work in recent years has shifted to offshore locations. All of the major outsource companies now have operations abroad, and there are many local outsource vendors in China, India, Russia, and a score of other countries.

The diversity of organizations within the outsource and contract community runs from a host of one-person individual contractors up to the size of Electronic Data Systems with perhaps 45,000 software professionals. The total number of contract and outsource software personnel in the United States is roughly about 185,000. Some of the major outsource and software contracting organizations include Andersen, Keane, EDS, ISSC, Lockheed, Computer Aid Inc., Computer Sciences Corporation, and a host of others.

The volume of domestic outsourcing agreements between large information systems clients and outsource contractors is quite significant: currently about 10 percent of all information systems software in the United States is produced under contract or outsource agreements, and that volume is increasing rather rapidly.

Offshore Outsourcing

When the second edition was published in 1995, the total volume of IS software from the United States that had been outsourced internationally seemed to have been less than 1 percent of total IS applications, based on preliminary data collected during assessments performed by Software Productivity Research. The number of offshore outsource workers was perhaps 85,000. Today in 2008, the number of offshore workers involved with U.S. software is about 300,000.

The result of the explosive growth in offshore outsourcing has led to significant layoffs among IS software personnel. In fact, the domestic U.S. outsource companies could also face significant problems, unless they establish international subsidiaries, which most of them already have.

The United States is not the only country that is exploring international software outsource arrangements. Most countries with high labor costs, such as Japan, Sweden, the United Kingdom, Germany, etc. are also interested in software development or maintenance in countries with lower labor costs.

However, a natural but unexpected economic phenomenon has been occurring. The countries with the most successful outsourcing businesses have had higher rates of inflation than the United States itself. Therefore the very low labor rates that initially attracted U.S. companies to offshore outsourcers in China, India, Russia, and elsewhere are gradually disappearing. Indeed, as this book is written in 2008 and as mentioned previously, the cost of living in Shanghai recently pulled ahead of New York. India is undergoing rapid inflation and increasing labor rates. Moscow has become one of the most expensive cities in the world.

While there are still other countries with low labor costs (Bangladesh, Malaysia, Vietnam, North Korea, etc.) their stability, safety, and political climates do not always encourage software outsourcing. In any case, a basic economic principle seems to be emerging: success in outsourcing leads to rapid inflation. By about 2015, the major cost differentials between today's outsource leaders and the United States will probably have disappeared. In fact, if the dollar keeps declining against the Euro, the United States may once again be cheap enough to enter the global outsource market.

Commercial Software

Commercial software was not broken out as a separate category in the first edition of this book. The phrase "commercial software" refers to applications that are produced for large-scale marketing to hundreds or even millions of clients. Examples of commercial software include word processors such as WordPerfect or Microsoft Word, spreadsheets such as Excel, accounting packages such as Pacioli, project management tools such as Timeline, Microsoft Project, or Checkpoint, and a myriad of other kinds of software. At the upper end of the size spectrum of commercial software are enormous enterprise-resource planning tools such as SAP, Oracle, and PeopleSoft that approach 300,000 function points. Not far below that would be Microsoft Vista, at almost 200,000 function points in size.

Some commercial software houses are more specialized and aim at niches rather than broad-based general populations. For example,

companies such as Bellcore serve the communications industry. Software Productivity Research (SPR) and Quantitative Software Management (QSM) build specialized software cost-estimating tools. Relativity Technologies specializes in renovation of legacy applications.

The commercial software domain employees about 220,000 software personnel out of a current U.S. total of perhaps 2,800,000. Although there are more than 3,000 companies that produce commercial software in the United States, the major players are getting bigger and rapidly acquiring the smaller players.

The larger commercial software players include the pure software shops such as Microsoft, Computer Associates, Oracle, Knowledgebase, Symantec, and Lotus. The larger players also include many hybrid companies that build hardware as well as software: IBM, SUN, Apple, AT&T, and many others. There are also a host of specialized niches within the commercial software world: accounting software companies, gaming software companies, CASE companies, and many more.

Although commercial software packages are built throughout the world, the United States is currently the dominant player in global markets. Software Productivity Research estimates that about 85 percent of the commercial packages running in the United States originated here and that about 40 percent of the commercial software everywhere in the world has a U.S. origin. This is perhaps the most one-sided balance of trade of any U.S. industry.

As personal computers become more powerful, the size range of commercial software applications is exploding. In the DOS era, typical PC applications were in the size range of 200 to 1,000 function points. When Windows 95, Windows NT, and Windows XP unlocked the painful size restrictions on memory and addressability, modern PC applications are as large as mainframe systems of a decade ago. Indeed, Windows 95, Windows NT, and OS/2 were in the 50,000 function point range compared to MVS, UNIX, and other massive applications. The newer Windows Vista is even larger and is in excess of 200,000 function points. It is one of the largest software applications in the world and not far from the massive ERP packages in total size, due in part to all of the extra functions over and above the basic features of an operating system.

Many commercial word processors, spreadsheets, and other standard Windows applications are now in excess of 5,000 function points in size. This explains why 8 to 16 megabytes of memory, gigabyte hard drives, or quad-speed CD ROM drives or higher are now the preferred equipment for running modern PC Windows-based applications. With Windows Vista, more than a gigabyte of real memory is needed just to achieve moderate performance levels.

There is a great deal of diversity in the commercial software domain, and this is due in part to the need to support diverse operating systems

and multiple hardware platforms: i.e., UNIX, Linux, Apple, Windows Vista, etc.

A common aspect of the commercial software domain is that the applications are built to appeal to a mass market, rather than built upon commission for a specific client or a small number of clients with explicit requirements.

The commercial software domain needs to balance a number of factors in order to be successful in the market place: the feature sets of the products, the quality levels of the delivered software, and the time to reach the relevant market.

Systems Software

In the context of this book "systems software" is defined as that which controls physical devices. The original archetype of systems software were operating systems such as CPM, DOS, OS/2, MVS, Windows NT, or UNIX that controlled computer hardware. Other examples of systems software includes telephone switching systems such as AT&T's ESS5 central office switching systems, smaller private branch exchange (PBX) switches, local area network (LAN) controllers and the like.

Also included under the general definition of systems software would be software that controls automobile fuel injection systems, civilian aircraft flight control software, medical instrument software, and even the software embedded in home appliances such as microwaves and clothes dryers.

The systems software world tends to build very large applications that are sometimes in excess of 200,000 function points in size. This domain also builds applications that may control complicated and expensive hardware devices where failures might be catastrophic; i.e., aircraft, spacecraft, telephone switching systems, manufacturing equipment, weapons systems, medical instruments, and the like.

The dual need for methods that can build large systems with high reliability has given the systems software world some powerful quality control approaches. Systems software is not very good in terms of productivity, but is a world leader in terms of quality.

The systems software domain in the United States contains overt 530,000 professional software personnel and managers, or roughly 25 percent of the approximate total of 2,800,000 U.S. software personnel. The number of companies that produce systems software total to more than 2,500 and include some of the largest software employers in the world. For example, both AT&T and IBM have more than 25,000 software personnel, and roughly half are in the systems software domain. Many other companies have 10,000 or more software personnel, such as Boeing, Lockheed, Ford, General Motors, General Electric, and Hewlett-Packard.

Systems software is often built for custom hardware platforms and may use custom or proprietary operating systems. Systems software also tends to utilize the UNIX operating system to a higher degree than other software classes. The systems software domain is also the main user of high-end development workstations. As a result of the special needs of systems software, the vendors serving this community are often quite distinct from those serving the information systems domain or the military domain.

Military Software

The phrase "military software" refers to software produced for a uniformed military service such as a nation's air force, army, or navy, or software produced for the Department of Defense (or the equivalent in other countries).

This broad definition includes a number of subclasses, such as software associated with weapons systems, with command, control, and communication systems (usually shortened to C3 or C cubed), with logistical applications, and also with software virtually identical to civilian counterparts such as payroll applications, benefits tracking applications, and the like.

The military software domain employs more than 550,000 software professionals in the United States, out of a total of about 2,800,000. There are about 1,500 defense contracting companies in the U.S., although probably 85 percent of all defense contract dollars goes to the top 50 of these groups.

The Department of Defense and the uniformed military services alone are reported to employ more than 80,000 although that number is partly speculative. (The annual software conferences sponsored by the Air Force's Software Technology Support Center attract about 3,000 attendees and ranks as one of the largest pure software conferences in the United States.)

The major defense and military employers are predominantly large corporations such as Lockheed Martin, Grumman Northrop, AT&T, Hughes, SAIC, Litton, Computer Sciences Corporation, GTE, Loral, Logicon, and the like.

The U.S. defense industry is currently going through a downsizing and shrinking phase. This is having some reduction on the defense software community. However, software is so important to modern military weapons systems that the reductions will hopefully not erode U.S. defense software capabilities.

The military software world has built the largest software systems in human history: some have exceeded 350,000 function points in overall size.

The United States is far and away the major producer and consumer of military and defense software in the world. The volume and sophistication of U.S. military software is actually a major factor of U.S. military capabilities. All those pictures of cruise missiles, smart bombs, and Patriot missiles destroying Scuds that filled television news during the First Gulf War had an invisible background: it is the software and computers on-board that makes such weapons possible.

Since the NATO countries tend to use many weapons systems, communication systems, logistics systems, and other software systems produced in the United States, it appears that the volume of U.S. defense and military software may be larger than the next five countries put together (Russia, China, Germany, United Kingdom, and France). Many countries produce military and defense software for weapons and communications systems that they use or market, such as Israel, Brazil, South Korea, Sweden, and Japan.

The total number of military software personnel outside of the United States is estimated by SPR to be about 1,200,000. The bulk of these are in Russia, the Ukraine, and China but there are also active military software projects and defense contractors in Iran, North Korea, Brazil, Argentina, Mexico, the United Kingdom, Canada, France, Israel, Sweden, Germany, Egypt, Turkey, Vietnam, Poland, Cuba, Iraq, Iran, and essentially every country with a significant military establishment.

Over the years the U.S. defense community has evolved an elaborate set of specialized standards and practices that differ substantially from civilian norms. Although these military practices and Department of Defense standards are not without merit, they have tended to be so cumbersome and baroque that they sometimes served more as impediments to progress rather than benefiting either the defense contractors or the DoD itself.

Since the United States is the world's largest producer and consumer of military software, the way the U.S. goes about the production of military software is of global importance. In the United States in 1994, William Perry, the Secretary of Defense, issued a major policy statement to the effect that DoD standards no longer needed to be utilized. Instead, the armed services and the DoD were urged to adopt civilian best current practices. This change in policy is in the early stages of implementation, and it is too soon to know how effective or even how pervasive the changes might be. However, since 2000 military projects have continued to outpace civilian projects in adopting the capability maturity model, so progress is reasonably good through 2008.

Variations in Software Development Practices Among Seven Sub-Industries

The seven different sub-industries included in this new edition vary significantly in productivity rates, quality levels, schedules, and many other quantitative matters. The quantitative differences are really reflections of some very significant variations in how software is built within each of the domains.

Variations in Activities Performed

One of the most visible differences that can easily be seen from SPR's assessment and baseline studies are the variations in the activities that are typically performed. SPR's standard chart of accounts for software baselines includes 25 activities. Only large military projects perform all 25, and the variances from project to project and company to company are both interesting and significant.

The new web domain is an interesting combination of fairly small projects and comparatively flexible and free-form development methods. This approach has been fairly successful, but needs to be reevaluated as web applications grow in size.

The end-user software domain averages four activities, and in some cases, may perform only a single activity, coding.

The management information systems domain averaged from 12 to 18 activities for mainframe software. Client-server projects are averaging only 6 to 12 activities, which explains in part why quality has declined in the MIS domain.

The outsource domain is somewhat more rigorous than the MIS domain and runs from 15 up to perhaps 20 activities.

The commercial software, systems software, and military software domains all perform more than 20 activities on average, and the military usually performs all 25. Since many of the activities in these domains are quality related, it can be seen why the quality control results are typically better than the rudimentary MIS domain.

Table 3-12 shows two important factors:

- The most common activities that are likely to be performed
- The approximate percentage of effort devoted to each activity

Since there are major variations in activity patterns associated with project size as well as with domain, assume that Table 3-12 is derived from software projects that are nominally 1,500 function points in size.

In Table 3-12 the activity of "independent verification and validation" is too long to spell out completely so it has been abbreviated. This activity involves hiring an independent contractor to check standards and

TABLE 3-12 Software Development Activities Associated with Seven Sub-Industries (Percentage of Staff Months by Activity)

Activities Performed	End-User	MIS	Outsource	Commercial	Systems	Military	Web
01 Requirements		7.50	9.00	4.00	4.00	7.00	
02 Prototyping	10.00	2.00	2.50	1.00	2.00	2.00	10
03 Architecture		0.50	1.00	2.00	1.50	1.00	
04 Project plans		1.00	1.50	1.00	2.00	1.00	
05 Initial design		8.00	7.00	6.00	7.00	6.00	
06 Detail design		7.00	8.00	5.00	6.00	7.00	
07 Design reviews			0.50	1.50	2.50	1.00	
08 Coding	35.00	20.00	16.00	23.00	20.00	16.00	33
09 Reuse acquisition	5.00		2.00	2.00	2.00	2.00	5
10 Package purchase		1.00	1.00		1.00	1.00	1
11 Code inspections				1.50	1.50	1.00	
12 Ind. Verif. & Valid.						1.00	
13 Configuration management		3.00	3.00	1.00	1.00	1.50	
14 Formal integration		2.00	2.00	1.50	2.00	1.50	
15 User documentation	10.00	7.00	9.00	12.00	10.00	10.00	10
16 Unit testing	40.00	4.00	3.50	2.50	5.00	3.00	30
17 Function testing		6.00	5.00	6.00	5.00	5.00	
18 Integration testing		5.00	5.00	4.00	5.00	5.00	
19 Systems testing		7.00	5.00	7.00	5.00	6.00	
20 Field testing				6.00	1.50	3.00	
21 Acceptance testing		5.00	3.00		1.00	3.00	
22 Independent testing						1.00	
23 Quality assurance			1.00	2.00	2.00	1.00	
24 Installation/ training		2.00	3.00		1.00	1.00	1
25 Project management		12.00	12.00	11.00	12.00	13.00	10
Total	**100**	**100**	**100**	**100**	**100**	**100**	**100**
Activities	5	16	20	21	22	25	8

quality activities. This activity is restricted to military projects where it is a requirement. The normal abbreviation in military parlance is "IV&V" but many civilians do not know what the abbreviation means so it seemed best to explain it.

The data in Table 3-12 should not be used as a cost estimating template. Whenever data is shown using percentages, note that the inclusion or removal of any activity would throw off all of the percentages. Table 3-12 illustrates the fact that the various domains have significant variances in the processes usually followed for building software.

Even a cursory examination of Table 3-12 shows why productivity variations are so large from domain to domain: variation in the number of activities performed is sufficient to cause significant productivity differences.

A more careful examination also sheds light on why quality variations occur, too. Note the percentages of effort devoted to quality control, inspections, and testing in the systems and military domains compared to the percentages in the end-user and MIS domains.

Variations in Defect Potentials, Defect Prevention, Defect Removal, and Quality Control

There are notable differences in the sets of defect prevention and defect removal activities among the seven domains. In order to give a context to these differences, three important concepts need to be understood: (1) Defect potentials; (2) Defect prevention; and (3) Defect removal.

The concept of "defect potential" refers to the sum total of possible errors in software from five separate sources: (1) Errors in requirements; (2) Errors in design; (3) Errors in source code; (4) Errors in user documentation; and (5) Errors associated with "bad fixes" or secondary errors introduced while fixing a primary error.

The defect potentials of software applications have been derived from long-range studies of software defect reports over periods of several years. The first such studies known to the author were carried out at IBM during the 1960s and 1970s on products such as the OS/360 operating system and the IMS database.

All defect reports were accumulated from requirements through development and out into the field for several years of maintenance and enhancement. The defect origins were also explored to see how many errors could be traced back to requirements, to design, to code, to user manuals, or to poor quality control during the defect repair process.

The current range of potential defects is from a low of less than one defect per function point to a high of more than ten defects per function point, as will be shown later in this chapter. However, the function point metric provides a fairly useful approximation for overall defect

potentials: Raise the function point total of the application to the 1.25 power and the result will give a rough estimate of the total volume of errors or bugs that may be encountered.

Table 3-13 illustrates the overall distribution of software errors among the various categories of origin points.

Because percentages are abstract, Table 3-13.1 shows the probable number of defects per function point for the same combination of industry segments and defect origins shown in Table 3-13.

The concept of "defect prevention" is the most difficult to study and quantify. Here is an example of how assumptions on defect prevention are derived.

Assume that you have two projects of the same size and nominal complexity; say 1,000 function points. Assume that one of the projects developed a prototype, while the other project did not.

When the defects for the two projects are accumulated, assume that design reviews found 200 bugs or errors in the project that did not build a prototype and only 100 bugs or errors in the project that did build a prototype. It can be hypothesized that the prototype had the effect of preventing 100 potential bugs or errors that might otherwise have occurred.

Of course, one example is not enough to really form such a hypothesis. But if 50 projects that built prototypes were compared to 50 similar projects that did not, and the number of design defects had a 2 to 1 average difference in favor of prototypes, then the hypothesis would be reasonable.

The concept of defect removal efficiency is a very important one for quality control and quality assurance purposes. Defect removal efficiency is normally calculated on the anniversary of the release of a

TABLE 3-13 Software Defect Origin Percent by Industry Segment

	Require. Bugs	Design Bugs	Code Bugs	Document Bugs	Bad Fix Bugs	Total
MIS	15%	30%	35%	10%	10%	100%
Web	40%	15%	25%	5%	15%	100%
U.S. outsource	20%	25%	35%	10%	10%	100%
Offshore outsource	25%	25%	25%	12%	13%	100%
Commercial	10%	30%	30%	20%	10%	100%
Systems	10%	25%	40%	15%	10%	100%
Military	20%	20%	35%	15%	10%	100%
End-user	0%	15%	55%	10%	20%	100%
Average	**18%**	**23%**	**35%**	**12%**	**12%**	**100%**

TABLE 3-13.1 Software Defects Per Function Point by Industry Segment

	Require. Bugs	Design Bugs	Code Bugs	Document Bugs	Bad Fix Bugs	Total Bugs
MIS	0.75	1.50	1.75	0.50	0.50	5.00
Web	1.68	0.63	1.05	0.21	0.63	4.20
U.S. outsource	0.95	1.19	1.66	0.48	0.48	4.75
Offshore outsource	1.38	1.38	1.38	0.66	0.72	5.50
Commercial	0.60	1.80	1.80	1.20	0.60	6.00
Systems	0.65	1.63	2.60	0.98	0.65	6.50
Military	1.40	1.40	2.45	1.05	0.70	7.00
End-user	–	0.45	1.65	0.30	0.60	3.00
Average	**0.93**	**1.25**	**1.79**	**0.67**	**0.61**	**5.24**

software product. Suppose that during development, a total of 900 bugs or errors were found. During the first year of use, customers found and reported another 100 bugs.

On the first anniversary of the product's release, the 900 bugs found during development are added to the 100 bugs customers reported to achieve a total of 1,000 bugs for this example. Since the developers found 900 out of 1,000, their defect removal efficiency can easily be seen to be 90 percent.

The measurement of defect removal efficiency levels is one of the signs of a "best in class" software producer. Not only do best in class organizations measure defect removal efficiency levels, but they average more than 95 percent removal efficiency levels for their entire software portfolio.

It is comparatively easy to go above 95 percent in defect removal efficiency levels for small applications of less than 100 function points in size. As the size of the application grows, defect removal efficiency levels typically decline unless very energetic steps are taken.

For applications larger than 10,000 function points and especially so for those approaching 100,000 function points, defect removal efficiency levels in excess of 95 percent are only possible by using a multistage set of pre-test removal activities such as formal inspections, coupled with an extensive suite of formal testing stages.

A simple rule of thumb can approximate the number of discrete defect removal operations needed to go above 95 percent in cumulative defect

removal efficiency: Raise the size of the application in function points to the 0.3 power and express the result as an integer:

Function Points	Defect Removal Stages
1	1
10	2
100	4
1,000	8
10,000	16
100,000	32

Table 3-14 illustrates the variations in typical defect prevention and defect removal methods among the major industry domains.

Table 3-14 oversimplifies the situation, since defect removal activities have varying efficiencies for requirements, design, code, documentation, and bad fix defect categories. Also, each defect removal operation has a significant range of performance. Unit testing by individual programmers, for example, can range from less than 20 percent efficiency to more than 50 percent efficiency.

Also, and this is an important point, the numbers of defect removal operations will vary considerably from those shown in Table 3-14. For example, not all systems software projects actually use 16 different kinds of defect prevention and removal operations nor does every military project use 18. There are variations in every industry segment. The data in Table 3-14 for systems software, for example, reflects the combined defect removal methods noted in 25 companies and more than 200 projects.

From observations among the author's clients, the minimum number of defect removal activities is one (unit test). The average number of defect removal operations is six:

- Prototypes
- Desk checking
- Unit testing
- New function testing
- Regression testing
- Acceptance testing

The maximum number of defect prevention and removal operations noted by the author was 22 for a large defense application.

TABLE 3-14 Patterns of Defect Prevention and Defect Removal Activities

	End-User	Web	MIS	Outsource	Commercial	Systems	Military
Prevention							
Prototypes	20.00%	20.00%	20.00%	20.00%	20.00%	20.00%	20.00%
Clean rooms						20.00%	20.00%
JAD sessions			30.00%	30.00%			
QFD sessions						25.00%	
Scrum sessions		30.00%					
Subtotal	20.00%	44.00%	56.00%	44.00%	20.00%	52.00%	36.00%
Pretest Removal							
Desk checking	15.00%	15.00%	15.00%	15.00%	15.00%	15.00%	15.00%
Requirements revision				30.00%	25.00%	20.00%	20.00%
Design review				40.00%	45.00%	45.00%	30.00%
Document review					20.00%	20.00%	20.00%
Code inspections					50.00%	60.00%	40.00%
Independent verification and validation							20.00%
Correctness proofs							10.00%
Usability labs					25.00%		
Subtotal	15.00%	15.00%	15.00%	64.30%	89.48%	88.09%	83.55%
Testing Activities							
Unit test	30.00%	30.00%	25.00%	25.00%	25.00%	25.00%	25.00%
New function test			30.00%	30.00%	30.00%	30.00%	30.00%
Regression test				20.00%	20.00%	20.00%	20.00%
Integration test			30.00%	30.00%	30.00%	30.00%	30.00%
Performance test					15.00%	15.00%	20.00%
Systems test			35.00%	35.00%	35.00%	40.00%	35.00%
Independent test							15.00%
Field test					50.00%	35.00%	30.00%
Acceptance test		25.00%		25.00%		25.00%	30.00%
Subtotal	30.00%	52.50%	76.11%	80.89%	91.88%	92.58%	93.63%
Cumulative Efficiency	52.40%	75.01%	88.63%	96.18%	99.32%	99.58%	99.33%
Number of Activities	3	4	7	11	14	16	18

Table 3-14 shows the defect prevention and removal operations noted among many clients in the same industry segment. Each individual company can use either fewer or more operations than the ones indicated.

However, Table 3-14 does indicate that systems and defense software applications use more defect removal activities than the other segments and are, therefore, higher in removal efficiency levels.

Overall, formal inspections have the highest average efficiency levels: both design and code inspections average about 80 percent in efficiency, and can go above 90 percent. Inspections are higher in efficiency than any known form of testing, with one exception. The exception is the kind of high-volume beta testing practiced by Microsoft, where more than 10,000 clients test a software package concurrently.

Another useful rule of thumb is that any given defect removal operation will find about 30 percent of the bugs that are present. This fairly low level of average defect removal efficiency explains why it takes up to 18 discrete defect removal operations to top 99 percent in cumulative defect removal efficiency levels.

Exploring Activity-Based Costing

Table 3-15 shows the overall ranges of software productivity expressed in two complementary formats: (1) Function points per staff month; and (2) Work hours per function points.

One of the purposes of all three editions of this book is to move further toward the concept of activity-based costing of software projects. Software data that is collected only to the levels of complete projects, or to the levels of six to eight software development phases, is not accurate enough for either serious economic analysis or for cost estimating of future projects.

The arithmetic mean of the data in Table 3-15 is not very useful information. The cumulative results, on the other hand, are actually both interesting and useful. Experience has shown that the concept of cumulative results is difficult to grasp, so a simple example may clarify the situation.

Suppose we were building a simple application of 100 function points in size, and we spent one month in coding the application and one month in testing the application.

Our productivity rate for coding is obviously 100 function points per month, and our productivity rate for testing is also 100 function points per month. Therefore the arithmetic mean of our work is also 100 function points per month, since we performed two activities at the same rate.

However, the sum of our work on coding and testing was two months of effort. Therefore our real productivity for this project is 50 function points per staff month using the cumulative results. This is easily calculated by

TABLE 3-15 Maximum, Minimum, and Modal Productivity Ranges for Software Development Activities

	Function Points per Month			Work Hours per Function Point		
Activities Performed	Minimum	Mode	Maximum	Maximum	Mode	Minimum
01 Requirements	50.00	175.00	350.00	2.64	0.75	0.38
02 Prototyping	25.00	150.00	250.00	5.28	0.88	0.53
03 Architecture	100.00	300.00	500.00	1.32	0.44	0.26
04 Project plans	200.00	500.00	1,500.00	0.66	0.26	0.09
05 Initial design	50.00	175.00	400.00	2.64	0.75	0.33
06 Detail design	25.00	150.00	300.00	5.28	0.88	0.44
07 Design reviews	75.00	225.00	400.00	1.76	0.59	0.33
08 Coding	15.00	50.00	200.00	8.80	2.64	0.66
09 Reuse acquisition	400.00	600.00	2,000.00	0.33	0.22	0.07
10 Package purchase	350.00	400.00	1,500.00	0.38	0.33	0.09
11 Code inspections	75.00	150.00	300.00	1.76	0.88	0.44
12 Ind. verification & validation	75.00	125.00	200.00	1.76	1.06	0.66
13 Configuration management	1,000.00	1,750.00	3,000.00	0.13	0.08	0.04
14 Formal integration	150.00	250.00	500.00	0.88	0.53	0.26
15 User documentation	20.00	70.00	100.00	6.60	1.89	1.32
16 Unit testing	70.00	150.00	400.00	1.89	0.88	0.33
17 Function testing	25.00	150.00	300.00	5.28	0.88	0.44
18 Integration testing	75.00	175.00	400.00	1.76	0.75	0.33
19 System testing	100.00	200.00	500.00	1.32	0.66	0.26
20 Field testing	75.00	225.00	500.00	1.76	0.59	0.26
21 Acceptance testing	75.00	350.00	600.00	1.76	0.38	0.22
22 Independent testing	100.00	200.00	300.00	1.32	0.66	0.44
23 Quality assurance	30.00	150.00	300.00	4.40	0.88	0.44
24 Installation/ training	150.00	350.00	600.00	0.88	0.38	0.22
25 Project management	15.00	100.00	200.00	8.80	1.32	0.66
Cumulative Results	1.90	6.75	13.88	69.38	19.55	9.51
Arithmetic mean	**133**	**284.8**	**624**	**2.78**	**0.78**	**0.38**

dividing the 100 function points in our application by the total amount of effort expended.

Assuming that there are 132 productive working hours in a month, we can perform the same analyses using work hours per function point. Our effort devoted to coding was 132 hours, and our effort devoted to

testing was 132 hours. Since the project is 100 function points in size, each activity amounted to 1.32 work hours per function point. Therefore the arithmetic mean of our two tasks was also 1.32 work hours per function point.

Here too, our total effort summed to two months, so our real productivity using the harmonic mean is 2.64 hours per function point. This is because we spent a total of 264 hours on this 100 function point project; i.e., 264 hours divided by 100 function points.

Software Productivity Improvement Rates

Cumulative results do not change very rapidly, because the weight of past experience tends to act as a damper. In order to see how productivity changes over time, it is interesting to aggregate projects by decade, by half-decade, or by year. For example, it is interesting to compare all projects that entered production in 2005 against all projects that entered production in 1990, 1985, and 1980, and so on back into the past.

The author has had the good fortune of having access to software productivity and quality data for almost 35 years. While working for IBM in the 1960s and 1970s on various productivity and quality improvement programs, all of IBM's internal data was readily available. In addition, the author served as an IBM liaison to various customers on productivity and quality matters, so quite a bit of external data was also available.

The author also had access to internal and external data associated with ITT and with many clients of the Nolan & Norton consulting company. The author's own company, Software Productivity Research, has hundreds of clients and hence data on thousands of software projects. Recent data has also become available from the International Software Benchmarking Standards Group (ISBSG).

As an experiment, the author "backfired" selections from his available historical data as far back as 1945 and plotted the results at five-year intervals using, when possible, projects entering production in each specific year (see Table 3-16). To complete the experiment, the author also projected the data forward to the year 2010 A.D. This experiment has a high margin of error and especially so for data collected prior to 1975.

The results are interesting even if unreliable. It is particularly interesting to see the impact of end-user software projects on overall results.

Table 3-16 attempts to show a long-range picture starting at the end of World War II and then going forward to about 2010. The data between 1985 and 2005 is reasonably accurate. Before 1985 much of the historical data could not be validated. After 2005 much of the data is projected rather than historical and so has a high but unknown margin of error.

The overall rate of improvement is fairly significant, and especially so when end-user applications are added to national results. The author is of two minds about the inclusion of end-user projects in national results.

TABLE 3-16 U.S. Software Productivity Rates at Five-Year Intervals from 1945 to 2010 AD (Productivity Expressed in Terms of Function Points per Staff Month)

Year	End-User	MIS	Web	Outsource	Commercial	Systems	Military	Average
1945							0.10	0.10
1950		1.00				0.50	0.30	0.60
1955		1.50				0.60	0.40	0.83
1960		2.00			1.50	1.00	0.50	1.25
1965		3.00			2.50	1.50	0.70	1.93
1970		4.00			3.50	2.00	1.00	2.63
1975	9.00	5.00		5.50	6.00	2.50	1.50	4.83
1980	12.00	5.50		6.00	6.50	3.50	2.25	5.96
1985	20.00	6.00		7.00	7.00	4.00	2.50	7.75
1990	35.00	8.75		9.25	10.00	4.50	3.00	11.75
1995	40.00	11.00	14.00	13.00	10.50	6.00	3.50	14.00
2000	50.00	13.00	18.00	16.00	11.00	9.00	5.00	17.43
2005	52.00	15.00	24.00	17.00	12.00	9.50	5.25	19.25
2010	54.00	17.00	28.00	18.00	14.00	10.00	5.50	20.93
Average	**34.00**	**7.13**	**21.00**	**11.47**	**7.68**	**4.20**	**2.00**	**12.50**

On the pro side, end-user software applications are a significant part of the U.S. software world. End-user applications might grow in volumes and sophistication as computer literacy improves among knowledge workers.

On the con side, end-user applications are not usually "owned" by corporations or government agencies. They have a short life expectancy, and can appear and disappear at random intervals based on job changes. End-user applications are not part of formal software portfolios, are not under formal configuration control, and usually have no quality assurance reviews of any kind. They also introduce significant bias into other results, because end-user software lacks requirements, specifications, test plans, test libraries, and often user documentation. Hence productivity levels are high enough to dominate averages if end-user results are included.

Figure 3-1 gives a visual representation of approximate software productivity trends from just after World War II until 2010.

A final caution about end-user software is that essentially *all* end-user applications are less than 100 function points in total size. The other domains of software have applications whose sizes run up to more than 300,000 function points.

There is a tendency in the software press to assert that because end-user productivity rates are high compared to everything else, that end-user development will put professional software personnel

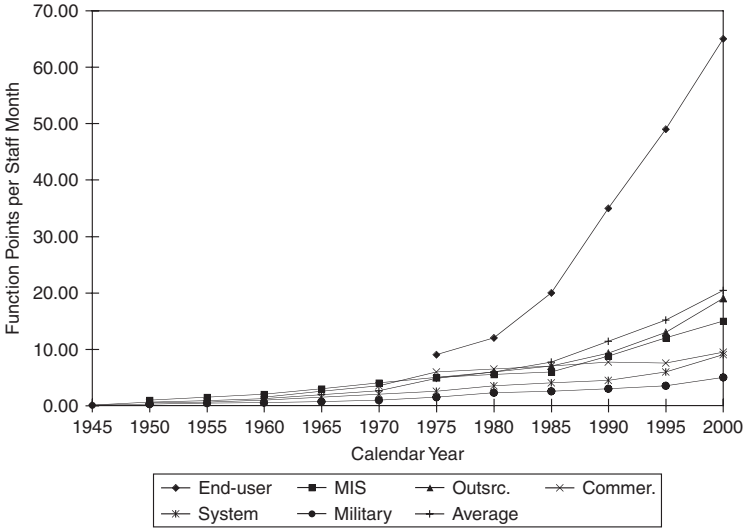

Figure 3-1 U.S. software productivity rates at five-year intervals

out of business. This is a distorted and untrue hypothesis. Large systems cannot be done at all by end users. For any application where quality, reliability, or formal change management are necessary, end-user development is a poor choice.

End-user development is also a poor choice for situations where there are whole classes of users, such as the class of project managers or the class of insurance sales personnel. It would be sheer folly to have 1,000 managers or 2,000 salespeople each attempt to develop end-user applications in the same general domain.

The chapter heading of "U.S. National Averages for Software Productivity and Quality" is probably an exaggeration when all of the sources of error are considered. However, as stated earlier, if preliminary results remain unpublished, there is no incentive to correct errors and improve on those results because other researchers do not have access to the information.

Hopefully this third edition of the information on U.S. national averages might have the following results:

- Development of international guidelines for what constitutes "national averages." This guideline should address important topics such as whether end-user applications should or should not be included in national results.
- A meeting or conference where all organizations with national databases or knowledge bases of software productivity, schedule, and quality data

can meet and discuss the differences and similarities of their contents. Right now the Gartner Group, the Air Force, Quantitative Software Management (QSM), the Software Engineering Institute (SEI), The Software Productivity Consortium (SPC), The International Software Benchmark Standards Group (ISBSG), the Australian Software Metrics Association (ASMA), the International Function Point User's Group (IFPUG), the British Mark II Function Point Users Group, and Software Productivity Research (SPR) all have significant volumes of data, but the results vary by as much as several hundred percent for unknown reasons. Since all of these groups are more or less in competition with one another, a general council of software data collection groups would need to be sponsored by a neutral organization such as the IEEE, DPMA, or perhaps the emerging National Software Council. The goal is to move toward common data collection and analysis practices.

- An international standard for software benchmarking that would define the activities to be included, the kinds of influential "soft" factors to be collected, and the kind of statistical analysis that should be performed on the results.
- Conversion rules among the various flavors of function point metrics; i.e., how to convert data from IFPUG to Mark II to Boeing 3D to COSMIC to SPR feature points and so forth. At the moment, the various function point associations are somewhat narrow and parochial and tend to ignore rival methods.
- General agreement on the ranges and values to utilize when "backfiring" or converting data between the older "lines of code" metric and the newer family of function point metrics.
- A methodology for determining the taxable value of software that is not as fraught with errors and ambiguity as the current approach used by the U.S. Internal Revenue Service (IRS) and the equivalent organizations in other countries.

Ranges, Averages, and Variances in Software Productivity

The most common method for expressing data is to use average values, primarily the arithmetic mean. Although averages are interesting, they tend to be deceptive. This is because the ranges on either side of average values are quite broad for every software topic of interest:

- Productivity
- Schedules

- Staffing
- Costs
- Defect potentials
- Defect removal efficiency levels
- Delivered defects
- Maintenance
- Enhancements

Because ranges are so broad, a major factor in exploring software productivity, quality, schedules, or other tangible matters is that of the range of possible outcomes. This is not an uncommon phenomenon. For example, the ranges in the height or weight of adult humans is also very broad. Similarly, the ranges of per capita incomes, the range of the time required to run one mile, and many other phenomena have broad ranges.

For software, an interesting question is "What is the probability of a given software project achieving a specific productivity rate?" Table 3-17 gives the probabilities based on application size as reflected from the SPR knowledge base.

In Table 3-17, the mode or most likely productivity range is underscored so that it can be easily seen. For the most common size ranges of software projects, i.e., projects between 10 and 1,000 function points in size, the overall range is from less than 1 function point per staff month to more than 100 function points per staff month.

Another way of looking at the data examines the probabilities of achieving various productivity rates based on the domain or sub-industry. Table 3-18 shows information similar to Table 3-17, only based on the various software domains that are now dealt with separately.

TABLE 3-17 Productivity Probability Ranges for Selected Software Application Sizes (Productivity Expressed in Terms of Function Points per Staff Month)

Productivity	1FP	10FP	100FP	1000FP	10000FP	100000FP
>100	10.00%	7.00%	1.00%	0.00%	0.00%	0.00%
75–100	20.00%	15.00%	3.00%	1.00%	0.00%	0.00%
50–75	**34.00%**	22.00%	7.00%	2.00%	0.00%	0.00%
25–50	22.00%	**33.00%**	10.00%	3.00%	0.00%	0.00%
15–25	10.00%	17.00%	**35.00%**	20.00%	2.00%	0.00%
5–15	3.00%	5.00%	26.00%	**55.00%**	15.00%	20.00%
1–5	1.00%	1.00%	10.00%	15.00%	**63.00%**	35.00%
<1	0.00%	0.00%	8.00%	4.00%	20.00%	**45.00%**
Total	**100.00%**	**100.00%**	**100.00%**	**100.00%**	**100.00%**	**100.00%**

TABLE 3-18 Productivity Probability Ranges for Selected Industry Segments (Data Expressed in Terms of Function Points per Staff Month)

Productivity	End-User	Web	MIS	Outsource	Commercial	Systems	Military
>100	7.00%	2.00%	1.00%	2.00%	3.00%	0.00%	0.00%
75–100	15.00%	8.00%	4.00%	5.00%	7.00%	0.00%	0.00%
50–75	**40.00%**	20.00%	7.00%	12.00%	10.00%	1.00%	0.00%
25–50	22.00%	**45.00%**	12.00%	16.00%	15.00%	10.00%	7.00%
15–25	10.00%	17.00%	23.00%	25.00%	20.00%	17.00%	12.00%
5–15	5.00%	5.00%	**35.00%**	**28.00%**	**27.00%**	**33.00%**	24.00%
1–5	1.00%	3.00%	13.00%	10.00%	15.00%	27.00%	**42.00%**
<1	0.00%	0.00%	5.00%	2.00%	2.00%	12.00%	15.00%
Total	**100.00%**	**100.00%**	**100.00%**	**100.00%**	**99.00%**	**100.00%**	**100.00%**

As with Table 3-17, the mode or most likely productivity range is underlined so that it stands out from the other information.

When viewed as independent data collections, each of the columns in Tables 3-17 and 3-18 produce more or less normal bell-shaped curves. However, the two tables tend to peak at different places because the sizes of typical applications vary with the class of application. That is, military software projects are usually larger than civilian projects, and end-user software projects are usually smaller than any other category.

Another significant variance that changes with both size and domain of the application is the overall distribution of effort into major activities. Table 3-19 shows four overall categories of work: (1) Work associated with production of paper documents; (2) Work associated with production of source code; (3) Work associated with finding and eliminating bugs or defects; and (4) Work associated with project management tasks.

As can be seen from Table 3-19, coding is the dominant activity for projects below 1,000 function points in size, but coding is by no means the most expensive activity for software applications larger than 1,000 function points in size.

TABLE 3-19 Variations in Software Effort Associated with Software Size Ranges

	Management	Paperwork	Defect Removal	Coding	Total
1FP	5.00%	5.00%	20.00%	70.00%	100.00%
10FP	10.00%	7.00%	18.00%	65.00%	100.00%
100FP	12.00%	15.00%	23.00%	50.00%	100.00%
1000FP	15.00%	25.00%	30.00%	30.00%	100.00%
10000FP	16.00%	31.00%	35.00%	18.00%	100.00%
100000FP	17.00%	35.00%	36.00%	12.00%	100.00%
Average	**12.50%**	**19.67%**	**27.00%**	**40.83%**	**100.00%**

Table 3-20 presents the same kinds of variations, only now the variances are based on domain or software sub-industry.

Note that here, too, coding is not the dominant activity for applications other than end-user software. Software paperwork and software defect removal are often more expensive than coding. Surprisingly, for some projects, even managerial tasks can outweigh coding costs. This phenomenon occurs primarily in the military domain and is due to the extensive oversight and managerial work associated with defense contracts.

Because the ranges are so broad in Tables 3-17 through Table 3-20, it is obvious that quite a large number of factors are influencing the overall outcomes. Two have already been discussed: (1) Size and (2) Domain or class of software such as the seven categories shown here.

However, more than 100 other factors can influence the final results, including but not limited to:

- Novelty or difficulty of the application
- Volumes of reusable material available
- Volumes of paper documents produced
- Volumes of defects or errors encountered
- Experience levels of technical staff and project management
- Rate of "creeping requirements"
- Programming languages used
- Tools used
- Processes and methodologies used
- Geographic dispersal of software team
- Physical office environment for software team

TABLE 3-20 Variations in Software Effort for Selected Industry Segments

	Management	Paperwork	Coding	Defect Removal	Total
End-user	0.00%	5.00%	50.00%	45.00%	100.00%
Web	11.00%	7.00%	42.00%	40.00%	100.00%
MIS	12.00%	22.00%	33.00%	33.00%	100.00%
Outsource	14.00%	25.00%	27.00%	34.00%	100.00%
Commercial	11.00%	26.00%	27.00%	36.00%	100.00%
Systems	13.00%	34.00%	17.00%	36.00%	100.00%
Military	16.00%	47.00%	12.00%	25.00%	100.00%
Average	**11.00%**	**23.71%**	**29.71%**	**35.57%**	**100.00%**

Because it is easier to perceive trends from charts than from graphs, Figure 3-2 shows the overall ranges of software productivity from the SPR knowledge base.

The variances in software productivity rates are obviously greater at the small end of the spectrum than at the large end of the spectrum. Small projects have considerable latitude in methods, tools, and languages. Really large systems, on the other hand, tend to follow similar patterns or they tend to run out of control and get canceled.

Because many readers are not aware of application sizes, it is interesting to show the overall size range of software applications produced circa 2008. The size data in Table 3-21 is synthetic rather than actual, and the sizes are derived from mathematical sizing algorithms developed by the author. The function point sizes assume IFPUG counting rules, version 4.2. The source code counting rules assume logical statements rather than physical lines of code.

Because Table 3-21 uses mathematical algorithms for size prediction rather than actual counts for most examples, the margin of error is significant. However, Table 3-21 does span the approximate size range of software applications from the largest down to the smallest. Relative placement in the table is probably close to the mark, even if the actual sizes are in error.

Perhaps the most interesting feature of Table 3-21 is the pervasiveness of software in the modern world. Almost every complex machine in daily use is now controlled by software. In fact, the larger and more complex the machinery becomes, the more software it is likely to contain.

Not only complex physical devices, but also business transactions, government operations, military and defense, and security activities are now dominated by computers and software.

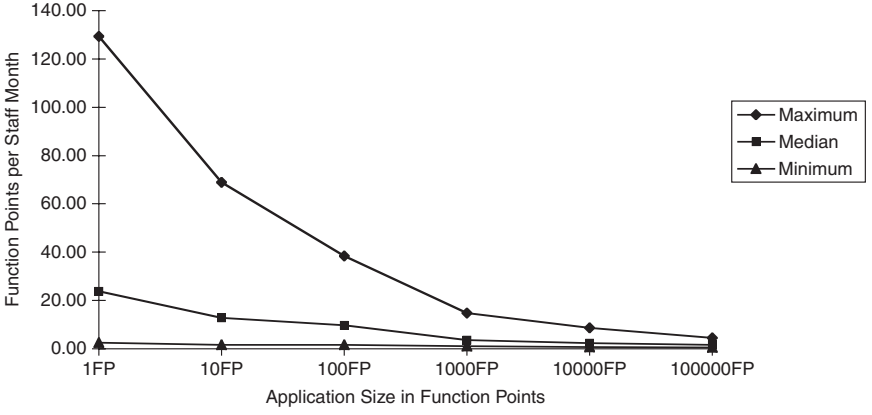

Figure 3-2 Overall ranges of U.S. software productivity levels

TABLE 3-21 Estimated Size Ranges of Selected Applications

Application	Size in Function Points (IFPUG 4.2)	Estimated Total Source Code	Lines per Function Point
1. U.S. Air Traffic control	306,324	65,349,222	213
2. Star Wars missile defense	352,330	32,212,992	91
3. WWMCCS	307,328	28,098,560	91
4. North Korean Border defenses	273,961	25,047,859	91
5. Oracle	310,346	24,827,712	80
6. Israeli air defense system	300,655	24,052,367	80
7. Iran's air defense system	260,100	23,780,557	91
8. SAP	296,764	23,741,088	80
9. NSA Echelon	293,388	20,863,147	71
10. Aegis destroyer C&C	253,088	20,247,020	80
11. IBM MVS	104,738	11,172,000	107
12. Microsoft VISTA	157,658	10,090,080	64
13. Microsoft XP	126,788	8,114,400	64
14. Airline reservation system	38,392	6,142,689	160
15. Microsoft Office Professional	93,498	5,983,891	64
16. VA Patient monitoring	23,109	4,929,910	213
17. NSA code decryption	35,897	3,829,056	107
18. FBI Carnivore	31,111	3,318,515	107
19. IBM IMS database	14,912	3,181,283	213
20. FBI fingerprint analysis	25,075	2,674,637	107
21. Nuclear reactor controls	19,084	2,442,747	128
22. NASA space shuttle	23,153	2,116,878	91
23. F115 avionics package	22,481	2,055,438	91
24. LexisNexis Legal analysis	22,434	2,051,113	91
25. Russian weather satellite	22,278	2,036,869	91
26. NASA Hubble controls	21,632	1,977,754	91
27. Shipboard gun controls	21,199	1,938,227	91
28. Oil refinery process controls	17,203	1,834,936	107
29. M1 Abrams battle tank	19,569	1,789,133	91
30. Boeing 787 avionics package	19,446	1,777,951	91
31. NASA Mars rover	19,394	1,773,158	91
32. Order entry system	18,052	1,650,505	91
33. Oil refinery process control	17,471	1,597,378	91
34. Corporate cost accounting	17,378	1,588,804	91
35. Tomahawk cruise missile	17,311	1,582,694	91
36. ITT System 12 telecom	17,002	1,554,497	91
37. ADP Payroll application	16,390	1,498,554	91
38. Inventory management	16,239	1,484,683	91
39. Natural language translation	20,350	1,447,135	71
40. American Express billing	20,141	1,432,238	71
41. Patriot missile controls	15,392	1,407,279	91

(*Continued*)

TABLE 3-21 Estimated Size Ranges of Selected Applications (*Continued*)

Application	Size in Function Points (IFPUG 4.2)	Estimated Total Source Code	Lines per Function Point
42. IRS income tax analysis	19,013	1,352,068	71
43. Brain/Computer interface	25,327	1,350,757	53
44. Cruise ship navigation	18,896	1,343,713	71
45. MRI medical imaging	18,785	1,335,837	71
46. Denver Airport luggage system	16,661	1,332,869	80
47. Google search engine	18,640	1,192,958	64
48. Skype	21,202	1,130,759	53
49. Data Warehouse	21,895	1,077,896	49
50. Motor vehicle registrations	10,927	999,065	91
51. America On Line (AOL)	14,761	944,713	64
52. FEDEX shipping controls	17,378	926,802	53
53. Ask search engine	16,895	901,060	53
54. Animated film graphics	21,813	872,533	40
55. Travelocity	19,383	775,306	40
56. Insurance claims handling	10,491	745,995	71
57. EBAY transaction controls	16,233	742,072	46
58. State-wide child support	13,823	737,226	53
59. Linux	17,505	700,205	40
60. Toyota robotic mfg.	14,019	690,152	49
61. Vonage VoIP	13,811	679,939	49
62. Quicken 2006	11,339	604,761	53
63. NVIDIA graphics card	3,573	571,637	160
64. Apple I Phone	19,366	516,432	27
65. SAS statistical package	10,380	511,017	49
66. Oracle CRM Features	6,386	510,878	80
67. Amazon web site	18,080	482,126	27
68. Apple Leopard	17,884	476,898	27
69. U.S. Patent applications	4,429	404,914	91
70. Second Life web site	14,956	398,828	27
71. Lasik surgery (wave guide)	3,505	373,832	107
72. Chinese submarine sonar	4,017	367,224	91
73. EZPass vehicle controls	4,571	325,065	71
74. Enterprise Java Beans	5,877	313,434	53
75. Microsoft DOS	1,344	286,709	213
76. Software renovation tool suite	5,170	275,750	53
77. Computer BIOS	857	274,243	320
78. Microsoft Excel 2007	3,969	254,006	64
79. Patent data mining	4,751	253,400	53
80. ITMPI web site	11,033	252,191	23
81. All-in-one printer	1,780	227,893	128
82. DNA Analysis	6,213	220,918	36

TABLE 3-21 Estimated Size Ranges of Selected Applications (*Continued*)

Application	Size in Function Points (IFPUG 4.2)	Estimated Total Source Code	Lines per Function Point
83. Citizens bank online	3,917	208,927	53
84. Microsoft Outlook	3,200	204,792	64
85. SpySweeper antispyware	2,108	192,757	91
86. Microsoft Word 2007	2,987	191,152	64
87. Bank ATM controls	3,625	178,484	49
88. Artemis Views	2,507	178,250	71
89. Sun D-Trace utility	3,309	176,501	53
90. Laser printer driver	1,248	159,695	128
91. Adobe Illustrator	2,151	152,942	71
92. MapQuest	3,793	151,709	40
93. PBX switching system	1,592	145,517	91
94. SPR Checkpoint	1,579	144,403	91
95. AutoCAD	1,768	141,405	80
96. Seismic analysis	1,492	136,438	91
97. Automobile fuel injection	842	134,661	160
98. Software code restructuring	1,658	132,670	80
99. Sidewinder missile controls	1,450	132,564	91
100. Anti-lock brake controls	826	132,144	160
101. SPR KnowledgePlan	1,785	126,963	71
102. Microsoft Project 2007	1,963	125,631	64
103. Microsoft Visual Basic	1,900	121,631	64
104. Palm OS	1,310	119,772	91
105. Windows Mobile	1,858	118,900	64
106. Norton antivirus software	2,068	110,300	53
107. ChessMaster 2007 game	2,227	109,647	49
108. PRICE-S	1,486	105,642	71
109. Toyota Prius engine	1,092	99,867	91
110. Property tax assessments	1,379	98,037	71
111. SLIM	1,355	96,342	71
112. Cochlear implant (internal)	1,041	95,146	91
113. IRA account management	1,281	91,096	71
114. Music synthesizer	1,134	90,736	80
115. Sony PlayStation game controls	1,622	86,502	53
116. Software complexity analyzer	1,202	85,505	71
117. COCOMO II	1,178	83,776	71
118. Microsoft Links golf game	1,564	83,393	53
119. Digital camera controls	1,285	82,243	64
120. Motorola cell phone	1,507	80,347	53
121. APAR analysis and routing	866	79,197	91
122. Configuration control	1,093	77,705	71

(*Continued*)

TABLE 3-21 Estimated Size Ranges of Selected Applications (*Continued*)

Application	Size in Function Points (IFPUG 4.2)	Estimated Total Source Code	Lines per Function Point
123. Smart bomb targeting	1,154	73,864	64
124. Mozilla Firefox	1,340	71,463	53
125. Consumer credit report	1,267	67,595	53
126. Casio atomic watch	993	63,551	64
127. Java compiler	1,185	63,186	53
128. COCOMO I	883	62,794	71
129. GPS navigation system	1,518	60,730	40
130. Intel Math function library	1,627	57,842	36
131. Nintendo Game Boy DS	1,002	53,455	53
132. CAI APO (original estimate)	1,332	53,288	40
133. Football bowl selection	992	52,904	53
134. Google Gmail	1,306	52,232	40
135. Function point workbench	714	50,800	71
136. SPR SPQR/20	699	49,735	71
137. Apple IPod	1,408	45,054	32
138. Instant messaging	687	43,944	64
139. Quick Sizer Commercial	794	42,326	53
140. LogiTech cordless mouse	736	39,267	53
141. CAI APO (revised estimate)	761	30,450	40
142. Wikipedia	1,142	30,448	27
143. Golf handicap analyzer	662	26,470	40
144. Denial of service virus	138	17,612	128
145. ILOVEYOU computer worm	22	2,838	128
146. Keystroke logger virus	15	1,886	128
147. MYDOOM computer virus	8	1,045	128
148. Quick Sizer prototype	30	480	16
149. APAR bug report	3.85	352	91
150. Screen format change	0.87	62	71
Average	33,269	2,152,766	65

Note 1: Sizes are estimated using the Quick Sizer algorithms.
Note 2: IFPUG counting rules, version 4.2 are assumed.
Note 3: Source code size is in terms of logical code statements.

Before turning to the tables and charts that contain overall results, following are brief summaries of seven "zones" of productivity information for each of the sub-industries included in this edition: end-user software, web applications, management information systems, outsourcers or contractors, commercial software, systems software, and military software.

Zone of End-User Software Projects

End-user software was not included in the first edition of this book. However, the increase in journal articles on this topic, the very significant increase in the numbers of end-users who program, and the improved tools available for this task, makes the subject worthy of inclusion.

End-user applications have the highest productivity rates of any domain. However, end-user applications are almost always less than 100 function points in size. The "average" end-user application (mode) hovers around 15 function points in size.

End-user productivity rates usually run from less than 25 function points per staff month to more than 150 function points per staff month and average about 35 function points per staff month.

However, end-user applications do not perform many of the work activities of the other classes of software shown in this book; i.e., no formal requirements, no formal specifications, no formal integration or configuration control, no quality assurance, and usually no manuals or user information. Productivity is high because the amount of work performed is low.

Zone of Management Information Systems (MIS) Projects

The MIS domain produces software that ranges from less than 1 function point in size up to applications in excess of 100,000 function points in size. Because the MIS domain is sharply segmented into the subdomains of "new" projects and "enhancement" projects it is not as easy to present overall result.

The "average" size (mode) of new MIS projects that are started from scratch is about 1,500 function points. However, the average size of enhancement projects that add new features to existing software applications is only about 150 function points in size.

The MIS domain is undergoing rapid transformation due to the pervasive influence of the new Agile methods. Another new change in the MIS domain is the increase in offshore outsourcing. Agile projects are often more productive than traditional "waterfall" projects. Offshore projects are usually less expensive in terms of dollars due to labor cost differentials. However, when measured using standard metrics such as work hours per function point, the offshore projects tend to be somewhat less productive than in-house U.S. projects due to the complexity of transferring requirements to other countries.

Average productivity rates for new MIS applications in standard COBOL are in the range of 8–10 function points per staff month. Newer Agile projects run from about 12 to 26 function points per staff month.

For client-server applications using generators or object-oriented projects, productivity rates are about 16 function points per staff month. Other MIS approaches include information-engineering, which average about 14 function points per staff month, and rapid application development (RAD), which averages about 20 function points per staff month.

For enhancements to aging legacy systems, productivity varies sharply based on the size, age, structure, and decay of the application being updated. Assuming that 50 function points of new and changed material are being added to a 1,500 function point application that is moderately high in complexity, the productivity rate would average about 6–9 function points per staff month.

The MIS domain is generally competent and effective for applications that run up to about 5,000 function points in size. For very large systems and especially for applications that approach or exceed 50,000 function points in size, the MIS domain has many more failures and disasters than successes.

The MIS domain has been experimenting with a host of interesting technologies, including but not limited to, Agile methods, Extreme programming (XP), joint application design (JAD), rapid application development (RAD), computer-aided software engineering (CASE), information engineering (IE), reverse engineering, reengineering, generators, reusability, and many more. The overall impact of these technologies runs from slightly negative to very positive. However, all of the technology factors that affect MIS projects will be discussed after the basic quantitative averages are given.

Zone of Web Applications

Software written for the Internet and World Wide Web did not even exist when the first edition of this book was published in 1991. Web software was not studied in detail when the second edition was published in 1996, because the author had not yet been commissioned by clients to include web applications in assessments and benchmark studies.

However, as this third edition is being written in 2008, web applications are now the most numerous of any kind of new development project. There are literally millions of web applications circa 2008. Some of these web applications are the main technologies of major corporations such as Amazon, American On Line (AOL), EBay, Google, and Yahoo.

Because web applications are highly interactive and deal with "content" that runs the gamut from sophisticated 3D animation to music, images, and text they have evolved some unique development practices. Web applications are also supported by many new tools and programming languages.

The development of web software itself tends to feature short schedules (less than 12 calendar months) and high productivity. Quality levels are usually acceptable, even if not outstanding. However, the main business value of web applications is not the software per se, but the content that the software presents and controls. Web applications range from less than 1,000 function points in size up almost 20,000 function points for applications that also handle order entry, inventory management, and other business topics.

As of 2008, there are no effective metrics for quantifying the size of web content such as images, music, photos, or animation. As a result, there are no solid economic studies on the cost of ownership of web applications. The development and maintenance costs of web software can be studied, but the higher value and costs associated with web content are outside the scope of this book and indeed of most economic studies to date.

Zone of Contract and Outsource Software Projects

The contract and outsource community first came to the author's attention about 20 years ago when doing a benchmark study within the New England banking community. Several projects stood out from the rest because they were about twice as productive as others of the same size and complexity levels. These projects were done by a well-known software contract house that specialized in the financial services industry. Productivity rates for the contract work were about 16 function points per staff month, whereas similar projects produced by the employees of the various banks averaged about 8 function points per staff month.

There were four contributing factors that explained the doubled productivity:

- Clients were less likely to change requirements with contractors than with their own staff.
- The contractors had done similar jobs for other banks and new the application areas very well.
- The contractors had quite a lot of reusable material available for banking applications.
- The contractors worked quite a few extra hours that did not get charged back to the client.

As a general rule, the performance of outsource and contract houses is better than the typical results of their clients. If this were not the case, the contract and outsource business would eventually wither away.

As the situation now stands, the contract and outsource business is growing rapidly in several dimensions.

In the United States, specialized vendors such as Computer Aid Inc. that deal with maintenance and enhancement of legacy applications have been growing rapidly. For new development projects, offshore outsourcers in India, China, Russia, and other countries with labor costs lower than U.S. norms have been growing rapidly. However, the inflation rates of the successful outsource countries has been growing much faster than U.S. inflation rates. As a result, circa 2008, the cost of living in Moscow is higher than in New York, and Shanghai is in the same range. Inflation rates in India are increasing too.

At the inflation rate rises, today's cost differentials between the United States, China, India, and Russia will have disappeared by about 2015. Of course, there will always be some countries with lower costs than the United States but whenever these countries become successful in outsourcing, labor costs and inflation rates tend to rise rapidly.

Twenty-five years ago Japan and Taiwan were the low-cost technology leaders, but now their costs are comparable to the United States and Europe. Looking for bargains in faraway places is useful in the short term, but not necessarily in the long term.

Contract and outsource vendors can work on any size application, but their main strength seems to be applications that are large enough and complex enough to cause severe troubles if their clients attempted them; i.e., applications from 5,000 to more than 100,000 function points in size.

However, not every outsource agreement has a successful outcome. The author has been an expert witness in more than a dozen lawsuits involving outsource software in the 10,000 function point range. To optimize the odds of a successful outsource engagement, the following topics should be included in the contract:

- The size of the project should be estimated and included, preferably with function points.
- Changing requirements, which average more than 1 percent per month, should be included.
- The quality of the delivered application should be included.

An "average" software contract will be for an application of about 1,000 function points in size. Productivity rates by contractors and outsourcers are typically 25 percent or so higher than their client's nominal rates; i.e., an average of about 5 to 8 function points per staff month. In specific industries where outsourcers and contractors serve many clients and hence have substantial volumes of reusable material available, outsource productivity rates can average close to 20 function

points per staff month. (Note that there are exceptions to the rule that outsourcers do better than their clients. Outsourcers and contractors can fail as embarrassingly as anyone else, especially for projects in the 10,000 function point range where failures outnumber successes.)

Zone of Commercial Software Projects

The commercial software world is being transformed by hardware and platform changes. As recently as 20 years ago, all large commercial software projects were on mainframes or at least mini computers. Personal computers were limited to 64 K bytes of memory, puny hard disks (if any), and were not suitable for applications larger than around 500 function points in size.

That was true in 1985. Today's personal computers are increasing in capacity and speed so fast that it is hard to keep up. Essentially today's personal computers are as powerful as the mainframes of a generation ago.

Many Windows and UNIX applications are now well over 5,000 function points in size. Both Windows NT and Windows 95 themselves topped 50,000 function points in size. (Based on Microsoft's ads stating that Windows 95 contained more than 11,000,000 lines of source code, Windows 95 might have been in the range of 85,000 function points or roughly the same as IBM's mainframe MVS operating system.) Windows XP topped 100,000 function points in size and Windows Vista is around 160,000 function points in size.

As personal computer software balloons in size, the software development methods of companies such as Microsoft are edging closer to those of the traditional mainframe software producers such as AT&T and IBM in both methods and results. Even Microsoft had trouble with their older Windows projects and hence began to use mainframe approaches such as inspections and formal defect-tracking systems. The numerous reported bugs in Vista and Office 2007 indicate that even Microsoft is not quite able to tackle really effective levels of quality control for massive applications.

The commercial software world is now dominated by Microsoft, with other major players being Computer Associates, IBM, and a host of other companies such as the SAS Institute, Oracle, KnowledgeWare, and many others.

When the first edition of this book came out in 1991, an "average" commercial software package on a personal computer was about 500 function points in size. The average size for mainframe packages was about 5,000 function points.

When the second edition of this book came out, the average size of Windows-based PC applications approached 5,000 function points in size or very nearly the same as the mainframe domain. At the time of this

third edition, some applications such as Office 2007 top 100,000 function points and most are larger than 10,000 function points in size.

Typical commercial software productivity rates were in the range of about 4 to 10 function points per staff month. This comparatively low rate was because the commercial software domain required significant volumes of user manuals, HELP screens, and other amenities that were not always part of in-house software projects.

Quality control for commercial software is a weak link. Major vendors such as Microsoft, Symantec, Oracle, SAP, and so on, deliver their applications with far too many defects.

Zone of System Software Projects

Systems software controls physical devices. This definition is broad enough to encompass operating systems, switching systems, fuel-injection systems, aircraft flight control systems, medical instrument controls, and robotic manufacturing control systems.

The term "systems software" also encompasses "embedded software." Embedded applications are physically located inside the devices they control and are usually inaccessible to users or device owners. Examples of embedded software includes software inside automobile fuel injection systems and software that controls GPS devices, cameras, and even digital wristwatches.

Systems software applications are often large and complex. An "average" systems software package will be about 15,000 function points in size and will have development productivity rates of somewhere between 3 and 6 function points per staff month. Really major systems software projects, such as IBM's MVS operating system, can easily top 100,000 function points, whereas Vista is even larger, in the range of 160,000 to possibly 200,000 function points. Vista's very large size is not because it is an operating system per se, but because Microsoft usually stuffs many extraneous features into all of their applications for marketing purposes.

The object-oriented paradigm is moving rapidly through the systems software world. Some OO projects within this domain are achieving productivity rates in excess of 20 function points per staff month.

Because systems software controls physical devices that are usually expensive and may be dangerous to human life and safety if the software fails, the systems software domain has long been the top-ranked industry segment in terms of quality control. The systems software world builds some very complex applications, but usually manages to remove in excess of 95 percent of the bugs prior to delivery. The systems software quality emphasis explains why more systems houses use formal inspections that any other sub-industry.

The systems software world ranks second to the military world in the production of *very* large sets of specifications and planning documents. Paperwork is, therefore, a major component of systems software production.

Zone of Military Software Projects

Military software has long been dominated by Department of Defense (DoD) standards. A secondary effect of DoD standards has been the production of amazingly large volumes of software specifications, planning documents, change requests, user manuals, and other paper deliverables.

Much of the military paperwork is due to oversight and control requirements. Analysis of sample documents for military projects reveals that military software projects create about three times the number of paper documents as civilian systems software projects of the same size. There are no apparent benefits for the large military specification sets, but there are rather large costs associated with these documents.

Military software costs are driven more by paperwork than by any other factor. This phenomenon was almost invisible when "lines of code" or LOC metrics were used for productivity analysis. However, the function point metric clearly points out the magnitude of software paperwork costs for military projects.

Military software is the overall laggard in terms of productivity rates. This is not due to poor coding productivity, since historical coding results using both Ada83 and Ada95 compare very favorably to civilian norms. More recent military projects indicate similar results with Java, C++, C#, etc. Programming per se is not a military software weakness. The poor productivity of the defense sector can be attributed to massive volumes of paperwork, and the expensive oversight requirements associated with military contracts.

An "average" military contract project will be about 5,000 function points in size and will run from 1 to about 5 function points per staff month in overall productivity levels.

The world of military software has many subdivisions, including but not limited to, embedded real-time software; command, control, and communication (C3) software, logistics and supply software, personnel software, strategic and tactical simulation software, ordinary management information systems, and many kinds of tools and support software.

U.S. Software Staffing Levels

The first topic of significance is the staffing required to build software applications. This issue has considerable flexibility, since adding or subtracting staff members is one of the few changes that can be

accomplished while a project is actually underway, even if sometimes the changes are difficult or intrusive.

Table 3-22 shows the approximate overall staffing levels for U.S. software projects ranging from 100 function points to 100,000 function points in size. Below 100 function points staffing consists almost always of one person, who may even work part time on the application. Staffing ratios for projects above 100,000 function points tend to be the same as for 100,000 function point projects. Once applications reach that size, hundreds of specialists are deployed.

The staffing levels shown in Table 3-22 include software engineers for all size levels. For the larger applications, specialists are included too: quality assurance, technical writers, testing, database administration, web design, etc. Note that the third edition of this book uses data collected during the years of 2000 to 2007, so the results are somewhat different from the previous edition, where cumulative data between 1990 and 1997 was used.

Another change in this edition is the elimination of data for very small software projects in the 1 and 10 function point size ranges. Such small projects are always performed by a single individual and vary widely due to unforeseen circumstances. For example, interruption of a project to take a class or being absent due to sick days can throw off the results of projects performed by single individuals.

Table 3-22 provides average values for both columns and rows. Readers are cautioned that although the column averages are fairly accurate, the row averages are merely the arithmetic mean of the data, rather than weighted averages. This means that the weight of the large 100,000 function point projects will distort the averages. The earlier editions did not provide row averages for that reason.

TABLE 3-22 Application Staff Size for Software Projects (Application Size Expressed in Function Points)

	Function points				
Form of Software	100	1,000	10,000	100,000	Average
End-user	0.76	–	–	–	0.76
Web	0.70	3.91	20.83	–	8.48
MIS	0.79	3.39	44.80	544.66	148.41
U.S. outsource	0.79	3.63	44.37	488.28	134.27
Offshore outsource	0.75	3.33	45.27	529.10	144.61
Commercial	1.14	4.89	47.26	390.63	110.98
Systems	1.23	6.04	43.57	382.48	108.33
Military	1.79	5.79	43.86	661.38	178.20
Average	**0.99**	**4.42**	**41.42**	**499.42**	**136.57**

It may be of interest to readers that the rather large staff shown for MIS applications in the 100,000 function point size range is because most MIS producers are not qualified to build such large applications, so they add emergency personnel to the team when the projects begin to run late.

When a technology evolves and matures, a point is reached where one practitioner can no longer encompass all of the information needed to work effectively on every kind of problem that occurs.

When this point occurs, the most effective response is specialization. Instead of generalists who can deal with any topic within the scope of the technology, there will be those who are particularly experienced and knowledgeable in selected areas and who may know other areas only slightly. This phenomenon has been true for medical practice, legal practice, chemistry, physics, biology, mathematics, and most forms of engineering.

The software industry reached the point of needing specialists as long as 30 years ago. As the 21st century unfolds, there are many kinds of software specialists working in large software-intensive enterprises. For some large enterprises, there may be more than 100 different occupation groups within a general software organization and various kinds of specialists can exceed one-third of total employment. As of 2008 there are more questions than answers in the domain of specialization, but this is an important topic that will become more important as in the future.

From the consulting engagements the author's company performs with various clients, some preliminary observations are accumulating on the numbers and kinds of specialists associated within enterprises of specific sizes and types. Following are short discussions of the kinds of specialists encountered.

Very Small Enterprises with Less Than 10 Software Professionals Enterprises whose entire software staff is less than 10 normally have no formal specialties at all in terms of job titles. However, there are de facto specialties based on the needs of the organization. For example, a major role in quality assurance and testing may devolve to one or more of the staff. If Agile methods are used, one of the team may become the de facto "scrum master." If design or code inspections are used, one of the team may become the moderator.

Another key role is maintenance of prior versions, and here too, the work may be concentrated in a few individuals. Good technical writing is not a very common skill, so if anyone in a department has such a skill they tend to accumulate much of the writing tasks that may occur. Because small enterprises can only build small to medium applications (although they can maintain large systems), the lack of formal specialists is not a major problem.

Small Enterprises with up to 100 Software Professionals By the time an enterprise approaches 100 software personnel, they usually have several kinds of specialists on board. Some of the specialists observed within this size range include (in alphabetic order):

- Customer support specialists
- Database administration specialists
- Function point counting specialists
- Maintenance and enhancement specialists
- Network specialists (local, wide area)
- Quality assurance specialists
- Scrum masters
- Systems software support specialists
- Technical writing specialists
- Testing specialists
- Webmasters

The percentage of the overall software population that these specialists occupy would average about 10 percent to 25 percent with perhaps 15 percent being the median value.

Medium Enterprises with up to 1,000 Software Personnel The domain of medium software organizations with up to about 1,000 total software personnel is one of the most troublesome in the United States in terms of cost overruns, missed schedules, and other kinds of software problems. Organizations within this size range need specialization fairly urgently in order to perform the work at hand, but quite a few still maintain a preference for a generalist slant. This is one of the reasons why midsized companies often have the lowest productivity rates and the worst quality levels of any size range. The more sophisticated enterprises within this size range have quite a range of specialties:

- Configuration control specialists
- Cost estimating specialists
- Customer support specialists
- Database administration specialists
- Function point counting specialists
- Maintenance and enhancement specialists
- Measurement specialists

- Network specialists (local, wide area)
- Quality assurance specialists
- Scrum masters
- Systems software support specialists
- Technical writing specialists
- Testing specialists
- Web design specialists
- Webmasters

The range of specialization within this size group runs from perhaps 15 percent to 35 percent, with about 25 percent being a median value.

Large Enterprises with up to 10,000 Software Personnel By the time an enterprise is large enough to employ in the range of 10,000 software personnel they need a host different kinds of specialists. In fact, one of the reasons why quality levels and productivity levels in very large companies can exceed those of midsized companies can be attributed to the specialists who are available in large corporations. Following are some of the kinds of specialists observed in major corporations and large government groups:

- Architecture specialists
- CMM and CMMI specialists
- Configuration control specialists
- Cost estimating specialists
- Customer support specialists
- Database administration specialists
- Education and training specialists
- Function point counting specialists
- Human factors specialists
- Information systems specialists
- Integration specialists
- Maintenance and enhancement specialists
- Measurement specialists (customer satisfaction)
- Measurement specialists (productivity)
- Measurement specialists (quality)
- Network specialists (local, wide area)

- Package acquisition specialists
- Performance specialists
- Planning specialists
- Process improvement specialists
- Quality assurance specialists
- Reusability specialists
- Scrum masters
- Standards specialists
- Systems software support specialists
- Technical writing specialists
- Technology specialists (object-oriented methods, GUI, client-server, etc.)
- Testing specialists
- Tool development specialists
- Web design specialists
- Webmasters

The range of specialists within large enterprises runs from a low of perhaps 25 percent to a high of more than 50 percent, with about 35 percent being a median value. In large enterprises within this size range, specialization is definitely the norm. Note that the same phenomenon stays true for even larger enterprises in the 20,000 to 40,000 software personnel range.

Ratios of Specialists to General Software Personnel One of the major topics in the domain of specialization is deciding how many specialists of various kinds are needed to support the overall work of the software community. This topic was only just starting to be explored in depth in 1995, so the following ratios have a high margin of error. Indeed, for some kinds of specialization there are not yet any normative ratios available. Table 3-23 lists specialists in descending order of the numbers encountered. As of 2008 several new specialties have been added, but the ratios of the older specialties are still close to 1995 values.

Unfortunately, research into software specialization is such a new topic that ratios are not yet available for quite a few software specialists. Overall the software industry remains less than a true profession in 2008. Certification and formal specialization only exists for a few titles such as quality assurance, function point counting, and architecture.

TABLE 3-23 Ratios of Technical Specialists to General Software Populations

Specialist Occupation	Specialists to Generalists in Typical Populations
Maintenance and enhancement specialists	1 to 3
Scrum masters	1 to 5
Testing specialists	1 to 8
Technical writing specialists	1 to 15
Web design specialists	1 to 20
Database administration specialists	1 to 25
Quality assurance specialists	1 to 25
Configuration control specialists	1 to 30
Systems software support specialists	1 to 30
Integration specialists	1 to 50
Measurement specialists	1 to 50
Network specialists (local, wide area)	1 to 50
Function point counting specialists	1 to 75
Architecture specialists	1 to 75
Performance specialists	1 to 75
SEI CMM specialists	1 to 100
Reusability specialists	1 to 100
Cost estimating specialists	1 to 100
Human factors specialists	1 to 100
ISO 9000-9004 specialists	1 to 125
Package acquisition specialists	1 to 150
Process improvement specialists	1 to 200
Education and training specialists	1 to 250
Standards specialists	1 to 300
Webmasters	1 to 500

Because the numbers of technical staff members and the kinds of specialists can change independently, the overall ranges of software personnel are extremely broad. Table 3-24 illustrates the observed ranges in total software staffing levels.

TABLE 3-24 United States Ranges for Software Staffs (Technical + Management)

Function Points	Minimum	Median	Maximum
1 FP	0.14	0.52	1.03
10 FP	0.40	1.17	2.57
100 FP	0.72	1.69	3.38
1000 FP	5.30	10.60	20.14
10000 FP	43.35	85.00	153.00
100000 FP	486.75	885.00	1,548.75

Software staffing patterns are highly flexible. There is a tendency to staff to maximum levels for software projects under considerable schedule pressure. The fact that excessive staffing may slow things down rather than speed things up is understood in theory, but seldom considered in practice.

U.S. Software Schedule Ranges

Of all known factors associated with software, schedules stand out as being the most troublesome and the most frequent source of executive dissatisfaction with the software community.

The schedules shown here run from the nominal "start of requirements" up to the nominal "first customer ship" date of software projects. The starting point of software projects is usually ambiguous and difficult to determine. The starting points used in Table 3-25 were simply derived by querying the responsible project managers. Delivery dates are sometimes ambiguous too. The assumption in Table 3-25 is for delivery to the first paying customer, rather than delivery to beta test or field test customers.

Software schedules, like staffing patterns, are highly variable. In part, schedules are determined by the number of personnel assigned, the amount of overtime (both paid and unpaid), the volume of creeping user requirements, and the tools, technologies, and languages utilized.

The most troublesome aspect of software schedules is the major difference between the real schedules of software projects and the anticipated or desired schedules as determined by clients or senior executives.

Figure 3-3 illustrates the typical pattern of differences between the initial schedule established for the project during requirements and the final delivery of the software. Note how the gap between anticipation and reality grows steadily wider as the overall sizes of the applications increase in size and complexity.

TABLE 3-25 Application Schedules for Software Projects (Schedules Expressed in Terms of Calendar Months)

Form of Software	100	1,000	10,000	100,000	Average
End-user	2.50	–	–	–	2.50
Web	3.00	10.00	40.00	–	17.67
MIS	6.00	18.00	48.00	68.00	35.00
U.S. outsource	5.50	16.00	46.00	64.00	32.88
Offshore outsource	7.00	19.00	47.00	63.00	34.00
Commercial	8.00	22.00	46.00	64.00	35.00
Systems	9.00	24.00	45.00	63.00	35.25
Military	10.00	36.00	60.00	72.00	44.50
Average	**6.38**	**20.71**	**47.43**	**65.67**	**35.05**

Figure 3-3 represents what is probably the most severe and chronic complaint about software from corporate executives and high government and military officials: Large systems are often later than requested. They are often later than when the software managers promised, too, which is one of the reasons why the software management community is not highly regarded by senior executives as a rule.

The large gap between the actual delivery date and the anticipated delivery date is the cause of more friction between the software world, corporate executives, and clients than any other known phenomenon. This gap is also the cause of a great deal of expensive litigation.

Some of the factors that explain this gap are due to creeping user requirements. However, a problem of equal severity is malpractice in the way the original schedule estimates were developed. Often schedules are set by decree or client demand, rather than being carefully planned based on team capabilities.

Arbitrary schedules that are preset by clients or executives and forced on the software team are called "backward loading to infinite capacity" in project management parlance. The essence of the phrase is that irrational schedules trigger more disasters than any other known phenomenon.

The presence of this gap between anticipation and reality is one of the reasons why a sub-industry of commercial software estimating tools has emerged. As this book is written, at least 50 specialized commercial software cost estimation tools are on the market in the United States. There are more than 100 general-purpose project planning or "project management" tools on the market too.

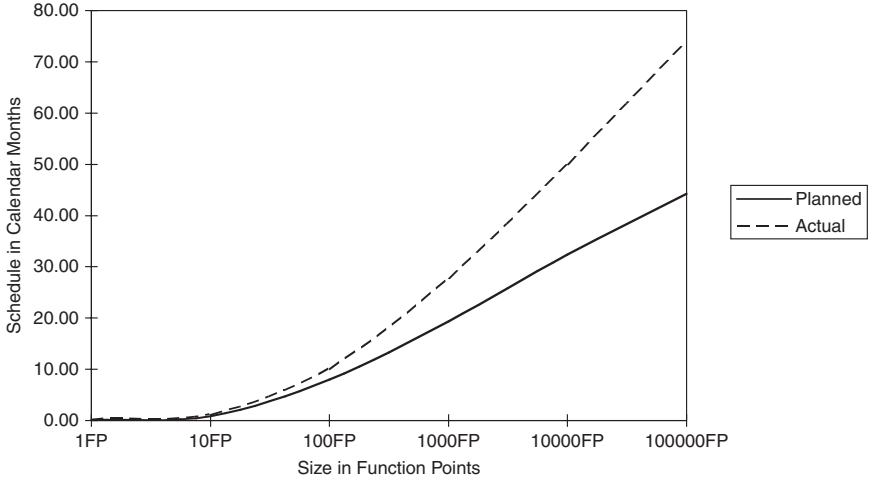

Figure 3-3 Differences between planned and actual software schedules

Table 3-26 illustrates the overall ranges of software schedules. Note how wide a range can exist between "best in class" and "worst in class" results.

The minimum column can only occur for software projects that are at state-of-the-art levels in terms of reusability, management methods of planning and estimating, staff experience levels, tools, and development processes. In fact, excellence in all of these attributes is necessary to move toward minimum schedules with a good chance of success.

The maximum column can occur on projects that are careless with quality control, that fail to control creeping requirements, or that make any of a host of other common mistakes. There are thousands of ways to mess up or damage software projects and only a few ways to do them well.

U.S. Ranges in Creeping User Requirements Because creeping requirements contribute to the gap between planned and actual schedules, it is appropriate to consider that topic next.

Table 3-27 shows the approximate volumes of creeping requirements that occur after the completion of the nominal requirements phase. The contents of the original "official" requirements are assumed to constitute 100 percent. The growth rate in Table 3-27 is expressed as a percentage change to the original full set of requirements.

Requirements creep is largely a function of development schedules. Larger projects with longer schedules naturally have higher volumes of creeping requirements than do small projects with shorter schedules.

However, the type of software also exerts an influence. For example, U.S. outsource projects are likely to have a lower rate of requirements creep than in-house projects because the outsource vendors are usually more experienced in gathering requirements.

TABLE 3-26 U.S. Software Schedule Ranges in Calendar Months

Function Points	Minimum	Average	Maximum
1 FP	0.03	0.16	0.40
10 FP	0.32	1.07	2.35
100 FP	3.00	6.38	19.00
1000 FP	16.56	20.71	44.16
10000 FP	29.88	47.43	84.66
100000 FP	44.28	65.67	132.84

TABLE 3-27 Application Probable Requirements "Creep" (Data Expressed in Percentage of Original Requirements)

Form of Software	100	1,000	10,000	100,000
End-user	1.75%	0.00%	0.00%	0.00%
Web	3.00%	12.00%	22.00%	0.00%
MIS	6.30%	16.80%	33.60%	50.40%
U.S. outsource	5.70%	13.20%	28.40%	40.80%
Offshore outsource	6.80%	16.00%	35.00%	52.00%
Commercial	8.80%	19.20%	36.80%	52.80%
Systems	6.00%	14.00%	23.50%	39.00%
Military	11.25%	28.50%	48.00%	63.75%
Average	**6.20%**	**14.96%**	**28.41%**	**37.34%**

Another way of expressing requirement creep is the average monthly rate during design and coding activities, where creeping requirements are most likely to occur:

Domain	Average Monthly Rate of Creeping Requirements
End-user software	0.5%
Management information systems	1.0%
U.S. outsource software	1.0%
Commercial software	3.5%
Systems software	2.0%
Military software	2.0%
Web software	5.0%
Agile software	10.0%

The high rate of creeping requirements for Agile projects is actually a deliberate part of the Agile method. Requirements are gathered in only a general way when the projects first start, not in a full way. As each iteration or "sprint" occurs, the requirements for the next wave are derived from the user's reactions to the most recent set of features.

Creeping requirements were difficult to measure during the "lines of code" or LOC era. However, one of the useful features of the function point metric is the ability to perform direct and rather accurate measures of the rate at which requirement changes occur.

Assume that you reach an initial agreement with a client to build an application of 1,000 function points in size. Any change to the features of the application will almost certainly affect the function point total. By comparing the size in function points at the time of the initial

agreement with the size at delivery, the rate of change can be measured directly and accurately.

U.S. Software Effort Ranges

Table 3-28 shows approximate averages expressed in terms of staff months of effort. The table assumes software engineers or programmers and also the many ancillary occupations such as quality assurance, database administration, testing, and the like. Project management is assumed to be present also.

The assumption in the table is that an average work month contains approximately 132 working hours. Note that Table 3-28 shows averages for the rows with reasonable accuracy, but not for the columns. It is easy to average columns of numbers, but the arithmetic average of projects that span a range from 1 function point to 100,000 function points are so misleading since the average is skewed by the 100,000 function point results.

Table 3-29 shows the overall ranges that occur in this important topic. There are a host of factors associated with the extreme conditions, such as reusability, experience, programming languages, tools, etc. Only an expert human estimator or one of the better expert-system software cost estimating tools can sort through all of the contributing factors and develop an accurate estimate for where any particular project is likely to fall.

The minimum side of the table can occur only with state-of-the-art technologies and expert technical and management personnel. The maximum side of the table can occur whenever too many mistakes are made. As always, there are many ways to fail and only a few ways to succeed.

TABLE 3-28 Average Software Effort in Staff Months (Assumes 132 Work Hours per Month)

Form of Software	100	1,000	10,000	100,000	Average
End-user	1.90	–	–	–	1.90
Web	2.11	39.06	833.33	–	291.50
MIS	4.76	60.98	2,150.54	37,037.04	9,813.33
U.S. outsource	4.35	58.14	2,040.82	31,250.00	8,338.33
Offshore outsource	5.26	63.29	2,127.66	33,333.33	8,882.39
Commercial	9.09	107.53	2,173.91	25,000.00	6,822.63
Systems	11.11	144.93	1,960.78	24,096.39	6,553.30
Military	17.86	208.33	2,631.58	47,619.05	12,619.20
Average	**7.06**	**97.47**	**1,988.37**	**33,055.97**	**8,787.22**

TABLE 3-29 Ranges of Software Effort in Person Months

	Minimum	Average	Maximum
1 FP	0.01	0.09	0.41
10 FP	0.14	1.42	6.02
100 FP	2.60	7.06	64.28
1000 FP	44.87	97.47	876.49
10000 FP	889.35	1998.37	12,952.98
100000 FP	21,553.29	33,055.97	205,735.95

U.S. Software Productivity Averages and Ranges for New Projects

The first two editions of this book utilized "function points per staff month" as the primary normalizing metric for expressing productivity. The second edition continued to use that metric, but also added the complementary metric "work hours per function point." In this book, the two are roughly equivalent if you assume that a staff month contains 132 work hours.

Table 3-30 assumes all activities from the start of requirements through delivery to the first external customer. As many as 25 activities can be performed, including many that have nothing at all to do with coding, such as creation of paper documents (i.e., specifications, user manuals, plans, etc.). Also assumed in Table 3-30 are activities such as quality assurance, testing, and project management.

This data here is somewhat lower than several other national databases of software productivity data. So far as can be determined, the primary reason for the differences is because the SPR knowledge base attempts to include *all* activities. Many other productivity data collections center on "design, code, and unit testing," but these activities

TABLE 3-30 U.S. Average Productivity in Function Points per Staff Month

Form of Software	100	1,000	10,000	100,000	Average
End-user	52.60	–	–	–	52.60
Web	47.30	25.60	12.00	–	28.30
MIS	21.00	16.40	4.65	2.70	11.19
U.S. outsource	23.00	17.20	4.90	3.20	12.08
Offshore outsource	19.00	15.80	4.70	3.00	10.63
Commercial	11.00	9.30	4.60	4.00	7.23
Systems	9.00	6.90	5.10	4.15	6.29
Military	5.60	4.80	3.80	2.10	4.08
Average	**23.56**	**13.71**	**5.68**	**3.19**	**11.54**

TABLE 3-31 U.S. Average Productivity in Work Hours per Function Point (Assumes 132 Hours per Month)

Form of Software	100	1,000	10,000	100,000	Average
End-user	2.51	–	–	–	2.51
Web	2.79	5.16	11.00	–	6.32
MIS	6.29	8.05	28.39	48.89	22.90
U.S. outsource	5.74	7.67	26.94	41.25	20.40
Offshore outsource	6.95	8.35	28.09	44.00	21.85
Commercial	12.00	14.19	28.70	33.00	21.97
Systems	14.67	19.13	25.88	31.81	22.87
Military	23.57	27.50	34.74	62.86	37.17
Average	**9.31**	**12.87**	**26.25**	**43.63**	**23.01**

only amount to about 25 percent of total effort expended on software projects.

Note that the averages displayed with Table 3-30 are merely the arithmetic means of the columns and rows. The data in Tables 3-31 and 3-32 come from projects noted during 2000 to 2007. The earlier editions of this book used data gathered between about 1985 and 1996.

Table 3-31 presents the same information, only expressed in terms of work hours per function point rather than function points per staff month.

The two forms of data in Tables 3-30 and 3-31 are mathematically equivalent under the assumption that there are 132 work hours, on average, in a calendar month.

The ranges of software productivity in the United States are very broad. Table 3-32 shows the range in function points per staff month.

Table 3-33 displays the equivalent information using the metric "work hours per function point."

It is obvious from the ranges in Tables 3-30 through 3-33 that many factors are simultaneously affecting the results: size, domain, tools, methods, experience, and so forth.

TABLE 3-32 U.S. Productivity Ranges in Function Points per Staff Month

	Maximum	Average	Minimum
1 FP	129.36	45.65	2.44
10 FP	71.62	34.70	1.66
100 FP	38.46	23.56	1.56
1000 FP	15.41	13.71	1.14
10000 FP	9.26	5.68	0.77
100000 FP	4.64	3.19	0.49

TABLE 3-33 U.S. Productivity Ranges in Work Hours per Function Point

	Minimum	Average	Maximum
1 FP	1.02	2.89	53.99
10 FP	1.84	3.80	79.51
100 FP	3.43	9.33	84.85
1000 FP	8.56	12.87	115.70
10000 FP	14.25	26.25	170.98
100000 FP	22.33	43.63	271.57

Since development methods are obviously one of the factors that spread the ranges of productivity levels, Table 3-34 illustrates the results of selected development methods: Agile, the CMM, Extreme programming, and some others. The traditional "waterfall" approach is also shown to provide a context for the other methods. See also Table 3-11 earlier in the chapter for the results of various hybrid methods.

Unfortunately, there are some gaps and omissions in Table 3-34. Agile projects in the 100,000 function point range have not yet been produced or at least not by the author's clients.

For the largest applications in the 10,000 to 100,000 function point range, CMM level 5 and Watts Humphrey's team software process (TSP) and personal software process (PSP) seem to provide the best overall results.

For small projects in the 1,000 function point range, the Agile and Extreme methods seem to produce the best results.

Since the results of Table 3-34 are significantly higher than the average results in the previous tables, readers may well wonder why this

TABLE 3-34 U.S. Average Productivity for Selected Methodologies (Data Expressed in Function Points per Staff Month)

Methodology	100	1,000	10,000	100,000	Average
Agile	37.00	25.00	7.50	–	23.17
CMM 1	12.00	7.20	4.50	1.70	6.35
CMM 3	21.00	10.20	6.30	4.80	10.58
CMM 5	23.00	12.60	8.90	6.50	12.75
Extreme (XP)	35.00	23.60	8.60	–	22.40
Iterative	19.00	9.90	6.70	4.20	9.95
Object-oriented	20.00	16.70	7.70	5.30	12.43
Six-Sigma	17.50	11.00	8.20	6.30	10.75
TSP/PSP	16.00	18.00	9.20	6.85	12.51
Waterfall	8.80	6.50	4.2	1.80	5.33
Average	**20.93**	**14.07**	**7.14**	**4.16**	**11.58**

situation occurs. The reason is because of the fairly low penetration of the various technologies illustrated. For example, the CMM and CMMI methods are used by less than 10 percent of U.S. projects and companies circa 2008. Even the Agile methods, which are growing rapidly in usage, are used by less than 80 percent of U.S. companies.

As a general rule, poor quality and requirements creep are the stumbling blocks for large software applications. Therefore methods that concentrate on quality control and that use formal change control techniques have the greatest success for large software projects.

U.S. Software Productivity Averages and Ranges for Enhancement Projects

New software projects that are started with requirements and entail complete development are fairly easy to characterize and measure. Enhancements, on the other hand, are much more complicated and much harder to deal with.

In addition to the other factors that influence results, such as experience, tools, languages, and so on, enhancement projects are influenced to a very large degree by these factors:

- The structure or complexity of the base legacy system being updated
- The number of latent defects still residing in the base or legacy system
- The structure or complexity of the new code being added
- The structure or complexity of any changed code
- The structure or complexity of any deleted code

The best case is the simple addition of new features to a well-structured existing application, with few latent bugs and no inner structural changes of any kind. The worst case is scattered updates and internal changes to a poorly structured existing application with many latent bugs or errors still remaining.

Another aspect of enhancement productivity measurement is difficult to explain, but is based on empirical evidence. Unlike new software projects where smaller projects are more productive than larger ones, small enhancements are very unproductive. The reason for this is that the overhead of these activities drags down productivity at the low end:

- Regression testing the base or legacy application
- Recompiling the base or legacy application
- Rewriting the specifications and user manuals in part
- Reshipping the entire product

Enhancement productivity tends to peak in a fairly narrow band, where the size of the enhancement is roughly 3 to 5 percent of the size of the existing application. This phenomenon tends to lead to the concept of fixed release intervals, with the interval roughly coinciding with a volume of change that approximates 5 percent. For personal computer software less than 1,000 function points in size, the optimal release interval is about every 6 to 9 months. For large applications above 5,000 function points in size, the optimal release interval is roughly every 12 to 18 months.

The simple graph shown in Figure 3-4 can clarify the situation. Assume that an existing application of 1,000 function points in size is going to be modified. If the size of the enhancement is about 5 percent (50 function points) productivity will tend to peak.

Although it is not a law of nature and some exceptions occur, most enhancements are much smaller than the base legacy application being updated. This means that a table of productivity data will appear to have some null records for situations where the enhancement is greater than the base.

Figure 3-5 shows what happens when enhancements vary from 1 function point upward, and the base system size also increases. To read Figure 3-5, note that the upper-left corner represents adding 1 function point to a 1 function point application. The lower-right corner represents the productivity of enhancing 100,000 function point systems.

Figure 3-5 has some seeming anomalies in the rates, until we recall that productivity tends to peak for enhancements that are neither very

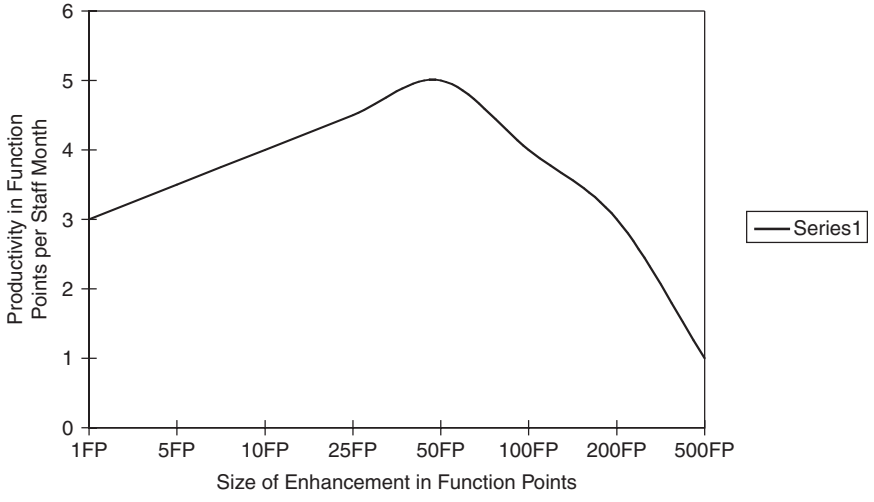

Figure 3-4 Productivity variations for enhancing a 1,000 function point application

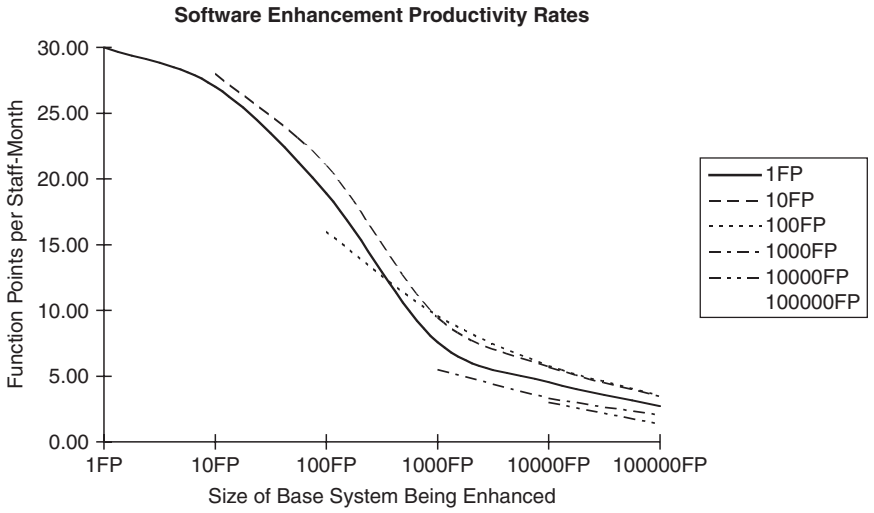

Figure 3-5 Relationship of enhancement size, base system size, and productivity

large nor very small, but about 5 percent of the size of the original application.

U.S. Averages for Software Maintenance (Defect Repair) Productivity

In ordinary usage, the word "maintenance" is highly ambiguous and can mean almost any change to a software application after it goes into production. In this book, the word "maintenance" is used in a narrower more restricted sense as meaning only defect repairs.

The reason for this is a practical one. The costs of defect repairs are usually absorbed by the developing organization, whereas the costs of adding new features or enhancements are often billed out to customers.

Some geriatric conditions, such as restructuring aging legacy systems, can be funded by either clients or the developing organization, depending upon various conditions.

In this book, the effort devoted to maintenance is primarily that of finding and fixing bugs in the source code. There are, of course, a host of other aspects of software maintenance that are expensive, including but not limited to

- Customer support organizations
- Field service organizations
- On-site maintenance teams

- Maintenance management
- Maintenance of user manuals
- Litigation associated with defects
- Enhancing legacy applications
- Renovating legacy applications
- Replacing legacy applications

Maintenance productivity is a complicated topic and also difficult to measure. One of the reasons for the difficulty is that the repairs for many software defects are very small: Sometimes only one or two lines of source code are in error. These small changes exceed the normal precision of function point metrics (one line of COBOL is roughly equivalent to only 0.01 function points).

The costs of maintenance are not really driven by the volume of code that is added or modified. The major maintenance cost driver is associated with the size, age, complexity, and latent defects of the application being repaired.

Complexity is a difficult topic in its own right. For day-to-day practical purposes, the most common measurement of software complexity is the McCabe "cyclomatic complexity" metric originally published by Tom McCabe in 1994. The cyclomatic complexity metric is a measure of the control flow of an application. It is derived from a graph of the application, and the general formula is that cyclomatic complexity equals the number of edges on the graph minus the number of nodes on the graph, plus two.

Perfectly structured source code with no branches of any kind has a cyclomatic complexity of 1. There will be a beginning and ending node for the graph, and one line that connects them (i.e., 1–2+2 = 1).

In practice, cyclomatic complexity levels of less than 10 are normally considered fairly well structured. Cyclomatic complexity levels from 10 to 20 are considered fair or average structure. Levels greater than 20 are considered hazardous or poorly structured.

Figure 3-6 shows the overall productivity for a typical defect repair scenario where the entire volume of code modified is equal or less than 1 function point in size. The base system being modified will vary from 1 function point (an unlikely occurrence) up to 100,000 function points. The complexity levels will vary from low through average up to high.

Note that in Figure 3-6 the unit of measure is "work hours per function point" rather than "function points per staff month." For activities such as finding and fixing bugs that are asynchronous and where each occurrence takes only a few hours, this form of data representation seems to be a better fit for the situation than "function points per staff month."

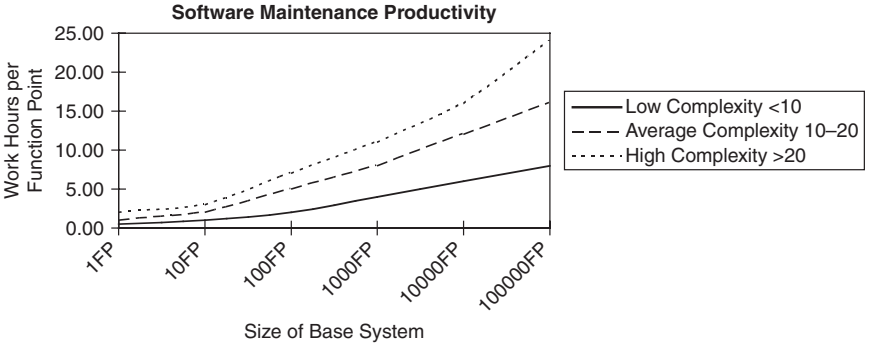

Figure 3-6 Maintenance productivity variances due to size of base system and cyclomatic complexity levels

The latter metric tends to imply some kind of continuous effort, and bug repairs are typically performed one by one using bursts of very intense effort.

There are no significant differences in the effort devoted to finding and fixing bugs among the seven domains discussed in this book. That is, end-user software, web application, information systems, systems software, commercial software, and military software are all subject more or less equally to the problems of size, age, latent defects, and complexity levels.

An important metric for maintenance and enhancement work is that of the "assignment scope" or the amount of legacy software one maintenance staff member can keep running in the course of a year. The maintenance assignment scope is affected by the structure of the legacy code, the tools available, the programming language(s), and the knowledge of the maintenance team. Table 3-35 shows the ranges in maintenance assignment scopes.

Table 3-35 can be used to predict the approximate maintenance team size for applications of any size, by selecting the most appropriate range for local conditions.

Since the second edition of this book was published, several kinds of companies have emerged that are improving maintenance activities. One of these companies is exemplified by Relativity Technologies, Inc. This company provides tools and methods for renovating legacy applications, thereby extending their useful lives before replacement. Renovations include simplifying code structures, identifying and eliminating error-prone modules, and improving the readability of existing code.

TABLE 3-35 Variations in Maintenance Assignment Scopes Based on Four Key Factors (Data Expressed in Terms of Function Points per Maintenance Team Member)

	Worst Case	Average Case	Best Case
1. Inexperienced staff Poor structure Low-level language No maintenance tools	100	200	350
2. Inexperienced staff Poor structure High-level language No maintenance tools	150	300	500
3. Inexperienced staff Poor structure Low-level language Full maintenance tools	225	400	600
4. Inexperienced staff Good structure Low-level language No maintenance tools	300	500	750
5. Experienced Staff Poor structure Low-level language No maintenance tools	350	575	900
6. Inexperienced staff Good structure High-level language No maintenance tools	450	650	1,100
7. Inexperienced staff Good structure Low-level language Full maintenance tools	575	800	1,400
8. Experienced staff Good structure Low-level language No maintenance tools	700	1,100	1,600
9. Inexperienced staff Poor structure High-level language Full maintenance tools	900	1,400	2,100
10. Experienced staff Poor structure Low-level language Full maintenance tools	1,050	1,700	2,400
11. Experienced staff Poor structure High-level language No maintenance tools	1,150	1,850	2,800

(*Continued*)

TABLE 3-35 Variations in Maintenance Assignment Scopes Based on Four Key Factors (Data Expressed in Terms of Function Points per Maintenance Team Member) (*Continued*)

	Worst Case	Average Case	Best Case
12. Experienced staff Good structure High-level language No maintenance tools	1,600	2,100	3,200
13. Inexperienced staff Good structure High-level language Full maintenance tools	1,800	2,400	3,750
14. Experienced staff Poor structure High-level language Full maintenance tools	2,100	2,800	4,500
15. Experienced staff Good structure Low-level language Full maintenance tools	2,300	3,000	5,000
16. Experienced staff Good structure High-level language Full maintenance tools	2,600	3,500	5,500
Average	1,022	1,455	2,278

A second kind of company is exemplified by Computer Aid, Inc. Computer Aid is a specialized maintenance outsource company that provides specialists rather than generalists for maintenance work. Specialization and full-time maintenance teams are much more efficient than having maintenance performed by part-time generalists who also do development. CAI also uses standard methods for all maintenance work, which allows team members to absorb new legacy applications with minimal learning curves.

The third kind of company is exemplified by Shoulders Corporation. This company uses sophisticated data-mining techniques to extract business rules and algorithms from legacy applications. Then these materials are used to develop new and improved versions of the legacy software, with additional features as well. The Agile methods are used for the redevelopment work. The most interesting aspect of this work is the data mining of the legacy code to extract business rules and algorithms. About 75 percent of the requirements for the new replacement

application can be derived from these operations, which ensure that the new application will offer identical features to the old, plus whatever new capabilities are needed.

Each of these capabilities are useful for improving maintenance efficiency. In fact aspects of all three could be combined as needed.

U.S. Averages for Volumes and Ranges of Software Paperwork

Software is an extraordinarily paper intensive industry. The overall volume of paper documents surrounding software projects is one of the largest of any manufactured product in human experience.

The author has been a direct participant in projects where the volume of specifications alone exceeded 60,000 pages. For very large military software projects, the total volume of paper documents can exceed 1,000,000 pages.

What is astonishing about paperwork volumes for very large systems is that the specifications and technical documents can be large enough to go beyond the normal lifetime reading speed of a single analyst!

It is hardly surprising that the Agile methods have hit upon software paperwork as being slow, cumbersome, and not especially conducive to achieving satisfactory results. The Agile approach prefers face-to-face contacts with users and the building of working prototypes or code segments as quickly as possible.

Assuming a technical reading speed of about 200 words per minute, there are some systems where an employee could spend an entire 40-year career doing nothing but reading plans and specifications without ever finishing.

The overall sets of paper documents for software projects include but are not limited to

- Requirements
- Initial functional specifications
- Design change requests
- Final functional specifications
- Program logic or internal design specifications
- Database design specifications
- Project development plans
- Project cost estimates

- Project milestone reports
- Project cost variance reports
- Review and inspection plans
- Test plans
- User manuals
- Maintenance manuals
- Internal defect reports
- Customer defect reports

Table 3-36 gives the overall volume of pages per function point for the aggregated sum of major document types produced. The table assumes normal U.S. paper stock (8.5 inches by 11 inches) with about 400 words per page.

The volume of paperwork varies with both size and domain. The smallest volumes of software paper documents are obviously associated with small end-user software applications, where there may be none at all. The largest volumes of paper documents are associated with large military applications, where the amount of effort devoted to producing paper can exceed 50 percent of total software budgets.

Although the phrase "paperwork" is used in this book, it is actually a misnomer. Quite a lot of textual and graphical material is available online. For example, this manuscript was prepared using Microsoft Word for Windows. The online HELP screens for this application total more than 100 pages of text. As an example of modern problems, both Word 2007 and Word 2003 were used because bugs and problems with 2007 slowed down the writing of the material.

The author and his company have not yet been commissioned to explore the impact of CD-ROM or hypertext on paperwork volumes. The effort to produce documents for these technologies is expected to

TABLE 3-36 Software Paperwork in Requirements, Specifications, and Documents (Data in Terms of Pages per Function Point)

Form of Software	100	1,000	10,000	100,000	Average
End-user	0.75	–	–	–	0.75
Web	1.20	3.40	3.75	–	2.78
MIS	2.50	4.00	5.00	6.00	4.38
U.S. outsource	2.60	4.25	5.25	6.50	4.65
Offshore outsource	2.80	4.75	5.60	6.80	4.99
Commercial	3.00	5.50	6.50	7.50	5.63
Systems	3.00	6.00	7.00	8.00	6.00
Military	6.00	9.00	12.00	15.00	10.50
Average	**2.73**	**5.27**	**6.44**	**8.30**	**4.96**

be somewhat greater than the effort needed to produce simple text and graphics materials on paper, however.

We have been commissioned to explore the costs of translating information from one national language to another. Indeed, both the screens, HELP text, and user manuals for the CHECKPOINT product, which the author's company markets, has been translated into French. A rule of thumb is that translation into a second national language takes about 35 percent of the effort of creating the materials in the original language.

Another rule of thumb is that Agile projects produce only about one third the volume of paper as traditional waterfall projects. The user guides and tutorial information are about the same size, but Agile requirements and specifications are largely supplanted by face-to-face meetings and prototyping.

U.S. Average Risk of Project Failure and Cancellation

The author and his company have been studying opposite ends of the software spectrum for many years. One end of the spectrum consists of projects that set new records for productivity, quality, or other tangible accomplishments. The opposite end of the spectrum consists of projects that were terminated due to cost and schedule overruns, failure to meet requirements, or other causes associated with poor planning, estimating, quality control, or excessive requirements creep.

Table 3-37 shows the overall probability that software projects will be terminated prior to completion. A larger research study on this project is described in another of the author's books, *Patterns of Software System Failure and Success* (Thomson International, 1995).

None of the seven domains are fully successful for large systems above 10,000 function points in size. Applications approaching or exceeding

TABLE 3-37 U.S. Average Probabilities of Software Project Cancellation (Data Expressed in Percentage of Projects Canceled)

Form of Software	100	1,000	10,000	100,000	Average
End-user	15.00%	95.00%	0.00%	0.00%	55.00%
Web	3.00%	11.00%	40.00%	0.00%	18.00%
MIS	5.00%	17.00%	45.00%	70.00%	34.25%
U.S. outsource	4.00%	14.00%	40.00%	45.00%	25.75%
Offshore outsource	6.00%	16.00%	47.00%	60.00%	32.25%
Commercial	5.00%	9.00%	35.00%	55.00%	26.00%
Systems	4.00%	8.00%	20.00%	30.00%	15.50%
Military	5.00%	12.00%	28.00%	48.00%	23.25%
Average	**5.88%**	**12.43%**	**36.43%**	**51.33%**	**28.75%**

100,000 function points are one of the most risky business ventures of the modern world. However, the systems software domain and the outsource domain have a somewhat better probability of finishing really large systems than do the other domains. The military software domain is in third place.

Within the past 10 years, the systems and military world have improved their ability to create really large applications, and as a result, failures have been reduced. The main reasons for this improvement are the SEI capability maturity model (CMM and CMMI), the use of Six-Sigma methods for software, and Watts Humphrey's team software process (TSP) and personal software process (PSP). Readers who have older editions of this book will note that the failure rate for large systems has improved somewhat.

The root causes identified for failed or canceled projects are too extensive to detail here, but include the following:

- Poor quality control
- Inadequate project planning
- Inadequate project estimating
- Inadequate milestone tracking
- Inadequate measurement methods
- Excessive schedule pressures
- Excessive requirements creep
- Project management malpractice
- Poor cooperation between clients and development team

Cancellations, major delays in excess of one calendar year, and cost overruns in excess of 100 percent remain endemic problems for software applications in the 100,000 function point size range, and larger.

As can easily be seen, small projects are comparatively "safe" in terms of premature mortality, but large software systems are one of the most hazardous and failure prone constructs of any industrial product.

The high failure rate on large systems is one of the key reasons why senior corporate executives seldom consider software executives as candidates for high-level corporate positions.

There is a general distrust and even dislike of the software community within many companies and government agencies. What fosters this dislike is the feeling that software is not fully under control and hence that software executives are not as capable as other kinds of executives who have fewer visible disasters in sight.

U.S. Averages for Software Life Expectancy

Once a software project reaches its clients, it is an interesting question as to how long the project will be utilized prior to being replaced by another product. As a general rule, the life expectancy of software is somewhat proportional to size and also varies by domain or subindustry.

Table 3-38 shows the approximate averages that are currently known. Neither the author nor his company have been commissioned to explore very small projects, so there are some gaps in the data for the lower end of the size ranges.

Note that the blanks in Table 3-38 are areas where the author does not have enough data to perform any kind of meaningful averages.

One alarming aspect of Table 3-38 is the fact that the life expectancy of large software applications is longer than the life expectancy of the programming languages used for development. As a result, many large software applications are written in languages that are now dead or orphaned, which means that there are no longer working compilers. Some large applications even run on custom hardware platforms that cannot easily be replaced or repaired.

Commercial software vendors such as Microsoft, Intuit, Symantec, and many others try to speed up the adoption of new versions and the replacement of older versions. The most common methods for doing this involve a "carrot and stick" approach. The "carrot" or beneficial aspect is to offer new and exciting features in the latest versions of software applications. The "stick" or harmful aspect is to adopt new file structures that are incompatible with older versions.

What the commercial vendors don't seem to realize is that their quality control is so poor that many users would prefer to keep older and stabler versions of software packages rather than using new and unstable versions. As an example, this manuscript was prepared using

TABLE 3-38 Average Software Life Expectancy in Calendar Years

Form of Software	100	1,000	10,000	100,000	Average
End-user	2.50	–	–	–	2.50
Web	1.00	3.00	5.00	–	3.00
MIS	3.50	6.00	11.00	20.00	10.13
U.S. outsource	3.00	6.00	10.00	17.00	9.00
Offshore outsource	3.00	6.00	10.00	15.00	8.50
Commercial	3.00	4.50	7.50	11.00	6.50
Systems	5.00	8.00	14.00	21.00	12.00
Military	5.00	8.00	19.00	26.00	14.50
Average	**3.25**	**5.93**	**10.93**	**18.33**	**8.27**

Office 2007. A number of annoying problems were noted in both Excel and Word, which slowed down the production of the manuscript.

Since the work of this third edition was based on the text of the second edition, it was also necessary to save the text and spreadsheets in the older format because Office 2007 adopted a new and different format. At least Microsoft has learned that it is mandatory to support older file formats, which they did not do previously.

A similar issue has been noted with Intuit's Quicken financial package. The older versions cannot open Quicken files saved with the Quicken 2008 version, which means that users are faced with buying new software whether they want to or not.

It is an interesting question as to why large software systems have very long life expectancies. The probable reason is that replacement is too expensive for large systems, whereas small projects are more or less disposable. With software development costs running to about $1,000 per function point, replacing a major application in the 100,000 function point size range would call for an investment of $100,000,000 on top of the money already spent for the current version. Not only that but the current version would still have to be maintained for at least five years while the new version is being constructed, and there is no guarantee that the new version would work well for at least two years after deployment due to quality problems.

By coincidence, the life expectancy patterns of software projects more or less resemble a totally unrelated topic. The life expectancy of animals is generally proportional to their overall body weight. Small animals such as chipmunks, squirrels, and rabbits seldom live more than a few years, whereas larger animals such as horses and cows can live more than 20 years.

Information on software life expectancy is useful data for planning maintenance and enhancement staffing over long periods. A useful rule of thumb for this purpose is that it takes about one full-time person to keep from 500 to 2,000 function points of software running and operational. This means that a software project of 10,000 function points in size may need a perpetual staff of 5 to 20 personnel to fix bugs and perform minor updates.

Well-structured applications with few latent bugs, with adequate comment densities, and with excellent maintenance materials need the fewest personnel. Poorly structured systems with many bugs and marginal or missing maintenance materials need the largest support staffs.

The comparative labor intensiveness of software maintenance and enhancements over long time periods explains why there is a growing sub-industry of companies providing geriatric tools and services for

aging legacy systems. Some of the components of the software geriatric care business include

- Complexity analysis tools
- Code restructuring tools
- Defect tracking tools
- Maintenance cost estimating tools
- Maintenance cost tracking tools
- Configuration control tools
- Reverse engineering tools
- Reengineering tools
- Maintenance workbenches

The overall effectiveness of an active geriatric program is rather high. The author has observed a tripling of maintenance assignment scopes on legacy systems in less than four years (i.e., function points supported by one person) as a direct result of formal geriatric programs that reduced complexity via restructuring and reengineering.

U.S. Average for Annual Enhancement Volumes

Once software goes into production it normally continues to grow and add features for much of its useful life. Since changes are made in response to new user requests, to changes in regulations or statutes, or to improve aspects of the current application it is difficult to characterize the nature of enhancements succinctly.

Table 3-39 gives the approximate annual volumes of new and changed features, compared against the function point totals. That is, an annual

TABLE 3-39 Average Rate of Annual Enhancements (Data is Based on Percentage Change of Application Function Points)

Form of Software	100	1,000	10,000	100,000	Average
End-user	12.00%	0.00%	0.00%	0.00%	12.00%
Web	15.00%	11.00%	8.00%	0.00%	11.33%
MIS	7.00%	6.50%	7.00%	4.00%	6.13%
U.S. outsource	7.00%	6.50%	6.00%	3.90%	5.85%
Offshore outsource	7.00%	7.00%	6.50%	4.75%	6.31%
Commercial	10.00%	9.00%	7.00%	5.00%	7.75%
Systems	7.00%	6.50%	8.00%	4.00%	6.38%
Military	7.00%	8.00%	9.00%	6.50%	7.63%
Average	**9.00%**	**7.79%**	**7.36%**	**4.69%**	**7.92%**

rate of 8 percent would imply that for a 100 function point application, 8 new or changed function points were added during the year.

The zeros in Table 3-39 are for topics where the author does not have enough data to form meaningful averages or for applications that don't actually exist such as end-user applications larger than 100 function points in size.

Average values such as those in Table 3-39 don't really tell the whole story. The maximum changes observed in applications for a single year can approach 100 percent when, for example, a mainframe application migrates to become a client-server application or when a Windows application is ported to Linux.

SPR has noted several companies where the volatility of their software projects is running in excess of 25 percent per year. That is, about one-fourth of the total quantity of function points in the application at the beginning of a year will be modified or added before the year is out. Applications dealing with volatile business topics tend to change very rapidly.

Conversely, SPR has noted a few projects where the volatility hovers around 0 percent. That is, for various business reasons, the software is not modified. Primarily applications that don't change are for relatively stable topics such as compilers for older programming languages.

U.S. Averages for Software Costs

As mentioned previously in this chapter, software costs are extremely variable. Staff compensation for identical jobs varies from country to country, industry to industry, region to region, and company to company. Even more variable are the overhead costs or burden rates applied on top of compensation for benefits, rent, depreciation, and the like.

Table 3-40 gives the approximate "average" compensation levels among the various domains discussed in this book. However, both

TABLE 3-40 U.S. Average Staff Compensation and Burden Rate Levels (Data Illustrates Monthly Compensation Levels)

Form of Software	Average Salary	Average Burden Percent	Monthly Burden Cost	Total Monthly Cost
End-user	$6,500.00	60	$3,900.00	$10,400.00
Web	$6,400.00	70	$4,480.00	$10,880.00
MIS	$5,850.00	60	$3,510.00	$9,360.00
U.S. outsource	$5,400.00	150	$8,100.00	$13,500.00
Offshore outsource	$2,200.00	125	$2,750.00	$4,950.00
Commercial	$5,700.00	60	$3,420.00	$9,120.00
Systems	$6,250.00	65	$4,062.50	$10,312.50
Military	$5,000.00	225	$11,250.00	$16,250.00
Average	**$5,412.50**	**102**	**$5,184.06**	**$10,596.56**

compensation levels and burden rates can change independently over ranges of about 3 to 1 in the United States and 8 to 1 abroad.

Using the cost profiles shown in Table 3-40, Table 3-41 shows the approximate cost per function point for the eight domains and the overall averages for the U.S. by size range. Table 3-41 shows the fully burdened costs; i.e., *salary* + *burden*. This table also uses productivity data from Table 3-30, earlier in the book.

Readers are cautioned that there is about a 2 to 1 range in cost structures and a 3 to 1 range in actual productivity compared to the values shown in Table 3-41. Averages are interesting, but they are not sufficient for serious business purposes such as cost estimating, budgets, or contracts.

Because abstract metrics such as cost per function point don't make it easy to visualize the real magnitude of software costs, Table 3-42 shows the approximate total costs for building software applications in various size ranges for the eight domains discussed in this book assuming fully burdened cost structures.

As can be seen from Table 3-42, software is a major business expense for U.S. business, military, and government operations. Few other parts of U.S. industry are simultaneously so expensive, so troublesome, and yet so necessary for success.

To provide a context for the high cost of software development, consider the costs of some other objects and items:

- 1 function point costs more than a high-definition television set.
- 10 function points cost more than motorcycle.
- 100 function points cost more than an automobile.
- 1,000 function points cost more than a family home.
- 10,000 function points cost more than an airliner.
- 100,000 function points cost more than a large office building.

TABLE 3-41 U.S. Average Development Cost per Function Point (Data Includes *Salary* + *Burden* or *Overhead*)

Forms of Software	100	1,000	10,000	100,000	Average
End-user	$197.72	$ –	$ –	$ –	$197.72
Web	$230.02	$425.00	$906.67	$ –	$520.56
MIS	$445.71	$570.73	$2,012.90	$3,466.67	$1,624.00
U.S. outsource	$352.17	$784.88	$2,755.10	$4,218.75	$2,027.73
Offshore outsource	$260.53	$313.29	$1,053.19	$1,650.00	$819.25
Commercial	$829.09	$980.65	$1,982.61	$2,280.00	$1,518.09
Systems	$1,145.83	$1,494.57	$2,022.06	$2,484.94	$1,786.85
Military	$2,901.79	$3,385.42	$4,276.32	$7,738.10	$4,575.40
Average	**$795.36**	**$1,136.36**	**$2,144.12**	**$3,639.74**	**$1,928.90**

TABLE 3-42 U.S. Average Software Development Costs (Data Includes *Salary + Burden* or *Overhead*)

Form of Software	100	1,000	10,000	100,000	Average
End-user	$19,772	$ –	$ –	$ –	$19,772
Web	$23,002	$425,000	$9,066,667	$ –	$3,171,556
MIS	$44,571	$570,732	$20,129,032	$346,666,667	$91,852,751
U.S. outsource	$35,217	$784,884	$27,551,020	$421,875,000	$112,561,530
Offshore outsource	$26,053	$313,291	$10,531,915	$165,000,000	$43,967,815
Commercial	$82,909	$980,645	$19,826,087	$228,000,000	$62,222,410
Systems	$114,583	$1,494,565	$20,220,588	$248,493,976	$67,580,928
Military	$290,179	$3,385,417	$42,763,158	$773,809,524	$205,062,069
Average	**$79,536**	**$994,317**	**$18,761,058**	**$272,980,646**	**$73,203,889**

It is no wonder that corporate executives are less than pleased with the overall performance of software managers and software engineers. The combination of extremely high costs, high failure rates, and high defect levels makes software development the most troublesome technology of the modern industrial era.

It is also of interest to consider the cost per function point using only basic salary or compensation levels, without including the burden rates. Table 3-43 gives the average cost per function point without applying the burden; i.e., only basic salary rates are included.

The data in Table 3-43 is interesting, but not really relevant since most organizations apply their burden rates when performing cost studies. What makes Table 3-43 interesting is the fact that the relative efficiency levels of outsource contractors and their clients (MIS development groups) favor the outsource organizations when burden rates are removed.

TABLE 3-43 U.S. Average Software Development Labor Costs (Data Includes Only Salaries; Burden Rates or Overhead are Omitted)

Form of Software	100	1,000	10,000	100,000	Average
End-user	$123.57	$ –	$ –	$ –	$123.57
Web	$135.31	$250.00	$533.33	$ –	$306.21
MIS	$278.57	$356.71	$1,258.06	$2,166.67	$1,015.00
U.S. outsource	$234.78	$313.95	$1,102.04	$1,687.50	$834.57
Offshore outsource	$115.79	$139.24	$468.09	$733.33	$364.11
Commercial	$518.18	$612.90	$1,239.13	$1,425.00	$948.80
Systems	$694.44	$905.80	$1,225.49	$1,506.02	$1,082.94
Military	$892.86	$1,041.67	$1,315.79	$2,380.95	$1,407.82
Average	**$374.19**	**$517.18**	**$1,020.28**	**$1,649.91**	**$890.39**

The author receives more than half a dozen requests each month for data on "average cost to build a function point." Sometimes these requests are due to fairly serious business situations such as contract litigation or tax litigation. The ranges are so broad for cost per function point that the author usually declines to provide "average" data and suggests instead that each company use its own specific compensation and burden rate levels.

Somewhat reluctantly, Table 3-44 reflects the author's judgment of what the most common (or mode) would be for average U.S. cost per function point in the domains included in this book, using round numbers to reflect the general lack of precision of the concept.

The averages shown in Table 3-44 are the arithmetic means of the two columns and hence should be used only with caution.

It should be recalled that the cost information shown here is an attempt to reflect 100 percent of the work effort associated with software. The costs start with requirements and include all development activities. Quality assurance, production of user manuals, and project management are included as well as normal design, code, and testing activities. The costs are much higher than might be observed under two other conditions:

- For companies whose measurements are centered on design, code, and unit testing (commonly abbreviated to DCUT) costs will appear to be only about 25 percent of the costs shown in these cost tables.
- For companies whose measurements are based on normal cost tracking systems that typically "leak" an average of about 50 percent of the real effort on software projects (i.e., unpaid overtime, time spent prior to turning on the tracking systems, specialists such as quality assurance, etc.) costs will only appear to be about 50 percent of the costs shown in these tables.

TABLE 3-44 U.S. Average Costs per Function Point With and Without Burden Rates

	Unburdened	Fully Burdened
End-user software	$123.57	$197.72–$250
Web software	$306.21	$520.36
Information systems software	$1015.00	$1624.00
U.S. outsource software	$834.57	$2027.74
Offshore outsource	$364.11	$819.25
Commercial software	$948.80	$1518,09
Systems software	$1082.94	$1786.85
Military software	$1407.82	$4575.40
Average	**$760.37**	**$1633.70**

On the whole, the cost of software is one of the most troublesome, ambiguous, and poorly recorded aspects of the entire software world.

U.S. Averages for Defect Potentials

The "defect potential" of a software application is the sum of all defects found during development and out in the field when the application is used by clients or customers. The kinds of defects that comprise the defect potential include five categories:

- Requirements defects
- Design defects
- Source code defects
- User documentation defects
- "Bad fixes" or secondary defects found in repairs to prior defects

The information on potential defects in this book is derived from observations of software projects that utilized formal design and code inspections plus full multistage testing activities. Obviously the companies also had formal and accurate defect tracking tools available.

There are normally several gaps in software defect statistics: (1) Desk checking or defects found privately by development personnel; (2) Defects found during unit testing by the programmers themselves; (3) Defects found during informal reviews by friends or colleagues of development personnel.

From time to time, companies perform experiments and ask for volunteers to provide data on defects found during these gaps in conventional defect statistics. Indeed, the author has been such a volunteer and has kept records of the numbers of errors found in his own software while desk checking and unit testing.

Table 3-45 gives the approximate U.S. average results in terms of defect potentials per function point. Note that this kind of data is clearly biased, since very few companies actually track lifecycle defect rates with the kind of precision needed to insure really good data on this topic.

Software development is a very difficult intellectual undertaking that is high in complexity and hence prone to a variety of bugs or errors. As a general rule, the major cost and schedule drivers associated with civilian software are those associated with defect removal activities, which cost more than anything else. (For military software, paperwork costs more than anything else.)

TABLE 3-45 U.S. Average Defect Potentials per Function Point (Sum of Requirements, Design, Code, Document, and Bad-Fix Defects)

Form of Software	100	1,000	10,000	100,000	Average
End-user	3.50	–	–	–	3.50
Web	4.00	4.60	5.60	–	4.73
MIS	4.00	5.00	6.00	7.25	5.56
U.S. outsource	3.80	4.50	5.60	6.75	5.16
Offshore outsource	4.10	5.05	6.30	7.40	5.71
Commercial	4.00	5.00	6.40	7.70	5.78
Systems	5.00	6.00	7.00	8.00	6.50
Military	5.50	6.75	7.80	8.50	7.14
Average	**4.24**	**5.27**	**6.39**	**7.60**	**5.87**

The overall ranking of U.S. software expense and schedule elements is of interest, since coding is only in fourth place:

1. Defect removal operations
2. Paperwork in all forms
3. Meetings and communication
4. Code development

Software quality cannot be ignored for projects that want to minimize costs and schedules. If quality control is not excellent upstream during requirements, design, and specification phases, then the downstream effort, schedule, and costs that accrue during testing can be of major proportions.

U.S. Averages for Cumulative Defect Removal Efficiency

The defect potential is not, of course, the quantity of errors or bugs delivered to customers. The potential defects are reduced by a series of reviews, inspections, and test steps so that only a fraction of the potential defects reach software clients.

Since each defect removal step, on average, is only about 30 percent efficient in finding bugs or errors, it is obvious that quite a few steps are needed in order to eliminate software errors.

Table 3-46 reflects the average "cumulative defect removal efficiency" or percentage of software defects found and eliminated prior to delivery of software to its intended users.

Note that defect removal efficiency declines with the overall sizes of software applications. There are also significant variations among the six sub-industries or domains included in this book.

TABLE 3-46 U.S. Average for Defect Removal Efficiency Before Delivery (Reviews, Inspections, and All Forms of Testing)

Form of Software	100	1,000	10,000	100,000	Average
End-user	70.00%	–	–	–	70.00%
Web	87.00%	87.00%	82.00%	–	85.33%
MIS	92.00%	85.00%	81.00%	65.00%	80.75%
U.S. outsource	95.00%	87.00%	84.00%	74.00%	85.00%
Offshore outsource	90.00%	84.00%	82.00%	70.00%	81.50%
Commercial	94.00%	92.00%	90.00%	88.00%	91.00%
Systems	97.00%	96.00%	95.00%	93.00%	95.25%
Military	96.00%	93.00%	92.00%	91.00%	93.00%
Average	**90.13%**	**89.14%**	**86.57%**	**80.17%**	**86.50%**

As a general rule, the best industries for software quality control are those where software controls large and complicated physical devices such as computers, switching systems, weapons systems, aircraft, medical instruments, and the like. The best industries overall in terms of software defect removal efficiency include

- Telecommunication manufacturers
- Computer manufacturers
- Aircraft manufacturers
- Medical instrument manufacturers
- Defense weapons systems manufacturers
- Commercial software manufacturers

The larger commercial software houses such as Microsoft are now starting to achieve results that compare favorably to the traditional large-systems houses such as AT&T and IBM. Now that Windows NT and Windows 95 are approaching the size and complexity levels of software such as IBM's MVS operating system or AT&T's ESS5 switching system, Microsoft is beginning to utilize similar quality control approaches.

U.S. Averages for Delivered Software Defects

If the defect potential and defect removal efficiency levels are known, it is straightforward to predict the number of software defects still latent at the time of software release. Table 3-47 shows the approximate number of software errors still present at delivery to software clients in terms of defects per function point.

TABLE 3-47 U.S. Average for Delivered Defects per Function Point

Form of Software	100	1,000	10,000	100,000	Average
End-user	1.05	–	–	–	1.05
Web	0.52	0.60	1.01	–	0.71
MIS	0.32	0.75	1.14	2.54	1.19
U.S. outsource	0.19	0.59	0.90	1.76	0.86
Offshore outsource	0.41	0.81	1.13	2.22	1.14
Commercial	0.24	0.40	0.64	0.92	0.55
Systems	0.15	0.24	0.35	0.56	0.33
Military	0.22	0.47	0.62	0.77	0.52
Average	**0.39**	**0.55**	**0.83**	**1.46**	**0.81**

Because abstract metrics such as "defects per function point" do not make it easy to visualize the actual quantities of software errors likely to remain in software at the time of delivery, Table 3-48 shows the overall probable quantity of delivered software defects.

From the rather significant volumes of defects still present at the time of delivery, it is easy to see why software producers have such large customer support staffs and such significant maintenance or defect repair budgets for many years after the delivery of software to clients.

Note that not all defects are of equal importance or equal severity. The software industry usually categorizes customer-reported defects using some kind of a weighting factor. A very common schema for this

TABLE 3-48 U.S. Average for Number of Delivered Defects

Form of Software	100	1,000	10,000	100,000	Average
End-user	105	–	–	–	105
Web	52	598	10,080	–	3,577
MIS	32	750	11,400	253,750	66,483
U.S. outsource	19	585	8,960	175,500	46,266
Offshore outsource	41	808	11,340	222,000	58,547
Commercial	24	400	6,400	92,400	24,806
Systems	15	240	3,500	56,000	14,939
Military	22	473	6,240	76,500	20,809
Average	**39**	**551**	**8,274**	**146,025**	**38,722**

purpose is the four-level severity scale originally developed by IBM in the early 1960s:

Severity 1	Critical problem (software does not operate at all)
Severity 2	Significant problem (major feature disabled or incorrect)
Severity 3	Minor problem (some inconvenience for users)
Severity 4	Cosmetic problem (spelling errors in messages; no affect on operations)

Table 3-49 shows the approximate overall distributions of high-severity software defects in severity levels 1 and 2 at the time of delivery to customers.

As can be seen, most of the really serious severity 1 and severity 2 defects are eliminated before delivery. However, for large systems, the number of latent high-severity software defects is very troublesome. The example of the long delays in opening up the Denver Airport due to software defects in the luggage handling system illustrate the point very clearly.

High-severity defects still present at delivery explain why installing and using new releases of major applications such as ERP packages, Microsoft Vista, Microsoft Office, and other large applications take so long and cost so much.

U.S. Averages for Potential and Actual Software Reusability

Software reuse is an emerging technology of major importance. Reuse holds the promise of making more quality and productivity improvements for software than any other technology to date.

However, reuse encompasses much more than just reusing source code. SPR is now beginning to collect information on 12 categories of reusable software artifacts.

TABLE 3-49 U.S. Averages for Number of Delivered High-Severity Defects

Form of Software	100	1,000	10,000	100,000	Average
End-user	23	144	–	–	23
Web	11	180	2,722	–	971
MIS	7	140	3,078	71,050	18,569
U.S. outsource	4	194	2,419	49,140	12,939
Offshore outsource	9	96	3,062	62,160	16,332
Commercial	5	58	1,728	25,872	6,916
Systems	3	113	945	15,680	4,185
Military	5	–	1,685	21,420	5,777
Average	**9**	**112**	**2,234**	**40,887**	**10,810**

While the literature on software reuse is expanding, as are tools that support reuse, a number of major elements are underreported. For example, there is little current information in print on reusable user manuals, reusable cost estimates, or reusable test materials.

As this book is being written, the outsource software community and the commercial software community tend to be somewhat ahead in overall volumes of reusable material. However, reuse is such an important topic that essentially every domain concerned with large software projects is mounting some kind of research and development program into reusable artifacts.

From discussions and conversations with reusability researchers, it is apparent that much more reuse would be theoretically possible than has, in fact, been achieved to date. The author's view of the maximum potential for reuse of these 12 artifacts would be

Software Artifacts	Potential Reuse
1. Reusable requirements	30%
2. Reusable architecture	60%
3. Reusable plans	30%
4. Reusable cost estimates	30%
5. Reusable designs	70%
6. Reusable source code	90%
7. Reusable data elements	60%
8. Reusable interfaces	95%
9. Reusable screens	50%
10. Reusable user manuals	25%
11. Reusable test plans	60%
12. Reusable test cases	55%
Average (arithmetic mean)	**55%**

If pushed to optimum levels, the volume of reusable material would more than double today's averages. This would be a notable step toward reducing the costs and improving the quality of the software industry.

The use of percentages is imprecise and indicates the following: When you examine a software artifact such as a test plan or test library, what percentage of the volume of material for the current application has been used with other applications?

In order for reusability to be effective it requires a very high level of quality control. Reusing garbage or defective components is not a viable strategy. Therefore, effective reusability programs must be associated with effective software quality assurance programs. The strong coupling of reuse and quality limits the applicability of reuse to enterprises that are somewhat better than average.

The Impact of Technology on Software Productivity and Quality Levels

One of the main uses of benchmark data, such as the Software Productivity Research knowledge base and the International Software Benchmark Standards Group (ISBSG) is to provide a research vehicle for judging the effects of various tools, programming languages, and methodologies on software productivity, quality, schedules, costs, and maintainability.

The usual mode of this research is to measure a sample of projects that used the topic in question (i.e., a particular software engineering tool, programming language, or some other new technology) and compare the results against a sample of similar projects that did not use the same approach.

This methodology allows fairly granular analyses that can actually distinguish fairly small differences in tools, languages, and methods. However, due to the very large numbers of specific vendors, it is more convenient to express the results at the levels of "classes" of technologies; i.e., lump all OO programming languages together rather than discuss C++, Objective C, Eiffel, Actor, etc. separately. Indeed, with more than 700 programming languages in use and often more than a dozen languages in the same application, it would be very difficult to measure a single programming language in isolation.

Since new languages, tools, and approaches appear frequently and at random intervals, this kind of research essentially never ends and is a continuous study. In the time between the first, second, and third editions of this book, quite a number of new technologies have emerged, and some older technologies such as the object-oriented paradigm have expanded into new prominence.

There are far too many tools and approaches in the software world to deal with each one individually, so of necessity the information in this book will deal with classes of technologies rather than specific instances. Table 3-50 gives an overview of the number of software tools and methods in use in the United States in 2008.

The overall numbers of software technologies that might be considered is far too large for a company the size of the author's, or really for any company or university, to deal with all of them. Even large software research houses such as IBM, Gartner Group, and the Software Engineering Institute (SEI) can only deal with small samples of the available software tools and technologies:

Not only are there many tools and methods currently available, but also the creation of new tools and methods is expanding as the software industry matures and becomes more sophisticated.

The author's company has been keeping a table of programming languages since 1986. New languages have been entered into this table

TABLE 3-50 Approximate Number of U.S. Software Tools and Methods Circa 2008

Technology Domain	Number of Commercial Tools
Agile support tools	20
Maintenance support tools	20
Client/server tools	20
Configuration control tools	20
Document support tools	25
Software metrics/measurement tools	35
Software testing tools	40
Software defect removal methodologies	40
Commercial software engineering tools	50
Commercial software cost estimating tools	50
Object-oriented programming tools	50
Software methodologies	65
Project management tools	100
Programming languages and dialects	500
Standalone software tools	3000

almost every week since it was started. The total number of languages in the table is now about 700, and there is no apparent end to the creation of new programming languages and new dialects. Languages such as E and Ruby which did not exist at the time of the 2nd edition 12 years ago are now in the mainstream.

One of the problems of evaluating software technologies is to measure exactly what changes and how to express the results of the changes. The method the author selected for this chapter encompasses eight aspects of using acquiring and using new tools or methods:

- The approximate learning curve in months to became familiar with the approach
- The initial productivity impact during the learning curve
- The long-range productivity impact after the technology is fully understood
- The impact on defect potentials, if any
- The impact on defect removal efficiency, if any
- The impact on software schedules, if any
- The approximate return on investment
- Cautions or counterindications for situations where the approach may be hazardous
- Usage of the technologies among the author's clients

Six of these nine factors can be expressed as a percentage change compared to "average" results. This is not a very accurate method, but it has the virtue of being consistent across any kind of tool, method, or programming language.

The learning curve is expressed in terms of calendar months required to study and use the new method long enough to become comfortable with it. This information is derived from interviewing practitioners about their experiences.

The course length is expressed in terms of business days required to take a normal introductory course to the topic in question. This information is derived from scanning the course catalogs of commercial software education groups and the in-house catalogs of large corporations such as AT&T and IBM. (University education operating on a semester or trimester basis is too sluggish for most software professionals.)

Almost every significant technology reduces productivity temporarily while software professionals learn how to use the new method and begin to ascend the learning curve. This fact is true outside of software, of course, but the software literature does not accurately reflect the short-term loss of productivity while the learning curve is being ascended. This information is derived from benchmark and assessment studies.

One of the most visible instances of a steep and protracted learning curve is that associated with various flavors of object-oriented analysis and design. The learning curve for OO analysis and design is so steep that roughly 50 percent of the projects attempting to use this approach for the first time abandon it before completion or bring in older methods to supplement the OO approach in order to complete the project.

After a few months to a year, when the new technology is fully absorbed, it is interesting to see what kinds of long-range improvements (if any) result from its utilization.

Two flavors of software quality improvements are considered: (1) Reduction in defect potentials; (2) Improvements in defect removal efficiency. Both are important, and some technologies can benefit both aspects simultaneously. The information is derived from SPR assessment and benchmark studies.

An excellent example of a technology that benefits both defect prevention and defect removal is the use of formal inspections. Not only are inspections about twice as efficient in finding bugs as most kinds of testing, but participants in inspections rapidly improve their own work from being exposed to common problems in their colleagues work.

Schedule reduction is one of the most important topics in the software world, so any attempt to evaluate new or emerging technologies must be sensitive to this topic. Unfortunately, this topic is heavily impacted by exaggerated claims and false assertions. The information is derived from long-range analysis over a multi-year period of companies that

have adopted various approaches (and have also adopted good measurement practices).

The return on investment is the most complex issue. What constitutes "value" from a new approach can really comprise six different dimensions:

- Cost reductions when building software initially
- Cost reductions when supporting or maintaining software
- Speed improvements or shortening of development schedules
- Quality improvements in terms of reduced defect potentials
- Quality improvements in terms of elevated defect removal efficiency
- Morale improvements on the part of management or staff

In this edition, these six factors have been amalgamated by the author into a single overall value dimension:

Excellent	> 20% tangible improvements
Very good	> 15% tangible improvements
Good	> 10% tangible improvements
Fair	> 5% tangible improvements
Poor	No tangible improvements
Very poor	Negative or harmful results

Needless to say, this approximation is not very accurate and has a high margin of error. Even so, the results are interesting and hopefully may have some utility.

The topic of counter-indications is also very complicated. For example, the classes of application generators and fourth-generation languages often benefit productivity and can be very helpful. Yet these approaches would not be recommended for embedded, real-time software such as air-traffic control or target sensing routines onboard the Patriot missile because of performance and speed of execution issues.

Another example of a counter-indication deals with "rapid application development" or RAD. RAD approaches can be very useful for small projects, and especially so for small information systems. Yet the RAD approach can also be hazardous for applications larger than a few thousand function points in size or for those like military weapons systems or large systems software projects where quality and reliability are of prime importance. Similar results are noted using the Agile methods and also the CMM when used for the wrong kinds of applications.

The technologies ranking as "excellent" typically produce significant benefits and sometimes in more than a single factor. For example, inspections improve defect removal more than anything else, but also

have a positive impact on defect prevention, schedules, and productivity rates. Examples of technologies in the excellent category: inspections, prototypes, reusable code and reusable design, and achieving a level of 5 on the SEI CMM scale—the TSP and PSP methods, and Agile development.

At the opposite end of the spectrum are technologies that are harmful or that return zero or negative value. Obviously there is little value from being a 1 on the SEI CMM scale. Two rather popular methods return negative value, however, using "physical lines of code" or LOC as a normalizing metric and the use of logical statements as a normalizing metric.

Perhaps the worst software technology of all time was the use of physical lines of code. Continued use of this approach, in the author's opinion, should be considered professional malpractice. Physical lines of code have no economic validity and hence cannot be used for serious economic studies. While logical statements can at least be converted into function point metrics by means of the "backfiring" approach, there are no published algorithms for doing this based on physical lines of code.

The end result is that neither software productivity nor software quality can actually be measured using physical lines of code, and companies attempting to do this are essentially wasting far more than $1.00 for every $1.00 they spend. The primary loss is simply spending money for measurement without getting any useful data. The more serious hidden loss is the fact that concentrating primarily on LOC metrics blinds researchers, managers, and software staff to many important issues associated with the costs, schedules, and quality implications of non-code activities.

Coding is not the major cost driver for large-system development. Defect removal and the construction of paper documents (specifications, plans, user manuals, etc.) cost much more than the code itself. These major cost drivers can be measured directly using function point metrics, but cannot be measured using physical LOC metrics. Hence, the use of physical LOC metrics tends to blind researchers to perhaps 80 percent of all known software cost elements!

Between these extreme conditions of "excellent" and "very poor" are a number of technologies that do return positive value in at least one dimension. It should be noted that there is no "silver bullet" or single technology that can yield 10 to 1 improvements in productivity, quality, or any other tangible factor.

Table 3-51 shows a sample of some of the technologies that the author and his colleagues have examined and are continuing to examine.

An interesting but disturbing fact about the latest version of this table is that usage of technologies seems to be inversely related to their effectiveness. That is, the most powerful technologies have the lowest

TABLE 3-51 Overall Rankings of Software Technologies for Productivity, Schedules, and Quality

Technology	Learning Curve (Months)	Course Length (Days)	Initial Prod. Results	Final Prod. Results	Defect Potential Results	Defect Removal Results	Schedule Results	ROI	ROI $	Usage	Average Usage
Service-oriented architecture	6.00	5.00	–7.00%	40.00%	–12.00%	12.00%	–40.00%	Excellent	$30.00	5.00%	Excellent
Reusable code (high quality)	9.00	5.00	–10.00%	25.00%	–15.00%	2.50%	–30.00%	Excellent	$25.00	15.00%	9.56%
TSP and PSP	3.00	5.00	–3.00%	30.00%	–20.00%	12.00%	–15.00%	Excellent	$15.00	5.00%	
Design inspections	1.00	3.00	5.00%	20.00%	–12.00%	20.00%	–20.00%	Excellent	$12.00	7.00%	
Reusable design	9.00	3.00	–4.00%	10.00%	–20.00%	10.00%	–12.00%	Excellent	$11.00	5.00%	
Agile development	1.00	3.00	5.00%	30.00%	–15.00%	0.00%	–25.00%	Excellent	$10.00	20.00%	
Extreme programming (XP)	1.00	3.00	5.00%	30.00%	–15.00%	10.00%	–25.00%	Excellent	$10.00	13.00%	
Six-Sigma for software	1.00	3.00	0.00%	15.00%	–20.00%	12.00%	–10.00%	Excellent	$9.00	4.00%	
Scrum	1.00	1.00	5.00%	15.00%	–7.00%	6.00%	–15.00%	Excellent	$9.00	14.00%	
Code inspections	1.00	3.00	5.00%	17.00%	–20.00%	20.00%	–12.00%	Excellent	$9.00	7.00%	
Cost estimating tools	2.00	2.00	5.00%	8.00%	–7.00%	90.00%	–8.00%	Excellent	$8.50	8.00%	
Formal change control	2.00	3.00	–1.50%	6.50%	–6.00%	5.00%	–5.00%	Excellent	$8.50	17.00%	
SEI CMM Level 5	18.00	5.00	0.00%	20.00%	–10.00%	10.00%	–12.00%	Excellent	$8.00	1.00%	
Code renovation	6.00	5.00	7.00%	12.00%	–8.00%	7.00%	–8.00%	Excellent	$8.00	5.00%	
Quality measures	2.50	2.00	–1.00%	8.00%	–12.00%	12.00%	–10.00%	Excellent	$8.00	12.00%	
Formal assessments	2.00	3.00	–1.50%	4.00%	–5.00%	6.00%	–1.00%	Excellent	$8.00	15.00%	
QFD	1.00	3.00	–1.00%	7.00%	–20.00%	5.00%	–7.00%	Very good	$7.50	2.00%	Very good

(*Continued*)

TABLE 3-51 Overall Rankings of Software Technologies for Productivity, Schedules, and Quality (*Continued*)

Technology	Learning Curve (Months)	Course Length (Days)	Initial Prod. Results	Final Prod. Results	Defect Potential Results	Defect Removal Results	Schedule Results	ROI	ROI $	Usage	Average Usage
OO development	6.00	5.00	–7.00%	10.00%	–4.00%	0.00%	–10.00%	Very good	$7.00	15.00%	11.22%
Prototypes	2.00	1.00	0.00%	6.00%	–10.00%	5.00%	–8.00%	Very good	$7.00	30.00%	
Formal testing	1.00	3.00	0.00%	6.00%	0.00%	10.00%	–7.00%	Very good	$7.00	8.00%	
OO design	6.00	5.00	–6.00%	10.00%	–3.00%	0.00%	–10.00%	Very good	$6.50	14.00%	
SEI CMM Level 4	12.00	4.00	0.00%	15.00%	–7.00%	7.00%	–10.00%	Very good	$6.50	3.00%	
Risk management	3.00	3.00	4.00%	9.00%	–6.00%	4.00%	–7.00%	Very good	$6.00	7.00%	
Function point metrics	4.00	3.00	–2.00%	8.00%	–5.00%	5.00%	–7.00%	Very good	$6.00	12.00%	
JAD	2.00	2.00	–1.00%	8.00%	–12.00%	4.00%	–80.00%	Very good	$6.00	10.00%	
Complexity analysis tools	2.00	2.00	0.00%	3.00%	–6.00%	5.00%	–5.00%	Good	$5.25	5.00%	Good
Productivity measures	3.00	2.00	–1.00%	6.00%	0.00%	0.00%	–5.00%	Good	$5.00	8.00%	11.45%
Formal benchmarks	3.00	2.00	0.00%	3.50%	–3.00%	3.00%	–5.00%	Good	$5.00	6.00%	
Formal test plans	1.00	2.00	0.00%	4.00%	0.00%	8.00%	–5.00%	Good	$5.00	5.00%	
Reengineering legacy code	6.00	5.00	–6.00%	12.00%	–8.00%	2.00%	–4.00%	Good	$4.25	4.00%	
Spiral development	1.00	2.00	–4.00%	6.00%	–7.00%	6.00%	–6.00%	Good	$4.00	4.00%	
Reusable test cases	6.00	3.00	–2.00%	6.00%	–7.00%	4.00%	–3.00%	Good	$4.00	3.00%	
Reverse engineering tools	3.00	3.00	–2.00%	6.00%	0.00%	0.00%	–2.00%	Good	$4.00	4.00%	

Maintenance workbenches	2.00	3.00	–2.50%	6.00%	–2.00%	2.00%	–4.00%	Good	$4.00	5.00%	
Automated testing	2.00	3.00	–2.00%	3.00%	0.00%	4.00%	–4.00%	Good	$4.00	5.00%	
Test case generators	2.00	3.00	–2.00%	3.00%	0.00%	–4.00%	–4.00%	Good	$4.00	5.00%	
SEI CMM Level 3	9.00	3.00	–3.00%	10.00%	–5.00%	5.00%	–7.00%	Good	$3.75	20.00%	
ITIL	3.00	3.00	–5.00%	3.00%	0.00%	–2.00%	–5.00%	Good	$3.50	12.00%	
RAD	2.00	3.00	–3.00%	4.00%	–3.00%	0.00%	–5.00%	Good	$3.00	7.00%	
Iterative development	1.00	2.00	–4.00%	5.00%	–7.00%	6.00%	–6.00%	Good	$3.00	5.00%	
Project management tools	2.00	3.00	4.00%	9.00%	0.00%	0.00%	–5.00%	Good	$3.00	30.00%	
Configuration control tools	3.00	3.00	–2.00%	3.00%	–1.00%	1.00%	–3.00%	Good	$3.00	70.00%	
Code restructuring	3.00	3.00	7.00%	10.00%	–5.00%	3.00%	–2.50%	Good	$3.00	5.00%	
Code refactoring	6.00	3.00	5.00%	8.00%	–4.00%	3.00%	–2.00%	Good	$2.50	8.00%	
Development workbenches	2.00	3.00	–3.00%	5.00%	0.00%	0.00%	–1.00%	Fair	$2.00	18.00%	Fair
Test library control tools	3.00	3.00	–1.00%	2.00%	2.00%	0.00%	–2.00%	Fair	$2.00	20.00%	11.70%
Use cases	6.00	5.00	–3.00%	3.00%	0.00%	–1.00%	–1.00%	Fair	$2.00	10.00%	
Formal milestones	1.00	1.00	0.00%	2.00%	0.00%	0.00%	–1.00%	Fair	$2.00	22.00%	
Defect tracking tools	1.00	1.00	0.00%	2.00%	2.00%	0.00%	–1.00%	Fair	$2.00	18.00%	
TQM	4.00	3.00	–1.00%	2.00%	–3.00%	3.00%	–1.00%	Fair	$1.75	1.00%	
ISO 9000-9004 standards	2.00	3.00	–5.00%	0.00%	–1.00%	1.00%	–1.00%	Fair	$1.75	8.00%	
CASE and ICASE	3.00	5.00	–5.00%	2.50%	0.00%	0.00%	–1.00%	Fair	$1.50	12.00%	

(*Continued*)

TABLE 3-51 Overall Rankings of Software Technologies for Productivity, Schedules, and Quality (*Continued*)

Technology	Learning Curve (Months)	Course Length (Days)	Initial Prod. Results	Final Prod. Results	Defect Potential Results	Defect Removal Results	Schedule Results	ROI	ROI $	Usage	Average Usage
Reusable plans	6.00	2.00	–1.00%	2.00%	0.00%	0.00%	–1.00%	Fair	$1.25	4.00%	
Reusable documents	9.00	3.00	–1.00%	1.00%	0.00%	0.00%	–1.00%	Fair	$1.25	4.00%	
SEI CMM Level 2	6.00	2.00	–5.00%	5.00%	–2.00%	3.00%	–3.00%	Fair	$1.00	22.00%	
LOC (logical statements)	1.50	1.00	0.00%	0.00%	0.00%	0.00%	0.00%	Poor	$1.00	25.00%	Poor
SEI CMM Level 1	1.00	1.00	–10.00%	–5.00%	10.00%	–5.00%	5.00%	Poor	$5.00	75.00%	60.60%
Waterfall development	1.00	1.00	–7.00%	–5.00%	5.00%	–5.00%	5.00%	Poor	$5.00	78.00%	
LOC (physical lines)	1.00	1.00	–10.00%	–10.00%	0.00%	0.00%	12.00%	Poor	$15.00	55.00%	
Reusable code (low quality)	3.00	1.00	–10.00%	–10.00%	30.00%	–10.00%	15.00%	Poor	$30.00	70.00%	
Average	**3.55**	**2.87**	**–1.48%**	**7.94%**	**–4.77%**	**5.11%**	**–7.56%**	**Good**	**$4.74**	**15.12%**	

usage percentages, whereas the technologies that are actually harmful and degrade productivity and quality are used by more than 50 percent of all contemporary software projects circa 2008.

This is not really a surprise given the normal patterns of human living. Only a very small percentage of adults work out regularly, avoid fats and excessive starches, and live truly healthy lives. A majority of adults over eat, many smoke, many indulge in alcohol or recreational drugs, and comparatively few exercise on a regular basis.

The overall cumulative results are interesting but not terribly useful. They do indicate that if a company plans on absorbing several technologies more or less concurrently, they should plan to spend quite a bit of time on education. They should also expect fairly substantial initial productivity reductions while ascending the learning curve.

Both the need for education and the rather lengthy learning curves required to get up to speed in the more complicated technologies tend to be ignored by technology vendors and are under reported in the software literature as well.

The averages of the overall results are not totally useful either. However, they do make clear an important point: There is no "silver bullet" or single technology that, by itself, will create order of magnitude improvements.

However, dealing with the simultaneous and concurrent results from multiple technologies is a very difficult topic for measurement and benchmark studies. Since no single technology, by itself, is likely to achieve really impressive results, the next topic of interest is how to express the results that might occur from *combinations of technologies?*

One of the basic uses of this kind of information is to put together patterns of best current practices observed in various software domains. Table 3-52 is built from the information in Table 3-51, and illustrates the current "best in class" patterns of technologies associated with application sizes between 100 and 100,000 function points.

It is significant and interesting that not every software technology is appropriate for every kind of application. For example, the very successful "joint application design" approach requires that users participate during the JAD sessions. This limits the usefulness of the JAD approach to applications where the users are known, and hence for applications such as Windows Vista with the potential of millions of unknown users, JAD cannot really be applied.

Because the management information systems (MIS) population is larger than any other software domain, there are more vendors, tools, and methodologies available for information systems than for any other kind of software. Of course, outsourcers and contractors who build information systems have access to the same kinds of approaches.

TABLE 3-52 Software Technologies Noted on Applications of Various Size Ranges (Size Expressed in Terms of Function Points)

Technology	100	1,000	10,000	100,000
Excellent				
Service-oriented architecture	No	No	Yes	Yes
Reusable code (high quality)	Yes	Yes	Yes	Yes
TSP and PSP	Yes	Yes	Yes	Yes
Design inspections	Maybe	Yes	Yes	Yes
Reusable design	Yes	Yes	Yes	Yes
Agile development	Yes	Yes	Maybe	Maybe
Extreme programming (XP)	Yes	Yes	Maybe	Maybe
Six-Sigma for software	Yes	Yes	Yes	Yes
Scrum	Yes	Yes	Yes	Yes
Code inspections	Yes	Yes	Yes	Yes
Cost estimating tools	Maybe	Maybe	Yes	Yes
Formal change control	Maybe	Yes	Yes	Yes
SEI CMM Level 5	Maybe	Maybe	Yes	Yes
Code renovation	Maybe	Yes	Yes	Yes
Quality measures	Maybe	Yes	Yes	Yes
Formal assessments	Maybe	Yes	Yes	Yes
Very Good				
QFD	No	Maybe	Yes	Yes
OO development	Yes	Yes	Yes	Yes
Prototypes	No	Maybe	Yes	Yes
Formal testing	Maybe	Yes	Yes	Yes
OO design	Maybe	Yes	Yes	Yes
SEI CMM Level 4	Maybe	Maybe	Yes	Yes
Risk management	Maybe	Maybe	Yes	Yes
Function point metrics	Yes	Yes	Yes	Yes
JAD	Maybe	Maybe	Yes	Yes
Good				
Complexity analysis tools	Maybe	Maybe	Yes	Yes
Productivity measures	Maybe	Yes	Yes	Yes
Formal benchmarks	Maybe	Yes	Yes	Yes
Formal test plans	Maybe	Maybe	Yes	Yes
Reengineering legacy code	Maybe	Maybe	Yes	Yes
Spiral development	Maybe	Maybe	Yes	Yes

TABLE 3-52 Software Technologies Noted on Applications of Various Size Ranges (Size Expressed in Terms of Function Points) (*Continued*)

Technology	100	1,000	10,000	100,000
Good (*Cont.*)				
Reusable test cases	Maybe	Maybe	Yes	Yes
Reverse engineering tools	Maybe	Maybe	Yes	Yes
Maintenance workbenches	Maybe	Maybe	Yes	Yes
Automated testing	Maybe	Maybe	Maybe	Maybe
Test case generators	Maybe	Maybe	Maybe	Maybe
SEI CMM Level 3	Yes	Yes	Yes	Yes
ITIL	Maybe	Maybe	Yes	Yes
RAD	Maybe	Maybe	Maybe	No
Iterative development	Maybe	Maybe	Maybe	Maybe
Project management tools	Maybe	Maybe	Yes	Yes
Configuration control tools	Yes	Yes	Yes	Yes
Code restructuring	Maybe	Maybe	Yes	Yes
Code refactoring	Yes	Yes	Yes	Maybe
Development workbenches	Yes	Yes	Yes	Yes
Fair				
Test library control tools	Maybe	Maybe	Yes	Yes
Use cases	Maybe	Maybe	Maybe	Maybe
Formal milestones	No	Maybe	Yes	Yes
Defect tracking tools	Maybe	Maybe	Yes	Yes
TQM	Maybe	Maybe	Maybe	Maybe
ISO 9000-9004 standards	Maybe	Maybe	Yes	Yes
CASE and ICASE	Maybe	Maybe	Maybe	Maybe
Reusable plans	Maybe	Maybe	Yes	Yes
Reusable documents	Maybe	Maybe	Maybe	Maybe
SEI CMM Level 2	Maybe	Maybe	No	No
Poor				
LOC (logical statements)	Maybe	Maybe	No	No
SEI CMM Level 1	Maybe	No	No	No
Waterfall development	Maybe	Maybe	No	No
LOC (physical lines)	No	No	No	No
Reusable code (low quality)	No	No	No	No

Some technologies work well with every size of application, such as object-oriented development. Others work well for small projects (such as RAD or Agile) but may not be suitable for very large applications.

Still other technologies, such as CMM Level 5 give excellent results for large systems but may be cumbersome for small projects of only 100 function points.

Note that several technologies, such as the use of "physical lines of code" as a normalizing metric, are "worst current practices" and should not be used within any domain under almost any circumstances since the end results are usually harmful.

The overall patterns of technology usage reflect those found in leading companies and government agencies. No single company and certainly no government agency is likely to use all of these best practices at the same time. However, if you perform assessments and baseline studies of hundreds of good to excellent companies, you can begin to see the patterns emerge.

You can also observe one of the distressing aspects of the software industry. Some companies neither explore best practices nor make any serious attempt to use them. For example, the author and his colleagues have done several consulting studies for large MIS shops with huge portfolios of aging legacy systems written in COBOL. It is obvious that a whole suite of geriatric tools and services might be useful for such companies, including but not limited to complexity analyzers, restructuring tools, reengineering tools, reverse engineering tools, renovation, etc. Yet may times the managers within the MIS domain have made no effort to acquire such technologies and are sometimes even surprised to find that geriatric tools are commercially available.

Evaluating the Productivity Impact of Multiple Technologies

Evaluating the simultaneous impact of multiple technologies is one of the most difficult research problems in the entire software world. Even though the author and his company have more data than most, it is still not an easy task. Earlier discussions in this chapter and the data in Table 3-8 illustrate some special combinations of technologies.

After experimenting with several ways of displaying the data derived from Software Productivity Research's multiple regression studies, an interesting method has been to show how four kinds of technologies or social factors interact with one another. This method involves limiting the selection to no more than four different approaches, and then showing the extreme conditions that can result from moving from "best" to "worst" for each factor alone, and then for all combinations and permutations of factors.

The reason that this method is limited to four approaches is that the 16 combinations that result is about the largest number that can be expressed in a table of convenient length. Each time a factor is added, the number of combinations doubles so that 5 factors would yield 32 combinations, 6 factors would yield 64 combinations, and so on.

Table 3-53 shows the interactions of the four most important factors that determine the outcomes of large software projects in the 10,000 function point range. The four key factors are

- The experience of the technical staff
- The experience of the project managers
- The sophistication of change control methods
- The sophistication of defect removal methods

There are, of course, many other factors that can influence the outcomes of software projects, such as the availability of high-quality reusable materials or the unexpected loss of key personnel. However, the four factors shown in Table 3-53 are of importance for all large projects of any kind.

The experience of the technical staff includes understanding both the application itself and also the tools and programming languages that will be used to develop it.

The experience of the project managers includes understanding of planning and estimating tools and understanding the importance of change control and quality control.

The sophistication of change controls includes formal change control boards, configuration control tools, and factoring all changes into cost and schedule estimates.

The sophistication of defect removal includes formal design and code inspections prior to testing, plus formal test planning and test case design. The specific test stages normally found on 10,000 function point applications include unit testing, new feature testing, regression testing, performance testing, security testing, system testing, external beta testing, and sometimes customer acceptance testing.

Table 3-53 shows the overall productivity ranges that occur from various combinations of these four key factors.

The last column of Table 3-53 shows the frequency of occurrence of each combination derived from the author's clients and from polls of the audience in various conferences and webinars. It is an unfortunate fact of life that the most sophisticated and effective development practices only occur rarely as of 2008.

The data is based on more than 250 large software projects explored between 1987 and 2007.

TABLE 3-53 Productivity Results on 10,000 Function Point Projects of Four Technology Factors (Date Expressed in Function Points per Staff Month)

		Low	Median	High	Frequency Percent
1.	Inexperienced staff Inexperienced managers Inadequate change control Inadequate defect removal	1.00	1.75	3.00	20
2.	Experienced staff Inexperienced managers Inadequate change control Inadequate defect removal	1.25	2.25	4.00	15
3.	Experienced staff Experienced managers Inadequate change control Inadequate defect removal	1.50	2.50	5.00	5
4.	Inexperienced staff Inexperienced managers Excellent change control Inadequate defect removal	2.00	2.75	5.25	2
5.	Inexperienced staff Inexperienced managers Inadequate change control Excellent defect removal	2.25	3.00	5.50	3
6.	Experienced staff Experienced managers Inadequate change control Inadequate defect removal	2.50	3.25	5.75	25
7.	Experienced staff Inexperienced managers Excellent change control Inadequate defect removal	2.75	3.75	6.25	3
8.	Experienced staff Inexperienced managers Inadequate change control Excellent defect removal	3.00	4.50	7.00	5

TABLE 3-53 Productivity Results on 10,000 Function Point Projects of Four Technology Factors (Date Expressed in Function Points per Staff Month) (*Continued*)

		Low	Median	High	Frequency Percent
9.	Inexperienced staff Experienced managers Excellent change control Inadequate defect removal	3.50	4.75	7.50	5
10.	Inexperienced staff Experienced managers Inadequate change control Excellent defect removal	3.75	5.00	8.00	5
11.	Inexperienced staff Inexperienced managers Excellent change control Excellent defect removal	4.00	5.50	8.50	1
12.	Experienced staff Inexperienced managers Excellent change control Inadequate defect removal	4.50	6.00	9.00	2
13.	Experienced staff Experienced managers Inadequate change control Excellent defect removal	5.00	6.50	9.50	2
14.	Experienced staff Inexperienced managers Excellent change control Excellent defect removal	5.50	7.00	10.00	3
15.	Inexperienced staff Experienced managers Excellent change control Excellent defect removal	6.00	7.50	10.50	3
16.	Experienced staff Experienced managers Excellent change control Excellent defect removal	7.00	10.00	13.00	1
					100

The purpose of the table is to illustrate that a single factor or technology, by itself, is not sufficient to make major improvements in software productivity. What is required is a combined approach with many concurrent improvements.

The four factors illustrated are only some of the topics that are known to impact software productivity and quality in favorable ways. Not shown because of space limitations are the impacts of reusability, object-oriented analysis and design, quality control methods such as inspections and quality function deployment, and many others. Each time a factor is included, the number of permutations doubles, so four factors is the maximum convenient number for a journal article.

As may be seen from the flow of information shown in Table 3-53 no single approach by itself is adequate to make large gains in software productivity. But multiple, concurrent improvements can create impressive results. Unfortunately, the best results also have the lowest frequency of usage.

The margin of error in the table is high. However, the overall flow of the information does reflect the results derived from hundreds of software projects in scores of companies and government agencies.

What Table 3-53 does not address, however, is the time and cost needed to make multiple concurrent improvements. If a company's technology base is as primitive as the one illustrated by the first plateau of Table 3-53 it can be three to five years and many thousands of dollars per staff member before the results shown in the sixteenth plateau can be achieved.

Evaluating the Quality Impact of Multiple Technologies

The same approach for dealing with the impact of multiple technologies on software productivity can also be used to illustrate the impact of various approaches on software quality.

It is obvious that no single defect removal operation is adequate by itself. This explains why "best in class" quality results can only be achieved from synergistic combinations of defect prevention, reviews or inspections, and various kinds of test activities.

Companies that can consistently average more than 95 percent in defect removal efficiency levels and keep defect potentials below about 3.0 per function point are moving toward "best in class" status.

The four quality factors selected for inclusion are these:

- Formal design inspections by small teams (usually three to five) of trained personnel. Formal inspections have the highest defect removal efficiency levels of any method yet studied. Design problems may outnumber code problems, so design inspections are a key technology for large systems. Inspections are used by essentially all "best in class" companies.

- Formal code inspections by small teams (usually three to five) of trained personnel. Formal code inspections differ from casual "walk-throughs" in that defect data and effort data is recorded and used for statistical purposes. Note that the defect data is *not* used for appraisal or personnel purposes.
- Formal quality assurance by an independent QA team, hopefully containing one or more certified QA analysts. The purpose of the formal QA team is to ensure that suitable defect prevention and removal operations have been selected and that relevant standards are adhered to.
- Formal testing by trained testing specialists. Testing is a teachable skill, although not many software development personnel have access to adequate training. Most of the "best in class" companies have established formal testing departments that take over testing after unit testing and deal with the more difficult topics of stress testing, regression testing, integration testing, system testing, and the like.

As with productivity, a useful way of showing the combined impacts of various defect removal operations is to show the permutations that result from using various methods singly or in combination. Since four factors generate 16 permutations, the results show that high quality levels need a multifaceted approach. Table 3-54 shows the cumulative defect removal efficiency levels of all 16 permutations of four factors.

Table 3-54 illustrates the "worst of the worst," which unfortunately is far too common in the software world. The essential message is that organizations that take no formal action to improve quality will probably not exceed 50 percent in overall defect removal efficiency.

Table 3-55 shows the impact of changing each factor individually. As before, no single change will yield results that are other than marginal improvements.

Table 3-56 shows the overall combinations of changing two factors at a time. As before, there are six combinations that might result.

The two-factor results approximate U.S. averages, and many companies and government groups are probably in this zone.

TABLE 3-54 Defect Removal Efficiency of 16 Combinations of 4 Defect Removal Methods

		Worst-Case Results		
		Worst	Median	Best
1.	No design inspections	30%	40%	50%
	No code inspections			
	No quality assurance			
	No formal testing			

TABLE 3-55 Defect Removal Efficiency of 16 Combinations of 4 Defect Removal Methods

		Single Factor Results		
		Worst	Median	Best
2.	No design inspections	32%	45%	55%
	No code inspections			
	Formal quality assurance			
	No formal testing			
3.	No design inspections	37%	53%	60%
	No code inspections			
	No quality assurance			
	Formal testing			
4.	No design inspections	43%	57%	66%
	Formal code inspections			
	No quality assurance			
	No formal testing			
5.	**Formal design inspections**	45%	60%	68%
	No code inspections			
	No quality assurance			
	No formal testing			

TABLE 3-56 Defect Removal Efficiency of 16 Combinations of 4 Defect Removal Methods

		Two Factor Results		
		Worst	Median	Best
6.	No design inspections	50%	65%	75%
	No code inspections			
	Formal quality assurance			
	Formal testing			
7.	No design inspections	53%	68%	78%
	Formal code inspections			
	Formal quality assurance			
	No formal testing			
8.	No design inspections	55%	70%	80%
	Formal code inspections			
	No quality assurance			
	Formal testing			
9.	**Formal design inspections**	60%	75%	85%
	No code inspections			
	Formal quality assurance			
	No formal testing			

TABLE 3-56 Defect Removal Efficiency of 16 Combinations of 4 Defect Removal Methods (*Continued*)

		Two Factor Results		
		Worst	Median	Best
10.	**Formal design inspections**	65%	80%	87%
	No code inspections			
	No quality assurance			
	Formal testing			
11.	**Formal design inspections**	70%	85%	90%
	Formal code inspection			
	No quality assurance			
	No formal testing			

The three factor results begin to achieve respectable levels of defect removal efficiency, as shown in Table 3-57.

For best of the best results, all defect removal operations must be present and performed capably, as shown by Table 3-58.

As may be seen from the progression through the 16 permutations in Tables 3-54 to 3-57, achieving high levels of software quality requires a multifaceted approach. No single method is adequate. In particular,

TABLE 3-57 Defect Removal Efficiency of 16 Combinations of 4 Defect Removal Methods

		Three Factor Results		
		Worst	Median	Best
12.	No design inspections	75%	87%	93%
	Formal code inspections			
	Formal quality assurance			
	Formal testing			
13.	**Formal design inspections**	77%	90%	95%
	No code inspections			
	Formal quality assurance			
	Formal testing			
14.	**Formal design inspections**	83%	95%	97%
	Formal code inspections			
	Formal quality assurance			
	No formal testing			
15.	**Formal design inspections**	85%	97%	99%
	Formal code inspections			
	No quality assurance			
	Formal testing			

TABLE 3-58 Defect Removal Efficiency of 16 Combinations of 4 Defect Removal Methods

		Best Case Results		
		Worst	Median	Best
16.	**Formal design inspections**	95%	99%	99.99%
	Formal code inspections			
	Formal quality assurance			
	Formal testing			

testing alone is not sufficient; quality assurance alone is not sufficient; inspections alone are not sufficient.

Technology Warnings and Counterindications

One important topic associated with software that is essentially never discussed in the literature is that of warnings, hazards, and counterindications. This topic deals with situations where various technologies might be counterproductive or even hazardous.

Following are observations derived from SPR assessments about selected technologies that can do harm in special situations.

Agile development and Extreme programming (XP) have both been effective for hundreds of applications ranging between about 100 function points and 1,000 function points. A few applications in the 10,000 function point range have reported success using either Agile or Extreme programming. However there are no reported results circa 2008 for really large applications in the 100,000 function point range. Also, both Agile and Extreme programming are new enough so that there is no data or literature on maintenance costs, bad-fix injections, or error-prone modules. Further, both Agile and Extreme seldom measure either productivity or quality. It may be that Agile and Extreme programming will be successful for large systems and successful for maintenance too. But as of 2008 there is not enough data to form a solid opinion.

Capability maturity model (CMM) developed by the Software Engineering Institute (SEI) originally covered less than a third of the factors known to impact software productivity and quality. For example, recruiting, compensation, and tools were not discussed at all. Further, the SEI CMM lacked any quantification of quality or productivity rates, so the whole concept could not be mapped to any empirical results. In addition, the SEI asserted that quality and productivity improved long before the method had been tested under field conditions. The end result is that climbing to a particular CMM level such as Level 2 or Level 3 does not guarantee success. The Navy has reported receiving software

from a CMM Level 3 contractor that had excessive defect levels. If the CMM is used only as a general scheme, it is fairly benign. If it is used rigidly, such as for determining contract eligibility, it is visibly defective and imperfect. The same statements apply to the newer CMMI. In general, both the CMM and CMMI give the best results for large systems above 10,000 function points in size. Below 1,000 function points both the CMM and CMMI are somewhat cumbersome.

However, recent data between 1997 and 2007 does indicate that both the CMM and CMMI have positive benefits for both quality and productivity when used for large systems.

Client-server architecture is much more complex than traditional monolithic architectures for software applications. The increase in complexity has not yet been accompanied by an increase in defect prevention or defect removal methods. The end result is that client-server applications typically have about 20 percent more potential defects than traditional applications and remove about 10 percent fewer of these defects before deployment. The result is twofold: (1) Near term quality and reliability problems; (2) Long-range elevations in maintenance costs as the client-server applications age to become legacy systems.

Computer-Aided Software Engineering (CASE) has two current deficiencies: (1) CASE tools seldom cover the "complete lifecycle" of software projects in spite of vendor claims; (2) CASE tools typically have a steep learning curve. The end result is that CASE tools have just about as great a chance to have zero or negative impacts on software costs and schedules as they do to have positive impacts. Any company that expects to achieve positive value from investments in CASE technology should prepare to spend about $1.00 in training on the tool's capabilities for every $1.00 spent on the tool itself.

ISO 9000-9004 standards tend to create volumes of paperwork that compare alarmingly with DoD 2167A. However, the author has not been able to find any solid empirical evidence that the ISO standards actually improve software quality. When this fact was broadcast via CompuServe and the Internet, the most surprising response was that "the ISO standards are not intended to improve quality but only to provide a framework so that quality improvements as feasible." The current end result of ISO certification for the software world is primarily an increase in paperwork volumes and costs and very little other tangible change. Of course, ISO certification is necessary for sale of some products in the European markets.

Lines of code metrics come in two flavors: (1) Logical statements; (2) Physical lines. Neither metric is suitable for economic studies or for serious quality analysis. However, the use of logical statements is marginally better because there are published rules and algorithms for converting logical statements into function points. There are no known

rules for converting physical lines into function points because of the random variations associated with individual programming styles interacting with hundreds of possible languages.

Over and above mere random variances and enormous variations, there are much deeper problems. The most important problem with LOC metrics of either kind is that the measured results run in the opposite direction from real results. That is, as economic productivity and quality improve, apparent results measured using LOC metrics will decline. It is this problem that causes the LOC metric to be assigned a "professional malpractice" label by the author.

Object-oriented analysis and design in all varieties tends to have a very steep and lengthy learning curve. The end result is that roughly half of the projects that use OO analysis and design for the first time either abandon the approach before completion or augment OO analysis and design by means of older, more familiar methods such as Warnier-Orr design, information engineering, or conventional structured analysis. If a company can overcome the steep learning curve associated with such a major paradigm shift, the use of OO analysis and design eventually turns positive. Don't expect much tangible improvement in the first year, however.

Further, use cases, which are often utilized during OO design, have very little published data on either design defects or even the volume of specifications produced. From examining the large systems in the 10,000 function point range where use cases were utilized, the number of design defects appeared close to U.S. averages of about 1.25 per function point. The volume or number of use cases is somewhat greater than conventional older forms of specifications such as HIPO diagrams, Nassi-Shneiderman charts, or Warnier-Orr diagrams.

Rapid Application Development (RAD) has been generally beneficial and positive for applications below about 1,000 function points in size. For really large systems above 5,000 function points in size, some of the shortcuts associated with the RAD concept tend to increase potential defects and reduce defect removal efficiency levels. The end result is that RAD may not be appropriate for large systems, nor for applications where quality and reliability are paramount such as weapons systems, systems software, and the like.

Using Function Point Metrics to Set "Best in Class" Targets

One of the useful by-products of the function point metric is the ability to set meaningful targets for "best in class" accomplishments. For the data shown here, no single company has been able to achieve all of the targets simultaneously. These "best in class" results are taken from

TABLE 3-59 U.S. "Best in Class" Productivity in Function Points per Staff Month

Form of Software	100	1,000	10,000	100,000	Average
End-user	150.00	–	–	–	150.00
Web	125.00	75.00	25.00	–	75.00
MIS	100.00	50.00	15.00	6.00	42.75
U.S. outsource	125.00	70.00	18.00	8.00	55.25
Offshore outsource	115.00	60.00	16.00	7.00	49.50
Commercial	100.00	60.00	15.00	9.50	46.13
Systems	80.00	50.00	15.00	7.50	38.13
Military	55.00	30.00	10.00	7.00	25.50
Average	**106.25**	**56.43**	**16.29**	**7.50**	**46.62**

companies that rank among the top 10 percent of the author's clients in terms of software productivity and quality levels in the various domains. As with any software data, there is a significant margin of error.

Note that the productivity targets are based on the standard SPR chart of accounts, which includes 25 development activities.

Table 3-59 indicates best in class software productivity rates expressed in terms of "function points per staff month."

The same information is presented in terms of "work hours per function point" in Table 3-60.

Any organization that can consistently approach or better the data shown in Tables 3-59 and 3-60 is very capable indeed. However, a strong caution must be given: the data in the tables assume *complete*

TABLE 3-60 U.S. "Best in Class" Productivity in Terms of Work Hours per Function Point (Assumes 132 Hours per Month)

Form of Software	100	1,000	10,000	100,000	Average
End-user	0.88	–	–	–	0.88
Web	1.06	1.76	5.28	–	2.70
MIS	1.32	2.64	8.80	22.00	8.69
U.S. outsource	1.06	1.89	7.33	16.50	6.69
Offshore outsource	1.15	2.20	8.25	18.86	7.61
Commercial	1.32	2.20	8.80	13.89	6.55
Systems	1.65	2.64	8.80	17.60	7.67
Military	2.40	4.40	13.20	18.86	9.71
Average	**1.35**	**2.53**	**8.64**	**17.95**	**7.62**

and accurate historical data for all activities starting with requirements and running through delivery to a real customer or client. If you compare only coding against the tables, it is easy to achieve the results. If you use data from cost tracking systems that "leak" half or more of the actual effort, it is not very difficult to approach the levels shown in these tables either.

Note that maintenance and enhancement projects in real life will have somewhat different results from those shown in the tables. However, when dealing only with "best in class" targets these differences are small enough to be negligible. When using these tables for maintenance and enhancement projects, note that it is the size of the enhancement or update, not the size of the base application, that should be the point of comparison.

Because software schedules are of such critical importance to the software industry, let's now consider best in class schedule results in Table 3-61.

These schedules are not easy to achieve, and if your organization can approach them, you can be justifiably proud of your accomplishments.

The best in class results for software quality don't differ quite so much from domain to domain. They do differ substantially with the size of the software application, however. For small projects of less than 100 function points, zero-defect levels are possible in terms of the numbers of delivered defects. For systems larger than 1,000 function points, and especially for software systems larger than 10,000 function points, zero-defect levels are theoretically possible but in fact the author has never seen it accomplished in more than 30 years.

Table 3-62 gives the best in class results in terms of "defect potentials." The term "defect potential" refers to the total number of possible

TABLE 3-61 "Best in Class" Application Schedules for Software Projects (Schedules Expressed in Terms of Calendar Months)

Form of Software	100	1,000	10,000	100,000	Average
End-user	1.50	–	–	–	1.50
Web	2.00	10.00	18.00	–	10.00
MIS	4.50	12.00	24.00	52.00	23.13
U.S. outsource	4.00	10.00	22.00	46.00	20.50
Offshore outsource	4.25	11.00	23.00	50.00	22.06
Commercial	5.00	12.00	24.00	48.00	22.25
Systems	5.00	14.00	28.00	48.00	23.75
Military	6.00	16.00	32.00	54.00	27.00
Average	**4.03**	**12.14**	**24.43**	**49.67**	**22.57**

TABLE 3-62 "Best in Class" Defect Potentials for Software Projects (Defects in Requirements, Design, Code, Documents, and "Bad Fixes")

Form of Software	100	1,000	10,000	100,000	Average
End-user	1.25	–	–	–	1.25
Web	1.50	2.25	2.75	–	2.17
MIS	1.75	2.75	3.50	4.60	3.15
U.S. outsource	1.50	2.50	3.25	4.25	2.88
Offshore outsource	1.60	2.60	3.30	4.50	3.00
Commercial	2.00	3.00	3.60	4.50	3.28
Systems	2.25	3.30	3.80	4.40	3.44
Military	2.50	3.65	4.00	4.80	3.74
Average	**1.79**	**2.86**	**3.46**	**4.51**	**3.16**

defects found in requirements, design, code, documents, plus "bad fixes" or secondary defects accidentally injected while repairing other defects. By way of context, the approximate U.S. average for potential defects is about 5.0 per function point circa 2008.

Table 3-63 shows the best in class results for "defect removal efficiency." The phrase "defect removal efficiency" is the percentage of total defects found and eliminated prior to delivery of software applications. As of 2008 the current U.S. average for defect removal efficiency is only about 85 percent, which is embarrassingly bad.

Now that best in class defect potentials and defect removal efficiency levels have been shown, Table 3-64 shows the number of delivered defects, expressed in terms of defects per function points.

TABLE 3-63 "Best in Class" Defect Removal Efficiency for Software Projects (Percentage of Defect Potentials Removed Before Delivery)

Form of Software	100	1,000	10,000	100,000	Average
End-user	100.00%	–	–	–	100.00%
Web	99.50%	97.00%	94.50%	–	97.00%
MIS	97.00%	96.00%	94.00%	92.50%	94.88%
U.S. outsource	99.00%	98.00%	96.00%	94.50%	96.88%
Offshore outsource	98.00%	97.00%	95.00%	93.50%	95.88%
Commercial	97.00%	95.00%	93.50%	92.50%	94.50%
Systems	98.50%	96.50%	96.00%	95.00%	96.50%
Military	98.00%	96.00%	95.50%	95.00%	96.13%
Average	**98.38%**	**96.50%**	**94.93%**	**93.83%**	**95.91%**

TABLE 3-64 "Best in Class" Delivered Defects for Software Projects (Data Expressed in Defects per Function Point)

Form of Software	100	1,000	10,000	100,000	Average
End-user	–	–	–	–	–
Web	0.01	0.07	0.15	–	0.08
MIS	0.05	0.11	0.21	0.35	0.18
U.S. outsource	0.02	0.05	0.13	0.23	0.11
Offshore outsource	0.03	0.08	0.17	0.29	0.14
Commercial	0.06	0.15	0.23	0.34	0.20
Systems	0.03	0.12	0.15	0.22	0.13
Military	0.05	0.15	0.18	0.24	0.15
Average	**0.03**	**0.10**	**0.17**	**0.28**	**0.15**

As of 2008, the current average in the U.S. for delivered defects per function point is around 0.75, so it can be seen that the best in class results are significantly better than average.

Table 3-65 illustrates the absolute numbers of defects still latent in software applications when the software is delivered to clients. Even for best in class applications above 10,000 function points in size, there are many thousands of delivered defects, unfortunately. However, many of these are low-severity defects that do not affect operations of the software; i.e., spelling errors or minor errors in terms of formatting.

TABLE 3-65 "Best in Class" Delivered Defects for Software Projects (Data Includes Defects of All Severity Levels)

Form of Software	100	1,000	10,000	100,000	Average
End-user	–	–	–	–	–
Web	1	68	1,513	–	527
MIS	5	110	2,100	34,500	9,179
U.S. outsource	2	50	1,300	23,375	6,182
Offshore outsource	3	78	1,650	29,250	7,745
Commercial	6	150	2,340	33,750	9,061
Systems	3	116	1,520	22,000	5,910
Military	5	146	1,800	24,000	6,488
Average	**3**	**102**	**1,746**	**27,813**	**7,416**

The data in Table 3-65 includes five classes of defects (requirements, design, code, user manuals, and bad fixes) and all four standard severity levels for software defects.

Severity 1	Total failure of application (1% of delivered defects)
Severity 2	Failure of a major function (11% of delivered defects)
Severity 3	Minor failure (50% of delivered defects)
Severity 4	Cosmetic error (38% of delivered defects)

Table 3-66 shows the approximate numbers of delivered defects in the categories of severity 1 and 2, which are the defects that actually interfere with operation of software applications.

Unfortunately for very large applications in the 100,000 function point class, there are still a great many serious defects delivered as of 2008. This situation is likely to continue into the indefinite future.

This chapter provides approximate U.S. averages for many different topics of interest, including productivity, quality, schedules, and costs. The margins of error are high, but unless data is published, there is no incentive to improve measurement practices and produce better data. Hopefully other researchers can use the data in this book as a starting point to improve both measurement practices and produce more accurate results.

In summary, software measurement is being rapidly transformed by the usage of function point metrics. The former lines of code (LOC) metric was reasonably effective during the early days of computing and software when low-level assembly language was the only major programming language in use. For assembly language programs and systems, code was the dominant cost driver.

TABLE 3-66 "Best in Class" Delivered Defects for Software Projects (Data Includes Only Severity 1 and Severity 2 Defects)

Form of Software	100	1,000	10,000	100,000	Average
End-user	–	–	–	–	–
Web	0	8	182	–	63
MIS	1	13	252	4,140	1,101
U.S. outsource	0	6	156	2,805	742
Offshore outsource	0	9	198	3,510	929
Commercial	1	18	281	4,050	1,087
Systems	0	14	182	2,640	709
Military	1	18	216	2,880	779
Average	**0**	**12**	**210**	**3,338**	**890**

More powerful languages changed the productivity equation and reduced the overall proportion of time and effort devoted to coding. With modern object-oriented programming languages, visual languages, and application generators, software can be built with coding effort dropping below 15 percent of the total work. In the modern world, usage of lines of code is ineffective and indeed harmful since this metric does not reveal the main cost drivers of modern software.

However, programming languages do not reduce the quantity of paper documents in the form of plans, specifications, and user manuals. The synthetic function point metric is now opening up new kinds of analyses and successfully showing the economic improvements associated with better tools, better programming languages, and better quality control approaches.

This chapter has a high margin of error, but it is encouraging that trends can be seen at all. Hopefully other researchers can correct any errors included here and expand the information as new and better data becomes available.

Chapter

4

The Mechanics of Measurement: Building a Baseline

Every company that produces software should have quantitative data on their productivity and quality rates. Every company should also have qualitative information about their development and maintenance methodologies. Furthermore, this information should be refreshed at least every two years, due to the rapid changes in software tools, methods, and programming languages. Unfortunately as of 2008, on-site observations at software companies and polls taken at software conferences indicate that less than 15 percent of U.S. software companies have either accurate quantitative data or reliable qualitative data. This lack of data explains, in part, why so many software projects fail, run late, or are delivered to customers with excessive volumes of serious bugs or defects.

Between the publication of the first edition in 1991 and this third edition, the related concepts of assessments, baselines, and benchmarks have entered the mainstream of software development. When the first two editions of this book came out in 1991 and 1996, respectively, quantitative data was very difficult to find. Data was available from companies such as Gartner Group and Software Productivity Research, but it was available only to clients. In 1997, however, the creation of the International Software Benchmarking Standards Group (ISBSG) made productivity and quality data widely available. It is now possible to acquire data on more than 4,000 projects in various forms such as CDs, monographs, or subscriptions.

Following are discussions of the kinds of information that companies should have to ensure professional levels of software development and maintenance:

- *Assessments* are structured evaluations of software processes, methods, and other influential factors using on-site interviews with software managers and staff.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

- *Baselines* are quantitative evaluations of an enterprise's current productivity and quality levels, to be used as a starting point for measuring rates of improvement.
- *Benchmarks* are quantitative evaluations of two or more enterprises to discover how each participant performs certain activities or achieves specific results.

These three are not mutually exclusive, and indeed using all three simultaneously results in the best understanding of software practices and how those practices impact quantitative results.

Software Assessments

A basic form of software qualitative measurement is an *assessment* performed by an independent, impartial team of assessment specialists. Software assessments have exploded in frequency under the impact of the well-known assessment technique developed by the Software Engineering Institute (SEI). SEI is a government-funded research institute located on the campus of Carnegie Mellon University in Pittsburgh, Pennsylvania.

The SEI assessment method is neither the first nor the only software assessment method used for software. But it has become the best known owing to widespread coverage in the software press and from the publication of a well-known book, Watts Humphrey's *Managing the Software Process*, which describes the assessment method used by the Software Engineering Institute (SEI). A second book dealing with software assessments was the author Capers Jones' *Assessment and Control of Software Risks*, which describes the results of the assessment method used by Software Productivity Research (SPR).

Both the SEI and SPR assessments are similar in concept to medical examinations. That is, both assessment approaches try to find everything that is right and everything that may be wrong with the way companies build and maintain software. Hopefully not too much will be wrong, but it is necessary to know what is wrong before truly effective therapy programs can be developed.

The SEI assessment approach has been in use since 1986. It was originally developed as a tool for exploring the capabilities of large defense contractors. The name of this assessment approach, the *Capability Maturity Model* or *CMM,* is now well known throughout the industry, and SEI-style assessments can be found in Europe, the Pacific Rim, South America, and essentially all over the world. There is also a newer expanded version of the CMM called *Capability Maturity Model Integration* or *CMMI.* There are other assessment methods too, such as the TickIT and SPICE assessments used in the United Kingdom and Europe. Many consulting companies can provide assessments as well.

Using the SEI CMM approach, software managers and staff are interviewed about the processes and methods deployed on software projects. The overall goal of the SEI CMM approach is to improve the sophistication or "maturity" with which software is built. To this end, the SEI has developed a five-plateau maturity scale, as shown here:

SEI Capability Maturity Model (CMM) Scoring System

CMM level	Frequency (%)	Approximate Definition
1 = initial	75.0	Primitive and random processes
2 = repeatable	15.0	Some standardized methods and controls
3 = defined	8.0	Well-structured methods with good results
4 = managed	1.5	Very sophisticated, with substantial reuse
5 = optimizing	0.5	State-of-the-art, advanced approaches

As can be seen, about 75 percent of all enterprises assessed using the SEI approach are at the bottom or "initial" level. SEI asserts that it takes about 18 months to move up a level, so software process improvement is a long-range goal.

A complete discussion of the SEI scoring system is outside the scope of this book. The SEI scoring is based on patterns of responses to a set of about 150 binary questions. The higher SEI maturity levels require "yes" answers to specific patterns of questions. For example, an SEI question dealing with quality assurance might look like this:

> Does your organization have a formal software quality assurance organization?

Software Productivity Research also has an assessment approach that is somewhat older than the SEI assessment approach. The SPR assessment approach also uses a five-point scale, but the results run in opposite directions.

An example of an SPR question that deals with quality assurance overlaps the same topic as the SEI question, although the formats are different. The SPR questionnaires use multiple-choice forms rather than the binary question formats used by SEI. One of the SPR questions associated with software quality is

Quality assurance function (select one choice)_____

1. Formal QA group with adequate resources
2. Formal QA group but understaffed (< 1 to 30 ratio)
3. QA role is assigned to development personnel
4. QA role is performed informally
5. No QA function exists for the project

TABLE 4-1 Distribution of Results Using the SPR Assessment Methodology, Percentage

	MIS/Web	Outsource	Commercial	Systems	Military	Average
1 = excellent	1.00	2.00	2.00	2.00	1.00	1.60
2 = good	12.00	22.00	23.00	24.00	16.00	19.40
3 = average	62.00	60.00	55.00	52.00	57.00	57.20
4 = poor	19.00	15.00	18.00	20.00	21.00	18.60
5 = very poor	6.00	1.00	2.00	2.00	5.00	3.20
	100.00	100.00	100.00	100.00	100.00	100.00

Like the SEI approach, the SPR assessment method also uses site visits and structured interviews covering several hundred factors that can influence the outcomes of software projects. The answers to the SPR questionnaires are input directly into a proprietary tool that performs a statistical analysis of the results and shows the mean values and standard deviations of all factors.

Table 4-1 is the SPR scoring system and the approximate percentages of results noted within five industry groups: commercial software, outsource contractors, military software, systems software, and management information systems software. As can be seen, most organizations are "average" in most topics covered. However, the systems, commercial, and outsource communities have a higher frequency of "good" and "excellent" responses than do the MIS, Web, and military communities. Note that in Table 4-1 the MIS and Web results are so close that the same column serves for both.

Because the SPR assessments produce typical bell-shaped curves and the SEI assessments have a skewed distribution, a simple inversion of the two scales is not sufficient to correlate the SPR and SEI results. Figure 4-1 illustrates the kinds of distribution associated with the SPR assessment results. However, by using both inversion and mathematical compression of the SPR scores, it is possible to establish a rough equivalence between the SPR and SEI scales. In order to achieve roughly the same distribution as the SEI results, some of the SPR scoring data has to be condensed, as follows:

SPR Scoring Range	Equivalent SEI Score	Approximate Frequency (%)
5.99–3.00	1 = initial	80.0
2.99–2.51	2 = repeatable	10.0
2.01–2.50	3 = defined	5.0
1.01–2.00	4 = managed	3.0
0.01–1.00	5 = optimizing	2.0

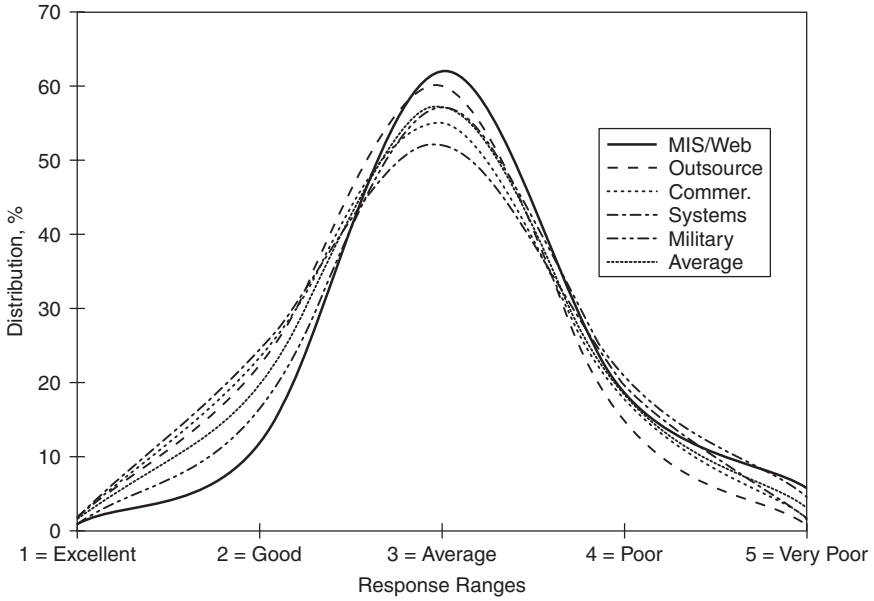

Figure 4-1 SPR software process assessment distribution of results

The inversion and compression of the SPR scores is not a perfect match to the SEI assessment distributions but is reasonably close.

A full and formal software assessment is a very good starting place for any kind of process improvement program. Because none of us can see our own faults clearly, a careful, independent assessment by trained specialists is the software world's equivalent of a complete medical examination. A software assessment should find everything we do wrong in building and maintaining software, and also everything we do right or better than average.

Software assessments normally take from a week to a maximum of about two months. The larger, longer assessments are those of multinational organizations with many different labs or locations that are in different cities or countries. A basic assessment of a single location should not require much more than a week to collect the data and an additional week to produce an assessment report.

However, an assessment by itself is not sufficient. The qualitative data collected during the assessment must be melded and correlated with "hard" productivity and quality data for optimum results. Therefore, a software assessment should not be a "stand-alone" event but should be carefully integrated with a baseline study, a benchmark study, or both.

Software Baselines

A software "baseline" analysis is the collection of hard, quantitative data on software productivity and quality levels. Both a baseline and an assessment are the normal starting points of long-range software process improvement programs. The fundamental topics of a software baseline include these elements in 2008:

- Basic productivity rates for all classes and types of software projects
- Comparative productivity rates by size of applications
- Comparative productivity rates by programming language
- Comparative productivity rates associated with various tools and methodologies
- Basic quality levels for all classes and types of software projects
- Comparative quality levels by size of software applications
- Schedule intervals for software of all classes and types of software projects
- Comparative schedules by size of applications
- Comparative schedules associated with various tools and methodologies
- Measurement of the total volume of software owned by your enterprise
- Measurement of the volumes of various kinds of software owned
- Measurement of software usability and user satisfaction levels

Unlike assessments, where external and independent assessment teams are preferred, software baseline data can be collected by an enterprise's own employees. However, some strong cautions are indicated, since there is no value in collecting erroneous data.

- Your software cost-tracking system (if any) probably "leaks," and therefore you should validate the data before using it for baseline purposes.
- You can validate your resource and cost data by the straightforward method of interviewing the participants in major software projects and asking them to reconstruct the missing elements, such as unpaid overtime or work performed before the tracking system was initialized for the project.
- If you use function points for normalizing your data, you will need to have one or more certified function point counters available to ensure that your counts are accurate. (Many function point courses are available in every part of the United States. The International Function

Point Users Group, or IFPUG, offers several certification examinations each year.)

- You cannot reconstruct missing quality data (although you can reconstruct missing cost and productivity data). Therefore, you may not be able to get really accurate quality data for perhaps a year after you start your baseline measurements. However, you can "model" your missing quality data using any of several commercial quality estimation tools.

Your initial baseline should probably contain information on somewhere between 10 and 30 software projects of various sizes and kinds. You will want to include all of the major kinds of software projects that are common, i.e., new work, enhancements, projects done by contractors, projects starting with packages, etc.

Expect it to take between a month and about six weeks to collect the initial baseline data if you do the work yourself.

Software Benchmarks

Software benchmark studies compare how two or more enterprises carry out the same activities, or compare their quantitative quality and productivity results. Sometimes benchmarks involve direct competitors, and they should always involve enterprises that are producing similar kinds of software.

Since benchmarks involve at least two separate enterprises, and sometimes many enterprises, the work of collecting the data is normally performed by an external, independent group. For example, many management consulting companies have benchmark practices. However, some companies have in-house benchmarking departments and executives.

As this third edition is written circa 2008, the International Software Benchmark Standards Group (ISBSG) is now the most widely used source of benchmark information for information technology projects. ISBSG has some data on systems and embedded software, some on commercial software, but not very much on military software. However for IT projects more than 4,000 data points are available circa 2008. The advantage of ISBSG is that the data is commercially available on CD and is also updated often. For commercial software, there are usually independent user associations that can sometimes perform benchmarks involving hundreds or even thousands of users of the same software. Some software journals and commercial benchmark companies also perform and publish software benchmark studies. For example, the Software Engineering Institute (SEI) has some significant benchmarks on the various levels of the CMM and CMMI. However, this data is only available to SEI affiliates or to SEI clients.

The most common forms of software benchmarks in 2008 include

- Benchmark comparisons of aggregate software expenditures
- Benchmark comparisons of total software portfolio sizes
- Benchmark comparisons of software staff compensation levels
- Benchmark comparisons of software productivity using function points
- Benchmark comparisons of software schedule intervals
- Benchmark comparisons of software quality
- Benchmark comparisons of user satisfaction
- Benchmark comparisons of development processes
- Benchmark comparisons of software specialization
- Benchmark comparisons of software assessment results, such as SEI's CMM/CMMI

The term "benchmark" is far older than the computing and software professions. It seemed to have its origin in carpentry as a mark of standard length on work benches. The term soon spread to other domains. Another early definition of benchmark was in surveying, where it indicated a metal plate inscribed with the exact longitude, latitude, and altitude of a particular point. Also from the surveying domain is the term "baseline," which originally defined a horizontal line measured with high precision to allow it to be used for triangulation of heights and distances.

When the computing industry began, the term "benchmark" was originally used to define various performance criteria for processor speeds, disk and tape drive speeds, printing speeds, and the like. This definition is still in use, and indeed a host of specialized benchmarks have been created in recent years for new kinds of devices such as CD ROM drives, multi-synch monitors, graphics accelerators, solid-state flash disks, and high-speed modems.

As a term for measuring the relative performance of organizations in the computing and software domains, the term "benchmark" was first applied to data centers in the 1960s. This was a time when computers were entering the mainstream of business operations and data centers were proliferating in number and growing in size and complexity. This usage is still common for judging the relative efficiencies of data center operations.

In the 1970s the term "benchmarking" also began to be applied to various aspects of software development. There are several flavors of software development benchmarking; they use different metrics and different methods and are aimed at different aspects of software as a business endeavor.

Cost and resource benchmarks are essentially similar to the classic data center benchmarking studies, only transferred to a software development organization. These studies collect data on the annual expenditures for personnel and equipment, number of software personnel employed, number of clients served, sizes of software portfolios, and other tangible aspects associated with software development and maintenance. The results are then compared against norms or averages from companies of similar sizes, companies within the same industry, or companies that have enough in common to make the comparisons interesting.

In very large enterprises with multiple locations, similar benchmarks are sometimes used for internal comparisons between sites or divisions. The large accounting companies and a number of management consulting companies can perform general cost and resource benchmarks.

Project-level productivity and quality benchmarks drop down below the level of entire organizations. These benchmark studies accumulate effort, schedule, staffing, cost, and quality data from a sample of software projects developed and/or maintained by the organization that commissioned the benchmark. Sometimes the sample is as large as 100 percent, but more often the sample is more limited. For example, some companies don't bother with projects below a certain minimum size, or they exclude projects that are being developed for internal use as opposed to projects that are going to be released to external clients.

Project-level productivity and quality benchmarks are sometimes performed using questionnaires or survey instruments that are mailed or distributed to participants. Such studies can also be performed via actual interviews and on-site data collection at the client site.

For example the author's company performs benchmarks by going on-site and interviewing project personnel using a standard questionnaire. The ISBSG organization also uses a standard questionnaire, but it is sent to the clients who fill it out and return the questionnaire. These two methods are both useful, but readers should understand the differences between them.

On-site benchmark data collection will usually collect information from about a dozen software projects. The consulting cost for such a study will range from about $25,000 to perhaps $50,000. Because interviews are effective in closing the common gaps in resource and cost data, on-site benchmarks are usually accurate to within about 5 percent, which is the current limit of precision of any form of software measurement.

Remote benchmarks using only questionnaires sent by email or regular mail usually collect information from one project at a time. The costs are only those for filling out the questionnaires, which usually takes between two and eight hours. However, since there are no trained consultants to deal with "leakage" from cost-tracking systems, unpaid

overtime, and other normal factors that reduce the accuracy of benchmarks, the remote benchmarks have higher margins of error than on-site benchmarks. Since most cost-tracking systems are only accurate to about 30 percent, it may be that the remote benchmarks are equally incorrect. However, there is also a chance that the projects selected for remote benchmarks were corrected and validated prior to submission. Unfortunately, there is no way of being sure.

In summary, on-site benchmarks are fairly expensive but also quite accurate. Remote benchmarks are very inexpensive, but of unknown accuracy.

What might be an interesting experiment for a university or nonprofit organization would be to perform a sample on-site benchmark after a remote benchmark has been submitted. The task would involve follow-up interviews of the project managers and team members to validate the accuracy of the original remote benchmark data.

To avoid "apples to oranges" comparisons, companies that perform project-level benchmark studies normally segment the data so that systems software, information systems, military software, scientific software, web software, and other kinds of software are compared against projects of the same type. Data is also segmented by application size to ensure that very small projects are not compared against huge systems. New projects and enhancement and maintenance projects are also segmented.

Activity-based benchmarks are even more detailed than the project-level benchmarks already discussed. Activity-based benchmarks drop down to the level of the specific kinds of work that must be performed in order to build a software application. For example, the 25 activities used by Software Productivity Research include requirements, prototyping, architecture, planning, initial design, detail design, design reviews, coding, reusable code acquisition, package acquisition, code inspections, independent verification and validation, configuration control, integration, user documentation, unit testing, function testing, integration testing, system testing, field testing, acceptance testing, independent testing, quality assurance, installation, and management.

Activity-based benchmarks are more difficult to perform than other kinds of benchmark studies, but the results are far more useful for process improvement, cost reduction, quality improvement, schedule improvement, or other kinds of improvement programs. The great advantage of activity-based benchmarks is that they reveal very important kinds of information that the less granular studies can't provide. For example, for many kinds of software projects the major cost drivers are associated with the production of paper documents (plans, specifications, user manuals) and with quality control. Both paperwork costs and defect removal costs are often more expensive than coding. Findings such

as this are helpful in planning improvement programs and calculating return on investment. But in order to know the major cost drivers within a specific company or enterprise, it is necessary to get down to the level of activity-based benchmark studies.

Software personnel and skill inventory benchmarks in the context of software are a fairly new arrival on the scene. As the 21st century unfolds, software has become one of the major factors in global business. Some large corporations have more than 25,000 software personnel of various kinds, and quite a few companies have more than 2,500. Over and above the large numbers of workers, the total complement of specific skills and occupation groups associated with software approaches 100 and new specialties such as "scrum masters" occur fairly often.

Large enterprises have many different categories of specialists in addition to their general software engineering populations, for example, quality assurance specialists, integration and testing specialists, human factor specialists, performance specialists, customer support specialists, network specialists, database administration specialists, technical communication specialists, maintenance specialists, estimating specialists, measurement specialists, function point counting specialists, and many others. There are important questions in the areas of how many specialists of various kinds are needed, how they should be recruited, trained, and perhaps certified in their area of specialization. There are also questions dealing with the best way of placing specialists within the overall software organization structures.

Benchmarking in the skill and human resource domain involves collecting information on how companies of various sizes in various industries deal with the increasing need for specialization in an era of downsizing and business process reengineering. A number of methodologies are used to gather the data for benchmark studies. These include questionnaires that are administered by mail or electronic mail, on-site interviews, or some combination of mailed questionnaires augmented by interviews.

Benchmarking studies can also be "open" or "blind" in terms of whether the participants know who else has provided data and information during the benchmark study. In a fully open study, the names of all participating organizations are known and the data they provide is also known. This kind of study is difficult to do between competitors and is normally performed only for internal benchmark studies of the divisions and locations within large corporations.

One of the common variations of an open study is a limited benchmark, often between only two companies. In a two-company benchmark, both participants sign fairly detailed nondisclosure agreements and then provide one another with very detailed information on methods, tools, quality levels, productivity levels, schedules, and the like. This kind of study

is seldom possible for direct competitors but is often used for companies that do similar kinds of software but operate in different industries, such as a telecommunications company sharing data with a computer manufacturing company.

In partly open benchmark studies, the names of the participating organizations are known, even though which company provided specific points of data is concealed. Partly open studies are often performed within specific industries such as insurance, banking, and telecommunications. In fact, studies of this kind are performed for a variety of purposes besides software topics. Some of the other uses of partly open studies include exploring salary and benefit plans, office space arrangements, and various aspects of human relations and employee morale.

In blind benchmark studies, none of the participants know the names of the other companies that participate. In extreme cases, the participants may not even know the industries from which the other companies were drawn. This level of precaution would be needed only if there were very few companies in an industry, or if the nature of the study demanded extraordinary security measures, or if the participants are fairly direct competitors.

Overall, the most accurate and useful kind of information is usually derived from benchmark studies that include at least a strong sample of on-site interviews and where the benchmark study is at least partly open. Because each of the different kinds of benchmark approaches can generate useful information, some companies use several different kinds concurrently. For example, very large corporations such as AT&T and IBM may very well commission data center benchmarks, cost and resource benchmarks, assessment benchmarks, and either project-level or activity-based productivity and quality benchmarks concurrently. They may also have special studies of security issues, of customer satisfaction, and of employee morale taking place as well.

Software benchmarking is continuing to expand in terms of the kinds of information collected and the number of companies that participate. Based on the ever-growing amount of solid data, it can be said that benchmarking is now a mainstream activity within the software world.

The objective of applied software measurement is insight. We don't measure software just to watch for trends; we look for ways to improve and get better. It should never be forgotten that the goal of measurement is useful information, not just data, and that the goal of information is improvement. A medical doctor would never prescribe medicines or therapies to patients without carrying out a diagnosis first. Indeed, any doctor who behaved so foolishly could not stay licensed. Software is not yet at this level of professionalism: Consultants and vendors prescribe their tools or methods without any diagnosis at all.

A good measurement program is a diagnostic study that can identify all software problems that need therapies. The therapies themselves can often be derived from the measurements. For example, if 20 projects are measured in a company and all 20 project teams report "crowded, noisy office conditions" as an environmental factor, it is a fair conclusion that improvements in office space are needed.

Measurement can take place at the strategic or corporate level or at the project or tactical level. Both are important, but the greatest insights are normally available by careful tactical measurement on a project-by-project basis. If such a study is carried out annually, it will allow a company to create a measured baseline of where it was at a given point in time and to use that baseline to judge its rate of improvement in the future.

Once baseline measurements begin, they normally become such valuable corporate assets that they quickly become standard annual events. This implies full-time staffing and adequate support, both of which are desirable. However, the very first baseline a company carries out is a special case. It differs from a normal annual software productivity survey in several key respects:

- The measurement team who collects the data is new and may be external management consultants.
- The concept of measurement is new, and the company is uncertain of what measurement is all about.
- The managers of the projects to be measured are probably apprehensive.
- There is no prior model to work from, in terms of what data to include or even what the report might look like.
- The timing and effort required for the baseline may be somewhat greater than in future baselines because of the need to climb up the learning curve.

Unless consultants are used, the first annual baseline will probably be staffed on a short-term basis by people who are fairly new to measurements and who are exploring metrics concepts. This also is a normal situation. An interesting phenomenon is that about 15 to 25 percent of the staff members who become involved with a company's initial baseline analysis become so intrigued with measurement technology that they enter new career paths and will wish to stay with measurement work permanently! That is a good sign for the software industry, since when metrics and measurements become serious enough for a career change, it indicates that a term like "software engineering" is becoming real instead of a painful misnomer.

Mechanics of a Baseline Study

The daily work of a physician involves diagnosis and treatment of patients. The daily work of a tactical measurement specialist involves diagnosis and treatment of projects. The project is the basic unit of study for tactical software measurement purposes. Although the term "project" is a common one, it is seldom defined. For the purposes of measurement, a *project* is defined as a set of coordinated tasks that are directed at producing and delivering a discrete software program, component, or system. The *project team* is assumed to consist of at least one manager and a varying number of technical staff members ranging from one to several hundred. The project team is also assumed to have a reasonably homogeneous set of methods and tools, which can encompass tools for requirements, design, coding, defect removal, and documentation.

Practical examples of what is meant by the term "project" include the following: an accounts payable system, an Ada compiler, a PBX switching system, a new release of a commercial software product such as Microsoft Excel 2007, or KnowledgePlan Releases 2 and 3, an order entry system, or a personnel benefits tracking system. All of these are "projects" in the measurement sense that the components and programs were part of a single overall scheme or architecture, the staff ultimately reported to a single executive, and a homogeneous tool set was employed.

Very large systems, such as IBM's MVS operating system, or Microsoft Vista, would normally consist of quite a few separate projects such as the scheduler component, the supervisor component, and the access methods. The practical reason for considering these technically related components to be separate projects is that they were developed in different cities under different managers who used different methods and tool sets. For the purposes of gaining insights, it is necessary to measure the tasks of each of the groups separately if for no other reason than because some were in California and others were in New York. Obviously, the final measures can be consolidated upward, but the data itself should be derived from the specific projects within the overall system.

An annual baseline measurement study is a well-defined and thorough diagnostic technique that can be administered either by professional consultants or by the measurement team that a company creates by using its own employees. To establish a baseline initially, a reasonable sample of projects is required. Normal annual baselines include all projects above a certain size that went into production in the preceding year. Thus, a company's 2008 baseline would include all of its 2007 delivered projects.

However, the very first annual baseline is usually somewhat of an experiment, and so it may not contain 100 percent of the prior year's projects. Indeed, the very first baseline may contain projects whose completions span many years. Yet, a reasonable sample is necessary to gain insights about software issues. Normally from 10 to 30 projects will constitute the volume of an initial baseline. The projects themselves

should be a mixture of new development, enhancements, and special projects (such as package acquisitions or contract projects) that reflect the kinds of software work the company actually carries out.

The baseline measurement study will examine all of the factors present in an enterprise that can affect the enterprise's software development, enhancement, and maintenance by as much as 1 percent. The baseline study can cover more than 200 soft environmental factors, and it is an attempt to bring the same level of discipline to bear on software as might be found in a thorough medical examination. As with medical examinations, a software baseline study is a diagnostic study aimed at finding problems or, in some cases, the lack of problems. After a careful diagnosis, it is possible to move toward a cure if problems are discovered. But before appropriate therapies can be prescribed, it is necessary to know the exact nature of the condition.

Why a Company Will Commission a Baseline

There are five common reasons why a company will commission a productivity analysis, and the measurement team should be very sensitive to them:

- The company is interested in starting a software measurement program, and it wants both reliable information on current productivity and assistance in selecting and starting a software improvement program of which measurement is a key component.
- The company wants to reduce the overall costs of software, data processing, and computing expenses within the enterprise and is looking for ways to economize.
- The company wants to shorten software development schedules and provide needed functions to users more quickly.
- The company has already achieved high software productivity, and it is seeking an independent validation of that fact, which will be credible with enterprise management and the users of data processing.
- Corporate management is beginning to think of outsourcing or offshore outsourcing, and the baseline is commissioned in order to aid in making a final decision as to whether outsourcing would be beneficial or not. (Sometimes the baseline is a defensive measure to avoid outsourcing.)

The Ethics of Measurement

Regardless of the company motivations for commissioning a baseline analysis, the staff involved in it has an ethical obligation to make the analysis accurate and valuable. There is no value, and there can be considerable harm, in incomplete or inaccurate information presented to higher management. Management has a fiduciary duty to run the enterprise capably.

The measurement team has an ethical duty to give the executives accurate data and clear explanations of the significance of the data.

It is important that both company management and the measurement team have a clear understanding of what results will come from a baseline study and what will not. What will result is a very clear picture of the software strengths and weaknesses of the enterprise that commissioned the study. For any weaknesses that are noted, possible therapies can be prescribed. What will not result is an instantaneous improvement in productivity. After the analysis is over, it can take from a month to a year to implement the therapies, and sometimes more than a year after that before tangible improvements can begin to show up. Very large corporations with more than 1,000 software professionals should extend their planning horizons out to three years or more.

It is unfortunate that Americans tend to like quick, simple solutions to difficult and complicated problems. Improving software productivity and quality in a large company is a difficult and complicated problem, and an annual baseline tactical analysis is a very important part of the entire process.

The Methodology of the Baseline Analysis

The exact methods and schedules for carrying out the baseline analysis depend upon the number of projects selected and on whether the analysis will be performed at a single location or carried out at multiple locations and cities. Defense contractors, and other kinds of companies as well, may start their baseline work with a self-assessment procedure developed by the Software Engineering Institute at Carnegie Mellon University and published by Watts Humphrey. This self-assessment procedure evaluates a number of technological factors and then places the company on a "software maturity grid." The results of the self-assessment are interesting and useful, but the SEI process is neither detailed enough to serve as the basis for an accurate diagnosis of deeper conditions, nor does it tend to suggest appropriate therapies.

The normal cycle for a full annual baseline analysis runs from two to three calendar months. The methodology for the full baseline described here is based on the concepts first used at IBM and ITT and subsequently adopted by other companies such as AT&T and Hewlett-Packard. The SPR methodology described here differs from the SEI self-assessment approach in a fashion that is analogous to a full medical examination vs. a self-checking procedure to be carried out at home. The SPR methodology is intended to be administered by professional consultants assisted by statistical analysis and multiple-regression techniques. The volume of information collected by using the SPR method may be more than an order of magnitude greater than the SEI method; it takes more time but leads to a fuller set of diagnoses and expanded therapies. Other results

were also published in the author's books *Programming Productivity* and *Assessment and Control of Software Risks*.

The very first time a full baseline is attempted, the additional startup work can stretch out the cycle to six months or more. Following are the general patterns.

Executive Sponsorship As pointed out in Chapter 1, the natural initial reaction to a baseline study is apprehension and fear, with the most severe alarms being felt by the managers whose projects will be measured. Therefore, an annual baseline study will probably require an executive sponsor at the vice presidential level, or even higher, the first time it is carried out.

Selection of the Measurement Team Set aside one or two months for the critical task of assembling a measurement team. No modern company would dream of turning its finances and corporate accounting over to well-intentioned amateurs without professional qualifications; yet many companies attempt to start measurement baseline programs with personnel whose only qualification is that they are between assignments and hence are available. That is not as it should be: The measurement manager selection should be based on capabilities, as should the selection of the rest of the measurement team.

The first time a company performs a baseline, the managers and staff will normally be somewhat inexperienced, but the situation is quite transient. Surprisingly, measurement technology is so interesting that perhaps 15 to 25 percent of the initial staff will decide to pursue measurement as a permanent career. As of 2008, no academic courses are available to software measurement specialists, but a new career is definitely emerging.

For the initial selection of a team, the manager should have a very good grounding in software methods and tools and sufficient knowledge of statistics to know how to apply statistics to practical problems. The measurement team should include at least one person with a good background in statistics, and all of the team should be experienced in software tools and methods.

Designing the Data Collection Instruments The first time a company starts to create an annual baseline, it is faced with the necessity of either acquiring or constructing questionnaires or survey forms. It is also faced with the necessity of acquiring or constructing software tools that can aid in the analysis and interpretation of the collected data. A minimum of about six months should be allotted if the company decides to construct its own instruments and tools. About two to three months should be allotted for selecting a consulting group if the company decides on the second option.

Some of the management consulting companies offering baseline services as part of their consulting practice include (in alphabetic order) A.D. Little; Computer Power; David Consulting Group, DMR; Gartner Group; Index Group; International Software Benchmark Standards Group (ISBSG); Quantitative Software Management (QSM), Roger Pressman Associates; Reifer Associates; Rubin Associates; and Software Productivity Research. Just as medical doctors do not diagnose patients by mail, a baseline study cannot be effectively carried out by means of mail surveys. Experienced, knowledgeable consultants must conduct face-to-face interviews with the project managers and team members to gain the really valuable insights that can occur.

It is not impossible to collect data remotely without on-site visits, but the results are not as accurate. Remote baselines or benchmarks are somewhat similar to a physician attempting to diagnose a patient over the telephone. It is not impossible, but not as accurate.

Nonprofit Associations with Function Point Baseline and Benchmark Data Now that the function point metric has become the dominant software metric throughout much of the world, a growing number of nonprofit measurement associations have come into existence that collect data from their members. A sample of these nonprofit organizations as of 2008 includes:

Country or Region	Acronym	Name
Australia	ASMA	Australian Software Metrics Association
Brazil	BFPUG	Brazilian Function Point Users Group
China	CSBSG	China Software Benchmarking Standards Group
Finland	FSMA	Finish Software Metrics Association
Quebec	CIM	Centre D'Interet Sur Les Metriques
Germany	DASMA	Deutschprachise Anwendergruppe für Software Metrik und Aufwandschutzung
Europe	EFPUG	European Association of Function Point Users Groups
France	FFPUG	French Function Point Users Group
Italy	GUFPI	Gruppo Utenti Function Point Italia
Netherlands	NEFPUG	Netherlands Function Point Users Group
New Zealand	SMANZ	Software Metrics Association of New Zealand
Switzerland	SwiSMA	Swiss Software Metrics Association
United Kingdom	UKSMA	UK Software Metrics Association
United Kingdom	COSMIC	Common Software Measurement International Consortium
United States	IFPUG	International Function Point Users Group
	NSC	National Software Council

Associated with the emerging National Software Council in the United States is the national software benchmark database sponsored by the U.S. Air Force. This database is somewhat primitive and does not

get to the level of activity-based costing, but it does support function point metrics in addition to the inadequate physical and logical "lines of code" metrics. In addition to these existing nonprofit associations, function point users are beginning to coalesce and form associations in many other countries such as Brazil, China, India, Japan, Russia, and South Korea.

"Vertical" associations of function point users are also beginning to form within specific industries such as banking, insurance, telecommunications, software, and healthcare. These special-interest groups usually get together at the general conferences held for software practitioners within the industries. There are also very active online function point metric discussions on the Internet, and also on various forums such as the Compuserve CASE forum, computer language forum, and cost estimating forums.

Although lagging somewhat owing to conservatism, the major software professional associations such as the IEEE Computer Society and the Data Processing Management Association (DPMA) are at least inviting speakers to discuss functional metrics at selected conferences. Even the very conservative Software Engineering Institute (SEI) has joined IFPUG and is finally awake to the power of functional metrics. In recent years, the SEI has become much more active in measurement and data collection than they were prior to the second edition of this book in 1996.

Introductory Session Prior to Commencing the Annual Baseline

The first public event of the baseline analysis begins with a one to three hour seminar on the rationale of the baseline and what is going to be accomplished. If available, it is helpful to show what other companies in the same line of business have been learning from their baselines or what is currently known about software productivity and quality in the industry as a whole. The purposes of the introductory session are to inform all staff and management of the intent of the baseline and to gain feedback about natural apprehensions and concerns. Although this task takes less than one day, at least a month of lead time is required to schedule the event. Also, if the session is to be given in more than one city or country (for multinationals) additional time will be required.

Preliminary Discussions and Project Selection

Plan on at least a week, and probably two weeks, for project selection. After the introductory session, the next step is to select from 10 to 30 specific projects to be examined, with 12 being the average number selected.

This is a delicate process in terms of corporate politics. Since the normal management reaction will be apprehension, there is sometimes a tendency to select only projects considered to be "successful." That is not as it should be. What is best is to select projects that are representative of what is really going on: good projects, bad projects, development projects, enhancement projects, completed projects, and unfinished projects are all candidates for measurement.

Scheduling the Project Interviews

Once the projects are selected, the first schedules will be set for the individual project analyses. A complete baseline analysis for 12 projects usually takes about 2 calendar months to complete, involves about 25 days of consulting time (16 of them on site at the project locations), and requires roughly 60 hours of client staff and management time to complete the questionnaires. Each interview session of a project team will range from about 2 hours to a maximum of about 4 hours. The participants will include the project manager and some or all of the project team, with a limit of perhaps 6 people in all. The limit of 6 people is not absolute, but it does represent the upper bounds of meetings that can be conversational without being discursive.

Individual Project Analysis

Each project selected by the company should be analyzed by using a standard technique so that the data is comparable. For example, the data collection instrument used by Software Productivity Research is a tactical project evaluation questionnaire that covers a total or more than 200 factors dealing with the projects themselves, the methods and tools used, the documents produced, the languages, the specialists and the organization structure, the defect prevention and removal techniques, and so forth.

Each project analysis requires from two to four hours. The participants for each analysis include the consultant, the manager of the project being analyzed, and several technical personnel: usually a design specialist, one or more programmers, and one or more technical writers if they were used for the project. If the project uses quality assurance, a quality assurance specialist should be part of the interview session. Thus, each analysis requires an investment by the project team of from 6 to perhaps 24 staff-hours.

All the participants in each analysis receive copies of the data collection questionnaire, which they are free to annotate during the analysis and to keep when the session is over. The master copy of the baseline questionnaire is kept with the actual project data by the consultant performing the analysis.

It is not uncommon for individuals on a project to have different opinions, and these ranges should be noted. Indeed, based on several hundred projects, it can be stated that managerial responses are normally about half a point more optimistic than technical staff responses when it comes to the usefulness of various methods and processes.

Although some apprehension is normal prior to the interview sessions, the apprehension vanishes almost immediately. Indeed, on the whole the managers and staff enjoy the interviews tremendously and for perfectly valid sociological reasons:

- They have never before had the opportunity to discuss their views of the tools, methods, and approaches in a serious and nonthreatening way, and they enjoy it.
- The interviews are a clear and definite sign that the company is starting to take software seriously. This is, of course, an encouraging fact for software professionals, who have long considered themselves to be outside the scope of their company strategy.
- The interviews provide a rare opportunity for people working on the same project to discuss what they are doing in a well-structured manner that includes, without exception, every facet that can influence the outcome of the project.

If the company is using automated tools for data collection and analysis, then before each analysis session is over, it may sometimes be possible to give the project manager a preliminary report that shows the responses for that project. Indeed, for unfinished projects it is possible to carry out a full cost estimate. However, if paper questionnaires are used, it is not convenient to provide immediate feedback.

Once the first wave of interviews and data collection takes place, a very common phenomenon is for other projects not initially selected for benchmarks to request to participate. This is because the team members from the early sessions are usually enthusiastic after the interviews are over.

Aggregation of Data and Statistical Analyses

Set aside at least ten working days for aggregation and analysis of the collected data. When all the selected projects have been individually analyzed, the combined data is aggregated and analyzed statistically. Hopefully, powerful statistical tools will be available to facilitate this aggregation. The factors that are significant include those in respect to which:

- All projects noted deficiencies or weaknesses.
- All projects noted successes or strengths.
- There were inconsistencies among projects, such as instances of some projects using formal inspections whereas similar projects did not.

Before production of the final reports, client management will be alerted to any significant findings that have emerged. The information can take the form of either an informal presentation or a draft report.

Preparation of the Baseline Presentation

There are two standard outputs from an annual baseline survey:

- A presentation to executives on the findings
- The full baseline report itself

The presentation to executives will normally be in the form of PowerPoint or some other graphics package , depending upon your corporate protocols. The presentation, which may run up to 100 screens (although 50 is more common) will discuss the background of the study, the weakness and strengths identified, the numerical hard data, and any possible therapies that appear to need immediate management attention. For preparation of the draft, about a week will normally be required. Production of slides and overheads may take up to one week depending upon the nature and speed of your production facilities.

For large companies, the time required to go through the presentation can range to more than half a day. The audience for the presentation will normally be senior executives, often including the CEO. The presentation will be given initially to the senior management of the company, but the normal protocols in business are such that it is highly advantageous to put on a road show and give the presentation to all company locations and managers who have any software responsibilities at all. Thus, in large corporations such as AT&T, Hewlett-Packard, ITT, and IBM, the annual baseline presentation may be given in more than 25 different cities.

Preparation of the Baseline Annual Report

After the full analysis of the data, the final results of the baseline are prepared under the supervision of the senior baseline measurement manager. The final report, typically 70 to 100 pages in length, contains the detailed results of the productivity analysis.

In public companies such as IBM and AT&T, the annual baseline reports are prepared on the same time scale as the corporate annual reports. Indeed, the annual software report tends to be about the same size and to more or less resemble in substance a true corporate annual report. The production of a true corporate annual report for a company such as IBM, however, is about a 3-month undertaking with costs that exceed $1 million for content, layout, and production. The cost and time of a software baseline is not in that league, of course, but about 2 to 3 weeks of effort to produce the draft, followed by 1 to 2 weeks of production is normal.

The published report will differ from the slide or screen presentation in these respects:

- It will have explanatory text that discusses the significance of the findings.
- It will normally have more detail than the presentation.

The final reports vary in content with the specifics of each company, but they always include the following items:

- An executive summary of significant diagnostic findings
- Specific problems diagnosed in the areas of management, organization, staff specialization, policies, physical environment, methodologies, tools, the production library, and the applications backlog
- Raw and normalized data on the projects and interpretation of the results
- Suggested therapies that the enterprise should explore to overcome the noted problems

Overall Scope and Timing of the Baseline

The normal scope of a baseline analysis generally runs from the initial meeting through the delivery of the final report. Although there are wide variances depending on the number of projects selected by the client, number of cities to be visited, and the like, a typical productivity analysis will take about 25 to 50 consultant days: 1 introductory day, 10 to 15 days of on-site data collection, 4 to 6 days of data analysis, 5 to 7 days of presentation creation, 4 to 7 days of written report preparation, 2 days of internal review among the measurement staff, 1/2 day with the senior company management for the preliminary report, 1 day for last-minute changes and corrections, and 1 full day with senior management to present the findings of the final report.

Follow-on Activities After the Baseline Is Complete

The baseline analysis will diagnose all software problems and all software strengths in an enterprise, with essentially no exceptions. As with a medical diagnosis, the diagnostic stage must be followed by a therapy stage in order to cure the condition. Exactly what follow-on activities will be needed varies from client to client. There is no single pattern that always emerges, just as medical therapy varies from patient to patient and illness to illness. There are, however, six general action items that often occur as a result of a productivity analysis.

Site visits A very frequent adjunct to a productivity analysis are site visits by client personnel to enterprises that have already achieved high levels of productivity. There are a number of leading-edge enterprises, already well ahead of U.S. averages in terms of software productivity, that allow or encourage visitors. Site visits give client enterprises an opportunity to see real companies in action, and they are very effective in showing the pragmatics of software productivity.

Tool acquisition Many enterprises are found to be deficient in software tooling, such as numbers of workstations and graphics and text design packages. A very common follow-on activity is the acquisition and installation of new tools. Tool acquisition is not inexpensive, and from $3,000 to more than $25,000 per software engineer may sometimes be committed.

Methodology selection Enterprises that lack technology exploration functions such as development centers or software research labs may be several years out of date in terms of methodology. A frequent result of a baseline analysis is the selection of and experimentation with such new methodologies as Agile or Extreme programming (XP), joint application design (JAD), high-speed prototyping, time-box prototyping, object-oriented languages, and inspections. Methodological changes do not require as heavy a capital investment as tool upgrades, but they do require time and educational commitments. Method upgrades can involve as much as several weeks of education time per software engineer and from $500 to $5,000 per staff member in training expenses.

Policy formation Changes in enterprise policies and cultures are the most difficult follow-on activities after a productivity analysis. For example, implementing a dual-compensation plan, installing opinion surveys, creating an open-door policy, and changing employment contracts are major topics that involve not only the software and data processing management but also the senior management of the enterprise up to the boards of directors, together with such other functions such as personnel and legal staffs.

Permanent measurement departments The baseline analysis itself will introduce metrics such as the function point technique and the McCabe Complexity Metrics to the enterprises that commission an analysis. The analysis will also introduce the soft and hard data collection principles and many other useful metrics.

A very frequent follow-on activity to the initial baseline is the establishment by the enterprise of a permanent software measurement program, which will continue to measure software development and maintenance productivity and create annual software state-of-the-art reports. This is a significant step, but it is not an inexpensive one.

A permanent measurement focal point requires at least one full-time expert, and the continuing costs of a measurement function can total several person-years each calendar year.

Software Quality Assurance (SQA) departments Since inadequate defect removal, lack of quality measures, and poor quality control are frequent findings, it often happens that one of the recommendations is to set up a software quality assurance function. Such organizations usually return much more value than the cost of creation, although not every company understands the economics of software quality.

Organizational changes Upgrading the organization structure of an enterprise is a relatively common follow-on activity. The establishment of formal maintenance departments occurs fairly often in large enterprises after a productivity analysis. Other organizational changes may be the creation of a formal testing organization, establishment of formal project offices, separating maintenance development, and so forth.

Identification of Problems Beyond the Current State of the Art

In the practice of medicine there are many diseases for which there is no current cure. In carrying out baseline analyses, there will be situations for which software engineering has no effective remedy. The following are examples of conditions for which no really effective therapies exist.

Irrational scheduling Unfortunately, in the United States more than half of the large projects are scheduled in an irrational manner: A predetermined end date is selected, and it is forced on the project by arbitrary decree. If the selected date exceeds staff capabilities by more than about 25 percent, the consequences are a potential disaster. The magnitude of the disaster can include attrition of staff, whose members may leave in disgust, catastrophic cost overruns, and, of course, no reasonable hope of making the irrational schedule.

The baseline analysis will point this situation out, but once an irrational schedule has been committed to a client or, even worse, published to the outside world, there is no easy solution. There are, of course, excellent commercial estimating tools available, but less than 15 percent of U.S. companies use such tools today. Once a schedule has been committed to a client, it is embarrassing to admit that it was arbitrary. In the long run, baseline measurements will provide accurate schedule information to prevent future occurrences. However, the first creation of a baseline will probably encounter projects with the distressing problem of having irrational schedules forced upon them.

Incompetence of management and staff As of 2008, software engineering and software management have no formal procedures for monitoring the equivalent of medical malpractice. A small but distressing number of projects will be found to be incompetently managed or, less often, developed. The situation seldom occurs in the context of baseline analyses for the pragmatic reason that companies with significant incompetence at the executive levels will probably not carry out baseline measurements at all. If the situation does occur, there is no easy therapy. The most visible manifestations of incompetence occur in litigation for cancelled projects or major software disasters such as the failure of the Denver Airport luggage handling systems. Depositions and court documents often reveal shocking failures on the part of project managers.

Aging non-COBOL production libraries For aging COBOL software, restructuring and renovation services and products can extend the useful lives of aging software by automatically restructuring, redocumenting, and isolating dead code. For other languages such as Assembler, FORTRAN, and PL/I, there were no commercially available automatic restructuring facilities in 2008. Clients with such software either have to redevelop it, replace it, or continue to live with it. Note that automatic restructuring is theoretically possible with other languages, but the companies that provide the service have chosen not to apply their methods to non-COBOL software. That may change in the future.

Layoffs or drastic reductions in force Enterprises that are faced with heavy competition, declining markets, or severe cash flow problems and have been forced to lay off substantial percentages of their work force may seek a baseline analysis in order to see if reduced staffing can continue to provide acceptable software functions and operations. Because so many of the powerful software technologies require upfront capital investment and substantial training expenses, clients with this situation should be warned that the set of available therapies not requiring some initial investment is neither large nor dramatically effective.

Large systems after the design phase From time to time, clients commission a baseline analysis because they are in the middle of a major effort (more than 5,000 function points or 500,000 source statements) that seems to be out of control and the schedules are stretching to unacceptable lengths. Unfortunately, if the key requirements and design phases are already past, there are no technologies that can dramatically reduce the rest of the cycle. New languages usually cannot be prescribed, since their performance may not be adequate for large systems. There are techniques, such as design and code inspections that can improve quality significantly and productivity slightly, but once the design phase has passed, options become few.

Context of the Baseline and Follow-on Activities

After the final report is presented and accepted by the client, the productivity analysis is normally complete. Whether or not the analysis leads to follow-on work depends upon the nature of the diagnosis, the kinds of therapies recommended, and the capabilities of the company organization.

A baseline analysis is often followed by changes in policy, methodologies, tools, staffing patterns, measurements, expectations, or any combination of the above. To demonstrate how a productivity analysis fits in context with the other activities, Figure 4-2 shows the normal sequences of events.

As can be seen, a baseline analysis is on the critical path to improving software productivity, but it must be followed by many other activities and events. Some of the activities, such as site visits and methodology selection, can occur fairly rapidly. Others, such as tool selection, require more extended analysis because capital investment of significant amounts may be required. The really long-term changes involve policies and organizational changes. It is seldom possible for a large company to introduce policy changes, such as dual salary plans, without many months of deliberation. Similarly, major organization changes, such as the creation of separate testing or maintenance groups, will also require substantial time for deliberation.

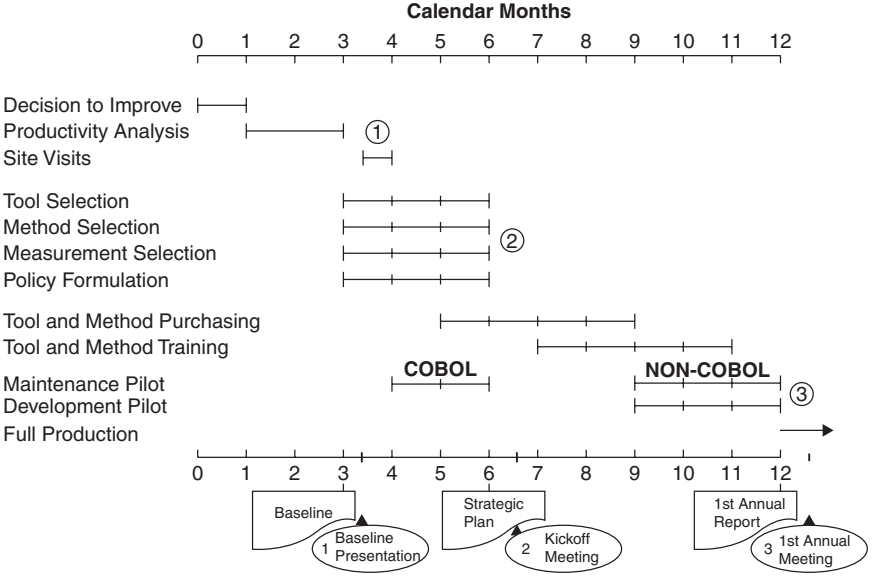

Figure 4-2 Chronological sequence of baseline analysis and follow-on improvements

What a Baseline Analysis Covers

A software baseline analysis explores essentially all of the soft and hard tactical factors that impact software development and maintenance: the project itself, the tools, the methodologies, morale and policy issues, languages, customer constraints, purchased software, training, and many other things. The following is an overview of the major topics and their significance.

Project factors First to be covered for each project studied are the attributes of the project itself. Whether the project is a new effort or an enhancement or a modification to an existing program is a key item, since productivity rates for enhancements are dramatically lower than for new development. The scope of the project also is important; notably different results will accrue depending upon whether the project is a prototype, a stand-alone program, a component of a large system, or a large system itself.

Also significant at the project level are any constraints against the project by users or enterprise management, i.e., schedule constraints, budgetary constraints, staffing constraints, and the like. Severely constrained projects are likely to run into difficulties, and are also difficult to analyze because of the large proportion of unpaid overtime that is typical on constrained projects.

In the United States, most software professionals are exempt from overtime payments, and yet they work 48 to 52 h/wk, which means that 8 to 12 h/wk (20 to 30 percent) of the total project effort can be in the form of unpaid overtime. Unless this factor is explored and understood, productivity measurements cannot be accurate or meaningful. The impact of unpaid overtime is also significant in doing productivity comparisons internationally. In the United States, 40-hour workweeks are the norm, but software professionals often work 48 to 52 hours. In Japan, 44-hour workweeks are the norm, but software professionals often work up to 60 hours. In Canada, 35- to 37-hour workweeks are the norm, but software professionals work 35 to 42 hours. In any case, unpaid overtime is a major variable and must be analyzed.

Management factors Achieving high productivity requires much more than just buying new tools and methods. The way employees are dealt with, the way morale issues are handled, and the way enterprises manage and organize software projects are very significant parameters. The software productivity analysis covers all of the critical managerial issues in depth. This portion of the baseline analysis may sometimes lead to the formulation of new policies and management practices.

Staff specialist factors More than 100 occupations and specialist types are now identified in software projects: software engineers, systems analysts, quality assurance specialists, maintenance specialists, database

administrators, and so on. The baseline analysis examines which specialist types are currently available within the enterprise and which additional specialists may be needed to meet the needs of future projects.

Physical environment factors The availability of sufficient workspace for software development and maintenance staffs and the kinds of equipment and workstations installed are explored fully. Although few enterprises recognize its significance, physical environment is one of the major determinants of software productivity.

Methodology factors The word "methodology" covers a broad spectrum of methods, tools, and procedures applied to software projects. The methodology portion of the productivity analysis covers all procedures used by the enterprise for software, including the way requirements are developed, the specification and design methods used, the way documentation is handled, and all other methodological factors.

Package acquisition and modification factors In many enterprises, a significant percentage of applications are purchased from outside vendors. The baseline analysis explores enterprise methods for evaluating, acquiring, and modifying software packages. Package acquisition is often a productivity enhancement factor, but it can sometimes be a productivity reduction factor as well.

Programming language factors With more than 700 languages available, most enterprises find it difficult to select a single language or a set of languages that is optimized for its needs. Part of the baseline analysis is to examine the enterprise's future projects and diagnose the most effective language choices. The most appropriate languages can range across all generations. It is not accurate to prescribe fourth-generation languages exclusively, since they are not appropriate for many program and system types.

Defect removal and quality assurance factors Eliminating bugs or defects is usually the most expensive single activity in the software world when all defect removal efforts are summed together. The baseline analysis also covers the methods and techniques used to find and remove errors: reviews, walk-throughs, inspections, all forms of testing, and proofs of correctness. For each project studied, the defect removal efficiency of the specific series of removal steps to be used is calculated. The overall effectiveness of enterprise defect removal is quantified as well. This portion of the productivity analysis often results in adoption of improved defect removal techniques.

Change control factors During development software projects add and change requirements at measured rates of between 1 and 3 percent each month. After deployment, software projects add new features at a rate

of about 7 percent per year, using the function point total of the application as the basis for judging changes. In spite of the fact that growing requirements and change are universal, many companies lack formal change control committees or even formal procedures for dealing with changes. A very frequent recommendation from baseline studies is to establish better and more formal change management procedures.

Maintenance and enhancement factors Since maintenance of legacy applications usually comprises more than 50 percent of the effort, it is normal to examine topics such as use of code restricting tools, complexity analysis tools, renovation, removal of error-prone modules, and other topics that are known to affect maintenance performance.

Measurement and normalization factors Most enterprises that commission a baseline analysis start with essentially no hard data at all on either productivity or quality. Therefore, a vital output from the analysis will be a solid benchmark of validated, hard productivity and quality data. The first baseline report will give most enterprises much better data than they have ever had or even knew was possible.

Standard design blueprints and reusable code The baseline analysis explores whether or not standard designs, termed "blueprints," and reusable code would be appropriate technologies for the projects and enterprises studied. Reusability is one of the most effective technologies yet developed, but it is not applicable to all enterprises and project types.

Developing or Acquiring a Baseline Data Collection Instrument

Most of us have had complete physical examinations. In the course of the examination, we normally fill out a medical history form that runs to perhaps ten pages and contains several hundred questions. A baseline measurement report is somewhat similar in logical content: It will be necessary to come to grips with several hundred factors, since software projects can be influenced by that many factors. This brings up an immediate need either to construct a data collection questionnaire or to acquire one from a commercial source. The questions used here to illustrate the concepts are taken from the proprietary CHECKPOINT questionnaire developed by the author for Software Productivity Research for carrying out large-scale baseline studies.

Regardless of its source, the data collection instrument must have these attributes: both hard data and soft data must be capable of statistical analysis. It is fairly straightforward to carry out statistical studies of the hard data, which is normally numeric in any case. The soft factors,

on the other hand, present a special problem. Bear in mind that the soft factors are usually subjective things about which individuals may give widely differing responses. It is not sufficient to just ask for opinions because the opinions cannot be analyzed statistically.

The approach that Software Productivity Research developed is to use multiple-choice questions built around a weighting scale, the software effectiveness level (SEL). The CHECKPOINT questions that evaluate skill, experience, novelty, and so on, are based on the following rationale: They use a scale from 1 to 5, with 3 representing the approximate U.S. average. Numbers less than 3 usually represent situations that are better than average; numbers greater than 3 usually represent situations that are hazardous or troublesome. The only exceptions to this rule are certain questions in which no risk or hazard at all is involved.

Note that the 1 to 5 scale used by SPR and the author runs in the opposite direction from the SEI 5-point scale. This is coincidental, and due to the fact that the SPR scale is more than one year older than the SEI scale, having originated in 1984.

Consider the following question about project novelty as it occurs in the CHECKPOINT questionnaire:

Project novelty?

1. Conversion or functional repeat of a well-known program
2. Functional repeat, but some new features
3. Even mixture of repeated and new features
4. Novel program but with some well-understood features
5. Novel program of a type never before attempted

It is easily seen that a 1 or 2 is less likely to be troublesome than a 4 or 5. The five possible answers somewhat resemble the Richter scale for earthquake magnitudes, in that as the numeric level of the answer goes up the potential for causing harmful situations also goes up.

The CHECKPOINT questions are so organized that 3 represents the approximate U.S. average for the parameter in question. Thus, for the above novelty question, an answer of 3, which means an even mixture of repeated and new features, is the approximate average for U.S. software. Since averages change over time, the SEL scale is recalibrated annually by Software Productivity Research.

To allow fine-tuning of client responses, the answers need not be integers, and up to two decimal places of precision can be used if desired. That is, answers such as 2.5 and 2.75 are perfectly valid and legitimate if the true situation falls between two points on the 5-point scale.

Because the questionnaire is so set up that 3 represents the approximate U.S. average for most questions, it is relatively easy to evaluate projects by using the 5-point scale:

- Projects averaging less than 1.5 are generally at the very extreme leading edge of modern software methods and practices. (*Note:* This would be equivalent to a rating of 5 on the SEI maturity level scale.)
- Projects averaging between 1.5 and 2.5 are generally well ahead of norms in terms of methods and practices.
- Projects averaging between 2.5 and 3.5 are generally in the normal range in terms of methods and practices.
- Projects averaging between 3.5 and 4.5 are generally deficient in terms of methods and practices, and significant therapies may be indicated. (*Note:* This would be equivalent to a rating of 1 on the SEI maturity level scale.)
- Projects averaging more than 4.5 are generally so poorly equipped and are so backward in technology that they have a very high probability of being absolute failures that should be canceled.

Software Productivity Research carries out baseline studies as a standard consulting engagement, and it is interesting to see the strengths and weaknesses that occur with high frequency in the United States. Table 4-2 shows the 12 most common weaknesses and Table 4-3 the 12 most common strengths that were noted between 2000 and 2008.

TABLE 4-2 Most Common Software Weaknesses between 2000 and 2008 in the United States

1. Schedules set before project is defined
2. Excessive schedule pressure
3. Major requirements change after the requirements phase
4. Inadequate project planning, tracking, measurement, and estimating methods
5. Inadequate pretest defect removal methods
6. Inadequate systems development methodology
7. Inadequate office space and poor environment
8. Inadequate training for management and staff
9. Inadequate support for reusable designs and code
10. Inadequate organizations and use of specialists
11. Too much emphasis on partial solutions such as the ISO 9000 standard or the SEI CMM
12. Attempting new technologies with inadequate training (Agile, 00 methods, client-server projects, etc.)

TABLE 4-3 Most Common Software Strengths between 2000 and 2008 in the United States

1. Staff experience in the application area
2. Staff morale and cohesiveness
3. Staff experience with programming languages
4. Staff experience with support tools
5. Staff experience with computer hardware
6. Availability of workstations or terminals
7. Availability of support tools
8. Use of adequate testing methods
9. Use of project library control methods
10. Use of structured code methods
11. Usage of formal assessments
12. Usage of geriatric tools for aging legacy software

A cursory examination of the more common U.S. software weaknesses and strengths leads to the following conclusion: The problems are centered around managerial, sociological, environmental, and organizational issues. The strengths are centered around tool and technical staff skills.

There is a second form of multiple-choice question that also is useful for baseline analyses. In this second case, the 5-point rating scale is used to discover the opinions of the respondents about whether something ranges from excellent to very poor.

Figure 4-3 shows this technique for allowing managers and staff to evaluate how well they perform various kinds of defect removal operations.

Administering the Data Collection Questionnaire

A baseline analysis session is a sometimes intense and always candid exploration of the particular methods and tools used on a project. The participants in the session include the project manager or supervisor, one or more technical people from the project, and the consultant who is handling the analysis. From time to time, additional people may attend: quality assurance personnel, guests from other projects, and the like.

Each of the participants should have his or her own blank copy of the questionnaire, which is usually passed out about a week before the session begins. It is important for participants to see the questions before

Business and Legal Reviews	Excl		Avg		Poor
1 Patent or legal reviews	1	2	3	4	5
2 Import/export license reviews	1	2	3	4	5
Nontesting Defect Removal					
1 Joint application development (JAD)	1	2	3	4	5
2 Prototyping	1	2	3	4	5
3 Requirements review	1	2	3	4	5
4 Functional design review	1	2	3	4	5
5 Logic design review	1	2	3	4	5
6 Data structure design review	1	2	3	4	5
7 Module design review	1	2	3	4	5
8 User documentation review	1	2	3	4	5
9 Maintenance documentation review	1	2	3	4	5
10 Code review and inspection	1	2	3	4	5
11 Quality assurance review	1	2	3	4	5
12 Correctness proofs	1	2	3	4	5
13 Independent verification and validation	1	2	3	4	5
Testing Defect Removal					
1 Unit testing	1	2	3	4	5
2 New function testing	1	2	3	4	5
3 Regression testing					
4 Integration testing	1	2	3	4	5
5 Stress or performance testing	1	2	3	4	5
6 System testing	1	2	3	4	5
7 Field testing	1	2	3	4	5
8 User acceptance testing	1	2	3	4	5
9 Independent testing organization	1	2	3	4	5
Postrelease Defect Removal					
1 Error-prone module analysis	1	2	3	4	5
2 Automated restructuring	1	2	3	4	5
3 User satisfaction survey	1	2	3	4	5
4 Maintenance defect repair	1	2	3	4	5

Figure 4-3 Use of a 5-Point scale for evaluating performance of multiple tasks

the session to reduce their natural apprehension. The participants may annotate their questionnaires if they wish. They can keep the questionnaires when the session is over, since the consultant administering the session keeps the master copy.

If, as in the case of the CHECKPOINT questionnaire, the questions are in automated form, it is technically possible to enter the responses immediately into a computer and produce a report on the spot.

Although desirable from one standpoint, the pragmatic result of having a computer in the room is not so good from a sociological standpoint. There is a tendency for the consultant who enters the data to get wrapped up in the mechanics of using the computer. There is also a certain unconscious intimidation attached to having the computer present, which may discourage free conversation. On the whole, it seems preferable to use paper questionnaires for the sessions themselves and produce computerized versions later.

Some of the questions on the baseline questionnaire are general and deal with global enterprise issues, but more than two-thirds of them are quite specific and deal with individual tactical project attributes. Consultants are cautioned not to try to handle multiple projects simultaneously in workshop mode. That has occasionally been done, but the results are unsatisfactory for these reasons: (1) The large number of people in the room leads to side conversations and distractions. (2) It is difficult to keep the focus on any single project. (3) It is easy to mix up responses and make mistakes in filling out the master copy of the questionnaires.

The time allotted to a productivity analysis session is typically 3.5 hours, although sessions are sometimes completed in 2 hours or less and they may sometimes run as long as 6 hours. If a single consultant is administering the sessions, then two sessions a day can usually be handled. The sessions are usually interesting to all concerned, and sometimes they are even enjoyable. They encourage both management and technical personnel to see that their problems and issues are being taken seriously, and the nature of the questionnaire and topics leads to very frank discussions of significant matters. Consultants and clients alike will learn a great deal from the productivity analysis sessions. The following are section-by-section discussions of a baseline questionnaire and the kinds of issues that are likely to come up during a productivity analysis session.

Recording the Basic Project Identity

The first items to be recorded are the basic identities of the project and the participants:

- Security level
- Name and location of the company
- Name of the project
- Names, phone numbers, and locations of all of the participants in the session

- Date of the session
- Start date of the project (if known)
- Current status of the project (i.e., completed, canceled, or partially completed)
- Comments or free-form information

A small but important point is very significant: security level. Unless the client gives specific written permission, the data collected during a baseline analysis is normally confidential and cannot be distributed to anyone. Indeed, a nondisclosure agreement should accompany such a study if external consultants are used. For military and defense projects, even more stringent security may be required. Be sure to note the security requirements for the project being analyzed in order to protect confidentiality. For military projects, the security level should be applied to each page.

If the questionnaire is acquired from a management consulting group such as Software Productivity Research, it may ask about the optional standard industry classification or SIC code for the enterprise. This can, of course, be omitted for single-company internal studies. The U.S. Department of Commerce has developed a coding scheme for all major industries and service companies as an aid in producing statistical reports. The SIC code is included for large-scale multiclient studies to identify the industry of the clients and to aid in concealing the identity of the specific companies that participated. The remainder of the information simply serves to give a context to the study: who was present, who performed the interviews, when the interviews took place, and so forth.

Project Natures, Scopes, Classes, and Types

An annual baseline for a large company will deal with a highly heterogeneous set of projects: new projects, enhancements, prototypes, programs, systems, MIS, military, batch, online, and so on, are likely to be present.

It is absolutely necessary to record the specifics of the individual projects included. Software Productivity Research has developed a taxonomy for dealing with these factors, which is shown in Figure 4-4.

- The "nature" parameter identifies the five major flavors of software projects that are common throughout industry and which tend to have different cost and productivity profiles.
- The "scope" parameter covers the range of possibilities from disposable prototypes through major systems.

Project Classification

Project Nature? _____
1 New program development
2 Enhancement (new functions added to existing software)
3 Maintenance (defect repair to existing software)
4 Conversion or adaptation (migration to new platform)

Project Scope? _____
1 Disposable prototype
2 Evolutionary prototype
3 Module or subelement of a program
4 Reusable module or macro
5 Complete stand-alone program
6 Program(s) within a system
7 Major system (multiple linked programs or components)
8 Release (current version of an evolving system)

Release Number _____

Project Class? _____
1 Personal program, for private use
2 Personal program, to be used by others
3 Academic program, developed in an academic environment
4 Internal program, for use at a single location
5 Internal program, for use at a multiple location
6 Internal program, developed by external contractor
7 Internal program, with functions used via time sharing
8 Internal program, using military specifications
9 External program, to be put in public domain
10 External program, leased to users
11 External program, bundled with hardware
12 External program, unbundled and marketed commercially
13 External program, developed under commercial contract
14 External program, developed under government contract
15 External program, developed under military contract

Project Type? _____
1 Nonprocedural (generated, query, spreadsheet)
2 Batch applications program
3 Interactive applications program
4 Batch database applications program
5 Interactive database applications program
6 Scientific or mathematical program
7 Systems or support program
8 Communications or telecommunications program
9 Process control program
10 Embedded or real-time program
11 Graphics, animation, or image-processing program
12 Robotics, or mechanical automation program
13 Artificial intelligence program
14 Hybrid project (multiple types)

For Hybrid Projects:

Primary Type? _____ Secondary Type? _____

Figure 4-4 Recording project nature, scope, class, and type

- Generally speaking, the "class" parameter is associated with the business aspects of a software project. Class determines the rigor and costs of project paperwork, and the volume of paperwork can be directly correlated to class number.
- The "type" parameter is significant in determining the difficulty and complexity of the code itself.

For statistical purposes, each of the 210 possible class-type combinations will have notably different productivity, quality, and skill "signatures" associated with them. Since the form of the questionnaire puts simple project attributes at the top and complicated attributes at the bottom, it is easy to predict (and easy to validate through measurement) that projects with low class-type numbers will be much more productive than those with high class-type numbers. And, indeed, projects of the class 1, type 1 form are the most productive in the United States and elsewhere. Projects of class 15 type 14 are the least productive in both the United States and elsewhere.

Fifteen classes are identified; they range from personal software through military contract software. Classes 1 and 2 are for personal software developed informally; class 3 is for software developed in an academic environment such as a university; classes 4 through 6 are for software developed by enterprises for their own internal use; and classes 9 through 15 are for software that will be delivered to external users, such as contract software, bundled software, and licensed software. The reason for recording class is that there are dramatic differences in productivity, quality, and technologies from class to class.

Consider the documentation, for example, that might be created to support a simple COBOL application program. If the program is developed for internal use (class 4) it might require 20 different supporting documents totaling perhaps 30 English words for every line of source code. If it were developed to be leased to external users, then it would probably require more than 50 supporting documents totaling almost 80 English words for every line of source code. If the program were developed for the U.S. Department of Defense under military specifications, it would require more than 100 supporting documents totaling almost 200 English words for every line of source code. In short, class is a significant parameter to consider when it comes to prescribing therapies for software problems.

The next question, project type, has 14 possibilities. The first 13 cover standard types such as batch and interactive applications and database programs. Type 14 is reserved for hybrid projects where more than a single type will be present in the same system. When this occurs, the primary and secondary types should be noted (and perhaps other types as well).

Project Goals and Constraints

When projects begin, management and staff will receive either explicit or implicit marching orders that they are asked to follow on the project. These goals and constraints should be recorded for each project included in a baseline analysis. There are many variations and flavors of goals and constraints, but the essential breakdown is simple:

- Most projects are directed to adhere to tight schedules, and this becomes the major goal and constraint. These projects have a dreadful tendency to become disasters.
- A few projects, such as mission-critical ones or those dealing with human life, are directed to achieve very high levels of quality and reliability.
- Some projects are in the middle ground, and they receive no goals or constraints at all.
- Some projects are back-burner types that are being done as fill-in between more significant projects.

Early on during the baseline analysis, the participants should discuss any goals or constraints levied against the project. This will normally generate lively discussions, and it will be the first sign that something useful is going to come out of the baseline measurement study.

Goals and constraints are critical productivity and quality issues, and consultants should be very sensitive to them. In real life, most projects start out handicapped by constraints established by client demands, managerial decrees, or some other extrinsic source. The most common, and also the most hazardous, constraints are those dealing with delivery schedules: More than 50 percent of all software projects in the United States have delivery dates established before the project requirements are fully defined!

The next most common set of constraints concerns staffing. Projects often have a locked staff and skill mix that cannot be extended but is clearly insufficient for the work at hand. A third set of constraints, to be dealt with later, concerns such technical constraints as performance, memory utilization, and disk space.

Staff Availability and Work-Habit Factors

One of the most important factors in coming to grips with productivity is that of establishing accurate corporate profiles of availability and work hours. Staff availability questions ask whether project personnel have worked full time on the project or divided their time among multiple projects. The normal assumption is that personnel will be assigned to a project 100 percent of the time, but sometimes this assumption is not correct.

If the answer is something other than 100 percent, the staff should estimate the percent of time assigned. It is also important to ascertain what percent of the project technical staff is exempt from overtime pay and will not generally be paid overtime. In the United States, most software professionals, except for junior, entry-level personnel, are exempt from overtime.

Your corporate accounting department can probably supply you with the average workday, workweek, and work year for your company. You must be very thorough in exploring how much effort was applied to the software project being analyzed. These are important questions from a measurement view, and also from an estimating standpoint. The normal U.S. workweek is 40 hours, yet software professionals frequently work in excess of 50-hour weeks. Thus, 20 to 30 percent of all effort applied to a project can be in the form of unpaid overtime, a very significant factor.

It is well known from time-and-motion studies that the productive time on software projects is much lower than the normal 40-hour accounting week. What is not so well known is that individual corporate cultures can make the productive week vary significantly. Therefore, capturing productive time is important. Table 4-4 shows a typical profile of work habits for knowledge workers in the United States.

Assuming that only 6 h/day is productive work and there are a practical 185 working days for actual tasks, the U.S. norm would seem to be 1,110 h/year applied to knowledge work. If the 24 days of meetings are included, then another 144 hours can be added to the total, bringing the total up to 1,254 hour of work per year. Now consider the work habits associated with critical software projects. Table 4-5 shows a software engineering work year on a schedule-driven project.

TABLE 4-4 Distribution of Staff Time During a Calendar Year

Time Use	Average Days
Normal working days	185
Weekend days (Saturdays and Sundays)	104
Meeting days	24
Vacation days	15
Public holidays	10
Slack days between tasks	7
Sick days	7
Business travel days	5
Education days	5
Conferences	3
Total	**365**

TABLE 4-5 Distribution of Software Staff Time During Critical Schedule-Driven Projects

Time Use	Average Days
Normal working days	197
Saturdays worked	40
Sundays worked	20
Nonworking weekends	44
Meeting days	24
Vacation days worked	10
Vacation days taken	5
Public holidays worked	6
Public holidays taken	4
Slack days between tasks	0
Sick days	0
Business travel days	5
Education days	0
Conferences	3
Total	**365**

The total number of project workdays, exclusive of meetings, is now up to 273 and includes many Saturdays, Sundays, some public holidays, and so forth. The average number of productive hours worked per day can also climb from 6 to 8, and the number of hours actually at work can climb from 8 to 10 or more. Thus, the probable number of work hours on a critical software project can total to 2,184, or to 2,376 if meetings are included. This number represents more than a 196 percent difference in the annual hours applied between ordinary knowledge work and critical software projects.

Obviously, this factor must be explored and studied in depth for software baselines to be accurate! Unfortunately, most corporate project-tracking systems do not record unpaid overtime. Therefore, one critical part of the baseline interview sessions is to get the software project managers and teams to reconstruct, from their memories, the real quantity of time they spent on their projects. It cannot be overemphasized that unless it captures such factors as unpaid overtime, tracking system data is essentially worthless for serious economic studies, and it is critical for accuracy to use the memories of the project team members.

Other related issues include use of contract personnel on the project and whether or not project personnel were dividing their time among several projects or were working on the project being studied in full-time mode. For contract personnel it is significant to ask what percent of

the project team was comprised of contract personnel. The number can range from 0 to 100 percent. This topic is significant for both productivity and quality purposes. Generally speaking, projects developed by contract personnel are often higher in productivity and are produced with shorter schedules than similar projects developed by full-time salaried employees. However, because of contract billing rates, the projects may cost more.

Inflation rates can be significant for long-term projects that may take a number of years to develop. For example, some large applications such as Oracle and SAP require more than five years for development. Obviously inflation will be significant over that span of years. For small projects that will typically take less than a year to develop, inflation rates can be omitted.

Determining the Size of the Project to Be Measured

Whether you use function points, feature points, lines of code, or something else, you will need to determine the size of an application if productivity studies are to be meaningful. There are no U.S. or world standards for counting source code, and there are very few enterprise or local standards. (Refer to Appendix A for the Software Productivity Research source-code-counting rules.) Therefore, when carrying out the first baseline analysis within a company, size information will be somewhat difficult to ascertain.

For example, on one of the first internal studies of software productivity carried out by the author at IBM in 1973, it was discovered that the major IBM divisions and labs were using six widely different conventions for counting source code size, which led to apparent differences of more than 300 percent in how large any given program or system would appear, depending on which rules were used. From that discovery, IBM standardized its source-code-counting rules and even developed an internal tool that automatically counted source lines in a consistent manner. Today, both function points and feature points provide size data independent of lines of source code. However, even though function and feature points are less variable than source code, there are still uncertainties in determining size.

More than half a dozen commercial tools have been marketed over the years, such as Battlemap, Pathview, Inspector, and ACT, which means analysis of complexity, and are capable of counting source code statements. There are also several tools that can aid in the calculation of function points and feature points, but as of 2008 there are no tools that can automatically backfire function points directly from source code even though this is not an excessively difficult accomplishment.

An emerging set of new tools may soon be able to predict function point and feature point totals as a by-product of design, but such methods are still experimental.

Even today, however, many and perhaps most companies have neither yet adopted functional metrics, nor have they as yet established counting rules, nor have they acquired a counting tool. Therefore, be very careful with project sizing when carrying out your first baseline assessment.

When ITT began its first baseline assessment in 1980, the ITT companies were reporting size in an astonishing variety of ways, including source lines with more than a dozen rule variations, object lines, bytes, and function points. The total range of variations, assuming one project had been counted under all possible rules, was more than an order of magnitude! It is easy to see why ITT established standard counting rules as an immediate by-product of the first baseline study.

If you use a commercial questionnaire such as CHECKPOINT, it will contain counting rules for source code, function points, and feature points. Size determination is a major topic for productivity analysis purposes, and especially so for large-scale studies involving different labs, divisions, or companies where it is quite unlikely that the projects will obey the same conventions. It is mathematically possible to convert source-code-counting rules from any set of counting rules into any other arbitrary set, but you must know the rules before you can do so. You can also convert size easily between source code and functional metrics by using the backfire method explained in Chapter 2. Table 4-6 illustrates some of the possible size variations based on how lines of code might be counted in PL/I for a typical business application.

It cannot be overemphasized that in the absence of local standards, any and all variations will be used. The variations are neither malicious, nor are they attempts to puff up sizes to make productivity look artificially high.

TABLE 4-6 Variations in Apparent Size of Software That Are Due to Alternative Methods for Counting Source Code

Method	Apparent Size in Source Lines
New executable lines only	100
New executable lines + data definitions	170
New executable lines, data definitions, comments	220
New + reused executable lines	250
New + reused executable lines + data definitions	430
New + reused executable lines + data definitions + commentary lines	500

It simply happens that size determination is too complicated an issue to be left to chance.

Although function points do not have the wide range of uncertainty that has always been associated with lines of code, even here there are many possible variations. For example, IFPUG function points and COSMIC function points can differ by more than 50 percent for some applications or come within 2 percent of being identical for other applications. So long as function point counting involves human judgment, there must necessarily be uncertainty. Table 4-7 shows the approximate ranges associated with the major flavors of function point counting. The data in Table 4-7 is based on a very small number of trials and experiments by the author as well as on secondhand reports, and must be regarded as premature for serious benchmarks. Additional projects and additional trials are needed. Note that Table 4-7 only shows about half of the function point variants in use circa 2008. A more extensive list in the author's previous book *Estimating Software Costs* noted 23 variations, and that list is not complete either.

Although it is hardly a valid mathematical procedure to aggregate and average the variations of unlike techniques, the average range of the entire set of methods in Table 4-7 is about 35 percent. Without multiplying examples, it can be seen that determining the size of projects for baseline purposes is not a trivial task. It requires both considerable

TABLE 4-7 Range of Variation Associated with Functional Metrics

Method	Automation Available	Range of Variation ± %
IBM 1979	No	50
DeMarco bang metric (1982)	Yes	50
Rubin variation (1983)	Yes	50
IBM 1984	Yes	20
SPR 1985	Yes	15
SPR backfire (1986)	Yes	35
SPR feature points (1986)	Yes	25
British Mark 11 (1988)	No	40
Herron approximation (1989)	Yes	35
IFPUG 1990 (Version 1-2)	Yes	20
IFPUG 1993 (Version 3)	Yes	20
IFPUG 1995 (Version 4)	Yes	25
IFPUG 1999 (Version 4.1)	Yes	05
IFPUG 2006 (Version 4.2)	Yes	05
Engineering function points (1995)	Yes	35
COSMIC function points (1998)	Yes	25
Jones "Quick Sizer" (2007)	Yes	15

effort and serious attempts at validation. (Note that this book assumes IFPUG function points, version 4.2.)

Also, over and above function point "clones," there are also metrics that use the idea of function points, but are called something else. A few examples include story points (often used with Agile projects), web object points, and use case points. These metrics produce results, but are not directly comparable to function points.

Standard Project Charts of Accounts

A chart of accounts is really nothing more than a convenient set of buckets into which cost and resource data can be placed. It is surprising that our industry has run for almost 50 years without any national or international standards, and very few corporate standards, on this topic. In the absence of a standard chart of accounts, many companies accumulate software project costs into a single cost bucket. Unfortunately, such an amorphous lump of data is worthless for serious economic studies because there is no way of validating it. The following sample illustrates the kinds of partial data that many companies have to utilize because they have no better method:

Project	Schedule	Effort	Staff	Cost
Billing system	18 months	2,000 hours	3	$60,000

Note the missing elements in the above example: How much time was devoted to any particular task? What was the breakdown of the 2,000-hour total? Were there any missing tasks? What were the three staff members doing? There is no way to validate such amorphous data. Even worse, there is no way to gain insight from such data, which is the main purpose of measurement. The absolute minimum for a chart of accounts to have any value at all is five tasks and a total. Table 4-8 illustrates a minimal chart of accounts and the data

TABLE 4-8 Minimal Chart of Accounts for Software Projects

	Schedule (months)	Effort (months)	Staff	Cost ($)
1. Requirements	1	1	1	5,000
2. Design	1	1	1	5,000
3. Code development	3	3	1	15,000
4. Integration and testing	2	2	1	10,000
5. Project management	–	2	1	10,000
Project total	**7**	**9**	**1.3**	**45,000**

elements to be recorded. Artificial data is inserted simply to give the chart the appearance of a completed one.

At least with a five-bucket chart of accounts, the tasks and the nature of the project begin to take on some semblance of reality. But five elements is by no means granular enough for real insight.

The CHECKPOINT standard chart of accounts, illustrated in Chapter 2, includes 25 development activities and a total, and it can be used for all current classes and types of software: civilian and military projects, large systems and small programs, new projects and enhancements, and so on.

That there are 25 tasks in a chart of accounts does not imply that every project performs all tasks. The set of 25 tasks is merely the smallest set that can be applied more or less universally. A good chart of accounts is revealing not only as a data collection tool but also as identifying tasks that should have been performed for a project but were not. For example, Table 4-9 illustrates the set of tasks performed for a PBX software project by a telecommunications company. It is based on interviews with the manager and staff.

Table 4-9 may appear to be reasonably complete and, indeed, useful, but it is highly significant that the following tasks that are useful for PBX software were not performed:

- Prototyping
- Project planning
- Reusable code acquisition

TABLE 4-9 Chart of Accounts for a PBX System Software Project

	Schedule (months)	Effort (months)	Staff	Cost ($)
1. Requirements	2	4	2.0	20,000
2. Initial design	2	6	3.0	30,000
3. Detail design	2	10	5.0	50,000
4. Code development	4	20	5.0	100,000
5. Unit testing	1	5	5.0	25,000
6. Function testing	2	10	5.0	50,000
7. Integration	2	2	1.0	10,000
8. System testing	3	15	5.0	75,000
9. User documentation	3	6	2.0	15,000
10. Installation	1	3	3.0	15,000
11. Project management	–	15	1.0	75,000
Total project	**22**	**96**	**4.4**	**480,000**

- Design reviews and inspections
- Code reviews and inspections
- Configuration control
- Quality assurance

The project in question was one that overran its budget by more than 40 percent and its schedule by more than 6 months. It quickly became obvious when looking at the tasks performed vs. what would be normal for such applications that the project had been in rush mode since it started, and the attempts to shortcut it by skipping quality control development practices led to disaster. Specifically, skimping on quality control and rushing to start coding paved the way to major delays later, when it was discovered during testing that the project had so many bugs that it could not work under normal operating conditions.

Note that exploration of factors that cause success or failure is possible when a granular chart of accounts is used, but it remains outside the scope of analysis when measures are less precise.

The most useful kind of chart of accounts is one which is derived from the full-work breakdown structure of the project being explored. In developing such a chart of accounts, it is significant to consider whether a single level of detail is desired or a multilevel "exploding" chart of accounts is to be used. In a single-level chart of accounts, all costs are accumulated against a small set of cost buckets. Single-level charts of accounts are easy to develop and are not burdensome to use, but they are somewhat coarse in terms of data precision. The following is an example of a single-level chart of accounts for six activities:

1. Requirements
2. Design
3. Coding
4. Documentation
5. Testing
6. Management

In multilevel charts of accounts, costs are accumulated against a variety of fairly granular "buckets" and then summarized upward to create higher levels of cost accumulation. Multilevel charts of accounts are the normal mode for accurate cost accounting, but they require more effort to establish and are somewhat more burdensome in day-to-day use.

The following is an example of a multilevel chart of accounts that expands on the preceding example:

1. Requirements
   1.1. Preliminary discussion
   1.2. Joint application design
   1.3. Requirements preparation
2. Design
   2.1. Initial functional design
   2.2. Initial data flow design
   2.3. Detailed functional design
   2.4. Detailed logic design
   2.5. Design reviews
3. Coding
   3.1. New code development
   3.2. Reusable code acquisition
   3.3. Desk checking
   3.4. Code inspections
4. Documentation
   4.1. Writing introduction
   4.2. Writing user's guide
   4.3. Writing operator's guide
   4.4. Writing maintenance manual
   4.5. Document editing
   4.6. Document printing
5. Testing
   5.1. Test planning
   5.2. Unit testing
   5.3. Function testing
   5.4. System testing
   5.5. Acceptance testing
6. Management
   6.1. Project planning and estimating
   6.2. Project milestone tracking

6.3. Project reviews

6.4. Project personnel management

There is no theoretical limit to the number of levels that can be used in a multilevel chart of accounts. In practice, more than 3 levels and more than about 200 subordinate tasks tend to become rather complicated. There are, of course, many commercial project management tools that can handle thousands of tasks, and large projects will normally utilize them.

It should be kept in mind that an appropriate level of granularity is one that provides meaningful information to project managers and the team itself. Data that is too coarse cannot lead to insights; data that is too granular will discourage both managers and staff alike because of the difficulty of recording the necessary information.

Identifying Occupation Groups and Staff Specialists

Now that software is becoming a full-fledged professional undertaking, many specialists are starting to become necessary for success. Just as medical practice has long been segmented into specialist areas, such as general practice, psychiatry, and neurosurgery, software is also starting to create special skill areas.

There are currently some 100 recognizable specialist skills in the area of software development and maintenance: application programmers, system programmers, database programmers, maintenance specialists, and so on. Small applications are usually done by individual programmers or software engineers; large applications not only require perhaps hundreds of workers but may also involve over dozens or even all 40 or so of the different occupation groups! Table 4-10 shows the numbers and kinds of the 10 most common software occupations typically engaged on business software applications of various sizes.

The significance of this section varies with the overall size of the enterprise being analyzed. As general rules of thumb, the following are the kinds of specialists usually found in various sizes of enterprises.

Small software staffs of fewer than 10 employees Small shops typically run on the generalist principle, with dual-purpose programmer-analysts being the most widely encountered occupation group. When specialization does occur, it usually consists of segmenting the analysis work from the programming work and sometimes segmenting applications programming from systems programming.

Medium software staffs with 10 to 100 employees When enterprises grow from small to medium, an increase in specialists is usually noted: Database programmers, sometimes network or communication programmers, and

TABLE 4-10 Software Occupation Groups Related to Software Size

Staffing Profile	Small Projects	Medium-size Projects	Large Projects
Project managers	1	2	10
Systems analysts	1	5	15
Application programmers	4	15	100
System programmers	1	10	15
Project librarians	1	1	3
Technical writers	1	1	5
Database administrators		1	3
Quality assurance		1	5
Test specialists		5	20
Scrum specialists	1	3	5
Performance specialists			2
Total staff	**10**	**45**	**183**

a definite split between system and application programming are frequent attributes.

Large software staffs with 100 to 1,000 employees Large enterprises enjoy the luxury of many specialists that smaller enterprises can seldom afford: professional writers for documentation, test specialists, maintenance specialists, tool and support specialists, and the like. From a productivity analysis point of view, large enterprises that do *not* have full specialization are candidates for staff structure improvements.

Very large software staffs with more than 1000 employees Very large enterprises usually have all 100 of the major specialist types and sometimes other types unique to their own businesses as well. Very large organizations that do not have maintenance specialists, test specialists, documentation specialists, and so on, are candidates for staffing upgrades.

Superlarge software staffs with more than 10,000 employees At the extreme end of the scale will be found enterprises such as AT&T, IBM, the Department of Defense, and a few other organizations with more than 10,000 software employees overall. The norm here is not only for full specialization but also for planned organizations built around the needs of the specialists. The adoption of full specialization and the supporting apparatus for them is one way to explain the phenomenon that productivity rates may be higher at the extremely large end than in the middle, in terms of staff sizes.

The hazards of generalists in specialist situations In all human situations, so far as can be determined, specialists tend to outperform generalists. It is certainly true that in knowledge work the advance from amateur

to professional status is marked by such an enormous increase in the knowledge content that one person can no longer absorb all of it, and so specialization is a necessary adjunct to the growth of a profession. Consider the number of medical and legal specialties, for example.

As for software, this phenomenon has only started to be explored and measured. Figure 4-5 gives the results to date. As can be seen, enterprise software productivity rates sag notably as the number of members on a professional staff begins to climb. In really large companies, however, the rate starts to climb again. One of the main reasons for that climb is that really large companies, such as IBM, Hewlett-Packard, and ITT, have gone beyond generalists and have moved toward full specialization and the organizations needed to support specialists.

Staff Skill and Personnel Variables

Psychologists who have studied software success patterns, such as Curtis, Schneiderman, and Weinberg, agree that experience is correlated with both productivity and quality. Breadth of experience is more important than length of time, incidentally.

Of all the variables known to affect software, "individual human variation" has the greatest known impact. In controlled studies, in which 8 programmers were asked to develop identical applications, the participants varied by more than 20 to 1 in productivity and by more than 10 to 1 in errors.

Yet this variable is difficult to capture and evaluate, and it is one of the most subjective of all soft factors. Not only is the factor subjective, but it must be measured carefully to avoid injustice to the people themselves. One of the chronic problems of personnel management is a phenomenon termed "appraisal skew," which means that managers tend to habitually appraise staff in accordance with their personal patterns

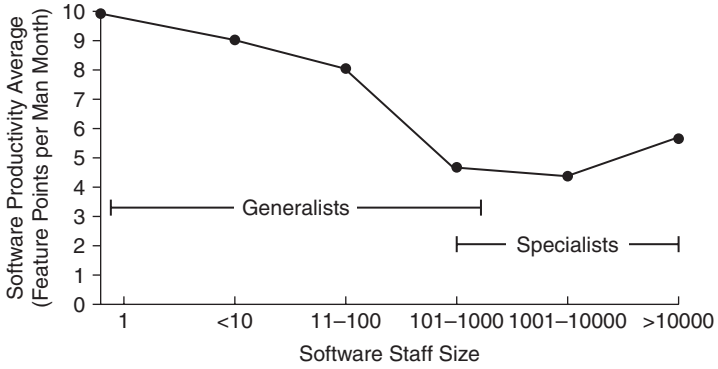

Figure 4-5 U.S. software productivity and total staff size

rather than objective facts. Some managers consistently appraise high; others consistently appraise low. (As a young programmer, one of the first serious programming tasks performed by the author was a project to measure appraisal skew of a large government agency.) Multinational companies must use extreme caution in measuring this parameter, since it is illegal in parts of Europe to record information about a worker's performance in a computerized database.

For the purposes of a baseline analysis, staff experience should be noted. There are eight major breakdowns of staff experience that are significant:

- Experience in the application area
- Experience with tools
- Experience with methods
- Experience with analysis and design
- Experience with languages
- Experience with reviews and inspections
- Experience with testing techniques
- Experience with the development hardware

Not only is staff experience significant, but, surprisingly, user experience and user cooperation provide two of the stronger positive correlations with productivity. There are three major user experience topics that should be recorded:

- User experience with software in general
- User experience with automation in his or her job area
- User experience as a participant in software projects

Management Project and Personnel Variables

Management topics are among the most sensitive and important sets of questions in the entire baseline analysis. The questions in this section should explore the fundamental policy and morale issues of the enterprise as well as the specific management environment for the project being analyzed. Most enterprises are highly political internally. Managerial power struggles, disputes, and dislikes can be very significant productivity parameters. Major projects are sometimes canceled or significantly thrown off track because key managers are fighting about goals, methods, or status.

From a productivity improvement standpoint, the problems and issues that the managerial section can highlight are very difficult to

treat or cure. Making changes in personnel and morale policies is outside the scope of authority of most software and data processing managers. Nonetheless, the managerial section is a critical one that will give both clients and consultants significant insights into enterprise cultural attributes.

Some of the topics to be explored in dealing with management issues include

- Management agreement or disagreement on project goals
- Management authority to acquire tools or change methods
- Management planning and estimating capabilities
- Management commitment to quality or schedule constraints
- The impact of corporate politics on the project

Of all of the factors that management can influence, and that management tends to be influenced by, schedule pressures stand out as the most significant. Some schedule pressure can actually benefit morale, but excessive or irrational schedules are probably the single most destructive influence in all of software. Not only do irrational schedules tend to kill the projects, but they cause extraordinarily high voluntary turnover among staff members. Even worse, the turnover tends to be greatest among the most capable personnel with the highest appraisals. Figure 4-6 illustrates the results of schedule pressure on staff morale.

It should be noted in conclusion that management has a much greater impact on both companies and projects than almost any other measured phenomenon. The leading-edge companies that know this, such as IBM, tend to go to extraordinary lengths to select managers carefully and to provide them with adequate training and support once they have been selected. Such companies are also careful to monitor manager performance and to find other work should a manager lack all of the attributes of successful management. Indeed, reverse appraisals in which staff members assess managers as well as management appraising staff members are utilized by leading-edge enterprises as part of their programs to ensure high management capabilities. A baseline study is not a direct tool for studying management capabilities, but those who have carried out baseline studies realize that it will point out strengths and weaknesses in this area.

Project Attribute Variables

The term "attribute" includes the impact of the physical office space for the software team and also the impacts of the workstations, tools, and methodologies in current use. Most of the attribute questions are reasonably self-explanatory, but the significance of the answers

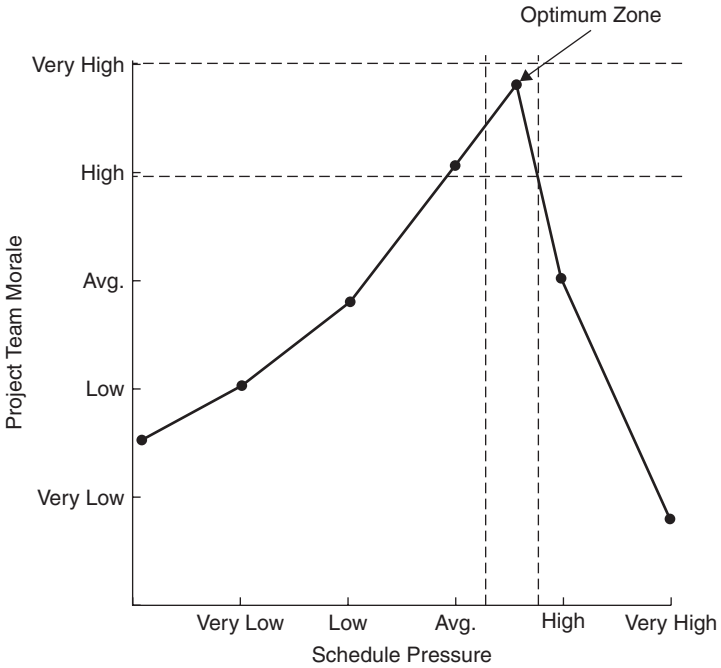

Figure 4-6 The impact of schedule pressure on staff morale

varies considerably. The most obvious kinds of attribute topics are concerned with the available tools and methods that the project team can use:

- Requirements methods
- Design methods and tools
- Formal or informal system development methodologies (Agile, RUP, etc.)
- Formal or informal standards (ISO, IEEE, etc.)
- Integration methods (discrete, continuous, etc.)

In studies such as those of Tom DeMarco and Tim Lister, some unexpected findings and conclusions regarding the physical environment itself have turned up. In a large-scale study involving more than 300 programmers, the DeMarco-Lister research noted the surprising finding that the programmers in the high quartile for productivity had office space that was approximately double the space available for programmers in the low quartile (more than 78 vs. 44 ft^2). In an older but even larger study by Gerald McCue that was commissioned by IBM and involved more than 1,000 programmers at the Santa Teresa Programming Center, the physical environment with all programmers having 10-by-10 ft private

offices resulted in productivity rates some 11 percent higher than the same personnel had achieved in their previous office buildings, which had 8-by-10-ft cubicles shared by two programmers.

These are significant and even poignant findings: Because of the rapid growth of numbers of computer professionals in most enterprises, lack of space is a national problem. Most companies allot scarcely more than 50 ft^2 of noisy space to their software professionals, with very unsatisfactory results. Unfortunately, physical space is one of the most expensive and difficult problems to solve. Poor available space is not quite an incurable problem, but it is among the most difficult problems in the industry.

Another aspect of environment is that of available computer resources and the impact of computer response time on development productivity. Thadani of IBM explored this, and he concluded that slow response times (>1 s) exerted a disproportionate loss in productivity because programmers tended to lose concentration and drift off for several additional seconds before recovering their train of thought. All physical, social, and technological aspects of the environment should be explored as part of the baseline analysis. If a commercial-grade questionnaire is used, such as the CHECKPOINT questionnaire, more than 50 environmental factors will be included.

From studying the environments of several hundred companies and government agencies in the United States, Canada, England, Australia, Europe, and Asia, quite a large number of chronic environmental and methodological problems have been noted. They appear to be endemic around the world.

- Requirements methodologies are deficient in more than 75 percent of all enterprises.
- Use of prototyping is starting to increase, but more than 85 percent of enterprises do not use prototype methods.
- Software design automation is almost totally lacking in more than 50 percent of all enterprises, and it is inadequate in more than 75 percent of all enterprises.
- Software documentation methods are inadequate in more than 85 percent of all enterprises.
- Office space and physical environment are inadequate in more than 80 percent of all enterprises.
- Management tools (estimating, tracking, and planning) are inadequate in more than 60 percent of all enterprises.
- Software measurements are inadequate in more than 90 percent of all enterprises.
- Software defect removal methods are inadequate in more than 80 percent of all enterprises.

Consultants and clients should keep in mind that a baseline analysis is a diagnostic technique that will find environmental problems but will not cure them. The cures will come from upgrading the environmental deficiencies.

Yet another aspect of the environment to be considered is the sociological aspect. In this case, topics to be explored are whether the organization reacts positively or negatively to change and how long it might take to carry out some of the following changes:

- Introduce a new methodology such as Agile, formal inspections, or JAD sessions
- Acquire new tools such as workstations
- Create a new department such as a quality assurance group, a measurement group, or a development center
- React to competitive situations

Surprising contradictions sometimes occur when sociological issues are explored. For example, in the early 1980s the stated goal of the ITT chairman was to improve software productivity rapidly. However, the executive charged with that task discovered that the purchase of a new tool required an average of eight approval signatures and more than six months of calendar time! Before it was possible to make rapid progress in software technologies, it was necessary to examine and streamline the purchasing process.

The topics of both bureaucratic friction and of change facilitation are normal parts of a baseline analysis. These topics are also supported by a substantial body of literature that deals not only with software but also with other areas of human endeavor. Since the natural human response to new ideas is to reject them until someone else has proven that they work, this topic is quite significant.

Contract Project Variables (Optional)

In a baseline analysis, contract projects should be included if the company or agency utilizes contract personnel in any significant quantity. The questions that might be asked about contract projects are optional, and they are used only when the baseline analysis is actually studying a project that is being developed by a contracting organization. Contract software generally encompasses five different contractual arrangements:

- Contracts with private individuals on a work-for-hire basis.
- Contracts with consulting organizations for staff and services on site at the client locations.
- Contracts with software houses or consulting organizations for custom packages: The work is performed at the contractor's location.

- Contracts with civilian government agencies at local, state, or national levels.
- Contracts with military services or the Department of Defense.

Each of these has different pros and cons, legal obligations, and the like.

Contract work can be surprisingly effective. A proprietary study of New England banking applications carried out by the author noted that contract projects averaged about twice the productivity, in function points per staff month, of projects the same size and class developed by employees of the banks. There were four reasons for this phenomenon: (1) The banks were more careful in defining requirements with contractors than with their own staff; (2) The contractors were fairly experienced in the applications; (3) The contractors had access to reusable code from similar applications; and (4) The contractors worked substantial amounts of overtime that was not billed back to the clients.

Since the second edition in 1996 and today's edition in 2008, offshore outsourcing to countries such as China, India, Russia, and so on, has increased more than 250 percent. Ten years ago when offshore outsourcing started to expand, the difference in costs between the U.S. and the outsource countries was about 6 to 1. That is to say, if a U.S. outsource contract was $10,000 per month the same work performed in India or China might only cost perhaps $1,700 per month.

However, success in outsourcing and other overseas economic progress has gradually raised offshore costs so that in 2008 the cost differences are now only about 1.5 to 1. The cost of living in Shanghai, Bombay, and Moscow in 2008 does not differ greatly from New York for professional work. If these trends continue, which seem likely, the cost savings from offshore outsourcing will essentially disappear by perhaps 2015. In fact as of 2008 several European companies have started outsourcing back to the United States due to the lower rate of inflation and the decline of the dollar against the Euro.

Software Package Acquisition (Optional)

Software packages and acquired software are definitely mainstream topics for a baseline analysis. When ITT carried out its initial baseline in the early 1980s, the team was surprised to discover that out of some 65,000 source code statements (equivalent to about 520,000 function points) some 26 percent of the total software owned by ITT had been acquired from vendors rather than developed by ITT personnel.

More recent studies of commercial off-the-shelf software (COTS) indicate that for large companies in the Fortune 500 set, COTS packages are more than 35 percent of total portfolios. For medium companies, COTS packages may top 50 percent. For small companies with less than 100

total personnel, COTS packages may actually account for 100 percent of the software in daily use, or close to that percentage.

Unfortunately the commercial software vendors such as Microsoft, SAP, Oracle, Symantec, and most of the others are pretty marginal with quality control. That means that purchasing software packages will necessarily involve finding and reporting bugs.

Also, many COTS packages may need customization in order to match corporate business needs. This brings up the problem that modifying COTS packages is often difficult and complex; and especially so if the vendors discourage modifications.

As a rule of thumb, if COTS packages require modification by more than 25 percent it may be less expensive in the long run to build the same kind of application. COTS economics are most favorable for 0 percent modification or, at any rate, for modifications that are supported and encouraged by the vendor.

The questions about packages are optional, and they are used only when the baseline analysis is studying a project that included or was based on a purchased package. In real life, purchasing packages is often a cost-effective way to get functions into production rapidly. Indeed, in terms of function points delivered per unit of effort, packages exceed all other forms in overall productivity. Depending upon the size and evaluation concerned, productivity rates in excess of 1000 function points per staff month are possible for packages. Cost per function point, on the other hand, may or may not be as favorable.

One major caution regarding packages should never be ignored: Package modification is a high-risk activity with a significant chance of failure and disaster. Not only is productivity normally low for modifying purchased packages, but if the vendors are reluctant to provide support, modification can easily be more expensive than custom development. As a rule of thumb, package modification is feasible only if the vendor is cooperative and supportive and if the package is well structured and well documented. Even then, changes that equal more than about 15 percent of the volume of the original software should be avoided. For older packages, poorly structured packages, and for those where vendor cooperation is minimal or even hostile, changes of more than 5 percent should be avoided. Indeed, searching for a second vendor or a more modern package should be considered.

Software Defect Removal Methods

Software defect removal methods were inadequate in about 80 percent of all enterprises in 1990 and that percentage has stayed more or less constant through 2007. A baseline analysis nearly always results in recommendations to upgrade defect removal methods via inspections, increased quality assurance funding, modernizing, testing, and the like. It is true

that some companies have adopted the CMM, Six-Sigma for software, or Watts Humphrey's TSP and PSP methods. Since all of these improve quality, it might be thought that quality control would be improving. But almost as many organizations have regressed or reduced their quality control activities. In fact in times of layoffs and downsizing, QA personnel are among the first to be released.

A baseline analysis is not the only method for actually measuring quality, since it can take more than a year to accumulate enough quality data to calibrate defect removal efficiency. (Chapter 5 discusses the techniques for quality and user satisfaction measurement.) However, baseline analysis can and should provide context and background information on these topics:

- Does the company really care about quality?
- Are defect prevention methods adequate?
- Are defect removal methods adequate?
- Is a formal quality assurance organization in existence?
- Is the quality assurance organization properly staffed and missioned?
- Does the company measure quality numerically?
- Are there any numeric quality targets for executives?

There are very significant differences in the kinds of defect removal activities practiced by companies on internal software, by vendors of commercial software, and by companies that work with the Department of Defense under military specifications. Defect removal for internal software is almost universally inadequate. Typically, enterprises use only perfunctory design reviews, no code inspections at all, no quality assurance, and a minimal test series that includes a unit test, function test, and customer acceptance test. The cumulative defect removal efficiency of the whole series seldom goes above 80 percent, which is why it takes a long time for most internal production software to stabilize: 20 percent of the bugs are still in it.

Defect removal for commercial software, at least that sold by major vendors such as IBM, is significantly more thorough than for internal software. Some vendors have a formal quality assurance review, design and code inspections, and in many cases, a separate testing department. The test series for commercial software starts with unit testing and includes regression testing, system testing, and field testing with early customers. Typical defect removal efficiencies for commercial software run from 90 to 96 percent. Unfortunately for other vendors such as Oracle and Microsoft, their major packages are so enormous that thousands of bugs or defects will still be present at delivery. In fact, these

large COTS packages above 100,000 function points in size usually are below 90 percent in terms of defect removal efficiency levels.

Defect removal for Department of Defense (DOD) software under military specifications is similar to that for commercial software, but it includes several activities unique to the DOD environment: independent validation and verification by an outside source, and independent testing by an outside source. Although DOD and military specification requirements add significantly to costs, there is no hard evidence that DOD software has either higher overall defect removal efficiencies or higher reliability than commercial software. Neither the DOD itself nor most contractors measure defect removal efficiency as a general rule, but the few that do seem to be about the same or slightly higher in efficiency than commercial houses: perhaps 85 to 95 percent cumulative efficiency. The baseline analysis methodology concentrates very heavily on defect prevention and removal methods for very pragmatic reasons:

- Defect removal is often the most expensive single software activity.
- It is not possible to make significant improvements in software productivity without first improving software quality.

An internal study within IBM noted the impact of quality on productivity for marketed commercial software products. The study revealed that the products with the highest quality after delivery had also achieved the highest development productivity. This study was surprising at the time, but when the economics of the situation were explored, the results were easily predictable. Projects that paid attention to quality did not get hung up during integration and testing, nor did they receive negative evaluations from the IBM quality assurance organizations. Such findings are in the public domain today, but in spite of that fact, very few companies really understand the linkage between quality and productivity.

Software Documentation Variables

There are 10 general classes of documentation that should be studied as part of a baseline analysis, and more than 100 different kinds of documents can be produced in support of software. Software is astonishingly paper-intensive: For software projects developed under military specifications and a DOD contract, paperwork will be the most expensive part of the project. DOD contract projects often generate more than 200 English words for every line of source code, and paperwork production can hit peaks of 60 percent of the entire project costs.

For commercial software produced by major vendors such as IBM, DEC, Hewlett-Packard, Raytheon, TRW, and Grumman, paperwork will usually be the second most expensive activity ranking just after defect removal. Here too, however, the costs are a significant proportion of

overall expenses. Commercial software will typically generate around 75 English words for every line of source code, and paperwork costs can amount to 40 percent of the total project.

For internal software and information systems, paperwork can be significant, but nothing nearly as dramatic as the two preceding cases. Typically, in-house information systems will generate about 30 English words for every line of source code, and perhaps 25 to 30 percent of the total project costs will be devoted to paperwork.

Although paperwork costs are always in the top three expense categories for software, and often are number one, most companies are totally unaware of the significance of this cost element. Baseline analyses carried out in computer companies and defense contractor environments almost always shock the managers and executives when they discover that 30 to 60 percent of their total software costs go to paperwork and less than 15 percent goes to coding. The 10 major classes of paperwork to be included in a baseline analysis include

1. Planning documents
   - Development plans
   - Business plans
2. Business and financial documents
   - Requests for proposals (RFPs)
   - Cost estimates
   - Budgets
   - Capital expenditure requests
3. Control documents
   - Milestone reports
   - Monthly progress reports
   - Budget variance reports
4. Technical documents
   - Requirements
   - Initial functional specifications
   - Final functional specifications
   - Module design specifications
5. Online documents
   - Help screens
   - Readme files
   - Hypertext links

6. User-oriented documents
   - Reference manuals
   - User's guides
   - Systems programmer's guides
   - Maintenance manuals
7. Tutorial documents
   - Introductions
   - Course materials
8. Marketing documents
   - Sales brochures
   - Advertisements
9. Quality and defect removal documents
   - Inspection reports
   - Quality assurance reports
   - Audit reports
   - Test reports
   - User bug reports
10. Correspondence and miscellaneous documents
    - Letters
    - Staff résumés
    - 35-mm slides
    - Overheads

One of the newer and very promising applications of the function point technique is the ability to predict and measure the volume of software paperwork: a task obviously beyond the capabilities of lines of code. Table 4-11 shows the use of function points for quantifying the volume of documents measured for a large commercial database product produced by a major computer company.

Maintenance and Enhancement Project Variables

The word "maintenance" is ambiguous in the software industry as of 2008, and it can mean either defect repairs or the addition of new functions and enhancements to a software product. Not only that, but for commercial software producers such as IBM, Microsoft, and Hewlett-Packard,

TABLE 4-11 Normalized Volumes of Paperwork for a Commercial Software Package

Documents	Pages per Function Point	Pages per Function Point	Words per Function Point
Planning	0.275	0.188	113
Business	0.138	0.000	38
Control	0.188	0.000	80
Technical	7.125	1.250	2,250
On-line	1.500	0.750	125
User-oriented	6.250	1.875	2,188
Tutorial	1.250	1.250	625
Marketing	0.063	0.123	63
Quality control	15.000	0.313	3,750
Correspondence	2.500	0.625	1,250
Totals	**34.300**	**6.375**	**10,450**

"maintenance" can include delivery support, field service, hotline support, and several other activities not usually found associated with internal software.

A baseline analysis will have quite a wide variance in scope for the maintenance area, depending on whether the client enterprise has commissioned the study for internal software, for commercial software, or for military software. In a baseline analysis, enhancements are generally treated as equivalent to new development and are studied as such, although their "hard" data should be broken out and shown separately because productivity rates for enhancements are quite different from new development. Specific maintenance questions should be included to cover defect repairs and the special considerations for commercial software, such as customer support and field service.

It is interesting that enhancement productivity follows a curve totally different from that for new projects. Enhancement productivity peaks, for example, when the size of the change is roughly 3 to 5 percent of the size of the system being modified. Smaller changes have high overhead costs associated with recompiling and regression testing, so productivity is low. Larger changes normally tend to damage the structure of the original system, so again productivity is low. Both enhancement and maintenance productivity are very sensitive to the structure of the base code being updated: Modifying well-structured code is more than twice as productive as modifying older unstructured software.

For maintenance or defect repairs of commercial software, totally new factors appear. Examples are "invalid" defect reports in which users report faults that are, in fact, caused by something other than the software itself, "duplicates" whereby several users report the same bug, and

"abeyances" whereby a problem found by a user cannot be replicated at the system repair center. The baseline analysis methodology should cover all of those aspects, but it varies significantly with the kind of software and client enterprise being analyzed.

A critical phenomenon often occurs when an industry approaches 50 years of age: It takes more workers to perform maintenance than it does to build new products. For example, by the time automotive production was 50 years of age, there were many more mechanics in the United States repairing automobiles than there were workers in Detroit and elsewhere building new cars.

Software will soon be 60 years of age, and we appear to be following a similar trend. Table 4-12 shows the approximate numbers of world programmers working on development, enhancements, and maintenance from 1950 to 2020. The data is partly empirical and partly extrapolation from client studies. The message of Table 4-12 is probably valid, but the reliability of the data itself is low.

Project Risk and Value Factors

A very important aspect of a baseline analysis is to try to come to grips with the value and also the risks of the projects that are included. Value and risks are opposite sides of a coin, so they should be studied simultaneously. The value factors to be explored include

- Tangible value (cost reduction primarily)
- Direct revenues (marketed software only)
- Indirect revenues (hardware drag along)
- Intangible value (enhanced capabilities, morale, etc.)
- Competitive value

TABLE 4-12 Population of Development, Enhancement, and Maintenance Programmers at Ten-year Intervals

Year	Programmers, New Projects	Programmers, Enhancement	Programmers, Repairs	Total
1950	90	3	7	100
1960	8,500	500	1,000	10,000
1970	65,000	15,000	20,000	100,000
1980	1,200,000	600,000	200,000	2,000,000
1990	3,000,000	3,000,000	1,000,000	7,000,000
2000	4,000,000	4,500,000	1,500,000	10,000,000
2010	5,000,000	7,000,000	2,000,000	14,000,000
2020	7,000,000	11,000,000	3,000,000	21,000,000

The risk factors to be explored include

- Risk of absolute failure or cancellation
- Risk of excessive schedule pressure
- Risk of unstable requirements
- Risk of poor quality
- Risk of inadequate staff or inadequate skills

Risk and value analyses are in rapid transition that should continue through the next decade. The early forms of value analysis dealt only with tangible cost savings associated with software projects. It was soon realized that the strategic value of software was often greater than that of any other aspect, but the measurement of strategic value is a complex task. The most recent approach to measuring the value of software is to use function points to explore consumption patterns of software once the software is deployed. Risk analysis too is evolving, and we should see substantial progress. The Department of Defense is extremely interested in it, and so it will continue to be researched.

Complexity and Source Code Factors

Every project studied in a baseline analysis should be evaluated for complexity in at least three different forms: (1) the complexity of the underlying problem and algorithms, (2) the complexity of the source code, and (3) the complexity of the data and data structures. Although the questions that might be asked about complexity are usually self-explanatory, the implications of the answers are not. Software complexity analysis is not an exact science. There are techniques, such as the McCabe cyclomatic complexity metric, with which source code structural complexity can be measured.

The McCabe technique yields very useful insights and provides a good quantification of practical software complexity. Small well-structured programs with straight-line logic will have a "cyclomatic complexity" of 1; that is, they have no branching logic. Empirical studies reveal that programs with cyclomatic complexities of less than 5 are generally considered simple and easy to understand. Cyclomatic complexities of 10 or less are considered not too difficult. When the cyclomatic complexity is greater than 20, the complexity is perceived as high. When the McCabe number exceeds 50, the software for practical purposes becomes untestable.

There are no measures at all of problem or algorithmic complexity, and empirical studies suggest that much of the observed complexity actually seen with source code is accidental or is caused by poorly trained programmers rather than by the problem itself. Data complexity is also generally

unmeasured, although counting the entity types referenced by the program or system holds significant promise. (Entities are persons or objects whose attributes comprise the data the program is manipulating.)

One other gap in complexity theory also is notable: All of the complexity research on software to date has been based on new programs. There are no pragmatic studies on complexity when updating an existing system, although empirical evidence reveals that updates have perhaps three times the error potential of new code for equal volumes and that errors correlate with structure. When the major forms of complexities that affect software projects are considered, there are at least 20 of them. As of 2008, only about half of them have been measured objectively and numerically; the rest still await exploration. The 20 varieties of complexity include the following:

- *Algorithmic complexity:* Deals with spatial complexity and algorithmic volumes. This form of complexity is one of the classic topics of software engineering. The basic concept is the length and structure of algorithms intended to solve various computable problems. Some algorithms are quite simple, such as one that finds the circumference of a circle, $C = pi^*$ diameter. Other problems, such as those involving random or nonlinear phenomena, may require extremely long algorithms. Problems with high complexity tend to be perceived as difficult by the humans engaged in trying to solve them. Examples of problems with high algorithmic complexity include radar tracking and target acquisition.
- *Computational complexity:* Deals with chronological complexity and run-time lengths. This form of complexity also is a classic topic of software engineering. The basic concern is the amount of computer time or the number of iterations required to solve a computational problem or execute an algorithm. Some forms of algorithms, such as those involving random or nonlinear phenomena, may require enormous amounts of computer time for solutions. Examples of problems with high computational complexity include long-range weather prediction and cryptographic analysis.
- *Informational complexity:* Deals with entities and relationships. This form of complexity has become significant with the rise of large database applications. The basic concern is with the kinds of entities about which information must be stored and with the relations among those entities. Examples of problems with high informational complexity include airline reservation systems, integrated manufacturing systems, and large inventory management systems.
- *Data complexity:* Deals with numbers of entity attributes and relationships. This form of complexity, similar in concept to informational

complexity, deals with the number of attributes that a single entity might have. For example, some of the attributes that might be used to describe a human being include sex, weight, height, date of birth, occupation, and marital status.

- *Structural complexity:* Deals with patterns and connections. This form of complexity deals with the overall nature of structures. Highly ordered structures, such as crystals, are often low in complexity. Other forms of structure include networks, lists, fluids, relational data structures, planetary orbits, and atomic structures.
- *Logical complexity:* Deals with combinations of AND/OR/NOR/NAND logic. This form of complexity deals with the kinds of logical operations that comprise syllogisms, statements, and assertions. It is much older than software engineering, but it has become relevant to software because there is a need for precise specification of software functions.
- *Combinatorial complexity:* Deals with permutations and combinations. This form of complexity deals with the numbers of subsets and sets that can be assembled out of component parts. In any large software project, there are usually many different programs and components that require integration to complete the tasks of the application.
- *Cyclomatic complexity:* Deals with nodes and edges of graphs. This form of complexity has been popularized by Tom McCabe. Its basic concern is with the graph formed by the control flow of an application. Unlike some of the other forms of complexity, this one can be quantified precisely. It is defined as the number of edges of a graph, minus the number of nodes, plus 2. Pragmatically, this is a significant metric for software, since modules or programs with high cyclomatic complexity are often difficult to maintain and tend to become error-prone.
- *Essential complexity:* Deals with nodes and edges of reduced graphs. This form of complexity is similar in concept to cyclomatic complexity, but it deals with a graph after the graph has been simplified by the removal of redundant paths. Essential complexity has been popularized by Tom McCabe.
- *Topologic complexity:* Deals with rotations and folding patterns. This form of complexity is explored widely by mathematicians but seldom by software engineers. It deals with the various forms of rotation and folding that are permissible in a basic structure. The idea is relevant to software, since it can be applied to one of the intractable problems of software engineering: attempting to find the optimal structure for a large system.
- *Harmonic complexity:* Deals with waveforms and Fourier transformations. This form of complexity is concerned with the various waveforms that together create an integrated wave pattern. The topic is

very important in physics and engineering, but it is only just being explored by software engineers.

- *Syntactic complexity:* Deals with grammatical structures of descriptions. This form of complexity deals with the structure and grammar of text passages. Although the field is more than 100 years old and is quite useful for software, it has seldom been utilized by software engineers. Its primary utility would be in looking at the observed complexity of specifications with a view to simplifying them for easier comprehension. It has a number of fairly precise quantifications, such as the FOG index and the Fleish index.
- *Semantic complexity:* Deals with ambiguities and definitions of terms. This form of complexity is often a companion to syntactic complexity. It deals with the definitions of terms and the meaning of words and phrases. Unlike syntactic complexity, it is rather amorphous in its results.
- *Mnemonic complexity:* Deals with factors affecting memorization. This form of complexity deals with the factors that cause topics to be easy or difficult to memorize. It is widely known that human memories have both a temporary and a permanent component. Memorization of text, verbal data, mathematics, and symbols normally involves temporary memory, which is quite limited in capacity. (A famous assertion holds that temporary memory can absorb only seven chunks of information at a time, plus or minus two.) Permanent memory is not fully understood, but it may involve some form of holographic distribution of engrams over the neural net of the brain. Visual data such as peoples' faces bypass temporary memory and are entered directly into permanent memory, which explains why it is easy to remember faces but not names.
- *Perceptional complexity:* Deals with surfaces and edges. This form of complexity deals with the visual appearance of artifacts and whether they appear complex or simple to the human perceiver. Regular patterns, for example, tend to appear simpler than random configurations with the same number of elements.
- *Flow complexity:* Deals with channels and fluid dynamics of processes. This form of complexity concerns fluid dynamics, and it is a major topic of physics, medicine, and hydrology. Since software systems also are dynamic and since information flow is a major aspect of design, this topic has a great relevance to software engineering. Within the past few years, a great deal of exciting research has taken place in the area of flow dynamics, and particularly in the area of turbulence. An entirely new sub-discipline of mathematical physics termed "chaos" has started to emerge, and it seems to have many interactions with software engineering.

- *Entropic complexity:* Deals with decay and disorder rates. All known systems have a tendency to move toward disorder over time, which is equivalent to saying that things decay. Software, it has been discovered, also decays with the passage of time even though it is not a physical system. Each time a change is made, the structure of a software system tends to degrade slightly. With the passage of enough time, the disorder accumulates sufficiently to make the system unmaintainable. This form of complexity is very significant in physics and astronomy and is starting to be explored in software engineering.
- *Functional complexity:* Deals with patterns and sequences of user tasks. When users of a software system call the system "complex," what exactly do they mean? This form of complexity concerns the user perception of the way functions within a software system are located, turned on, utilized for some purpose, modified if necessary, and turned off again.
- *Organizational complexity:* Deals with hierarchies and matrices of groups. This form of complexity deals not with a software project directly, but with the organizational structures of the staff who will design and develop it. It has been studied by management scientists and psychologists for more than 100 years, but only recently has it been discovered to be relevant to software projects. A surprising finding has been that large systems tend to be decomposed into components that match the organizational structures of the developing enterprise rather than components that match the needs of the software itself.
- *Diagnostic complexity:* Deals with factors affecting identification of malfunctions. When a medical doctor is diagnosing a patient, certain combinations of temperature, blood pressure, pulse rates, and other signs are the clues needed to diagnose specific illnesses. Similarly, when software malfunctions, certain combinations of symptoms can be used to identify the underlying cause. This form of complexity analysis is just starting to be significant for software projects.

Measuring the Hard Data for Software Projects

Each major task in a software project will produce some kind of a deliverable, and the volume of the deliverables should be measured as part of a baseline analysis. The normal kinds of deliverables for software projects can be divided into two sets: natural and synthetic. The natural deliverables of a software project are the tangible outputs of many tasks, and they include

- Pages of paper documents
- Source code
- Test cases

The synthetic deliverables of a software project are the volumes of abstract constructs, and they include

- Function points (in all variations)
- Feature points

Although the synthetic functional deliverables are superior to the natural deliverables for economic studies, it is also useful to record the volumes of natural deliverables as well. Table 4-13 shows a typical set of hard data measurements using natural deliverables.

There are several equations developed by the author for software measurement that can be used interchangeably with both natural and synthetic deliverables. The equations are concerned with relationships between assignment scopes and production rates for software deliverables. An *assignment scope* is the amount of a deliverable for which one person is normally responsible. A *production rate* is the amount of a deliverable which one person can normally produce in an hour, day, week, or month.

The equations have several interesting attributes: (1) They allow software projects to be measured with reasonably low error; (2) They work equally well with function points, pages of documentation, and lines of source code; (3) They can be used with any source language; (4) They can be used for estimating as well as measuring; (5) They can be used for individual activities, complete programs, large systems, or entire enterprises; (6) They establish regular and predictable relationships among the major attributes of software projects: staff sizes, human effort, schedules, and costs. There are three fundamental equations and three supplemental equations in the set. The fundamental equations are

- *Staff size = product size/assignment scope*
- *Effort = product size/production rate*
- *Schedule = effort/staff size*

TABLE 4-13 Measurement of Hard Data for Project Deliverables

Activity	Deliverable	Size	Assignment Scope	Production Rate per Month
Requirements	Pages	100	50	25
Design	Pages	250	150	25
Coding	Statements	25,000	7,500	1,250
User documents	Pages	300	200	50
Unit testing	Test cases	2,500	500	250
Function testing	Test cases	750	250	125

Although simple in concept and use, the fundamental equations are surprising. They allow any real software project to be matched with very low errors for the attributes of staff size, effort, and schedules.

The three supplemental equations are

- *Cost = (effort * salary) + other costs*
- *Assignment scope = product size/staff size*
- *Production rate = product size/effort*

It is surprising how useful equations as simple as these can be. Let us consider each one in turn.

The first equation solves the troublesome question of how large staff sizes are likely to be for any given project. To repeat, the assignment scope is the average quantity of work for which an average staff employee is responsible on any given project. For example, on small to medium-size software projects, an average programmer might be responsible for completing 5,000 lines of COBOL source code. Expressed in function points, the same assignment scope would be 47 function points.

Once an enterprise begins to think in these terms and to measure assignment scopes, a great many troublesome points can be resolved. For example, when dealing with a natural deliverable such as lines of source code, once an enterprise knows that its average coding assignment scope is 3,000, 4,000, or 5,000 lines (assignment scopes range up to 20,000 lines), then optimal staff sizes for new projects can easily be calculated. Assume an enterprise is planning a new 50,000-source-line COBOL system and its average coding assignment scope has been 5,000 lines. Dividing product size (50,000 lines) by assignment scope (5,000 lines) indicates that a staff of 10 programmers would be needed.

Assignment scopes can also be used with synthetic deliverables such as function points. For example, a typical maintenance assignment scope for a programmer who performs defect repairs and minor updates would be about 500 function points in size. In real life, assignment scopes vary in response to skill levels, source languages, product sizes, and environmental factors. But the assignment scope concept is worth understanding and using.

It is immediately apparent that the terms of the equation can be reversed. For example, to ascertain a typical coding assignment scope for historical projects that are completed, it is only necessary to divide product size (50,000 statements, for example) by the number of programmers (10, for example) to find the average assignment scope: 5,000 in this example.

The second fundamental equation solves the problem of how much human effort will be required for a software project. In this equation, product size is once again used (and either lines of source code or function

points will work). Product size is divided by production rate to find out how much human effort will be required. For example, in the 50,000 COBOL source-line system used in the previous example, a typical pure code production rate might be 2,000 lines of source code per month. Dividing product size (50,000 COBOL statements) by production rate (2,000 lines of source code per month) yields 250 person-months of effort to code the system.

Here too it is apparent that the terms of the equation can be reversed: To find the productivity rate for any completed project merely dividing product size by actual effort will give the productivity rate. The equation also works with function points or any other deliverable object such as documentation pages, test cases, or even bug reports.

Although the assignment scope parameter and the equations that use it are comparatively new to software engineering, the production rate factor has been in existence for many years.

The third fundamental equation solves the problem of development schedules. Effort (as calculated by the second equation) is divided by staff size (as calculated by the first equation) to yield the calendar time needed to develop the program. In the example already used, effort was calculated at 100 person-months and staff size was calculated at 10, so the schedule would be 10 calendar months.

The fourth equation, for cost, is not fundamental, but it is useful. Project cost is calculated by simply multiplying effort by average burdened salary. Assuming, for the example discussed thus far, that the average burdened salary rate per month is $5,000, the basic product cost can be calculated by multiplying the 100 months of effort by $5,000 to yield a basic product cost of $500,000.

Although this is a simplistic way to deal with costs, it is reasonably satisfactory. The remaining term of the fourth equation (*other costs*) is to be used for cost items outside the scope of normal burdened salaries—heavy capital equipment investments, moving and living costs, real estate, and the like.

Of the six equations, only the first one, which introduces the assignment scope variable, is comparatively new to software engineering, having been used for project estimating since about 1984.

However, since assignment scope allows the problem of staff size prediction to be handled relatively unambiguously, it is the keystone of the equation set.

Measuring Project, Activity, and Task Schedules

Among the most difficult and taxing forms of hard data measurement is that of project and task schedules. The initial difficulty with schedule measurement is a basic one: identifying the starting point of any given

project! Very seldom are projects started so crisply and precisely that a manager or user can assert "Project XYZ began on April 25, 2008." Usually there is a great deal of exploratory discussion and informal work, which can sometimes span a year or more, before an actual project finally coalesces. (Very often start dates are best determined by asking either the development team or the project manager for their personal views on the most appropriate starting point.)

Once an approximate start date for a project is identified, the next set of difficulties involves overlaps among tasks. The original waterfall model concept contained the naive assumption that a task would not begin until its predecessor ended, as shown in Figure 4-7. It quickly became evident to all who worked on real projects that the assumption of waiting until a task ended before the next task began was unrealistic. Given normal schedule pressures and software work habits, it is quite common to start the next task in sequence long before a predecessor task is over. Indeed, from analysis of several thousand projects since 1985 it can be asserted that the approximate average overlap is 25 percent; that is, about one-fourth of any given task will remain unfinished at the time the next task in sequence is begun. That average is often exceeded, and overlaps of 50 or even 100 percent (which implies true concurrency) are frequently encountered.

Figure 4-8 shows how the waterfall model tended to be implemented in real-life terms. Although overlap is simple and easy to understand in principle, it is quite difficult to measure in real life. The problems are not insoluble, however. If a formal project planning system is used, it will show not only the overlaps but also the actual calendar dates as well.

If formal project planning systems are not used, it is seldom possible to reconstruct the actual dates from memory. A useful approximation, however, is to ask the project managers and staff to express overlap in percentage terms; i.e., "coding overlapped design by 50 percent." Unless overlaps are measured and included in historical data, it will be very difficult to use such data for predicting future projects. The gross schedule

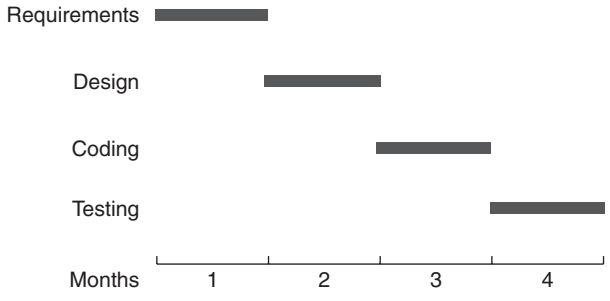

Figure 4-7 Original assumption of zero overlap with the waterfall model

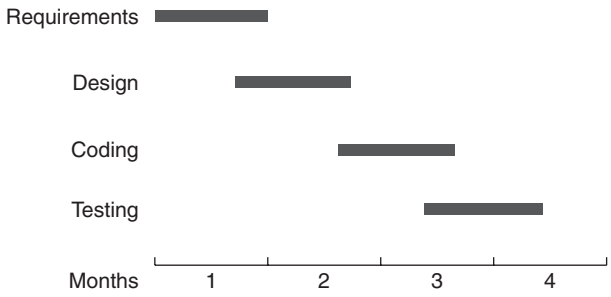

Figure 4-8 Revised assumption of 25 percent overlap with the waterfall model

from the beginning to the end of a project is insufficient for accurate estimation.

The newer Agile methods add to the complexity of scheduling. For Agile projects a number of small increments or "sprints" will be developed. Usually these sprints overlap, with the end of one sprint slightly overlapping the beginning of the next. There are also initial sprints, where the overall project is roughed out; and there are final sprints where all of the features are tested simultaneously.

Another topic that adds slightly to the complexity of scheduling is that of the method used by Extreme programming (XP) to develop the test cases before the code. This technique is effective for quality control, but since the test cases overlap coding somewhat, the scheduling needs to be carefully analyzed.

Reusable Design and Code Factors (Optional)

Reusable designs are less common than reusable code, although they are no less important. Reusable code is one of the most powerful productivity techniques available. Leading-edge companies such as Toshiba, Microsoft, Symantec, and IBM are able to develop new applications with up to 85 percent of the deliverable code coming from reusable sources. The topic of reusability should be included in all projects that actually make use of this technique—and in theory some 90 percent of all projects could.

One of the reasons why design and development personnel should be part of the baseline analysis sessions is that they have a much better idea about reused code availability than most managers. An interesting study by Barry Silverman of reused code at NASA found that managers thought that the reused code volumes in the projects studied totaled to less than 5 percent, whereas the programmers on the projects said that the volume approached 60 percent.

Whenever really high productivity rates occur (i.e., more than 5,000 source code statements or 50 function points per person-month) there is a very strong probability that at least half of the code in question was reused. Prescribing reusability as a therapy will occur in quite a large number of enterprises. Consultants should definitely consider reusability when doing productivity analyses within all large enterprises with more than 200 programmers and analysts. Certain industry types are especially good candidates for reusability: aerospace, banking, computers, electronics, communications, insurance, public utilities, and telephone operating companies, for example. These enterprises do large numbers of very similar applications, and that is exactly the domain where reusability is most effective.

There are many possible artifacts that can be reused, including:

- Reusable requirements
- Reusable designs and specifications
- Reusable test plans
- Reusable test cases
- Reusable database structures
- Reusable source code
- Reusable interface methods
- Reusable user documents
- Reusable project plans (with modifications)
- Reusable cost estimates (with modifications)

A strong caution about the relations between reuse and software quality is necessary. High quality reusable materials have the best return on investment of any known software technology. However, reusable materials of low quality that are filled with latent defects have about the worst negative return on investment of any known technology. Do not attempt large-scale reuse without state-of-the-art quality control approaches, or the results will be expensive and harmful to both customer satisfaction and business value.

Base Code Factors for Maintenance and Enhancement Projects

For projects that either enhance existing software or perform major defect repairs, an exploration of base code is quite important. For new programs and systems, the section should be omitted completely.

The long-range growth of software over time and also the growth in entropy or complexity can be modeled with high precision. Such

modeling can lead to decisions on what interventions or therapies may be needed to slow down or reverse the decay normally associated with aging software. The ten-year evolution of software projects was described by the author in the *Journal of Software Maintenance Practice and Experience*.

The costs, schedules, risks, and defect potentials of enhancing or modifying an existing system are dramatically different from those typical of entirely new software. Adding 1,000 COBOL source lines to an aging, poorly structured, and partially undocumented system can easily be three times as expensive and take more than twice as long as writing a new program of 1,000 COBOL statements. Moreover, because of the difficulty of updating aging software safely, the potential defects when updating old software can be three times higher than for new software, and the average defect removal efficiency is 5 to 10 percent lower for enhancement work than for new software.

It very often happens during a baseline analysis that clients want explicit advice and counsel on therapies that can improve aging software. Starting in 1980, a new software sub-industry began to emerge that by 2008 appears to be generating many millions of dollars in annual revenues. This industry consists of the companies that provide "geriatric care" for aging software—automated restructuring services or products and the even newer reverse engineering and reengineering products. By the end of the 2008, this sub-industry is likely to have 10 or 20 companies engaged with accumulated annual revenues exceeding $100 million.

The service offered by "geriatric care" companies include complexity analysis, code restructuring (automated), code refactoring (manual), data mining, error-prone module removal, reverse engineering, and even reengineering. Some "geriatric" companies such as Shoulders Corporation can even develop replacement versions of legacy applications, with much of the funding coming from reduced maintenance costs.

There are also companies offering specialized maintenance outsourcing, such as Computer Aid Inc., and some offering specialized renovation services such as Relativity Technologies.

The costs of restructuring range from perhaps 2 cents to more than $1 per source line depending on the volume of code, the service selected, and whether or not any manual intervention is required to perform such tasks as remodulization that are normally performed by the restructuring tools. Most of the companies in the restructuring business have some kind of free trial arrangement under which up to 5,000 source lines can be tried for nothing. If the baseline analysis turns up significant quantities of aging COBOL, restructuring is a recommended therapy. Unfortunately, many companies have aging libraries of programs in other languages that cannot be automatically restructured: Assembler, FORTRAN, Java, PL/I, PLS, RPG, APL, and the like.

Two newer technologies, reverse engineering and reengineering, have recently joined the restructuring concept in providing geriatric care. Reverse engineering uses expert system techniques to extract the hidden design elements from the source code itself, with a view to performing either manual or automatic conversion once the design (long lost from human view) is recovered. Reengineering is concerned with the automatic migration of aging software to new platforms, new languages, or more robust forms. Both reverse engineering and reengineering were so new when the second edition if this book was written that empirical evidence about them was scarce, although the technologies appear to be well formed and may be effective. New data over the past 10 years indicate that both reverse engineering and reengineering are valuable for renovating legacy applications.

The set of therapies for non-COBOL source code is both limited in effectiveness and expensive to apply. Pragmatically, the most effective therapy yet observed for other languages is the manual removal of error-prone modules.

Internal studies by IBM in the 1970s indicated that software errors are not smoothly or randomly distributed throughout the modules of large software systems. The errors tend to clump in a very small number of very buggy modules. In the case of one of IBM's commercial software products, the IMS Data Base, 57 percent of all errors were found in only 5 percent of the modules. More than 60 percent of the modules had no errors at all. Other companies studying software errors have made similar findings.

As of 2008, error-prone modules have been noted in more than 500 large applications and more than 100 corporations and government agencies. In fact, unless development is exceptionally good in terms of quality control, error-prone modules will occur in almost all large applications.

It is possible to analyze production software manually, isolate the error-prone modules, and either redevelop the modules or upgrade them substantially. That is not a particularly easy task, and it requires highly skilled professional programmers who may be occupied for very long periods of time—a year or more—while the work is going on.

Some other therapies in the maintenance and enhancement area are also worth noting. As of 2008, most companies with fewer than about 200 total software professionals usually handle maintenance on an informal basis whereby development personnel fix bugs as the need arises. Companies with more than 200 software personnel often have separate maintenance departments staffed by professionals who concentrate on defect repairs and small enhancements.

Maintenance specialists with full-time personnel are far more efficient and productive than having maintenance tasks performed informally by

developers. Interleaving development and maintenance tends to slow down both activities, makes estimating extremely difficult, and stretches out development schedules to excessive lengths.

Quite a few companies that have more than 200 professionals in 2008 only a few years before had 50 or fewer. Since organization structures typically change more slowly than staffs grow in size, it is often useful to consider the establishment of professional software maintenance units if the enterprise is of the right size and does substantial amounts of maintenance. These specialist groups are usually more productive than ad hoc maintenance by developers, and the separation of tasks allows both development and maintenance professionals to concentrate their skills.

Delta or Changed Code Factors for Maintenance and Enhancement Projects

The delta code examination (changed code) is used only to cover the structure and complexity of enhancements of or modifications to existing systems. The distinction between delta code and new code is minor but significant: The delta code will be added to the existing system as discrete blocks or modules, will require internal changes to the existing source code inside the current program, or will be both added and changed simultaneously. Productivity rates, risks, potential bad fixes, costs, and schedules are significantly worse when the delta changes require updating existing code than they are when the delta code is merely added on top of the original program.

When delta code is added to an existing system, the new code will either utilize or modify the existing data structures of the original program. Since modifying an existing data structure is sometimes very intricate, it is significant to note this factor if it occurs. The phrase "bad fix" refers to an error accidentally inserted into a program or system while trying to fix a previous error. This is a surprisingly common occurrence, and an internal study within IBM revealed that a full 20 percent of the customer-reported bugs against the MVS operating system were bad fixes.

Many other corporation also experience bad fixes. In fact, bad fixes are almost universal for software and have been noted in almost 100 percent of companies and projects where the topic is studied. Only a few very small and very simple applications are immune from bad fixes. Any application larger than 1,000 function points or with a cyclomatic complexity level higher than 10 will probably experience at least 5 percent bad fixes. As complexity rises, so do bad-fix injection rates.

The probability of making bad fixes is directly proportional to the structure of the original code: For cyclomatic complexity numbers of less than 10, the probability is about 1 bad fix per 20 changes (i.e., a 5 percent bad fix probability). For cyclomatic complexity numbers

of 20 to 30, the bad fix probability rises to about 20 percent, and for McCabe complexity numbers greater than 50, the bad fix probability is approximately 40 percent. In extreme cases, when the McCabe number approaches 100, bad fixes can hit 60 percent.

The therapies that minimize bad fix potentials include automatic restructuring (available for COBOL only), code inspections, and testing of all modifications before releasing the change. This last step, although intuitively obvious, is often not performed because of schedule pressures.

Descriptive Materials and Comments

The last set of topics of a baseline analysis is intended to explore specific information about the project that may require text and comments: the computers, operating systems, and tools used on the project, database packages if any, and the like. A section is also reserved for comments by the clients or consultants of any unusual aspects of the project being analyzed. Unusual aspects that can impact productivity and quality include, but are not limited to, the following:

- Transfer of the project from one city to another
- Physical movement of the project team from one location to another
- Significant expansion of requirements or project scope during mid-development
- Significant technical redirections or restarts
- Staff attrition rates higher than 20 percent per year
- Use of a new programming language or major new tool for the very first time
- Change in hardware or software environment after the project started
- Disasters, such as an earthquake, fire in a data center or office building, and so on

When the last comments are recorded, the analysis of the project is completed. The consultant administering the questionnaire should use the comments section for recording the start and stop times of a session, as a planning aid for future sessions. Finally, the consultant should close by asking the participants if there are any factors or issues that have not yet been covered.

Since a full baseline analysis will include from 5 to 20 projects, the consultant might tell the participants how many other projects are being analyzed and what the approximate schedules are for the aggregation and statistical analysis of the entire set.

Analysis and Aggregation of the Baseline Data

It is sufficient here to discuss the overall results from a human factors standpoint. The interview sessions themselves, and the kinds of information collected, are quickly perceived by all participants as benign and even helpful. This situation is natural and beneficial. Indeed, the results are favorable enough that spontaneous requests for additional interviews occur frequently.

The technology developed for carrying out software baseline analyses is thorough and accurate. Most enterprises have never before experienced such a detailed examination of their software policies, methods, tools, metrics, and environment. Although the practical results of a baseline analysis depend upon the specific diagnoses that are made, they often include the following improvements in client software environments:

- Establishment of a software metrics program and the adoption of leading-edge measurement techniques
- Enhanced credibility with users and enterprise management in terms of schedule and resource commitments
- Changes in the training and education made available to management and technical personnel
- Establishment of new policies dealing with morale and human relations issues
- Adoption of leading-edge software requirements and design techniques
- Adoption of leading-edge defect removal and quality assurance techniques
- Adoption of new technologies for restoring aging software systems
- Changes in the physical environment, the supporting tools, and the workstations available for software professionals
- Measurable and validated increases in productivity and quality simultaneously

The overall goal of the baseline analysis technology is to inform all participants of the exact strengths and weaknesses that were discovered during the diagnosis. Only from the basis of firm diagnostic knowledge is it possible to plan effective therapies.

The specific therapies prescribed can encompass tools, policy changes, methodological changes, organization changes, staffing level changes, or all of the above simultaneously. The baseline analysis will diagnose all known problems affecting software, but therapy selection requires skilled and knowledgeable consultants and clients.

Suggested Readings

Curtis, B. *Human Factors in Software Development*. IEEE catalog number EHO229-5, 2nd ed. Washington, D.C.: IEEE Press, 1986.
DeMarco, T. and T. Lister. *Peopleware*. New York: Dorset House, 1987.
Humphrey, Watts. *Measuring the Software Process*. Reading, MA: Addison-Wesley, 1989.
Jones, Capers. "A 10 Year Retrospective of Software Engineering within ITT." Burlington, MA: Software Productivity Research, Inc., 1989.
Jones, Capers. "Long-Range Enhancement Modeling." *Journal of Software Maintenance–Practice and Experience*, vol. 1, no. 2, 1990.
Jones, Capers. *Assessment and Control of Software Risks*. NJ: Prentice Hall, Englewood Cliffs, 1994.
Jones, Capers. *Programming Productivity*. New York: McGraw-Hill, 1986.
McCabe, T. "A Software Complexity Measure." *IEEE Transactions on Software Engineering*, vol. 2, December 1976: pp. 308–320.
McCue, G. "IBM's Santa Teresa Laboratory—Architectural Design for Program Development," *IBM Systems Journal*, vol. 17, no. 1, 1978. Reprinted in C. Jones. *Programming Productivity—Issues for the Eighties*. IEEE catalog number EH02394, 2nd ed. Washington, D.C.: IEEE Press, 1986.
Schneiderman, B. *Software Psychology—Human Factors in Computer and Information Systems.* Cambridge, MA: Winthrop, 1980.
Silverman, B. "Software Cost and Productivity Improvements: An Analogical View." *Computer*, May 1985: 86–96. Reprinted in C. Jones. *Programming Productivity—Issues for the Eighties*. IEEE catalog number EHO239-4, 2nd ed. Washington, D.C.: IEEE Press, 1986.
SPR, CHECKPOINT®. "Data Collection Questionnaire." Burlington, MA: Software Productivity Research, Inc., 1990.
Thadani, A. J. "Factors Affecting Programmer Productivity During Development," *IBM Systems Journal*, vol. 23, no. 1, 1984: pp. 19–35.
Weinberg, G. *The Psychology of Computer Programming*. New York: Reinhold, 1971.

Additional Readings

The literature on both corporate and software measurement and metrics is expanding rapidly. Following are a few samples of some of the more significant titles to illustrate the topics that are available.

Bakan, Joel. *The Corporation: The Pathological Pursuit of Profit and Power.* City: The Free Press, March 2004.
Boehm, Barry W. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.
Crosby, Philip B. *Quality is Free*. New York: New American Library, Mentor Books, 1979.
Ecora Corporation. Practical Guide to Sarbanes-Oxley IT Internal Controls. Portsmouth, NH: Ecora, 2005 (www.ecora.com).
Garmus, David and David Herron. *Function Point Analysis.* Boston: Addison Wesley Longman, 2001.
Garmus, David and David Herron. *Measuring the Software Process: A Practical Guide to Functional Measurement*. Englewood Cliffs, NJ: Prentice Hall, 1995.
Grady, Robert B. and Deborah L. Caswell. *Software Metrics: Establishing a Company Wide Program.* New Jersey: Prentice Hall, Inc., 1987.
Howard, Alan (ed.). *Software Metrics and Project Management Tools.* Phoenix, AZ: Applied Computer Research (ACR), 1997.
International Function Point Users Group. *IT Measurement*. Boston: Addison Wesley Longman, 2002.
Jones, Capers. *Estimating Software Costs.* 2nd ed. New York: McGraw-Hill, 2007.

Jones, Capers. "Sizing Up Software." *Scientific American,* Vol. 279, No. 6, December 1998: pp. 104–109.
Jones, Capers. *Software Assessments, Benchmarks, and Best Practices*. Boston: Addison Wesley Longman, 2000.
Jones, Capers. *Conflict and Litigation Between Software Clients and Developers*. Burlington, MA: Software Productivity Research, 2003.
Kan, Stephen H. *Metrics and Models in Software Quality Engineering.* 2nd Ed. Boston: Addison Wesley Longman, 2003.
Kaplan, Robert S. and David B. Norton. *The Balanced Scorecard*. Cambridge, MA: Harvard University Press, 1996.
Kaplan, Robert S. and David B. Norton. *Strategy Maps: Converting Intangible Assets into Tangible Outcomes.* Boston, MA: Harvard Business School Press, Boston, MA, 2004.
Pohlen, Terrance L. "Supply Chain Metrics." *International Journal of Logistics Management.* Vol. 2, No. 1; 2001: pp. 1–20.
Putnam, Lawrence H. *Measures for Excellence—Reliable Software On Time, Within Budget*. Englewood Cliffs, NJ: Yourdon Press—Prentice Hall, 1992.
Putnam, Lawrence H and Ware Myers. *Industrial Strength Software—Effective Management Using Measurement.* Los Alamitos, CA: IEEE Press, 1997.
Miller, Sharon E. and George T. Tucker. *Software Development Process Benchmarking.* New York: IEEE Communications Society, December 1991. (Reprinted from IEEE Global Telecommunications Conference, December 2–5, 1991).
Information Technology Infrastructure Library, www.wikipedia.org (accessed 2007).

Chapter

5

Measuring Software Quality and User Satisfaction

Two very important measurements of software quality that are critical to the industry are

- Defect potentials
- Defect removal efficiency

All software managers and quality assurance personnel should be familiar with these measurements, because they have the greatest impact on software quality and also software costs and schedules of any known measures.

The phrase "defect potentials" refers to the probable numbers of defects that will be found during the development of software applications. As of 2008 the defect potential of software includes five categories of defects:

- Requirements defects
- Design defects
- Coding defects
- Documentation defects
- Bad fixes (secondary defects accidentally included in repairs of prior defects)

As of 2008, the approximate U.S. averages for defects in these five categories have been measured in terms of defects per function point and rounded slightly so the cumulative results are an integer value. Note that defect potentials should be measured with function points and not with lines of code. This is because most of the serious defects

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

are not found in the code itself, but rather in requirements and design. Following are U.S. averages for defect potentials circa 2008:

Requirements defects	1.00
Design defects	1.25
Coding defects	1.75
Documentation defects	0.60
Bad fixes	0.40
Total	**5.00**

The measured range of defect potentials extends from just below 2.00 defects per function point to about 10.00 defects per function point. Defect potentials correlate with application size. As application sizes increase, defect potentials also rise.

The phrase "defect removal efficiency" refers to the percentage of the defect potentials that will be removed before the software application is delivered to its users or customers. As of 2008, the U.S. average for defect removal efficiency is about 85 percent.

If the average defect potential is 5.00 bugs or defects per function point and removal efficiency is 85 percent, then the total number of delivered defects will be about 0.75 defects per function point. However, some forms of defects are harder to find and remove than others. For example, requirements defects and bad fixes are much more difficult to find and eliminate than coding defects.

At a more granular level, the defect removal efficiency measured against each of the five defect categories is approximately as follows:

Defect Origin	Defect Potential	Removal Efficiency	Defects Remaining
Requirements defects	1.00	77%	0.23
Design defects	1.25	85%	0.19
Coding defects	1.75	95%	0.09
Documentation defects	0.60	80%	0.12
Bad fixes	0.40	60%	0.12
Total	**5.00**	**85%**	**0.75**

Note that the defects discussed in this section include all severity levels, ranging from severity 1 "show stoppers" to severity 4 "cosmetic errors." Obviously it is important to measure defect severity levels as well as recording numbers of defects.

There are large ranges in terms of both defect potentials and defect removal efficiency levels. The "best in class" organizations have defect potentials that are below 2.50 defects per function point coupled with defect removal efficiencies that top 95 percent across the board.

Defect removal efficiency levels peak at about 99.5 percent. In examining data from about 13,000 software projects over a period of 40 years, only two projects had zero defect reports in the first year after release. This is not to say that achieving a defect removal efficiency level of 100 percent is impossible, but it is certainly very rare.

Organizations with defect potentials higher than 7.00 per function point coupled with defect removal efficiency levels of 75 percent or less can be viewed as exhibiting professional malpractice. In other words, their defect prevention and defect removal methods are below acceptable levels for professional software organizations.

Most forms of testing average only about 30 percent to 35 percent in defect removal efficiency levels and seldom top 50 percent. Formal design and code inspections, on the other hand, often top 85 percent in defect removal efficiency and average about 65 percent.

As can be seen from the short discussions here, measuring defect potentials and defect removal efficiency provide the most effective known ways of evaluating various aspects of software quality control. In general, improving software quality requires two important kinds of process improvement: (1) defect prevention and (2) defect removal.

The phrase "defect prevention" refers to technologies and methodologies that can lower defect potentials or reduce the numbers of bugs that must be eliminated. Examples of defect prevention methods include joint application design (JAD), structured design, and also participation in formal inspections. (Formal design and code inspections are the most effective defect removal activities in the history of software and are also very good in terms of defect prevention. Once participants in inspections observe various kinds of defects in the materials being inspected, they tend to avoid those defects in their own work. All software projects larger than 1,000 function points should use formal design and code inspections.)

The phrase "defect removal" refers to methods that can either raise the efficiency levels of specific forms of testing, or raise the overall cumulative removal efficiency by adding additional kinds of review or test activity. Of course both approaches are possible at the same time.

In order to achieve a cumulative defect removal efficiency of 95 percent, it is necessary to use approximately the following sequence of at least eight defect removal activities:

- Design inspections
- Code inspections
- Unit testing
- New function testing
- Regression testing

- Performance testing
- System testting
- External beta testing

Since each testing stage will only be about 30 percent efficient, it is not feasible to achieve defect removal efficiency levels of 95 percent by means of testing alone. Formal inspection not only will remove most of the defects before testing begins but also raise the efficiency level of each testing stage. Inspections benefit testing because design inspections provide a more complete and accurate set of specification from which to construct test cases.

From an economic standpoint, the combination of formal inspections and formal testing is less expensive than testing by itself and will also yield shorter development schedules than testing alone. When testing starts after inspections, almost 85 percent of the defects will already be gone. Therefore, testing schedules will be shortened by more than 45 percent.

Measuring defect potentials and defect removal efficiency levels are among the easiest forms of software measurement and are also the most important. To measure defect potentials, it is necessary to keep accurate records of all defects found during the development cycle, which is something that should be done as a matter of course. The only difficulty is that "private" forms of defect removal such as unit testing will need to be done on a volunteer basis.

Measuring the numbers of defects found during reviews, inspections, and testing is also straightforward. To complete the calculations for defect removal efficiency, customer-reported defect reports submitted during a fixed time period are compared against the internal defects found by the development team. The normal time period for calculating defect removal efficiency is 90 days after release.

For example, if the development and testing teams found 900 defects before release, and customers reported 100 defects in the first three months of usage, it is apparent that the defect removal efficiency would be 90 percent.

Although measurements of defect potentials and defect removal efficiency levels should be carried out by *all* software organizations, unfortunately, circa 2008, these measurements are performed by only about 5 percent of U.S. companies. In fact, more than 50 percent of U.S. companies don't have any useful quality metrics at all. More than 80 percent of U.S. companies, including the great majority of commercial software vendors, have only marginal quality control and their defect removal efficiency levels are much less than the optimal 95 percent. This fact is one of the reasons why so many software projects fail completely or

experience massive cost and schedule overruns. Usually failing projects seem to be ahead of schedule until testing starts, at which point huge volumes of unanticipated defects stop progress almost completely.

As it happens, projects that average about 95 percent in cumulative defect removal efficiency levels tend to be optimal in several respects. They have the shortest development schedules, the lowest development costs, the highest levels of customer satisfaction, and the highest levels of team morale. This is why measures of defect potentials and defect removal efficiency levels are so important to the industry as a whole. These measures have the greatest impact on software performance of any known metrics.

From an economic standpoint, going from the U.S. average of 85 percent defect removal efficiency up to 95 percent actually saves money and shortens development schedules because most schedule delays and cost overruns are due to excessive defect volumes during testing. However, to rise above 95 percent defect removal efficiency up to 99 percent does require additional costs. It will be necessary to perform 100 percent inspections of every deliverable, and testing will require about 20 percent more test cases than normal.

New Quality Information Since the Earlier Editions

Since the first edition in 1991, the second edition in 1996, and this third edition in 2008, a number of quality-related topics have affected the software community. This is not surprising. New methods have been surfacing at approximately monthly intervals for more than 50 years. Some of these, such as structured design and the object-oriented methods, entered the mainstream. Others are more ephemeral and soon disappear. Among the more significant of the newer methods are

- The emergence of web applications
- The emergence of Agile methods
- The emergence of Six-Sigma for software
- The emergence of the Information Technology Infrastructure Library (ITIL)
- The emergence of service-oriented architecture (SOA)
- The creation of the International Software Benchmarking Standards Group (ISBSG)
- The formation of the Information Technology Metrics and Productivity Institute (ITMPI)

- The emergence of team software process (TSP) and personal software process (PSP)
- The emergence of the ISO 9000-9004 quality standards
- The emergence of data quality as a major topic
- The expansion of the Software Engineering Institute (SEI) maturity model
- The rise (and fall) of client-server software
- The rapid growth of the object-oriented (OO) paradigm
- The rise (and fall) of total quality management (TQM)

The Emergence of Web Applications

When the first edition of this book came out in 1991, the Web had only just begun. By the second edition in 1996, there were several hundred web sites and a few thousand web applications. By today, in 2008, there are more than a million web sites and more than 10,000,000 web applications. Web software has been the fastest growing segment of the computer industry, even in spite of the "dot com" failures at the end of the 20th century.

Web applications are typically fairly small and developed with some very sophisticated tools and perhaps 80 percent reusable material. The quality levels of web applications are not great, but the real issue is not so much web software as web content. Here we encounter a major problem: Because as this book is written in 2008, there are no effective metrics for quantifying the size of web content. Without size data, there is no effective way of quantifying web content quality.

Frequent web users have learned to work around problems such as data that does not display properly or links to incorrect web sites. Deeper and more complex problems such as out-of-date information or errors of fact are numerous, but difficult to avoid.

The Web and the Internet rank among the greatest inventions of human history. The Web has done more to facilitate knowledge transfer than any invention since the printing press, the telephone, radio, and television. In fact, as of 2008, the amount of technical information available on the Web in PDF or readable formats is probably as large as the total number of pages in all of the world's technical libraries. In terms of numbers of entries, the web-based *Wikipedia* encyclopedia is the largest reference source in the world (www.wikipedia.org) and is growing faster than any other reference source in human history. This is astonishing given the fact that every entry is written by unpaid volunteers, whose opinions on specific topics are more or less random. Future generations of sociologists and psychologists will have years of interesting studies ahead of them as they analyze the growth and origins of Wikipedia.

Technical libraries are organized using the Dewey decimal system or some other rational categorization scheme. As of 2008, the organization of data on the Web can best be described as chaotic and unstructured. There are some emerging attempts at improving this situation, including some very interesting metaphors that construct visual images of the Web as kind of enormous library with sections and shelves illustrated more or less as in a real library.

The Web is a powerful research tool, and the author visited more than 100 web sites in the course of this book. The usual starting point is a Google search with phrases of interest such as "Six-Sigma metrics" or "quality function deployment." Queries such as this open up paths to thousands of information sources.

The Emergence of Agile Methods

The second edition of this book was published in 1996. The famous "Agile Manifesto" was published in 1997. Therefore, the previous edition had no data on Agile projects because the various Agile methods had not been developed. This edition does contain a significant amount of new data on several forms of Agile including Extreme programming (XP).

In terms of quality, the Agile methods have been effective in reducing defect potentials for software applications below 1,000 function points. The reasons for this reduction can be attributed to the daily face-to-face meetings with users and to the daily "Scrum" sessions where developers meet to discuss issues and problems.

For large applications of 10,000 function points or larger, the Agile methods have not yet been used with enough frequency to have created much data. Anecdotal evidence is not unfavorable, but statistical studies have not been performed for major applications.

One of the weaknesses of the Agile approach is the widespread failure to measure projects using standard metrics such as function points. There are some Agile measures and estimating methods, but the metrics used such as "story points" have no available benchmarks. Lack of Agile measures include both productivity and quality. However, independent studies using function point metrics do indicate substantial productivity benefits from the Agile approach.

The Emergence of Six-Sigma for Software

The famous Six-Sigma approach was first applied to hardware components. The term "Six-Sigma" stems from mathematics and means one error in 3.4 million opportunities. For software, that level of quality is unachievable. It would be roughly equivalent to having a defect removal efficiency level of 99.999999 percent, which essentially never occurs. Or alternatively, Six-Sigma for software would imply defect potentials of

about 0.00005 defects per function point rather than current U.S. average of about 5.0 defects per function point. Here, too, such low volumes of delivered defects essentially never occur for software.

However, just because achieving actual Six-Sigma defect levels is beyond the state of the art for software, that does not mean the methodology has no value. In fact, the use of Six-Sigma principles for software does lower defect potentials by as much as 50 percent sometimes. By studying and measuring defect removal operations, the Six-Sigma approach can also raise defect removal efficiency levels.

Some of the Six-Sigma metrics are equivalent to the metrics in this book, although they go by different names. For example the term "total containment effectiveness" (TCE) used with software Six-Sigma is equivalent to "defect removal efficiency" and is calculated in the same way. The term "defect removal efficiency" originated in IBM in the late 1960s, whereas the term "total containment effectiveness" did not occur until about 2000; however, the underlying calculations are the same.

The Emergence of the Information Technology Infrastructure Library (ITIL)

The well-known Information Technology Infrastructure Library is a large set of some 30 books and manuals that discuss all aspects of software change control, software defect repairs, and software "Help desks" and maintenance tasks.

The virtue of the ITIL approach is that it suggests targets and metrics for key issues such as mean time to failure, recovery time, and elapsed times from reporting defects until repairs are made.

The problem with the ITIL approach is that it is somewhat abstract and does not establish firm correlations between service measurements and measures of defect potentials and defect removal efficiency. The bottom line is that unless defect removal efficiency is 95 percent or higher, achieving satisfactory service intervals will be essentially impossible.

The Emergence of Service-Oriented Architecture (SOA)

The origins of service-oriented architecture are fairly recent and only date back to about 2000. The SOA approach is an attempt to elevate software reuse from reusing small code segments of perhaps 10 function points up to reusing major components of perhaps 1,000 function points in size. Thus, a large system of 10,000 function points might be assembled from 10 existing components, each being 1,000 function points in size. These components can either be in-house software or acquired from vendors. These components might also be found on the Web, as is the case with email systems.

Under the SOA approach, there is no direct contact or internal linkage between the components. What happens is that the outputs from one component are analyzed and then routed to some other component by means of the SOA infrastructure. This means that no inner changes are needed in any component, although some of outputs may require conversion to different formats or translation of one kind or another.

In theory, the SOA approach can lead to very high productivity rates because this approach makes use of existing components. In terms of quality, the results are not yet clear. Obviously, the individual components need to be close to zero defect levels, which seldom occurs. Unless the defect potentials of the original components is below 2.5 defects per function point and the removal efficiency levels are above 95 percent, the SOA approach will not have adequate quality and reliability for success and there will be many failures.

However, as components continue to be used and latent defects are removed, it is possible that many kinds of applications can evolve into becoming pieces of service-oriented systems.

The Creation of the International Software Benchmarking Standards Group (ISBSG)

The International Software Benchmarking Standards Group (ISBSG) was founded in 1997, one year after the publication of the second edition of this book. Now in 2008, the volume of software benchmark data collected by ISBSG contains more than 4,000 projects, and the volume is increasing by perhaps 500 projects per year.

Unlike older benchmarking organizations such as Gartner Group, David Consulting Group, SPR, and so on, the data collected by the ISBSG can be purchased commercially on CD or in paper form. As a result, the ISBSG is now a valuable asset for the software industry.

Comparisons between the ISBSG data and the data published in this book show that the two collections of data are generally similar, and the ISBSG results will often be within a few percentage points of matching the data in this book.

The main differences between the ISBSG data and the data in this book are

- The ISBSG data tops out at about 20,000 function points whereas this book goes beyond 100,000 function points.
- The ISBSG data contains few military or systems software projects, which total about 50 percent of projects described here.
- The ISBSG data is self-reported, whereas the data shown here is primarily derived from site visits.
- The ISBSG data does not have very much quality information, but more is being added.

However, for information technology projects ranging between 100 and 10,000 function points in size, the ISBSG data and this book are very similar. The advantage of the ISBSG data is that it goes into more detail than the tables in this book. Further, once a company acquires the ISBSG data in CD format, many interesting statistical and analytical studies can be performed that would not be convenient using only static data derived from published tables. The ISBSG data is a valuable resource and becoming more valuable as the quantity of data increases. For more information, visit the ISBSG web site (www.ISBSG.org).

The Creation of the Information Technology Metrics and Productivity Institute (ITMPI)

Tony Salvaggio, the president of Computer Aid Inc. (CAI), had a vision of assembling the best articles and the best researchers under one umbrella. Therefore, in 2004, he founded the Information Technology Metrics and Productivity Institute. In about a four-year period, ITMPI has assembled hundreds of articles and speeches and reached agreements with many of the notable software engineering and management researchers in the industry. Some of the authors whose work can now be found on the ITMPI web site include, in alphabetical order: Victor Basili, Bob Charette, Gary Gack, Dan Galorath, David Garmus, David Herron, Watts Humphrey, Herb Krasner, Tom Love, Larry Putnam, Howard Rubin, Charles Symons, and Ed Yourdon. There are many others as well, and new authors are being added on a monthly basis.

For those interested in the state of the art of software engineering and software project management, the ITMPI web site is worth a visit (www.ITMPI.org). Not only does this site contain hundreds of interesting documents, but it also contains many recent studies created over the past few years.

The Emergence of the Team Software Process (TSP) and the Personal Software Process (PSP)

Watts Humphrey was formerly IBM's director of software process improvement. After that he joined the Software Engineering Institute (SEI) where he pioneered the famous capability maturity model (CMM). Obviously, Watts is one of the top experts in software development practices. The team software process (TSP) and personal software process (PSP) that he developed have been added to the higher CMM and CMMI levels. They require several weeks of training, but the empirical data to date indicates significant improvements in defect prevention and also in development productivity.

Although few managers realize it, reducing the quantity of defects during development is, in fact, the best way to shorten schedules and reduce costs. The TSP and PSP are very rigorous methods, and they do include collection of data on effort, defects, etc. As a general rule, the value of the TSP and PSP approaches increases with the size, complexity, and difficulty of the applications being developed. Defect potentials are cut back by at least 50 percent compared to older informal "waterfall" methods. Defect removal efficiency levels often top 95 percent using TSP and PSP.

For smaller simpler projects, the results from TSP and PSP are similar to those from Agile methods, but above 10,000 function points in size, the TSP and PSP approaches seem to be superior to the available alternatives. Both TSP and PSP are defined in books published by Addison Wesley, and additional information can be found on the SEI web site (www.SEI.org).

The Emergence of the ISO 9000-9004 Quality Standards

The International Standards Organization (ISO) quality standards 9000 through 9004 became mandatory in 1992 for selling various products throughout the European Community. By 2007 these standards have many years of field usage. Unfortunately, the information that is available indicates that the ISO standards have very little tangible impact on software quality but do tend to elevate software costs in a noticeable way owing to increasing the volumes of paper material produced.

In the course of SPR assessment and baseline surveys between 1992 and 2008, a number of companies were contacted that were in the process of completing ISO certification or that had already been certified. Similar companies were also studied that were not applying for certification. So far as can be determined from limited samples, there is not yet any tangible or perceptible difference in the quality levels of companies that have been certified and similar groups that have not.

As of 2008, there is still a remarkable lack of quantified results associated with the ISO 9000-9004 standard set. To date it does not seem to have happened that the ISO-certified groups have pulled ahead of the noncertified groups in some tangible way. The only tangible and visible impact of ISO certification appears to be the costs of the ISO certification process itself, the time required to achieve certification, and an increase in the volume of quality-related paper documents.

It is obvious that the ISO standards have had a polarizing effect on software producers, as well as on hardware manufacturers. Some companies such as Motorola have challenged the validity of the ISO standards, whereas others have embraced the ISO standards fully. The reaction to ISO certification among software groups to date has been somewhat

more negative than hardware and electronic component manufacturing. However, in several industries contacted (fiber optics, electronics manufacturing), the overall results of ISO certification are viewed as ambiguous. Some of the reactions are negative. On the other hand, some companies are using ISO certification as a marketing opportunity.

Some ISO adherents have observed that it is not the place of the ISO 9000-9004 standards to actually improve quality. Their purpose is simply to ensure that effective practices are followed. They are not in and of themselves a compilation of best current practices. Indeed, some companies have pointed out that compliance is fairly easy if you already have a good quality control approach in place, and that ISO certification tends to raise the confidence of clients even if quantification is elusive.

On the whole, the data available from the ISO domain as of mid-2007 remains insufficient to judge the impact of the ISO standards on these topics: (1) Defect potentials, (2) Defect removal efficiency levels, (3) Defect prevention effectiveness, (4) Reliability, (5) Customer satisfaction levels, (6) Data quality.

Quite a bit of information about ISO 9000-9004 is available via the Internet and various web sites. Attempts to find empirical data that ISO standards improved software quality using these channels elicited no useful factual information. However, many ISO enthusiasts responded with the surprising message that the ISO standards were not actually intended to improve quality and hence should not be faulted if they failed to do so.

The Emergence of Data Quality and Data Metrics

The topic of data quality was not included in the original 1991 edition but has been catapulted into significance in the world of data warehouses, repositories, online analytical processing (OLAP), and client-server applications. Unfortunately, as of 2008, research into data quality is handicapped by the fact that there are no known normalizing metrics for expressing the volume of data a company uses, the quality levels of the data, and the costs of creating data, migrating data from platform to platform, correcting errors, or destroying obsolete data. However, starting in late 2007, MIT may be performing research on a "data point" metric.

There is so little quantified information on data quality that this topic is included primarily to serve as a warning flag that an important domain is emerging, and substantial research is urgently needed. It is suspected that the databases and repositories owned by major corporations are filled with errors, redundant data, and other significant sources of trouble. But with no effective data volume or quality metrics, researchers are handicapped in even determining averages and ranges of data quality.

Several commercial companies and Dr. Richard Wang of MIT have begun to address the topic of data quality, but to date there are no known published statistical studies that include the volumes of data errors, their severity levels, or the costs of removing data errors. There is not even any accepted standard method for exploring the "cost of quality" for data errors in the absence of any normalizing metrics. (Note that the author's company, Software Productivity Research, is now exploring the concept of a "data point" metric that can be mathematically related to the function point metric. This is a research project, and all technical contributions would be welcome.)

Expansion of the SEI Capability Maturity Model (CMM and CMMI)

The Software Engineering Institute's (SEI) capability maturity model (CMM) concept is one of the most widely discussed topics in the software literature, although it has recently been replaced by the newer capability maturity model integration (CMMI). SEI has claimed that software quality and productivity levels correlate exactly with maturity level. There are similar claims for the newer CMMI approach. As of 2008, there is solid empirical data that supports these assertions. The higher CMM levels of 3, 4, and 5 do yield better quality levels and higher productivity than similar projects at Level 1. Interestingly, the larger the application the more successful the CMM approach becomes. Below 1,000 function points, Agile methods seem to be superior to the CMM, but above 10,000 function points the CMM pulls ahead.

Some years ago Software Productivity Research, Inc., was commissioned by the Air Force to perform a study of the economic impact of various SEI CMM levels. Raw data was provided to SPR on Levels 1, 2, and 3 by an Air Force software location. In terms of quality, the data available to date indicated that for maturity Levels 1, 2, and 3 average quality tends to rise with CMM maturity level scores. However, this study had a limited number of samples. By contrast, the Navy has reported a counterexample and has stated that at least some software produced by a Level 3 organization was observed to be deficient in terms of software quality and had high defect levels.

There is clearly some overlap among the various SEI levels. Some of the software projects created by organizations at SEI CMM Levels 1 and 2 are just as good in terms of quality as those created by SEI Level 3. Not only that, but achieving SEI Level 3 does not guarantee that all software projects will have exemplary quality.

More recent studies between 2000 and 2007 indicate that the higher CMM Levels 3, 4, and 5 do improve both productivity and quality. However, there is a caveat. Below 1,000 function points the CMM approach tends to be somewhat cumbersome, and the Agile methods

yield better productivity—although not better quality. But above 10,000 function points, the rigor of CMM Levels 3, 4, and 5 lead to improvements in both quality and productivity. Above 100,000 function points in size, only CMM Level 5 augmented by the TSP and PSP lead to a higher ratio of successful projects than failures. Six-Sigma is also beneficial for large applications in the 10,000 and 100,000 function point ranges.

On the whole, the SEI CMM and CMMI are in need of much more solid empirical data. Some of the results from ascending the SEI CMM are favorable, but the SEI approach is by no means a "silver bullet" that will solve all software quality problems.

Finally, there has been no quantitative comparison between software produced by the few hundred enterprises that have used the SEI maturity level concept and the more than 30,000 U.S. software producers that have not adopted the maturity concept at all. The topic of the exact costs and the value of the SEI maturity levels needs a great deal more research and quantification before definite conclusions can be reached. Ideally, what needs to be mounted is a large-scale survey involving perhaps 50 organizations and at least 250 projects at various SEI CMM levels.

The Rise (and Fall) of Client-Server Quality

The client-server phenomenon has been present in the software industry for more than 15 years. Ten years ago the growth of the client-server approach was one of the most rapid ever experienced by the software community, although it has slowed down since 2000.

No methodology is fully perfected when it first appears, and client-server is no exception. From talking to client-server users, developers, and listening to conversations at client-server conferences, the following critical problem areas will need to be addressed by client-server tool vendors and developers in the future:

- Client-server quality levels are suspect.
- Client-server development methodologies are still immature.
- Data consistency among client-server applications is questionable.
- Client-server maintenance costs are higher than normal.

Here are some observations:

- Client-server applications have higher delivered defect rates than other forms of software. Quality data from client-server applications is somewhat alarming and indicates a need for pretest inspections and better quality control.
- Compared to the way mainframe applications have been built, the overall set of methods used for client-server software can only be characterized as "immature." Quite a bit more rigor is indicated.

- How client-server applications will impact corporate data administration, data dictionaries, and repositories has not yet been fully worked out. Initial findings are somewhat ominous. It is hard to escape the conclusion that client-server applications are starting to suffer from inconsistent data definitions, unique or quirky data elements, and carelessness in data design.
- Client-server applications were usually brand new in 1995, but they are not brand new in 2008. How much it will cost a company to maintain client-server applications now that they are aging legacy systems is an important topic, and the costs seem about 30 percent higher than ordinary mainframe applications of the same size.

Unlike stand-alone personal computer applications, client-server projects are distributed software applications that involve multiple computers, networks or linkages between mainframes and personal computers, and quite a bit of sharing and synchronization of the data residing in the server portion of the application. These factors mean that client-server development is more complex than developing older monolithic applications. It also means that the operational complexity is higher.

Although the complexity of typical client-server applications is fairly high, development practices for client-server applications are often less formal than for older mainframe applications. For example, formal inspections of specifications and source code are less common for client-server applications than for mainframe applications. The combined results of higher complexity levels and less rigorous development practices tend to have both a near-term and a long-term impact: In the near term, client-server quality is often deficient, and in the long term, client-server maintenance costs may well exceed those of traditional mainframe applications.

Although the data is still far from complete and has a high margin of error, it is of interest to consider some of the differences between monolithic mainframe applications and client-server applications.

Following is a typical defect pattern for monolithic mainframe applications, using the metric "defects per function point" for normalizing the data.

By contrast, following that is the equivalent defect pattern for distributed client-server applications. The number of requirements problems are approximately equal between the two domains. However, owing to increased complexity levels associated with distributed software, design and coding defect rates are typically more numerous. Also more numerous is the category of "bad fixes," which are secondary defects or bugs accidentally introduced into software during the fixing of previous bugs. The category of bad fixes is somewhat ominous since it will have an impact on the long-range maintenance costs of client-server applications.

As can be seen, client-server applications are generating more than 20 percent higher defect potentials than monolithic mainframe software.

Even more troublesome than the elevated defect potentials associated with client-server software are somewhat lower defect removal efficiency rates. Defect removal efficiency reflects the combined effectiveness of all reviews, inspections, and tests carried out on software before delivery. For example, if a total of 90 bugs are found during the development of a software project and the users find and report 10 bugs, then the defect removal efficiency for this project is 90 percent, since 9 out of 10 bugs were found before release.

Software Defect Pattern for Monolithic Mainframe Applications (Defects per Function Point)

Requirements Defects	Design Defects	Code Defects	Bad Fixes	Overall Defect Total
1.00	1.25	1.75	0.50	4.50

Software Defect Pattern for Distributed Client-Server Applications (Defects per Function Point)

Requirements Defects	Design Defects	Code Defects	Bad Fixes	Overall Defect Total
1.00	1.75	2.00	0.75	5.50

The average defect removal efficiency for typical monolithic mainframe applications has been about 85 percent. (Note that best-in-class companies average above 95 percent.) To date, the equivalent results within the client-server domain have been less than 80 percent, although the data on this topic is very sparse and has a high margin of error.

When the mainframe defect potentials of about 4.5 bugs per function point are reduced by 85 percent, the approximate quantity of delivered defects can be seen to average about 0.675 defects per function point. For distributed client-server applications, the higher defect potential of 5.5 bugs per function point interacts in an alarming way with the reduced removal efficiency of 80 percent. The results indicate a defect delivery level of about 1.1 defects per function point, which is uncomfortably close to twice the typical amounts latent in mainframe software.

Although client-server quality is often marginal today, that does not mean that it will stay bad indefinitely. The same kinds of methods, tools, and approaches that have been used successfully by "best-in-class" software producers for many years can also be used with client-server applications. These approaches can be summarized in terms of defect prevention methods, defect removal methods, and quality tools.

Among the kinds of defect prevention approaches that work with client-server applications, the concept of joint application design (JAD), can be very useful. The JAD approach calls for joint requirements

development between the client organization and the software group. The overall impact of the JAD method is to reduce the rate of unplanned creeping requirements and to reduce the number of defects in the requirements themselves.

In terms of defect removal, testing alone has never been sufficient to ensure high quality levels. All of the best-in-class software producers such as AT&T, Hewlett-Packard, Microsoft, IBM, Raytheon, or Motorola utilize both pretest design reviews and formal code inspections. Design reviews and code inspections can both be used with client-server applications and should improve defect removal efficiency notably.

In terms of tools that can benefit client-server quality, any or all of the following tools have a useful place: (1) Quality and defect estimating tools, (2) Defect tracking and measurement tools, (3) Complexity analysis tools, (4) Design and code inspection support tools, (5) Prototyping tools, (6) Code restructuring tools, (7) Configuration control tools, (8) Record and playback test support tools, (9) Test coverage analysis tools, and (10) Test library control tools.

The client-server domain is being replaced by web-enabled applications. Therefore, comparatively few new client-server applications are being built in 2008 compared to web applications.

The Rapid Growth of Object-Oriented Quality Levels

The object-oriented world has been on an explosive growth path. The usage of OO methods has now reached a sufficient critical mass to have created dozens of books, several OO magazines, and frequent conferences such as OOPSLA and Object World. Unfortunately, this explosion of interest in OO methodologies has not carried over into the domain of software quality. There are a number of citations on software testing, but little or no empirical data on actual software quality levels associated with the OO paradigm. Even in 2008, there is little data on topics such as defect potentials, defect removal efficiency levels, bad-fix injections, and error-prone modules (or error-prone class libraries) in the OO domain.

From very preliminary and provisional results, it appears that object-oriented languages such as C++ and SMALLTALK may have reduced defect levels compared to normal procedural languages such as C or COBOL or Fortran. Defect potentials for OO languages are perhaps in the range of 1 per function point, as opposed to closer to 1.75 per function point for normal procedural languages. However, the defect potentials for object-oriented analysis and design have not yet showed any reduction, and indeed may exceed the 1.25 defects per function point associated with the older standard analysis and design approaches such as Yourdon, Warnier-Orr, and Gane & Sarson. Use cases, for example, show no significant reduction in levels of design defects compared to other methods.

Unfortunately, OO analysis and design tend to have high rates of abandonment, as well as projects that started with OO analysis and design but switched to something else in midstream. These phenomena make it difficult to explore OO quality. There is obviously a steep learning curve associated with most of the current flavors of OO analysis and design, and steep learning curves are usually associated with high levels of errors and defects.

After more than 20 years, there is not yet enough data available in 2008 to reach a definite conclusion, but there is no convincing published evidence that OO quality levels are dramatically better than non-OO quality levels for OO analysis and design. The data on OO languages is somewhat more convincing but still needs more exploration. On theoretical grounds, it can be claimed that OO quality levels should be better, owing to inheritance and class libraries. However, that which is inherited and the class libraries themselves need to be validated carefully before theory and practice coincide.

The Rise (and Fall) of Total Quality Management (TQM) for Software

When the first edition of this book was published in 1991, total quality management (TQM) was on an upswing in the software industry. At the time of the second edition, TQM appeared to be declining in the software world, owing to the visible lack of success as a software QA approach. As of the third edition in 2008, TQM is more or less relegated to history. TQM is not part of Agile or Extreme programming (XP) or most other recent methods.

This is not to say that age alone makes methodologies less useful. In fact, formal design and code inspections are approaching 40 years of age, and they remain the most efficient and effective ways of eliminating software defects.

Unfortunately, only about half of the TQM experiments in the United States were successful and improved quality, and the other half were failures. The successful use of TQM correlated strongly with the seriousness of the commitment and the depth of understanding by executives and management. TQM worked only if it was really used. Giving the TQM concepts lip service but not really implementing the philosophy only led to frustration.

The TQM concept was not a replacement for more traditional software quality approaches. Indeed, TQM worked best for organizations that were also leaders in traditional quality approaches. Specifically, some of the attributes of companies that have been successful in their TQM programs include the following: (1) They also have effective software quality measurement programs that identify defect origins, severities, and removal efficiency rates; (2) They utilize formal reviews and

inspections before testing begins; (3) Their software quality control was good or excellent even before the TQM approach was begun.

Conversely, enterprises that lacked quality metrics, which failed to utilize pretest defect prevention and removal operations, and which lagged behind their competitors in current software quality control approaches tended to gain only marginal benefits, if any, from the adoption of the TQM method. Such enterprises were very likely to use TQM as a slogan, but not to implement the fundamental concepts.

The operative word for total quality management is *total*. Concentrating only coding defects, measuring only testing, and using the older KLOC metric violates the basic philosophy of total quality management. These approaches ignore the entire front end of the lifecycle and have no utility for requirements problems, design problems, documentation problems, or any of the other sources of trouble that lie outside of source code.

It is obvious that there are three sets of factors that must be deployed in order for TQM to be successful with software:

- Adopting a culture of high quality from the top to the bottom of an enterprise
- Using defect prevention methods to lower defect potentials
- Using defect removal methods to raise prerelease efficiencies

Synergistic combinations of defect prevention methods can reduce defect potentials by more than 50 percent across the board, with the most notable improvements being in some of the most difficult problem areas, such as requirements errors.

The set of current defect removal methods includes such things as formal inspections, audits, independent verification and validation, and many forms of testing. However, to bring TQM to full power in an organization, the culture of the management community must be brought up to speed on what TQM means and how to go about it.

In general, no one method by itself is sufficient, and a major part of the TQM approach is to define the full set of prevention and removal methods that will create optimal results. This, in turn, leads to the need for accurate measurement of defect potentials and defect removal efficiency.

Quality Control and International Competition

There are four critical business factors that tend to determine successful competition in global markets:

- Cost of production
- Time to market

- Quality and reliability
- User satisfaction

All four are important, and all four depend upon careful measurement for a company to make tangible improvements. Cost of production has been a major factor for hundreds of years, and it has proven its importance in both natural products (agriculture, minerals, metals, and so on) and manufactured durable goods as well as services. Indeed, cost of production was the first business factor to be measured and explored in depth.

Time to market has become progressively important in recent years as new manufacturing technologies and rapid communication make international competition more speed-intensive than the former durable goods economy. Time to market was particularly important in the 20th century and even more so today in the 21st; it is a key consideration for consumer goods and such personal electronics as iPod and cell phones, MP3 players, and compact disc players. It is also a major factor for industrial and commercial products.

Only since the 1950s have quality, reliability, and user satisfaction been recognized as the major driving force for high-technology products and hence a key ingredient in long-range national and industrial success. Quality, reliability, and user satisfaction are turning out to be the driving force of competition for high-technology products, including automobiles, computers, telecommunications equipment, and many modern consumer products such as kitchen appliances and power tools.

It is primarily quality control, rather than cost of production or time to market, that explains the unprecedented success of Japan in modern high-technology competition. Indeed, the pioneering work of Deming on quality measurement and statistical quality control, which was widely adopted by Japanese industry, has perhaps been the single most important factor in Japan's favorable balance of trade vis-à-vis the United States.

Further, in the specific case of software, quality control is on the critical path for reducing cost of production and shortening time to market. It is also important in determining user satisfaction. The importance of quality control cannot be overstressed: software producers who do not control quality risk being driven from their markets and perhaps out of business by competitors who do control quality! Before dealing with the measurement of quality, it is useful to consider the significance of this factor on high-technology products in general and on software in particular.

Since the reemergence of Japanese industry after World War II, Japan has followed one of the most successful industrial strategies in all human history. The basic strategy has two aspects:

- Concentrate on high-technology products because the value per shipped ton is greater than for any other kinds of product.
- Compete by achieving quality levels at least 25 percent higher than those of U.S. and European companies while holding prices to within ±10 percent of U.S. and European levels.

In research of the factors that contribute to market share of high technology products, the customer perception of quality has turned out to be the dominant factor for large market shares. For high technology products, quality and user satisfaction even outweigh time to market and cost of production in terms of overall market share and competitive significance.

A question that arises is why the widely publicized Japanese strategy has been so successful against U.S. and European companies when those companies can easily follow the same strategy themselves. The answer is that U.S. and European business strategies evolved more than 100 years ago during an earlier industrial era when factors such as least-cost production were dominant in creating market shares and quality was often a secondary issue. Many U.S. and European executives either ignored quality or perceived it to be an optional item that was of secondary importance. That was a mistake, and the United States and Europe are now paying the price for it.

As of 2008 both India and China are beginning to follow the Japanese business pattern. As of 2008, India has more high-level CMM organizations than any other country. China too has begun to move toward both the CMM and other quality-control approaches. Because inflation rates in India and China are increasing rapidly, the long-range strategy for both countries is to achieve quality levels significantly above U.S. and European averages, just as Japan has done.

The whole postwar generation of U.S. and European executives have been playing by the rules of a 100-year-old game whereas Japanese executives, and now Chinese and Indian executives, are playing a modern high-technology game. The following is a list of ten high-technology products for which quality is the dominant factor leading to market share:

- Automobiles
- Cameras
- Compact disc players
- Computers
- Machine tools
- Medical instruments
- Stereo receivers

- Telephone handsets
- Television sets
- Watches

Japan has become the world's largest producer of five of the ten products and is a major producer of all of the others. The United States has steadily lost global market share in all of the areas except computers. However, China is rapidly catching up to Japan, due in large part to improved quality levels.

Another significant aspect of industry is that microcomputers and software are embedded in all 10 of the product types listed. The 2007 model automobiles, for example, included cars with as many as 12 separate onboard computers governing fuel injection, cooling and heating, audio equipment, odometers and tachometers, and other instruments!

As computers and software continue to expand into consumer product areas, companies must recognize that success in software is on the critical path to corporate survival. The future success of the United States against global competition requires that the CEOs of U.S. industries understand the true factors of global competition in the 21st century:

- High-technology products are critical to U.S. success.
- Quality is the dominant marketing factor for high-technology products.
- High-technology products depend upon computing and software.
- Quality is the key to success in computing and software.
- Quality must start at the top and become part of the U.S. corporate culture.

By the end of the 20th century, it could be seen that two new business laws were about to become operational and drive successful businesses and industries through the 21st century:

- **Law 1** Enterprises that master computers and software will succeed; enterprises that fall behind in computing and software will fail.
- **Law 2** Quality control is the key to mastering computing and software; enterprises that control software quality will also control schedules and productivity. Enterprises that do not control quality will fail.

Defining Quality for Measurement and Estimation

Readers with a philosophical turn of mind may have noticed a curious anomaly in this chapter so far. The term "quality" has been used many times on every page but has not yet been defined. This anomaly

is a symptom of a classic industrial paradox that deserves discussion. Both quality and quality control are agreed by all observers to be major international business factors, yet the terms are extraordinarily difficult to define precisely. This phenomenon permeates one of the finest although unusual books on quality ever written: *Zen and the Art of Motorcycle Maintenance,* by Robert Pirsig. In his book, Pirsig ruminates that quality is easy to see and is immediately apparent when encountered. But when you try to pin it down or define what it is, you find the concept is elusive and slips away. It is not the complexity but the utter simplicity of quality that defies explanation.

In the writings of software quality specialists, and in informal discussions with most of them, there are a variety of concepts centering around software quality. Here are the concepts expressed by several software specialists, in alphabetical order:

- Dr. Barry Boehm, formerly of TRW and DARPA and now at USC, tends to think of quality as "achieving high levels of user satisfaction, portability, maintainability, robustness, and fitness for use."
- Phil Crosby, the former ITT Vice President of Quality, has created the definition with the widest currency because of its publication in his famous book *Quality Is Free.*" Phil states that quality means "conformance to user requirements."
- W. Edwards Deming, in his lectures and writings, considers quality to be "striving for excellence in reliability and functions by continuous improvement in the process of development, supported by statistical analysis of the causes of failure."
- Watts Humphrey, of the Software Engineering Institute, tends to speak of quality as "achieving excellent levels of fitness for use, conformance to requirements, reliability, and maintainability."
- Capers Jones, the author of this book, defines software quality as "the absence of defects that would make software either stop completely or produce unacceptable results. Defects can be traced to requirements, to design, to code, to documentation, or to bad fixes of previous defects. Defects can range in severity from minor to major."
- Steve Kan, author *Metrics and Models in Software Quality Engineering* and a top software quality expert at IBM defines quality in both popular and professional modes. For the purposes of this book, Kan's professional definition, which is based on the early work of J. M. Juran, uses the classic "fitness for use" as a prime definition.
- James Martin, in his public lectures, has asserted that software quality means being on time, within budget, and meeting user needs.

- Tom McCabe, the complexity specialist, tends to define quality in his lectures as "high levels of user satisfaction and low defect levels, often associated with low complexity."
- John Musa of Bell Laboratories, the well-known reliability modeler, states that quality means a combination of "low defect levels, adherence of software functions to user needs, and high reliability."
- Bill Perry, head of the Quality Assurance Institute, has defined quality in his speeches as "high levels of user satisfaction and adherence to requirements."

Considered individually, each of the definitions has merit and raises valid points. Taken collectively, the situation resembles the ancient Buddhist parable of the blind men and the elephant: The elephant was described as being like a rope, a wall, or a snake depending upon whether the blind man touched the tail, the side, or the trunk. For practical day-to-day purposes, a working definition of quality must meet two criteria:

- Quality must be measurable when it occurs.
- Quality should be predictable before it occurs.

Table 5-1 lists the elements of the major definitions just quoted and indicates which elements meet both criteria.

Although Table 5-1 does not contain many surprises, it is important to understand the limitations for measurement purposes of two important concepts: conformance to requirements and user satisfaction.

TABLE 5-1 Predictability and Measurability of Quality Factors

Quality Factor	Predictable	Measurable
Defect levels	Yes	Yes
Defect origins	Yes	Yes
Defect severity	Yes	Yes
Defect removal efficiency	Yes	Yes
Product complexity	Yes	Yes
Project reliability	Yes	Yes
Project maintainability	Yes	Yes
Project schedules	Yes	Yes
Project budgets	Yes	Yes
Portability	Yes	Yes
Conformance to requirements	No	Yes
User satisfaction	No	Yes
Fitness for use	No	Yes
Robustness	No	No

For software, Phil Crosby's widely quoted definition of quality as "conformance to requirements" suffers from two serious limitations:

- Although this can be measured after the fact, there are no a priori estimating techniques that can predict the phenomenon ahead of time.
- When software defect origins are measured, "requirements" is one of the chief sources of error. To define quality as conformance to a major source of error leads to problems of circular reasoning and prevents either measurement or estimation with accuracy. It should not be forgotten that the famous "Y2K" problem with two-digit dates originated as a specific user requirement. Many serious problems originate in requirements, so they cannot serve as a standard for measuring quality.

User satisfaction with software is certainly capable of being measured with high precision after the fact. But there are currently no effective estimating techniques that can directly predict the presence or absence of this phenomenon far enough ahead of time to take effective action. There are, however, very strong correlations between user satisfaction and factors that can be estimated. For example, there is a very strong inverse correlation between software defect levels and user satisfaction. So far as can be determined, there has never been a software product with high defect levels that was satisfactory to its users. The opposite situation, however, does not correlate so well: There are software products with low defect levels that are not satisfactory to users.

Another aspect of user satisfaction is an obvious one: Since this factor depends on users, it is hard to carry out early measurements. In the case of defects, for example, measurements can start as early as the requirements phase. As for user satisfaction, it is hard to begin until there is something to use. These considerations mean that, in a practical day-to-day measurement system, user satisfaction is a special case that should be dealt with on its own terms, and it should be decoupled from ordinary defect and removal metrics. The bulk of the other metrics in Table 5-1 meet the two criteria of being both predictable and measurable. Therefore, any or all of them can be used effectively for quality control purposes.

In recent years, the Agile approach has added user representatives to development teams, so that daily contact between the user representative and the team occurs. Although this approach does give early insights into user satisfaction, the method only works for small applications with comparatively few users in total. The Agile approach would not be effective for applications such as Microsoft Word or Vista where the number of users will number in the millions. No single user can possibly reflect the combined needs of the entire community, nor can one user serve to judge user satisfaction.

Five Steps to Software Quality Control

These five steps to software quality control have been observed in the course of software management consulting in leading corporations.

Step 1: Establish a Software Quality Metrics Program

Software achieved a notorious reputation during the first 50 years of its history as the high-technology occupation with the worst track record in terms of measurements. Over the last 25 years, improvements in measurement technology have enabled leading-edge companies to measure both software quality and productivity with high precision. If your company has not yet adopted leading-edge quality metrics for key products, you are headed for a disturbing future as your more advanced competitors pull ahead.

The significance of this step cannot be overstated: If you look inside the leading companies within an industry, you will find full-scale measurements programs and hence executive abilities to receive early warnings and cure problems. For example, IBM and AT&T have had quality measurements since before World War II and software quality measurements since the 1960s. Quality measurement is a critical factor in high-technology products, and all the companies that have tended to become household words have quality measurement programs: Microsoft, Hewlett-Packard, IBM, and many others. The lagging enterprises that have no software measures also have virtually no ability to apply executive control to the software process.

Step 2: Establish Tangible Executive Software Performance Goals

Does your enterprise have any operationally meaningful software quality or productivity goals? The answer for many U.S. companies would be no, but for leading-edge companies such as IBM and Hewlett-Packard it would be yes. Now that software can be measured, it is possible to establish tangible, pragmatic performance goals for both software quality and productivity. Since the two key aspects of software quality are defect removal efficiency and customer satisfaction, reasonable executive targets would be to achieve higher than 95 percent efficiency in finding software bugs and higher than 90 percent "good" or "excellent" customer satisfaction ratings.

Note that for sociological reasons, quality goals should be targeted at the executive level rather than at the worker level. The reason for this is that executives are authorized to spend funds and acquire training for their employees. It is valid for individual employees to have personal

goals and to strive for quality, but official corporate quality targets need to be set for those who are empowered to introduce organizational changes, not for those whose span of control is limited to their personal behavior.

Step 3: Establish Meaningful Software Quality Assurance

One of the most significant differences between leading and lagging U.S. enterprises is the attention paid to software quality. It can be strongly asserted that the U.S. companies that concentrate on software quality have higher productivity, shorter development schedules, and higher levels of customer satisfaction than companies that ignore quality. Since the steps needed to achieve high quality include both defect prevention and defect removal, a permanent quality assurance organization can facilitate the move toward quality control.

It is possible, in theory, to achieve high quality without a formal quality assurance organization. Unfortunately, American psychology often tends to need the external prompting that a formal quality assurance organization provides. An effective, active quality assurance team can provide a continuous boost in the quality arena.

As business profits and income rise and fall, there are layoffs and downsizings from time to time. Unfortunately it often happens that quality assurance specialists are among the first to be let go in times of economic stress. In fact specialists are usually let go before the general software engineering population. This is unfortunate and actually counterproductive. Reducing corporate emphasis on quality will degrade productivity, lengthen development schedules, and lower customer satisfaction.

Step 4: Develop a Leading-Edge Corporate Culture

Business activities have a cultural component as well as a technological component. The companies that tend to excel in both market leadership and software engineering technologies are those whose corporate cultures reflect the ideals of excellence and fair play. If your corporate culture stresses quality, service to clients, innovation, and fairness to employees, there is a good chance that your enterprise is an industry leader. If your corporate culture primarily stresses only schedule adherence or cost control, as important as those topics are, you may not ultimately succeed. Am interesting correlation can be noted between corporate culture and quality control. Of the companies that produce software listed in *The 100 Best Companies to Work for in America* most have formal software quality programs.

There is no external source of corporate culture. The board of directors, the CEO, and the senior executives are the only people who can forge

a corporate culture, and it is their responsibility to do it well. As Tom Peters has pointed out in his landmark book, *In Search of Excellence,* the truly excellent enterprises are excellent from top to bottom. If the top is not interested in industry leadership or doesn't know how to achieve it, the entire enterprise will pay the penalty.

Step 5: Determine Your Software Strengths and Weaknesses

More than 200 different factors can affect software productivity and quality; they include the available tools and workstations, the physical environment, staff training and education, and even your compensation plans. In order to find out how your enterprise ranks against U.S. or industry norms and whether you have the appropriate set of tools and methods in place, it is usually necessary to bring in one of the many U.S. management consulting organizations that specializes in such knowledge.

This step is logically equivalent to a complete medical examination in a major medical institution. No physician would ever prescribe therapies without a thorough examination and diagnosis of the patient. The same situation should hold true for software: Do not jump into therapy acquisition without knowing what is right and wrong in all aspects of your software practice.

Software Quality Control in the United States

Computing and software have become dominant issues for corporate success and even corporate survival. As the 20th century drew to a close, the enterprises that could master computing and software were poised to probably be the key enterprises of the next century. The enterprises that did not master computing and software may not survive this new century!

Quality control is the key to mastering computing and software, and the enterprises that succeed in quality control will succeed in optimizing productivity, schedules, and customer satisfaction also. Following are short discussions of some of the quality control methods used in the United States.

Quality Assurance Organizations

Most United States commercial, military, and systems software producers have found it useful to establish formal quality assurance (QA) organizations. These QA organizations vary significantly in their roles authorities and responsibilities. The most successful of the QA organizations, which are typical of large computer manufacturers and major software vendors, are capable of serving as general models.

Quality assurance organization types There are two common forms of QA organization; they might be termed "active" and "passive." Of these, the active QA organization is usually more successful, although more expensive as well.

Active quality assurance The phrase "active quality assurance" means a formal quality assurance organization that plays an active part in software projects. This is the form of quality assurance most often practiced by leading-edge computer manufacturers, defense contractors, and other high-technology enterprises. This form is recommended above the passive form. Typical roles encompassed by active QA organizations include

- Moderating design and code inspections
- Collecting and analyzing defect data
- Developing and running test scenarios
- Estimating defect potentials
- Recommending corrective actions for quality problems
- Teaching courses on quality-related topics

Active quality assurance groups normally start work on projects during or shortly after the requirements phase and are continuously engaged throughout a project's development cycle.

Because of the scope and amount of work carried out by active quality assurance groups, the staffing requirements are not insignificant: Most active QA organizations are in the range of 3 to 5 percent of the size of the software development organizations they support. Some, such as the IBM QA organization supporting commercial and systems software, approach 10 percent of the entire software staff. Active QA organizations normally are managerially independent of development, which is a pragmatic necessity to guarantee independence and avoid coercion by development project management.

Passive quality assurance The phrase "passive quality assurance" refers to a group whose primary task is observation to ensure that the relevant quality standards and guidelines are, in fact, adhered to by development personnel. The passive QA organizations can, of course, be much smaller than the active ones, and typically they are in the range of 1 to 2 percent of the size of the development groups they support. However, the effectiveness of passive QA is necessarily less than that of the active form. Some of the roles performed by the passive QA organizations include

- Observing a sample of design and code inspections
- Analyzing defect data collected by development personnel
- Recommending corrective actions for quality problems

Passive QA organizations, like the active forms, are normally managerially independent of development, which is a pragmatic necessity to guarantee independence and to avoid coercion by development project management.

Reviews, Walk-Throughs, and Inspections

Informal reviews have been held spontaneously since the earliest days of software, and the first such reviews may even have taken place prior to 1950. However, in the burst of really large system projects starting in the 1960s and 1970s, it became obvious that testing was too late in the development cycle, too inefficient, and not fully effective.

Several IBM researchers turned their attention to the review process, and three alternative approaches surfaced more or less at the same time: Design reviews and code reviews originated at the IBM San Jose laboratories in the early 1970s. Structured walk-throughs originated concurrently at the IBM Poughkeepsie laboratory. Formal design and code inspections originated slightly after walk-throughs; they were created at the IBM Kingston laboratory and were based on the analysis of Michael Fagan, Lew Priven, Ron Radice and several others. Tom Gilb and his colleagues in Europe also made contributions to inspection methods.

After considerable debate and trial, the formal inspection process tended to come out ahead in terms of overall rigor and efficiency and gradually moved toward becoming a standard technique within IBM. Indeed, inspections appear to be the most efficient form of defect removal yet developed, and only formal inspections tend to consistently exceed 60 percent in defect removal efficiency. External publication of the method in 1976 by Fagan brought the inspection technique into more widespread use. IBM began to offer external training in inspections by 1979, and other companies such as ITT and AT&T adopted the method as an internal standard. By 1990, a dozen or more computer, consulting, and educational companies including SPR were teaching inspections, and the method was continuing to expand at the time the second edition of this book was published in 1996. Still, today in 2008, formal design and code inspections rank as the most effective methods of defect removal yet discovered. The defect removal efficiency levels of formal inspections, which can top 85 percent, are about twice those of any known form of testing. Further, inspections have secondary benefits as well. Participation in inspections is an excellent form of defect prevention. Additionally, inspections raise the efficiency of subsequent testing by providing better and more accurate specifications.

Formal inspections differ from reviews and walk-throughs primarily in the rigor of the preparation, the formal roles assigned, and the post-inspection follow-up. To be deemed an "inspection," it is necessary to obey the following protocols:

1. The participants must receive training prior to taking part in their first inspection.
2. There will be sufficient time available for preparation before the inspection sessions take place.
3. In the inspection sessions, there will be at least the following staff present:
   - A moderator (to keep the discussions within bounds)
   - A reader (to paraphrase the work being inspected)
   - A recorder (to keep records of all defects)
   - An inspector (to identify any problems)
   - An author (whose work is being inspected)

All five (or more) may be present for large projects. For small projects dual roles can be assigned, so that the minimum number for a true inspection is three: moderator, author, and one other person.

Formal inspections typically follow these patterns of behavior:

- The inspection sessions will follow normal protocols in terms of timing, recording defects, and polling participants.
- There will be follow-up after the inspection process to ensure the identified defects are repaired.
- The defect data will not be used for appraisal or punitive purposes.

The purpose of inspections is simply to find, early in the process, problems that if not eliminated could cause trouble later. It should be emphatically stated that the purpose of inspections is neither to humiliate the authors, nor to fix the problems. Defect repairs take place later. The purpose, once again, is simply to find problems.

Inspections are not inexpensive, but they are highly efficient. Prior to first utilizing this method, there will be a natural reluctance on the part of programmers and analysts to submit to what may seem an unwarranted intrusion into their professional competence. However, this apprehension immediately disappears after the first inspections, and the methodology is surprisingly popular after the start-up phase.

Any deliverable can be inspected, and the method has been used successfully on the following:

- Requirements
- Initial and detailed specifications
- Development plans
- Test plans

- Test cases
- Source code
- User documentation
- Training materials
- Screen displays

For initial planning purposes, Table 5-2 shows the normal times required to prepare for and carry out inspections for typical deliverables. Table 5-2 is taken from the SPR inspection training materials as updated for 1991. Note that the variations for both preparation and meeting effort can exceed ±50 percent of the figures in Table 5-2, based upon individual human factors, interruptions, and the numbers of problems encountered. Normal single-column pages in a 10-pitch type with a normal volume of graphics are assumed: roughly 500 words per text page and about 150 words per page containing charts, diagrams, or illustrations. For source code, normal formatting of about 50 statements per page of listing and a single statement per physical line is assumed.

The business advantages of inspections can be illustrated by two simple figures. Figure 5-1 shows the distressing gaps in the timing between defect introduction and defect discovery in situations where testing alone is used. Figure 5-2 shows the comparatively short intervals between defect creation and defect discovery that are associated with inspections.

Inspections tend to benefit project schedules and effort as well as quality. They are extremely efficient in finding interface problems between components and in using the human capacity for inductive reasoning to find subtle errors that testing will miss. They are not very effective, of course, in finding performance-related problems where actual execution of the code is usually necessary.

Quality Function Deployment (QFD)

Quality function deployment is another interesting and useful quality control method that originated in Japan. The basic method of quality function deployment is to have users and software developers meet

TABLE 5-2 Typical Inspection Preparation and Execution Times

Deliverable	Preparation	Meeting
Requirements	25 pages/h	12 pages/h
Functional specification	45 pages/h	15 pages/h
Logic specification	50 pages/h	20 pages/h
Source code	150 LOC/h	75 LOC/h
User documents	35 pages/h	20 pages/h

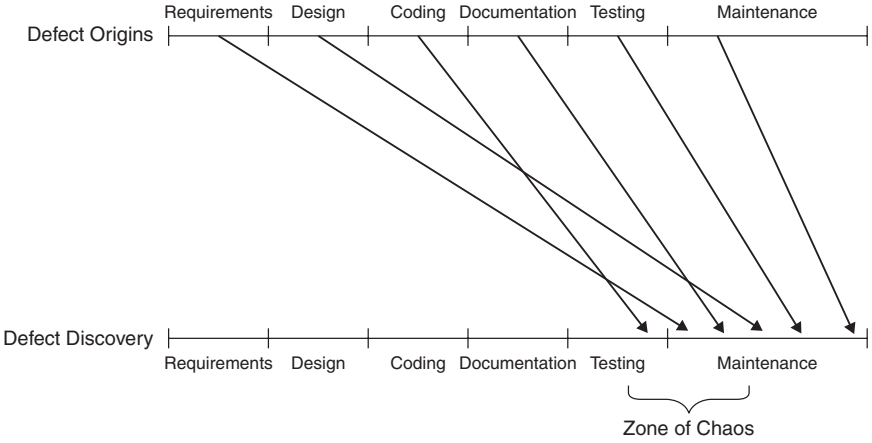

Figure 5-1 Delays between defect creation and discovery when testing is the primary removal method

and discuss user quality issues in a formal way, with the discussions being documented using a special kind of graph called "house of quality" because of its unusual peaked triangular top, which serves to link user quality needs with engineering responses. The user quality needs are termed "the voice of the customer" or VOC.

As with many quality approaches, QFD originated for hardware and was transferred and slightly modified for software later. As of 2008 QFD is used primarily for systems and weapons systems software, but it has been quite successful in many cases, though not always. Some organizations have balked at the time required (several weeks) whereas others have corporate cultures that tend to gloss over complex issues.

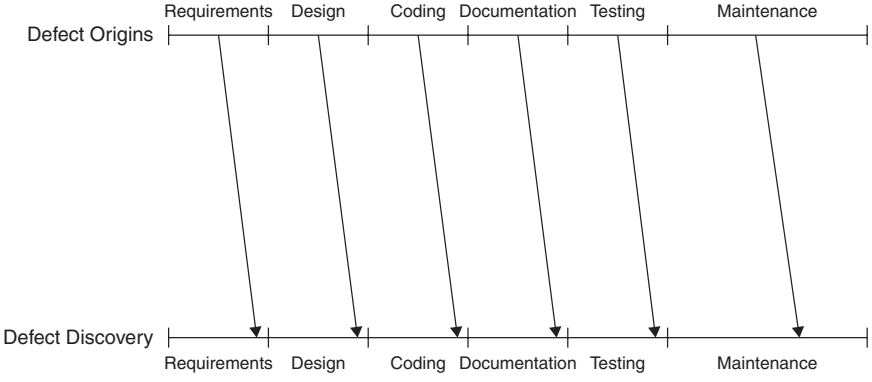

Figure 5-2 Reduced delays between defect creation and discovery associated with formal inspections

When used successfully, QFD has the effect of reducing requirements defects by as much as 70 percent, and it has the effect of raising defect removal efficiency against requirements defects up to more than 95 percent.

Six-Sigma for Software

The Six-Sigma approach was first developed by Motorola for use with telecommunications hardware, and was later transferred to software. The term "Six-Sigma" derives from mathematics and means reducing defects below a level of 3.4 per 1,000,000 opportunities.

Actually achieving Six-Sigma levels of quality is beyond the state of the art for software circa 2008. However, the principles of Six-Sigma are useful and effective. Among these useful principles are those of accurate measurements of defects, root cause analysis, and continuous improvement. The combination of accurate measures coupled with continuous improvement in both defect prevention and defect removal are right on target for software.

Several recent variations to traditional Six-Sigma have emerged and are proving useful to software. A method called "lean Six-Sigma" is somewhat quicker to learn and deploy than conventional Six-Sigma. Hybrid methods that couple Six-Sigma with other methods such as the CMM are valuable too.

Quality Circles

The technique of quality circles became enormously popular in Japan, where many thousands of quality circles have been registered. The concept is that ordinary workers, given the opportunity and some basic training in cause-effect analysis, can provide a powerful quality assurance and productivity boost for their employers. Quality circles have demonstrated their value in manufactured goods, electronics, automotive production, and aircraft production. In Japan at least, the technique has also worked well for software. Here in the United States, the approach has neither been widely utilized for software, nor is there enough empirical data reported in the literature to even judge the effectiveness for U.S. software.

Zero-Defect Programs

The concepts of zero defects originated in aviation and defense companies in the late 1950s. Halpin wrote an excellent tutorial on the method in the mid-1960s. The approach, valid from a psychological viewpoint, is that if each worker and manager individually strives for excellence and zero defects, the final product has a good chance of achieving zero defects. The concept is normally supported by substantial public

relations work in companies that adopt the philosophy. Interestingly, some zero-defect software applications of comparatively small size have been developed. Unfortunately, however, most software practitioners who live with software daily tend to doubt that the method can actually work for large or complex systems. There is certainly no empirical evidence that zero-defect large systems have yet been built.

Proofs of Correctness

The concepts of correctness proofs originated in the domain of systems software, and they have been made well known by such authors as Mills and Martin. Unfortunately, there is no empirical data that demonstrates that software proven correct has a lower defect rate under field conditions than software not proven correct. Anecdotal evidence indicates that proofs are fairly hard to create, and indeed there may be errors in a substantial number of the proofs themselves. Also, proofs are very time-consuming. On the whole, this technology lacks empirical validation.

Independent Verification and Validation

Military projects following military specifications such as 2167A and 498 are constrained to use outside specialists for reviewing major deliverables and indeed for independent testing as well. Independent verification and validation is normally termed "IV&V," since the whole name is too long for convenience. Although the concept seems sound enough, there is insufficient empirical evidence that military projects have significantly higher quality or reliability than ordinary systems software projects of the same size and complexity, which do not use IV&V. Mission-critical military projects seem to have reasonably high levels of quality and reliability, however.

There are also commercial IV&V companies that offer testing and verification services to their clients. Since testing is a technical specialty, these commercial testing houses often do a better job than the informal "amateur" testing carried out by ordinary programmers who lack training in test methods.

Professionally Staffed Testing Departments

In almost every human activity, specialists tend to outperform generalists. That is decidedly true of software testing, which is a highly sophisticated technical specialty if it is to be performed well. As a rule of thumb, companies that have testing departments staffed by trained specialists will average about 10 to 15 percent higher in cumulative testing efficiency than companies that attempt testing by using their ordinary programming staffs. Normal unit testing by programmers is

seldom more than 25 percent efficient, in that only about 1 bug in 4 will be found, and most other forms of testing are usually less than 30 percent efficient when carried out by untrained generalists. A series of well-planned tests by a professionally staffed testing group can exceed 35 percent per stage, and 90 percent in overall cumulative testing efficiency. Microsoft is one of the strongest enthusiasts for professional testing groups, which are necessary for products such as Windows Vista, Office, and other complex applications.

In recent years automated testing has been improving in terms of tools and thoroughness. Automated testing tools analyze source code and create test cases for all major paths and features. The advantages of automated testing include speed and consistency. However automated testing cannot find missing features, nor can it find defects in the code introduced due to earlier defects in requirements or design. For example automated testing would not have found the famous "Y2K" problem because the applications which contained that problem had two-digit dates fields on purpose due to incorrect user requirements.

Clean-Room Development

The concept of clean-room development as described by Dr. Harlan Mills and his colleagues at IBM is based on the physically clean rooms used to construct sensitive electronic devices, where any impurity can damage the process. For software, clean-room development implies careful control of all deliverables prior to turning the deliverable over to any downstream worker. The clean-room method also envisions the usage of formal specification methods, formal structural methods, and also proofs of correctness applied to critical algorithms. The clean-room concept is new, and it is intuitively appealing. However, empirical evidence is scarce, and no large-scale successes have yet been reported. The federal and military support groups of the IBM corporation have been the most active U.S. adherents to the clean-room concept.

Prototyping

Prototyping is more of a defect prevention than a defect removal technique, although the impact on a certain class of defects is very high. Projects that use prototypes tend to reach functional stability earlier than those that do not. Therefore, prototyped projects usually add only 10 percent or less of their functions after the requirements phase, whereas unprototyped projects tend to add 30 percent or more of their functions after the requirements phase. Since the defect rates associated with late, rushed functions are more than twice as high as normal averages, it can be stated that prototyping is a very effective defect prevention method. It is most successful for midsize projects—between about 100 and 1000 function points. For very small projects, the project itself serves the same

purpose as the prototype. For very large projects, it is not usually possible for the prototype to reflect all of the functionality.

Joint Application Design (JAD)

The joint application design (JAD) methodology originated in Canada at the IBM Toronto programming laboratory, and it has now spread throughout the world. More than a dozen companies now teach JAD-like techniques and offer consulting services to facilitate JAD sessions. Like prototyping, the JAD technique is more of a defect prevention method than a defect removal method. It is an improved method of deriving the requirements for a software project by means of structured, joint sessions of user representatives and the software design team. The sessions follow a structured format, and they are facilitated by a trained moderator. As with prototypes, projects using the JAD approach reach functional stability earlier, and they usually add less than 10 percent of their final functionality as afterthoughts. It should be noted that the JAD method is synergistic with prototyping, and the two techniques together seem to be a very effective combination.

The JAD method originated in the domain of management information systems and assumes that there will be an available set of users who can participate. Thus, for projects where users are either not available or where there may be thousands of them (as with spreadsheets or word processing software), the JAD method may not be useful.

Software Complexity Analysis

There is a strong and direct correlation between code complexity and software defect rates. Since code complexity can be directly measured by means of a dozen or so commercially available tools, a useful first step in improving quality is to analyze the complexity of existing software. The programs or components of systems that have dangerously high complexity levels can then be isolated and corrected.

Error-Prone Module Removal

In all of the large software systems that have yet been studied by major corporations, including IBM, EDS, Raytheon, AT&T, and Hewlett-Packard, the bugs or defects have not been randomly distributed through the systems. Instead, they have clumped in surprisingly localized sections termed "error-prone modules." As a rule of thumb, about 5 percent of the modules in a large system may receive almost 50 percent of the total defect reports. Once such modules have been identified, the problems can often be corrected. Although high complexity and poor coding practices are often associated with error-prone modules, there are other factors too. Often error-prone modules are those added to the system late, after

requirements and design were nominally finished. In many cases, the late arrival of the code led to the embarrassing situation of having no test cases created for it, since the test and quality assurance groups did not know the code had been added!

Restructuring, Reverse Engineering, and Reengineering

In 1985, a new sub-industry started to form; it comprised companies that offered various geriatric services for aging software. The first such services to be offered were automatic or semiautomatic restructuring of COBOL source code. The general results were favorable, in that the restructured code was easier to modify and maintain than the previous unstructured versions. However, restructuring did not change the size of the modules, so manual remodularization was often used as an adjunct to reduce the sizes of the modules down to manageable levels (fewer than 500 source code statements). Here, too, the results have generally been favorable. Some companies, indeed, have experimented with using the restructuring techniques on new applications since the standardized results make it easier for programmers to maintain applications other than those they wrote themselves. Here, too, the results are favorable.

The impact of restructuring and remodularization on software quality is significant, but it is largely indirect. Restructuring and remodularization will find some bugs and dead code, of course, but the primary benefit is a reduction in the effort associated with defect repairs and enhancements once the transformation is finished. More recently, such companies as Computer Aid Inc.; Relativity Technologies, and Shoulders Corporation among others can analyze existing source code and create a synthetic specification from latent information present in the code itself. This process, termed "reverse engineering," is based on the thesis that the original specifications have long since fallen into decay, leaving the enterprise with nothing but aging source code and human knowledge to work from when updating applications. Obviously, aging source code is not a good starting point for major enhancements or updates of the application.

The next step beyond reverse engineering is reengineering, or semiautomatic conversion of an aging application into a modern one, perhaps by using different languages and running on a different platform. The concepts of reverse engineering and reengineering are appealing, but not enough empirical data exists at present to state whether they will be truly successful.

The Malcolm Baldrige Awards

One does not ordinarily associate governments with quality, yet the government-sponsored Baldrige awards for quality have become a major

factor in U.S. business and hopefully will remain so in the future. To win a Baldrige award, it is necessary to not only achieve high quality in real life but also measure the results in a statistically valid way. Associated with the Baldrige concept are seven pillars, including measures, strategy, customer focus, human skill enhancement, process, and results. Thus far, the Baldrige awards have stirred up more action in the engineering world than in the software world, but several leading software producers are actually attempting a run at one of these prestigious awards. All of us in the software industry should wish them luck, since winning the award would benefit the entire industry. For the Baldrige awards to stay effective, great care must be exercised in whom they are given to. Nothing can damage the impact of such an award more quickly than giving to an enterprise that does not truly deserve it!

Measuring Software Defect Removal

The word "quality" has so many possible definitions for software that it has tended to become an intangible term. As already discussed, it can mean user satisfaction, conformance to requirements, or any combination of a host of concepts that end with "ility" such as reliability, maintainability, and portability. These abstract definitions can be measured subjectively, but the measurements are often vague and unsatisfying.

There is one major aspect of quality that can be measured in a tangible and convincing way. The hard, tangible aspect of quality is the measurement of defect removal. Not only can defect removal be measured with high precision, but this metric is one of the fundamental parameters that should be included in all corporate software measurement programs.

Projects that perform well in terms of defect removal often perform well with such other aspects of quality as conformance to requirements and user satisfaction. On the other hand, projects with inadequate defect removal are seldom successful in the other aspects of quality either.

Reporting Frequency for Defect Measurements

The natural frequency for defect measurement is monthly. That is, every month a standard report that shows all of the defects found in the course of the prior month should be produced. The monthly report should contain at least the following sections:

- Defects found by reviews and inspections
- Defects found by testing
- Defects found and reported by users

More sophisticated defect reports, such as those produced by large commercial software producers like IBM, are capable of showing this additional information:

- Defects found by product and product line
- Defects found by geographic region (country, state, city, etc.)
- Defects found by customer
- Defect found by industry (banking, insurance, etc.)

All of this data can be condensed and summarized on an annual basis, but the fundamental reporting frequency should be monthly.

Turning on the Defect Count "Clock"

Productivity data can be reconstructed from the memories of the project teams. Indeed, such reconstructed data is sometimes better than the data that comes out of a tracking system because it lacks the errors of the typical tracking system.

Quality data, on the other hand, cannot be reconstructed. It must be measured, and it must be accumulated for quite some time before enough of it is available to be useful. If you start counting today, it can be more than a year before you accumulate enough data to really understand your company's quality levels. However, if you do not start today, or at least fairly soon, you may never understand your company's quality levels. Indeed, if your competitors measure quality and your company does not, they may very well put your company out of business before you can even know why! There are three natural starting places for beginning to count defect levels:

- As early as possible, such as during requirements
- When testing begins
- When customers begin to use the software

From consulting engagements with several hundred clients in large corporations, the norm (among those who measure defects at all) is, unfortunately, to begin measuring when customers begin to use the software. That is not as it should be. The true starting place, in order to gain insights that can make major improvements, is to start as early as possible. IBM, for example, starts its defect "clock" during the requirements phase and keeps it running through the entire lifecycle. This kind of data is invaluable for insights and subsequent defect prevention.

Late starts with defect counting tend to obscure problems with requirements and design, which typically comprise the bulk of all problems for major systems. The earlier the counting begins, the better the insights

that can be gained. Early starting, of course, involves cultural changes within a company and some careful preparation in order to make defect measures a useful corporate tool.

Some defect removal activities such as desk checking and unit testing are "private" in the sense that the work is carried out by individuals without any supervision or group contact. This raises the issue of gaps in defect data. A solution to this problem was found by IBM and ITT. Not every programmer or analyst participated, but requests were made for volunteers to record "private" defects on a sample of projects. If an organization has 1,000 programmers or software engineers, a sample of 50 volunteers is probably enough to gain useful statistics on the efficiency of personal defect removal activities. However, when samples are used, there is always a chance that the volunteers are "top guns" who know they do a good job, so the results may be higher than they would be with 100 percent participation.

Needless to say, the data submitted by the volunteers must not be used for harm, such as appraisals. The purpose of asking for volunteers is primarily to ascertain the average defect removal efficiency of otherwise immeasurable activities such as unit testing. Readers may find it of interest that the average efficiency reported by volunteers is only about 25 percent for unit testing. After unit testing is over, subsequent test stages plus uses-reported defects were about four times more numerous than those found during unit testing. Even top-guns seldom go about 50 percent removal efficiency during unit test.

Definition of a Software Defect

A software defect is simply a bug that if not removed would cause a program or system to fail or to produce incorrect results. *Note:* The very common idea that a defect is a failure to adhere to some user requirement is unsatisfactory because it offers no way to measure requirements defects themselves, which constitute one of the larger categories of software error. Recall that the famous Y2K problem did not originate as a coding bug. The Y2K problem originated as a specific user requirement to conserve space by using only two digits for dates.

Defects can have five different origins in a software project, and it is normal to assign an origin code to each discovered defect. Examples of the five defect origins include requirements errors, such as accidentally leaving out a necessary input screen; design defects, such as a mistake in an algorithm; coding defects, such as branching to a wrong location; documentation defects, such as giving the incorrect command for starting a program; and bad fixes, such as making a fresh mistake while fixing a previous bug.

When measuring defect removal, it is normal to assign the responsibility for determining whether or not a given problem is a defect to

the quality assurance manager. If the project does not have a quality assurance manager assigned, the project manager or supervisor should assign the origin code after the defect has been technically analyzed.

Some defects will be reported more than once, especially when the software has many users. Although duplicate reports of the same bug are recorded, the normal defect removal metric is based on a "valid unique defect," or the first report of any given bug. From time to time, defect reports that are submitted turn out, upon examination, to be either user errors or problems that are external to the software itself. These are termed "invalid defect reports," and they are excluded from the software defect measures. They are, however, recorded for historical and statistical purposes. There are also various plateaus of defect severity. The four-point severity scale used by IBM for software looks like this:

Severity 1	System or program inoperable
Severity 2	Major functions disabled or incorrect
Severity 3	Minor functions disabled or incorrect
Severity 4	Superficial error

An initial defect severity code is often assigned by the client or user of the software such as a tester. However, since users have a natural tendency to code most defects severity 1 or severity 2, the final severity level is usually determined by a quality assurance manager or an objective source. In summary, defects are counted as totals and then sorted into subcategories as follows:

- Unique defect reports versus duplicate defect reports
- Valid defect reports versus invalid defect reports
- Defect reports by origin:
  - Requirements defects
  - Design defects
  - Coding defects
  - Documentation defects
  - Bad fixes
- Defect reports by severity:
  - Severity 1 defects
  - Severity 2 defects
  - Severity 3 defects
  - Severity 4 defects

TABLE 5-3 Distribution of Software Defect Origins and Severities

	Severity Level				
Defect Origin	1 (%)	2 (%)	3 (%)	4 (%)	Total (%)
Requirements	5.0	5.0	3.0	2.0	15.0
Design	3.0	22.0	10.0	5.0	40.0
Coding	2.0	10.0	10.0	8.0	30.0
Documentation	0.0	1.0	2.0	2.0	5.0
Bad fixes	0.0	2.0	5.0	3.0	10.0
Total defects	**10.0**	**40.0**	**30.0**	**20.0**	**100.0**

If you measure software defects using these concepts, a typical distribution of valid unique defects for a medium to large system might look like the data shown in Table 5-3.

Note the interesting skew in the distribution in Table 5-3: For critical severity 1 defects, requirements errors are dominant; for severity 2 defects, design errors are dominant. Coding errors are in second place overall, but they do not comprise the largest source of errors except for the comparatively unimportant severity 4 category. Of course, the table reflects the results of medium-size to large-size systems of 50,000 COBOL statements and greater. For small programs of 5000 COBOL statements or less, coding errors would comprise more than 50 percent of the total error density. In addition to the valid unique defects that the table reflects, you can expect an invalid defect for each valid defect. If you have multiple users of or clients for your software, you can expect one duplicate defect report for every five users.

Measuring Defect Removal Efficiency

Once a satisfactory encoding scheme for your basic defect counts has been adopted, the next stage is to measure the defects found by each removal activity. Typical MIS projects go through pretest requirements and design reviews, partial code walk-throughs, and from two to five testing steps depending on the size and nature of the project. A typical MIS defect removal series might look like this:

1. Requirements review
2. Design review
3. Desk checking by individual programmers (not measured)
4. Code walk-through
5. Unit testing by individual programmers (not measured)

6. Integration testing of entire program or system
7. Acceptance testing by users

At this point, it should be noted that tasks 3 and 5 (desk checking and unit testing) are often not measured, since they are carried out by the individual programmers themselves. (However, some programmers have volunteered to record defects found during these activities, simply to complete the data necessary to evaluate a full defect removal series.) Assuming that you will measure only the public defect removal activities and not the private ones, your data might look like the information in Table 5-4 for a 50,000-source-statement COBOL application that also contains some 500 function points. Note that Table 5-4 makes the simplifying assumption that it takes 100 COBOL statements to implement one function point. The average value is actually 105 statements per function point, but for tutorial purposes an even amount was selected.

At this point, you will not yet be able to complete the final set of quality measures. That requires collecting defect data reported by actual users of the project. You will normally collect data from users for as long as the project is in use, but after one year of production and then annually for each additional year of use, you will be able to complete a very significant analysis of the defect removal efficiencies of your review, inspection, and test series. Assume that the first year of production by your users generated the number of defects shown in Table 5-5. You will now be able to calculate one of the most important and meaningful metrics in all software: your defect removal efficiency.

The general formula for defect removal efficiency is quite simple: It is the ratio of bugs found prior to delivery by your staff to the total number of bugs. The total number of bugs is found by summing bugs you discovered with the bugs your clients discovered in a predetermined time period. In this example, the bugs that your staff removed before delivery totaled 1,125. The bugs found by your users in the first year

TABLE 5-4 Software Defect Distributions by Removal Step

Removal Activity	Defect Found	Defects per KLOC	Defects per Function Point
Requirements review	125	2.5	0.25
Design review	250	5.0	0.50
Code inspection	500	10.0	1.00
Integration testing	150	3.0	0.30
Acceptance testing	100	2.0	0.20
Total defects	**1125**	**22.5**	**2.25**

TABLE 5-5 User-Reported Defects from One Year of Production Runs

Activity	Defects Found	Defects per KLOC	Defects per Function Point
First-year user defect reports	400	8.0	0.8

totaled 400. If you add those two values together, the sum is 1,525. That is equal to 30.5 bugs per KLOC, or 3.05 bugs per function point, which are middle-range values for COBOL applications of this size. (Leading-edge applications will total fewer than 15 bugs per KLOC, whereas the real disasters may go above 75 bugs per KLOC.)

Cumulative defect removal efficiency is defined as the percentage of bugs found by the development team before a software product is delivered to its users. Efficiency cannot be measured until after a predetermined time period has gone by, and the first year of use is a convenient interval for the first calculation. Of course, not all bugs will be found after only one year, so recalculation after two or even three years would also be useful.

In this example, finding 1,125 bugs out of 1,525 amounts to only 73.8 percent, which is a typical but mediocre result. The normal defect removal efficiency for MIS projects is somewhere between 50 and 75 percent when measured. Leading-edge MIS companies will find more than 90 percent of the bugs, whereas leading-edge commercial and military software producers will find more than 95 percent of the bugs.

Defect removal efficiency is a very important metric, because raising efficiency above 90 percent can improve quality, productivity, and user satisfaction at the same time. Indeed, a cumulative total of 95 percent appears to be one of the most powerful node points in all of software engineering, since schedules, quality, effort, and user satisfaction all tend to approach optimum levels when this target is achieved.

Not only can the cumulative defect removal efficiency of a series of removal steps be measured; it is also possible to measure the individual net efficiency of each step. In real life, each defect removal step will have a differential efficiency against each source of defect. Table 5-6 shows typical efficiencies against four defect origins for a variety of reviews, inspections, and tests. The efficiencies in Table 5-6 were calculated from empirical studies, although significant variations can and do occur.

The table of defect removal efficiencies actually matches real-life results fairly closely. Coding defects are the easiest to remove, and hence they have the highest overall efficiencies against them. Requirements defects are the most troublesome, and normally they have the lowest efficiencies. Although documentation defects are not intrinsically difficult

TABLE 5-6 Defect Removal Efficiencies by Defect Origin

	Defects by Origin (%)			
Removal Step	Requirements	Design	Coding	Documentation
JAD	50	25	10	15
Prototyping	40	35	35	15
Requirements review	40	15	0	5
Design review	15	55	0	15
Code inspection	20	40	65	25
Subtotal	75	85	73	50
Unit testing	1	5	20	0
Function testing	10	15	30	5
System testing	10	15	35	20
Field testing	20	20	25	25
Subtotal	35	45	75	43
Cumulative	**87**	**92**	**94**	**72**

to remove (journals such as *Scientific American* exceed 99.5 percent in net efficiency), most software projects simply do not use high-efficiency techniques such as copyediting by professional editors or proofreading by pairs of readers.

In terms of the removal methods themselves, detailed code inspections are the most efficient form of defect removal yet measured, whereas most testing steps are less than 35 percent efficient. That is, most forms of testing find less than one bug out of every three bugs that actually exist. Once you have measured a reasonably large sample of projects (more than 50), you will be able to use the data you've collected to make very accurate quality and reliability estimates.

Some of the implications of Table 5-5 deserve discussion. First, given the low efficiencies of testing, it is obviously impossible to achieve high levels of cumulative efficiency without up-front activities such as prototyping, reviews, or inspections. To be blunt, companies that only perform testing will never go much beyond 75 percent in cumulative defect removal efficiency; that is, they will deliver at least one out of every four bugs to their clients.

Second, testing ranges from moderate to useless in its ability to come to grips with front-end defects such as requirements and design. Indeed, it is surprising that testing can find requirements defects at all. Therefore, it is imperative to utilize the high-efficiency pretest activities in order to control these major sources of system problems.

Given the magnitude of defects associated with requirements and design, it is obvious that these important defect sources must be included in quality control planning. Even a cursory inspection of removal efficiencies

demonstrates that JADs, prototyping, and up-front inspections are logically necessary to control front-end bugs.

Third, there are some special calculations needed to include bad fixes, or bugs accidentally introduced as by-products of fixing previous bugs. The total quantity of bad fixes averages about 5 to 10 percent, and it will be directly related to the complexity of the work product being repaired.

Fourth, given the low average efficiencies of most removal steps, it is obvious that to achieve a high cumulate efficiency, it will be necessary to use many steps. Commercial and military software producers, for example, may include as many as 20 to 25 discrete removal activities.

This is one of the key differences between systems software and MIS projects. MIS projects normally utilize only five to seven different kinds of removal, and they seldom utilize the rigorous inspection process.

Fifth, serious quality control requires a synergistic combination of multiple techniques, with each removal step aimed at the class of defects for which its efficiency is highest. Figure 5-3 illustrates a typical synergy among defect removal methods. The moral of the figure is quite simple: Choose the combination of defect removal steps that will achieve the highest overall efficiency for the lowest actual costs.

Here is a final point on defect measurement and defect removal in general. Finding and fixing bugs has been among the most expensive, if not the most expensive, activity for software since the industry began. Companies that do measure quality and defect removal have a tremendous competitive advantage against those that do not. Historically, certain industries such as computers and telecommunications paid serious attention to both hardware quality and software quality and introduced quality measurement programs, in some cases, more than 30 years ago. These industries have been capable of withstanding overseas competition much better than such industries as automotive construction,

	Requirements Defects	Design Defects	Code Defects	Document Defects	Performance Defects
Reviews/ Inspections	Fair	Excellent	Excellent	Good	Fair
Prototypes	Good	Fair	Fair	Not Applicable	Good
Testing (all forms)	Poor	Poor	Good	Fair	Excellent
Correctness Proofs	Poor	Poor	Good	Fair	Poor

Figure 5-3 Defect removal methods

which learned about quality control far too late. Quality control is the key to corporate survival in the 21st century, and measurement is the key to quality control. Now is the time to start!

Finding and Eliminating Error-Prone Modules

In the late 1960s Gary Okimoto, a researcher at the IBM Endicott laboratory, carried out a study in which he looked at the distribution of defects within the OS/360 operating system. To his surprise, and to the surprise of everyone else who knew of the study, the defects were not smoothly or randomly distributed throughout the modules of the system. Instead, they clumped in a small number of very buggy modules. Some 4 percent of the modules contained 38 percent of the errors in the entire operating system. This study is perhaps the original discovery of the error-prone module concept. In any case, it is the first such study known to this author.

The study was replicated against other IBM software products, and it invariably produced striking results. For example, when west coast researchers at IBM's Palo Alto laboratory carried out a similar study on the IMS database product, some 57 percent of the errors were concentrated in only 31 modules, which constituted about 7 percent of the modules of the product and about 12 percent of the code.

Other companies, including AT&T, ITT, and Hewlett-Packard, have come to regard error-prone module analysis as a normal aspect of quality control. So far as can be determined from all of the studies yet carried out, error-prone modules are a very common phenomenon and will occur in all large systems unless deliberate corrective steps are taken. Fortunately, a number of corrective steps that can eliminate error-prone modules from software products are available. The most basic step is to measure defects down to the level of modules in inspections, testing, and maintenance. Any module with more than about ten defects per KLOC or one defect per function point is a candidate for a full review to explore its status. From the error-prone modules that have been explored in depth, a number of causative factors have been identified. The most significant among them are

- Excessive schedule pressure on the developers
- Excessive complexity that is due to either:
  - Failure to use proper structured techniques
  - Intrinsic nature of the problem to be encoded
- Excessive size of individual modules (>500 statements)
- Failure to test the module after code was complete

The first three factors are intuitive and even self-explanatory, but the fourth factor is something of a surprise. Even more surprising is the high frequency with which this phenomenon occurs. Normally, error-prone modules that have not been tested are those that were created very late in a product's development cycle, often while testing was nearing completion. These modules were rushed into production without bothering to update the specifications or the test case libraries! The solution to this problem is a rigorous module promotion process, which includes "locked" master copies of a software product with no way to add modules unless careful quality control procedures are followed.

Using Metrics to Evaluate Test-Case Coverage

As of 2008, there are perhaps a dozen commercially available tools that can be used to assist in measuring test-case coverage. These tools normally trace the execution sequence of code when it is executing, and they can isolate and identify sequences that are not executed while testing. Although such tools are very useful, it should be clearly realized that there is a striking discontinuity between test coverage and testing efficiency. Even though a particular test step, such as function testing, for example, causes the execution of more than 90 percent of the instructions in a program, that does not imply that 90 percent of the bugs will be found. Indeed, normally less than 30 percent of the bugs will be found by function testing. The reason for the discontinuity is fairly straightforward: Just because instructions have been executed does not guarantee that they are doing what was actually intended.

It should also be noted that there is no guarantee that the test cases themselves are correct. Indeed, a study at IBM's Kingston laboratory in the middle 1970s found that test cases often had a higher error content than the products for which the test cases were constructed! Another surprising finding was that about one-third of the test cases appeared to be duplicates, which added nothing to testing rigor but quite a bit to testing costs.

Not only are errors in test cases seldom studied, but studying them is difficult. Normally test cases are not counted with function point metrics—or with any metrics for that matter. Some test cases are small enough so that they would need "micro function points" since they would be less than 15 function points in size.

Further, test case errors are not studied by normal quality assurance groups and are seldom studied by test groups either. Essentially no one in a company is routinely assigned to examine test cases for either correctness or for redundancy with other similar test cases.

Anecdotal evidence leads to the tentative conclusion that purging test libraries of incorrect and redundant test cases might speed up testing by 20 percent.

The bottom line is that test case errors or bugs are numerous and are extremely resistant to either defect prevention or defect removal activities as of 2008.

Using Metrics for Reliability Prediction

Quality and reliability are logically related topics. Surprisingly, they are often decoupled in the software engineering literature. Many articles on quality do not mention reliability, and many articles on reliability do not mention quality.

The reliability domain has built up a large body of both theoretical and empirical models and a number of supporting metrics such as mean time to failure (MTTF) and mean time between failures (MTBF). A good overview of the reliability domain is provided by Musa, Iannino, and Okumoto.

Because the topics of quality and reliability are so often separated, it is useful to show at least the crude correlations between them. Table 5-7 is derived from empirical data collected by the author at IBM in the 1970s for systems software written in Assembler. The data was collected from unit, function, component, and system test runs.

Similar results occur when delivered defects are measured with function point metrics. Since the current U.S. average circa 2008 is about 0.75 defects per function point, that correlates to an MTTF of roughly 4 to 24 hours before a defect is encountered.

Software delivered with more than about 1.2 defects per function point will run for only a few minutes without encountering a defect or stopping entirely. To achieve continuous around-the-clock operation without interruption by defects, delivery rates need to be below about 0.05 defects per function point.

TABLE 5-7 Relation between Defect Levels and Reliability

Defect Levels in Defects per KLOC	Approximate Mean Time to Failure (MTTF)
More than 30	Less than 2 min
20–30	4–15 min
10–20	5–60 min
5–10	1–4 h
2–5	4–24 h
1–2	24–160 h
Less than 1	Indefinite

Measuring the Costs of Defect Removal

Since the computing and software era began, the largest single identifiable cost element has been that of finding and fixing bugs. It is astonishing therefore, that so few companies have measured this cost with accuracy. It is also embarrassing that attempts to measure defect removal have so often been paradoxical in economic terms.

The Paradox of "Cost per Defect"

For perhaps 40 years, the most common unit of measure for assessing defect removal has been "cost per defect." At least 50 referenced journal articles and perhaps half a dozen software management books have contained the statement, "It costs 100 times as much to fix a bug in production as it does during design." The concept of cost per defect is to accumulate the total costs associated with defect removal for a particular activity class such as testing and then divide the cost by number of defects found. Thus, if a particular test step found 100 defects and cost $2,500 to carry out, the cost per defect would be $25.

Unfortunately, as it is commonly calculated, cost per defect is one of the worst and most foolish metrics ever devised. It contains a built-in mathematical paradox that causes it to penalize high quality and to do so in direct proportion to the level of quality achieved. The best programs and systems with the fewest defects will look the worst and the most decrepit and bug-ridden applications will look the best!

To understand the nature of the problems with cost per defect, it is necessary to look at the detailed economic picture of removing defects in software. Every software defect removal activity, regardless of whether it is a review, an inspection, a test, or maintenance, will have three basic cost elements associated with it:

- Preparation costs
- Execution costs
- Repair costs

Preparation costs consist of the things that must be performed prior to carrying out a specific form of defect removal. For example, prior to testing a program, it is necessary to create test cases. Prior to carrying out a design review, it is necessary to read the specifications. Preparation costs are essentially fixed; they will remain comparatively constant regardless of how many bugs are present. Thus, even for zero-defect software it will still be necessary to write test cases and read the descriptive materials.

Execution costs are associated with the actual events of the defect removal activity. For testing, execution consists of running the application against the set of prepared test cases. For reviews and

inspections, execution is the cost of actually holding the review and inspection sessions. Execution costs are not fixed costs, but they are somewhat inelastic. That is, the cost of executing a review or test is only partly associated with the number of defects present. The bulk of the costs is associated with the mere mechanics of going through the exercise. Thus, even zero-defect applications will accumulate some costs for carrying out reviews and inspections and running test cases, and even zero-defect applications can have maintenance costs. That is surprising but true, since user errors and invalid defects will probably be reported against the product.

Defect repair costs are those associated with actually fixing any bugs or defects that were identified. Defect repairs are true variable costs, and they are the only cost elements that will drop to zero if zero bugs are present in a program or system. When quality improves, the cost per defect will tend to get higher rather than lower, since the fixed and inelastic preparation and execution costs will become progressively more important. Table 5-8 illustrates the paradox of cost per defect in the case of high quality for two applications in Ada, both of which were 150 function points, or 10,000 source statements, in size. Assume $5,000 per person-month as the normal labor rate.

Consider the economic fallacy of cost per defect. The true costs of defect removal declined from $35,000 to only $10,000, or better than 70 percent. The costs of the critical defect repair component declined by a full 10 to 1 between the low-quality and high-quality examples. Yet while the real economics of defect removal improved tremendously, the "cost per defect" skyrocketed from $70 to $2,000! It is obvious that if the high-quality product had achieved zero-defect status, there still would have been expenses for preparation and execution. In this case, the cost per defect would have been infinity, since there would be tangible costs divided by zero defects!

TABLE 5-8 The Paradox of Cost per Defect for Quality Measures

	Low-Quality Ada Application	High-Quality Ada Application
Size in KLOC	10	10
Size in function points	150	150
Defects found	500	5
Preparation costs	$5,000	$5,000
Execution costs	$25,000	$2,500
Defect repairs	$25,000	$2,500
Total removal cost	$35,000	$10,000
Cost per defect	$70	$2,000
Cost per function point	$233.33	$66.66

TABLE 5-9 Preparation, Execution, and Defect Repair Effort

Removal Step	Preparation (hours per function point)	Execution (hours per function point)	Repairs (hours per defect)
JAD	0.15	0.25	1.00
Prototyping	0.25	1.00	1.00
Requirements review	0.15	0.25	1.00
Design inspection	0.15	0.50	1.50
Code inspection	0.25	0.75	1.50
Unit testing	0.50	0.25	2.50
Function testing	0.75	0.50	5.00
Systems testing	1.00	0.50	10.00
Field testing	0.50	0.50	10.00

Plainly, cost per defect is not a suitable metric for economic studies associated with software quality. Here, too, function points are much better for economic purposes. Since both versions of the application contained 150 function points, it can be seen that the metric defect removal costs per function point matches economic assumptions perfectly. For the low-quality version, $233.33 per function point was spent for defect removal; for the high-quality version, only $66.66 was spent.

Collecting Defect Removal Data

Although cost per defect is not valid in the way it is normally used—with preparation and execution simply clumped with repairs, the metric can be explored by time-and-motion studies and utilized in conjunction with functional metrics. Table 5-9 shows the effort associated with common forms of defect removal in terms of preparation, execution, and defect repairs. Preparation and execution are measured in work hours per function point, and repair (and only repair) is measured in hours per defect repaired. The final total of effort is normalized to person-hours per function point.

For economic study purposes, it is necessary to convert all of the final effort data into a per function-point basis. For example, assume that you are carrying out a design inspection with five participants on a project that totals 100 function points and in the course of the inspection you find 50 bugs that need repairs. From the data in Table 5-9, the preparation effort would amount to 25 hours and the inspection sessions would amount to 50 work hours (but only 10 clock hours). The 50 bugs would require 75 hours. Thus, the entire process is totaled as follows:

Preparation	Execution	Repair		
25 h +	50 h +	75 h	=	150 h

The overall inspection, including preparation, execution, and repair, netted out to 1.5 work hours per function point. In this example, the effort per bug would be 3.0 hours per bug for the sum of preparation, execution, and repair. Consider the same basic example, only this time assume that only 10 bugs were found:

Preparation	Execution	Repair		
25 h +	50 h +	15 h	=	90 h

In this second situation, the overall inspection process amounted to only 0.9 hours per function point, which is substantially below the previous rate of 1.5 hours per function point. However, the "effort per bug" has now tripled and is up to 9.0 hour per bug! As can easily be seen, the fixed and inelastic costs of preparation and execution tend to distort the per-bug metric so that it becomes paradoxically more expensive as quality improves. A metric that penalizes the very goal you are seeking is hardly suitable for serious economic studies.

The Cost-of-Quality Concept

Phil Crosby's famous book *Quality Is Free,* made popular a cost collection method termed "cost of quality." Although this concept originated in the domain of manufactured products, it has been applied to software projects as well. It is fairly thorough in its approach, and it captures costs associated with rework, scrap, warranty repairs, complaint handling, inspections, and testing. There are three large cost buckets associated with the concept:

- Prevention costs
- Appraisal costs
- Failure costs

Crosby's descriptions of these costs are based on manufactured products, and they are somewhat orthogonal to software. For software purposes, prevention would encompass methods that simplified complexity and reduced the human tendency to make errors. Examples of prevention costs include the costs of training staff in structured design and coding techniques. Joint application design (JAD) also comes under the heading of prevention since it is one of the key by-products of the approach.

Appraisal costs for software include inspections and all forms of testing. For military projects, appraisal costs also include independent verification and validation, or IV&V, as it is called. In one sense, fixing bugs found by testing might be thought of as failure costs, but it seems

more appropriate to count the costs as appraisal elements, since they normally occur prior to delivery of the software to its final customers. Failure costs of software are the costs associated with post-release bug repairs: field service, maintenance, warranty repairs, and in some cases, liability damages and litigation expenses.

Evaluating Defect Prevention Methods

Defect removal deals with tangible things such as bug reports that can be counted fairly easily. Defect prevention, on the other hand, is much harder to come to grips with. This phenomenon is true of other human activities as well. For example, the cost and efficacy of preventive medicine is much more uncertain than the cost and efficacy of treating conditions once they occur. The term "defect prevention" means the overall set of technologies that simplify complexity and minimize a natural human tendency to make errors while performing complex tasks. Examples of defect prevention technologies include prototypes, JAD sessions, graphic design methods, formal architectures, structured techniques, and high-level languages. Indeed, some defect removal methods, such as inspections, are also effective in terms of defect prevention because participants will spontaneously avoid making mistakes that they observe during the inspection sessions. Figure 5-4 illustrates some of the synergies among defect prevention methods.

	Requirements Defects	Design Defects	Code Defects	Document Defects	Performance Defects
JADs	Excellent	Good	Not Applicable	Fair	Poor
Prototypes	Excellent	Excellent	Fair	Not Applicable	Excellent
Structured Methods	Fair	Good	Excellent	Fair	Fair
CASE Tools	Fair	Good	Fair	Fair	Fair
Bluprints & Reusable Code	Excellent	Excellent	Excellent	Excellent	Good
QFD	Good	Excellent	Fair	Poor	Good

Figure 5-4 Defect prevention methods

TABLE 5-10 Long-Range Improvements in Defect Prevention and Defect Removal

	1985	1986	1987	1988	1990	1995
Potential defects per function point	5.0	4.5	4.0	3.5	3.0	2.5
Removal efficiency (%)	70	80	85	90	95	99
Delivered defects per function point	1.5	0.9	0.6	0.35	0.15	0.025

Long-Range Monitoring of Defect Prevention and Defect Removal

After quality measurement programs get underway, they can be used to monitor long-term effects over a period of many years. For example, Table 5-10 illustrates a ten-year trend by a major computer manufacturer for both applications and systems software. The original data has been simplified and rounded to demonstrate the trends.

The overall combination of defect prevention and defect removal improvements led to a full order of magnitude improvement in defects as received by clients. It cannot be overemphasized that both defect prevention and defect removal need to be part of an effective quality program: Neither is sufficient alone; together, they are synergistic. Finally, it cannot be overemphasized that without measurements carefully carried out over many years, none of the improvements would be either possible or visible.

Measuring Customer-Reported Defects

For many years, relations between defects actually delivered to customers and the number of defects found and reported back by customers has been known only to perhaps three or four major computer and telecommunication manufacturers. The relations are not intrinsically mysterious, but only a few companies had measurement systems that were sophisticated enough to study them. The fundamental problem posed by the situation is this: If you deliver a software product to a certain number of customers and it has 100 latent defects still present at the time of delivery, how many of those defects will be found and reported back by the customers in the first year? How many in the second year? The answers obey two general rules:

1. The number of defects found correlates directly with the number of users; the more users, the greater the number of defects found.
2. The number of defects found correlates inversely with the number of defects that are present; the more defects, the fewer the defects found.

TABLE 5-11 Relations Between Users and First-Year Defect Reports

	Case 1	Case 2	Case 3	Case 4
Number of users	1	10	100	1000
Defects present	100	100	100	100
Number of defects found in year 1	20	40	75	95
Percent of defects found in year 1	20	40	75	95
Number of defects remaining in year 2	80	60	25	5
Number of years to remove all defects	5	3	2.5	2

The first rule is intuitive and easy to understand; the second is counterintuitive and, at first glance, very hard to understand. It would seem that the more bugs present in software, the greater the number that would be found and reported, but that is not the case. As it happens, rule 2 tends to be in direct conflict with rule 1 for the following reason: Very buggy software cannot or will not be used. Therefore, if your company ships software with more than a certain quantity of bugs latent within it, those bugs will prevent the utilization of the software, will slow down sales, and will in general stretch out the time before the bugs are found and fixed.

Tables 5-11 and 5-12 illustrate these phenomena. They show the impacts of changing numbers of users and then the impact of changing numbers of latent defects with a specific number of users.

Table 5-11 illustrates the phenomenon that somewhere between about 20 and 95 percent of the bugs will normally be found in the first year of production, based on the number of software users. Since only high-volume production by large numbers of users can approach 100 percent in overall removal efficiency, it can clearly be seen that the greater the number of users, the higher the percentage of first-year bug removal.

Table 5-12 illustrates the counterintuitive phenomenon that the number of bugs found by users is inversely related to the number of

TABLE 5-12 Relations Between Defect Quantities and First-Year Defect Reports

	Case 1	Case 2	Case 3	Case 4
Number of users	100	100	100	100
Number of delivered defects	100	200	300	400
Number of defects found in year 1	60	80	100	100
Percent of defects found in year 1	60	40	33	25
Number of defects after year 1	40	120	200	300
Number of years to remove all defects	2	4	5	7

bugs present. The unfortunate truth is that buggy software will not (and sometimes cannot) be put into high-volume production. The initial users of the software will generally have such a bad experience with buggy software that references are impossible. Indeed, if the software package and the vendor are of sufficient magnitude that user groups exist, word of the poor quality will spread like wildfire and will slow down subsequent sales.

The findings in Table 5-12 are counterintuitive, but they are derived from empirical studies with actual projects. The distressingly low rate of only 25 percent in the high-defect Case 4 is due, in such situations, to the fact that users do not trust the products and will not use them except in the most timid and careful fashion. Only software with a low level of initial defects will go into fully productive use fast enough to flush out the latent defects in a short time.

The data in Tables 5-11 and 5-12 implies that a calculation of defect removal efficiency after one year of service will probably be artificially high, since not all of the bugs will have been found in only one year. That is true, and it explains why companies such as IBM will recalculate defect removal efficiency after two or more years.

One other aspect of customer-reported bugs also is counterintuitive and that is the severity levels reported by users. In this case, the reason is that the reported data is simply wrong because of business factors. Table 5-13 shows the numbers of customer-reported bugs for commercial-grade software received by a major computer company.

What is counterintuitive about the table is the astonishing 47 percent shown in the left column for severity 2 levels. At first glance, it would seem that the manufacturer was seriously amiss in quality control. In this case, however, appearances are deceiving. What was happening is that the vendor tried to fix severity 1 bugs within a week and severity 2 bugs within two weeks. Severity 3 bugs were fixed within about a month, and severity 4 bugs were not fixed until the next

TABLE 5-13 Severity Levels of Customer-Reported Defects

Severity Level	Percent of Defect Reports	Probable Distribution (%)
Severity 1 (system unusable)	3	3
Severity 2 (major function disabled)	47	15
Severity 3 (minor function disabled)	35	60
Severity 4 (no functions disabled: superficial problem)	15	22
Total	**100**	**100**

normal release. Obviously, every customer wanted his or her bug report processed promptly, so this tended to create an artificial bulge of severity 2 defects. The probable number of real severity 2 bugs was something approaching 15 percent, as shown in the second column.

Measuring Invalid Defects, Duplicate Defects, and Special Cases

Companies that produce commercial software are aware that they not only must deal with real bugs but also must deal with enormous quantities of bugs that are not really the fault of the software against which the bug was reported. For example, in a modern multivendor environment utilizing commercial operating systems, databases, query languages, and user applications, it is quite easy to mistake the origin of a defect. Thus, a customer can send in a bug report for a seeming error in a query language when it might actually be against the user application, the operating system, or something else. As a rule of thumb, software vendors will receive two invalid defect reports for every valid report of an actual bug.

An even more common problem is duplicate reports of the same bug. For example, when WordPerfect release 5.0 was first issued, a fairly simple bug in the installation procedure generated more than 10,000 telephone calls from users on the same day to report the same bug, temporarily shutting down phone service into Utah! The costs and effort to service the duplicate calls far exceeded the costs of actually fixing the bug itself. As a rule of thumb, about 70 percent of all commercial software bugs will be found by more than one user. About 15 percent will be found by many users.

A third problem that should be measured is that of "abeyant defects." This term is used for a bug such that the system repair center cannot re-create it or make the failure happen. Obviously, some special combination of circumstances at the user location is causing the bug, but finding out exactly what it is may require on-site assistance and considerable expense! As a rule of thumb, about 20 percent of commercial software bugs will require additional information because the bugs will not occur at the repair location. The collected costs of processing invalid bug reports, duplicates, and abeyant bug reports can exceed 25 percent of the total cost of fixing real bugs. That is too large an amount to ignore, and certainly too large to leave out of quality and maintenance plans.

Finally, the most expensive single aspect of mainframe commercial software has been field service. Companies, such as IBM, Oracle, and Hewlett-Packard, that send service representatives on-site to customer locations to aid in defect identification and repair can, for some products,

spend more effort and costs on field service than on the entire total of software development and internal maintenance!

Measuring User Satisfaction

Measurement of user satisfaction differs from measurement of software defects in a number of important respects:

- User satisfaction measures are normally annual events; defect measures are normally monthly events.
- User satisfaction data requires active effort in order to collect it; defect reports may arrive in an unsolicited manner.
- The staff that measures user satisfaction is normally not the staff that measures defects.
- The changes required to improve user satisfaction may go far beyond the product itself and may encompass changes in customer support, service policies, and corporate goals.

For commercial software, user satisfaction surveys are normally carried out by the vendor's sales force on an annual basis. The sales personnel will interview their clients and will then report the findings back to the sales organization, which in turn will pass them on to the development groups. It should be noted that many commercial software products have large user associations. In that case, the user satisfaction studies may even be carried out by the user association itself. It should also be noted that for the most successful and widely used commercial software packages, user satisfaction surveys will probably be carried out by one or more of the major industry journals such as *Datamation, ComputerWorld,* and *Software.* These studies tend to include multiple vendors and multiple products at the same time, and they are exceptionally good sources of comparative information. For internal software such as MIS projects, user satisfaction surveys can be carried out by quality assurance personnel (assuming the company has any) or by designated staff assigned by software management.

Contents and Topics of User Satisfaction Surveys

Because of the multiple kinds of information normally included on user satisfaction surveys, it is appropriate to give an actual example of some of the topics discussed. Following are actual excerpts from a user satisfaction survey developed by Software Productivity Research. The survey questions included stress the major topics of the survey, but in order to concentrate on essential factors, they omit basic boilerplate information such as the name of the product and the name of the company.

Excerpts from the SPR User Satisfaction Questionnaire

Nature of product usage?

1. Job-related or business usage only
2. Mixture of job-related and personal usage
3. Personal usage only

Frequency of product usage?

1. Product is used continuously around the clock.
2. Product is used continuously during business hours.
3. Product is used as needed on a daily basis.
4. Product is used daily on a regular basis.
5. Product is used weekly on a regular basis.
6. Product is used monthly on a regular basis.
7. Product is used annually on a regular basis.
8. Product is used intermittently (several times a year).
9. Product is used infrequently (less than once a year).

Importance of product to your job functions?

1. Product is mandatory for your job functions.
2. Product is of major importance to your job.
3. Product is of some importance to your job.
4. Product is of minor importance to your job.
5. Product is of no importance to your job.

How product functions were performed previously?

1. Functions could not be performed previously.
2. Functions were performed manually.
3. Functions were performed mechanically.
4. Functions were performed electronically.
5. Functions were performed by other software.

Primary benefits from use of current product?

1. Product performs tasks beyond normal human abilities.
2. Product simplifies complex decisions.
3. Product simplifies tedious calculations.

4. Product shortens critical timing situations.
5. Product reduces manual effort.
6. Other: _____
7. Hybrid: product has multiple benefits.

Primary benefits? _____

Secondary benefits? _____

Product User Evaluation

Ease of learning to use product initially?

1. Very easy to learn
2. Fairly easy to learn
3. Moderately easy to learn, with some difficult topics
4. Difficult to learn
5. Very difficult to learn

Ease of installing product initially?

1. Little or no effort to install
2. Fairly easy to install
3. Moderately easy to install, with some difficult spots
4. Difficult to install
5. Very difficult to learn

Ease of customizing to local requirements?

1. Little or no customization needed
2. Fairly easy to customize
3. Moderately easy to customize, with some difficult spots
4. Difficult to customize
5. Very difficult to customize

Ease of logging on and starting product?

1. Very easy to start
2. Fairly easy to start
3. Moderately easy to start, with some difficult spots
4. Difficult to start
5. Very difficult to start

Ease of product use for normal tasks?

1. Very easy to use
2. Fairly easy to use
3. Moderately easy to use, with some difficult spots
4. Difficult to use
5. Very difficult to use

Ease of product use for unusual or infrequent tasks?

1. Very easy to use
2. Fairly easy to use
3. Moderately easy to use, with some difficult spots
4. Difficult to use
5. Very difficult to use

Ease of logging off and exiting product?

1. Very easy to exit
2. Fairly easy to exit
3. Moderately easy to exit, with some difficult spots
4. Difficult to exit
5. Very difficult to exit

Product handling of user errors?

1. Very natural and safe error handling.
2. Fairly good error handling.
3. Moderately good error handling, but some caution needed.
4. User errors can sometimes hang up system.
5. User errors often hang up system or stop product.

Product speed or performance in use?

1. Very good performance.
2. Fairly good performance.
3. Moderately good normal performance but some delays.
4. Performance is sometimes deficient.
5. Performance is unacceptably slow or poor.

Product memory utilization when in use?

1. No memory utilization problems with this product.
2. Minimal memory utilization problems with this product.
3. Moderate use of memory by this product.
4. Significant memory required to use this product.
5. Product memory use is excessive and unwarranted.

Product compatibility with other software products?

1. Very good compatibility with other products.
2. Fairly good compatibility with other products.
3. Moderately good compatibility with other products.
4. Significant compatibility problems.
5. Product is highly incompatible with other software.

Product quality and defect levels?

1. Excellent quality with few defects
2. Good quality, with some defects
3. Average quality, with normal defect levels
4. Worse than average quality, with high defect levels
5. Poor quality with excessive defect levels

Product reliability and failure intervals?

1. Product has never failed or almost never fails.
2. Product fails less than once a year.
3. Product fails or crashes a few times a year.
4. Product fails fairly often and lacks reliability.
5. Product fails often and is highly unreliable.

Quality of training and tutorial materials?

1. Excellent training and tutorial materials
2. Good training and tutorial materials
3. Average training and tutorial materials
4. Worse than average training and tutorial materials
5. Poor or unacceptable training and tutorial materials

Quality of user reference manuals?

1. Excellent user reference manuals
2. Good user reference manuals
3. Average user reference manuals
4. Worse than average user reference manuals
5. Poor or unacceptable user reference manuals

Quality of on-screen prompts and help messages?

1. Excellent and lucid prompts and help messages
2. Good prompts and help messages
3. Average prompts and help messages
4. Worse than average prompts and help messages
5. Poor or unacceptable prompts and help messages

Quality of output created by product?

1. Excellent and easy to use product outputs
2. Good product outputs, fairly easy to use
3. Average product outputs, normal ease of use
4. Worse than average product outputs
5. Poor or unacceptable product outputs

Functionality of product?

1. Excellent—product meets all functional needs.
2. Good—product meets most functional needs.
3. Average—product meets many functional needs.
4. Deficient—product meets few functional needs.
5. Unacceptable—product meets no functional needs.

Vendor support of product?

1. Excellent—product support is outstanding.
2. Good—product support is better than many.
3. Average—product support is acceptable.
4. Deficient—product has limited support.
5. Unacceptable—little or no product support.

Status of product versus major competitive products?

1. Clearly superior to competitors in all respects
2. Superior to competitors in many respects
3. Equal to competitors, with some superior features
4. Behind competitors in some respects
5. Clearly inferior to competitors in all respects

Value of product to you personally?

1. Excellent—product is highly valuable.
2. Good—product is quite valuable.
3. Average—product has acceptable value.
4. Deficient—product is not valuable.
5. Unacceptable—product is a loss.

List the five best features of the product:

1. _____
2. _____
3. _____
4. _____
5. _____

List the five worst features of the product:

1. _____
2. _____
3. _____
4. _____
5. _____

List five improvements you would like to see in the product.

1. _____
2. _____
3. _____
4. _____
5. _____

As can be seen, a full user satisfaction survey covers a wide variety of topics. This brings up a significant decision for software vendors:

- Should user surveys be carried out by live interviews?
- Should user surveys be carried out by mail survey?
- Should user surveys include both live interviews and mail surveys?

The pragmatic answer to these options depends upon the number of users, the available staff, and the geographic scatter of the users themselves. Normally, for major mission-critical systems and large mainframe packages, it is desirable to carry out the survey in the form of live interviews. Mail surveys are appropriate under the following conditions:

- The product has more than 1,000 users. (That is the minimum number for which mail surveys will typically generate an adequate volume of responses.)
- The product is distributed through secondary channels, such as distributors or by mail itself.
- The customers are widely dispersed geographically.

A hybrid methodology that utilizes both mail questionnaires and live surveys is a technique frequently employed by computer manufacturers and large software houses. The mail surveys, normally somewhat simplified, will be widely distributed to several thousand customers. Sales personnel will then carry out in-depth user satisfaction surveys with perhaps a 5 percent sample of customers or with customers that meet certain prerequisites. Typical prerequisites might include number of copies of the product acquired by the customer, size of the customer's company, or other salient factors.

Combining User Satisfaction and Defect Data

Although user satisfaction data and defect data are collected at different intervals, by different staff, and for different purposes, combining these two measurements at least once a year is an extremely useful undertaking. A very useful technique for combining the two kinds of information is a simple scatter graph that shows user satisfaction on one axis and user-reported defect levels on the other axis. Figure 5-5 illustrates such a combination graph. Obviously, the four different conditions shown by the figure can lead to four different business responses.

- **High levels of user satisfaction and low levels of defects** This situation implies an excellent product, and it is the desirable goal

USER SATISFACTION

	HIGH	LOW
HIGH DEFECT LEVELS	ZONE OF URGENT REPAIRS	ZONE OF URGENT REPLACEMENT
LOW DEFECT LEVELS	ZONE OF EXCELLENT APPLICATIONS	ZONE OF FUNCTIONAL ENHANCEMENT

Figure 5-5 Scatter graph of user satisfaction and defect data

of all software. Companies that measure both user satisfaction and numeric defect levels quickly realize the strong correlation between the two phenomena. Software products with high defect levels never generate satisfactory levels of user satisfaction. Leading software vendors and computer manufacturers can achieve more than 90 percent of all products within this quadrant.

- **Low levels of user satisfaction and low levels of defects** Products falling within this quadrant have obviously missed the mark in some attribute other than pure defect levels. The most common factors identified for products in this quadrant are insufficient functionality, cumbersome human interfaces, and inadequate training or documentation.
- **High levels of user satisfaction and high levels of defects** This particular quadrant is included primarily for consistency, and it will normally have very few if any projects falling within it. Any software product that does fall here is a candidate for full inspections and quality upgrading. Typically, the only projects within this quadrant will be initial releases with fairly important new functionality.
- **Low levels of user satisfaction and high levels of defects** Products falling within this quadrant are candidates for emergency handling and perhaps replacement. The steps that can be handled on an emergency basis include searching for and eliminating error-prone modules, full inspections, and perhaps retesting. The longer-range solutions may include restructuring the product, rewriting the documentation, and including additional functions.

Since this quadrant represents project failure in its most embarrassing form, it is significant to note that a majority of these products were rushed during their development. Excessive schedule pressures coupled with inadequate defect prevention and removal technologies are the factors most often associated with projects in this quadrant.

Summary and Conclusions

The measurement of quality, reliability, and user satisfaction is a factor that separates the leading-edge companies from the laggards. On a global basis, high quality is the main driving force for high-technology products. The companies that recognize this basic fact are poised for success in the 21st century; the companies that do not recognize it may not survive this new century!

Reading List

Because quality control and user satisfaction are such major topics, it is desirable to conclude with a short reading list of some of the more notable books that expand upon the topics discussed in this chapter.

Japanese Competitive Methods

Walton, Mary, *The Deming Management Method,* New York: Putnam, 1986, 262 pages. No single individual in all history has had more impact on the global economic balance than W. Edwards Deming. Deming moved to Japan after World War II, and he was the primary source of Japan's manufacturing quality control methods. He returned to the United States, and he was belatedly being listened to by U.S. companies that refused his advice for more than 30 years.

Noda, Nabuo, *How Japan Absorbed American Management Methods,* Tokyo: Asian Productivity Organization Press, 1981, 37 pages. The Asian Productivity Organization is sponsored by the national governments of Japan, Korea, Thailand, the Republic of China, and several other countries. This short pamphlet is an interesting chronology of the absorption of statistics-based quality control and psychology-based management practices in Japan.

Matsumoto, Koji, *Organizing for Higher Productivity: An Analysis of Japanese Systems and Practices,* Tokyo: Asian Productivity Organization Press, 1982, 75 pages. This is yet another of the interesting studies of Japanese methods and techniques. The author was an official of the Ministry of International Trade and Industry (MITI), and he is in a good position to speak about Japanese industrial and management practices.

Ohmae, Kenichi, *The Mind of the Strategist: The Art of Japanese Business,* New York: McGraw-Hill, 1982, 283 pages. Kenichi Ohmae is a director of the prestigious McKinsey & Company management consulting organization. This book gives his observations as a leading business consultant to major Japanese corporations. Since strategic planning in Japan is often more sophisticated than in the United States, this book is quite valuable to U.S. readers.

Crocker, Olga, C. Charney, and J. S. L. Chiu, *Quality Circles,* New York: Mentor, 1984, 361 pages. Quality circles are derived from the work of the American W. Edwards Deming, but they are much more popular in Japan, where more than 7,000 of them are registered with the Japanese national registry of quality control. The appealing concept of quality circles is that, given the chance and training, employees want to improve quality and help their employers. The quality circle methodology formalizes that concept, and this book shows managers and executives what they can do to tap this valuable resource.

Pascale, Richard Tanner and A. G. Athos, *The Art of Japanese Management,* New York: Warner, 1982, 363 pages. This interesting book contrasts Matsushita from Japan and ITT from the United States. Both enterprises had charismatic senior executives and were introducing novel and exciting management principles, but each reflected the paradigm of the nation in which the enterprise developed.

Jones, Capers, *Software Productivity and Quality Today—The Worldwide Perspective*, Carlsbad, CA: Information Systems Management Group, 1993, 200 pages. This book is the first to attempt to quantify software productivity and quality levels on a global basis. Demographic information is included on the software populations of more than 90 countries and 250 global cities. Productivity data is ranked for systems software, information systems, military software, and other categories. This book is comparatively unique in that it uses the SPR Feature Point metric, rather than the more common Function Point metric. The book also discusses international variations in software effectiveness, quality control, and research programs.

Morita, Akio, E. M. Reingold, and M. Shimomura, *Made in Japan,* New York: Penguin, 1986, 343 pages. This is an autobiography of Akio Morita, the founder of Sony corporation. The book is interesting because it reveals not only the details of Sony's industrial growth but also the personality of its founder. Sony shared the typical high quality of other Japanese companies in product manufacture, but it also had an unusually perceptive sense of market trends.

Random, Michel, *Japan: The Strategy of the Unseen,* Wellingborough, England: Aquarian, 1987, 205 pages. This book, translated from the original French, is a tutorial for European, British, and American business travelers visiting Japan. It discusses the historical and philosophical background of Bushido, the code of the Samurai in medieval Japan. This is a major part of modern Japanese business practice, and Westerners who have no knowledge of Samurai history will be at a disadvantage.

Masuda, Youeji, *The Information Society,* Bethesda, MD: World Future Society, 1981, 170 pages. This book is now somewhat dated, but in one sense that may add to its importance. It discusses some of the large-scale experiments carried out in Japan to explore the concepts of full

computerization and communication capabilities in ordinary households. Entire villages were equipped with terminals and communication networks that allowed such functions as remote purchasing from stores, remote emergency medical advice, and even some direct participation in town government functions. Similar large-scale experiments have taken place in other countries such as Sweden, France, and Canada, but the Japanese experiments are perhaps the largest yet carried out.

Halberstam, David, *The Reckoning,* New York: Avon, 1987, 786 pages. This book is a massive history of the automotive industry that concentrates on Ford and Toyota, and it deals with both the companies and the executives who founded them. The major point of the book is that, through a combination of arrogance and ignorance, the mightiest industry in America was struck a damaging blow by Japanese competition. U.S. automotive executives ignored quality and disregarded its importance until it was almost too late to recover. They also were guilty of ethnocentrism and the naive assumption that U.S. technology would permanently dominate the postwar world.

Quality Control and Corporate Excellence

Crosby, Philip B., *Quality Is Free*, New York, NY: Mentor, 1980, 270 pages. Phil Crosby was the ITT Vice President of Quality, and he was the person who introduced modern quality control practices into the ITT system. His famous book, long a best-seller, derives its title from the empirical observation that the economic advantages of quality control far overshadow the costs. Phil Crosby and W. Edwards Deming were two of the pioneers in this field of study.

Geneen, Harold and A. Moscow, *Managing,* New York: Avon, 1984, 305 pages. Harold Geneen was the chairman and CEO that took ITT from a medium-size telecommunications company to the most diverse and largest conglomerate in U.S. history. His style was unique, but his observations on running very large enterprises are worthy of note. It is interesting that Phil Crosby dedicated his *Quality Is Free* to Harold Geneen, since Harold's support was vital to establishing statistical quality methods within ITT.

Peters, Tom, *In Search of Excellence,* New York: Random House, 1985, 389 pages. This was one of the most widely discussed books of the 1980s. In it, Peters discusses the characteristics that tend to separate leading enterprises from laggards. Excellence in all its forms, human, cultural, and business, is part of the pattern of success.

Andrews, Dorine C. and Susan K. Stalick, *Business Reengineering—The Survival Guide,* Englewood Cliffs: Prentice Hall, 1994, 300 pages. Business process reengineering (BPR) became a cult of the 1990s. When BPR is used to provide better customer service, it is often successful. However, BPR has become synonymous with massive layoffs

and cutbacks and is sometimes a step to corporate disaster. This book covers the pros and cons of the topic from the point of view of someone who may have to live through a major BPR analysis.

Pirsig, Robert, *Zen and the Art of Motorcycle Maintenance,* New York: Bantam Books, 1974. In spite of the unusual title and the fact that it is a novel rather than a technical book, Pirsig's book is, in fact, a book on quality that is good enough to have been used by IBM as part of its quality assurance training.

The book deals with a cross-country motorcycle trip taken by Pirsig and his son. In the course of the trip, the author discusses the meaning of quality and the difficulty of coming to grips with this elusive concept. The fundamental points are that doing things well provides great satisfaction to the doer as well as to the beneficiary, and that quality is easy to experience but difficult to define.

Software Engineering and Software Quality Control

Jones, Capers, *Patterns of Software System Failure and Success,* Boston: International Thomson, 1995, 250 pages. This book was published in December of 1995. The contents are based on large-scale studies of failed projects (i.e., projects that were either terminated prior to completion or had severe cost and schedule overruns or massive quality problems) and successful projects (i.e., projects that achieved new records for high quality, low costs, short schedules, and high customer satisfaction). On the whole, management problems appear to outweigh technical problems in both successes and failures. Other factors discussed include the use of planning and estimating tools, quality control approaches, experience levels of managers, staff, and clients, and stability of requirements.

Freedman, D. and G. M. Weinberg, *A Handbook of Walkthroughs, Inspections, and Technical Reviews,* Boston: Little, Brown, 1982, 450 pages. Daniel Freedman and Jerry Weinberg are two well-known consultants to the software industry. This book provides an excellent introduction to the various forms of review and inspection that have proven to be so effective for software projects. The book is in question-and-answer form, and it covers all of the topics that naturally occur to first-time participants of reviews and inspections.

DeMarco, Tom and Tim Lister, *Peopleware,* New York: Dorset House, 1987, 188 pages. This book deals with the human side of software; it discusses the concept that so long as software is a human occupation, optimizing the treatment and conditions of software staff will be beneficial. The book goes beyond merely stating a point, however, and discusses large-scale empirical studies that buttress some of its conclusions. Among the most surprising observations was the discovery that

the physical office environment had a direct correlation with software productivity: programmers in the high quartile had more than 78 ft^2 of office space, and those in the low quartile had either open offices or less than 44 ft^2. DeMarco was the recipient of the 1987 Warnier prize for outstanding contributions to computer and information science.

DeMarco, Tom, *Controlling Software Projects,* New York: Yourdon Press, 1982, 284 pages. This book gives the advice of one of the industry leaders in keeping software projects under control in terms of a variety of dimensions: quality, schedule, costs, and others. The book also introduced DeMarco's bang functional metric.

Boehm, Barry, *Software Engineering Economics,* Englewood Cliffs, NJ: Prentice Hall, 1981, 767 pages. This book is perhaps the largest ever written on the costs associated with software projects. It certainly has one of the largest and most complete sets of references to the relevant literature. The book contains the original equations for the COCOMO estimating model, and it presents one of the few nonproprietary descriptions of how a software cost-estimating model might work. It has been a best-seller since it originally appeared.

Humphrey, Watts, *Managing the Software Process,* Reading, MA: Addison-Wesley, 1989, 489 pages. Humphrey, formerly at IBM, was head of the prestigious Software Engineering Institute (SEI) associated with Carnegie Mellon University. This book contains both his observations on the software process and also his method for evaluating the five stages of maturing of software development organizations.

Bigger staff, T. and A. Perlis, *Software Reusability,* Vols. 1 and 2, Reading, MA: Addison-Wesley, 1989; Vol. 1, 424 pages; Vol. 2, 386 pages. Dr. Biggerstaff was part of the ITT Programming Technology Center. Together with Dr. Perlis of Yale, he was cochairman of the famous 1983 ITT conference on software reusability. This two-volume set builds upon a kernel of papers that were presented at that conference, but it brings the entire subject up to date.

Glass, Robert L., *Modern Programming Practices: A Report from Industry,* Englewood Cliffs, NJ: Prentice Hall, 1982, 311 pages. Books on software engineering theory outnumber books on software engineering practices by at least 20 to 1. This one is based on interviews and empirical observations carried out within large companies—Martin Marietta, TRW, CSC, and others. Although somewhat dated, it is an interesting attempt to describe what goes on day-to-day when large companies build software.

Dunn, Robert and Richard Ullman, *Quality Assurance for Computer Software,* New York: McGraw-Hill, 1982, 349 pages. This book is yet another that originated with research carried out within ITT. Dunn and Ullman were quality assurance specialists for one of ITT's divisions and later for the corporate offices. It summarizes, broadly, all of the major

tasks and issues regarding setting up a quality assurance organization for modern software engineering projects.

Dunn, Robert, *Software Defect Removal,* New York: McGraw-Hill, 1984, 331 pages. Bob Dunn continues his explication of software quality methods with a broad-based survey of all of the major forms of defect removal. As with the preceding title, this book comprises part of the set of methods for quality assurance used within the ITT system.

Chow, Tsun S., *Software Quality Assurance,* New York: IEEE Press, 1984, 500 pages. Tsun Chow's tutorial volume is part of the IEEE series on software engineering. It contains some 50 articles by various authors covering all of the major software QA topics.

Myers, Glenford, *The Art of Software Testing,* New York: Wiley, 1978, 177 pages. Although this book is now 30 years old, it is still regarded as a classic text on software testing and is still a textbook in many university and graduate school curricula. Myers is a multifaceted researcher who was also one of the originators of structured design and a contributor to computer architecture as well. He was the 1988 recipient of the Warnier prize for outstanding contributions to computer and information science.

Brooks, Fred, *The Mythical Man-Month,* Reading, MA: Addison-Wesley, 1995, 195 pages. This book has sold more copies than any other book on a software topic since the industry began. (A revised new version came out in 1995 with many new insights.) Brooks was the IBM director who first developed the operating system software for the IBM System/360 computer line, which was one of IBM's first major efforts to run far beyond planned budgets and schedules. His report is a classic account of why even large and well-managed enterprises such as IBM run into trouble when building large software systems. The cover art of the book captures the problems exactly: giant prehistoric ground sloths struggling to free themselves from the La Brea tar pits! The message is that, once the problems of large systems get started, no amount of money or extra staff can easily recover the situation.

Pressman, Roger, *Software Engineering: A Practitioner's Approach,* New York: McGraw-Hill, 1982, 352 pages. This book, widely used as a college text, is an excellent introduction to all of the major topics of concern to both software engineers and software managers as well. It is organized chronologically and is based on the phases of a software lifecycle. It is much broader in scope than many other software engineering texts in that it includes management topics such as planning, estimating, and measurement. It also covers requirements, design, coding, testing and defect removal, and maintenance. It would be difficult to find a more suitable introduction to all of the critical topics than is contained in this book. As of 2008 the 6th edition of this book is now in print.

Suggested Readings

Crocker, O., L. S. Charney, and J. S. L. Chieu. *Quality Circles.* New York: Mentor, 1984.
Crosby, Phil. *Quality Is Free: The Art of Making Quality Certain.* New York: McGraw-Hill, 1979.
Deutsch, M. *Software Verification and Validation: Realistic Project Approaches.* Englewood Cliffs, NJ: Prentice Hall, 1982.
Fagan, M. "Design and Code Inspections to Reduce Errors in Program Development," *IBM Systems Journal,* vol. 15, no. 3 (1976): pp. 182–211.
Halpin, J. F. *Zero Defects—A New Dimension in Quality Assurance.* New York: McGraw-Hill, 1966.
Levering, R., M. Moskowitz, and M. Katz. *The 100 Best Companies to Work for in America.* New York: Addison-Wesley, 1985.
Martin, J. *System Design That Is Provably Bug Free.* Englewood Cliffs, NJ: Prentice Hall, 1985.
Mills, H. "The New Math of Computer Programming," *CACM,* vol. 18 (January 1975): pp. 43–48.
Musa, J., A. Iannino, and K. Okumoto. *Software Reliability: Measurement, Prediction, Application.* New York: McGraw-Hill, 1987.
Peters, Tom. *In Search of Excellence.* New York: Random House, 1985.
Pirsig, R. *Zen and the Art of Motorcycle Maintenance.* New York: Bantam Books, 1984.
SPR. "User Satisfaction Survey Questionnaire." Burlington, MA: Software Productivity Research, 1989.
SPR. *Software Design Inspections.* Burlington, MA: Software Productivity Research, 1990.
Walton, Mary. *The Deming Management Method.* New York: Perigree, 1986.

Additional References on Software Quality and Quality Measurements

Ewusi-Mensah, Kweku. *Software Development Failures*. Cambridge, MA: MIT Press, 2003.
Galorath, Dan. *Software Sizing, Estimating, and Risk Management: When Performance Is Measured Performance Improves.* Philadelphia: Auerbach Publishing, 2006.
Garmus, David and David Herron. *Function Point Analysis—Measurement Practices for Successful Software Projects*. Boston, MA: Addison Wesley Longman, 2001.
Gilb, Tom and Dorothy Graham. *Software Inspections*. Reading, MA: Addison Wesley, 1993.
Glass, R. L. *Software Runaways: Lessons Learned from Massive Software Project Failures*. Englewood Cliffs, NJ: Prentice Hall, 1998.
International Function Point Users Group (IFPUG). *IT Measurement—Practical Advice from the Experts*. Boston, MA: Addison Wesley Longman, 2002.
Johnson, James et al. *The Chaos Report*. West Yarmouth, MA: The Standish Group, 2000.
Jones, Capers. *Assessment and Control of Software Risks*. Englewood Cliff, NJ: Prentice Hall, 1994.
Jones, Capers. *Patterns of Software System Failure and Success*. Boston: International Thomson Computer Press, 1995.
Jones, Capers. *Software Quality—Analysis and Guidelines for Success*. Boston: International Thomson Computer Press, 1997.
Jones, Capers. "Sizing Up Software." *Scientific American Magazine,* vol. 279, no. 6 (December 1998): pp. 104–111.
Jones, Capers. *Software Assessments, Benchmarks, and Best Practices*. Boston: Addison Wesley Longman, 2000.
Jones, Capers. *Conflict and Litigation Between Software Clients and Developers*. Burlington, MA: Software Productivity Research, Inc., 2001.
Jones, Capers. *Estimating Software Costs*. New York: McGraw-Hill, 2007.

Jones, Capers. *Software Project Management Tools.* Narragansett, RI: Software Productivity Research LLC, 2008.
Kan, Stephen H. *Metrics and Models in Software Quality Engineering.* 2nd Ed. Boston: Addison Wesley Longman, 2003.
Radice, Ronald A. *High Qualitiy Low Cost Software Inspections*. Andover, MA: Paradoxicon Publishing, 2002.
Wiegers, Karl E. *Peer Reviews in Software—A Practical Guide*. Boston: Addison Wesley Longman, 2002.
Yourdon, Ed. *Death March—The Complete Software Developer's Guide to Surviving "Mission Impossible" Projects*. Upper Saddle River, NJ: Prentice Hall PTR,1997.
Yourdon, Ed. *Outsource: Competing in the Global Productivity Race*. Upper Saddle River, NJ: Prentice Hall PTR, 2005.

Chapter

6

Measurements, Metrics, and Industry Leadership

Software measurement as discussed earlier in this book is only one of more than a dozen forms of measurement performed by large corporations and government agencies. This chapter places Applied Software Measurement in context with other forms of corporate measurement. Collectively, all forms of measurement are expensive. The measurement costs for large companies total to about 6 percent of their sales. However, the value of measurements is far greater than their costs.

Historically, industry leaders and innovators do not achieve success blindly. A wide spectrum of business and technical metrics are used both to plan future strategies and to head off potential problems. Some of the key measurements and metrics noted among large corporations are those of market penetration, customer satisfaction, quality, productivity, schedules, costs, and personnel factors. Benchmarks, baselines, and assessments are also tools used by industry leaders.

A successful measurement program is of necessity a multifaceted activity. This chapter summarizes the measurement practices of Software Productivity Research (SPR) clients who have built successful businesses and are also technical and market leaders within their fields. Note that while SPR concentrates on software practices, we also observe broader corporate and engineering measurements and metrics too.

As business practices change with the impact of the Internet and World Wide Web, new measurements and metrics must evolve to meet new situations. In 2008, there are gaps in the measurements and metrics of even the best enterprises. Currently, there are no effective measurements or metrics for dealing with data quality, database volumes, the quality of knowledge-based systems, or the quality or costs of web content.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

Metrics and measurement programs tend to lead to behavioral changes among managers and staff because they improve in the factors that are being measured. Therefore, it is important to select measures and metrics carefully to ensure that they do not cause harmful results.

Within every industry, there are significant differences between the leaders and the laggards in terms of market shares, technological sophistication, and quality and productivity levels. One of the most significant differences is that leaders *know* their market performance, customer satisfaction, quality and productivity levels because they measure them. Lagging companies do not measure, and so they don't know how good or bad they are compared to competitors.

Measurement is not the only factor that leads to excellence. Measurement is only one part of a whole spectrum of issues, including (1) Good management at all levels; (2) Good technical staffs with high morale; (3) Good development and maintenance processes; (4) Effective and complete tool suites; (5) Good and flexible organizational structures; (6) Specialized staff skills as needed; (7) Continuing on-the-job training and staff education; (8) Good personnel policies; (9) Good working environments; (10) Good communications.

However, measurement is the technology that allows companies to make visible progress in improving the other factors. Without measurement, progress is slow and sometimes negative. Companies that don't measure tend to waste scarce investment dollars in "silver bullet" approaches that consume time and energy but generate little forward progress. In fact, investment in good measurement programs has one of the best returns on investment of any known technology.

In 2002, Congress passed the 2002 Sarbanes-Oxley Act (SOX). SOX was a response to the financial irregularities of Enron, WorldCom, Tyco, and the other corporate accounting scandals that swept through U.S. business. The SOX act had a phased implementation and began to be fully deployed only in November 2004. The SOX act affects large public companies with revenues in excess of $75,000,000 per annum.

The impact of adhering to section 404 of the new SOX measures was reported to cost Fortune 500 companies $7,800,000 each to perform the SOX measures and another $1,900,000 per company to have the measures audited. Although costs will probably decrease once companies get up to speed, there is no question that the Sarbanes-Oxley act has introduced permanent and important measurement changes into U.S. public companies.

There is one curious aspect of measurement in large corporations. Measurement is seldom used recursively to explore the value of the measures themselves. The total cost of measurement in all forms in large corporations can approach 6 percent of sales: a very high cost. What is the value of these measurements? Unfortunately, this topic is not

addressed by the current literature. From examining the uses made of measurement data, it can be stated that a good measurement program is a key contributor to market share, customer satisfaction, and internal operating cost. Hopefully the return on investment in a full and comprehensive measurement program is positive and significant, but there is very little empirical data in 2008 to actually quantify the ROI.

What Do Companies Measure?

The best way for a company to decide what to measure is to find out what the "best in class" companies measure and do the same thing. Following are the kinds of measurements used by companies that are at the top of their markets and succeeding in global competition. If possible, try to visit companies such as GE, Microsoft, IBM, AT&T, or Hewlett-Packard and find out first hand what kinds of measurements tend to occur.

Business and Corporate Measures

There are a number of important measurements at the corporate level. Some measurements are required by government regulatory agencies. However, many metrics and measurements are optional and utilized to provide visibility into a variety of topics. Here are just a few samples of corporate measures to illustrate what the topics of concern are.

Sarbanes-Oxley (SOX) Measures As mentioned previously, the Sarbanes-Oxley act was passed in 2002 and became effective in 2004. The SOX measures attempt to expand and reform corporate financial measurements to ensure absolute honesty, integrity, and accountability. Failure to comply with SOX criteria can lead to felony charges for corporate executives, with prison terms of up to 20 years and fines of up to $500,000. However, the Sarbanes-Oxley measures apply only to major public companies with revenues in excess of $75,000,000 per annum. The first implementation of SOX measures seemed to require teams of 25 to 50 executives and information technology specialists working for a year or more to establish the SOX control framework. Many financial applications had to be modified, and of course, all new applications must be SOX compliant. The continuing effort of administering and adhering to SOX criteria will probably amount to the equivalent of perhaps 20 personnel full-time for the foreseeable future. Because of SOX's legal requirements and the severe penalties for noncompliance, corporations need to get fully up to speed with SOX criteria. Legal advice is very important.

Balanced Scorecard Measures Dr. Robert Kaplan and Dr. David Norton of the Harvard Business School are the originators of the "balanced

scorecard" approach. This approach is now found in many major corporations and some government agencies as well. The balanced scorecard approach is customized for specific companies, but includes four major measurement topics: (1) The learning and growth perspective; (2) The business process perspective; (3) The customer perspective; (4) The financial perspective. Although the balanced scorecard approach is widespread and often successful, it is not a panacea.

Financial Measures Corporations have been measuring revenues, expenses, profits, losses, cash flow, capital expenditures, and a host of other financial topics for many centuries. Financial measurements are usually the most complete of any form of corporate measurement. Financial measurements are also governed by a variety of regulations and policies as required by the Internal Revenue Service, Securities and Exchange Commission, state agencies, and other governmental bodies. Of course, the new Sarbanes-Oxley (SOX) act introduced the most important changes in financial measures in U.S. history. SOX was implemented because historical financial measurements were not always well done. Indeed, the fairly recent spate of bankruptcies, lawsuits, and criminal charges levied against corporate financial officers and other executives indicate how imperfect financial measures can sometimes be in large corporations. Financial measures are normally in accordance with generally accepted accounting principles (GAAP), which are set by the Financial Accounting Standards Board (FASB) in the United States and by similar organizations abroad. Non-GAAP measures are also utilized. Of course auditors and accountants have also been implicated in some of the recent corporate financial scandals. In the United States, the Securities and Exchange Commission (SEC) requires very detailed financial reports known as 10-Ks. These reports discuss the corporation's evolution, management structure, equity, subsidiaries, earnings per share, and many other topics.

Return on Investment (ROI) The measurement of return on investment or ROI has long been an important corporate measurement. The mathematics for calculating internal rates of return (IRR) and net present value (NPV) have been used for more than a hundred years. This is not to say that ROI calculations are simple and easy. Often long-range assumptions or even guesses must be utilized. Some companies include total cost of ownership (TCO) as part of their ROI calculations, whereas others prefer a limited time horizon of three to five years for a positive ROI. Because of unique situations, each company and sometimes each division or business unit may calculate ROI differently. Additionally, not all value is financial. When investments or potential development projects benefit corporate prestige, customer loyalty, staff morale, safety, security, health,

or perhaps national defense, then financial ROI calculations are not completely adequate. ROI is one of the key factors that affect corporate innovation because almost all companies (more than 75 percent at last count) require formal ROI studies for technology investments.

Shareholder Measures Public companies in the U.S. are required to produce reports on their financial and equity conditions for all shareholders. Some of the elements included in annual and quarterly reports to shareholders include revenues, assets, expenses, equity, any litigation in progress or upcoming, and a variety of other topics. One of the items of interest to shareholders includes market capitalization, or the number of shares in existence multiplied by the current price per share. Another item of interest is total shareholder return, or the change in share price from the original purchase date (plus dividends accrued) divided by the original price. The total number of topics is too large for a short discussion, but includes economic value added (EVA), cash flow return on investment (CFROI), economic margin, price-earnings ratio (P/E ratio), and many others.

Market Share Measures Industry and global leaders know more about their markets, market shares, and competitors than the laggards do. For example, industry leaders in the commercial software domain tend to know how every one of their products is selling in every country and how well competitive products are selling in every country. Much of this information is available from public sources. Subscriptions to various industry reports and customized studies by consulting groups also provide such data.

Competitive Measures Few companies lack competitors. Industry leaders know quite a bit of information about their competitors' products, market share, and other important topics. Much of this kind of information is available from various industry sources such as Dun & Bradstreet, Mead Data Central, Fortune magazine, and other journals, and from industry studies produced by organizations such as Auerbach, Meta Group, the Gartner Group, and others. For many topics, industry benchmarks performed by neutral consulting groups provide information to many companies but conceal company specifics in order to protect proprietary information.

Human Resource Measures Since good people are the cornerstone of success, industry leaders strive to hire and keep the best knowledge workers. This means that measurements of morale and attrition are important. Other personnel measures include demographic measures of the number and kinds of personnel employed, skills inventories, and training curricula.

Research and Development (R&D) Measures Some companies have substantial investments in research facilities and employ scientists and researchers as well as white- and blue-collar workers. The IBM Research Division and Bell Labs are prominent examples of major corporate research facilities. Because research and development projects are not always successful, simple measurements are not sufficient. Some of the specialized measures associated with R&D laboratories include topics such as patents, invention disclosures, publications, and the migration of research topics into industrial products. For large companies, R&D measures provide a glimpse of organizations that are likely to be innovative. For example, a company that files for 150 patents each year can probably be assumed to have a higher level of innovation than a similar company filing for five patents each year.

Manufacturing Measures Companies that build products from scratch have long found it valuable to measure their development cycles from end to end. Manufacturing measures include procurement of raw materials, shipping and freight, manufacturing steps, scrap and rework, inventory and warehousing, and a host of other topics. The specific kinds of measurements vary based on whether the manufacturing cycle is discrete or involves continuous flow processes. A variety of commercial tool vendors and consultants have entered the manufacturing measurement domain. Enterprise Resource Planning (ERP) packages are now available to handle many key measurements in the manufacturing sector, although such packages are not perfect by any means.

Outsource Measures Outsourcing within the United States has been common for many years. Outsourcing abroad to low-cost countries such as India, China, Pakistan, Russia, Guatemala, and so on, has become very common and appears to be accelerating rapidly. Specific measures for outsourcing vary with the nature of the work being outsourced. For example, outsourcing human resources or payroll administration is very different from outsourcing software development or manufacturing. For software outsourcing, an area tht Software Productivity Research specializes in, some of the key measurement topics include service-level agreements, quality agreements, productivity agreements, schedule agreements, and rates of progress over time. Also important are some specialized topics such as requirements growth rates, dealing with project cancellations or termination, and methods for handling disagreements between the outsource vendor and the client.

Supply Chain Measures Measuring the performance of an entire supply chain for a major corporation is a challenging task. The full supply chain may include dozens of prime and scores of subcontractors, possibly

scattered around the globe. In 1996 and 1997, an organization called the Supply Chain Council was formed. Some of the founders included perhaps a dozen large companies such as Bayer, AMR Research, Procter & Gamble, Lockheed Martin, and similar companies. The council developed a set of measurement methods called the Supply Chain Operations Reference (SCOR). Some of the topics that are significant in dealing with supply chain measures include orders filled without backordering, orders filled without errors, arrival time of materials from subcontractors, defects or returns to subcontractors, on-time delivery to customers, delivery speed, costs of materials, freight and storage costs, inventory size, and inventory turnover. There are also some specialized measures having to do with taxes paid by supply chain partners. Some of these measures overlap traditional measures used by individual companies of course. Supply chain measures are a fairly new phenomenon and are evolving rapidly.

Warranty and Quality Measures For both manufactured goods and custom built products such as software, quality and warranty costs are important topics and should be measured fully and accurately. The topic of warranty measures is fairly straightforward and includes the total costs of customer support and product replacements under warranty guarantees, plus any additional costs attributed to good will. The topic of quality itself is large and complex. Many quality metrics center around defect levels and defect repairs. The costs of repairing defects are often handled under the topic of "cost of quality." The cost of quality topic was made famous by Phil Crosby of ITT in his book *Quality is Free*. It is somewhat ironic that the cost of quality topic is really about the cost of poor quality. Traditional cost of quality measures cover four topics: external failures, internal failures, inspection costs, and prevention costs. However, there are many variations and extensions to Crosby's original themes. Some more recent approaches to quality measures are those associated with total quality management (TQM), Six-Sigma quality, and Quality Function Deployment (QFD), and the quality standards in the ISO 9000-9004 set. However, these topics are too complex and extensive to discuss here.

Benchmarks and Industry Measures

Most companies are in industries that contain similar companies. It is often of interest to have comparative data that can show how a specific company is performing against the background of many similar companies. Since some or all of the companies of interest may be competitors, special methods are needed to gather data from companies without revealing proprietary information. It is also necessary to avoid even the appearance of collusion or price fixing or sharing insider information.

The most common way for carrying out multi-company benchmarks is to use an independent consulting group that specializes in benchmarking various topics. Examples of such benchmark organizations include the Gartner Group, Meta Group, the Standish Group, Forrester Research, the International Function Point Users Group (IFPUG), and of course, the author's company, Software Productivity Research. Some of the measurements normally collected by independent consulting groups include the following:

Compensation and Benefits Corporations cannot legally share compensation and benefit data with competitors, nor would it be prudent to do so. Compensation and benefit studies are normally "blind" studies where the names of the participants are concealed. Companies within the same industry report compensation and benefit amount to a neutral independent consulting group, which then averages and reports on the results. Each company that contributes information receives a report showing how their compensation compares to group averages, but the specific values for each company are concealed from the others in the group.

Attrition and Turnover Every large company keeps internal statistics on attrition rates. But to find out whether a company is better or worse than others in the same industry requires that a neutral independent company collect the information and perform statistical analysis. The results are then reported back to each company that contributes data to the study, but the specifics of each company are concealed from the others.

Research and Development Spending Patterns Companies with substantial investments in research and development are always interested in how they compare to similar companies. As with other topics involving multiple companies in the same industry, the R&D studies are performed blind with each company contributing information about R&D spending and personnel.

Software Measures and Metrics

As the name of the author's company implies, Software Productivity Research (SPR) is primarily involved with measurements, assessments, estimation, and process improvements in the software domain. Of course, software development is only one of many organizations in modern enterprises. However software development is notoriously difficult. The failure rate of software projects is alarmingly high. Even when software projects are successfully completed, poor quality, high maintenance costs, and customer dissatisfaction are distressingly common. Therefore, measuring software projects, and especially key indicators affecting quality and risk, is vital to major corporations.

Software metrics have been troublesome over the years. The first attempt at a metric for measuring software quality and productivity was "lines of code." This was appropriate in the 1960s when only a few low-level programming languages were used. By the end of the 1970s hundreds of programming languages were in use. For some high-level programming languages such as Visual Basic, there were no effective rules for counting lines of code. As the work of coding became easier thanks to high-level programming languages, the costs and quality of handling requirements and design became more important. Lines of code metrics could not be used for measuring non-coding work.

In the mid 1970s IBM developed a general-purpose metric for software called the "function point" metric. Function points are the weighted totals of five external aspects of a software project: inputs, outputs, logical files, interfaces, and inquiries. Function point metrics quickly spread through the software industry. In the early 1980s a nonprofit corporation of function point users was formed: the International Function Point Users Group (IFPUG). This association expanded rapidly and continues to grow. As of 2008 there are IFPUG affiliates in 23 countries and the IFPUG organization is the largest software measurement association in the world.

Since 1990, function points have been replacing lines of code metrics for studies of software economics, productivity, and quality. Today in 2008, there is more data expressed in function point form than all other metrics put together. This is not to say that function point metrics are perfect. But they do have the ability to measure full software lifecycles, including project management, requirements, design, coding, inspections, testing, user documentation, and maintenance. With function point metrics, it is possible to show complete expense patterns for large software projects from early requirements all the way through development and out into the field.

Thanks to function point metrics, we now know that for large software projects the cost of producing paper documents is more expensive than the code itself. But outranking both paper and code, the cost of repairing defects is the single most expensive activity. It is also known that for large software projects requirements grow and change at a rate of about 2 percent per calendar month.

The software world has two major segments: systems software and information technology (IT) projects. Systems software controls physical devices and includes operating systems, telephone switching systems, embedded software, aircraft flight controls, process controls, and the like. IT software controls business functions and includes payrolls, accounting packages, ERP packages, and the like.

Many large companies such as IBM, AT&T, and Microsoft produce both kinds of software. Interestingly, the systems software groups and the IT

groups tend to be in separate organizations, use different tools and programming languages, and perform different kinds of measurements.

The systems software domain is much better at measuring quality and reliability than the IT domain. The IT domain is much better at measuring productivity and schedules and is much more likely to use function point metrics. The sociological differences between the systems software and IT domains have caused problems in combining overall data about software in very large companies.

Every "best in class" company measures software quality. There are no exceptions. If your company does not do this, it is not an industry leader, and there is a good chance that your software quality levels are marginal at best. Quality is the most important topic in software measurement. The reason for this is because, historically, the cost of finding and fixing software defects has been the most expensive activity during both software development and software maintenance. Over a typical software lifecycle, more than 50 percent of the total accrued costs will be associated with finding and repairing defects. Therefore, it is obvious that attempts to prevent defects from occurring and to remove them as quickly as possible are of major economic importance to the software industry.

Following are some of the more important software measurements circa 2008.

Software Benchmarks Because software is such a troublesome topic, large companies are anxious to find out if other companies have their software development under control. Therefore, software benchmarks for IT investments, quality, productivity, costs, and customer satisfaction are very common. Another form of benchmark in the software domain involves demographics. As of 2008, large corporations can employ more than 70 kinds of specialized personnel in their software organizations. As with other kinds of benchmarks, software benchmarks are usually carried out by independent consulting organizations. *Data is* collected from a sampling of companies, analyzed, and then reported back to the participants.

Portfolio Measures Major corporations can own from 250,000 to more than 10,000,000 function points of software, apportioned across thousands of programs and dozens to hundreds of systems. Leading enterprises know the sizes of their portfolios, their growth rates, replacement costs, quality levels, and many other factors. For companies undergoing various kinds of business process reengineering, it is important to know the quantity of software used by various corporate and business functions such as manufacturing, sales, marketing, finance, and so forth. Portfolio measures are often subset into the various categories of software that corporations build and acquire. Some of the categories include in-house systems software,

in-house information systems, in-house web applications, outsourced software of various kinds, and vendor-provided applications.

For vendor-acquired applications, reported defects, acquisition costs, and maintenance costs are the most common data points. Since most vendors don't report on the sizes of their packages in terms of LOC or function points, normalized analysis is usually impossible.

Earned Value Measurements Traditional methods for monitoring progress on various kinds of projects involve estimating costs before the projects begin and accumulating actual costs while the projects are underway. At monthly intervals, the estimated costs are compared to the actual accrued costs. Any deviations are highlighted in the form of variance reports.

The problem with this methodology is that it does not link either estimates or accrued costs to actual deliverable items or work products. In the late 1960s, defense and civilian engineered projects began to utilize a somewhat more sophisticated approach termed "earned value." Under the earned-value approach, both time and cost estimates were made for specific deliverables or work products such as requirements, initial functional specifications, test plans, and the like.

As projects proceed, actual costs and schedules are recorded. But actual accrued costs are linked to the specific deliverables that are supposed to be produced under the earned-value approach. This link among planned costs, accrued costs, and actual completion of work packages or deliverables is a better indicator of progress than the older method of simply estimating costs and accruing actual costs without linking to milestones or work packages.

Balanced Scorecards for Software The balanced scorecard approach was not developed for software. However, the balanced scorecard approach can be applied to software organizations just as it can to other operating units. Under this approach, conventional financial measures are augmented by additional measures that report on the learning and growth perspective, the business process perspective, the customer perspective, and the financial perspective.

Function point metrics are starting to be used experimentally for some aspects of the balanced scorecard approach. Some of the function point metrics associated with the balanced scorecard approach include

- Learning and growth perspective
  - Rate at which users can learn to use software (roughly 1 hour per 100 function points)
  - Tutorial and training material volumes (roughly 1.5 pages per 100 function points)

- Business process perspective
  - Application and portfolio size in function points
  - Rate of requirements creep during development
  - Volume of development defects per function point
  - Ratios of purchased function points to custom-developed function points
  - Ratios of reused function points to custom-developed function points
  - Annual rates of change of applications or portfolios in function points
  - Productivity (work hours per function point or function points per staff month)
- Customer perspective
  - Number of function points delivered
  - Number of function points used by job title or role
  - Customer support costs per function point
  - Customer-reported defects per function point
- Financial perspective
  - Development cost per function point
  - Annual maintenance cost per function point
  - Termination or cancellation cost per function point for retired projects

The balanced scorecard approach and function point metrics are just starting to be linked together circa 2008.

Goal Question Metrics Dr. Victor Basili of the University of Maryland developed a general methodology for linking important software topics to other business issues using a technique called "goal question metrics." This method is increasing in usage and popularity circa 2007. Important business or technical goals are stated, and then various questions are posed as to how to achieve them. The combination of goals and questions leads to sets of metrics for examining progress. What would be an interesting adjunct to this method would be link it to the North American Industry Classification System (NAICS) developed by the Department of Commerce. Since companies within a given industry tend to have similar goals, developing common sets of metrics within the same industry would facilitate industry-wide benchmarks.

Software Outsource Measures About 70 percent of software outsource agreements are satisfactory to both sides. For about 15 percent, one side or both will not be pleased with the agreement. For about 10 percent, the agreement will be terminated within two years. For about 5 percent, disagreements may even reach the level of litigation for breach of contract or even fraud. For software outsourcing, which is the area Software Productivity Research specializes in, some of the key measurement topics include service-level agreements, quality agreements, productivity agreements, schedule agreements, and rates of progress over time. Also important are some specialized topics such as growth rates of requirements, handling new and changing requirements, handling project cancellations or termination, and methods for handling disagreements between the outsource vendor and the client if they occur.

In the outsource litigation where the author and his colleagues have been expert witness, the major issues that caused the litigation were outright failure of the project; significant cost and schedule overruns of more than 50 percent; delivery of software that could not be used as intended; excessive error content in delivered software; and high volumes of requirements changes.

Customer Satisfaction Leaders perform annual or semiannual customer satisfaction surveys to find out what their clients think about their products. Leaders also have sophisticated defect reporting and customer support information available via the Web. Many leaders in the commercial software world have active user groups and forums. These groups often produce independent surveys on quality and satisfaction topics. There are also focus groups, and some large software companies even have formal "usability labs" where new versions of products are tried out by customers under controlled conditions.

Software Usage Measures A new kind of analysis is beginning to be used within the context of business process reengineering. The function point metric can be used to measure the quantity of software used by various workers within corporations. For example, project mangers often use more than 10,000 function points of tools for planning, estimating, sizing, measuring, and tracking projects. Such information is starting to be available for many other occupations including accounting, marketing, sales, various kinds of engineering, quality assurance, and several others.

Defect Quantities and Origins Industry leaders keep accurate records of the bugs or defects found in all major deliverables, and they start early during requirements or design. At least five categories of defects are

measured: requirements defects, design defects, code defects, documentation defects, and bad fixes or secondary bugs introduced accidentally while fixing another bug. Accurate defect reporting is one of the keys to improving quality. In fact analysis of defect data to search for root causes has led to some very innovative defect prevention and defect removal operations in many SPR client companies. Overall, careful measurement of defects and subsequent analysis of the data is one of the most cost-effective activities a company can perform.

Defect Removal Efficiency Industry leaders know the average and maximum efficiency of every major kind of review, inspection, and test, and they select an optimum series of removal steps for projects of various kinds and sizes. The use of pretest reviews and inspections is normal among Baldrige winners and organizations with ultra-high quality, since testing alone is not efficient enough. Leaders remove from 95 percent to more than 99 percent of all defects prior to delivery of software to customers. Laggards seldom exceed 8 percent in terms of defect removal efficiency and may drop below 50 percent.

Delivered Defects by Application Industry leaders begin to accumulate statistics on errors reported by users as soon as the software is delivered. Monthly reports are prepared and given to executives, which show the defect trends against all products. These reports are also summarized on an annual basis. Supplemental statistics such as defect reports by country, state, industry, client, and so on, are also included.

Defect Severity Levels All of the industry leaders, without exception, use some kind of a severity scale for evaluating incoming bugs or defects reported from the field. The number of plateaus vary from one to five. In general, severity 1 problems cause the system to fail completely, and the severity scale then descends in seriousness.

Complexity of Software It has been known for many years that complex code is difficult to maintain and has higher than average defect rates. A variety of complexity analysis tools are commercially available that support standard complexity measures such as cyclomatic and essential complexity. It is interesting that the systems software community is much more likely to measure complexity than the information technology (IT) community.

Test Case Coverage Software testing may or may not cover every branch and pathway through applications. A variety of commercial tools are available that monitor the results of software testing and help to identify portions of applications where testing is sparse or nonexistent. Here,

too, the systems software domain is much more likely to measure test coverage than the information technology (IT) domain.

Cost of Quality Control and Defect Repairs One significant aspect of quality measurement is to keep accurate records of the costs and resources associated with various forms of defect prevention and defect removal. For software, these measures include: (1) The costs of software assessments; (2) The costs of quality baseline studies; (3) The costs of reviews, inspections, and testing; (4) The costs of warranty repairs and post-release maintenance; (5) The costs of quality tools; (6) The costs of quality education; (7) The costs of your software quality assurance organization; (8) The costs of user satisfaction surveys; (9) The costs of any litigation involving poor quality or customer losses attributed to poor quality. In general, the principles of Crosby's "Cost of Quality" topic apply to software, but most companies extend the basic concept and track additional factors relevant to software projects.

Information Technology Infrastructure Library (ITIL) Measures Although some of the ITIL materials originated in the 1980s and even before, the ITIL books started to become popular in the 1990s. Today, in 2008, the ITIL materials are probably in use by more than 30 percent of large corporations in Europe and North America. Usage is higher in the United Kingdom and Europe because many of the ITIL books were developed in the United Kingdom. The ITIL library focuses on "service-oriented" measurements. Thus some of the ITIL measurement topics deal with change requests, incidents, problem reports, service or "help" desks, availability, and reliability.

In spite of the usefulness of the ITIL materials, there are still some notable gaps in coverage. For example, there are no discussions of "error prone modules" whose presence in large systems is a major factor that degrades reliability and availability. Neither is there good quantitative coverage of the growth in complexity and size of legacy applications over time. Also, the very important topic of "bad fix injection" is not discussed. Since about 7 percent of all changes and defect repairs contain new defects, this is a critical omission. The bottom line is that the ITIL materials alone are not yet a full solution for achieving optimal service levels. Additional materials on software quality, defect removal efficiency, Six-Sigma, and other topics are needed also.

Service Desk Measures Under the ITIL concept a "service desk" is the primary point of contact between application users and the software engineers who maintain the applications and add new features. Thus, service desks are the main point for handling both change requests and defect reports. Unfortunately most service desks for commercial software are

severely understaffed and also may be staffed by personnel who lack both the knowledge and tools for providing effective responses. In order to minimize the wait time for actually talking to someone at a service desk, the staffing ratio should be in the order of 1 service specialist for every 150 customers, adjusted for the quality levels of the software they support. Very few service desks have staffing ratios of more than about 1 service specialist per 1,000 customers. Even fewer make adjustments for poor or marginal quality. As a result, very long wait times are common.

Change Requests Under both the ITIL concept and normal maintenance concepts, change requests are submitted by authorized users to request new features for both existing software applications and also for applications currently under development. Using function points as a unit of measure, development changes average about 2 percent per calendar month from the end of requirements through the design and coding phases. Thus, if an application is nominally 1,000 function points in size at the end of the requirements phase, changes amounting to about 20 function points in size will occur every month during the subsequent design and coding phases. For legacy applications that are already in use, changes amount to about 7 percent per calendar year. Thus, for an installed application of a nominal 1,000 function points in size, about 70 new and changed function points will occur on an annual basis for as long as the application is used. Failure to anticipate and plan for change requests is a major cause of cost overruns and schedule slips. Failure to include effective methods for sizing and costing change requests is a cause of litigation between outsource vendors and clients.

Problem Management Under both the ITIL concept and normal maintenance concepts, a "problem" is an event that either stops a software application from running or causes the results to deviate from specified results. That being said, problems vary widely in severity levels and overall impact. The worst kinds of problems are high-severity problems that stop an application dead. An equally bad kind of problem is one that destroys the validity of outputs, so the application cannot be used for its intended purposes. Minor problems are those that might degrade performance slightly or small errors that do not affect the usability of the application. In addition to valid and unique problems, there are also many invalid problems reported. Invalid problems are situations that, upon investigation, turn out to be hardware issues, user errors, or in some cases, reports submitted by accident. There are also many duplicate problem reports. These are common for applications with hundreds or thousands of users. Since software applications normally find only about 85 percent of known bugs before delivery to customers, there will obviously be many problem reports after deployment.

Service-Level Agreements One of the key concepts of the ITIL approach and also of normal maintenance is to define and measure service levels of software applications. These levels include reliability measures, availability measures, and performance measures. Sometimes security measures are included as well. One of the gaps of the ITIL materials is the failure to provide quantitative data on the very strong correlations between defect levels and reliability levels. Reliability is inversely proportional to the volume of delivered defects.

Security Measures In the modern world where viruses are daily occurrences, identify theft is a global hazard, and spyware is rampant, all applications that utilize important business, personal, or defense information need to have fairly elaborate security plans. These plans may include encryption of data, firewalls, physical security, and other approaches with varying degrees of sophistication.

Application Deliverable Size Measures Industry leaders measure the sizes of the major deliverables associated with software projects. Size data is kept in two ways. One method is to record the sizes of actual deliverables such as pages of specifications, pages of user manuals, screens, test cases, and source code. The second way is to normalize the data for comparative purposes. Here the function point metric is now the most common and the most useful. Examples of normalized data would be the pages of specifications produced per function point, source code produced per function point, and test cases produced per function point. The function point metric defined by the International Function Point Users Group (IFPUG) is now the major metric used for software size data collection. The total number of projects sized with function points circa 2008 probably exceeded 60,000 in the U.S. and 100,000 on a worldwide basis.

Activity-Based Schedule Measures Some but not all leading companies measure the schedules of every activity and how those activities overlap or are carried out in parallel. The laggards, if they measure schedules at all, simply measure the gross schedule from the rough beginning of a project to delivery, without any fine structure. Gross schedule measurements are totally inadequate for any kind of serious process improvements. One problem, however, is that activities vary from company to company and project to project. As of 2008, there are no standard activity definitions for software projects.

Activity-Based Cost Measures Some but not all leaders measure the effort for every activity, starting with requirements and continuing through maintenance. When measuring technical effort, leaders measure all activities, including technical documentation, integration, quality assurance, etc.

Leaders tend to have a rather complete chart of accounts, with no serious gaps or omissions. Laggards either don't measure at all or collect only project or phase-level data, both of which are inadequate for serious economic studies. Three kinds of normalized data are typically created for development productivity studies: (1) Work hours per function point by activity and in total; (2) Function points produced per staff month by activity and in total; (3) Cost per function point by activity and in total.

Maintenance Productivity Measures Because maintenance and enhancement of aging software is now the dominant activity of the software world, companies also measure maintenance productivity. An interesting metric for maintenance is that of "maintenance assignment scope." This is defined as the number of function points of software that one programmer can support during a calendar year. Other maintenance measures include numbers of customer supported per staff member, numbers of defects repaired per time period, and rate of growth of applications over time. Collecting maintenance data has led to some very innovative results in a number of companies. IBM commissioned a major study of its maintenance operations some years ago and was able to reduce the defect repair cycle time by about 65 percent. This is another example of how accurate measurements tend to lead to innovation.

Indirect Cost Measures The leading companies measure costs of both direct and indirect activities. Some of the indirect activities, such as travel, meeting costs, moving and living expenses, legal expenses, and the like are so expensive that they cannot be overlooked.

Gross Productivity Measures In addition to measuring the productivity of specific projects, it is also interesting to measure gross productivity. This metric is simple to calculate. The entire work force from the chief information officer down through secretaries are included. The total effort expended by the entire set of personnel is divided into the total number of function points developed in the course of a year. The reason that gross productivity is of interest is because it includes overhead and indirect effort and thus provides a good picture of overall economic productivity. However, compared to net project productivity rates, the gross figure will be much smaller. If a company averages 10 function points per staff month for individual projects, it is unlikely that they would top 2 function points per staff month in terms of gross productivity. This result is because of all of the work performed by executives, managers, secretaries, and administrative personnel.

Rates of Requirements Change The advent of function point metrics has allowed direct measurement of the rate at which software requirements change. The observed rate of change in the United States is about

2 percent per calendar month. The rate of change is derived from two measurement points: (1) The function point total of an application when the requirements are first defined; (2) The function point total when the software is delivered to actual customers. By knowing the size of the initial requirement, the size at delivery, and the number of calendar months between the two values, it is possible to calculate monthly growth rates.

Measurement of the rate at which requirements grow and change can also reveal the effectiveness of various methods that might slow down change. For example, it has now been proven that joint application design (JAD) and prototypes can reduce the rate of requirements change to less than 10 percent in total. Here, too, collecting data and analyzing it is a source of practical innovations.

Software Assessment Measures

Even accurate quality and productivity data is of no value unless the reasons *why* some projects are visibly better or worse than others can be explained. The domain of the influential factors that affect the outcomes of software projects is normally collected by means of software assessments, such as those performed by the Software Engineering Institute (SEI), Software Productivity Research (SPR), R.A. Pressman Associates, Howard Rubin Associates, Quantitative Software Management (QSM), Real Decisions, or Nolan & Norton. In general, software process assessments cover the following topics.

Capability Maturity Model (CMM) Level In 1985, the nonprofit Software Engineering Institute (SEI) was chartered by DARPA to find ways of making significant improvements in the development of military software engineering. One of the first efforts by SEI was the creation of a schema for evaluating software development expertise or "maturity." This method of evaluation was published under the name of "capability maturity model" or CMM. The CMM approach assigns organizations to one of five different levels of software development maturity: (1) Initial; (2) Repeatable; (3) Defined; (4) Managed; (5) Optimizing.

The specifics of each level are too complex for this chapter. But solid empirical evidence has been collected that demonstrates organizations at levels 3, 4, and 5 are, in fact, much more successful at building large and complex software projects than organizations at levels 1 and 2. Ascertaining the CMM level of a company is a task normally carried out by external consulting groups, some of which are licensed by the SEI to perform such assessments. It is significant that levels 3, 4, and 5 have very good and complete measurements of software quality and productivity. Indeed, one of the criteria for even reaching level 3 is the presence of a good measurement system. Because the CMM originated for military and

defense software, it is still much more widely used in the defense sector than it is for pure civilian organizations. Comparatively few information technology groups have been assessed or have CMM levels assigned. The systems software community, on the other hand, has also adopted the CMM approach fairly widely.

Software Processes This topic deals with the entire suite of activities that are performed from early requirements through deployment. How the project is designed, what quality assurance steps are used, and how configuration control is managed are some of the topics included. This information is recorded in order to guide future process improvement activities. If historical development methods are not recorded, there is no statistical way for separating ineffective methods from effective ones.

Software Tool Suites There are more than 2,500 software development tools in the commercial market and at least the same number of proprietary tools that companies have built for their own use. It is of considerable importance to explore the usefulness of the available tools and that means each project must record the tools utilized. Thoughtful companies identify gaps and missing features and use this kind of data for planning improvements.

Software Infrastructure The number, size, and kinds of departments within large organizations are an important topic, as are the ways of communication across organizational boundaries. Whether a project uses matrix or hierarchical management, and whether a project involves multiple cities or countries exert a significant impact on results.

Software Team Skills and Experience Large corporations can have more than 70 different occupation groups within their software domains. Some of these specialists include quality assurance, technical writing, testing, integration and configuration control, network specialists, and many more. Since large software projects do better with specialists than with generalists, it is important to record the occupation groups used.

Staff and Management Training Software personnel, like medical doctors and attorneys, need continuing education to stay current. Leading companies tend to provide from 10 to 15 days of education per year, for both technical staff members and for software management. Assessments explore the topic. Normally training takes place between assignments and is not a factor on specific projects, unless activities such as formal inspections or joint application design are being used for the first time. It is an interesting fact that companies providing their technical staff members with about 10 days or more of training each year have higher

productivity rates than similar companies that provide little or no training. Thus measurements have demonstrated a positive return on investment in staff education.

Environment and Ergonomics The physical office layout and noise levels exert a surprisingly strong influence on software results. The best-in-class organizations typically have fairly good office layouts, whereas laggards tend to use crowded cubicles or open offices that are densely packed. There may also be telecommuters or remote personnel involved, and there may be subcontractors at other locations involved. Several studies have been made of office organizations, and the conclusion is that for software projects private offices lead to higher productivity and quality levels than shared offices, although this finding may seem counterintuitive.

Measures and Metrics of Industry Leaders

The phrase "industry leadership" is somewhat ambiguous. The phrase usually refers to a company that has a dominant market share, such as Microsoft. But other components of industry leadership also include research, inventions, product development, shareholder value, profitability, customer support, quality, and reliability. From our consulting studies within several hundred corporations, the following factors are those that stand out in companies that have an excellent combination of market success, a continuous stream of interesting new products, and generally satisfied customers.

Research and Development Developing new products is a hit or miss proposition. Seldom do more than 10 percent of possible new product ideas turn into marketable commodities. Of course, some small companies get started with only a single product. But for large corporations, it is necessary to have a formal structure devoted to research and development. Industry leaders are major investors in research into new product ideas, and may have hundreds of researchers and scores of potential product ideas under development at the same time. IBM's Research Division and AT&T's Bell Labs are examples of major corporate research organizations.

Market Share The most common aspect of industry leadership is that of having major market shares in primary business areas. Examples of major market shares include Exxon, Ford, General Motors, Toyota, Microsoft, IBM, General Electric, Sony, and Boeing. There are also smaller companies that have major market shares in specific niches, such as Seagate with storage devices, Symantec with software utilities, or Meade with optics.

Market Growth The topic of market growth includes both the expansion of sales of current products and also the introduction of new products or even new kinds of products. Market growth may also involve acquisitions of companies with related products, which is sometimes an alternative to in-house research and development.

Shareholder Value For both public and private corporations with shareholders, achieving positive levels of shareholder value is a major responsibility for corporate executives and boards of directors. As witnessed by the many recent corporate financial and accounting scandals, this responsibility has not always been carried out with ethics and honesty. Industry leaders tend to be long-range leaders in returning shareholder value and achieve those results by means of legitimate business activities. Of course, many organizations such as civilian government agencies and the military services have no shareholders.

Time to Market Every company that builds products to sell is concerned with time to market. While industry leaders are very good at achieving fast time to market, they temper the desire to be first in the market with the knowledge that being first with a defective product is not the way to achieve long-range success. Some smaller companies have achieved success by being "fast followers" or bringing out improved versions of products actually pioneered elsewhere. There is some debate in the business literature as to whether being first in the market or being second with additional features leads to the best long-range success.

Unit Development Costs All companies that build products for sale are sensitive to the manufacturing costs of those products. Industry leaders are proactive in both measuring their manufacturing costs and also in making process improvements as the need arises. Reductions in unit development costs for manufactured products can come from three separate strategies: (1) Improvements in development processes; (2) Value engineering or reducing the costs of components in the products themselves; (3) Moving the manufacturing activity to countries or locations with low labor costs. The third of these strategies has become a major concern to the United States since the 1990s as manufacturing work and some technical work have been leaving the country for lower-cost locations. Some successful companies, and in particular many Japanese companies, have found that customers will pay more for products that are highly reliable. Therefore, having the least expensive product is not a guarantee of success.

Customer Satisfaction In every industry having repeat business from satisfied customers is a highly desirable goal. The topics leading to

high levels of customer satisfaction vary from product to product. But some general observations are true across broad ranges of products. Customers demand reliability above features, and when something goes wrong, they demand rapid and competent service. For example, in the 1950s and 1960s IBM grew to dominance in the mainframe computer world by having the best service and customer support of any computer manufacturer. In several large-scale studies of customer satisfaction and buying preferences, quality was the top-ranked factor for many consumer products and also for home appliances. Industry leaders measure customer satisfaction often and carefully. Many also have customer associations such as the SHARE and GUIDE associations of IBM mainframe computer customers. A number of companies, such as Apple, also sponsor major customer conferences both to announce new products and also to gather customer feedback on a variety of topics.

Service and Support Even excellent products can have problems. When problems occur, getting competent support rapidly is a major issue for all corporations. Customer support includes telephone contacts, email, web sites, carry-in or mail-in repair centers, and on-site service. Industry leaders try to be proactive in service and support, although even the best may have lapses. Nothing frustrates customers more than long hold times when reporting problems or customer support personnel who are poorly informed or evasive.

Warranty Repairs The nature of warranties varies widely from product to product and industry to industry. For automobiles and some consumer products, there may be state government regulations that mandate certain kinds of repairs or certain kinds of warranty support. The software industry has long been notorious for an almost total lack of warranties either explicit or implied. A careful reading of software warranties usually reveals that the vendor does not guarantee that the software will even work, and the vendor is to be held harmless if the software causes problems. About the only kind of explicit software warranty is replacement of a disk if it is unreadable. But in most industries the leaders are proactive about warranty repairs and may even provide service or replacement after the nominal warranty period is over. Also, industry leaders keep all kinds of data and statistics about warranty repairs and use root-cause analysis to discover problems that need to be fixed during the manufacturing cycle.

Staff Morale One of the first things consultants notice when working with industry leaders is the excellent morale of almost everyone in the company. People enjoy working for good companies, and they enjoy working on products that are exciting. Of course, every large organization will

experience corporate politics and may have a few disgruntled employees. But on the whole, the staff morale of industry leaders is much better than average. Industry leaders know this, because most of them carry out annual or semiannual opinion surveys. Some also have other channels available for making employee feelings known. IBM, for example, had a program called "Speak Up" though which employees could voice concerns or questions. IBM also had a formal "open-door" policy of appealing to senior executives when an employee felt mistreated in some way. There was a guarantee that such appeals would not result in any punitive or negative treatment.

There are many other topics where industry leaders stand out as being better than average. But the topics cited here have been noted in many different companies, many different industries, and even many different countries.

Measures, Metrics, and Innovation

There are two kinds of innovations that are important to corporations: product and process. Product, or external innovations, involve developing new or improved products that will excite customers. Process or internal innovations, involve developing new or improved methods of development that can shorten development times, reduce costs, or improve quality. Both forms of innovation are important. Both forms are needed for corporations to grow and prosper.

One of the more prominent examples of a person who excelled in both forms of innovation is Henry Ford. Ford's Model A automobile was a classic example of an external product innovation that excited customers. The assembly lines and manufacturing plants that Ford pioneered to construct the Model A are prime examples of internal process innovations. Indeed without Ford's genius at perfecting the assembly line, the manufacturing costs of Model A Fords would have been so high that they might not have replaced horses as the major form of transportation.

In the modern world, both forms of innovation are found in successful corporations that are industry leaders. For example, IBM was a pioneer in the external innovations of computing and storage technologies. IBM also developed very effective internal innovations centering around manufacturing techniques for circuit boards, semiconductors, disk drives, printers, and other computing hardware.

It is interesting that sometimes external product innovations and internal process innovations are at differing levels of sophistication. The imbalance between sophisticated products and primitive manufacturing is illustrated by the software industry. Even in 2008, very sophisticated and complex pieces of software are still constructed by manual methods with extraordinary amount of labor and very distressing quality levels.

Another example of an imbalance between product innovations and process innovations can be seen in the migration of technology jobs from the United States to India, China, and other countries with low labor costs. Many sophisticated products designed in the United States are now being manufactured abroad because the U.S. has not been able to introduce internal manufacturing innovations in sufficient quantities to stay cost competitive.

Measures and Metrics Supporting External Product Innovation

The measures and metrics that can highlight external innovations are those that deal with inventions themselves and also with the success of products in the marketplace. Some examples of measurements and metrics that deal with external innovations include

- Patents issued to employees
- Invention disclosures
- Technical publications
- Research and development spending
- Morale surveys of technical workers
- Market share measurements
- Market growth measurements
- Customer focus group results
- Customer survey results
- Trade show editorial reviews
- Successful completion of government tests
- Loss or gain of research and development jobs

In general, external innovations excite customers and often create new markets for new kinds of products. As this chapter is being written, some of the external innovations that are attracting press coverage and expanding markets include wireless communications devices and protocols, genetically modified plants and food products, laser surgery, and digital cameras with higher and higher pixel counts.

Measures and Metrics Supporting Internal Process Innovation

Internal innovations are subtle and sometimes difficult to measure. Because they often involve improvements in development or manufacturing techniques compared to prior approaches, it is necessary to

have long-range measurements that cover both the "before" and "after" methodologies.

The standard economic definition of productivity is "goods or services produced per unit of labor or expense." One of the main goals of internal innovation is to improve economic productivity. Thus an improvement in economic productivity is a useful though indirect measure of internal innovation.

In the modern hi-tech world, a major component of economic productivity centers around warranty repairs, recalls, and repairing defects in technical products. Therefore, it is apparent that quality and reliability are also in need of internal innovations. Some of the measures and metrics that can highlight internal innovations include

- Time to market: inception to delivery
- Manufacturing cycle time
- Baseline studies showing improvement over time
- Benchmark studies against similar companies
- Cost of quality measures
- Six-Sigma quality measures
- Total cost of ownership measures
- Defect removal efficiency measures
- Scrap and rework during manufacturing
- Manufacturing work hours per unit
- Manufacturing cost per unit
- Warehouse and storage cost per unit
- Distribution cost per unit
- Warranty repairs per unit
- Defect reports per unit per time period
- Defect repair rates
- Recalls of defective units
- Morale surveys of manufacturing staff
- Morale surveys of support and maintenance staff
- Litigation alleging defective products
- Government withdrawal of safety certificates
- Internal Quality Assurance reviews
- Inspection reports
- Test reports

- Customer surveys dealing with quality and reliability
- Customer surveys dealing with service and support
- Loss or gain of manufacturing jobs

As can be seen, the measurement of internal innovations includes measures of cost, measures of time to market, measures of quality, and measures of customer satisfaction. Success in all of these is necessary.

The overall goal of corporate innovation is to have a good balance of both external and internal innovations. New and exciting products need to be coupled with efficient and reliable manufacturing steps.

Measurements, Metrics, and Outsource Litigation

The author and his colleagues have worked as expert witnesses in about 20 lawsuits involving alleged breach of contracts between outsource vendors and clients. We have also worked on about half a dozen tax case disputes between companies and the Internal Revenue Service. We have also been queried on litigation for theft of intellectual property or copyright violations.

It has been surprising that poor measurements and questionable metrics tend to be associated with most disputes and litigation between outsource vendors and clients. Some of the measurement and metric problems observed are discussed next.

Lack of Clauses Dealing with Quality Many outsource agreements have no clauses or sections dealing with the quality of the delivered application. Thus when the applications are put into service and begin to encounter bugs or defects, there is no agreed-to method for handling the problems. Litigation has occurred because of the numbers of defects encountered, their severity, the speed of repairs or due to "bad fixes" or the injection of new defects as a by-product of fixing old defects.

Impossible Quality Demands Some outsource agreements that do include clauses or sections dealing with quality contain impossible demands. For example, one lawsuit was due to a contractual clause that demanded zero defects after deployment of a large system. So far as can be determined, achieving zero defects is beyond the state of the art for large software applications. Of course the vendor and the vendor's attorney should never have agreed to such an impossible demand.

Requirements Changes Software requirements have been measured to change and grow at a rate of about 2 percent per calendar month during the design and coding phases of software development projects.

Some fixed-cost outsource agreements demand that the vendor absorb all of the costs for changes, which can cause the vendor to lose money. Other outsource agreements are ambiguous as to what kinds of changes are to be undertaken at the vendor's expense and what kinds of changes are to be funded by the client. In the absence of any satisfactory clauses in the contracts, litigation may be necessary to resolve the issue.

Schedule Responsibilities for Both Parties Many software projects run late. When the delays begin to exceed 12 calendar months on an outsourced project, litigation may occur. It sometimes happens that the contract had no clauses dealing with milestones other than the final delivery. In other contracts, the responsibility of the vendor is spelled out, but not the responsibility of the client. For example, clients are expected to review and approve particular documents such as design specifications. It has sometimes happened that the documents were delivered on schedule, but the review and approval were delayed and thus threw off downstream activities.

Ambiguous Baselines Some outsource agreements call for productivity improvements over time. In order to determine whether the vendor is meeting the terms of the agreement, it is necessary to have a valid baseline of performance before the outsource agreement commenced. It may also be necessary to have annual baselines for the duration of the contract. In several cases, disputes arose involving the validity of the starting baseline.

Software Development Costs In a number of tax cases, one of the issues of the litigation involved ascertaining the original development cost of software projects. It was surprising to find that some companies do not keep records of software development costs or they keep records in such an ambiguous manner that the costs are suspect. In this situation both the IRS and the company may try to estimate the original development costs. Thus there are often debates between opposing experts as to what the original costs were.

Measurements, Metrics, and Behavioral Changes

One of the major purposes of starting a corporate measurement program is to use the resulting information to achieve behavioral changes. Obviously, the executives and managers who sponsor the measurement program want the behavioral changes to be positive and beneficial. While many behavioral changes in response to measurements are beneficial, it is well known among measurement consultants that sometimes the behavioral changes may be harmful. It also happens

that measurements may move employee groups in directions far from the intended direction.

Here is an example of a measurement program that caused problems. In the early 1970s, IBM attempted to achieve software schedule improvements by issuing outstanding contribution awards to project teams and software engineers who bettered the planned completion schedules of their projects. It was quickly discovered that at least some of the awarded projects had embarrassingly poor quality levels and also poor levels of customer satisfaction, due to poor quality. Therefore the basis of the award was changed. While accelerated schedules were still given awards, the software had to have at least six months of deployment with quality levels at or better than IBM averages and customer satisfaction of at least "good" before the awards were issued. Focusing only on schedule reduction clearly led to results that were harmful in other ways.

IBM had better results with another measurement-based award program. In the late 1970s, IBM's System Development Division made a portion of software executive bonuses contingent upon the software in their organizations achieving customer-reported defect levels 30 percent better than the current corporate average. Within three years, over 90 percent of the division's major software projects had significant declines in customer-reported defect rates. Indeed, the results were so good that IBM's average quality levels improved significantly. The quality data was collected by IBM's independent Quality Assurance groups, and was certified to be accurate. Customer satisfaction improved too. Surprisingly, there were also small improvements in both schedules and development costs. The reasons for this were because finding and fixing bugs are major sources of schedule delays, so when bugs are prevented or removed early, schedules and costs improve.

There are two interesting aspects to this second case. The first is that the 30 percent improvement was actually achievable. This target was set by IBM's Quality Assurance group and was about one standard deviation better than corporate averages. However, there were a number of projects that had already achieved the results, so the target was definitely possible.

Quite a few companies have attempted to set improvement targets for productivity or quality that are ten times better than existing levels. This kind of arbitrary target generally has no behavioral impact, or at least not a beneficial one. Everyone knows that 10 to 1 changes are virtually impossible to achieve so nobody even tries to achieve them.

But a 30 percent improvement target, coupled with real projects already at those levels, is an achievable target for the right kinds of methodologies. Indeed, the managers whose projects were better than the target were in great demand to explain how they had gone about it. (Formal design and code inspections were the key methods used.)

The second interesting aspect of the IBM case is that the behavioral changes were aimed at senior executives who controlled large software projects and many software teams. The rationale was that it was important to set goals primarily for executives who are authorized to spend funds and have the authority to introduce better methods and tools. Since individual software engineers have little authority to introduce process improvements, other than by suggestion, it is important to set improvement targets for higher management levels rather than for individual employees.

Without multiplying examples, it is important to use measurement data carefully and to set targets responsibly. The targets should be realistic and achievable. Do not set arbitrary targets such as "10 to 1" unless you are absolutely certain that the target has been achieved and can be achieved.

Also, do not set targets for people who are not authorized to make changes happen. It is of little value to set targets for individual staff members because they can't buy better tools or introduce better methods by themselves. Targets are only effective when aimed at management levels with spending and methodology authority.

Measurement Targets with Positive Results

It is interesting that most of the quantified targets that have achieved tangible benefits are those associated with quality matters. Some targets that have proven to be successful in actual use are discussed next.

Zero Error-Prone Modules It was discovered by IBM in the 1960s that software defects tend to "clump" in a small number of modules within large systems. For example, a study of IBM's OS/360 operating system found that more than 50 percent of customer reported defects were found in only 5 percent of the modules. Once this was known, other large systems were examined and error-prone modules were found to be very common in all large software applications. IBM set targets of having zero error-prone modules. Within about four years, all known error-prone modules had been removed or corrected and development methods had been established to prevent new ones from occurring. Other large corporations followed suit and also achieved benefits. Formal design and code inspections are the technology that can inoculate software against error-prone modules.

Ninety-five Percent Defect Removal Efficiency Most forms of software testing are only about 30 percent efficient in finding bugs or defects. Even for large software projects that use sequences of test stages such as unit testing, new function testing, stress testing, system testing, and field testing, removal efficiency seldom tops 75 percent. The U.S. average is

only about 85 percent for defect removal before delivery to customers. However, design and code inspections have been measured for years and average about 65 percent each. They also raise the efficiency of machine-based testing. Projects using a synergistic combination of inspections and multiple test stages can achieve better than 95 percent defect removal efficiency levels. These projects will also have shorter schedules and lower costs than similar projects using only testing.

Achieving Level 3 or Higher on the CMM The Software Engineering Institute was founded in 1985 and chartered to find methods for improving government software practices. One of SEI's key accomplishments was the development of a multistage software "capability maturity model" or CMM. (This was recently upgraded to the newer CMMI, or integrated capability maturity model.) The CMM approach defines five levels of software development maturity; 1 initial, 2 repeatable, 3 defined, 4 managed, and 5 optimizing. In the mid-1990s, the U.S. Air Force commissioned a study on the empirical results of the CMM. Overall the data supported the CMM and found that projects produced by Levels 3, 4, and 5 were indeed of higher quality than those produced by Levels 1 and 2. For projects of the same size, schedules and costs were also better at the high end of the CMM spectrum.

Six-Sigma Quality Levels The concept of Six-Sigma quality levels was made famous by Motorola circa 1985. The usual definition of Six-Sigma quality is to have a development process that releases no more than about 3.4 defects in terms of "opportunities" or no more than about 3.4 defects per million parts. Expressed another way, in terms of defect removal efficiency, achieving Six-Sigma quality implies a defect removal efficiency of 99.999999 percent. Some of the companies that have adopted and used the Six-Sigma approach for manufacturing operations include Allied Signal (which merged with Honeywell in 1999), Ford, and General Electric. All of these have reported fairly significant ROI from their use of the Six-Sigma approach. However some cautions are needed. The Six-Sigma approach is complex and pervasive and requires several years to achieve peak performance. It also requires significant training for practitioners, high-level executive commitments, and very thorough and complete measurements. Also, the Six-Sigma approach is still largely experimental for some kinds of enterprises or products, such as those involved with software.

Measurement Targets with Negative Results

It is interesting that the measurement targets that seem to have no positive benefits, or even to be harmful, have one of two major characteristics

in common: one is that they are arbitrary numbers rather than being based on actual results; the second is that they ignore quality. Following are some targets that have behaved badly when actually used.

Improving Productivity by 10 to 1 Over the past 25 years, the author has encountered no less than 50 corporations that have formulated targets of improving productivity levels by 10 to 1. So far as can be determined, none of these companies achieved anything like a 10 to 1 improvement. Indeed, many or most did not achieve any positive gains in productivity at all. The fundamental problem with having a 10 to 1 target for improving productivity is that there is no set of tools, methods, or both that can actually reach the target. In the absence of any proven pathway to achieve the target, the target is quickly ignored. Even worse, setting an arbitrary target such as 10 to 1 without demonstrating that it can be achieved makes the executives who set the target look foolish to their subordinates, employees, and eventually to their superiors and shareholders as well.

Short-Term Productivity Gains in One Year Most of the technologies that benefit quality and/or productivity require substantial start-up training before they can be used successfully. Not only that, but the transition from older technologies to newer technologies may take quite a few months. For example, projects already more than half complete using older technologies cannot just stop and retool in the middle of development. The new approaches have to be phased in on new projects. As a result, it is very difficult to achieve major benefits in productivity in less than about 24 months. Quality is a little easer, but even so, benefits are not likely to become visible in less than about 18 months. Therefore, short-term goals such as "improve productivity by 25 percent next year" are very likely to be failures.

Productivity Improvements Decoupled from Quality As already illustrated in the IBM example at the start of this chapter, attempts to improve productivity that are not linked to quality are not likely to be successful. From examining the data on thousands of software projects, it is clear that the most expensive and time-consuming activities in software development are those associated with finding and fixing bugs or defects. It is not really possible to make major cost or schedule improvements on software projects without using methods that prevent defects from occurring, remove them early, or both. Thus it is not a safe practice to set productivity targets without also setting quality targets. Indeed, the reverse concept is strongly supported by empirical data. Improving software quality will also improve software schedules and software costs even though these benefits are unintentional. This was discovered by

IBM in the 1970s. The original IBM efforts were triggered by customer complaints about poor quality levels. IBM discovered that by improving quality they had also managed to reduce the major source of schedule delays and cost overruns.

Arbitrary Schedule Targets Software schedules should be derived from careful analysis of application requirements coupled with full understanding of team capabilities. Far too often software schedules are set arbitrarily by either client demands or senior executive mandates. When this happens, the project may well be facing disaster. In order to achieve significant schedule reductions, it is necessary to identify and reduce the major sources of schedule delays. From measurements of hundreds of projects within scores of corporations, the two dominant sources of schedule delays are: (1) the time required to find and fix bugs or defects and (2) the time required to incorporate changing requirements introduced late in the development cycle after requirements definitions are nominally complete. Although brute-force approaches such as massive bursts of overtime can compress schedules slightly, it is not possible to make substantial schedule reductions without improving quality and controlling requirements creep. Setting an arbitrary deployment schedule without using methods to control quality and requirements change will probably lead to either massive overruns or outright cancellation of the project in question.

Increasing "Lines of Code per Month" In the past, several companies have offered various rewards to software engineers to spur them to achieve higher productivity rates measured using lines of code. None of these attempts have been successful. Usually one or both of two harmful results occur: (1) Extraneous code is written simply to puff up the code count; (2) Low-level programming languages are used because they require more code to complete any given set of requirements. Neither of these reactions improve economic productivity. All they do is make "lines of code per month" look greater than before, without achieving any economic benefit whatsoever. Thus "lines of code" is a metric that must be used with caution.

Reducing Cost per Defect Reducing the costs of defect repairs is definitely advantageous to all corporations. Unfortunately the metric "cost per defect" tends to go up as defect volumes go down. This leads to a paradoxical result: As quality improves cost per defect will go higher until zero defects are achieved, at which point the cost per defect will be infinite! The problem with cost per defect resides in the fixed costs that are always part of any business activity. If you have a maintenance specialist who earns $5,000 per month and fixes ten defects this month, then

the cost per defect will be $500. But suppose next month only five defects are reported. The cost per defect will double to $1,000. The reason is that the maintenance specialist is still going to get paid for a whole month's work whether fixing defects or not. Suppose in the third month, zero defect are reported. The maintenance specialist will still be paid $5,000, but this month there were no defects so the cost per defect goes to infinity. Of course, it might be possible to reassign the maintenance specialist to other projects, but by and large, the cost per defect metric causes more trouble than necessary.

In conclusion, measurement and metrics have often been shown to introduce behavioral changes into the way companies go about their business. This means that measurements and metrics must be carefully planned and thought out to ensure that the behavioral changes are positive and beneficial. It often happens that simplistic metrics or metrics known to cause problems will inadvertently trigger harmful behavioral changes instead of beneficial ones.

Topics Outside the Scope of Current Measurements

Although measurement and metrics have made significant progress in recent years, there are still some important topics that are outside the scope of current measurements. The primary reason is because there are no effective size metrics or normalizing metrics for these "missing links" in the measurement chain.

Database Measures The most important missing link in corporate and software measurements concerns the data contained in databases, repositories, and data warehouses. It is suspected that corporations own much more data than they own software and that the costs of development and maintenance are higher. It is also suspected that the error density of data is actually higher than the error data of software, which is an alarming hypothesis. Unfortunately there is no "data point" metric for examining the size or volume of data that companies own. As a result, there are no reliable studies on database economics, data quality, data longevity, nor any other significant economic topic associated with data and information.

Web Content Measures Corporate web sites may contain thousands of pages of text, thousands of images, and also sounds, music, recordings, and dynamic images such as video segments. Corporate web sites may also contain "applets" or possibly full-scale applications that can be invoked by users on demand. There are no effective measurements of the size or volume of the contents of a web site, other than the amount of storage required to hold everything. As a result, there is little or no

economic knowledge dealing with the cost of ownership, maintenance costs, quality, or return on investment of web sites.

Intangible Value Measures Although there are effective measurements for return on investment (ROI) and rate of return, there are no common or widespread measurements for the intangible value of software. For example, if a software application benefits national security, human life and safety, employee morale, or corporate prestige, there is no way to measure these factors in a quantitative way. The relationship between intangible value and ROI is one of the critical junctions for corporate investment and sometimes a barrier to innovation. If strict financial ROI is the prime criterion for technology investment, then some projects or acquisitions may not go forward.

Canceled Projects and Software Overruns Because software is a difficult technology to control, failures and disasters are quite common. Indeed most studies have indicated that failed projects outnumber successful projects for applications larger than 10,000 function points in size. Because unfinished projects may not have accurate size data, it is hard to explore the economics of software failure. What is needed is some kind of relative scale for failing projects, possibly similar to the Richter scale for earthquakes or the scale used to measure the strength of hurricanes. For example, a class 1 failure might be one that cost less than $100,000, whereas a class 2 failure might cost between $100,000 and $1,000,000. A class 3 failure might cost between $1,000,000 and $10,000,000 and so on.

Cautions Against Simplistic and Hazardous Measures and Metrics

Metrics and measurement consultants are painfully aware that there is a widespread tendency to seek simplistic measurements such as "industry averages." In the course of a business year, the author receives at least 50 emails or telephone calls asking for industry averages for software productivity, quality, and costs. Unfortunately many of the callers want this information in order to ascertain how their own organizations compare to the overall average. What they should want to know is how they compare to similar organizations within their own sub-industry.

Industry Average Productivity While it might be possible to construct such an average productivity value from the results of all measured software projects, it would not have much practical use. Small software projects can be more than ten times as productive as large systems. Information systems can be more than three times as productive as military systems of the same size. Software projects performed by groups that rank

above 3 in terms of CMM levels can be more than twice as productive as the same kinds of projects developed by groups that rank at level 1 on the CMM scale. Thus an overall average might be interesting, but of no practical value in assessing productivity or quality levels of actual projects in progress. An overall industry average is somewhat equivalent to the average life expectancy of a country. While a value of about 8 function points per staff month is sometimes cited as the industry average for the United States, the measured range runs from less than 1 function point per staff month to more than 50 function points per staff month. What is much more effective than a generic industry average is to consider the productivity rates of projects organized by size, by type, and by sub-industry.

Average Cost per Function Point Software projects developed within industries that have high compensation levels and overhead levels, such as banks, can cost three times more than the same project done in low-cost sectors such as education or state government. Projects done by large companies with high overhead rates can be more than twice as costly as the same project done by a small company with low overhead rates. Due to inflation, projects developed 10 years ago should be about 35 percent less expensive than projects developed in 2008. Projects developed in high-cost countries such as Switzerland can cost more than four times as much as similar projects in low-cost countries such as China. While an average cost of about $1,000 per function point is cited for the United States, the measured range is from less than $100 per function point to more than $60,000 per function point. What is more effective is to consider the costs of projects of similar sizes, similar types, and taken from the same or similar sub-industries.

Without multiplying examples, it is hazardous to think that a single value such as an industry average has any practical effect. Some other examples of hazardous simplistic metrics also include the following.

Cost per Defect This metric has been in use since the 1960s for software, and probably for at least 50 years before that for hardware and engineered products. It is often reported in the trade press that "it costs 100 times as much to fix a defect in the field than in design."

The basic problem with the cost per defect metric is that all defect removal operations have a significant quantity of fixed costs associated with them. Thus, as the number of defects reported declines, the cost per defect must rise. Thus cost per defect is not valid for serious economic analysis because every downstream activity will have a higher cost per defect than upstream activities.

Assume that a software application contains 100 defects. Now assume that it will go through three consecutive test stages, each of which will find 50 percent of the latent defects. Writing the test cases for each test

stage costs $1,000. Running the tests for each stage costs $1,000. Fixing each discovered defect costs $100. Consider the economics of the three test stages.

In the first test stage, the costs were $1,000 for writing test cases, $1,000 for running test cases, and $5,000 for fixing 50 defects, or $7,000 in all. The "cost per defect" for test stage 1 would be $70.

In the second test stage, the costs were $1,000 for writing test cases, $1,000 for running test cases, and $2,500 for fixing 25 defects, or $4,500 in all. The "cost per defect" for test stage 2 would be $180.

In the third test stage, the costs were $1,000 for writing test cases, $1,000 for running test cases, and $1,200 for fixing 12 defects, or $3,200 in all. The "cost per defect" for test stage 3 would be $266.

As can be seen from this simplistic example, the fixed costs of writing and running test cases tend to drive up the later test stages where few defects are found. If we continued this sequence, we would find that every downstream test stage has a higher cost per defect than the one before it, because the downstream tests find fewer defects than the upstream tests. This example is simplistic, but it does illustrate that fixed costs can lead to incorrect economic conclusions as defect volumes decline.

Interestingly a switch to "defect removal costs per function point" will track the economic costs of testing quite well. Assume the project just illustrated contains 100 function points. Thus the defect removal cost per function point for test stage 1 would be $70; for test stage 2 it would be $45; and for test stage 3 it would be $32.

Cost per Line of Code The basic problem with this metric is that it penalizes high-level programming languages and makes low-level languages seem better than they are.

The lines of code or LOC metric has been used for almost 50 years, but has never been standardized for even a single language. In 2008 there are more than 700 programming languages in use throughout the world, and some large systems may utilize as many as a dozen languages simultaneously. About one-third of software applications utilize at least two languages simultaneously, such as COBOL mixed with SQL.

The real problem with LOC metrics is not the lack of standardization, but the fact that this metric is paradoxically misleading when used with high-level languages. It is also misleading when analyzing portfolios or sets of projects that involve more than a single programming language.

The reason that LOC metrics give erroneous results with high-level languages is because of a classic and well-known business problem: the impact of fixed costs. Coding itself is only a small fraction of the total effort that goes into software. Defect repairs cost more. Also, paperwork in the form of plans, specifications, and user documents often cost much more, as does defect removal.

Here is a simple example, showing the lines-of-code results for doing the same application in two languages: basic assembly and C++. For simplicity, assume that the programming staff in both examples earn $5,000 per month.

Assume that the assembly language program required 10,000 lines of assembly code, and the various paper documents (specifications, user documents, and so on) totaled to 100 pages. Assume that coding and testing required ten months of effort, and writing the paper documents took four months of effort.

The entire project took a total of 14 months of effort and so had a productivity rate of 714 LOC per month. At $5,000 per month, the cost for this project was $70,000. With 10,000 lines of code, the cost per line of code was $7.00.

Assume that the C++ version of the same application required only 1,000 lines of code. The design documents probably were smaller as a result of using an OO language, but the user documents are the same size as in the previous case: assume a total of 75 pages were produced. Assume that coding and testing required 1 month and document production took three months. Now we have a project where the total effort was only four months, but productivity expressed using LOC is only 250 LOC per month.

The cost for this project at $5,000 per month was $20,000. With only 1,000 lines of code created, the cost per line of code is $20.00.

Consider the bizarre results of using lines of code metrics. Although the C++ version of the project cost $50,000 less than the assembler version, its cost per line of code is $20.00 as compared to the $7.00 per line of code for the assembly program.

Since the standard economic definition for productivity is "goods or services produced per unit of labor or expense," it can be seen that LOC metrics do not measure economic productivity at all.

If function point metrics were used, the true economic benefits of the high-level programming language would be apparent. Since both the assembler version and the C++ version do the same things, their function point totals would be identical. Assume 50 function points for both cases. Now we see that the cost per function point for the assembler project was $1,400 ($70,000 divided by 50) whereas the cost per function point for the C++ version was only $400 ($20,000 divided by 50). Thus function point metrics actually measure economic productivity, whereas LOC metrics do not.

Commercial Software Measurement Tools

Measurement using manual methods is difficult and expensive. For many years, the only effective measurement tools were proprietary

ones built by various corporations for their own internal use. Starting about 20 years ago, a new sub-industry began to emerge of companies that built measurement tools. For general corporate measurements and manufacturing, the vendors of enterprise resource planning (ERP) tools have achieved very significant market penetration. Some of these companies include SAP, Baan, and PeopleSoft. Curiously, ERP packages usually have little or no capabilities for software projects. There are other vendors selling measurement tools for software quality, complexity, defect tracking, cost tracking, schedule tracking, tools inventories, function point tracking, and many other measurement topics such as the David Consulting Group, Quantitative Software Management (QSM), and Software Productivity Research (SPR).

As of 2008, the measurement tool sub-industry is composed of at least 50 companies and more than 100 products in the United States alone. The best way to explore the rapidly growing measurement sub-industry is to visit the vendor showcases of software conferences dealing with quality, application development, or the metrics conferences sponsored by nonprofit groups such as the International Function Point Users Group (IFPUG), the Society of Cost Estimating and Analysis (SCEA), or the International Society of Parametric Analysis (ISPA).

Summary and Conclusions

Measurements and metrics are important corporate activities. Leading corporations tend to use more measurements than laggards and to use the results more wisely. The best companies do not just passively collect data. They analyze the results and then make deliberate attempts to improve weak areas and invest more into strong areas.

International competition for U.S. companies is increasing every day. The best plan for continued success in global markets is for U.S. companies to achieve world-class levels of quality, reliability, customer satisfaction, and time to market. The only proven way to improve these factors is to measure current results, analyze the data, and then plan improvement strategies based on solid, tangible measurements. Thus, good measurement is on the critical path to successful competition in world markets.

The U.S. software industry is struggling to overcome a very bad reputation for poor quality and long schedules and disastrous project failures. The companies that have been most successful in improving quality and shortening schedules have also been the ones with the best measurements. Even better, the companies that have achieved the best software quality results have also lowered their development and maintenance costs at the same time. This is because defect removal, historically, has been the top ranked expense element for software.

The U.S. software industry is facing major challenges from overseas vendors with markedly lower labor costs than U.S. norms. Measurements of software quality and productivity are already important business tools. As offshore software vendors use metrics and measurements to attract U.S. clients, good measurements may well become a business weapon.

Suggested Readings on Measurement and Metrics

The literature on both corporate and software measurement and metrics is expanding rapidly. Following are a few samples of some of the more significant titles to illustrate the topics that are available.

Bakan, Joel. *The Corporation: The Pathological Pursuit of Profit and Power*. New York, NY; The Free Press, 2004.

Boehm, Barry W. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.

Crosby, Philip B. *Quality is Free*. New York: New American Library, Mentor Books, 1979.

Ecora Corporation. Practical Guide to Sarbanes-Oxley IT Internal Controls. Portsmouth, NH: Ecora, 2005: www.ecora.com.

Garmus, David and David Herron. *Function Point Analysis*. Boston: Addison Wesley Longman, 2001.

Garmus, David and David Herron. *Measuring the Software Process: A Practical Guide to Functional Measurement*. Englewood Cliffs, NJ: Prentice Hall,1995.

Grady, Robert B. and Deborah L. Caswell. *Software Metrics: Establishing a Company Wide Program*. Englewood Cliffs, NJ: Prentice Hall, Inc., 1987.

Howard, Alan (ed.). *Software Metrics and Project Management Tools*. Phoenix: Applied Computer Research, 1997.

International Function Point Users Group. *IT Measurement*. Boston: Addison Wesley Longman, 2002.

Jones, Capers. *Estimating Software Costs*. 2nd Ed. New York: McGraw-Hill, 2007.

Jones, Capers. "Sizing Up Software." *Scientific American,* vol. 279, no. 6 (December 1998): pp 104–109.

Jones, Capers. *Software Assessments, Benchmarks, and Best Practices*. Boston: Addison Wesley Longman, 2000.

Jones, Capers. *Conflict and Litigation Between Software Clients and Developers*. Burlington, MA: Software Productivity Research, 2003.

Kan, Stephen H. *Metrics and Models in Software Quality Engineering.* 2nd Ed. Boston: Addison Wesley Longman, 2003.

Kaplan, Robert S. and David B. Norton. *The Balanced Scorecard*. Cambridge: Harvard University Press, 1996.

Kaplan, Robert S. and David B. Norton. *Strategy Maps: Converting Intangible Assets into Tangible Outcomes*. Boston: Harvard Business School Press, 2004.

Pohlen, Terrance L. "Supply Chain Metrics." *International Journal of Logistics Management;* vol. 2, no. 1 (2001): pp 1–20.

Putnam, Lawrence H. *Measures for Excellence—Reliable Software On Time, Within Budget*. Englewood Cliffs, NJ: Yourdon Press, Prentice Hall, 1992.

Putnam, Lawrence H and Ware Myers. *Industrial Strength Software - Effective Management Using Measurement*. Los Alamitos, CA: IEEE Press, 1997.

Miller, Sharon E. and George T. Tucker. "Software Development Process Benchmarking." *IEEE Communications Society* (December 1991). Reprinted from IEEE Global Telecommunications Conference, December 2–5, 1991.

Information Technology Infrastructure Library, www.wikipedia.org (accessed 2007).

Chapter

7

Summary of Problems in Software Measurement

This final chapter of *Applied Software Measurement* summarizes and condenses all of the measurement and metrics problems observed in the software industry over the past 40 years of study.

Until software size, quality, and productivity can be measured with precision the phrase "software engineering" is a misnomer. There can be no serious engineering without accurate measurements. At the present time, effective software measurements are not used widely enough to support the word "engineering" for software occupations.

Many common software metrics such as lines of source code, cost per defect, and work hours per function point have not been evaluated under controlled or standard conditions. When they are evaluated under controlled conditions, some of these, for instance, "cost per defect" and "lines of code," prove to have errors of alarming seriousness. Metrics such as lines of source code that are widely viewed as being objective are, in fact, highly subjective. Even worse, such metrics tend to fail when used for economic analysis.

Measurement, metrics, and statistical analysis of data are the basic tools of science and engineering. Unfortunately, the software industry has existed for almost 60 years with a dismaying paucity of measured data, with metrics that have never been formally validated, and with statistical techniques that are at best questionable.

As the 21st century unfolds, the phrase "software engineering" must cease being an oxymoron and become a valid description of a true engineering discipline. An important step in this direction will be to evaluate and validate all of the common metrics and measurement techniques for software under controlled conditions, in order to select a standard set that can be used with little or no ambiguity and with minimal subjectivity.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

This chapter recapitulates a number of critical problems in the practices of measuring software projects discussed earlier.

Synthetic vs. Natural Metrics

Consider some of the basic metrics that confront us in a normal day. Upon arising, we may check the outside *temperature* and perhaps note the *barometric pressure* to see if it might rain. At breakfast, we may think about the *cholesterol levels* and the *calories* in the food we eat. If we purchase gasoline on the way to work, we perhaps consider the *octane rating* of the fuel we are purchasing. We might also reflect on the *horsepower* of the automobile engine or calculate the *miles per gallon* we received with our last tank of gasoline. If we use computers at work, we are probably aware of the processing speed expressed in *megahertz* and the capacities of our disk drives expressed in *megabytes* or *gigabytes*. Other common metrics that occur less often but are still part of our basic metrics vocabulary include *amps*, *watts*, *volts*, *ohms*, and *British thermal units*.

All of the above metrics are interesting in several respects: (1) They are synthetic metrics that deal with invisible phenomena that cannot be counted directly; (2) More than 95 percent of adult humans understand the significance and basic meaning of these metrics; (3) Less than 1 percent of adult humans know how to calculate these metrics or understand their mathematical derivations.

In our daily lives, we also deal with natural metrics, too. Natural metrics are usually simple integers that are used for counting tangible objects. For example, at a grocery store we might buy a *dozen eggs* or a *pound of coffee*.

It is quite instructive to read Asimov's *Biographical Encyclopedia of Science and Technology* and note the evolution of various sciences and engineering disciplines. In the formative years of chemistry, physics, meteorology, and electrical engineering, there were no synthetic metrics. The pioneering scientists and engineers started by exploring unnamed, unquantified phenomena and created the metrics as part of their initial studies. This is why so many synthetic metrics are named in honor of pioneering researchers: Ampere, Henry, Joule, Ohm, Volta, Celsius, and Fahrenheit, for example.

Now let us consider the metrics history of software engineering. Prior to the 1970s, the phrase "software engineering" did not exist. Those of us in the field were simply called "programmers" or sometimes "programmer/analysts." Having been one myself, I think the term "software engineer" was and still is a misnomer.

The only metrics that we used were simple natural metrics such as integers. We tended to use lines of code because we wrote our programs

on special programming tablets where the lines were clearly visible and often numbered. Some programming languages such as Basic could not even be used without line numbers.

In the 1960s and 1970s, with languages such as Assembly, COBOL, and FORTRAN being used for the bulk of commercial programming, the effort devoted to programming was the dominant activity of software, and other activities such as requirements, design, and user documentation were seldom measured at all. When they were measured, we used simple natural metrics such as integer counts of the pages or words. Graphics seldom occurred in commercial software, and topics such as reusable code, object-oriented languages, pull-down menus, graphical interfaces, mice, etc. were still in the future.

What we programmers did was often useful and sometimes carefully constructed (and sometimes not), but it was not true engineering. The results were not predictable, and if two programmers were given the same task they usually produced two notably different programs. The best that could be said is that a good programmer was a good craftsman.

The first powerful synthetic metric developed for software was the function point, which was created in the middle 1970s by Allan Albrecht and his colleagues at IBM. This metric was placed in the public domain in October of 1979 at a joint SHARE/GUIDE/IBM conference in Monterey, California.

If future historians want to explore the evolution of software engineering as a true engineering discipline, October 14, 1979, is a strong contender for being the exact starting point. Allan Albrecht's presentation in Monterey marks the first day in software history that an effective synthetic metric for software was publicly stated.

(Note: Tom DeMarco independently developed his "Bang" functional metric at about the same time as Albrecht and his colleagues at IBM developed function points, but Albrecht's work was published prior to DeMarco's.)

The pioneering work of Bohm and Jacopini in the 1950s on software structure and Edsger Dijkstra's famous 1968 letter to the *Communications of the ACM* on "GOTO's Considered Harmful" are often considered the starting points of software engineering. But without effective metrics there is no true engineering. Bohm, Jacopini, Dijkstra, Mills, and other pioneers contributed greatly to the craft of programming, but they did not turn the craft into an engineering discipline because their work did not lead to effective metrics or measurement methods.

The power of synthetic metrics lies in their generality. For example, the metric horsepower can be used to measure electric motors, gasoline engines, and diesel engines with equal precision. The function point metric also offers generality to software engineers and managers: it can be used to measure requirements, design, coding, technical writing,

management, and even user contributions to software. The function point metric can handle reusability, object-oriented methods, graphical user interfaces, and many other objects and activities that far exceed the capabilities of natural metrics such as lines of code.

If software engineering follows the path of other engineering disciplines such as electrical engineering or telecommunications engineering, many other synthetic metrics will also be developed. McCabe's "cyclomatic complexity" and "essential complexity" metrics are also synthetic, and are also general enough to be used with specifications as well as code, although it is obvious that much more work is needed in the domain of complexity metrics. The future is difficult to see, but one point has been true throughout scientific and engineering history: Without accurate measurement and without effective synthetic metrics, there is no science and there is no engineering. Our joint task for software engineering is to make our craft a true engineering discipline. This chapter discusses the critical measurement topics that must be resolved for software engineering to become a true engineering discipline.

Ambiguity in Defining the Nature, Scope, Class, and Type of Software

As discussed earlier in this book, the most basic ambiguity in software engineering is the lack of a good taxonomy that can be used to establish the positioning of various kinds of software products for the purposes of economic study.

In our daily lives, the existence of useful taxonomies serve a variety of economic purposes. In another context, all of the following vehicles will have an average manufacturing cost:

- Bicycle
- Motorcycle
- Automobile
- Farm tractor
- Six-wheel cab truck
- Indianapolis racing car

It would be very unusual to see a journal article recommending, for example, that farmers stop buying tractors and start buying bicycles because the costs are so much less. Obviously bicycles can't do some of the things that farm tractors do, such as pull plows and harvest equipment. Neither would a farm tractor compete against an Indianapolis racing car in terms of speed.

However, almost every month, software journal articles are published that recommend switching to end-user developed software or Agile methods rather than commercially produced software because the costs are so much lower. There are plainly things that end-user software can't do or at least can't do safely: Would you want to fly in airplanes where the navigational software was written by the crew shortly before take off? The Agile method is more serious and sophisticated than end-user software, but is still not certified for embedded and systems software that control complex equipment such as medical devices or weapons.

The fundamental point for software is that "apples to oranges" comparisons are the norm, rather than the exception, because the software industry has only a hazy understanding of how to classify projects for economic purposes. After some 60 years of existence, there is still no satisfactory taxonomy that can be used to place any given software project in a given category for the purposes of value analysis, cost analysis, quality analysis, and economic understanding.

As another example, consider the usefulness of the Kelley Blue Book for finding the costs of automobiles. Part of its utility is due to the taxonomy that readers take for granted. Consider the probable cost differences between two automobiles based on their relative taxonomies:

	Automobile A	Automobile B
Age:	Used Car	New Car
Origin:	American	German
Style:	Four-door sedan	Sports Utility
Engine:	4 cylinder; 150 horsepower	V8; 285 horsepower
Transmission:	Manual transmission	Six-speed automatic transmission
Accessories:	AM/FM radio	DVD; satellite radio; On-Star
Probable cost:	<$15,000	>$50,000

Without knowing the specifics of either automobile, it is obvious from the taxonomy itself that Automobile B is going to cost a great deal more than Automobile A. What we need for software projects is a taxonomy that will also allow us to know at a glance the probable costs and outcomes.

In exploring this problem, the author has concluded that a flexible and expandable multitier taxonomy is the most practical. This approach is still evolving and is not a standard, but it has been used for more than 20 years with useful results.

The current taxonomy encompasses five separate aspects of software: nature, scope, class, type, and size. A specific software project can be identified with considerable precision by combining these five parameters.

The "nature" parameter denotes whether a given software project is brand new, is being modified, or is an evolution of another kind of software. As of 2008, the nature parameter encompasses these forms:

PROJECT NATURE

1. Package acquisition (no new development)
2. Package acquisition and modification (some customization)
3. New program development
4. Enhancement (new functions added to meet new requirements)
5. Maintenance (defect repairs)
6. Mixed (both new functions and defect repairs)
7. Code conversion (code moving to a new language)
8. Code renovation (dead code removal; error-prone module removal)
9. Full conversion (software moving to a new platform, with new documentation)

Consider the implications of comparing two software projects without knowing their nature, such as comparing the costs of an enhancement to the costs of a new project. It is obvious that if the enhancement and the new software are both the same size, say 100 function points or 10,000 source code statements, that the enhancement will cost more. The reason is that when existing software is being modified or enhanced, it is necessary to recompile some of it and to test much of the original software to ensure that has not been damaged. The user documentation and other paper deliverables will also require updating. Thus comparing new projects to enhancements, without realizing that two different kinds of projects were being considered, might well lead to erroneous conclusions.

The second of the five parameters is termed "scope." The *scope* parameter is used for both estimation and measurement purposes and attempts to bound the project under consideration. As of 2008, the scope parameter includes some 15 sub elements:

PROJECT SCOPE

1. Subroutine
2. Module or subelement of a program
3. Reusable module
4. Disposable prototype
5. Evolutionary prototype
6. Complete stand-alone program

7. Component of a system (Similar programs performing major functions)
8. New departmental system (multiple programs or components)
9. Minor release (defect repairs and small enhancements to existing system)
10. Major release (major new functionality for existing system)
11. New enterprise system
12. New corporate system
13. New service-oriented architecture (SOA) system
14. New national system
15. New global system

Here too, this kind of information should be a permanent part of any productivity or economic study. Failure to record the scope of projects has caused no end of trouble for the software industry, and indeed is one of the primary contributing factors to exaggerated claims and false advertising by vendors who often compare results from small projects such as stand-alone programs against major projects such as new systems.

The specific problem is the side-by-side comparison of projects with unlike scopes. Many of the misleading productivity claims stem from comparing prototypes to complete applications. Prototypes seldom have much in the way of specifications, user documentation, or defect removal rigor. Therefore, the productivity rates of prototypes are often rather high (averaging more than 25 function points or 2,000 code statements per month). But prototypes are not complete applications, in terms of rigor of the user's guides, tutorial materials, quality control methods, and many other aspects. It is not a fair comparison to simply show a prototype side by side against a full project and use the difference for advertising purposes without clearly stating that the two versions were not equivalent.

There are similar scale differences between stand-alone programs and systems comprised of multiple programs and multiple components. Systems, for example, require a number of activities that are not usually part of stand-alone program development: system-level design, architecture, integration of multiple components, system-level testing, and system-level documentation are all expense elements that are outside the domain of any given program within the system.

The point is that *scope* information should be recorded and should be identified when comparing any two projects for economic or productivity purposes.

The next parameter is termed the "class" of the application. The term "class" denotes the specific business arrangement under which the

project will be funded. The class parameter has major implications in terms of project costs, since as a rule of thumb the costs of software paperwork are directly correlated to the class of software. As of 2008, the author's class list identifies 16 discrete business arrangements:

PROJECT CLASS

1. Personal program for private use
2. Personal program, to be used by others
3. Academic program, developed in an academic environment
4. Internal program, to be used at a single location
5. Internal program, to be used at multiple locations
6. Internal program to be put on an intranet
7. Internal program, developed by external contractor
8. Internal program, developed by outsource contractor
9. Internal program, with functions accessed via network
10. Internal program (produced by a military service)
11. External program, to be put in the public domain
12. External program to be put on the Web
13. External program, leased to users
14. External program, bundled with hardware
15. External program, unbundled and marketed commercially
16. External program, developed under commercial contract
17. External program, developed under government contract
18. External program, developed under military contract

There are also supplemental factors associated with the class parameter:

- For the military classes (10 and 18), it is significant to know which military standards such as DoD 2167A or others (or, in some cases, none) will be adhered to.
- For the contract classes, it is significant to know whether the form of the contract will be fixed price, time and materials, or something else. It is also significant to know whether the contracting work will be carried out on the client's premises, or on the contractor's premises. It is significant to know if the contract will be let to a single individual, a single company, or to multiple contractors. It is also significant to know if the contract will be domestic or international.

- For projects in classes 1 through 4, paperwork costs for requirements, plans, specifications, and user documents are comparatively inexpensive and may not exceed 15 percent of the total costs of the application. For the upper classes, 10 through 18, paperwork costs will be major cost elements, and often more expensive than the code itself. For the two military classes (10 and 18) paperwork may cost twice as much as coding and absorb more than 50 percent of the total effort devoted to the software. Here too, the point is that this kind of information is an essential part of software measurement. Without knowledge of the class of software, there is no way to perform serious economic and productivity analyses.

The fourth element in the taxonomy is the "type" of software that must be produced. The type parameter is one of the key determinants of coding costs. As a general rule, the complexity of the applications with higher type numbers is much greater than the complexity of applications with lower type numbers. As of 2008, there are 22 types used in the author's taxonomy:

PROJECT TYPE

1. Nonprocedural (generators, query, spreadsheet, etc.)
2. Web application
3. Batch applications
4. Interactive applications
5. Batch database applications
6. Interactive database applications
7. Client-server database applications
8. Computer games
9. Scientific or mathematical
10. Expert or knowledge-based system
11. Systems or middleware
12. Communications or telecommunications
13. Process control
14. Embedded or real-time program
15. Trusted system with high reliability and security levels
16. Graphics, animation, or image processing program
17. Multimedia application
18. Robotics, or mechanical automation program

19. Expert system with substantial knowledge acquisition
20. Artificial intelligence program
21. Neural net application
22. Hybrid application (multiple types)

Type 22, hybrid, is very common. About a third of all projects can be classed as hybrid. As a rule, hybrid projects are more troublesome and expensive than the "pure" types. They are also harder to estimate and often difficult to measure.

Unless "type" information is available, it is very easy to make serious mistakes when comparing software productivity, quality, or any other tangible aspect. Consider the different kinds of work that must be performed when building type 2 software (web applications) and type 18 (robotics or mechanical automation program). Obviously, quite different approaches are needed for activities such as requirements analysis, design, and testing between these two classes. The point, of course, is that type data should be a standard item of software measurement.

The next element of the author's taxonomy is the size of the application. The term "size" is highly ambiguous within the software industry. In common usage, it refers to either the quantity of functions in the application (as expressed with metrics such as function points) or to the quantity of source code in the application (as expressed with metrics such as LOC, KLOC, or KSLOC). The term "size" also refers to specific deliverables, such as the number of pages in a specification, or to the number of test cases.

For measurement purposes, it is very important to know if the size of an application is small, medium, large, or very large. Here, there is substantial ambiguity. So far as can be determined, there are no industry standard definitions for what it means, exactly, for an application to be small or large.

In reviewing the major software journals, such as the *IEEE Transactions on Software Engineering for the Communications of the ACM,* it is quite astonishing to see applications of some nominal size be referred to as "small" in one article and "large" in another. For example, in the commercial literature produced by authors employed by major companies such as AT&T, IBM, and Motorola, a size of 500 function points or 50,000 source code statements is usually considered to be medium size or less. Large systems are often viewed as more than 10,000 function points and 1,000,000 source code statements in size. However, in articles produced by universities or academics, 500 function points or 50,000 source code statements are often considered to be a large system. Some academic articles actually cite applications of 50 function points or 5,000 source code statements as "large" and reserve

the phrase "small" for those that hover around 1 function point or 100 source code statements in size. The point is that the software literature uses terms such as "small," "medium," and "large" with so much ambiguity that a more precise quantification would be helpful.

The SPR taxonomy for size is based on observations of the kinds of activities and deliverables that are usually associated with software. Since 1991, the SPR size taxonomy has been based on roughly the following assumptions and seven size plateaus:

Descriptive Term	Size in Function Points	Size in KLOC
1 = Very small applications	1–100	1–10
2 = Small applications	100–1,000	10–100
3 = Low-medium applications	1000–2,500	100–250
4 = Medium applications	2500–10,000	250–1,000
5 = Large systems	10,000–25,000	1,000–2,500
6 = Very large systems	25,000–100,000	2,500–10,000
7 = Super large systems	>100,000	>10,000

This size range taxonomy imperfect, but serves as a rough starting point for carrying out meaningful conversations about the work effort that will be required to construct an application or a system.

Unfortunately, there is the possibility for substantial discrepancies between the two columns. For example, if a project is using a very low-level language such as Assembly language, it might easily be in a different size category depending upon whether functionality or code volumes were used to determine size.

As an example, a typical basic Assembly language program averages about 320 statements to encode 1 function point. Thus an Assembly language application of 500 function points (ranking as a "medium" application using function points) would require some 160,000 source code statements (hence ranking as a "large system" in terms of code quantity).

On the other hand, for a more modern higher-level language such as Ada, the average ratio of source code statements of function points is 71 Ada statements to encode 1 function point. Thus, the same 500 function point application (in the "medium" size range) would take only 35,500 Ada statements and hence would also be judged "medium" using either metric.

To actually estimate the costs and schedules for any given project, some 25 different size quantities must be known:

1. The functionality of the application (in function points)
2. The volume of information in databases or data warehouses (in data points)
3. The complexity of the problems facing the development team

4. The complexity of the source code produced by the development team
5. The quantity of new source code to be produced
6. The number of programming languages utilized
7. The quantity of changed source code, if any
8. The quantity of deleted source code, if any
9. The quantity of base code in any existing application being updated
10. The quantity of borrowed code from other applications
11. The quantity of reusable code from certified sources
12. The number of paper deliverables (plans, specifications, documents, etc.)
13. The sizes of paper deliverables (pages, words)
14. The number of national languages (English, French, Japanese, etc.)
15. The number of online screens
16. The number of graphs and illustrations
17. The number of test cases that must be produced
18. The size of nonstandard deliverables (i.e., music, animation)
19. The number of bugs or errors in requirements
20. The number of bugs or errors in specifications
21. The number of bugs or errors in source code
22. The number of bugs or errors in user manuals
23. The number of secondary "bad fix" bugs or errors
24. The defect removal efficiency of various inspections and tests
25. The severity levels of software defects

Why such a complex five-layer taxonomy is necessary can be demonstrated by a thought experiment comparing the productivity rates of two dissimilar applications. Suppose the two applications have the following aspects:

Parameter	Application A	Application B
Nature	1 = New	1 = New
Scope	4 = Disposable prototype	12 = Major corporate system
Class	1 = Personal program	18 = Military contract
Type	1 = Non procedural	15 = Trusted system
Size	1 = Very small	6 = Very large system
Probable cost	<$10,000	>$10,000,000

The productivity rate of Application A can very well exceed the productivity rate of Application B by more than two orders of magnitude. The absolute amount of effort devoted to Project B can exceed the effort devoted to Project A by more than 1,000 to 1. Does that mean the technologies, tools, or skills on Project A are superior to those used on Project B? It does not—it simply means two very different kinds of software project are being compared and great caution must be used to keep from drawing incorrect conclusions.

Application A would probably be developed in a few days by one person with a productivity rate approaching 100 function points per staff month. By contrast, Application B would probably require more than six calendar years of development with a team of perhaps 1,000 people and achieve a productivity rate of no more than about 2.5 function points per staff month.

In particular, software tool and methodology vendors should exercise more care when developing their marketing claims, many of which appear to be derived exclusively from comparisons of unlike projects in terms of the nature, scope, class, type, and size parameters.

The software taxonomy for software illustrated here is actually sufficient to allow fairly accurate size and cost estimates for software projects by using nothing more than "pattern-matching" based on the taxonomy itself. By placing future projects within the taxonomy, their approximate costs, schedules, and other attributes can be predicted with acceptable accuracy and very little effort. This method can be used at least six months earlier than other forms of sizing and estimating. It can also be used for legacy applications, commercial software, and even classified applications where the specifications are not available to the general public.

Ambiguity in Defining and Measuring the Activities and Tasks of Software Projects

Once the barrier of attempting to describe the basic nature of the software project has been passed, another major barrier is encountered. This is the serious ambiguity in measuring the tasks and activities that will be performed during software development.

When a software project is measured for productivity purposes, or when productivity data is being published, exactly what activities should be included? It is surprising that after 50 years this fundamental question has not yet been answered unambiguously. The observed variations in this basic topic are enormous. Some of the patterns of activities used as the basis for published productivity results include

- Measuring only coding
- Measuring design, code, and unit testing

- Measuring requirements, design, code, and all forms of testing
- Measuring requirements, design, code, testing, documentation, and management

For a project of 10,000 statements in the C language, the ranges in apparent productivity rates using the metric "lines of source code per person month" would be approximately the following: Method A = 3000, Method B = 1500, Method C = 1000, and Method D = 500 (based on observations of typical systems software projects such as PBX telephone systems and embedded applications).

Using the metric "function point per staff month" and assuming 125 C statements per function point, the range of results for the four examples would be Method A = 24, Method B = 12, Method C = 8, and Method D = 4.

The difference in overall results spans a range of 6 to 1. Recall that all four examples are for exactly the same project. The only things that differ are the activities included in the measurements. From time to time, the author has been tasked to investigate claims of very high productivity (>25 function points per staff month). It often happens that these claims are associated with using only Method A or measuring only coding, but then comparing the results against other projects where all the development activities were included in the data.

Two common software practices must be pointed out as being both inaccurate and unscientific: aggregation of data to the level of *development phases* or to the level of *complete projects*.

The concept of a software phase structure is an arbitrary simplification that usually makes it impossible to replicate findings or perform serious economic studies. A typical phase structure for software projects will identify from four to eight phases. A basic four-activity phase structure might consist of

1. Requirements
2. Design
3. Development
4. Testing

These are the problems with phase-structure data: (1) There is no way of knowing what went on within a phase; (2) There is no way of dealing with activities such as user documentation that typically commence in one phase and end in another; (3) There is no way of dealing with activities that span all phases, such as project management.

If project data is only recorded to the level of phases, consider how ambiguous something basic like the "testing phase" can be. The testing

phase of software projects at the low end can consist of a single unit test by an individual programmer. At the high end, the "testing phase" for a software project might contain a full series of eight discrete testing stages with seven of them carried out by testing specialists rather than by developers, i.e., unit testing, function testing, regression testing, stress testing, integration testing, system testing, independent testing, and field testing.

Using an entire project as the unit of measurement is highly inaccurate and unscientific, since there is no way at all of exploring the inner structure of the work that took place. For example, a small project might consist only of coding and some rudimentary user instructions. Yet a major project might consist of several hundred formal activities and many thousands of tasks.

Only if the activities or tasks of two projects are similar, or if elaborate safeguards exist, can project-level data be directly compared. The author's approach for performing comparisons between projects where the activities and tasks are not identical is to ask users to determine how they want the comparisons performed.

- Do you want the comparison to be based only on activities that are common to both projects?
- Do you want the comparison to be based on the activities of one or the other of the activity sets?
- Do you want the comparison to be carried out regardless of the similarity of the activity sets?

Unfortunately, much of the software technical literature does not define the set of activities on which productivity rates are based. The refereed scholarly journals such as the *IEEE Transactions on Software Engineering* are particularly careless in this respect. It is almost embarrassing to read the software measurement articles in this journal, since it is seldom possible to replicate the studies and assertions of the authors. The ability to replicate findings is a basic canon of scientific publication, and this ability is essentially missing from the software engineering literature.

The hard science journals that deal with physics, chemistry, medicine, or any mature science all contain a striking feature that has no parallel in the software engineering journals. From a third to half of the text in standard scientific articles is devoted to discussions of the experimental methods, measurements, and procedures for collecting the data used and to the methods for analyzing this data. From this information, other scientists and researchers can replicate the experiments and either confirm or challenge the conclusions.

In a survey of the *IEEE Transactions on Software Engineering, IEEE Software, Communications of the ACM, the IBM Systems Journal,* and other software engineering journals the equivalent discussion of measurement methods and data collection averages less than 5 percent of article text and is entirely absent in about a third of refereed articles. This is an embarrassing situation.

Readers might wish to replicate this assertion by reviewing the contents of several issues of the *IEEE Transactions on Software Engineering* or the *Communications of the ACM* and noting the volume of information on measurement methods and experimental practices. Review a sample of articles published by the IEEE or the ACM and consider these points:

- Are the nature, scope, class, type, and sizes of projects defined?
- If LOC or KLOC metrics are used, does the author define the counting rules used?
- If productivity is discussed, does the author define the activities that were measured?
- If resource tracking data is used, did the author check for resource leakage?
- Could you repeat the studies using the author's published description?

The author was formerly an editor of research papers and monographs in physics, chemistry, and medicine as well as in software. Unfortunately, most software managers and software engineers, including referees and journal editors, appear to have had no training at all in the basic principles of scientific writing and the design of experiments. The undergraduate curricula of physicists, physicians, and chemists often include at least a short course on scientific publication, but this seems to be omitted from software engineering curricula.

False Advertising and Fraudulent Productivity Claims

The popular press and almost all of vendor advertisements for products such as CASE tools are even worse. One would have to step back almost 100 years in some other discipline such as medicine to find the kinds of exaggerated claims and unsupported assertions that show up in software tool advertisements today.

A short survey of CASE advertisements in journals such as *Software Magazine, CASE Trends, CASE Outlook,* and *Datamation* found that none of the productivity assertions by tool vendors even identified which activities were used in producing the published results. More than a

dozen advertisements made assertions for productivity improvement in the range of "10 to 1," and "20 to 1" if the tools in question were used. The outliers were advertisements that featured "50 to 100 to 1" improvement and one extreme case that cited productivity gains of "1000 to 1" quoting (without any substantiating data) a nominal "world's most famous productivity expert."

One of the major causes for this kind of misleading advertising is improper measurement. For example, CASE Vendor A measures only code production using their tools and then compares the effort devoted to coding against the effort devoted to complete projects (i.e., requirements, design, coding, testing, user documentation, installation, management, etc.) This kind of "grapes to watermelon" comparison is unprofessional, unethical, and ultimately embarrassing to vendors as well as to the industry. Even worse, some CASE, tool, and language vendors make productivity claims without any data whatsoever.

Software does not presently have a canon of ethics, and it is not a controlled industry such as pharmaceuticals where advertisements must be based on controlled studies and make provable claims. Unfortunately, the unsupported and exaggerated claims published today accomplish little except to bring embarrassment to the phrase "software engineering."

To minimize this form of ambiguity, the author and his colleagues at Software Productivity Research utilize a standard 25-activity checklist for measurement purposes. Project managers and technical personnel are asked to identify all activities that were performed and included in the project. Table 7-1 illustrates the SPR checklist.

The SPR chart of accounts table of activities has been in use since 1985 and has resulted in a number of productivity findings. For example, projects of less than 10,000 source code statements will typically utilize an average of only six activities, whereas projects larger than 100,000 source code statements will utilize an average of 17 activities. Information systems utilize an average of 12 activities, whereas systems software will utilize an average of 20 activities, and U.S. military software projects may utilize all 25 activities.

The SPR 25-activity chart of accounts is only the *minimum* level of granularity needed for accurate software measurement. It is desirable to expand this chart of accounts from activities down to the levels of tasks and subtasks. A full Work Breakdown Structure (WBS) for a large system may include several thousand tasks and subtasks.

Variations in the activities performed on software projects appear to be one of the chief causes for the enormous range of variation in published software productivity results. The gap between the activities actually performed and the activities normally measured is a contributing factor to the next sources of ambiguity to be discussed.

TABLE 7-1 Checklist of Common Software Activities

1.	Requirements
2.	Prototyping
3.	Architecture
4.	Planning
5.	Initial design
6.	Detail design
7.	Design reviews
8.	Coding
9.	Reusable code acquisition
10.	Package acquisition
11.	Code inspections
12.	Independent verification and validation
13.	Configuration control
14.	Integration
15.	User documentation
16.	Unit test
17.	Function test
18.	Integration test
19.	System test
20.	Field test
21.	Acceptance test
22.	Independent test
23.	Quality assurance
24.	Installation
25.	Project management

The Absence of Project Demographic and Occupation Group Measurement

As the overall software industry has grown, we have developed an ever-increasing number of technical specialists. Unfortunately, software measurement practices have not advanced to the level of even recording the numbers and kinds of specialists that participate in large software projects. This means that there is very little empirical data available on an important topic: How many specialists of various kinds will a software organization of 1,000 total staff employ?

Some of the specialists who are noted during SPR assessments include application programmers, systems programmers, programmer/analysts, systems analysts, testing specialists, maintenance specialists, human factors experts, cognitive psychologists, performance specialists, standards specialists, configuration control specialists, integration specialists, security specialists, quality assurance specialists, cost estimators and cost analysts, auditors, measurement specialists, function point

counting specialists, technical writers, course developers, editors, marketing specialists, network and communication specialists, reusability specialists, and assessment specialists.

For a large software organization with 1,000 total staff members or more, specialists can occupy perhaps up to half of all positions. This means that to improve overall organizational productivity or effectiveness, it is necessary to make reasonable assumptions on the quantity and kinds of specialists required. Not only is this kind of information often omitted from project measurements, but several major companies such as AT&T and ITT tend to use blanket job titles such as "member of the technical staff" for all specialists.

Ambiguity in the Span of Control and Organizational Measurements

The number of personnel reporting to a manager is termed the "span of control" and has been a topic of industrial research for more than 100 years. The span of control variances observed during SPR assessments has run from a low of 1 (a manager had only one employee) to a high of 30. Government and military projects often have smaller spans of control (averaging 5 or 6) than civilian projects (which average 7 to 10). This basic parameter should be a standard item of software measurement reports, but unfortunately is often omitted entirely.

About a third of large software projects tend to use an organization style termed "matrix management." In a matrix organization, individual employees typically report to a *career manager* for appraisal and compensation purposes and are assigned to one or more *project managers* for day-to-day work purposes. A more traditional organization structure is termed "hierarchical" and means a vertical project organization where employees report in a single chain of command, more or less along the lines of military command hierarchies. Here, too, the organization structures used on software projects should be standard items on software measurement reports, but unfortunately this information is often completely omitted.

The matrix versus hierarchical organization question is a good example of why this information should be recorded. Projects organized in a matrix fashion tend to have a much higher probability of being canceled or running out of control than hierarchical projects. The point, however, is that there is no way of determining which method is more effective unless measurement practices include explicit measurements of organization structures utilized. SPR's assessment and measurement approaches call for recording organizational data, so that the impact of this phenomenon can be evaluated.

Since 1997 many software projects have adopted the Agile methods. The Agile methods include some new kinds of occupations such as "Scrum master" and also permanent user representatives who are part

of the development teams. Here, too, there is ambiguity in terms of the numbers of software engineers that one Scrum master can support or the number of user representatives for any given software team.

The Missing Link of Measurement: When Do Projects Start?

When projects are delivered is usually fairly easy to determine. However, when projects *start* is the single most ambiguous point in the entire lifecycle. For many projects, there can be weeks or even months of informal discussions and preliminary requirements gathering before it is decided that the application looks feasible. (If the application does not look feasible and no project results, substantial resources might still have been expended that it would be interesting to know about.) Even when the decision to go forward with the project occurs, that does not automatically imply that the decision was reached on a particular date that can be used to mark the commencement of billable or formal work.

So far as can be determined there are no standards or even any reasonable guidelines for determining the exact starting points of software projects. The methodology used by SPR to determine project starting dates is admittedly crude: We simply ask the senior project manager for his or her opinion as to when the project began and utilize that point unless there is a better source.

Sometimes a formal request for proposal (RFP) exists and also the responses to the request. For contract projects, the date of the signing of the contract may serve as the starting point. However, for the great majority of systems and internal MIS applications, the exact starting point is clouded in uncertainty and ambiguity.

It would be highly desirable for IFPUG, the IEEE, ISO, or some other standards group to develop at least some basic guidelines for the industry to assist in determining this most mysterious of all milestones.

Ambiguity in Measuring Milestones, Schedules, Overlap, and Schedule Slippage

Since software project schedules are among the most critical of all software project factors, one might think that methods for measuring schedules would be fully matured after some 60 years of trial and error. There is certainly no shortage of project scheduling tools, many of which can record historical data as well as plan unfinished projects. (As of 2008 no less than 30 project scheduling/planning tools are on the commercial market in the United States.)

However, the measuring original schedules, slippages to those schedules, milestone completions, missed milestones, and the overlaps among partially concurrent activities is still a difficult and ambiguous undertaking.

One of the fundamental problems is the tendency of software projects to keep only the current values of schedule information, rather than a continuous record of events. For example, suppose the design of a project was scheduled to begin in January and end in June. In May, it becomes apparent that June is unlikely, so July becomes the new target. In June, new requirements are levied, so the design stretches to August when it is nominally complete. Unfortunately, the original date (June, in this example) and the intermediate date (July, in this example) are often lost. Each time the plan of record is updated, the new date replaces the former date, which then disappears from view.

It would be very useful and highly desirable to keep track of each change to a schedule, why the schedule changed, and what were the prior schedules for completing the same event or milestone.

Another ambiguity of software measurement is the lack of any universal agreement as to what constitutes the major milestones of software projects. A large minority of projects tend to use "completion of requirements," "completion of design," "completion of coding," and "completion of testing" as major milestones. However, there are many other activities on the critical path for completing software projects, and there may not be any formal milestones for these: examples include "completion of user documentation" and "completion of patent search."

Readers of this chapter who work for the Department of Defense or for a defense contractor will note that the "earned value" approach is only cited in passing. There are several reasons for this. First, none of the lawsuits where the author was an expert witness involved defense projects so the earned-value method was not utilized. Second, although the earned-value method is common in the defense community, its usage among civilian projects including outsourced projects is very rare. Third, empirical data on the effectiveness of the earned-value approach is sparse. A number of defense projects that used earned-value methods have run late and been over budget. There are features of the earned-value method that would seem to improve both project estimating and project tracking, but empirical results are sparse.

Once a software project is underway, there are no fixed and reliable guidelines for judging its rate of progress. The civilian software industry has long utilized ad hoc milestones such as completion of design or completion of coding. However, these milestones are notoriously unreliable.

Tracking software projects requires dealing with two separate issues: (1) Achieving specific and tangible milestones; (2) Expending resources and funds within specific budgeted amounts. For an industry now more than 60 years of age, it is somewhat surprising that there is no general or universal set of project milestones for indicating tangible progress. From SPR's assessment and baseline studies, Table 7-2 lists some representative milestones that have shown practical value.

TABLE 7-2 Representative Tracking Milestones for Large Software Projects

1.	Requirements document completed
2.	Requirements document review completed
3.	Initial cost estimate completed
4.	Initial cost estimate review completed
5.	Development plan completed
6.	Development plan review completed
7.	Cost tracking system initialized
8.	Defect tracking system initialized
9.	Prototype completed
10.	Prototype review completed
11.	Complexity analysis of base system (for enhancement projects)
12.	Code restructuring of base system (for enhancement projects)
13.	Functional specification completed
14.	Functional specification review completed
15.	Data specification completed
16.	Data specification review completed
17.	Logic specification completed
18.	Logic specification review completed
19.	Quality control plan completed
20.	Quality control plan review completed
21.	Change control plan completed
22.	Change control plan review completed
23.	User information plan completed
24.	User information plan review completed
25.	Code for specific modules completed
26.	Code inspection for specific modules completed
27.	Code for specific modules unit tested
28.	Test plan completed
29.	Test plan review completed
30.	Test cases for specific testing stage completed
31.	Test case inspection for specific testing stage completed
32.	Test stage completed
33.	Test stage review completed
34.	Integration for specific build completed
35.	Integration review for specific build completed
36.	User information completed
37.	User information review completed
38.	Quality assurance sign-off completed
39.	Delivery to beta test clients completed
40.	Delivery to clients completed

Note that these milestones assume an explicit and formal review connected with the construction of every major software deliverable. Formal reviews and inspections have the highest defect removal efficiency levels of any known kind of quality control activity and are characteristic of "best in class" organizations. Table 7-2 shows suggested milestones.

The most important aspect of Table 7-2 is that every milestone is based on completing a review, inspection, or test. Just finishing up a document or writing code should not be considered a milestone unless the deliverables have been reviewed, inspected, or tested.

In the litigation where the author worked as an expert witness, these criteria were not met. Milestones were very informal and consisted primarily of calendar dates, without any validation of the materials themselves.

Also, the format and structure of the milestone reports were inadequate. At the top of every milestone report, problems and issues or "red flag" items should be highlighted and discussed first.

During depositions and review of court documents, it was noted that software engineering personnel and many managers were aware of the problems that later triggered the delays, cost overruns, quality problems, and litigation.

At the lowest levels, these problems were often included in weekly status reports or discussed at team meetings. But for the higher-level milestone and tracking reports that reached clients and executives, the hazardous issues were either omitted or glossed over.

A suggested format for monthly progress tracking reports delivered to clients and higher management would include these sections.

Suggested Format for Monthly Status Reports for Software Projects

1. Status of last month's "red flag" problems
2. New "red flag" problems noted this month
3. Change requests processed this month versus change requests predicted
4. Change requests predicted for next month
5. Size in function points for this month's change requests
6. Size in function points predicted for next month's change requests
7. Schedule impacts of this month's change requests
8. Cost impacts of this month's change requests
9. Quality impacts of this month's change requests
10. Defects found this month versus defects predicted

11. Defects predicted for next month
12. Costs expended this month versus costs predicted
13. Costs predicted for next month
14. Deliverables completed this month versus deliverables predicted
15. Deliverables predicted for next month

Although the suggested format somewhat resembles the items calculated using the earned value method, this format deals explicitly with the impact of change requests and also uses function point metrics for expressing costs and quality data.

An interesting question is the frequency with which milestone progress should be reported. The most common reporting frequency is monthly, although exception reports can be filed at any time it is suspected that something has occurred that can cause perturbations. For example, serious illness of key project personnel or resignation of key personnel might very well affect project milestone completions and this kind of situation cannot be anticipated.

The simultaneous deployment of software sizing tools, estimating tools, planning tools, and methodology management tools can provide fairly unambiguous points in the development cycle that allow progress to be judged more or less effectively. For example, software sizing technology can now predict the sizes of both specifications and the volume of source code needed. Defect estimating tools can predict the numbers of bugs or errors that might be encountered and discovered. Although such milestones are not perfect, they are better than the former approaches.

Project management is responsible for establishing milestones, monitoring their completion, and reporting truthfully on whether the milestones were successfully completed or encountered problems. When serious problems are encountered, it is necessary to correct the problems before reporting that the milestone has been completed.

Failing or delayed projects usually lack serious milestone tracking. Activities are often reported as finished while work is still on going. Milestones on failing projects are usually dates on a calendar rather than completion and review of actual deliverables.

Delivering documents or code segments that are incomplete, contain errors, and cannot support downstream development work is not the way milestones are used by industry leaders.

Another aspect of milestone tracking among industry leaders is what happens when problems are reported or delays occur. The reaction is strong and immediate: Corrective actions are planned, task forces assigned, and correction begins to occur. Among laggards, on the other hand, problem reports may be ignored and very seldom do corrective actions occur.

In more than a dozen legal cases involving projects that failed or were never able to operate successfully, project tracking was inadequate in every case. Problems were either ignored or brushed aside, rather than being addressed and solved.

Because milestone tracking occurs throughout software development, it is the last line of defense against project failures and delays. Milestones should be established formally and should be based on reviews, inspections, and tests of deliverables. Milestones should not be the dates the deliverables more or less were finished.

Problems with Overlapping Activities

It is widely known that those of us in software normally commence the next activity in a sequence long before the current activity is truly resolved and completed. Thus design usually starts before requirements are firm, coding starts before design is complete, and testing starts long before coding is complete. The classic "waterfall model," which assumes that activities flow from one to another in sequential order, is actually a fiction that has almost never occurred in real life.

When an activity such as design starts before a predecessor activity such as requirements are finished, SPR uses the term and metric called "overlap" to capture this phenomenon. For example, if the requirements for a project took four months, but design started at the end of month three, then we would say that design overlapped requirements by 25 percent. As may be seen, overlap is defined as the amount of calendar time still remaining in an unfinished activity when a nominal successor activity begins.

The amount of overlap or concurrency among related activities should be a standard measurement practice, but in fact is often omitted or ignored. This is a truly critical omission because there is no way to use historical data for accurate schedule prediction of future projects if this information is missing. (The average overlap on projects assessed by SPR is about 25 percent, but the range goes up to 100 percent.)

Simplistic schedule measurements along the lines of, "The project started in August of 2005 and was finished in May of 2007" are essentially worthless for serious productivity studies. Accurate and effective schedule measurement includes the schedules of specific activities and tasks and the network of concurrent and overlapping activities.

Leakage from Software Project Resource Tracking Data

Companies that build and market software estimating tools are invariably asked a fundamental question: "How accurate is your estimating tool when compared against actual historical data?" A much more

fundamental question is essentially never asked, but should be: "How accurate is the historical data itself?"

Unfortunately, normal software measurement practices tend to "leak" and seldom record or collect more than about 80 percent of the actual effort applied to software projects. Indeed, the average accuracy of the historical resource data on projects validated by the author and his colleagues at Software Productivity Research is less than 60 percent. That is, some 40 percent of the total effort devoted to software projects is unrecorded via standard resource tracking systems. Sometimes more than 50 percent of the actual effort is not recorded.

The major sources of resource data leakage are the following:

- Failure to record all activities that were performed
- Failure to record voluntary, unpaid overtime
- Failure to record user effort
- Failure to record managerial effort
- Charging time to the wrong project codes

From analyzing productivity data at the activity level, it was discovered that a number of activities are often not tracked. For example, many resource tracking systems are not even initialized or turned on until after the requirements are complete. Business functions such as planning are seldom tracked. The entire set of information engineering tasks for building corporate data models are seldom tracked. Specialist groups such as quality assurance, technical writing, database administration, and configuration control may not record their time for every project they support. More than 20 percent of a project's total effort may be part of these unrecorded activities, when the missing data is reconstructed via interviews. Recently it has been noted that some of the permanent users assigned to Agile projects may not charge their time to the project's budget.

Unpaid overtime by software technical staff is a common phenomenon and appears to average in the vicinity of 12 percent on large systems. The unpaid overtime is not equally distributed over the development phases and is particularly heavy during late coding and testing. Since this phenomenon is seldom tracked, it is necessary to carry out interviews with project personnel and have them reconstruct their overtime patterns from memory and personal data.

Users often perform technical work on MIS software projects, such as participating in JAD sessions and design reviews, assisting with prototypes, producing their own user documentation, carrying out acceptance testing, and the like. User effort on MIS projects averages about 19 percent of the total effort applied. However, user effort is almost

never recorded via standard project time recording. It is necessary to carry out interviews with users and have them reconstruct their activities from memory. This is true for some of the new Agile projects as well as for traditional projects.

Management effort on software projects averages between 12 percent and 15 percent of the total effort applied, but this effort is seldom recorded. Here, too, interviews with management personnel are necessary to validate gaps in standard measurement practices.

The most disturbing errors in cost tracking systems are those where time is charged to the wrong project. Sometimes this occurs by accident, but more often than not the situation is because one project is running low on funds, whereas another still has a surplus. Obviously any kind of cost analysis or economic study that includes data with incorrect charges will be essentially worthless.

MIS projects are the least accurate in collecting resource data and often omit more than 60 percent of the total project effort. Systems software is somewhat better when developed by large corporations and computer companies that run about 70 percent in terms of accuracy. However, for many small systems software companies, accuracy is essentially zero, since they have no recording mechanisms at all. Only in the case of military software does tracking accuracy approach 90 percent on the average and even here unpaid overtime is sometimes omitted.

Ambiguity in Standard Time Metrics

Software work time can be recorded in units of minutes, hours, days, weeks, months, and years (or not at all). The most accurate of the widely used methods is hours, and the higher-level units grow progressively more ambiguous. The most serious problems occur when comparing productivity between projects or enterprises that use differing base metrics for recording time. For example, if company A records time in hourly units based on time actually recorded via worksheets, while company B records time in monthly increments and does not use task or activity-level timesheets, there can easily be more than a 100 percent differential in the apparent productivity of the two companies, which may not actually exist.

In order for time recording to be accurate enough for cost and economic studies, it is necessary to record a number of basic assumptions:

- The number of hours in the company's working day
- The number of *productive hours* during a working day (i.e., about five to six hours out of eight in the United States)
- The number of *paid overtime hours* during the week and on weekends

- The number of *unpaid overtime hours* for exempt employees and managers
- The number of *slack hours* between assignments or waiting for work
- The impact of *split assignments* when working concurrently on several projects
- The number of days in the company's working year
- The company's annual number of holidays and planned closings
- The company's vacation day policies (i.e., 10, 15, or 20 days per year)
- The company's experience on annual sick days
- The company's experience with unplanned events (snow storms, hurricanes, etc.)
- How non-project days (i.e., meetings, classes, interviews, and travel) are dealt with

Hourly recording is fairly common for contract software and for all small projects where only one or two technical workers are engaged. As the size of the applications grow, time recording tends to switch from hours to coarser units, with days and months being the two most common units.

From interviews with project personnel carried out as part of SPR assessments, it appears that the "noise" or incorrect information that is typically contained in software tracking systems is large enough so that productivity studies based on uncorrected tracking data are essentially worthless. Here are two examples of such noise:

- The programming staff of a software contract house were directed to charge no more than 40 hours per week to a specific project since its budget was nearly exhausted but were, in fact, working at an average of almost 56 hours a week.
- A programmer employed by a major computer manufacturer reported that he had transferred from one department to another earlier in the year. However, his time continued to be charged to the former department for a full three months after the transfer. When the error was finally noted, it was considered "too insignificant to bother updating the records." This meant, of course, that the project he actually worked on for three months would be under-reported, whereas the former project would show effort that was not actually applied. Problems such as these are endemic to the software industry.

When companies measure time in terms of days, they often fail to establish ground rules for the number of productive hours in a standard day.

In the United States, the standard accounting day is eight hours, but productive work normally occupies only five to six such hours. However, about an hour of unpaid overtime each day is also common in the United States, and this too must be considered.

When companies measure time in terms of weeks, they often fail to explicitly state the number of hours in a standard week. Here the ambiguity spans both company and national boundaries. The normal accounting workweek in the United States is 40 hours, but in Canada it is 37.5 and for some enterprises only 32. Unpaid overtime is more common in the United States than in Canada. In Japan, the standard accounting workweek is 44 hours and unpaid overtime averages more than 10 hours per week. As may be seen, unless the ground rules are recorded, the range of uncertainty can be very high. Obviously, a direct comparison between Canadian and Japanese productivity rates that used "weeks" would favor Japan.

When companies measure time in terms of months, the ambiguity is enormous. Vacations and holiday periods vary significantly from country to country. For example, the standard vacation period in Mexico is only 6 days, the U.S. standard is 10 days, Germany is 18 days, Australia is 20 days, England is 22 days, Sweden is 27 days, and Austria is 30 days.

The ambiguity in time recording may not be serious for a single project or even for a single company. However, when carrying out large-scale multi-company and international studies the ambiguities can be devastating. The most accurate of the widely used methods for time recording is to measure resources in terms of work hours and to also measure schedules in terms of work hours. Hourly data can be converted into daily, weekly, monthly, or annual rates by means of conversion formulas that include locally validated assumptions on productive time and unpaid overtime.

There is a tendency among software managers and software engineers to disregard accurate time reporting as being "unprofessional." However, other professionals such as attorneys and physicians are normally much more precise in recording time than software personnel. Attorneys, for example, often record time in 15 minute increments and can provide complete historical data on time and effort for every client.

Unfortunately, for software, there are no standard conventions for recording productive time and unpaid overtime. Most time tracking systems in the United States do not allow entry of such phenomena as "slack time" or "unpaid overtime." Such omissions create errors in resource tracking that, by themselves, sometimes exceed 25 percent of the total effort applied to software projects.

Finally, the gaps and leakage of resource tracking and time recording are perhaps the least studied factors of software project measurement and estimation. The number of articles and books that even discuss accurate resource tracking and time recording for software projects

comprise less than 1 percent of the total measurement and metrics literature. So far as can be determined, in the last ten years neither the *Communications of the ACM* nor the *IEEE Transactions on Software Engineering* have published even one article that explored or discussed resource tracking errors. It is also striking that the accuracy of resource tracking data is not even included in SEI assessments.

Inadequate Undergraduate and Graduate Training in Software Measurement and Metrics

Can an electrical engineering student matriculate through a normal curriculum without learning how to use an oscilloscope and volt-ohm meter, or without learning the derivation of amps, ohms, joules, volts, henrys, and other standard metrics? Certainly not. Could a physician successfully pass through medical school without understanding the measurements associated with blood pressure, triglyceride levels, or even chromatographic analysis? Indeed not. Could a software engineer pass through a normal software engineering curriculum without learning about functional metrics, complexity metrics, or quality metrics? Absolutely. (While this chapter was being prepared, the author was speaking at a conference in San Jose, California. After the session, a member of the audience approached and said, "I'm a graduate software engineering student at Stanford University, and I've never heard of function points. Can you tell me what they are?")

Unfortunately, most universities are far behind the state of the art in teaching software measurement and metrics. The measurement curricula of even well-known schools is quite embarrassing: Some of nominal U.S. leaders in software engineering are more than ten years out of date in measurement and metrics subject matter; for example, Carnegie-Mellon, Harvard, UCLA, the University of Maryland, the University of Michigan, the University of North Carolina, and Stanford University are all prestigious, but are all painfully behind the state of the art in measurement topics, although some progress has occurred over the past few years. For example the University of Maryland is the source of the well-known "goal-question" metrics developed by Dr. Victor Basili.

There is a simple test for revealing inadequate metrics training at the university level. Ask a recent software engineering graduate from any U.S. university this question: "Did you learn that the 'lines of source code' metric tends to work backward when used with high-level languages?" Try this question on 100 graduates and you will probably receive about 98 "No" answers. It is embarrassing that graduate software engineers can enter the U.S. workforce without even knowing that the most widely utilized metric in software's history is seriously flawed.

Universities are ordinarily about ten years behind the state of the art, because it takes approximately that long to turn advances in technology into undergraduate courses and to produce textbooks. The first college textbook on function points, Dr. Brian Dreger's *Function Point Analysis*, was published in 1989—exactly ten years after function points were initially put into the public domain. However, as of 2008, some 30 years after the publication of function points, more than 85 percent of U.S. colleges and universities do not include courses in function point analysis. Yet IFPUG, the nonprofit function point users group, ranks as the largest professional association of software measurement personnel in the United States!

Inadequate Standards for Software Measurement

Unfortunately, formal standards organizations such as the Department of Defense (DoD), IEEE, The International Standards Organization (ISO), and the National Bureau of Standards are many years behind the state of the art in dealing with software measurements and metrics. This statement is also true of quasi-standards organizations such as the Software Engineering Institute (SEI) and the European PRISM group. Even the International Function Point Users Group (IFPUG) has gaps in a number of domains, although it is more modern than the classical software engineering associations.

For example, as of 2008 it has been observed that the ISO 9000-9004 quality standards add to software costs, but do not improve quality in any detectable fashion. The ISO functional size standards are vague enough so that four competing function point metrics are all certified, but their results can differ by more than 35 percent when sizing the same applications.

As of 2008, every single known standard on software productivity, including ISO, IEEE, all current DoD standards on software, the European PRISM reports, and the current SEI measurement reports on software productivity measurement etc., fails to provide adequate cautions that "LOC and KLOC" are decoupled from standard economic assumptions and behave paradoxically. This is a serious enough omission to be viewed as constituting professional malpractice.

In the United States, the DoD and SEI are the most regressive and most heavily locked in to the obsolete LOC metric. Both organizations have never carried out formal analyses of LOC using different programming languages to note the reversals of economic results.

Of all of the major software measurement organizations involved with standards, the International Function Point User's Group (IFPUG) is the most promising, since this group is not tied down to an unworkable metric.

However, IFPUG will need to broaden its guidelines from the mechanics of function point counting to deal with the other critical issues. For example, IFPUG counting practices need expanded discussion of software work breakdown structures, methods, methods for recording influential factors such as tools and languages utilized, and time recording methods that encompass international projects. IFPUG also needs to extend their work on schedule and overlap recording. Finally, IFPUG needs new working groups or research committees on the application of functional metrics to software quality, business value, and software usage.

A number of corporations have internal measurement standards: AT&T, IBM, Hewlett-Packard, and Siemens provide examples of some. These internal measurement standards range from fairly good to being quite obsolete. However, all internal standards that are based on LOC or KLOC normalization and fail to provide cautions about the hazards of LOC are candidates for remedial modernization.

Let us turn now to the problems of LOC and KLOC metrics, and examine one of the major hazards preventing "software engineering" from becoming a true engineering discipline.

Lack of Standardization of "Lines of Source Code" Metrics

The subjectivity of "lines of source code" can be amusingly illustrated by the following analogy: Ask an obvious question such as, "Is the speed of light the same in the United States, Germany, and Japan?" Obviously the speed of light is the same in every country.

Then ask the following question: "Is a line of source code the same in the United State, Germany, and Japan?" The answer to this question is, "No, it is not"—software articles and research in Germany have tended to use physical lines more often than logical statements, whereas the reverse is true for the United States and Japan.

There have been other metrics that differed from country to country, such as U.S. gallons and Imperial gallons. Also statute miles and nautical miles differ significantly. These differences are common knowledge, whereas the differences in "lines of source code" definitions are comparatively obscure, and sometimes not fully stated by software authors.

The most widely used software metric since the industry began has been lines of source code (LOC). Either this metric or KLOC (where K stands for 1,000) have been used in print in more than 10,000 articles and books since 1946.

Most users of LOC and KLOC regard this metric as being objective, and indeed a number of standard reference books and articles on metrics have cited the objectivity of LOC as a key virtue. However, from discussions with more than a thousand software managers and professionals,

it is unfortunate to report that the LOC metric may be the most subjective metric used in refereed articles in the last 50 years.

When LOC and KLOC originated as software metrics, the only languages in use were machine language and basic assembly language. For basic assembly language, physical lines and logical lines were equal: Each source statement occupied one line on a coding sheet or one tab card. From 1946 until about 1960, LOC and KLOC metrics were reasonably well defined and reasonably objective. The explosion of languages from 1960 onward destroyed the objectivity of LOC and KLOC and their validity for economic studies as well.

As programming languages evolved, the relationship between logical and physical lines broke down. More sophisticated basic assembly languages such as 1401 Autocoder allowed several logical statements per physical line, and the Basic language also allowed a number of logical statements per physical line. Macro assembly languages began to change the fundamental concept of a "line of source code" by introducing pseudo-codes and macro expressions. Other languages, such as COBOL, allowed conditional statements to span several physical lines.

Using the Basic language as an example, the difference between physical lines and logical statements can be as great as 500 percent, and the average difference is about 200 percent, with logical statements outnumbering physical lines. On the other hand, for COBOL, the difference is about 200 percent in the opposite direction, with physical lines outnumbering logical statements.

Other variations also must be considered in the LOC counting arena: counting or not counting commentary lines (about 80 percent of managers do not, but 20 percent do count comments), counting or not counting job control languages, counting procedural statements and/or data definition statements, and so forth.

From surveys of counting practices carried out by the author and his colleagues at Software Productivity Research, the varieties of subjective methods associated with LOC counting create a total range of apparent size of more than one order of magnitude for the software industry as a whole. The largest number of major code counting variations observed within a single company was six, and the range for counting the size of a single control project within that company was approximately 5 to 1. This is far too broad a range to be tolerated for an engineering metric.

Consider the fundamental topic of whether to count physical lines or logical statements. From SPR surveys, about 65 percent of U.S. project managers and published articles use counts of physical lines as the basis for size and productivity data, whereas about 35 percent use logical statement counts. Unfortunately, only about 50 percent of the published articles clearly state which method was used or gave the counting rules used for sizing.

The standard dictionary definition of subjectivity is "Particular to a given individual; personal." Under that definition, it must be concluded that LOC and KLOC are, in fact, subjective metrics and not objective ones.

Code counting subjectivity could be eliminated by establishing standard counting conventions for each major language. Indeed, Software Productivity Research, the IEEE, and the Software Engineering Institute published preliminary draft LOC counting proposals within a year of one another. Unfortunately, the SPR, IEEE, and SEI draft standards differ, so even in the domain of standardization of LOC counting practices, subjectivity is present. Note that for many modern languages such as 4GLs, spreadsheets, query languages, object-oriented languages, and graphics icon-based languages, none of the current draft standards are technically acceptable.

Paradoxical Behavior of "Lines of Source Code" Metrics

The LOC and KLOC metrics have a much deeper and more serious problem than a simple lack of standardization. They are also paradoxical and move backward! Indeed, the tendency of LOC and KLOC to move backward as economic productivity improves is a much more serious problem for software economic studies than the subjectivity of LOC and KLOC. The mathematical problems with LOC are severe enough that they make the phrase "software engineering" seem ridiculous. It is embarrassing for a major industry such as software engineering to continue to use a metric that does not work—and to do so without even realizing what is wrong with it!

Unfortunately, many well-known books on software measurement and economics do not contain even a single statement about this well-known problem. To cite but two examples, both Barry Boehm's *Software Engineering Economics* and Robert Grady and Deborah Caswell's *Software Metrics: Establishing a Company-Wide Program* use LOC and KLOC metrics without any warnings or cautions to the readers of the paradoxical nature of these metrics for high-level languages. Thus both books are flawed as reliable sources of software engineering data; although in other respects, both are valuable and contain useful information.

The ambiguity with LOC and KLOC is caused by the impact of the fixed and inelastic costs of certain activities that are always part of software projects. The problem of measuring productivity in the presence of fixed costs has long been understood for manufacturing economics. However for software, it was initially described by the author in 1978 and fully explained in 1986 in the book *Programming Productivity*.

There is a basic law of manufacturing economics that if a manufacturing process includes a high percentage of fixed costs, and the number of

units produced goes down, the cost per unit will go up. This same law also applies to software, and an illustration can clarify the concept.

Consider a program that was written in basic Assembly language and required 10,000 lines of code. The noncoding tasks for requirements, specifications, and user documentation would take about three months of effort, and the code itself would take about seven months of effort for development and testing. The total project would thus require ten months of effort, and net productivity would be 1,000 LOC per person month.

Now consider the same program written in Ada. Only about 2,000 Ada statements would be required as opposed to the previous 10,000 basic Assembly statements. The Ada coding and testing effort would be one month rather than seven months, for a real savings of six months. However, the noncoding tasks for requirements, specifications, and user documentation would still take three months of effort. Thus, the total project would require only four months of total effort, for a clear productivity improvement of 60 percent in terms real economic productivity (i.e., goods or services produced per unit of labor or expense).

However, when productivity of the total project is measured with LOC, the Ada version of the same project averages only 500 LOC per month for an apparent *reduction* in productivity of 50 percent versus basic assembly language. This is counter to the improvement in real economic productivity and constitutes a dramatic paradox for the software industry.

(Since both versions of this application perform the same functions, the function point total of the Assembly and Ada version would obviously be the same: assume 35 function points. Using the function point metric, the Assembly version has a productivity rate of 3.5 function points per month, whereas the Ada version is 8.75 function points per month. Function points match the increase in real economic productivity, whereas the LOC metric conceals the power of Ada and other high-level languages.)

Using LOC and KLOC metrics for a single language can produce valid results if standard counting rules are applied. However, for cross-language comparisons, or for projects containing multiple languages (such as COBOL and SQL), the results are almost always invalid and paradoxical.

Unfortunately all modern programming languages such as Java, C++, C#, Objective C, SMALLTALK, Visual Basic, etc., are penalized by the use of LOC metrics. Comparisons of any of these modern languages to older low-level languages such as Assembler and COBOL will yield incorrect results that are not valid economically.

Since Ada was such an important aspect of military software, let us take a more detailed look at two versions of the same project using Ada and Assembly language and compare the productivity and levels in a fairly detailed fashion.

Assume that we are concerned with two versions of a military application that provide *exactly* the same functions to end users. The quantity of function points is, therefore, the same for both the Ada and Assembly language versions—assume 15 function points for both versions.

The code volumes, on the other hand, are quite different. The Assembly language version required 5,000 statements, whereas the version in the more powerful Ada language required only 1,000 statements, or only 20 percent of the Assembly language code.

Assume that the burdened salary rate in both the Ada and Assembly versions of the project was $10,000 per staff month. Here are the two versions shown in side-by-side form.

	Assembly Language Version (5,000 source lines, 15 function points)		Ada Language Version (1,000 source lines, 15 function points)	
Activity	Months	$	Months	$
Requirements	1.0	$10,000	1.0	$10,000
Design	2.0	$20,000	1.0	$10,000
Coding	3.0	$30,000	0.75	$7,500
Testing	2.0	$20,000	0.75	$7,500
Documentation	1.0	$10,000	1.0	$10,000
Management	1.0	$10,000	0.5	$5,000
Totals	**10.0**	**$100,000**	**5.0**	**$50,000**
LOC per month	500		200	
$ per LOC	$20		$50	
FP per month	1.5		3.0	
$ per function point	$6,666		$3,333	

In standard economic terms (i.e., goods or services produced per given amount of labor or expense), the Ada version is clearly twice as productive as the Assembly language version since delivering the same application costs half as much and takes half as much effort.

However, are these economic advantages of Ada visible when productivity is measured with "source lines per month" or "cost per source line?" Indeed they are not. The productivity of the Assembly language version of the project is 500 LOC per month. The productivity of the Ada version is only 200 LOC per month.

There are similar reversals when costs are normalized: the cost per LOC for the Assembly language version is $20.00 whereas the cost per LOC for the Ada version has soared to $50.00.

Plainly something is wrong here: The more expensive version (Assembly language) looks more than twice as productive as the Ada version. Yet the Ada version only costs only half as much. What is wrong?

The fundamental problem is that there are fixed costs tied up in activities such as requirements, design, and documentation that are the same in both versions. Thus, when LOC is selected as a manufacturing unit, and there is a switch from a low-level language to a high-level language, then the number of units that must be manufactured is clearly reduced, but the fixed costs are not reduced.

Since both the Assembly language and Ada versions produce the same functions, observe how closely productivity can be when measured with function points. The Ada version nets out at 3.0 function points per month, which is double that of the Assembly language version, which nets out at only 1.5 person-months per staff month.

The cost per function point also favors Ada, too: The Ada cost per function point is $3,333 whereas the Assembly version is $6,666. As can be seen, function points actually correlate with standard manufacturing economics and productivity assumptions, whereas LOC metrics move paradoxically in the wrong direction.

When only coding is measured, the Ada version nets out at a scorching 20 function points per staff month, whereas the Assembly language version slogs along at only 5 function points per staff month: only one-fourth the productivity rate of Ada.

But when LOC metrics are used to measure coding productivity, the Assembly version is 1,666 LOC per month whereas the Ada version is 1,333 LOC per month. But the Assembly version took 3 months as opposed to the 0.75 months for Ada. Here, too, LOC metrics do not reveal economic advantages of more powerful programming languages.

The LOC metric, compared to function points, also distorts quality measurements. The situation with LOC is so paradoxical and absurd in the presence of high-level languages that it is fair to state that the LOC metric has slowed the advance of software engineering as a true engineering discipline. It is time to step up to this problem and declare LOC metrics to be an example of professional malpractice.

Usage of LOC and KLOC Metrics Considered to be Professional Malpractice

So long as LOC and KLOC metrics are used inappropriately and without awareness of their paradoxical nature, software engineering will never be a serious engineering discipline. At this point in our history, a strong stand for eliminating the paradox seems desirable. As far back as 1992, to speed up the process of eliminating LOC and KLOC metrics, the author proposed that starting in 1995, the use of LOC and KLOC under conditions known to be paradoxical should be viewed as professional malpractice. However, still in 2008, thousands of articles are published every year that still use LOC metrics without even mentioning the fact that such metrics violate standard economic assumptions.

Malpractice is a serious situation and implies the usage of an approach known to be harmful under certain conditions, which should have been avoided through normal professional diligence. For example, a medical doctor who prescribed penicillin for a patient known to be allergic to that antibiotic is an illustration of professional malpractice. Using LOC and KLOC metrics to evaluate languages of different levels without cautioning about the paradoxical results that occur is unfortunately also an example of professional malpractice.

What happens, of course, is that when LOC is considered as a manufacturing unit, and there is a migration from a low-level language to a high-level language, the number of "units" obviously declines. When there is a decline in manufactured units in the presence of fixed costs (i.e., paperwork), then the cost per unit must inevitably rise.

The LOC and KLOC metrics grow progressively more ambiguous and counterintuitive as the level of languages goes up or for multi-language studies. Following are situations where LOC and KLOC are ambiguous enough to be harmful to economic understanding and their usage should constitute malpractice.

- LOC and KLOC metrics should be avoided for economic studies involving object-oriented languages, 4GLs, generators, spreadsheets, and graphic-icon-based languages.
- LOC and KLOC metrics should never be used to compare unlike languages, such as COBOL and Java.
- LOC and KLOC metrics should not be used for applications containing multiple languages, such as Java and HTML, C and Assembly, or COBOL and SQL.
- LOC and KLOC metrics should not be used for quality normalization (i.e., defects per KLOC) for studies involving multiple languages.

Consider the similar problem of carrying out international cost surveys that involve multiple currencies such as dollars, yen, pounds, lire, deutschmarks, francs, etc. There are two methods for carrying out accurate international cost surveys: (1) One of the currencies such as the dollar is selected as the base currency, and all other currencies are converted into equivalent amounts; (2) A synthetic base metric such as European Currency Units (ECU) is selected, and all quantities are expressed in those units.

The acceptable methods for dealing with multiple currencies provide a useful model for software studies dealing with multiple languages: (1) One of the languages such as Assembly is selected as the base language, and all other languages are converted into equivalent amounts; (2) A synthetic base metric such as IFPUG function points is selected, and all quantities are expressed in those units. Of these two methods for dealing with multiple languages, method 2 is preferred today.

The Sociology of LOC and KLOC Usage

Changing paradigms or abandoning metrics such as LOC and KLOC is a sociological issue as well as a technical issue. Several books can be recommended as background information on this topic: Kuhn's *The Structure of Scientific Revolutions,* Cohen's *Revolution in Science,* Starr's *The Social Transformation of American Medicine,* and Mason's *A History of the Sciences*.

At conferences and seminars, it is interesting to ask a series of questions and take informal polls of the responses. Since the author speaks at conferences hosted by individual companies, universities, nonprofit groups such as the IEEE, ISPA, and IFPUG, and fee-based conferences by organizations such as DCI, QAI, and Extended Intelligence, the audiences are a full spectrum of military and civilian software professionals who are concerned with all kinds of software from artificial intelligence through ordinary MIS applications. Following are observations derived from such events in 2007.

Question 1: How many measure with LOC, function points, or not at all?
Answers: Not at all = 50%; LOC = 30%; function points = 20%.

Question 2: How many think that LOC is an objective metric?
Answers: LOC is objective = 70%; LOC is not objective = 30%.

Question 3: How many know that LOC has never been standardized?
Answers: Thought it had been = 50%; have local standard = 30%; did know = 20%.

Question 4: How many know that the LOC metric penalizes high-level languages?
Answers: Did not know = 75%; we don't use such languages = 5%; did know = 20%.

Question 5: Why do you use the LOC metric? (asked of LOC users)
Answers: Management insists = 60%; easiest to use = 30%; no specific reason = 10%.

Question 6: Now that you've seen what's wrong with LOC, will you still use this metric?
Answers: Not allowed to change = 50%; will change = 25%; not ready to change = 25%.

The most troubling answers in the whole series are those to questions 5 and 6, where the LOC metric appears to be forced on unwilling users by corporate policies, military standards, or managerial insistence. This is a harmful situation from both a business and a sociological viewpoint. This is not good engineering management, but rather professional malpractice.

It is also of interest to pose the same questions to attendees from specific companies and see what kinds of answers are received within more homogeneous environments than large-scale heterogeneous conferences.

The technical problems and paradoxes of LOC and KLOC metrics are easily demonstrated, and have been described fairly often in books and journal articles. Therefore, it can be assumed that personnel within leading companies will be aware of some of the problems. The question that arises, then, is why does this metric continue to be so widely used.

When project managers and technical personnel were queried from companies considered to be industry leaders, such as AT&T, Hewlett-Packard, and IBM, the most common response at conferences has been: "We know it doesn't work, but we use LOC because our senior managers demand it." When queried as to whether anyone has apprised senior management of the problems of LOC and KLOC, the most common response is, "No, it might make them look bad."

When the senior executives in the same companies are asked about why they want LOC and KLOC data when it is visibly defective for economic and comparative studies across multiple languages, the most common response at the higher levels is, "No one ever told me about the problems."

Interestingly, when a discussion of the problems of LOC and KLOC does reach the top via upward information flow or from external sources, senior management is often quite willing to change and take corrective action.

What often appears to occur in leading companies is a kind of negative feedback loop: Subordinates are reluctant to embarrass their superiors so information on LOC and KLOC problems is not passed upward; the superiors themselves may sense that LOC and KLOC may have flaws, but assume that their subordinates would have cautioned them about major problems.

When the same question about LOC and KLOC usage is posed to attendees from second-tier companies, the most common response is, "We use LOC and KLOC because AT&T, IBM, and Hewlett-Packard use them." The next most common response is an ad hominem argument citing some well-known industry figure or organization, such as "We use LOC metrics because the SEI uses them."

Below the second tier of companies, measurement is such a scarce undertaking that no metrics at all are typically used, so the questions are moot.

When the LOC and KLOC usage question is posed to military software personnel, the most common response is, "We use LOC and KLOC because the DoD demands it."

When academics and university faculty are queried as to why they use LOC and KLOC metrics in the face of substantial proof that the metrics are subjective and paradoxical, the most common answer is

fairly depressing: "LOC and KLOC metrics must be valid, because the software engineering literature that uses these metrics is so large."

There appears to be an incorrect assumption that because several hundred books and several thousand refereed articles have been published using LOC and KLOC data, that this provides a kind of "proof" that the metrics are valid. (Prior to Copernicus, there were many published books and papers that assumed the sun circled the Earth, but that did not make it a valid hypothesis). LOC and KLOC metrics are still flawed and paradoxical, regardless of the number of refereed publications that use these metrics or the prestige of some of the authors.

Not all uses of LOC and KLOC are caused by duress, of course. Other common responses as to why these metrics continue to be utilized include

- "We only care about a single language, and LOC is accurate for single languages."
- "There isn't anything better available."
- "Function points are for MIS and we do embedded software."
- "We use COCOMO, and so we need LOC data to estimate."
- "We don't measure design or documentation and LOC works for coding."

When reviewing the history of scientific paradigm shifts in other disciplines, such as the shift to continental drift and plate tectonics in geology, the shift to quantum mechanics in physics, and the shift to sterile surgical procedures in medicine, it appears that from 10 to 25 years is needed before a major paradigm shift can occur, with the average time being perhaps 14 to 15 years. There is no quick way to change social phenomena, even unfortunate ones.

The Hazards and Problems of Ratios and Percentages

Almost every month, software productivity data is published in a form that resembles this: "Design tool 40 percent of the effort, coding took 20 percent, and testing took 40 percent." In a similar fashion, most papers dealing with maintenance and speakers at conferences on maintenance include the phrase, "Maintenance takes 70 percent of the effort, and development is only 30 percent."

Quality data is also frequently discussed in percentage and ratio form, in a fashion that resembles the following: "Requirements bugs were 15 percent of the total; design bugs were 25 percent of the total; coding bugs were 50 percent of the total; and other bugs made up 10 percent of the total."

This form of representation is unfortunately worthless for serious study. Ratios and percentages are prey to the same kinds of ambiguity as LOC and KLOC metrics. Ratios and percentages vary from language to language. Obviously, the percentage of a project's effort devoted to coding will be *highest* for low-level languages such as Assembly, C, FORTRAN, and the like. The percentage of a project's effort devoted to coding will be *lowest* for more powerful languages such as SMALLTALK, 4GLs, application generators, and the like.

It would be far more useful and effective to abandon percentages and ratios and switch to synthetic metrics that actually contain useful information. For example, if the author of a paper on productivity wishes to allow other researchers to replicate his or her findings, then expressing information in these terms is much more accurate than using percentages: "Functional specifications were produced at a rate of 50 function points per month; coding proceeded at a rate of 20 function points per month; and unit testing proceeded at a rate of 75 function points per month. Project management was calculated at a rate of 150 function points per month. Net productivity for the entire project was 11 function points per month."

With this kind of representation, it is possible to use the information for estimating other projects. With ratios and percentages, estimation is dangerous and unsafe if the project being considered will use a language or languages other than the ones from which the ratios were derived.

Moving now to other problems, the fundamental subjectivity and ambiguity of LOC and KLOC metrics and derivatives such as percentages, are compounded by ambiguity in the next problem to be considered.

Ambiguity in Measuring Development or Delivery Productivity

Another area of basic uncertainty in software measurement is whether productivity rates should be derived from what the programming staff *develops* or what is actually *delivered* to users or both. This uncertainty comes into play in four separate domains: enhancements, deletions, reuse, and packages.

Enhancement Ambiguity

Assume that a software project is set up to enhance an existing application. The existing application might be 100,000 source statements in size in the C language, and the enhancement will add 10,000 new C statements. Assume also that the enhancement requires a total of 20 months of effort for all activities that are performed from requirements through delivery, including management.

Under this scenario, the development productivity rate for the enhancement itself is 500 LOC per month. But if the productivity rate

is calculated on the basis of what is delivered to users, then the productivity rate of the new release is 5,500 LOC per month or more than an order of magnitude higher.

Using function points the same situation might look as follows: The existing application is 800 function points in size, and the enhancement will add 80 function points to it. The enhancement itself has a productivity rate of 4 function points per month, but if the existing software is included, the rate balloons up to 44 function points per month.

The measurement rules used by Software Productivity Research in the case of enhancements is that the existing, base system is not counted for productivity purposes. The rationale for the SPR method is that manufacturing economics operate in this fashion for dealing with additions to buildings and manufactured products. However, the software literature is highly ambiguous on this point, almost 50 percent of the published works on enhancement productivity do include the base in the productivity calculations.

Deletion Ambiguity

Assume that a software project is established to enhance an existing application by replacing a particular set of functions with a new version. Assume the base application is 100,000 C statements in size, and that in the enhancement 10,000 new C statements will replace 10,000 previous C statements. Assume that the effort for all activities in making the deletion is 5 person months, and the effort for all activities of adding the new functions from requirements through delivery and including management is 20 person months.

Unfortunately, the ambiguity of the software literature in this situation spans every possible combination and permutation of possibilities. The SPR measurement method in this case follows generally accepted manufacturing practices and measures the deletion effort and the enhancement effort separately and then aggregates them. Using the SPR method, deletion productivity is 2000 LOC per month and the new enhancement is 500 LOC per month. The aggregated net productivity is 400 LOC per month for the deletion and addition together.

Using function points for the same scenario, the base application is 800 function points in size. The new features add 80 function points, which replaces 80 function points of prior software. The productivity for the new addition is 4 function points per month, and for the deletion the rate is 16 function point per month. The aggregated total effort for both new and deleted work is 3.2 function points per staff month.

Reuse Ambiguity

Assume that a software project is able to make use of a formal reusable code library. The total application as delivered is 20,000 C statements

in size, and reusable code constitutes 50 percent of the total, or 10,000 C statements. Assume that the effort for extracting and linking the reusable code totals to one day. Assume that the effort for developing the 10,000 LOC totals to 20 months for all activities for the project from requirements to delivery, including management. In this case, development productivity is 500 LOC per month, and delivery productivity is almost 1,000 LOC per month. If the effort for extracting and linking the reused code is converted into a monthly rate, then the productivity rate would be 200,000 LOC per month.

The measurement rules used by Software Productivity Research in this case are to measure both development and delivery productivity, since both are significant. Here, too, there are no actual standards, although the literature in this case is weighted toward measuring delivery productivity. Obviously delivery productivity is more important for economic purposes. Also, in the domain of object-oriented languages, it is much more appropriate to consider delivery productivity than development productivity. The reuse of software components is treated in the same fashion that standard subassemblies are dealt with in manufacturing economics.

When function points are used, the volume of reusable material is 80 function points. The volume of new work is also 80 function points. The development productivity rate is 4 function points per staff month, and the delivery productivity is almost 8 function points per staff month. The effort just for extracting and linking the reused code is almost 1,600 function points per staff month.

Package Ambiguity

Assume that a software package is acquired from an external vendor that met most of the user requirements for a new application. The package totals some 50,000 C statements in size. However, it was necessary to add 10,000 C statements to the package in order to customize to local needs. Assume the effort to evaluate and acquire the package is 4 person months. Assume that the effort for all enhancement activities for this project totals to 20 person months from requirement to delivery, including management. In this case, development productivity is 500 LOC per month, and delivery productivity is 2500 LOC per month including the effort for package evaluation and development. The productivity rate for package acquisition itself would be 12,500 LOC per month.

Expressed in function points, the package totals to 400 function points, and it is necessary to add 80 function points to it. Development productivity is 4 function points per staff month, and delivery productivity is 20 function points per staff month. The productivity rate for just the package acquisition tasks is 100 function points per staff month.

The measurement rule used by Software Productivity Research, in this case, is to measure acquisition, development, and delivery productivity. This is also the way manufacturing economics deals with purchased products. However, the software literature is ambiguous on this point. Unlike the previous case of reuse, the literature dealing with packages is weighted toward measuring development productivity, since neither the actual size of many packages nor the acquisition effort are known precisely.

Ambiguity in Measuring Complexity

One of the most important factors that influences both software productivity and quality is that of "complexity." It is surprising, therefore, that such an important topic has such a sparse literature. Indeed, many of the different kinds of real-world complexity have essentially zero citations in the software engineering literature.

The kinds of complexity that affect software projects can be divided into three general classes: (1) Complexity in the problem set surrounding the application; (2) Complexity in the data and relationships among the entities associated with the application; (3) Complexity of the structure of the application's code and control flow.

Only the topic of code and control flow complexity has a reasonable number of citations, which tend to center around McCabe's cyclomatic and essential complexity concepts. However, even here the literature is ambiguous as to the effects of cyclomatic or essential complexity on actual programming performance.

Several forms of complexity that would seem relevant to software engineering, such as the syntactic and semantic complexity of specifications, have few citations. This is surprising, since linguistic literature is quite voluminous and a number of commercial tools can carry out standard textual complexity analyses such as the Fog and Fliesch indexes. There are some interesting studies comparing text, graphics, and various symbol sets for perceptual or mnemonic value, but on the whole, the literature is rather sparse and the results ambiguous.

The subject of topologic complexity, or the ability to transform software into different structural configurations is not widely published, even though there are commercial tools available that can transform software structures topologically for languages such as COBOL.

Perhaps the most notable shortage of citations, and an area of substantial ambiguity, is the domain of information, data, entities, and relationships complexity. This topic obviously has relevance to databases and repositories and clearly is an influential factor for software as a whole. Yet there are a shortage of standard metrics and normalization methods in this important domain. One would think that the repository vendors

such as IBM the database companies such as Oracle and SAP, or the Information Engineering (IE) community such as Texas Instruments or JMA would be performing substantial research on data complexity and associated metrics, but this appears not to be the case as of 2008. It can be flatly stated that without adequate synthetic metrics that encompass data volumes and complexity, the phrase "information engineering" is a misnomer. There is no engineering without powerful synthetic metrics.

Ambiguity in Functional Metrics

As of 2008 function point metrics are the major tool for performing economic studies of software. That being said, it is also necessary to add that as of 2008 there are at least 38 different variations of "function points" and their results can differ by large but unknown amounts from one another.

Historically, functional metrics for software originated in the mid-1970s within IBM. Allan Albrecht and his colleagues developed a synthetic metric for measuring the size of information, systems called function points. As stated previously, the original function point metric was publicly announced in October of 1979 at a jointly sponsored conference by SHARE, GUIDE, and IBM. A major revision to the IBM function point metric occurred in 1984, and the 1984 version became the basis for the most widely used function point counting method in the United States today.

In 1986, the International Function Point Users Group (IFPUG) was created and today includes more than 400 member companies in the United States alone, plus members in Europe, Australia, South America, and the Pacific Rim. The IFPUG counting practices committee is the de facto standards organization for function point counting methods in the United States. The IFPUG counting practices manual is the canonical reference manual for function point counting techniques in the United States and much of the world.

However, for various reasons, the success of function point metrics spawned the creation of many minor variations. As of 2008, the author has identified about 38 of these variants, but this may not be the full list. No doubt at least another 20 variations exist that either have not been published or not published in journals yet reviewed by the author.

This means that the same application can appear to have very different sizes, based on whether the function point totals follow the IFPUG counting rules, the British Mark II counting rules, COSMIC function point counting rules, object-point counting rules, the SPR feature point counting rules, the Boeing 3D counting rules, or any of the other function point variants. Thus, application sizing and cost estimating based on function point metrics must also identify the rules and definitions of the specific form of function point being utilized.

Here is an example derived from the author's previous book, *Estimating Software Costs*: Suppose you are a metrics consultant with a client in the telecommunications industry who wants to know what methods and programming languages give the best productivity for PBX switching systems. This is a fairly common request.

You search various benchmark databases and find 21 PBX switching systems that appear to be relevant to the client's request. Now the problems start:

- Three of the PBXs were measured using "lines of code." One counted physical lines, one counted logical statements, and one did not define which method was used.
- Three of the PBXs were object-oriented. One was counted using object points and two were counted with use-case points.
- Three of the PBXs were counted with IFPUG function points.
- Three of the PBXs were counted with COSMIC function points.
- Three of the PBXs were counted with NESMA function points.
- Three of the PBXs were counted with feature points.
- Three of the PBXs were counted with Mark II function points.

As of 2008, there is no easy technical way to provide the client with an accurate answer to what is really a basic economic question. You cannot average the results of these 21 similar projects nor do any kind of useful statistical analysis because so many different metrics were used.

In the author's opinion, the developers of alternate function point metrics have a professional obligation to provide conversion rules from their new metrics to the older IFPUG function point metric. It is not the job of IFPUG to evaluate every new function point variation. The function point sizes in this book are based on IFPUG counts, with Version 4.2, as stated earlier.

Over and above the need to be very clear as to which specific function point is being used, there are also some other issues associated with function point sizing that need to be considered.

The rules for counting function points using most of the common function point variants are rather complex. This means that attempts to count function points by untrained individuals generally lead to major errors. This is unfortunate but is also true of almost any other significant metric.

Both the IFPUG and the equivalent organization in the United Kingdom, the United Kingdom Function Point (Mark II) Users Group, offer training and certification examinations. Other metrics organizations, such as the Australian Software Metrics Association (ASMA) and

the Netherlands Software Metrics Association (NESMA) may also offer certification services. However, most of the minor function point variants have no certification examinations and have very little published data.

When reviewing data expressed in function points, it is important to know whether the published function point totals used for software cost estimates are derived from counts by certified function point counters, from attempts to create totals by untrained counters, or from six other common ways of deriving function point totals:

- Backfiring from source code counts, using mathematical ratios of lines of code to function points
- Pattern-matching using the taxonomy discussed earlier in this book
- Automatic generation of function points from requirements and design, using tools
- Automatic generation of function points from data mining of legacy applications
- Deriving function points by analogy, such as assuming that Project B will be the same size as Project A, a prior project that has a function point size of known value
- Counting function points using one of the many variations in functional counting methods (i.e., Boeing 3D function points, COSMIC function points, Mark II function points, Netherlands function points, etc.)

As a result of the lack of written information for legacy projects, the method called "backfiring," or direct conversion from source code statements to equivalent function point totals, has become one of the most widely used methods for determining the function point totals of legacy applications. Since legacy applications far outnumber new software projects, this means that backfiring is actually the most widely used method for deriving function point totals.

Backfiring is highly automated, and a number of vendors provide tools that can convert source code statements into equivalent function point values. Backfiring is very easy to perform, so that the function point totals for applications as large as one million source code statements can be derived in only a few minutes of computer time.

The downside of backfiring is that it is based on highly variable relationships between source code volumes and function point totals. Although backfiring may achieve statistically useful results when averaged over hundreds of projects, it may not be accurate by even plus or minus 50 percent for any specific project. This is due to the fact that individual programming styles can create very different volumes of source code for the same feature. Controlled experiments by IBM in which eight

programmers coded the same specification found variations of about 5 to 1 in the volume of source code written by the participants.

Also, backfiring results will vary widely based upon whether the starting point is a count of physical lines or a count of logical statements. In general, starting with logical statements will give more accurate results. However ,counts of logical statements are harder to find than counts of physical lines.

In spite of the uncertainty of backfiring, it is supported by more tools and is a feature of more commercial software-estimating tools than any other current sizing method. The need for speed and low sizing costs explains why many of the approximation methods, such as backfiring, sizing by analogy, and automated function point derivations, are so popular: They are fast and cheap, even if they are not as accurate. It also explains why so many software-tool vendors are actively exploring automated rule-based function point sizing engines that can derive function point totals from requirements and specifications, with little or no human involvement.

Since function point metrics have splintered in recent years, the family of possible function point variants used for estimation and measurement include at least 38 choices as shown in Table 7-3. Note that Table 7-3 is taken from the author's previous book *Estimating Software Costs*.

Note that this listing is probably not 100 percent complete. The 38 variants shown in Table 7-3 are merely the ones that have surfaced in the software measurement literature or been discussed at metrics conferences. No doubt, at least another 20 or so variants may exist that have not yet been published or presented at metrics conferences.

Here in the United States, membership in IFPUG has been growing at a rate of nearly 25 percent per year. As early as 2000, IFPUG had become the largest software measurement organization in the United States. If membership continues to expand at the current rate, then IFPUG will eventually become one of the largest software professional groups of any kind.

To be useful, the variability of the IFPUG function point method itself must be known. A national study commissioned by IFPUG and carried out by Kemerer and his colleagues at MIT revealed about a 11 percent U.S. variance in counting function points. An 11 percent variation is much lower than the order-of-magnitude variation associated with LOC and KLOC counting. Standard IFPUG function point counts are normally within the range of accuracy for carrying out useful economic studies. Indeed, the accuracy of function point counting using IFPUG rules is about equal to the accuracy of large-scale economic studies caused by variations in currency exchange rates.

Although the variations in functional metrics do introduce substantial ambiguity, the absolute range of ambiguity with functional metrics

TABLE 7-3 Function Point Counting Variations, Circa 2007

1.	The 1975 internal IBM function point method
2.	The 1979 published Albrecht IBM function point method
3.	The 1982 DeMarco bang function point method
4.	The 1983 Rubin/ESTIMACS function point method
5.	The 1983 British Mark II function point method (Symons)
6.	The 1984 revised IBM function point method
7.	The 1985 SPR function point method using three adjustment factors
8.	The 1985 SPR backfire function point method
9.	The 1986 SPR feature point method for real-time software
10.	The 1994 SPR approximation function point method
11.	The 1997 SPR analogy-based function point method
12.	The 1997 SPR taxonomy-based function point method
13.	The 1986 IFPUG Version 1 method
14.	The 1988 IFPUG Version 2 method
15.	The 1990 IFPUG Version 3 method
16.	The 1995 IFPUG Version 4 method
17.	The 1989 Texas Instruments IEF function point method
18.	The 1992 Reifer coupling of function points and Halstead metrics
19.	The 1992 ViaSoft backfire function point method
20.	The 1993 Gartner Group backfire function point method
21.	The 1994 Boeing 3D function point method
22.	The 1994 Object Point function point method
23.	The 1994 Bachman Analyst function point method
24.	The 1995 Compass Group backfire function point method
25.	The 1995 Air Force engineering function point method
26.	The 1995 Oracle function point method
27.	The 1995 NESMA function point method
28.	The 1995 ASMA function point method
29.	The 1995 Finnish function point method
30.	The 1996 CRIM micro–function point method
31.	The 1996 object point method
32.	The 1997 data point method for database sizing
33.	The 1997 Nokia function point approach for telecommunications software
34.	The 1997 full function point approach for real-time software
35.	The 1997 ISO working group rules for functional sizing
36.	The 1998 COSMIC function point approach
37.	The 1999 Story point method
38.	The 2003 Use Case point method

is still less than one-fifth of the range of LOC and KLOC metrics. Also, the IFPUG counting practices committee has the ability to evaluate and endorse or reject some of the more exotic variations that have surfaced in the domain of functional metrics.

On the whole, functional metrics appear to be more stable, more objective, and to have a better correlation with standard manufacturing economics than LOC and KLOC metrics.

Ambiguity in Quality Metrics

The domain of quality metrics is among the most subjective and ambiguous area in the entire literature of software engineering. The problem starts with the lack of a logical and consistent definition for what the word "quality" actually means.

The definition in the European ISO 9000 through 9004 standards are both subjective and ambiguous. For example, the ISO definitions of quality include "portability" as an adjunct of software quality. Portability is an important topic, but it has essentially nothing to do with quality for standard manufactured products.

The common U.S. definitions are even worse: For example, the widely used concept that quality means "conformance to requirements" is illogical as a serious quality definition. Requirements errors themselves constitute one of the major problem areas of the software industry (about 15 percent of defect totals). To define conformance to a major source of error to mean "quality" is an example of circular reasoning. Worse, under this definition there is no way to measure or explore errors in the requirement themselves.

For the purposes of creating unambiguous quality metrics, an acceptable definition of "quality" needs to possess two key attributes: (1) It should be predictable before it occurs; (2) It should be measurable when it does occur.

With these two ground rules in mind, software quality metrics can be centered around the following concepts: (1) Defect origins, (2) defect quantities, (3) defect severities, (4) root causes of defects, (5) defect removal efficiency, (6) user-reported defects, (7) duplicate defect reports, (8) invalid defect reports, and (9) user satisfaction.

Two very common software quality metrics are so ambiguous that it would seem desirable to cease using them: defects per KLOC and cost per defect.

Ambiguity with the Defects per KLOC Metric

Measuring defects in terms of KLOC is paradoxical and will automatically penalize high-level languages. Also, measuring defects with KLOC is psychologically harmful in that it leads away from the analysis of requirements, design, and documentation errors.

Consider the following examples: Assume that an application of 10 KLOC was written in basic Assembly language and had the following defect levels: 50 requirements defects, 100 design defects, 300 coding

defects, and 50 user documentation defects. The total number of defects is 500, and therefore, this version has a level of 50 defects per KLOC, which is fairly typical of basic Assembly programs.

Now assume that Ada was used rather than Assembly, and the program required only 2.5 KLOC of Ada statements. Following are the defect levels associated with the Ada version: 50 requirements defects, 100 design defects, 50 coding defects, and 50 user documentation defects.

The total number of defects is reduced to 250, or 50 percent lower than the assembly case. This is clearly a significant improvement in real software quality. But because requirements, design, and documentation defect totals are unchanged, the Ada defects per KLOC rate has now doubled, and jumped up to 100 per KLOC! As may be seen the metric, "defects per KLOC" is extremely harmful for studying defects quantities when high-level languages are used.

(Since both the Assembly and Ada versions are functionally identical, they would both have the same function point total: 35 function points. Using function points instead of KLOC, the Assembly version had 14.3 defects per function point, whereas the Ada version had only 8.6 defects per function point. For quality measurement purposes, function points reveal important information that the KLOC metric conceals and distorts.)

The problem of using KLOC metrics for exploring requirements, design, and documentation defects is so pervasive that even major companies such as AT&T, Raytheon, Hewlett-Packard, and Siemens have comparatively little valid data on front-end problems. Indeed, within the companies just named the defect tracking systems normally commence only with testing. The defect quantities and severities associated with the entire front-end of the lifecycle is seriously underreported, and much of the blame can be attributed to the use of KLOC as a normalization metric for quality reporting.

Ambiguity with the Cost per Defect Metric

Almost every month for more than 50 years, a published book or article has contained the following phrase: "It costs 100 times as much to fix a bug after delivery as it does during design." Unfortunately, as it is commonly calculated, the "cost per defect" metric penalizes quality and achieves its lowest costs wherever the greatest numbers of defects are found. This explains why the cost per defect rate rises steadily throughout the lifecycle: The number of defects steadily declines, so the cost per defect will steadily rise.

The economic problem with cost per defect is that it ignores the fixed costs associated with the preparation and execution associated with defect removal operations.

Consider the cost per defect encountered when unit testing low-quality and high-quality programs. The low-quality program will have 50 bugs that were detected via unit testing, and the high-quality program will have only 10 bugs that were detected via unit testing. Assume $300 per day is the fully burdened cost.

For the low-quality program with 50 bugs, test case preparation took one day, running the test cases took one day, and repairing the defects took five days. The total cost for the seven day test and repair cycle is $2,100. The cost per defect is $42.00.

For the high-quality program with 10 bugs, test case preparation took one day, running the test cases took one day, and repairing the defects took one day. The total cost for the three day test cycle is only $900, or 57 percent less than the low-quality case. However, the cost per defect in this case is $90.00, or more than twice that of the low-quality example.

As may be seen, cost per defect rises as defect quantities decline due to the impact of fixed or inelastic costs. This phenomenon is counter to the goals of quality improvement, and also counter to the principles of manufacturing economics. It is a severe enough problem to cease using the cost per defect metric.

As a software product moves through the lifecycle, normal defect removal operations will find more defects in the beginning of the cycle than at the end. The fixed costs associated with defect removal, however, are comparatively constant. The inevitable result is the cost per defect will be cheap at the beginning and expensive at the end.

A final caution must be given about cost per defect: Zero-defect software is not impossible, and in the future may become fairly common. It is obvious that cost per defect cannot be used to assess zero-defect software, and it is also obvious that zero-defect software will still have substantial quality costs in the forms of reviews, inspections, testing, and maintenance training and preparation.

Fortunately, functional metrics provide an acceptable alternative. Assume that both the low-quality and high-quality examples were both 30 function points in size. The unit testing cost for the low-quality version was $70.00 per function point. The unit testing cost for the high-quality version was only $30.00 per function point. As may be seen, functional metrics agree with the standard economic definition of productivity and with concepts of manufacturing economics.

Even if the product in question had zero defects, function points could still be used. Since preparation and execution of test cases cost $600, the cost per function point for testing zero-defect software in this case would be $20.00. Obviously, the cost per defect metric cannot be used in situations where there are zero defects.

Failure to Measure Defect Potentials and Defect Removal Efficiency

As mentioned earlier in this book, two of the most important topics in the domain of software quality are those of "defect potentials" and "defect removal efficiency." The phrase "defect potentials" refers to the sum total of software defects or bugs that originate in all software deliverables, including at least (1) requirements defects, (2) design defects, (3) source code defects, (4) user documentation defects, and (5) "bad fixes" or secondary defects accidentally injected while fixing a prior defect.

The phrase "defect removal efficiency" refers to the number of defects removed, divided by the number of defects present, with the results expressed as a percentage. Thus, if an activity such as unit testing found 25 bugs, and subsequent testing plus user-reported bugs totaled to 75, then the efficiency of unit testing in this example is 25 percent. The 25 bugs eliminated and the 75 other bugs constitute the "defects present" value.

A few leading companies such as AT&T, IBM, ITT, Hewlett-Packard, and Motorola have been measuring this kind of information since the 1960s, so the literature on the subject is moderately extensive. For example, Dunn and Ullman from ITT published books with such data in both 1982 and 1984. In 1992 Robert Grady of Hewlett-Packard published similar data from that company's defect potential and removal efficiency experience. The author of this book and his colleagues have continued to measure defect potentials and defect removal efficiency, and plan to do so in the future.

In spite of the outstanding results that these companies have achieved from measures of defect potentials and removal efficiency, the topic remains underreported in the software engineering literature. Indeed, the current ISO 9000-9004 quality standards do not even mention defect potentials and defect removal efficiency! Even the capability maturity model (CMM) of the Software Engineering Institute is silent on the topics of defect potentials and defect removal efficiency levels.

Studies of defect potentials and defect removal efficiencies are quite shocking when first encountered and tend to discredit many standard beliefs. For example, the very common concept that "quality means conformance to requirements" is thrown into question when it is realized that requirement is one of the chief sources of software defects.

The equally common belief that testing is the most effective form of defect removal is also overthrown: Most forms of testing are less than 30 percent efficient, whereas formal inspections can achieve defect removal efficiencies in excess of 85 percent.

Estimating and measuring defect potentials and defect removal efficiency levels should be standard practices for every software project in

the world. This is because projects with low defect potentials and high defect removal efficiency also have the shortest schedules, lowest costs, and best customer satisfaction levels. The current U.S. average is a defect potential of about 5.0 defects per function point coupled with a removal efficiency of only about 85 percent. Best in class organizations are below 2.5 potential defects per function point and eliminate at least 95 percent of these defects before delivery. This pays off with shorter schedules, lower costs, and much lower maintenance costs.

The Problems of Measuring the Impact of "Soft" Factors

Assume your organization had a measurement system in place that could capture resource and cost data with a precision of 99.999 percent and no leakage. Assume also that your company had adopted the IFPUG function point metric and had abandoned obsolete LOC metrics. You have just completed your first major productivity study and have measured 10 projects, where the productivity ranges from 1.25 function points per staff month to a high of 25.0 function points per staff month. When you present these new findings to your vice president of software, he or she asks, "What causes the difference between our low productivity projects and our high productivity projects?" What answer would you give to this basic question?

Of all measurement topics, the least explored and the least understood is that of capturing "soft" information on why projects vary. (The term "soft data" is used in opposition to "hard data" and means factors where the subjective opinions of human beings provide the information. Examples of "soft" data include the skill and experience levels of team members, usefulness of tools and methods, and the clarity and completeness of requirements. In all of these topics, the subjective opinions of human beings must be captured. The problem is to capture and utilize subjective information with minimum bias and maximum reliability.) Unfortunately, this very important topic was seldom addressed in the software engineering literature.

There are fundamental gaps in the software engineering literature dealing with this topic, including:

- How many different factors are known to influence software projects?
- What is the range of impact of the influential factors?
- Is there a practical taxonomy for organizing influential factors?
- Is it possible to rank order influential factors in a meaningful way?
- What is the minimum set of influential factors that should be recorded?

- What is the best method for recording such largely subjective information?
- What kind of sample size is needed to minimize bias with subjective information?
- How often should new factors be added to the set of influential factors?

As of 2008, several sets of soft-factor collection methods are used in the United States. The oldest is the process assessment questionnaire developed by Software Productivity Research and called "the SPR assessment method" that came out in 1984 a year before the Software Engineering Institute was formed. More recently, the assessment method of the Software Engineering Institute associated with the famous "capability maturity model" or CMM has become the most widespread. However there are other assessment methods such as those used by the David Consulting Group, by QSM, and by several other consulting companies. In Europe the TickIT method is widely used.

Several leading U.S. organizations such as AT&T, Hewlett-Packard, and Motorola use both the SPR and SEI soft factor assessment methods concurrently, since the two are somewhat complementary. Miller and Tucker of AT&T have published an interesting discussion of their concurrent usage.

SPR uses a set of linked multiple-choice questionnaires covering both strategic corporate issues and tactical project issues that affect quality and productivity and user satisfaction. The total number of SPR questions is about 400.

The SPR questions are based on a five-point performance scale, where "1" means *excellent,* "2" means *good,* "3" means *average,* "4" means *marginal,* and "5" means *poor*.

For their original capability studies, the SEI used a set of some 129 binary questions with "Yes" or "No" answers. The questions are essentially process oriented and tactical in nature. Strategic and environmental topics, such as compensation, hiring practices, office environments, and so on, are not covered. To date, user satisfaction is not covered by SEI either. A subset of some 85 questions were used for establishing the "maturity level" of enterprises.

Although developed independently, the SPR and SEI assessments share a number of features. (Both Capers Jones of SPR and Watts Humphrey of SEI worked at IBM during the same time periods, and both were performing process assessments within IBM.) The SPR assessments and the SEI assessments begin with kick-off meetings; include training of the local staff in the questionnaire, interviews of selected projects, aggregation and analysis of the results, a report of strengths and weaknesses, and suggested improvements.

Differences Between the SPR and SEI Assessment Questions

The SPR strategic and tactical questionnaires are based on a 5-level performance scale. The "strategic" questions deal with general corporate issues, and the "tactical" questions deal with topics that impact specific projects. Following are examples of strategic and tactical questions:

Example of an SPR strategic question:
Enterprise Compensation Policy?

1. Enterprise pay is much higher than competitors.
2. Enterprise pay is somewhat higher than competitors.
3. Enterprise pay is at competitive averages.
4. Enterprise pay is somewhat below competitors.
5. Enterprise pay is much lower than competitors.

Example of an SPR tactical question:
Quality Assurance Function?

1. Formal QA group with adequate resources.
2. Formal QA group but understaffed (<1 to 30 ratio).
3. QA role is formally assigned to development personnel.
4. QA role is performed informally.
5. No QA function exists for the project.

The SPR questions are all structured so that "3" is the approximate U.S. average for the topic being explored. The goal of the SPR assessment method is twofold: (1) To find out how companies and government agencies compare to normal U.S. practices; (2) To identify specific strengths and weaknesses so that weaknesses can be corrected.

Structure and Goals of the SEI Question Set

Since the SEI questions are binary and not based on a five-point performance scale, the SEI assessments are not usually capable of determining *how well* activities are being performed. The SEI method primarily is used to determine if an appropriate set of activities either *is* or *is not* being performed.

Here is an example of the SEI question that deals with the quality assurance function:

Example of an SEI tactical question:
Does the software quality assurance function (SQA) have a management reporting channel separate from the software development project management? (**Yes** or **No**)

From the patterns of key answers to some 85 of the 129 questions, the SEI assessment process next determines the "maturity level" of the company. The maturity level is based on a five-plateau scale. However, unlike SPR, the SEI scale uses 1 as the "worst case" and 5 as the "best case." Also, SEI does not have a midpoint or "average" in its maturity schema. Indeed, of the enterprises assessed by SEI to date, the numeric average is only about 1.5. The five SEI maturity levels are

- **Level 1: "Initial" processes** Unpredictable and poorly controlled
- **Level 2: "Repeatable" processes** Can do follow-on projects without making the same mistakes
- **Level 3: "Defined" processes** Uses a well-defined and effective methodology
- **Level 4: "Managed" processes** Methods are not only effective, but also are measured!
- **Level 5: "Optimizing" processes** Leading-edge, with full reusability, modern methods, etc.

This maturity-level schema is interesting, but also somewhat abstract. Several analyses of the SEI maturity-level assumptions have been published. A very careful analysis by McGowan and Bollinger of Contel deserves thoughtful reading by those interested in the SEI assessment approach.

The SEI assumptions are that productivity increases monotonically with level, so that optimum productivity rates will be achieved at level 5. This has been shown to be true, but the costs of achieving SEI levels 4 and 5 are so high that some organizations reaching those levels might not be able to win contracts in open competitive bidding.

Also, the SEI questions appear to be biased in favor of large organizations that produce mainframe software. There appears to be an intrinsic bias with the SEI questions that is unfavorable to small companies. It is unlikely that software organizations with fewer than 30 employees can ever achieve SEI maturity levels greater than 2 because the overhead staffing and expenses are so extreme at levels 3, 4, and 5.

Correlating the SPR and SEI Assessment Results

In spite of the differences in the form and structure of the SPR and SEI questions, it is possible to carry out rough correlations between the SPR and SEI results. Indeed, the CHECKPOINT and CHECKMARK tool produces an overall "SPR Performance Level" that is more or less equivalent to the SEI maturity level, although the numbering systems run in opposite directions. Here are the rough equivalencies between the SPR and SEI levels:

SPR PERFORMANCE LEVEL		SEI MATURITY LEVEL	
1.00 to 1.99	Excellent	5	Optimizing
2.00 to 2.49	Good	4	Managed
2.50 to 3.49	Average	3	Defined
3.50 to 3.99	Marginal	2	Repeatable
4.00 to 5.00	Poor	1	Initial

The above correlations are crude and imperfect, but show that at least both groups are concerned with similar issues. In particular, the SPR performance levels include staff experience factors that were not included in the original SEI assessment questions, although Dr. Bill Curtis later developed an auxiliary CMM that did include personnel factors. In general, for projects and companies where staff and managerial experience is high, the standard SPR performance level will give more positive results than the SEI assessment method.

In recent years, the SEI has developed a new and expanded capability model called the "Capability Maturity Model Integration" or CMMI. This model is now widely used in the United States.

It should be noted that other "soft factor" collection methods also exist, such as the TickIT method used in Europe and the assessment methods used by the David Consulting Group, Quantitative Software Management (QSM), and several others.

Problems in Measuring Software Value

Complete economic analysis of software should also be able to explore the value, consumption, and usage patterns of software applications. The literature on the topic of software business value, usage patterns, and the impact of software on knowledge work is not extensive, and much of it appears to be subjective.

Financial value of software is more or less under control, and the standard forms of measurement include both cost reductions and revenue increases associated with various kinds of software. However, there are a wide range of intangible value topics that are very difficult to measure. For example, if a software application can improve customer satisfaction, team morale is difficult to quantify. Even harder to measure are applications such as medical software that can save human lives or military software that improve national defense.

What appears to be needed is a "value point" metric based on the structure and format of function point metrics that is capable of integrating financial value and intangible value into a single metric.

Also, the usage of software applications has lagged in study and research. As a sample of this kind of research, consider the usage pattern of a typical spreadsheet such as Excel or Quattro. These tools contain in

the vicinity of 2,000 function points, and all cost in the vicinity of $500, which means they cost about $0.25 per function point to acquire.

In the course of a typical business day, an accountant, auditor, or financial analyst will use such tools for about three to four hours and utilize perhaps 500 of the 2,000 function points such tools contain. Thus, it can be theorized that financial personnel are utilizing spreadsheets at a rate of about 125 to 150 function points per hour.

Performing the same work manually with only a calculator would require at least 15 to 20 hours. (Performing the same work manually without using even a calculator would require more than 100 hours.) Thus, it can be hypothesized that without a spreadsheet, the same work could proceed at a rate of only 25 to 35 function points per hour.

Without going into elaborate calculations, it can be derived that spreadsheets improve financial tasks by a range of between 3.5 and 6 times.

The utilization of function points for business value and usage profiles is such a new undertaking that it is not clear if this approach will make a permanent contribution or be only a transient experiment. However, the preliminary results are intriguing enough to continue the exploratory work. Table 7-4 is an example of a study that measured the usage patterns of project management tools.

TABLE 7-4 Numbers and Size Ranges of Project Management Tools (Size Data Expressed in Terms of Function Point Metrics)

Project Management	Lagging	Average	Leading
Project planning	1,000	1,250	3,000
Project cost estimating			3,000
Statistical analysis			3,000
Methodology management		750	3,000
Renovation analysis			2,000
Quality estimation			2,000
Assessment support		500	2,000
Project measurement			1,750
Portfolio analysis			1,500
Risk analysis			1,500
Resource tracking	300	750	1,500
Value analysis		350	1,250
Cost variance reporting		500	1,000
Personnel support	500	500	750
Milestone tracking		250	750
Budget support		250	750
Function point analysis		250	750
Backfiring: LOC to FP			750
Function point subtotal	*1,800*	*5,350*	*30,250*
Number of tools	*3*	*10*	*18*

By analyzing the numbers of tools used and their approximate size in function points, it is possible to make a preliminary evaluation of the usefulness of tools to software project managers.

It is appropriate that two of the first domains where function points are being used to explore business value and usage patterns are those of software project management tools and software engineering tool suites. Software managers who utilize a full suite of project planning tools, estimating tools, tracking tools, and so on, may have in excess of 30,000 function points of processing power available to them. Programmers and software engineers with fully integrated tool suites may have in excess of 50,000 function points of processing power available today and perhaps more than 100,000 function points of processing power by the end of the century.

Because many military and defense projects use the "earned value" concept, there are at least a dozen tools on the commercial market that support earned-value calculations. There are also commercial estimating tools such as SEER and SLIM that include earned-value capabilities.

Lack of Effective Measurement and Metrics Automation

Measurement carried out informally by human beings tends to be inaccurate. The scope and breadth of software measurement tools has been deficient since the industry began and is still deficient in 2008, although improvements are on the horizon.

Tools for Measuring Functional Metrics

Function points can, in theory, be derived at least semiautomatically from specifications produced under standard approaches such as the Yourdon structured design approach, use cases, the information engineering approach, the Warnier-Orr approach, U.S. military standards, and a number of other specification protocols.

It is also possible to arrive at functional metrics by "backfiring," or the direct conversion from LOC counts. Here Software Productivity Research (SPR) and many other companies have produced tools that can handle backfire calculations. Although once the levels of programming languages are known, backfiring is easy to perform using a spreadsheet or even a pocket calculator. Backfiring is a bit complex for applications using multiple programming languages, so for these projects formal backfiring tools are more convenient.

As of the Spring of 2008, Relativity Technologies has announced a new form of function point counting tool for legacy applications that examines source code and extracts business rules, which can then be

used for automated function point analysis. The accuracy of this method is stated to be equivalent to manual function point counts, but the time required for such an automated count is only minutes as opposed to days or weeks with manual function point counting. However only about a dozen programming languages out of more than 700 are capable of using this new quick form of function point analysis. Even so, it is a very positive advance in function point methods.

In theory, it is not difficult to produce function point or feature point totals by direct scanning and parsing of source code. Many of the commercial code structure analyzers or complexity analyzers could easily be extended to include backfiring logic. Even better, tools that parse and analyze source code are often used for data mining to extract business rules. Once the business rules have been extracted from the code, they could serve as a kind of surrogate specification that could be used to derive standard IFPUG function points both speedily and accurately. The Relativity Technologies tool is the first commercial example, but not doubt other companies will follow suit.

Software estimating and measurement tools that support function points as explicit metrics are fairly common. The SPQR/20 estimating tool, on the market in 1984, was the first such commercial tool to provide explicit support for function points as well as for LOC and KLOC metrics. Other estimating and measurement tools that support functional metrics of various flavors include (in alphabetic order): Asset-R, the Bridge, BYL, CHECKMARK (in the United Kingdom), CHECKPOINT (in the United States and Europe), COCOMO II, ESTIMACS, GECOMO, KnowledgePlan, MARS, PADS, Price-S, SEER, and SLIM.

The newest kind of management tool utilizes pattern-matching for both sizing and estimating. For example, the author's "Risk Master" prototype can predict application sizes in terms of both function points and lines of code in less than one minute and can perform full development and maintenance cost estimates at the same time.

Potential Expansion of Functional Metrics to Solve New Problems

In spite of the considerable success of function point metrics in improving software quality and economic research, there are a number of important topics that still cannot be measured well or even measured at all in some cases. Here are some areas where the concept of function points need to be expanded to solve other measurement problems. Indeed an entire family of functional metrics can be envisioned.

As of 2008, function points have been successfully utilized for almost every kind of software project including but not limited to embedded software such as fuel-injection, systems software such as operating systems and switching systems, defense software such as weapons

control systems, commercial packages such as project management tools, information systems, client-server applications, and web applications using Java. This versatility suggests that functional metrics could be expanded into other areas.

The Need for Data Point Metrics

In addition to software, companies own huge and growing volumes of data and information. As topics such as repositories, data warehouses, data quality, data mining, and online analytical processing (OLAP) become more common, it is obvious that there are no good metrics for sizing the volumes of information that companies own. Neither are there good metrics for exploring data quality, the costs of creating data, migrating data, or eventually retiring aging legacy data.

A metric similar to function points in structure but aimed at data and information rather than software would be a valuable addition to the software domain.

Aspects of the British Mark II function point metric that include entities and relationships are moving in the direction of data metrics. A hypothetical data point metric might include the following factors:

- **Data points**
  - Logical files
  - Entities
  - Relationships
  - Attributes
  - Inquiries
  - Interfaces

Surprisingly, database and data warehouse vendors have performed no research on data metrics. As a result, there is little or no empirical information on data quality, costs of data base construction, data maintenance, or other economic topics. The problem is that the industry lacks an effective size metric for data.

The Need for Service Point Metrics

The utility of function points for software studies has raised the question as to whether or not something similar can be done for service groups such as customer support, human resources, sales personnel, and even health and legal professionals.

What would be useful would be a metric similar in structure to function points, only aimed at service functions within large corporations. Right now, there is no easy way to explore the lifetime costs of systems that include extensive human service components as well as

software components. A hypothetical service point metric might include the following factors:

- **Service points**
  - Customers (entities)
  - Inputs
  - Outputs
  - Inquiries
  - Reference sources
  - Rules and Regulations (constraints)

Experiments with variations on the function point metric have been carried out for customer support groups, insurance claims handling, and medical office practices. The results have been encouraging, but are not yet at a point for formal publication.

The Need for Engineering Point Metrics

The U.S. Navy and the other military services have a significant number of complex projects that involve hardware, software, and microcode. Several years ago, the Navy posed an interesting question: "Is it possible to develop a metric like function points for hardware projects, so that we can do integrated cost analysis across the hardware/software barrier?"

The ability to perform integrated sizing, cost, and quality studies that could deal with software, hardware, databases, and human service and support activities would be a notable advance indeed. A hypothetical engineering point metric might include the following factors:

- **Engineering points**
  - Inputs
  - Outputs
  - Constraints
  - Innovations
  - Algorithms

Integrated cost estimates across the hardware/software boundary would be very welcome in many manufacturing and military domains.

The Need for a COTS-point Metric

Many small corporations and some large ones buy more software than they build. The generic name for packaged applications is "commercial off-the-shelf software," which is usually abbreviated COTS.

Right now, COTS packages are outside the boundaries of function point metrics. It would be possible to include COTS packages if vendors published the function point sizes of commercial software applications. This is unlikely to happen in the near future. A hypothetical COTS metric might include the following factors:

- **Packaged application points**
  - Inputs
  - Outputs
  - Inquiries
  - Logical files
  - Interfaces
  - User satisfaction
  - Gaps or missing features

The inclusion of COTS points is desirable for dealing with "make or buy" decisions in which possible in-house development of software is contrasted with possible acquisition of a commercial package.

The Need for Value Point Metrics

One of the major weaknesses of the software industry has been in the area of value analysis and the quantification of value. All too often what passes for "value" is essentially nothing more than cost reductions or perhaps revenue increases. While these are no doubt important topics, there are a host of other aspects of value that also need to be examined and measured: customer satisfaction, employee morale, national security, safety, and a host of other topics. A hypothetical value point metric might include the following factors:

- **Value points**
  - Safety improvement
  - National security improvement
  - Risk reduction
  - Synergy (compound values)
  - Cost reduction
  - Revenue increases
  - Market share increases
  - Schedule improvement
  - Competitive advantages

- Customer satisfaction increase
- Staff morale increase
- Mandates or statutes

Note that although cost reduction and revenue increases are both value factors, a host of other factors also need to be examined, weighted, and included.

The Need for Risk Point Metrics

Software projects are nothing if not risky. Indeed, the observed failure rate of software projects is higher than almost any other human artifact. Although software risk analysis is a maturing discipline, there are still no metrics that can indicate the magnitude of risks. Ideally, both risks and value could be analyzed together. A hypothetical value risk point metric might include the following factors:

- **Risk points**
  - Risks of death or injury
  - Risks to national security
  - Risks of property destruction
  - Risks of theft or pilferage
  - Risks of security vulnerabilities
  - Risks of litigation
  - Risks of business interruption
  - Risks of business slowdown
  - Risks of market share loss
  - Risks of schedule delays
  - Risks of cost overruns
  - Risks of competitive actions
  - Risks of customer dissatisfaction
  - Risks of staff dissatisfaction

Large software projects fail almost as often as they succeed, which is a distressing observation that has been independently confirmed. It is interesting that project management failures in the form of optimistic estimates and poor quality control tend to be the dominant reasons for software project failures.

Tools for Measuring Complexity

The set of tools for measuring complexity appears to be growing at a rapid rate. Several varieties of complexity (i.e., cyclomatic complexity, essential complexity, N-path complexity, etc.) are now available from both commercial sources and as freeware or shareware from public web sites, forums, and from many companies. McCabe & Associates, CADRE, KnowledgeWare, Relativity Technologies, and ViaSoft are examples of commercial vendors that market complexity analysis tools.

Tools for Measuring Resources and Costs

Many project planning tools such as Artemis and Prestige by Lucas Management Systems can also collect resources. Also specialized tools such as Tracer and Application Intelligence by Computer Aid Incorporated capture resources for maintenance projects. Many estimating tools also have measurement modes, such as CHECKPOINT, SEER, or SLIM. In addition, a majority of large corporations tend to utilize various proprietary measurement systems built to assist their cost accounting functions.

The capabilities of these tools range from minimal to quite sophisticated, but all of them share an endemic problem: they leak resources. The problem of resource leakage is sometimes built into the tool itself, such as tools that do not allow slack time or unpaid overtime to be recorded. More often, however, the tools have the basic capability to record such factors but management discourages this kind of data from being recorded.

Another major resource leak consists of the time spent by users of applications when they are working in a technical capacity during requirements, reviews, doing documentation, or participating in acceptance testing. This is very common for Agile projects, where users are a permanent part of the development team. The quantity of user time on a major MIS project approaches 20 percent of the total effort expended, but user effort is almost never captured by standard resource tracking tools since the users are not part of the formal project team for accounting purposes and hence are not assigned charge codes.

Tools for Measuring Schedules, Schedule Overlaps, and Schedule Slippages

Essentially all of the 70 or so commercial project planning tools (such as Artemis, Prestige, Harvard Project Manager, Microsoft Project, Project Managers Workbench, Timeline, Superproject, etc.) can record basic project schedules and schedule overlaps.

Many project planning tools can record schedule slippages, too, if they are used correctly. However, if such tools are utilized in a mode whereby only *current* schedules are preserved, and *prior* schedules are not stored for historical purposes, then important information is being wasted. This is exactly the kind of information that is useful for serious studies of schedule slippages on long-term projects.

Several project estimating and measurement tools can also record schedules, overlaps, and slippages. Examples of estimating and measurement tools with schedule overlap and slippage recording capabilities include COCOMO II, KnowledgePlan, SEER, SLIM, and PADS.

Tools for Measuring Lines of Code

Most large corporations have built proprietary source code counting engines that embody their corporate counting standards: Examples of such proprietary tools exist within AT&T, IBM, Hewlett-Packard, and Siemens.

In addition, most compilers and interpreters create some kind of code counting output. There are also a number of commercial tools such as KnowledgeWare's Inspector, Peat Marwick's PATHVUE, McCabe & Associates BattleMap, and Analysis of Complexity Tool (ACT) that generate code counts, as does the renovation tool from Relativity Technologies.

There are even some code counting tools available as freeware and shareware from web sites and online information services.

Tools for Metrics Conversion

Tools that can convert data back and forth *between* various metrics counting conventions are not on the commercial market as of 2008. For example, in order to teach measurement internationally, the author had to build a proprietary code-size conversion tool that could: (1) Convert physical line counts into logical statement counts and vice versa; (2) Convert size between rules that include or exclude comments and blank lines; (3) Convert counts between rules than included or excluded reusable code. (Given almost 50 years of continuous usage of the LOC metric, it might be thought that such a tool would be readily available on the commercial markets, but in fact none could be discovered.)

Tools that can count multiple languages within the same application are also unavailable on the commercial market. About one-third of U.S. applications contain at least two languages (such as COBOL and SQL). The maximum number of languages within the same system, as observed by the author and his colleagues at SPR, is 13 languages.

Several companies are starting to build and market tools that can convert data back and forth between LOC metrics and function points or feature points. The first commercial tool to do this was SPQR/20,

marketed in 1985. CHECKPOINT, marketed in 1989, was the first tool to support feature points and offer full bidirectional conversion between LOC metrics, IFPUG function points, SPR function points, and SPR feature points. Other tools are attempting the same conversions such as COCOMO II, SEER, SLIM, etc. Surprisingly, the Software Engineering Institute (SEI) has neither published anything on this topic, nor appear to be building any tools to facilitate metrics conversion.

As of 2008, metric conversion remains a weak link in the software measurement world. With dozens of software metrics in existence the industry needs reliable conversion rules between common metrics such as physical and logical code counts; IFPUG and COSMIC function points, story points, use-case points, web-object points, and dozens of similar metrics.

Integrated Software Management Tools That Measure All Factors

As of 2008, there have only been a few attempts at integrated software management tools that can deal simultaneously with all of the critical factors: (1) Sizes of deliverables (i.e., pages, source code function points, test cases, etc.); (2) Work breakdown structures; (3) Schedules, schedule overlaps, and schedule slippages; (4) Resources planned and expended; (5) Costs planned and expended; (6) Staffing by time period; (7) Occupation groups and specialists utilized; (8) Organization structures utilized; (9) Quality and defect removal; (10) Critical path analysis; (11) "Soft" factors that influence the outcomes of the project (compensation, environment, tools, methods, etc.); (12) Multiple project aggregation and statistical analysis; (13) Complexity analysis and supporting metrics.

Because no single tool records all important metrics and assessment topics, it is necessary to acquire multiple tools and either link them together or perform stand-alone linkages using a spreadsheet. Early in 2008, Computer Aid Inc. announced an "Application Intelligence" tool that seems to come close to capturing all of the information needed. In addition, this tool allows used-defined metrics and can share data with other commercial tools.

Social and Political Resistance to Software Measurements

It is not possible to implement a software measurement program inside a corporation or government agency without considering the sociology of metrics and dealing with political opposition that normally occurs.

Most large enterprises are microcosms of the outside world and are subject to the same kinds of political and sociological issues. Executives and

managers often form power blocks that contest one another. At the technical staff and individual worker level, there are also polarized groups that are often at odds with one another.

When the subject of measurement is first broached in a typical organization, the normal reactions are public support but private opposition.

There are also political fights within the measurement domain itself. One of the most bitter political and technical disputes of the century is now in full swing: the war between "lines of source code" metrics and "function point" metrics. This dispute is arguably the most severe technological debate in the last 100 years and is roughly equivalent in anger and intensity to the debate between alternating current adherents and direct current adherents in 1892.

The war between function points and lines of source code is a reflection of yet another political dispute: the conflict between systems software and management information systems (MIS) software. The systems software domain, without really exploring the issues, tends to discredit function points because they were first used to measure MIS applications. The fact that Allan Albrecht is himself an electrical engineer and always envisioned function points as an engineering metric is a topic not generally known.

Many of the recent function point clones such as COSMIC function points and full function points were created to provide support for real-time and embedded software due to lack of appreciation that standard IFPUG function points could actually be used for such applications.

Let us now consider why a measurement program engenders resistance in software management. In today's world, without accurate and effective software metrics, there is no clear and obvious way for distinguishing excellent software performance from mediocre. When measurements are introduced, there is an immediate fear that the data might be used for punitive purposes.

Interestingly, the greatest opposition to measurement is often at the middle management and lower executive level: second and third line managers, directors, and in very large enterprises, perhaps vice presidents. These people typically live in a highly competitive, highly political environment in most large organizations. Without measurements, factors such as charisma, strength of character, and other intangible attributes tend to be the topics that lead to success or failure.

When measurements are introduced, middle management's natural fear is that their organization may rank below organizations managed by enemies or opponents. Rather than risk possible embarrassment, many middle managers would rather not have quantitative data at all. This rationale is often masked or concealed. For example, the most common reaction to measurement by middle managers is, "Measurements are fine for the other groups, but our projects are so complex that we might be penalized."

An unfortunate byproduct of middle management fears is the very common practice of hidden data. That is, specific projects, departments, and divisions are concealed in the measurement data. Each manager or executive is told how his or her organization compared against corporate averages, but Jane Doe is not told how her group compared against John Smith's group.

In the long run, hidden data is folly. Imagine trying to run a business if profits, losses, and financial data were concealed from senior executives. Can you imagine a presentation to a board of directors by a CFO that ran: "Some of our units lost money this year but we promised the unit managers we would not reveal which ones because we didn't want to embarrass anyone."

Yet the author has been present when a CIO gave a presentation to a corporate chairman and several directors that ran: "Here are the current productivity and quality levels by unit, but we promised confidentiality to the unit managers so we've disguised the unit names and locations."

When the covers are finally removed from software productivity and quality data, and specific units and major projects are identified, two remarkable sociological phenomena occur: (1) The units that rank *below* corporate averages in the first annual report will usually rank *above* corporate averages in the second and subsequent annual reports; (2) The aggregate performance of all groups will improve faster than at any time in corporate history.

At the individual contributor level, there is also a fear that measurement data might be misused for punitive purposes. This fear has a certain basis in fact, since from time to time measurement data is misused to set targets for individual contributors. As this chapter is being written, the author received a phone call from a software department manager who made the following query: "I want to know average schedules, so I can chew out the guys that are running late."

However, when measurements are done well, individual contributors typically benefit from accurate measurement data. Such data clearly reveals inadequacies in tools, office space, noise levels, and other factors, and is often used to improve the working conditions.

All of the sociological and political issues in the measurement arena are not yet fully understood. However, from close observations of what works and what doesn't work when measurements are introduced, following are some recommendations:

- Unit-level data should not be hidden or concealed from senior executives.
- Measurement data should not be used to set performance targets for individual contributors.

- Setting numeric quality targets for executives (directors, vice presidents) is quite effective.
- Setting numeric productivity targets for executives (directors, vice presidents) can be effective if done carefully.
- Setting irrational schedule targets for managers or individual contributors is harmful.
- Do not set unachievable targets such as "improve schedules 10 to 1 in 5 years."
- The purposes of measurement and uses of the data should be explained to all staff members.
- Hard data and soft data are an integrated set. Neither by itself has much value.

Without sensitivity to corporate politics and sociological issues, measurement programs can cause more harm than good and will not achieve their true benefits of revealing areas of strength and weakness. A good measurement program is like an annual physical exam or a semiannual dental checkup: the purpose is to prevent problems from occurring and to cure problems that have occurred before they spin out of control.

Measurement also has a positive value: it is quite exciting to see real improvements start to occur in quality, productivity, and cycle times. When actual improvements start to show up using accurate measurements, the psychological boost to a company and its morale is striking.

Ambiguity in Software Measurement and Metrics Terminology

One of the most common problems encountered when performing both SPR and SEI assessments is inconsistent usage of terminology. Even fairly commonly used terms such as "productivity," "quality," "inspection," and "unit testing" vary widely from company to company and even department to department within a company.

When SPR consultants and assessors are collecting information by interviewing project teams, more time is spent on explaining terms and dealing with terminology issues than on any other aspect of the assessment. In a typical three-hour assessment session, from 60 to 90 minutes will usually be devoted to terminology issues.

Most sciences and engineering disciplines that are growing rapidly go through periods of rapidly expanding terminology. Software engineering is no exception. What follows from this expansion of the basic vocabulary of a discipline is a period of confusion and ambiguity that can last from 10 to more than 50 years! Eventually, the inconsistencies will be

resolved and standard dictionaries of terms will become widely utilized. Unfortunately, that time is still in the future for software.

There is no shortage of software engineering dictionaries or glossaries, but they are often inconsistent among themselves and sometimes woefully inadequate. For example, the IEEE standard glossary of software engineering (Std 610.12-1990) lacked all references to functional metrics, in spite of the fact that IFPUG was the largest measurement association in the United States at the time the standard was published. The IEEE glossary also had very few management terms, measurement terms, or those connected with assessments and risk management.

In software as in other disciplines, advances in fundamental technologies bring expansion in the vocabulary associated with the discipline. New concepts and new terms go hand in hand. Indeed, the rate of growth of this glossary has been surprising. When SPR began performing its assessments in 1986, it soon became obvious that many terms were being questioned and discussed in every session. The first glossary produced in support of SPR assessments in 1987 was only about 20 pages in size and defined only about 100 terms. The more recent SPR glossaries have grown to more than 100 pages and more than 2,000 terms dealing with measurement, metrics, and software management topics. If the software industry continues to generate new terms and concepts at the current rate of terminology, then eventually a software measurement, metrics, and management dictionary will approach the size of *Dorland's Medical Dictionary*, which is to say almost 2,000 pages. This will probably be necessary because software is rapidly developing its own argot of special terms, or standard words used in special ways.

Software assessors and measurement specialists quickly learn some of the hazards that can result from inconsistent terminology. For example software productivity has been measured with source code metrics for almost 50 years. Yet this basic term can mean either physical source code lines or delimited source code statements. The apparent difference in the size of program can vary by 500 percent depending upon which choice is made. Much of the literature that uses source code metrics does not even state which choice the author made. Plainly, to become a true engineering discipline software engineering needs to adopt standard definitions for its basic terminology.

Coming to Grips with Maintenance and Enhancements

Nowhere is the software vocabulary more ambiguous than in the domain of "maintenance." This basic term has more meanings than almost any other word in the software vocabulary and has been used to discuss

defect repairs, enhancements, migrations to new platforms, and a host of other concepts.

Subordinate terms are even more troubling. For example, the military standard DoD 2167A uses the terms "corrective maintenance" for defect repairs, "adaptive maintenance" for new features, and "perfective maintenance" for things such as productivity improvements.

IBM, on the other hand, as a result of an antitrust suit in the 1970s restricted the term "maintenance" to mean *only* defect repairs. Other terms, such as "enhancements," were used when the changes were based on new features or customer demands. Although brought about by litigation, this terminology split made good business sense. The costs and work content of defect repairs do not resemble in any way the tasks of adding new functionality to software.

To match other business and engineering domains outside of software, the term "maintenance" should be used to discuss only defect repairs. Those repairs should be subcategorized into sets done under explicit warranties, implicit warranties, or for normal good technical practice. Maintenance costs, under this definition, should be absorbed by the developer or vendor.

For new features added to meet new customer requirements, the term "enhancements" is recommended. These enhancements are normally funded by users or at least will result in some form of external charge and cost recovery.

When changes to software are made as a result of law, government requirements, company policy, or some other mandate, the phrase "mandatory modification" is suggested. The costs for such required updates will probably have to be born by the vendor or development group.

When software is being moved to a new platform (DOS to UNIX or LINUX, for example), the term "porting" is suggested, since it is already widely used for that purpose. The assumption with porting is that multiple versions of the software will then exist. Porting costs are often chargeable or will open up new markets for the software.

If the software is being moved to a new platform, and the original software is to be discarded thereafter, then the term "migration" is suggested. This differs from porting in that the assumption is that a single version of the software will continue under a different guise. The charges for migration will vary with circumstances.

Various actions that might be taken to augment aging software should be defined separately and not lumped together under the blanket term "maintenance." Thus, the terms "restructuring," "reverse engineering," and "reengineering" should be used for cost collection and analysis purposes. The charges for these activities will vary with circumstances.

There are several difficulties in exploring software maintenance costs with accuracy. One of these difficulties is the fact that maintenance

tasks are often assigned to development personnel who interleave both development and maintenance as the need arises. This practice makes it difficult to distinguish maintenance costs from development costs because the programmers are often rather careless in recording how time is spent.

Another and very significant problem is the fact that a great deal of software maintenance consists of making very small changes to software applications. Quite a few bug repairs may involve fixing only a single line of code. Adding minor new features such as perhaps a new line item on a screen may require less than 50 source code statements.

These small changes are below the effective lower limit for counting function point metrics. The function point metric includes weighting factors for complexity, and even if the complexity adjustments are set to the lowest possible point on the scale, it is still difficult to count function points below a level of perhaps 15 function points.

An experimental method called "micro function points" is in development for small maintenance changes and bug repairs. This method is similar to standard function points, but drops down to three decimal places of precision. Thus changes that involve only a fraction of a standard IFPUG function point can be measured. The micro function point method should become available by 2008.

Of course, the work of making a small change measured with micro function points may be only an hour or less. But if as many as 10,000 such changes are made in a year, the cumulative costs are not trivial. Micro function points are intended to eliminate the problem that small maintenance updates have not been subject to formal economic analysis.

Quite a few maintenance tasks involve changes that are either a fraction of a function point or may at most be less than 10 function points or about 1,000 COBOL source code statements. Although normal counting of function points is not feasible for small updates and micro function points are still experimental, it is possible to use the "backfiring" method or converting counts of logical source code statements to equivalent function points. For example, suppose an update requires adding 100 COBOL statements to an existing application. Since it usually takes about 105 COBOL statements in the procedure and data divisions to encode 1 function point, it can be stated that this small maintenance project is "about 1 function point in size."

If the project takes one work day consisting of 6 hours, then at least the results can be expressed using common metrics. In this case, the results would be roughly "6 staff hours per function point." If the reciprocal metric "function points per staff month" is used, and there are 20 working days in the month, then the results would be "20 function points per staff month."

All software projects should measure certain key factors that are known to affect costs of aging applications. These include

- Size of all legacy applications, preferably in function points
- Volume of new features added, preferably in function points
- Maintenance staffing and maintenance assignment scopes
- Numbers of customers or users of the software
- Code structure using cyclomatic and essential complexity
- Rate at which code structure decays over time
- Numbers of bugs reported and fixed per calendar period
- Bugs reported against "error prone modules"
- Customer satisfaction, measured annually
- Maintenance team morale, measured annually

There are other maintenance measures as well, but the ones listed above should be standard for all legacy applications.

Failure to Use Metrics for Establishing Goals and Targets

As a management consultant to more than a hundred commercial enterprises and government agencies, the author frequently meets executives, managers, and software technical staffs who sincerely want to improve their software methods, tools, and approaches. Often to encourage progress, senior executives are willing to set goals or targets for the organization.

The earliest effective target observed by the author occurred more than 35 years ago. In September of 1973 one of IBM's major software divisions set a target for executives and managers of reducing incoming user-reported software bugs by 80 percent. The division average for incoming user-reported bugs in 1973 was about 3 bugs per KLOC per year (equivalent to about 0.375 bugs per function point).

What triggered this aggressive target were customer complaints and threats of lawsuits made at an annual IBM stockholders' meeting. After the meeting the IBM Chairman issued a strong directive to improve quality and do it quickly.

The three-year target was to lower the average value down to 0.6 bugs per KLOC per year (equivalent to about 0.075 bugs per function point.) This target was achieved in about three and a half years, and the social impact on the division was quite remarkable. The deployment of formal design and code inspections, which had just been developed, was the key to the reduction. Improved testing contributed also, but inspections were the primary factor.

When IBM's quality measurement program and user satisfaction measurement program started to show tangible improvements, the morale of both managers and technical staff workers was visibly bolstered, and a sense of pride and accomplishment became obvious. Obviously customer satisfaction improved as well.

There is a real art to setting goals or targets. The goals should obviously be much better than current norms, but they should actually be achievable. (To leave software for a moment, it might be reasonable to set a target for a track star to "run a mile in less than 3 minutes and 40 seconds." However, to set a target to "run a mile in less than 1 minute" would be ridiculous.) Also, the goals should be set for managers and executives, who are authorized to spend money and make changes happen. It is of little value to set goals for individual contributors, since they have only limited abilities to acquire improved methods, tools, or education on their own.

Unfortunately, some of the more common goals encountered even within leading companies tend to be abstract and ambiguous. Such ambiguous goals include

- Improve productivity by 10 to 1 within five years.
- Improve quality by 10 to 1 within five years.

There are two problems with such ambiguous goals. The most obvious problem is that the goals are beyond the actual state of the art so neither one could be achieved. This makes the person who set the goal look stupid. Also, unachievable goals do not lead to any improvements at all. The second problem is more subtle. The starting point for the goal is undefined. If your starting point is not quantified, then there is no way to judge progress. It is obvious that such goals were set by people who are well-intentioned, but who lacked any real depth of knowledge about measurement, metrics, or how to judge rates of improvement.

It is appropriate to close with sets of numeric, quantitative goals that are known to be achievable and that have been derived from the performance figures of the leading 10 percent of U.S. software producing enterprises. Any enterprise that can achieve these targets can be justifiably proud of its accomplishments.

Software Quality and User-Satisfaction Goals

- Achieve defect potentials of less than 2.5 defects per function point. This is a reduction of 50 percent from U.S. averages of 5.0 defects per function point.
  (Sum of defects in requirements, design, code, documents, and bad fixes)

- Achieve bad-fix injection rates of less than 3.5 percent.
  This is a reduction of 50 percent from U.S. averages of 7 percent bad-fix injections.
  (Sum of defects in requirements, design, code, documents, and bad fixes)
- Achieve an average cumulative defect removal efficiency higher than 95 percent.
  This is an increase of 10 percent from U.S. averages of 85 percent.
  (All defects found during development compared to first year's usage)
- Achieve less than 0.125 delivered defects per function point.
  The current U.S. average is 0.75 delivered defects per function point.
  (Measured against valid, unique defect reports)
- Achieve 90 percent "excellent" ratings from user-satisfaction surveys.
  (Measured on topics of functionality, quality, and customer support)
- Allow zero error-prone modules in released software.
  (Modules that receive disproportionate quantities of defect reports)
- Strive to improve software quality levels by less than 40 percent per year.
  (Baseline is current year's volume of customer reported defects)

Development Cycle and Time to Market Goals

- Less than 6-month development cycles for software less than 500 function points in size.
- Less than 12-month development cycles for software less than 1,000 function points in size.
- Less than 24-month development cycles for software less than 5,000 function points in size.
- Less than 36-month development cycles for software less than 10,000 function points in size.
- Less than 54-month development cycles for software less than 100,000 function points in size.
- Raising function points to the 0.4 power gives average schedule in calendar months. Try and achieve an exponent of only 0.34 power for development schedules.
- Strive to reduce current cycle times by greater than 15 percent per year. (Baseline is current year's average cycle time for each class and size of software)

Productivity Goals for New Development Projects

- Average greater than 25 function points per staff month on projects up to 500 function points. (Including all activities: requirements, design, code, testing, documents, management)
- Average greater than 15 function points per staff month from 500 to 1,500 function points. (Including all activities: requirements, design, code, testing, documents, management)
- Average greater than 12 function points per staff month on projects from 1,500 to 5,000 function points. (Including all activities: requirements, design, code, testing, documents, management)
- Average greater than 6 function points per staff month on projects from 5,000 to 20,000 function points. (Including all activities: requirements, design, code, testing, documents, management)
- Average greater than 4.50 function points per staff month greater than 20,000 function points. (Total function points delivered divided by total software employees × 12)
- Strive to improve new development productivity levels by greater than 25 percent per year (Baseline is current year's average productivity rate.)

Productivity Goals for Enhancement Projects

- Average greater than 30 function points per staff month on minor enhancements
  (Enhancements ranging from 1 to 50 function points in size)
- Average greater than 20 function points per staff month on medium enhancements
  (Enhancements ranging from 50 up to 250 function points in size)
- Average greater than 15 function points per staff month on large enhancements
  (Enhancements larger than 250 up to 1,000 function points in size)
- Average greater than 8 function points per staff month on major releases
  (Enhancements larger than 1,000 function points in size)
- Use complexity analysis tools on 100 percent of aging software.
  (Software in production for more than three years)
- Use restructuring tools on 100 percent of aging COBOL, C, or other languages where such tools are supported

- Strive to improve enhancement productivity levels by greater than 25 percent per year
  (Baseline is current year's average rate for enhancement productivity)

Productivity Goals for Migration and Conversion Projects

- Average greater than 50 function points per staff month for migration to a new platform
- Average greater than 100 function points per staff month for conversion to a new database
- Average greater than 100 function points per staff month for conversion to a new language

Productivity Goals for Maintenance Projects

- Average greater than 1,500 function points in maintenance assignment scope
  (Total portfolio size divided by number of people working on maintenance)
- Strive to improve the maintenance assignment scope by greater than 25 percent per year
  (Baseline is current year's average value)
- Provide essentially zero wait time for customers reporting a bug or defect
  (Online defect recording is necessary to achieve this target)
- Average less than 24 hours in repairing severity 1 (high severity) defects
- Average less than 5 days in repairing severity 2 defects
- Average less than 15 days in repairing severity 3 defects
- Average less than 60 minutes in providing temporary "work-around" solutions for defects
  (Online defect recording is necessary to achieve this target)
- Average less than 1.0 minutes from customer dial-in to service representative contact
- Achieve a bad fix rate of less than 2.5 percent
  (Secondary defects inserted as a byproduct of repairing previous defects)

Quantified Goals for Sociological and Morale Factors

- Provide at least 10 days of annual training for technical staff members
- Provide at least 15 days of training for software managers

- Provide at least 5 days of training for software and corporate executives
- Provide at least 90 square feet of quiet office space for software staff members
- Achieve 90 percent "good" or "excellent" responses on annual morale and opinion surveys

Quantified Goals for Reusability Technologies

- Achieve levels of greater than 50 percent reuse for project plans and estimates
- Achieve levels of greater than 50 percent reuse for design and specifications
- Achieve levels of greater than 75 percent reuse for source code in every application
- Achieve levels of greater than 75 percent reuse for test cases and test scripts
- Achieve levels of greater than 85 percent reuse for data and information

Quantified Goals for Management Technologies

- Use automated defect tracking tools on 100 percent of software projects
- Use automated estimating tools on 100 percent of software projects greater than 500 function points
- Use automated planning tools on 100 percent of software projects greater than 500 function points
- Measure 100 percent of completed projects greater than 500 function points

Summary and Conclusions

Although software is approaching 60 years of age as an industry, it is still young compared to chemistry, physics, medicine, electrical engineering, and other scientific and engineering disciplines.

The subjectivity and ambiguity of software measurement and metrics are troublesome today, but metrics problems are troublesome in every young science. A hundred years ago the basic metrics of electronics were troublesome, and a hundred years before that the basic metrics of chemical reactions were troublesome.

The long-range prognosis for software measurement and metrics is guardedly favorable. Some improvements have been seen since the first edition of this book in 1991 and today with the publication of the third edition in 2008. Certainly function point metrics have continued to expand at a rapid rate. Within perhaps another 10 years it is possible, if not likely, that the problems discussed in this third edition will have been resolved. If so, then software engineering can take its place as a true engineering discipline.

Unless the critical measurement problems are resolved, then software engineering will, in fact, be an oxymoron. Without accurate metrics and effective measurement practices, programming will neither be able to advance beyond the level of a craft, nor take its place in the ranks of true professions.

Suggested Readings

It is interesting that most of the suggested readings are not software books or project management books. They are books dealing with the evolution of older scientific disciplines. Starr, Paul, *The Social Transformation of American Medicine*, New York, NY: Basic Books, Perseus Books, 1984. This thought-provoking book won the Pulitzer Prize in 1984. Medicine in 2008 is the most prestigious of any profession. Yet this book points out that only a little more than 100 years ago receiving a medical degree required only two years in medical school. Neither a college degree nor even a high-school diploma were prerequisites. There were no internships or residencies. Less than 10 percent of physicians were college graduates. There were no medical boards or recognized specialties. Quacks and trained physicians were competing for patients. Drugs and pharmaceuticals were uncontrolled and some were harmful. Many medical students and some physicians never even saw the inside of a hospital. The hospitals themselves were unsanitary and hazardous to patients and staff alike. This book shows the steps taken by the American Medical Association (AMA) to transform medicine from an occupation that was even less professional than software is today into the most prestigious and powerful profession in history. There are many lessons in this book that we in software should be learning.

Kuhn, Thomas, *The Structure of Scientific Revolutions*, 3rd Ed., Chicago: University of Chicago Press, 1996. Thomas Kuhn's book has been listed among the 100 most influential books of the past 50 years. It tackles the interesting question of why scientific revolutions occur. It is interesting that a majority of scientific breakthroughs were initially resisted and opposed, sometimes very strongly. Examples of scientific advances that were initially opposed include the Copernican worldview, sterile surgical procedures, continental drift, evolution, and scores of

others. Not only scientific advances but also military and mechanical advances have been rejected. Naval screw propellers, iron clad ships, self-leveling naval canon, rifles, and tanks were all initially rejected by military authorities when first presented. What seems to happen is that cognitive dissonance causes many scholars to dismiss evidence that opposes the "standard" views of the time. As evidence mounts, resistance increases until suddenly the evidence becomes overwhelming and there is an abrupt transition to a new way of thinking.

Joseph, Christopher, *A Measure of Everything–An Illustrated Guide to the Science of Measurement,* Buffalo, NY: Firefly Books, 2005. This interesting book is an excellent illustration of the importance of measurement to both science and to our daily lives. The book discusses more than 1,000 measurements from physics, biology, economics, construction, medicine, agriculture, and scores of other disciplines. Simply scanning the pages and reading information about various metrics is a good way to understand how important metrics are to progress in every field.

Asimov, Isaac, *Biographical Encyclopedia of Science and Technology,* New York: Equinox Books, 1972. Isaac Asimov is a famous and prolific science fiction author. But he has also written a number of excellent scientific books. This compendium gives short biographies of 1,125 scientists and scholars from ancient Greece and Egypt through the modern era. It is interesting that several hundred of these scientists have metrics named after them and almost half were pioneers in metrics and measurement. Science cannot progress without measurement.

Gribben, John, *The Scientists,* New York: Random House, 2002. This book is similar to Asimov's in theme, but it has more extensive biographical information including descriptions of some of the actual experiments and measurements made by the scientists. Here, too, many of the scientists are memorialized by having metrics named after them (ohm, volt, ampere, joule, henry, and many other metrics are all derived from the names of the scientists who explored their properties). This book illustrates the close connection between accurate measurement and scientific progress. Not all of the measures are accurate. For example the first attempt to measure the speed of light was off by more than 20 percent. But before those early measurements were taken, many scholars thought the speed of light was infinite. Thus measurements and progress are closely intertwined.

Beard, Miriam, *A History of Business,* Ann Arbor: University of Michigan Press, 1962. There are not very many histories of business compared to histories of warfare or histories of science. This book runs from Babylon through the modern era, with very thoughtful analyses of Roman business practices, medieval business practices, Viking business practices, and of course, the business practices of the Renaissance and the Industrial Revolution. However, the book is included here

primarily because of the close and vital relationship between business and measurement. Obviously accurate financial measures have been important for business transactions since the earliest history of Babylon and Sumer. The book discusses many advances in accounting, measurements of weights, measurement of land areas, and measurement of production that originated due to business needs.

Additional References on Software Measurements

Albrecht, A. J. "Measuring Application Development Productivity." *Proceedings of the Joint IBM/SHARE/GUIDE Application Development Symposium.* October 1979. pp. 83–92.

Albrecht, A.J. "AD/M Productivity Measurement and Estimation Validation." IBM Corporate Information Systems. Purchase, NY: IBM Corporation, May 1984.

Asimov, Isaac. *Biographical Encyclopedia of Science and Technology*. New York: Avon Books, 1972.

Bakan, Joel. *The Corporation: The Pathological Pursuit of Profit and Power*: The Free Press, March 2004.

Boehm, Barry W. *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.

Bollinger, Terry B. and Clem McGowan. "A Look at SEI Software Process Assessments." *IEEE Software* (April 1991).

Cohen, I. Bernard. *Revolution in Science*. Cambridge, MA: Harvard University Press, 1985.

Crosby, Philip B. *Quality is Free*; New American Library, New York, NY: Mentor Books, 1979.

DeMarco, Tom. "Developing a Quantifiable Definition of Bang," in *Controlling Software Projects*. New York: Yourdon Press, 1982: pp. 92–110.

Draft Standard for Software Productivity Metrics, P1045/D3.1. IEEE Software Productivity Metrics Working Group. December 1990.

Dreger, J. Brian. *Function Point Analysis*. Englewood Cliffs, NJ: Prentice Hall, 1989.

Dunn, Robert and Richard Ullman. *Quality Assurance for Computer Software*. New York: McGraw-Hill, 1982.

Dunn, Robert and Richard Ullman. *Software Defect Removal*. New York: McGraw-Hill, 1984.

Ecora Corporation. *Practical Guide to Sarbanes-Oxley IT Internal Controls*. Portsmouth, NH; Ecora, 2005 (www.ecora.com).

Garmus, David & Herron, David. *Function Point Analysis*. Boston, MA: Addison Wesley Longman, 2001.

Garmus, David & Herron, David. *Measuring the Software Process: A Practical Guide to Functional Measurement*. Englewood Cliffs, NJ: Prentice Hall, 1995.

Garmus, D. (ed.). *IFPUG Counting Practices Manual*. Release 3.0. Westerville, OH: International Function Point Users Group, April 1990.

Grady, Robert B. and Caswell, Deborah L. *Software Metrics: Establishing a Company Wide Program*. Prentice Hall, Inc., 1987.

Grady, Robert B. and Deborah L. Caswell. *Software Metrics: Establishing a Company-Wide Program*. Englewood Cliffs, NJ: Prentice Hall, 1987.

Grady, Robert B. *Effective Software Measurements*. Englewood Cliffs, NJ: Prentice Hall, 1992.

Howard, Alan (Ed.) *Software Metrics and Project Management Tools; Applied Computer Research (ACR)*. Phoenix, AZ, 1997.

Humphrey, Watts. *Managing the Software Process*. Reading, MA: Addison Wesley, 1989.

Information Technology Infrastructure Library, www.wikipedia.org, (accessed 2007).
International Function Point Users Group; IT Measurement; Boston, MA: Addison Wesley Longman, 2002.
International Standards Organization (ISO). Quality Management and Quality Assurance Standards - Part 3: Guidelines for the Application of 9001 to the Development, Supply, and Maintenance of Software.
Jones, Capers. *Estimating Software Costs*. 2nd edition, New York, NY: McGraw-Hill, 2007.
Jones, Capers. *"Sizing Up Software"; Scientific American*, New York, NY, December 1998, vol. 279 no. 6: pp. 104–109.
Jones, Capers. *Software Assessments, Benchmarks, and Best Practices*; Boston, MA: Addison Wesley Longman, 2000.
Jones, Capers. *Conflict and Litigation Between Software Clients and Developers; Software Productivity Research*, Burlington, MA: 2003.
Jones, Capers. "Measuring Programming Quality and Productivity." *IBM Systems Journal,* vol. 17, no. 1 (1978): pp. 39–63. (Reprinted in *Programming Productivity—Issues for the Eighties*. Washington DC: IEEE Press, 1986.
Jones, Capers. "The 1985 Software Productivity Research Function Point Methods" in *Applied Software Measurement*." New York: McGraw-Hill Publishing Company, 1991, pp. 69–78.
Jones, Capers. *Programming Productivity*. New York: McGraw-Hill Publishing Company; 1986.
Jones, Capers. *Table of Programming Languages and Levels*. Version 10.0. Burlington, MA: Software Productivity Research, Inc, May, 2005.
Jones, Capers. *The Software Productivity Research (SPR) Assessment Method*. Burlington, MA: Software Productivity Research Inc., May 1991.
Kan, Stephen H. *Metrics and Models in Software Quality Engineering*, 2nd edition; Boston, MA: Addison Wesley Longman, 2003.
Kaplan, Robert S. and Norton, David B. *The Balanced Scorecard*. Cambridge, MA: Harvard University Press, 1996.
Kaplan, Robert S. and Norton, David B. *Strategy Maps: Converting Intangible Assets into Tangible Outcomes*. Boston, MA: Harvard Business School Press, 2004.
Kemerer, C. F. "Reliability of Function Point Measurement: A Field Experiment." MIT Sloan School Working Paper 3193-90-MSA. January 1991.
Kuhn, Thomas S. "The Structure of Scientific Revolutions" in *International Encyclopedia of Unified Sciences*. Chicago: The University of Chicago Press, 1970.
Mason, Stephen F. *A History of the Sciences*. New York: Collier Books, Macmillan Publishing Company, 1962.
McCabe, Tom. "A Software Complexity Measure." *IEEE Transactions on Software Engineering*, vol. 2 (December 1976): pp. 308–320.
Miller, Sharon E. and George T. Tucker. *Software Development Process Benchmarking*. IEEE Communications Society Reprint. CH2980 1/910000-0153. New York: IEEE Press, 1991.
Pinis, Mark (ed.). *CHECKPOINT User's Guide*. Burlington, MA: Software Productivity Research, Inc., June 1992.
Pohlen, Terrance L. *Supply Chain Metrics; International Journal of Logistics Management*; vol 2, no. 1; 2001: pp. 1–20.
Putnam, Lawrence H. *Measures for Excellence—Reliable Software On Time, Within Budget*. Englewood Cliffs, NJ: Yourdon Press - Prentice Hall, 1992.
Putnam, Lawrence H and Myers, Ware. *Industrial Strength Software—Effective Management Using Measurement*; Los Alamitos, CA: IEEE Press, 1997.
Software Metrics Definition Working Group. *Software Size Measurement with Application to Source Statement Counting*. Pittsburgh, PA: Software Engineering Institute, August 1991.
Software Productivity Research (SPR), Inc. CHECKPOINT (TM) Questionnaire. Version 1.2. Burlington, MA: Software Productivity Research, 1989.

Software Productivity Research, Inc. "Rules for Counting Procedural Source Code." Burlington, MA: Software Productivity Research, 1990. Reprinted in *Applied Software Measurement*. New York: McGraw-Hill Publishing Company, 1991: pp. 309–316.

Starr, Paul. *The Social Transformation of American Medicine*. New York: Basic Books Inc., 1982.

Symons, Charles R. "Function Point Analysis—Difficulties and Improvements." *IEEE Transactions on Software Engineering,* vol. 14, no. 1 (January 1988): pp. 2–11.

Symons, Charles R. *Software Sizing and Estimating: MK II FPA (Function Point Analysis)*. Chichester, West Sussex. England: John Wiley & Sons Ltd., 1989.

Appendix

Rules for Counting Procedural Source Code

As this book is written in 2008, there are more than 700 programming languages in existence. Since the author first began to collect information on programming languages in 1985, new programming languages have been introduced at a rate of more than one per month.

There are no apparent technical reasons to explain why the software industry needs to have 700 programming languages. Many of these languages were created, experienced a few brief years of popularity, and then were no longer used for software development. In fact, the average age of a large software application is older than the average age of a programming language. As a result, thousands of working applications were written in "dead" programming languages that are now used only for maintenance of aging legacy applications. Examples of some of these dead languages include Coral, Chill, Mumps, and Bliss.

This means there are still working applications written in many old and obscure languages but very few trained programmers who know these languages and sometimes no working compilers in existence. Some of these older applications operate only on obsolete hardware platforms or run under obsolete operating systems such as DOS or early IBM operating systems.

In addition to the plethora of languages created by the software engineering industry, there is also the curious phenomenon that many of these languages are so specialized they can be used only for a narrow range of purposes, such as querying databases. As a result, a majority of large applications in the 10,000 function point size range utilize multiple programming languages, such as COBOL and SQL or Java and HTML. The author has noted applications containing a dozen languages, and there are reports of some applications containing 20 or more programming languages.

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

The reason for the plethora of languages dates back to the early days of computing. Some of the first "high-level" languages were created for specialized purposes. For example, FORTRAN stands for "formula translator" whereas COBOL stands for "common business-oriented language." From these names, it can be seen that FORTRAN was intended for mathematical applications and COBOL was intended for business applications. Many other specialized niche languages soon followed. LISP was created for list processing and APL was created for mathematical analysis. SQL was created for querying databases and HTML was created for putting documents on the Web.

Although the plethora of languages soon became troublesome for maintenance work and made recruiting programmers difficult, attempts to create more robust languages that could be used for wider ranges of software were generally unsuccessful. IBM's attempt to replace both COBOL and FORTRAN with PL/I did not succeed. The attempt by the Department of Defense to replace Jovial, FORTRAN, COBOL, and many other languages with Ada did not succeed. These attempts did create powerful and useful languages, but they did not have the intended effect of eliminating the other languages. As a result, several more new languages were added to the mix already in existence.

The economic costs of having 700 or so programming languages are rather high. The author estimates that the world's maintenance software workforce is about 25 percent larger than it would have to be if there were only a few programming languages. This has led to the creation of companies and tools that attempt to convert older languages into new languages, such as turning COBOL into Java or Powerbuilder into Java. Unfortunately, if the old code is poorly structured and error-prone, the modern translations won't be particularly elegant and may even include some of the error-prone modules carried over from the older versions. Some form of renovation should be performed before jumping into code conversion.

With "lines of code" having been a common metric for measuring productivity and quality since the software industry began, one would think that language developers themselves would have provided suggested counting rules as a by-product of developing a new language. However, most language developers did not regard counting as a topic they were responsible for, so counting rules usually arrive long after a language has been developed, if at all. For at least 50 of the 700 or so programming languages, no satisfactory counting rules have been developed. An example of this lack is Visual Basic, where some "programming" is done with buttons and pull-down menus rather than with actual source code. As a result, even in 2008 there are no satisfactory counting rules for what was one of the most widely used languages in history.

It is quite astonishing that in 60 years of software engineering and programming history, there has never been a true international standard that defined exactly what is meant by "a line of source code." The following rules for counting source code were created by Software Productivity Research (SPR). They are intended to provide at least minimal consistency with clients when discussing productivity issues in terms of "lines of source code."

Indeed, the rules have been embodied in a size conversion tool that can convert apparent size by using any arbitrary combination of rules into the equivalent size when some other combination is used. Such a tool is more or less necessary when an attempt is made to compare productivity between companies, departments, and even projects by using lines of source code.

The general principle behind the following rules can be stated briefly as "Count as you think." Counting should come as close as possible to being intuitive and natural, and it should avoid being artificial and stilted. In the rules, the SPR standard is indicated by an asterisk, i.e., *. The SPR standard is not necessarily the optimum choice, but in general, it seems to be pragmatic and make sense.

* = SPR standard counting technique

Project Source Code Counting Rules

Multiple-language counting

*1. Each language is counted separately.
2. All languages are lumped and counted together.

Source code termination

*1. Delimiters
2. Physical lines

Source code counting

1. Executable statements only
*2. Executable statements + data definitions
3. Executable statements + data definitions + comments

Macro instructions and expansion counting

0. Project does not have macro instructions or expansions.
1. Macro instructions and expansions are not counted.

2. Macro instructions are counted, but not expansions.
*3. Macro instructions and unique expansions are counted.
4. Macro instructions and all expansions counted separately.
5. Macro instructions and all expansions lumped with product.

Reusable code counting

0. Project does not have reusable code.
1. Reusable modules are not counted.
*2. Unique reusable modules are counted separately.
3. All uses of reusable modules are counted separately.
4. Reused modules are lumped and counted with product code.

Program and application generator counting

0. Project does not have generator statements.
1. Generator statements are not counted.
2. Generator input statements are counted.
*3. Generator inputs and outputs are counted separately.
4. Generator inputs and outputs are lumped together.

Job control language counting

0. Project does not have job control language.
1. Job control language is not counted.
*2. Job control language is counted separately.
3. Job control language is lumped and counted with product.

Changed code counting

0. Project does not have changed code.
1. Changed code is not counted.
*2. Changed code is counted separately from new code.
3. Changed code is lumped and counted with new code.

Base code counting

0. Product does not have base code.
1. Base code is not counted.
*2. Base code is counted separately.
3. Base code is lumped and counted with new and changed code.

Deleted code counting

0. Project does not have deleted code.
1. Deleted code is not counted.

*2. Deleted code is counted separately.

3. Deleted code is lumped with new and changed code.

Scaffold code counting

0. Project does not have scaffold code.
1. Scaffold code is not counted.

*2. Scaffold code is counted separately from product code.

3. Scaffold code is lumped and counted with product code.

Support code counting

0. Project does not have support code.
1. Support code is not counted.

*2. Support code is counted separately from product code.

3. Support code is lumped and counted with product code.

Test case code counting

0. Project does not have test case code.
1. Test case code is not counted.

*2. Test case code is counted separately from product code.

3. Test case code is lumped and counted with product code.

General Rules for Counting Code Within Applications

- Do not count commentary lines or information appended to executable lines for informational purposes.
- Count verbs or action statements, such as assignments, print commands, conditionals, and loops.
- Count equations and mathematical expressions.
- Count data definitions, including both variables and constants, and also type definitions.
- Count initialization of data definitions.
- Count procedure definitions.
- Count each formal parameter within a procedure.

- Count procedure labels.
- Count unexpanded macro calls if you are interested in development size.
- Count expanded macro instructions if you are interested in delivered size.
- Clearly identify and explain the specific counting rules you have selected.
- Be aware that individual human variation can and will cause notable differences in the sizes of programs that are logically identical. In a controlled study, eight programmers varied by 5 to 1 in the number of statements they used to implement the same algorithm.

Examples of the SPR Source Code Counting Rules

- Do not count commentary lines or statements that are used purely for information purposes; i.e., the following statement would not be counted:

  ```
  REM THIS ROUTINE CALCULATES GROSS PAY
  ```

- For strongly typed languages, count data definition statements as one statement; i.e., the following example counts as one statement:

  ```
  WEEKDAY = (MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
  SATURDAY, SUNDAY)
  ```

- For weakly typed languages, count data definition statements as one statement; i.e., the following example counts as one statement:

  ```
  DATA = MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
  SATURDAY, SUNDAY
  ```

- Count assignment statements as one statement; i.e.,

  ```
  A = B + C
  ```

- Count equations as you would treat them in mathematics, i.e., as a single unit or line. The following expression counts as one statement:

  ```
  A = ((B * C)/(D - E)) + ((F + G + H) * I))
  ```

- For languages that allow multiple logical statements per physical line, count the logical statements. (This is the same as counting delimiters.) The following example counts as three statements:

  ```
  BASE = 0: BASE = HOURS * RATE: PRINT BASE
  ```

  Note: The two colons and the carriage return at the end of the line are the three delimiters.

- Count procedure or function calls as one statement, i.e.,

```
CALL PRINTDRV
```

- Count IFs and THENs as separate statements; i.e., the following example would count as two statements:

```
IF A OCCURS
THEN UPDATE B
```

- Count IFs followed by GOTOs as separate statements; i.e., the following example would count as two statements:

```
IF A OCCURS THEN GOTO NEWSTART
```

- Count IFs, THENs, and ELSEs as separate statements; i.e., the following example would count as three statements:

```
IF A OCCURS
THEN UPDATE B
ELSE UPDATE C
```

- Count CASE statements as separate statements; i.e., the following example would count as five statements:

```
SELECT CASE NUMBER$
 CASE "ONE": PRINT "1"
 CASE "TWO": PRINT "2"
 CASE "THREE": PRINT "3"
 CASE "FOUR": PRINT "4"
END SELECT
```

- Count DO-WHILE constructions as one statement; i.e., the following example would count as four statements:

```
WHILE J >10 DO
 PRINT HEADER
 PRINT LINENUM
 PRINT MESSAGE
```

- Count FOR-NEXT constructions as one statement; i.e., the following example would count as four statements:

```
FOR J = 1 TO 10
 PRINT HEADER
 PRINT LINENUM
 PRINT MESSAGE
NEXT J
```

- Count REPEAT-UNTIL constructions as one statement; i.e., the following example would count as four statements:

```
REPEAT
 PRINT HEADER
 PRINT LINENUM
 PRINT MESSAGE
UNTIL INDEX >10
```

- Count nested constructions in accordance with the individual element rules; i.e., the following example would count as eight statements:

```
FOR J = 1 TO 10
 PRINT HEADER
  FOR K = 1 TO 3
   PRINT NUMBER
   PRINT STREET
   PRINT CITY
  NEXT K
 PRINT LINENUM
 PRINT MESSAGE
NEXT J
```

Software Productivity Research COBOL-Counting Rules

Although COBOL was the most widely used language in the world and approximately 50 percent of all software was written using it during the 1970's, 1980's, and part of the 1990's, there are currently no national or international standards that actually define exactly what is meant by "a line of COBOL source code" for the purposes of measuring productivity.

The most basic problem with COBOL counting is the fundamental decision whether to count physical lines or logical lines. Among SPR clients, about 75 percent count physical lines, which is definitely the most convenient method since many library tools and COBOL compilers themselves can provide such data.

However, since the actual work required to produce an application correlates rather poorly with physical lines, it is often more useful for productivity studies to count logical lines. Because COBOL tends to have conditional statements that span several physical lines, a count of logical lines can reduce the apparent size of the application by more than 2 to 1. For productivity purposes, this is very troublesome—especially in light of the fact that few productivity authors actually state which rules they used for line counting!

If you do wish to count logical lines, you face two problems: (1) What are the exact rules that define logical lines? (2) How can logical lines be counted other than by a laborious manual analysis? It is remarkable that, after so many years of COBOL's history, these problems remain. The answer to both questions is the same: It is necessary to build or acquire a line-counting tool that embodies your corporate standards (since there are no global standards). Several COBOL-line-counting tools are commercially available, and they can be set to match your corporate rules.

The following rules by Software Productivity Research are a surrogate for a true standard, and they should be used with caution. *Note:* The COBOL rules are based on the general rules for procedural languages given in the preceding section.

As stated before, try to count as you think: If a statement represents a unit of thought, it should also represent a unit of work and hence deserves to be counted.

1. Statements within the IDENTIFICATION DIVISION are not counted by SPR, since they are essentially comments. If you choose to count them, suggested rules are as follows:
   - PROGRAM ID and the ID itself count as one line.
   - AUTHOR and the author's name count as one line.
   - INSTALLATION and following text count as one line.
   - DATE WRITTEN and following text count as one line.
   - DATE COMPILED and following text count as one line.
   - SECURITY and following text count as one line.
   - COMMENTS and following text do not count, since commentary lines are normally excluded.
2. Statements within the ENVIRONMENT DIVISION are similar to comments in some ways, but because they are volatile and change when applications migrate, it is appropriate to count the FILE CONTROL SELECT statements.
3. Count statements within the DATA DIVISION in accordance with these rules:
   - Each statement in the FILE section counts as one line, e.g., BLOCK, RECORD, and VALUE OF.
   - COPY statements themselves count as one line each. The code that is actually copied from a source statement library is reused code and should be counted as such. If you are interested in development productivity, it can be ignored. If you are interested in delivery productivity, it should be counted.
   - Each numbered FIELD or SUBFIELD statement (01, 02, 66, etc.), including the PICTURE clause, counts as one line.
   - Statements in the WORKING STORAGE section follow the same rules as the FILE section.
4. Count statements within the PROCEDURE DIVISION in accordance with these rules:
   - Count procedure labels as one line; i.e., MAIN-ROUTINE of END-OF-JOB routine would count as one line each.
   - Count verbal expressions as one line; i.e., OPEN, WRITE, CLOSE, MOVE, ADD, etc., count as one line each.
   - Count CALL statements as one line; i.e., CALL 'GRCALC' by USING CR-HOURS, CR-RATE, WS-GROSS counts as one line.

- Count PERFORM statements as one line; i.e., PERFORM RATE-LOOKUP THROUGH RATE-EXIST counts as one line.
- Count IF logic as separate statements; i.e., the following expression counts as three lines:

```
IF HOURS IS GREATER THAN 40
  SUBTRACT 40 FROM HOURS GIVING OVERTIME
  MULTIPLY OVERTIME BY 1.5 GIVING PREMIUM
```

- Count IF-ELSE-GO TO logic as separate statements; i.e., the following expression counts as three lines:

```
IF LINE-COUNT = 50 OR LINE-COUNT >50 GOTO PRINT-HEADINGS
  ELSE GO TO PRINT-HEADINGS-EXIT
```

This concludes the COBOL example. In 2008 COBOL has declined as a development language but continues to be the major language for maintenance of aging legacy applications.

With new languages popping up at a rate of more than one per month and more than 700 languages in the total inventory, it is not possible to do more than provide general principles for counting lines of code.

Also, for many important languages such as Visual Basic, there are no effective counting rules because much of the "programming" is done using buttons or pull-down menus rather than using procedural code.

The plethora of languages, the utilization of more than one language in large applications, and the lack of effective counting rules are yet more reasons for function point metrics to become the industry standard for measuring productivity and quality.

Index

A
abandonment of successful software processes, 188
abeyant defects, 491–492
Abran, Alain, 125
academic institutions, 44
activities
  ambiguity in defining, 561–564
  checklist of common software activities, 566
  overlapping, 573
activities performed, variations in, 256–258
activity-based benchmarks, 360–361
activity-based cost measures, 33, 525–526
  example, 46–47
activity-based costing, 263–265
activity-based schedule measures, 33, 525
Ada, 583–585
advertising, false, 564–566
age of software applications, 14–15
Agile development, 342
Agile manifesto, 227
Agile methods, 12
  emergence of, 439
Agile project metrics, 173–176
Agile software, development of, 226–229
Agile story points, 79
aging non-COBOL production libraries, 376
airlines manufacturers, 41
Albrecht, A.J., 11, 58, 73, 74, 118, 551, 594
algorithmic complexity, 416
algorithms, 119
  examples of in a software estimating program, 123
  See also feature points
Analysis of Complexity Tool (ACT), 165
application deliverable size measures, 32–33, 525
application schedules, best in class, 346
application size, variations in productivity rates and, 159–163
applied software measurement, 2–3
  costs of programs, 61–62
  and future progress, 66
  hard data, 6–9
  normalized data, 10–17
  placement and organization, 62–63
  sequence of creating a program, 63–66
  skills and staffing of teams, 62
  soft data, 9–10
  structure of, 48–53
  value of programs, 58–61

Copyright © 2008 by The McGraw-Hill Companies. Click here for terms of use.

Assembly language, 583–585
assessments, 351–355
assignment scopes, 37–38, 420
attribute variables, 403–406
attrition, 23, 516
automatic backfiring from source code, 164–165
automatic conversion, 165
automatic derivation, 79–80
  of function points from requirements and design, 164
automotive manufacturers, 41
average cost per function point, 544
averages in productivity, 268–276
averaging techniques, variations in, 223–226

B

backfiring, 80, 81, 92, 109, 596–597
  automatic backfiring from source code, 164–165
  backfiring function points, 107–112
  See also reverse backfiring
backlog measures, 50–51, 64
bad fixes, 220
balanced scorecard measures, 18, 176–177, 511–512
  for software, 26–27, 519–520
bang function points. *See* DeMarco "bang" function points
banking industry, 43
baselines, 352, 356–357
  5-point scale, 383–384
  aggregation of data and statistical analysis, 371–372
  analysis and aggregation of data, 430
  context of and follow-on activities, 377
  developing or acquiring a baseline data collection instrument, 380–383
  ethics of measurement, 365–366
  follow-on activities after completion of baseline, 373–375
  identification of problems, 375–376
  individual project analysis, 370–371
  introductory session, 369
  mechanics of a baseline study, 364–365
  methodology of baseline analysis, 366–369
  nonprofit measurement associations, 368–369
  preliminary discussions and project selection, 369–370
  preparation of annual report, 372–373
  preparation of presentation, 372
  reasons for commissioning, 365
  scheduling project interviews, 370
  scope and timing, 373
  what a baseline covers, 378–380
Basili, Victor, 27, 520, 578
Battlemap, 165
behavioral changes, 536–542
benchmarking, 229–230, 358
benchmarks, 352, 357–363, 518
  attrition and turnover, 23
  compensation and benefits, 23
  and industry measures, 22–23, 515–516
  research and development spending patterns, 23

software measures and metrics, 23–25
benefits, 23, 516
"best in class" targets, 344–350
blind benchmark studies, 361–362
Boehm, Barry, 455, 582
Bootsma, Fred, 126
burden rates, 199–201
average costs per function point with and without, 315
average staff compensation and burden rate levels, 312
business and corporate measures, 18, 511–515

C

canceled projects, 543
cancellation, risk of, 307–308
capability maturity model (CMM), 243–244, 342–343, 352–353, 527–528
achieving level 3 or higher, 539
expansion of, 445–446
levels, 35
and soft data, 10
capability maturity model integrated (CMMI), 352
expansion of, 445–446
and soft data, 10
Caswell, Deborah, 582
cautions, 543–546
change requests, 31–32, 524
changes in software technologies since 1991
beneficial changes, 186–187
harmful changes, 187–188
chart of accounts
example of, 8
selecting for resource and cost data, 167–172
standard project charts of accounts, 395–399
checklist of common software activities, 566
CHECKPOINT, 90–91, 380–381
data normalization feature, 94
China, 453
class, 388
ambiguity in defining, 552–561
clean-room development, 468
client-server applications, 231
client-server architecture, 343
client-server quality, rise and fall of, 446–449
CMM. *See* capability maturity model (CMM)
CMMI. *See* capability maturity model integrated (CMMI)
combinatorial complexity, 417
commercial off-the-shelf (COTS) software, 407–408
COTS-point metrics, 612–613
commercial software, 231–232, 251–253
measurement tools, 546–547
productivity improvement, 17
zone of commercial software projects, 281–282
commercial-grade estimation tools, 3
compensation, 23, 197–199, 516
average staff compensation and burden rate levels, 312
competition, international, 451–454
competitive measures, 20, 513
completed project measures, 65
complexity analysis, 469
complexity calculations, extension and refinement of, 165–166
complexity measurement
ambiguity in, 593–594
tools for, 615
complexity of software, 30, 415–419, 522

computational complexity, 416
Computer Aid Incorporated, 238, 302–304
Computer-Aided Software Engineering (CASE), 343
constraints, 389
contract project variables, 406–407
contract software
  offshore, 250–251
  in the U.S., 249–250
  zone of contract software projects, 279–281
corporate measurements, 17
  balanced scorecard measures, 18, 511–512
  business and corporate measures, 18
  competitive measures, 20, 513
  financial measures, 18–19, 512
  human resources measures, 20, 513
  manufacturing measures, 21, 514
  market share measures, 20, 513
  outsource measures, 21, 514
  research and development (R&D) measures, 20–21, 514
  return on investment (ROI), 19, 512–513
  Sarbanes-Oxley (SOX) measures, 18, 511
  shareholder measures, 19–20, 513
  supply chain measures, 21–22, 514–515
  warranty and quality measures, 22, 515
COSMIC function points, 112–113, 161
cost ambiguity, 197–201
cost errors in data, 192–206
cost of quality, 486–487
cost of quality control and defect repairs, 30, 523
cost per defect, 544–545
  ambiguity in, 600–601
  paradox of, 483–485
cost per line of code, 545–546
costs
  average costs per function point with and without burden rates, 315
  average development cost per function point, 313
  average software development costs, 314
  average software development labor costs, 314
  averages for software costs, 312–316
  of counting function points, 78–81
  tools for measuring, 615
creeping requirements, ranges in, 292–294
Crosby, Phil, 455, 457, 486
customer satisfaction, 28, 521, 530–531
customer-reported defects, measuring, 488–491
cyclomatic complexity, 301–302, 417

D

data collection instruments
  designing, 367–368
  developing or acquiring a baseline data collection instrument, 380–383
data collection questionnaire, 383–385
  base code factors for maintenance and enhancement projects, 425–428

complexity and source code factors, 415–419
contract project variables, 406–407
delta or changed code factors for maintenance or enhancement projects, 428–429
descriptive materials and comments, 429
determining size of project to be measured, 392–395
identifying occupation groups and staff specialists, 399–401
maintenance and enhancement project variables, 412–414
management project and personnel variables, 402–403
measuring hard data for software projects, 419–422
measuring project, activity and task schedules, 422–424
project attribute variables, 403–406
project goals and constraints, 389
project natures, scopes, classes and types, 386–388
project risk and value factors, 414–415
recording the basic project identity, 385–386
reusable design and code factors, 424–425
software defect removal methods, 408–410
software documentation variables, 410–412
software package acquisition, 407–408
staff availability and work-habit factors, 389–392
staff skill and personnel variables, 401–402
standard project charts of accounts, 395–399
data complexity, 416–417
data confidentiality, sociology of, 54–55
data files, 136
data metrics, emergence of, 444–445
data point metrics, need for, 611
Data Processing Management Association (DPMA), 369
data quality, emergence of, 444–445
data representation, variations in, 223–226
database measures, 542
David Consulting Group, 71, 124
dead languages, 212–213
defect, defined, 473–475
defect origin, by industry, 259
defect potential, 220, 433–437
averages for, 316–317
best in class, 347
failure to measure, 602–603
variations in, 258–259
defect prevention, 435
evaluating methods, 487–488
patterns, 262
variations in, 259
defect quantities and origins, 29, 521–522
defect removal, 435
measuring, 471–475
measuring costs of, 483–487
defect removal efficiency, 29, 100–101, 220–221, 434–437, 522
of 16 combinations of 4 defect removal methods, 339–342

defect removal efficiency *(Continued)*
averages for cumulative defect removal efficiency, 317–318
best in class, 347
failure to measure, 602–603
measuring, 177–179, 475–480
ninety-five percent, 538–539
patterns, 262
variations in, 259–263
defect removal methods, 408–410
defect repair, U.S. averages for software maintenance productivity, 300–305
defect severity levels, 29–30, 522
defects per function point, by industry, 260
defects per KLOC metric, ambiguity in, 599–600
defense industry, 42–43
deletion ambiguity, 591
delivered defects, 221–222
by application, 29, 522
averages for, 318–320
best in class, 348, 349
DeMarco, Tom, 114, 404, 551
DeMarco "bang" function points, 113–115
Deming, W. Edwards, 455
diagnostic complexity, 419
Dijkstra, Edsger, 551
distribution of U.S. software applications, by type of application, 14
distribution of U.S. software projects, by nature of work, 15
documentation variables, 410–412
downsizings, 232–233, 376
Dreger, Brian, 579
duplicate defects, 491–492

E

earned value measurements, 26, 179–181, 519
effort
ranges in, 294–295
variations in associated with software size ranges, 270
variations in for industry segments, 271
effort errors in data, 192–206
embedded software, 282
See also systems software
emerging companies, 238
end-user benefits, 76
end-user programming, 230–231
end-user software, 247–248
zone of end-user software projects, 277
energy companies, 41, 43
engineering function points, 115–117
engineering point metrics, need for, 612
enhancements, 236–240
ambiguity, 590–591
average for annual enhancement volumes, 311–312
base code factors, 425–428
delta or changed code factors, 428–429
productivity averages and ranges for enhancement projects, 298–300
project variables, 412–414
terminology, 621–624
enterprise demographic measures, 53, 66
enterprise opinion survey, 52–53, 66
Enterprise Resource Planning (ERP), 21
entropic complexity, 419

environment, 36–37, 529
ergonomics, 36–37, 529
error-prone module removal, 469–470, 480–481, 538
errors in data, 190–191
  effort, resource, and activity errors, 192–206
  sizing errors, 191–192
  statistical and mathematical errors, 206–226
essential complexity, 417
estimated size ranges of selected applications, 273–276
estimating templates, publication of, 166
estimation, 3
ethics of measurement, 365–366
European industry, 453
executive sponsorship, of baseline studies, 367
external product innovation, 533
Extreme programming (XP), 342

F

Fagan, Michael, 462
failure, risk of, 307–308
false advertising, 564–566
feature points, 118–124
feedback loops, 121
Financial Accounting Standards Board (FASB), 19, 512
financial companies, 43
financial measures, 18–19, 512
flow complexity, 418
FP Lite, 124–125
fraudulent productivity claims, 564–566
full function points, 125–126
Function Point Analysis (Dreger), 579
function point calculations, 121–122
function point metrics
  3D function points, 107
  ambiguity in, 594–599
  automatic backfiring from source code, 164–165
  automatic conversion, 165
  automatic derivation from requirements and design, 164
  backfiring function points, 107–112
  chart of accounts for resource and cost data, 167–172
  COSMIC function points, 112–113, 161
  cost of counting, 78–81
  counting variations circa 2007, 598
  DeMarco "bang" function points, 113–115
  development of, 1, 24
  economic validity of, 89
  engineering function points, 115–117
  evolution of, 73–78
  expansion of, 233–235
  extending into areas that lack effective measurements, 167
  extension and refinement of complexity calculations, 165–166
  external attributes of, 104
  feature points, 118–124
  FP Lite, 124–125
  fragmentation and competition of variations, 186
  full function points, 125–126
  goals of, 172
  hybrid development practices, 234–235
  IFPUG function points, 126–136

function point metrics *(Continued)*
ISO standards on functional metrics, 137
Mark II Function Points, 137–140
micro function points, 111, 140–142
Netherlands function points (NESMA), 142–143
object points, 143
pattern-matching and function point sizing, 143–149
potential expansion of, 610–611
productivity comparison of function point variants, 162
prognosis for, 167
publication of estimating templates based on functional metrics, 166
rankings of productivity levels, 97
setting "best in class" targets, 344–350
size comparison of function point variants, 160
SPR function points, 149–155
story points, 155–156
tools for measuring, 609–610
unadjusted function points, 156–157
use case points, 157–158
uses for, 74
utilization of for software value analysis, 166
utilization of for studies of software consumption, 166
varieties of, 104–107
web object points, 158–159
function points
number of owned by selected U.S. companies, 77
ratio of source code statements to, 110
used by selected U.S. occupations, 78
function points light. *See* FP Lite
function points per staff month, best in class productivity, 345
functional complexity, 419
functional metrics. *See* function point metrics

G

Gartner Group, 71
generally accepted accounting principles (GAAP), 19
geriatric care companies, 426
Gilb, Tom, 462
goal question metrics, 27–28, 181–182, 520
goals, 389
development cycle and time to market, 626
failure to use metrics to establish, 624–629
productivity goals for enhancement projects, 627–628
productivity goals for maintenance projects, 628
productivity goals for new development projects, 627
quantified goals for management technologies, 629
quantified goals for reusability technologies, 629
quantified goals for sociological and morale factors, 628–629
software quality and user satisfaction, 625–626
government agencies, 44
GQM. *See* goal question metrics
Grady, Robert, 582

granularity, 7–9
gross productivity measures, 34, 526

H
Halstead, Maurice, 115, 116
hard data, 6–9
harmonic complexity, 417–418
Herron, David, 124–125
high-level languages, reversed productivity for, 87–103
human resources measures, 20, 513
Humphrey, Watts, 1, 12, 366, 442, 455
hybrid development practices, 234–235

I
IBM, 537–538
  value of measurement, 59–61
IEEE Computer Society, 369
IFPUG function points, 126–131, 136
  14 influential or value adjustment factors, 131–135
  data files, 136
  inputs, 135
  inquiries, 136
  interface, 136
  outputs, 136
incompetence, 376
independent verification and validation, 467
India, 453
indirect cost measures, 34, 526
industry average productivity, 543–544
industry leaders, measures and metrics of, 529–532
industry measures, 22–23
Information Technology Infrastructure Library (ITIL) measures, 30–31, 523
  emergence of, 440
Information Technology Metrics and Productivity Institute (ITMPI), creation of, 442
informational complexity, 416
innovation, 532–535
inputs, 135
inquiries, 136
inspections, 462–464
insurance industry, 43
intangible value measures, 543
interface, 136
internal process innovation, 533–535
international competition, and quality control, 451–454
International Function Point Users Group (IFPUG), 92, 594
  counting rules version 4.2, 1
  See also IFPUG function points
International Organization for Standardization. *See* ISO
International Software Benchmark Standard Group (ISBSG), 71, 225–226, 230, 357
  creation of, 186, 441–442
invalid defects, 491–492
ISO
  9000-9004 standards, 235–236, 343, 443–444
  standards on functional metrics, 137
ITIL. *See* Information Technology Infrastructure Library (ITIL) measures
IV&V, 467

J
JAD. *See* joint application design
Japanese industry, 452–453, 454
joint application design, 331, 469
Jones, Capers, 455
journals, 563

K
Kan, Steve, 455
Kaplan, Robert, 18, 176
Kemerer, Chris, 93
KLOC
  data normalization feature, 101
  defects per, 599–600
  lack of standardization, 580–582
  and professional malpractice, 585–586
  sociology of, 587–589
  See also lines of code (LOC)

L
languages, represented in the data, 211–217
layoffs, 232–233, 376
leading-edge companies, vs. trailing-edge companies, 3
leakage of resource data, 573–575
Leitgeb, Arthur, 115
life expectancy, averages for, 309–311
"light" function points, 80
lines of code (LOC), 24, 343–344
  evolution of, 72–73
  KLOC, 101
  lack of a standard definition for, 83–86
  lack of standardization, 580–582
  paradoxical behavior of LOC metrics, 582–585
  problems with and paradoxes of, 81–87
  and professional malpractice, 585–586
  range of costs, 82
  rankings of productivity levels, 97
  sociology of, 587–589
  tools for measuring, 616
Lister, Tim, 404
logical complexity, 417
logical lines, 92
  counting, 83

M
maintenance, 236–240
  assignment scope, 33
  base code factors, 425–428
  delta or changed code factors, 428–429
  project variables, 412–414
  terminology, 621–624
  U.S. averages for defect repair productivity, 300–305
  variations in assignment scopes, 303–304
maintenance productivity measures, 33–34, 526
Malcolm Baldridge Awards, 470–471
management, 4
  training, 36, 528–529
management information systems. *See* MIS
manual counting, 79, 81, 82
manufacturing industry, 43
manufacturing measures, 21, 514
Mark II Function Points, 137–140
market growth, 530
market share measures, 20, 513, 529
Martin, James, 455

mathematical errors, 206–226
McCabe, Tom, 456
McCabe cyclomatic complexity metric, 301–302, 415
McCue, Gerald, 404
mean time between failures (MTBF), 482
mean time to failure (MTTF), 482
measurement, 509–511
measurement expertise, sociology of, 57–58
measurement team, selection of, 367
methodologies, 4
methods, number of, 323
metrics conversion, tools for, 616–617
micro function points, 111, 140–142
Microsoft Office, 310
milestone tracking, 568–573
military software, 254–255
  productivity improvement, 16–17
  zone of military software projects, 283
Mills, Harlan, 468
MIS, 248–249
  productivity, 16
  vs. systems software, 57
  zone of MIS projects, 277–278
mnemonic complexity, 418
MOOSE metrics, 93, 94
morale, 531–532
Musa, John, 456

N

National Software Council, 368–369
natural metrics, vs. synthetic metrics, 550–552
nature, 386
  ambiguity in defining, 552–561
NESMA indicative, 80
Netherlands Function Point Users Group (NESMA), 80
Netherlands function points (NESMA), 142–143
nonprofit measurement associations, 368–369
normalized data, 10–17
Norton, David, 18, 176

O

object points, 143
object-oriented analysis and design, 344
object-oriented paradigm, 240
object-oriented quality levels, growth of, 449–450
occupation groups
  absence of measurement, 566–567
  identifying, 399–401
office environment, 5
offshore outsourcing, 250–251
oil production companies, 41
Okimoto, Gary, 480
Oligny, Serge, 125
one-person projects, sociology of measuring, 56
ongoing project measures, 64
on-site benchmarks, 359–360
open benchmark studies, 361–362
operational measures, 52, 64
organization of measurement function, 62–63
organization structures, 4
organizational complexity, 419
organizational measurements, 567–568
outputs, 136
outsource litigation, 535–536

outsource measures, 21, 514
outsource productivity, 16
outsourced software
  offshore, 250–251
  in the U.S., 249–250
  zone of outsourced software projects, 279–281
outsourcing, 241
overhead costs, 199–201
overlap tracking, 568–573
overlapping activities, 573

P

package ambiguity, 592–593
paperwork, averages for volumes and ranges of, 305–307
pattern-matching, 80–81, 82
  and function point sizing, 143–149
percentages, 589–590
perceptional complexity, 418
Perry, Bill, 456
personal software process (PSP), 12
  emergence of, 442–443
personnel variables, 401–403
physical lines, 92, 326
  counting, 83
Pirsig, Robert, 455
placement of measurement function, 62–63
planning, 3
political resistance to software measurements, 617–620
portfolio measures, 25–26, 518–519
Priven, Lew, 462
probabilities of project cancellation, 307
probability ranges for application sizes, 269
probability ranges for industry segments, 270
problem management, 32, 524
production library measures, 50–51, 64
production rates, 37–38, 420
productivity claims, fraudulent, 564–566
productivity comparison of function point variants, 162
productivity levels
  impact of technology on, 322–338
  rankings of using function point metrics and LOC metrics, 97
productivity measures, 49–50
productivity ranges
  in function points per staff months, 296
  in work hours per function point, 297
productivity rates, 296
  average productivity for selected methodologies, 297
  average productivity in function points per staff month, 295
  average productivity in work hours per function point, 296
  averages and ranges for enhancement projects, 298–300
  averages and ranges for new projects, 295–298
  changes in between 1990 and 2005, 189–190
  distribution in function points, 13
  impact of multiple technologies, 334–338
  improvement rates, 265–268
  maintenance, 300–305
  ranges, averages and variances in, 268–276

results on 10,000-function point projects of four technology factors, 336–337
reversed productivity for high-level languages, 87–103
and service-oriented architecture, 219
by size of application, 220
by type of application, 16
variations in application size and, 159–163
professional malpractice, 98–99, 585–586
programming languages, represented in the data, 211–217
project, defined, 364
project cancellation, risk of, 307–308
project demographics, absence of measurement, 566–567
project failure, risk of, 307–308
project team, defined, 364
proofs of correctness, 467
prototyping, 468–469
public utilities, 43
publishing industry, 43–44

Q

QA organizations, 460–462
quality, defining for measurement and estimation, 454–457
quality assurance organizations, 460–462
quality circles, 466
quality control
five steps to, 458–460
and international competition, 451–454
reviews, walk-throughs and inspections, 462–464
in the U.S., 460–471
Quality Function Deployment (QFD), 22, 464–466
Quality Is Free (Crosby), 486
quality levels
for applications of 10,000 function points, 221
impact of multiple technologies, 338–342
impact of technology on, 322–334, 338–342
quality measures, 48–49
quality metrics, ambiguity in, 599
Quantitative Software Management (QSM), 71
Quicken, 310

R

RAD. *See* rapid application development
Radice, Ron, 462
ranges in productivity, 268–276
rankings of technologies for productivity, schedules, and quality, 327–330
rapid application development, 325, 344
rates of requirements change, 34–35, 526–527
ratios, 589–590
reengineering, 427, 470
Reifer, Don, 116, 158
Relativity Technologies, 238, 302, 609–610
reliability prediction, using metrics for, 482
remote benchmarks, 359–360
renovation, 15
requirements creep, ranges in, 292–294
research and development (R&D) measures, 20–21, 514, 529

research and development spending patterns, 23, 516
resources, tools for measuring, 615
restructuring, 470
return on investment (ROI), 19, 325, 512–513
  value of applied software measurement programs, 58–61
reusability, 5–6, 242–243
  ambiguity, 591–592
  averages for potential and actual reusability, 320–321
  reusable design and code factors, 424–425
reusable code, counting, 84–85
reverse backfiring, 82
  See also backfiring
reverse engineering, 427, 470
reversed productivity, for high-level languages, 87–103
reviews, 462–464
risk factors, 414–415
risk point metrics, need for, 614
Roemer, Olaus, 190

S

Sarbanes-Oxley (SOX), 19, 510
  financial measures, 19
  measures, 18, 511
schedule ambiguity, 195–196
schedule ranges, 290–294
schedule reduction, 324–325
schedule tracking, 568–573
schedules, tools for measuring, 615–616
scheduling, irrational, 375
scope, 386
  ambiguity in defining, 552–561
Scrum sessions, 173
searching, 120–121
security measures, 32, 525
semantic complexity, 418
service and support, 531
service desk measures, 31, 523–524
service level agreements, 32, 525
service point metrics, need for, 611–612
service-oriented architecture, 241–242
  emergence of, 440–441
  and productivity rates, 219
shareholder measures, 19–20, 513
shareholder value, 530
Shoulders Corporation, 238, 304–305
Six-Sigma, 242, 466
  emergence of, 439–440
  quality levels, 539
Six-Sigma Quality, 22
size comparison of function point variants, 160
size ranges of applications, 273–276
sizing errors in data, 191–192
skills, 401–402
  of measurement teams, 62
SOA architecture. *See* service-oriented architecture
social resistance to software measurements, 617–620
soft data, 9–10
soft factors, problems of measuring impact of, 603–607
soft-factor measures, 51–52, 65
software artifacts, reusable, 5–6
software assessment measures, 35, 527–529
software benchmarks, collections of, 71
software consumption, 166

software control, characteristics of, 2
software defect measures, 65–66
software defect removal methods, 408–410
Software Engineering Economics (Boehm), 582
Software Engineering Institute (SEI), 71, 369
  assessments, 352–355
  capability maturity model (CMM), 243–244
  measuring impact of soft factors, 604–607
  using physical lines, 92
  See also capability maturity model (CMM); capability maturity model integrated (CMMI)
software industry, evolution of, 72–78
software infrastructure, 36, 528
software life cycle, and measurement, 44–47
software life expectancy, averages for, 309–311
software measurement, sociology of, 53–54
software measures and metrics, 23–25, 516–518
  activity-based cost measures, 33, 46–47, 525–526
  activity-based schedule measures, 33, 525
  application deliverable size measures, 32–33, 525
  assignment scopes and production rates, 37–38
  balanced scorecards for software, 26–27, 519–520
  capability maturity model (CMM) level, 35
  change requests, 31–32, 524
  complexity of software, 30, 522
  cost of quality control and defect repairs, 30, 523
  current measurement experiences by industry, 40–44
  customer satisfaction, 28, 521
  defect quantities and origins, 29, 521–522
  defect removal efficiency, 29, 522
  defect severity levels, 29–30, 522
  delivered defects by application, 29, 522
  earned value measurements, 26, 519
  environment and ergonomics, 36–37
  evolution of, 72–78
  goal question metrics, 27–28, 520
  gross productivity measures, 34, 526
  indirect cost measures, 34, 526
  Information Technology Infrastructure Library (ITIL) measures, 30–31, 523
  maintenance productivity measures, 33–34, 526
  not based on function points, 173–182
  portfolio measures, 25–26, 518–519
  problem management, 32, 524
  rates of requirements change, 34–35, 526–527
  security measures, 32, 525
  service desk measures, 31, 523–524
  service level agreements, 32, 525

software measures and metrics *(Continued)*
social and political resistance to, 617–620
software assessment measures, 35
software benchmarks, 25, 518
software infrastructure, 36
software outsource measures, 28, 521
software processes, 36
software team skills and experience, 36
software tool suites, 36
software usage measures, 28–29, 521
staff and management training, 36
strategic and tactical software measurement, 38–40
terminology, 620–624
test case coverage, 30, 522–523
training in, 578–579
Software Metrics (Grady and Caswell), 582
software overruns, 543
software package acquisition, 407–408
software populations, comparison of by industry, 246
software processes, 36, 528
software productivity, major factors influencing, 14
Software Productivity Research (SPR), 71
ages of project in SPR knowledge base, 209–211
assessments, 352–355
measuring impact of soft factors, 604–607
using logical statements, 92
software quality analysis, 100–102
software quality, changes since 1991, 189
software team skills and experience, 36, 528
software technologies, represented in the data, 217–222
software tool suites, 36, 528
software value, problems in measuring, 607–609
sorting, 120
source code, 415–419
ratio of to function points, 110
span of control, 567–568
specialists, 288–290
identifying, 399–401
SPICE, 352
SPQR/20 tool, 149, 152–153
SPR function points, 149–155
staff availability, 389–392
staff morale, 531–532
staff performance targets, sociology of using data for, 55–56
staff training, 36, 528–529
staffing, of measurement teams, 62
staffing levels, 283–290
staff months, 94
standards for software measurement, 579–580
start of projects, 568
statistical errors, 206–226
status reports, suggested format for, 570–572
step-rate calculation functions, 121
story points, 155–156
strategic software measurement, 38–40
strengths, most common software strengths, 383
structural complexity, 417
suggested readings, 630–632

Supply Chain Council, 21
supply chain measures, 21–22, 514–515
Supply Chain Operations Reference (SCOR), 21
Symons, Charles, 113, 137–139
syntactic complexity, 418
synthetic metrics, vs. natural metrics, 550–552
systems software, 253–254
  productivity improvement, 16–17
  zone of systems software projects, 282–283
systems software vs. MIS, sociology of, 57

T
tactical software measurement, 38–40
targets, failure to use metrics to establish, 624–629
tasks, ambiguity in defining, 561–564
team software process (TSP), 1, 12
  emergence of, 442–443
technical staffs, 4
technologies noted on applications of various size ranges, 332–333
telecommunications manufacturing, 41
telecommunications operating companies, 41
terminology, 620–624
test case coverage, 30, 522–523
  using metrics to evaluate, 481–482
testing departments, 467–468
third edition, changes in structure, format, and contents, 244–246
3D function points, 107
TickIT, 352
time metrics, ambiguity in, 575–578
time to market, 530
tools, 4
  integrated software management tools, 617
  for measuring function point metrics, 609–610
  number of, 323
topological complexity, 417
Total Quality Management (TQM), 22
  rise and fall of, 450–451
trailing-edge companies, vs. leading-edge companies, 3
training in software measurement and metrics, 578–579
turnover, 23, 516
type, 388
  ambiguity in defining, 552–561
types of lines, counting, 83–84

U
Umholtz, Donald, 115
unadjusted function points, 156–157
unit development costs, 530
use case points, 157–158
user satisfaction measures, 65, 492
  combining with defect data, 499–501
  survey, 492–499

V
value adjustments, 131
value factors, 414–415
value point metrics, need for, 613–614
variances in productivity, 268–276

variations in data representation and averaging, 223–226

W

walk-throughs, 462–464
warranty and quality measures, 22, 515
warranty repairs, 531
Watt, James, 93
weaknesses, most common software weaknesses, 382
web applications, 247
  emergence of, 438–439
  zone of, 278–279
web content measures, 542–543
web object points, 158–159
Whitemire, Scott, 107
wholesale-retail industry, 43
Wikipedia, 438
work hours per function point, best in class productivity, 345
work period ambiguity, 201–206
work-habit factors, 389–392
World Wide Web, 244

Z

Zen and the Art of Motorcycle Maintenance (Pirsig), 455
zero-defect programs, 466–467
Zipf, George, 117